

GBDK 2020 Docs

Generated on Sat Dec 27 2025 17:53:30 for GBDK 2020 Docs by Doxygen 1.9.2

Sat Dec 27 2025 17:53:30

1 General Documentation	2
1.1 Introduction	2
1.2 About the Documentation	2
1.3 About GBDK	2
1.4 Historical Info and Links	3
2 Getting Started	3
2.1 1. Download a Release and unzip it	3
2.1.1 Known Issue: Windows and folder names with spaces on non-C drives	3
2.2 2. Compile Example projects	3
2.2.1 Windows (without Make installed):	3
2.2.2 Linux / macOS / Windows with Make installed:	4
2.3 3. Use a Template	4
2.4 4. If you use GBTD / GBMB, get the fixed version	4
2.5 5. Review Coding Guidelines	4
2.6 6. Hardware and Resources	4
2.7 7. Set up C Source debugging	5
2.8 8. Try a GBDK Tutorial	5
2.9 9. Read up!	5
2.10 10. Need help?	5
2.11 Migrating From Pre-GBDK-2020 Tutorials	5
2.11.1 Also see:	5
2.11.2 Use auto-banking	5
2.11.3 Non-standard types (UINT8, etc)	6
2.11.4 If using GBTD / GBMB, get the fixed version	6
2.11.5 LCC and SDCC flags that are not needed	6
2.11.6 ROM Header Settings (such as Color, SGB, etc)	6
2.11.7 GBDK Header include changes	6
2.11.8 Include .h headers, not .c source files	6
2.11.9 Use the Template Projects	6
2.11.10 Use hUGTracker instead of gbt_player	6
3 Links, Tools and Debugging	6
3.1 SDCC Compiler Suite User Manual	7
3.2 Getting Help	7
3.3 Game Boy Documentation	7
3.4 Sega Master System / Game Gear Documentation	7
3.5 Mega Duck / Cougar Boy Documentation	7
3.6 Tutorials	7
3.7 Example code	8
3.8 Graphics Tools	8
3.9 Music And Sound Effects for the Game Boy	8
3.10 Music And Sound Effects for the SMS/Game Gear	8

3.11 Emulators	9
3.12 Debugging tools	9
3.13 Optimizing Assembly	10
3.14 Continuous Integration and Deployment	10
4 Using GBDK	10
4.1 Interrupts	10
4.1.1 Available Interrupts	10
4.1.2 Adding your own interrupt handler	11
4.1.3 Using your own Interrupt Dispatcher	11
4.1.4 Returning from Interrupts and STAT mode	11
4.2 What GBDK does automatically and behind the scenes	11
4.2.1 NES console	11
4.2.2 OAM (VRAM Sprite Attribute Table)	11
4.2.3 Graphics Tile Maps and Data on Startup	12
4.2.4 Font tiles when using stdio.h	12
4.2.5 Default Interrupt Service Handlers (ISRs)	12
4.2.6 Ensuring Safe Access to Graphics Memory	12
4.3 Compression	12
4.4 Copying Functions to RAM and HIRAM	13
4.5 Mixing C and Assembly	13
4.5.1 Inline ASM within C source files	13
4.5.2 In Separate ASM files	13
4.6 Including binary files in C source with incbin	14
4.7 Known Issues and Limitations	14
4.7.1 SDCC	14
5 Coding Guidelines	14
5.1 Learning C / C fundamentals	14
5.1.1 General C tutorials	15
5.1.2 Embedded C introductions	15
5.1.3 Game Boy games in C	15
5.2 Understanding the hardware	15
5.3 Writing optimal C code for the Game Boy and SDCC	15
5.3.1 Tools	15
5.3.2 Avoid Reading from VRAM	15
5.3.3 Variables	15
5.3.4 Code structure	17
5.3.5 GBDK API/Library	17
5.3.6 Toolchain	17
5.3.7 Constants, Signed-ness and Overflows	18
5.3.8 Chars and vararg functions	18
5.4 When C isn't fast enough	19

5.4.1 Reusable Local Labels and Inline ASM	19
5.4.2 Variables and registers	19
5.4.3 Segments / Areas	19
5.4.4 Calling convention	20
6 ROM/SRAM Banking and MBCs	21
6.1 ROM/SRAM Banking and MBCs (Memory Bank Controllers)	21
6.1.1 Non-banked cartridges	21
6.1.2 MBC Banked cartridges (Memory Bank Controllers)	22
6.1.3 Recommended MBC type	22
6.2 Working with Banks	22
6.2.1 Setting the ROM bank for a Source file	22
6.2.2 Setting the Cart SRAM bank for a Source file	23
6.2.3 Setting the MBC and number of Cart ROM & SRAM banks available	23
6.2.4 MBC Type Chart	23
6.2.5 Getting Bank Numbers	24
6.2.6 Banking and Functions	24
6.2.7 Const Data (Variables in ROM)	26
6.2.8 Variables in Cart SRAM	26
6.2.9 Far Pointers	26
6.2.10 Bank switching	26
6.2.11 Wrapper Function for Accessing Banked Data	26
6.2.12 Currently active bank: CURRENT_BANK	26
6.3 Auto-Banking	27
6.4 Errors related to banking (overflow, multiple writes to same location)	27
6.5 Bank space usage	28
6.5.1 Other important notes	28
6.6 Banking example projects	28
6.7 SMS/Game Gear Banking	28
6.7.1 ROM Banks	28
6.7.2 Cart SRAM Banks	29
6.7.3 Auto-Banking	29
6.8 NES Banking	29
6.8.1 ROM Banks	29
6.8.2 Cart SRAM Banks	29
7 GBDK Toolchain	29
7.1 Overview	29
7.2 Data Types	30
7.2.1 Using variables in High RAM on the Game Boy	30
7.3 Changing Important Addresses	30
7.4 Compiling programs	31
7.4.1 Makefiles	31

7.4.2 Using Makefiles	31
7.4.3 Linker Files and ROM Auto Banking	31
7.5 Build Tools	32
7.5.1 lcc	32
7.5.2 sdcc	32
7.5.3 sdasgb	32
7.5.4 bankpack	32
7.5.5 sldlgb	32
7.5.6 ihxcheck	33
7.5.7 makebin	33
7.6 GBDK Utilities	33
7.6.1 GBCompress	33
7.6.2 png2asset	33
7.6.3 makecom	35
7.6.4 makenes	35
7.6.5 png2hicolorgb	35
7.6.6 romusage	36
8 Supported Consoles & Cross Compiling	36
8.1 Consoles Supported by GBDK	36
8.2 Cross Compiling for Different Consoles	36
8.2.1 lcc	36
8.2.2 sdcc	36
8.2.3 Console Port and Platform Settings	37
8.3 Cross-Platform Constants	37
8.3.1 Console Identifiers	38
8.3.2 Console Hardware Properties	38
8.4 Using <gbdk/...> headers	38
8.5 Cross Platform Example Projects	39
8.5.1 Cross Platform Asset Example	39
8.6 Hardware Summaries	39
8.6.1 Safe VRAM / Display Controller Access	40
8.7 Using Game Boy Color (GBC/CGB) Features	41
8.7.1 Differences Versus the Regular Game Boy (DMG/GBP/SGB)	41
8.7.2 Game Boy Color features in GBDK	41
8.7.3 CGB Examples	41
8.8 Porting Between Supported Consoles	41
8.8.1 From Game Boy to Analogue Pocket	41
8.8.2 From Game Boy to SMS/GG	42
8.8.3 From Game Boy to NES	44
8.8.4 From Game Boy to Mega Duck / Cougar Boy	48
9 Example Programs	50

9.1 banks (various projects)	50
9.2 comm	51
9.3 crash	51
9.4 colorbar	51
9.5 dscan	51
9.6 filltest	51
9.7 fonts	51
9.8 galaxy	51
9.9 gb-dtmf	51
9.10 gbdecompress	51
9.11 irq	51
9.12 large map	51
9.13 metasprites	52
9.14 lcd isr wobble	52
9.15 paint	52
9.16 rand	52
9.17 ram_fn	52
9.18 rpn	52
9.19 samptest	52
9.20 sgb (various)	52
9.21 sound	52
9.22 space	53
9.23 templates	53
10 Frequently Asked Questions (FAQ)	53
10.1 General	53
10.2 Licensing	53
10.3 Graphics and Resources	53
10.4 ROM Header Settings	53
10.5 Editors	54
10.6 Errors and Warnings	54
10.7 Debugging / Compiling / Toolchain	56
10.8 API / Utilities	56
11 Migrating to new GBDK Versions	57
11.1 GBDK-2020 versions	57
11.1.1 Porting to GBDK-2020 4.5.0	57
11.1.2 Porting to GBDK-2020 4.4.0	57
11.1.3 Porting to GBDK-2020 4.3.0	58
11.1.4 Porting to GBDK-2020 4.2.0	58
11.1.5 Porting to GBDK-2020 4.1.1	59
11.1.6 Porting to GBDK-2020 4.1.0	59
11.1.7 Porting to GBDK-2020 4.0.6	59

11.1.8 Porting to GBDK-2020 4.0.5	60
11.1.9 Porting to GBDK-2020 4.0.4	60
11.1.10 Porting to GBDK-2020 4.0.3	60
11.1.11 Porting to GBDK-2020 4.0.2	60
11.1.12 Porting to GBDK-2020 4.0.1	60
11.1.13 Porting to GBDK-2020 4.0	61
11.1.14 Porting to GBDK-2020 3.2	61
11.1.15 Porting to GBDK-2020 3.1.1	61
11.1.16 Porting to GBDK-2020 3.1	61
11.1.17 Porting to GBDK-2020 3.0.1	61
11.2 Historical GBDK versions	61
11.2.1 GBDK 1.1 to GBDK 2.0	61
12 GBDK Release Notes	61
12.1 GBDK-2020 Release Notes	62
12.1.1 GBDK-2020 4.5.0	62
12.1.2 GBDK-2020 4.4.0	63
12.1.3 GBDK-2020 4.3.0	65
12.1.4 GBDK-2020 4.2.0	68
12.1.5 GBDK-2020 4.1.1	69
12.1.6 GBDK-2020 4.1.0	70
12.1.7 GBDK-2020 4.0.6	71
12.1.8 GBDK-2020 4.0.5	73
12.1.9 GBDK-2020 4.0.4	74
12.1.10 GBDK-2020 4.0.3	75
12.1.11 GBDK-2020 4.0.2	76
12.1.12 GBDK-2020 4.0.1	76
12.1.13 GBDK-2020 4.0	77
12.1.14 GBDK-2020 3.2	77
12.1.15 GBDK-2020 3.1.1	77
12.1.16 GBDK-2020 3.1	78
12.1.17 GBDK-2020 3.0.1	78
12.1.18 GBDK-2020 3.0	78
12.2 Historical GBDK Release Notes	78
12.2.1 GBDK 2.96	78
12.2.2 GBDK 2.95-3	79
12.2.3 GBDK 2.95-2	79
12.2.4 GBDK 2.95	79
12.2.5 GBDK 2.94	80
12.2.6 GBDK 2.93	80
12.2.7 GBDK 2.92-2 for win32	81
12.2.8 GBDK 2.92	81

12.2.9 GBDK 2.91	81
12.2.10 GBDK 2.1.5	82
12.2.11 GBDK 2.0b11 (DOS binary only) - 24 November 1997	82
12.2.12 GBDK 2.0b10 (DOS binary only) - 6 November 1997	82
12.2.13 GBDK 2.0b9 (DOS binary only)	82
12.2.14 GBDK 2.0b8 (DOS binary only)	82
12.2.15 GBDK 2.0b7 (DOS binary only)	82
12.2.16 GBDK 2.0b6	83
12.2.17 GBDK 2.0b5	83
12.2.18 GBDK 2.0b4	83
12.2.19 GBDK 2.0b3	83
12.2.20 GBDK 2.0b2	83
12.2.21 GBDK 2.0b1	84
12.2.22 GBDK 1.1	84
12.2.23 GBDK 1.0-1 1996	84
13 Toolchain settings	84
13.1 lcc settings	84
13.2 sdcc settings	84
13.3 sdasgb settings	86
13.4 sdasz80 settings	87
13.5 sdas6500 settings	87
13.6 bankpack settings	87
13.7 sldlgb settings	88
13.8 sldlz80 settings	88
13.9 sldl6808 settings	89
13.10 ihxcheck settings	89
13.11 makebin settings	89
13.12 makecom settings	90
13.13 makenes settings	90
13.14 gbcompress settings	90
13.15 png2asset settings	90
13.16 png2hicolorgb settings	91
13.17 romusage settings	91
14 Todo List	92
15 Module Index	92
15.1 Modules	92
16 Data Structure Index	92
16.1 Data Structures	92
17 File Index	93

17.1 File List	93
18 Module Documentation	95
18.1 List of gbdk fonts	95
18.1.1 Detailed Description	95
18.1.2 Variable Documentation	95
19 Data Structure Documentation	96
19.1 __far_ptr Union Reference	96
19.1.1 Detailed Description	96
19.1.2 Field Documentation	96
19.2 _fixed Union Reference	96
19.2.1 Detailed Description	97
19.2.2 Field Documentation	97
19.3 atomic_flag Struct Reference	97
19.3.1 Field Documentation	97
19.4 isr_nested_vector_t Struct Reference	97
19.4.1 Field Documentation	98
19.5 isr_vector_t Struct Reference	98
19.5.1 Field Documentation	98
19.6 joypads_t Struct Reference	98
19.6.1 Detailed Description	99
19.6.2 Field Documentation	99
19.7 metasp Sprite_t Struct Reference	100
19.7.1 Detailed Description	100
19.7.2 Field Documentation	100
19.8 OAM_item_t Struct Reference	101
19.8.1 Detailed Description	101
19.8.2 Field Documentation	101
19.9 sfont_handle Struct Reference	102
19.9.1 Detailed Description	102
19.9.2 Field Documentation	102
20 File Documentation	103
20.1 docs/pages/01_getting_started.md File Reference	103
20.2 docs/pages/02_links_and_tools.md File Reference	103
20.3 docs/pages/03_using_gbdk.md File Reference	103
20.4 docs/pages/04_coding_guidelines.md File Reference	103
20.5 docs/pages/05_banking_mbc5.md File Reference	103
20.6 docs/pages/06_toolchain.md File Reference	103
20.7 docs/pages/06b_supported_consoles.md File Reference	103
20.8 docs/pages/07_sample_programs.md File Reference	103
20.9 docs/pages/08_faq.md File Reference	103

20.10 docs/pages/09_migrating_new_versions.md File Reference	103
20.11 docs/pages/10_release_notes.md File Reference	103
20.12 docs/pages/20_toolchain_settings.md File Reference	103
20.13 docs/pages/docs_index.md File Reference	103
20.14 gbdk-lib/include/asm/mos6502/provides.h File Reference	103
20.14.1 Macro Definition Documentation	103
20.15 provides.h	103
20.16 gbdk-lib/include/asm/sm83/provides.h File Reference	103
20.16.1 Macro Definition Documentation	104
20.17 provides.h	104
20.18 gbdk-lib/include/asm/z80/provides.h File Reference	104
20.18.1 Macro Definition Documentation	104
20.19 provides.h	104
20.20 gbdk-lib/include/asm/mos6502/stdarg.h File Reference	104
20.20.1 Macro Definition Documentation	105
20.20.2 Typedef Documentation	105
20.21 stdarg.h	105
20.22 gbdk-lib/include/asm/sm83/stdarg.h File Reference	105
20.22.1 Macro Definition Documentation	105
20.22.2 Typedef Documentation	106
20.23 stdarg.h	106
20.24 gbdk-lib/include/asm/z80/stdarg.h File Reference	106
20.24.1 Macro Definition Documentation	106
20.24.2 Typedef Documentation	107
20.25 stdarg.h	107
20.26 gbdk-lib/include/stdarg.h File Reference	107
20.27 stdarg.h	107
20.28 gbdk-lib/include/assert.h File Reference	107
20.28.1 Macro Definition Documentation	107
20.28.2 Function Documentation	108
20.29 assert.h	108
20.30 gbdk-lib/include/ctype.h File Reference	108
20.30.1 Detailed Description	109
20.30.2 Function Documentation	109
20.31 ctype.h	110
20.32 gbdk-lib/include/duck/laptop_io.h File Reference	110
20.32.1 Detailed Description	112
20.32.2 MegaDuck Laptop Peripheral IO support	112
20.32.3 Macro Definition Documentation	112
20.32.4 Function Documentation	114
20.32.5 Variable Documentation	117
20.33 laptop_io.h	117

20.34 gbdk-lib/include/duck/laptop_keycodes.h File Reference	119
20.34.1 Macro Definition Documentation	121
20.35 laptop_keycodes.h	128
20.36 gbdk-lib/include/duck/model.h File Reference	130
20.36.1 Macro Definition Documentation	131
20.36.2 Function Documentation	131
20.37 model.h	131
20.38 gbdk-lib/include/gb/bcd.h File Reference	131
20.38.1 Detailed Description	132
20.38.2 Macro Definition Documentation	132
20.38.3 Typedef Documentation	132
20.38.4 Function Documentation	132
20.39 bcd.h	133
20.40 gbdk-lib/include/gbdk/bcd.h File Reference	134
20.41 bcd.h	134
20.42 gbdk-lib/include/nes/bcd.h File Reference	134
20.42.1 Detailed Description	134
20.42.2 Macro Definition Documentation	134
20.42.3 Typedef Documentation	135
20.42.4 Function Documentation	135
20.43 bcd.h	136
20.44 gbdk-lib/include/sms/bcd.h File Reference	136
20.44.1 Detailed Description	136
20.44.2 Macro Definition Documentation	137
20.44.3 Typedef Documentation	137
20.44.4 Function Documentation	137
20.45 bcd.h	138
20.46 gbdk-lib/include/gb/bgb_emu.h File Reference	138
20.46.1 Detailed Description	138
20.47 bgb_emu.h	138
20.48 gbdk-lib/include/gb/cgb.h File Reference	139
20.48.1 Detailed Description	139
20.48.2 Macro Definition Documentation	140
20.48.3 Typedef Documentation	142
20.48.4 Function Documentation	142
20.49 cgb.h	145
20.50 gbdk-lib/include/gb/crash_handler.h File Reference	145
20.50.1 Detailed Description	145
20.50.2 Function Documentation	145
20.51 crash_handler.h	146
20.52 gbdk-lib/include/gb/drawing.h File Reference	146
20.52.1 Detailed Description	147

20.52.2 Macro Definition Documentation	147
20.52.3 Function Documentation	148
20.53 drawing.h	150
20.54 gbdk-lib/include/gb/emu_debug.h File Reference	151
20.54.1 Detailed Description	151
20.55 emu_debug.h	151
20.56 gbdk-lib/include/gbdk/emu_debug.h File Reference	151
20.56.1 Detailed Description	152
20.56.2 Macro Definition Documentation	152
20.56.3 Function Documentation	154
20.56.4 Variable Documentation	155
20.57 emu_debug.h	155
20.58 gbdk-lib/include/gb/gb.h File Reference	156
20.58.1 Detailed Description	161
20.58.2 Macro Definition Documentation	161
20.58.3 Typedef Documentation	172
20.58.4 Function Documentation	172
20.58.5 Variable Documentation	205
20.59 gb.h	206
20.60 gbdk-lib/include/gb/gbdecompress.h File Reference	213
20.60.1 Detailed Description	214
20.60.2 Function Documentation	214
20.61 gbdecompress.h	215
20.62 gbdk-lib/include/gbdk/gbdecompress.h File Reference	216
20.63 gbdecompress.h	216
20.64 gbdk-lib/include/sms/gbdecompress.h File Reference	216
20.64.1 Function Documentation	216
20.64.2 Variable Documentation	217
20.65 gbdecompress.h	217
20.66 gbdk-lib/include/gb/hblankcpy.h File Reference	217
20.66.1 Function Documentation	217
20.66.2 Variable Documentation	218
20.67 hblankcpy.h	218
20.68 gbdk-lib/include/gb/isr.h File Reference	219
20.68.1 Detailed Description	219
20.68.2 Macro Definition Documentation	219
20.68.3 Typedef Documentation	220
20.69 isr.h	221
20.70 gbdk-lib/include/gb/sgb.h File Reference	221
20.70.1 Detailed Description	222
20.70.2 Macro Definition Documentation	222
20.70.3 Function Documentation	223

20.70.4 Variable Documentation	224
20.71 sgb.h	224
20.72 gbdk-lib/include/gbdk/console.h File Reference	224
20.72.1 Detailed Description	225
20.72.2 Function Documentation	225
20.73 console.h	225
20.74 gbdk-lib/include/gbdk/far_ptr.h File Reference	226
20.74.1 Detailed Description	226
20.74.2 Macro Definition Documentation	226
20.74.3 Typedef Documentation	228
20.74.4 Function Documentation	228
20.74.5 Variable Documentation	228
20.75 far_ptr.h	228
20.76 gbdk-lib/include/gbdk/font.h File Reference	229
20.76.1 Detailed Description	230
20.76.2 Macro Definition Documentation	230
20.76.3 Typedef Documentation	230
20.76.4 Function Documentation	230
20.77 font.h	231
20.78 gbdk-lib/include/gbdk/gbdk-lib.h File Reference	232
20.78.1 Detailed Description	232
20.79 gbdk-lib.h	232
20.80 gbdk-lib/include/gbdk/incbin.h File Reference	232
20.80.1 Detailed Description	232
20.80.2 Macro Definition Documentation	232
20.81 incbin.h	234
20.82 gbdk-lib/include/gbdk/platform.h File Reference	234
20.83 platform.h	234
20.84 gbdk-lib/include/gbdk/rledecompress.h File Reference	235
20.84.1 Detailed Description	235
20.84.2 Macro Definition Documentation	235
20.84.3 Function Documentation	235
20.85 rledecompress.h	236
20.86 gbdk-lib/include/gbdk/version.h File Reference	236
20.86.1 Macro Definition Documentation	236
20.87 version.h	236
20.88 gbdk-lib/include/gbdk/zx0decompress.h File Reference	236
20.88.1 Function Documentation	236
20.89 zx0decompress.h	237
20.90 gbdk-lib/include/limits.h File Reference	237
20.90.1 Macro Definition Documentation	237
20.91 limits.h	238

20.92 gbdk-lib/include/gb/hardware.h File Reference	239
20.92.1 Detailed Description	245
20.92.2 Macro Definition Documentation	245
20.92.3 Variable Documentation	258
20.93 hardware.h	264
20.94 gbdk-lib/include/msx/hardware.h File Reference	269
20.94.1 Detailed Description	270
20.94.2 Macro Definition Documentation	270
20.94.3 Variable Documentation	275
20.95 hardware.h	276
20.96 gbdk-lib/include/nes/hardware.h File Reference	277
20.96.1 Detailed Description	279
20.96.2 Macro Definition Documentation	279
20.96.3 Typedef Documentation	281
20.96.4 Function Documentation	281
20.96.5 Variable Documentation	282
20.97 hardware.h	282
20.98 gbdk-lib/include/sms/hardware.h File Reference	284
20.98.1 Detailed Description	287
20.98.2 Macro Definition Documentation	287
20.98.3 Variable Documentation	296
20.99 hardware.h	297
20.100 gbdk-lib/include/gb/metaspri.es.h File Reference	300
20.100.1 Detailed Description	301
20.100.2 Metasprite support	301
20.100.3 Metaspri.es composed of variable numbers of sprites	301
20.100.4 Metaspri.es and sprite properties (including cgb palette)	301
20.100.5 Macro Definition Documentation	302
20.100.6 Typedef Documentation	302
20.100.7 Function Documentation	303
20.100.8 Variable Documentation	306
20.101 metaspri.es.h	307
20.102 gbdk-lib/include/gbdk/metaspri.es.h File Reference	308
20.103 metaspri.es.h	308
20.104 gbdk-lib/include/msx/metaspri.es.h File Reference	308
20.104.1 Macro Definition Documentation	309
20.104.2 Typedef Documentation	309
20.104.3 Function Documentation	309
20.104.4 Variable Documentation	311
20.105 metaspri.es.h	311
20.106 gbdk-lib/include/nes/metaspri.es.h File Reference	312
20.106.1 Detailed Description	312

20.106.2 Metasprite support	312
20.106.3 Macro Definition Documentation	312
20.106.4 Typedef Documentation	313
20.106.5 Function Documentation	313
20.106.6 Variable Documentation	317
20.107 metasprites.h	317
20.108 gbdk-lib/include/sms/metaspites.h File Reference	318
20.108.1 Detailed Description	319
20.108.2 Metasprite support	319
20.108.3 Metasprite support	319
20.108.4 Macro Definition Documentation	319
20.108.5 Typedef Documentation	319
20.108.6 Function Documentation	320
20.108.7 Variable Documentation	323
20.109 metasprites.h	323
20.110 gbdk-lib/include/msx/msx.h File Reference	324
20.110.1 Detailed Description	328
20.110.2 Macro Definition Documentation	328
20.110.3 Typedef Documentation	335
20.110.4 Function Documentation	335
20.110.5 Variable Documentation	350
20.111 msx.h	352
20.112 gbdk-lib/include/nes/nes.h File Reference	358
20.112.1 Detailed Description	361
20.112.2 Macro Definition Documentation	361
20.112.3 Typedef Documentation	370
20.112.4 Function Documentation	370
20.112.5 Variable Documentation	397
20.113 nes.h	398
20.114 gbdk-lib/include/nes/rgb_to_nes_macro.h File Reference	403
20.114.1 Macro Definition Documentation	403
20.115 rgb_to_nes_macro.h	403
20.116 gbdk-lib/include/rand.h File Reference	404
20.116.1 Detailed Description	404
20.116.2 Macro Definition Documentation	404
20.116.3 Function Documentation	405
20.116.4 Variable Documentation	405
20.117 rand.h	406
20.118 gbdk-lib/include/setjmp.h File Reference	406
20.118.1 Macro Definition Documentation	407
20.118.2 Typedef Documentation	407
20.118.3 Function Documentation	407

20.119 setjmp.h	407
20.120 gbdk-lib/include/sms/sms.h File Reference	408
20.120.1 Detailed Description	413
20.120.2 Macro Definition Documentation	413
20.120.3 Typedef Documentation	420
20.120.4 Function Documentation	420
20.120.5 Variable Documentation	434
20.121 sms.h	435
20.122 gbdk-lib/include/stdatomic.h File Reference	441
20.122.1 Function Documentation	441
20.123 stdatomic.h	442
20.124 gbdk-lib/include/stdbool.h File Reference	442
20.124.1 Macro Definition Documentation	442
20.125 stdbool.h	442
20.126 gbdk-lib/include/stddef.h File Reference	443
20.126.1 Macro Definition Documentation	443
20.126.2 Typedef Documentation	443
20.127 stddef.h	444
20.128 gbdk-lib/include/stdint.h File Reference	445
20.128.1 Macro Definition Documentation	446
20.128.2 Typedef Documentation	449
20.129 stdint.h	450
20.130 gbdk-lib/include/stdio.h File Reference	454
20.130.1 Detailed Description	454
20.130.2 Function Documentation	454
20.131 stdio.h	455
20.132 gbdk-lib/include/stdlib.h File Reference	456
20.132.1 Function Documentation	456
20.133 stdlib.h	459
20.134 gbdk-lib/include/stdnoreturn.h File Reference	460
20.134.1 Macro Definition Documentation	460
20.135 stdnoreturn.h	460
20.136 gbdk-lib/include/asm/mos6502/string.h File Reference	460
20.136.1 Detailed Description	461
20.136.2 Macro Definition Documentation	461
20.136.3 Function Documentation	461
20.137 string.h	465
20.138 gbdk-lib/include/asm/sm83/string.h File Reference	465
20.138.1 Detailed Description	466
20.138.2 Function Documentation	466
20.138.3 Variable Documentation	469
20.139 string.h	469

20.140 gbdk-lib/include/asm/z80/string.h File Reference	470
20.140.1 Detailed Description	470
20.140.2 Function Documentation	470
20.141 string.h	473
20.142 gbdk-lib/include/string.h File Reference	474
20.142.1 Detailed Description	474
20.143 string.h	474
20.144 gbdk-lib/include/time.h File Reference	474
20.144.1 Detailed Description	474
20.144.2 Macro Definition Documentation	474
20.144.3 Typedef Documentation	475
20.144.4 Function Documentation	475
20.145 time.h	475
20.146 gbdk-lib/include/typeof.h File Reference	475
20.146.1 Macro Definition Documentation	476
20.147 typeof.h	477
20.148 gbdk-lib/include/asm/mos6502/types.h File Reference	478
20.148.1 Detailed Description	478
20.148.2 Macro Definition Documentation	478
20.148.3 Typedef Documentation	478
20.149 types.h	479
20.150 gbdk-lib/include/asm/sm83/types.h File Reference	479
20.150.1 Detailed Description	480
20.150.2 Macro Definition Documentation	480
20.150.3 Typedef Documentation	480
20.151 types.h	480
20.152 gbdk-lib/include/asm/types.h File Reference	481
20.152.1 Detailed Description	481
20.152.2 Macro Definition Documentation	481
20.152.3 Typedef Documentation	482
20.153 types.h	483
20.154 gbdk-lib/include/asm/z80/types.h File Reference	484
20.154.1 Detailed Description	484
20.154.2 Macro Definition Documentation	484
20.154.3 Typedef Documentation	484
20.155 types.h	485
20.156 gbdk-lib/include/types.h File Reference	485
20.156.1 Detailed Description	486
20.156.2 Macro Definition Documentation	486
20.156.3 Typedef Documentation	486
20.157 types.h	486

1 General Documentation

- [Getting Started](#)
- [Links, Tools and Debugging](#)
- [Using GBDK](#)
- [Coding Guidelines](#)
- [ROM/SRAM Banking and MBCs](#)
- [Supported Consoles & Cross Compiling](#)
- [GBDK Toolchain](#)
- [Example Programs](#)
- [Frequently Asked Questions \(FAQ\)](#)
- [Migrating to new GBDK Versions](#)
- [GBDK Release Notes](#)
- [Toolchain settings](#)

1.1 Introduction

Welcome to GBDK-2020! The best thing to do is head over to the [Getting Started](#) section to get up and running.

If you are upgrading please check [GBDK Release Notes](#) and [Migrating to new GBDK Versions](#)

1.2 About the Documentation

This documentation is partially based on material written by the original GBDK authors in 1999 and updated for GBDK-2020. The API docs are automatically generated from the C header files using Doxygen.

GBDK-2020 is an updated version of the original GBDK with a modernized SDCC toolchain and many API improvements and fixes. It can be found at: <https://github.com/gbdk-2020/gbdk-2020/>.

The original GBDK sources, documentation and website are at: <http://gbdk.sourceforge.net/>

1.3 About GBDK

The GameBoy Developer's Kit (GBDK, GBDK-2020) is used to develop games and programs for the Nintendo Game Boy (and some other consoles) in C and assembly. GBDK includes a set of libraries for the most common requirements and generates image files for use with a real GameBoy or emulators.

GBDK features:

- C and ASM toolchain based on SDCC with some support utilities
- A set of libraries with source code
- Example programs in ASM and in C
- Support for multiple ROM bank images and auto-banking
- Support for multiple consoles: Game Boy, Analogue Pocket, Mega Duck, Master System and Game Gear and NES

GBDK is freeware. Most of the tooling code is under the GPL. The runtime libraries should be under the LGPL. Please consider mentioning GBDK in the credits of projects made with it.

1.4 Historical Info and Links

Work on the original GBDK (pre-2020) was by:

Pascal Felber, Lars Malmberg, Michael Hope, David Galloway (djmijs), John Fuge, and others.

The following is from the original GBDK documentation:

Thanks to quang for many of the comments to the gb functions. Some of the comments are ripped directly from the Linux Programmers manual, and some directly from the pan/k00Pa document.

quangDX.com

The (original) gbdk homepage

[Jeff Frohwein's GB development page](#). A extensive source of Game Boy related information, including GeeBee's GB faq and the pan/k00Pa document.

2 Getting Started

Follow the steps in this section to start using GBDK-2020.

2.1 1. Download a Release and unzip it

You can get the latest releases from here: <https://github.com/gbdk-2020/gbdk-2020/releases>

2.1.1 Known Issue: Windows and folder names with spaces on non-C drives

There is a known issue on Windows where sdcc will fail when run from folder names with spaces on non-C drives. For the time being the workaround is as follows (with `D:\My Stuff\` as an example folder):

- Run Windows Command as administrator
- Run: `fsutil.exe 8dot3name query D:`
 - Output: The volume state is: 1 (8dot3 name creation is disabled). The registry state is: 2 (Per volume setting - the default). Based on the above settings, 8dot3 name creation is disabled on D:
- Run: `fsutil 8dot3name set D: 0`
 - Output: Successfully enabled 8dot3name generation on D:
- Run: `fsutil.exe 8dot3name query D:`
 - Output: The volume state is: 0 (8dot3 name creation is enabled). The registry state is: 2 (Per volume setting - the default). Based on the above settings, 8dot3 name creation is enabled on D:
- Only folders created AFTER the setting has been enabled will get 8.3 filename support, renaming folders does NOT appear to generate 8.3 filename support. However it is possible to manually generate a short path name for an existing folder:
 - Run: `D:\>fsutil file setshortname "D:\My stuff" "mystuf~1"`

2.2 2. Compile Example projects

Make sure your GBDK-2020 installation is working correctly by compiling some of the included [example projects](#).

If everything works in the steps below and there are no errors reported then each project that was built should have its own .gb ROM file (or suitable extension for the other supported targets).

2.2.1 Windows (without Make installed):

Navigate to a project within the example projects folder ("`examples\gb\`" under your GBDK-2020 install folder) and open a command line. Then type:

```
compile
```

or

```
compile.bat
```

This should build the example project. You can also navigate into other example project folders and build in the same way.

2.2.2 Linux / macOS / Windows with Make installed:

Navigate to the example projects folder ("`examples/gb/`" under your GBDK-2020 install folder) and open a command line. Then type:

```
make
```

This should build all of the examples sequentially. You can also navigate into an individual example project's folder and build it by typing `make`.

2.2.2.1 macOS security warnings If you get a security warning on macOS that says ("... developer cannot be verified, macOS cannot verify that this app is free from malware"), it does not mean that GBDK is malware. It just means the GBDK toolchain binaries are not signed by Apple, so it won't run them without an additional step. You will need to unquarantine the files in the `bin` folder in order to run them. This can be fixed using the following steps.

Open a terminal and navigate to the gbdk bin folder ("`bin/`" under your GBDK-2020 install folder). Then type:

```
xattr -d com.apple.quarantine *
```

2.3 3. Use a Template

To create a new project use a template!

There are template projects included in the [GBDK example projects](#) to help you get up and running. Their folder names start with `template_`.

1. Copy one of the template folders to a new folder name.
2. If you moved the folder out of the GBDK examples then you **must** update the GBDK path variable and/or the path to LCC in the `Makefile` or `compile.bat` so that it will still build correctly.
3. Type `make` on the command line in that folder to verify it still builds.
4. Open `main.c` to start making changes.

2.4 4. If you use GBTD / GBMB, get the fixed version

If you plan to use GBTD / GBMB for making graphics, make sure to get the version with the `const` fix and other improvements. See [const_gbtd_gbmb](#).

2.5 5. Review Coding Guidelines

Take a look at the [coding guidelines](#), even if you have experience writing software for other platforms. There is important information to help you get good results and performance on the Game Boy.

If you haven't written programs in C before, check the [C tutorials section](#).

2.6 6. Hardware and Resources

If you have a specific project in mind, consider what hardware you want to target. It isn't something that has to be decided up front, but it can influence design and implementation.

What size will your game or program be?

- 32K Cart (no-MBC required)
- Larger than 32K (MBC required)
- See more details about [ROM Banking and MBCs](#)

What console platform(s) will it run on?

- Game Boy (GB/GBC)
- Analogue Pocket (AP)
- Sega Master System (SMS)

- Game Gear (GG)
- Mega Duck (DUCK)
- See [Supported Consoles & Cross Compiling](#)

If targeting the Game Boy, what hardware will it run on?

- Game Boy (& Game Boy Color)
- Game Boy Color only
- Game Boy & Super Game Boy
- See how to [set the compatibility type in the cartridge header](#). Read more about hardware differences in the [Pandocs](#)

2.7 7. Set up C Source debugging

Tracking down problems in code is easier with a debugger. Emulicious has a [debug adapter](#) that provides C source debugging with GBDK-2020.

2.8 8. Try a GBDK Tutorial

You might want to start off with a guided GBDK tutorial from the [GBDK Tutorials section](#).

- **Note:** Tutorials (or parts of them) may be based on the older GBDK from the 2000's before it was updated to be GBDK-2020. The general principles are all the same, but the setup and parts of the [toolchain](#) (compiler/etc) may be somewhat different and some links may be outdated (pointing to the old GBDK or old tools).

2.9 9. Read up!

- It is strongly encouraged to read more [GBDK-2020 General Documentation](#).
- Learn about the Game Boy hardware by reading through the [Pandocs](#) technical reference.

2.10 10. Need help?

Check out the links for [online community and support](#) and read the [FAQ](#).

2.11 Migrating From Pre-GBDK-2020 Tutorials

Several popular GBDK Tutorials, Videos and How-to's were made before GBDK-2020 was available, as a result some information they include is outdated or incompatible. The following summarizes changes that should be made for best results.

2.11.1 Also see:

- [Migrating to new GBDK Versions](#)
- [Coding Guidelines](#)
- [Getting Started](#) (the section above this)

2.11.2 Use auto-banking

GBDK-2020 now supports auto-banking ([rom_autobanking](#)). In most cases using auto-banking will be easier and less error prone than manually assigning source and assets to banks.

- There is a source example `banks_autobank` project.

2.11.3 Non-standard types (UINT8, etc)

The old GBDK types `UINT8`, `INT8`, `UINT16`, `INT16` are non-standard and less portable.

The following should be used instead: `uint8_t`, `int16_t`, `uint16_t`, `int32_t`, `uint32_t` and `bool`.

These are standard types defined in `stdint.h` (`#include <stdint.h>`) and `stdbool.h` (`#include <stdbool.h>`).

2.11.4 If using GBTD / GBMB, get the fixed version

If you plan to use GBTD / GBMB for making graphics, make sure to get the version with the `const` fix and other improvements. See [const_gbtd_gbmb](#).

2.11.5 LCC and SDCC flags that are not needed

The following flag is no longer needed with `lcc` and `sdcc`, it can be removed without any loss of performance.

- `-DUSE_SFR`
 - Behavior formerly enabled by `USE_SFR_FOR_REG` is on by default now (no need to specify it, it isn't a tested `#ifdef` anymore). Check here why: <https://gbdev.gg8.se/forums/viewtopic.php?id=697>

2.11.6 ROM Header Settings (such as Color, SGB, etc)

Setting ROM bytes directly with `-Wl-yp0x<address>=0x<value>` is no longer supported. Instead use `makebin` flags. For example, use `-Wm-yC` instead of `-Wl-yp0x143=0xC0`. See [faq_gb_type_header_setting](#).

2.11.7 GBDK Header include changes

The following header files which are now cross platform were moved from `gb/` to `gbdk/`: `bcd.h`, `console.h`, `far_ptr.h`, `font.h`, `gbdecompress.h`, `gbdk-lib.h`, `incbin.h`, `metasprites.h`, `platform.h`, `version.h`

- When including them use `#include <gbdk/...>` instead of `#include <gb/>`

2.11.8 Include .h headers, not .c source files

Do not `#include` `.c` source files into other `.c` source files. Instead create `.h` header files for them and include those.

- https://www.tutorialspoint.com/cprogramming/c_header_files.htm

2.11.9 Use the Template Projects

Modern project templates are included with GBDK-2020. Using them (and their Makefile or compile.bat) as a starting point for projects is recommended and can help ensure better default settings and project organization.

2.11.10 Use hUGTracker instead of gbt_player

hUGTracker and its driver [hUGDriver](#) are smaller, more efficient and more versatile than `gbt_player`.

3 Links, Tools and Debugging

This is a brief list of useful tools and information. It is not meant to be complete or exhaustive, for a larger list see the [Awesome Game Boy Development](#) list.

3.1 SDCC Compiler Suite User Manual

- GBDK-2020 uses the SDCC compiler and related tools. The SDCC manual goes into much more detail about available features and how to use them.
<http://sdcc.sourceforge.net/doc/sdccman.pdf>
<http://sdcc.sourceforge.net>
- The SDCC assembler and linker (sdas / asxxxx and aslink) manual.
<https://sourceforge.net/p/sdcc/code/HEAD/tree/trunk/sdcc/sdas/doc/asmlnk.%20txt>

3.2 Getting Help

- GBDK Discord community:
<https://github.com/gbdk-2020/gbdk-2020/#discord-servers>
- Game Boy discussion forum:
<https://gbdev.gg8.se/forums/>

3.3 Game Boy Documentation

- **Pandocs**
Extensive and up-to-date technical documentation about the Game Boy and related hardware.
<https://gbdev.io/pandocs/>
- **Awesome Game Boy Development list**
A list of Game Boy/Color development resources, tools, docs, related projects and homebrew.
<https://gbdev.io/resources.html>

3.4 Sega Master System / Game Gear Documentation

- **SMS Power!**
Community site with technical documentation, reviews and other content related to the Sega 8-bit systems.
<https://www.smspower.org/>

3.5 Mega Duck / Cougar Boy Documentation

- **MegaDuck.dev**
A collection of technical information and resources for the Mega Duck / Cougar Boy console.
<https://megaduck.dev/>

3.6 Tutorials

- **Larold's Jubilant Junkyard Tutorials**
Several walk throughs about the fundamentals of developing for the Game Boy with GBDK-2020. There are simple examples with source code.
<https://laroldsjubilantjunkyard.com/tutorials/>
- **Gaming Monsters Tutorials**
Several video tutorials and code for making games with GBDK/GBDK-2020.
<https://www.youtube.com/playlist?list=PLeEj4c2zF7PaFv5MPYhNAkBGkx4i%20PGJo>
<https://github.com/gingemonster/GamingMonstersGameBoySampleCode>
- **Pocket League Tutorial**
<https://blog.ty-porter.dev/development/2021/04/04/writing-a-gameboy-game-in-2021-pt1.html>

3.7 Example code

- **Simplified GBDK examples**

https://github.com/mrombout/gbdk_playground/commits/master

3.8 Graphics Tools

-

- **Game Boy Tile Designer and Map Builder (GBTD / GBMB)**

Sprite / Tile editor and Map Builder that can export to C that works with GBDK.

This is an updated version with const export fixed and other improvements.

https://github.com/gbdk-2020/GBTD_GBMB

- A GIMP plugin to read/write GBR/GBM files and do map conversion:

<https://github.com/bbbbr/gimp-tilemap-gb>

- Command line version of the above tool that doesn't require GIMP (png2gbtiles):

<https://github.com/bbbbr/gimp-tilemap-gb/tree/master/console>

- **Tilemap Studio**

A tilemap editor for Game Boy, GBC, GBA, or SNES projects.

<https://github.com/Rangi42/tilemap-studio/>

3.9 Music And Sound Effects for the Game Boy

- **hUGEdriver and hUGEdriver**

A tracker and music driver that work with GBDK and RGBDS. It is smaller, more efficient and more versatile than gbt_player.

<https://github.com/SuperDisk/hUGEDriver>

<https://github.com/SuperDisk/hUGETracker>

- **CBT-FX**

A sound effects driver which can play effects created in FX Hammer.

<https://github.com/datguywitha3ds/CBT-FX>

- **VGM2GBSFX**

A sound effects converter and driver for DMG VGM files, FX Hammer and PCM WAV files.

<https://github.com/untoxa/VGM2GBSFX>

- **GBT Player**

A .mod converter and music driver that works with GBDK and RGBDS.

<https://github.com/AntonioND/gbt-player>

Docs from GBStudio that should mostly apply: <https://www.gbstudio.dev/docs/music/>

3.10 Music And Sound Effects for the SMS/Game Gear

- **Banjo**

A sound driver for Sega Master System and Game Gear which can import from Furnace tracker.

- <https://github.com/joffb/banjo> An example of using it with GBDK can be found in the CrossZGB engine.

- <https://github.com/gbdk-2020/CrossZGB/commit/8ecd46639f3af420cfe139fc155740e8aa>

3.11 Emulators

- **Emulicious**
An accurate emulator with extensive tools including source level debugging. <https://emulicious.net/>
- **BGB**
Accurate emulator, has useful debugging tools.
<http://bgb.bircd.org/>
- **Super Junior SameDuck** The only MegaDuck emulator with support for the Mega Duck Laptop models (Quique and Junior Super Computer) <https://github.com/bbbbr/SuperJuniorSameDuck>

Intellisense in VSCode may have trouble identifying some GBDK types or functions, and therefore flag them as warnings or unidentified.

GBDK platform constants can be declared so that header files are parsed more completely in VSCode. The following `c_cpp_properties.json` example may be adapted for your own project.

```
{
  "configurations": [
    {
      "name": "gameboy",
      "includePath": [
        "${workspaceFolder}/src/**",
        "${workspaceFolder}/res/**",
        "${workspaceFolder}/include/**",
        "${workspaceFolder}/../../../../gbdk/include/**"
      ],
      "defines": ["__PORT_sm83", "__TARGET_gb"],
      "compilerPath": "",
      "cStandard": "c11",
      "intelliSenseMode": "${default}",
      "compilerArgs": [],
      "browse": {
        "limitSymbolsToIncludedHeaders": true
      }
    }
  ],
  "version": 4
}
```

3.12 Debugging tools

- **Emulicious debug adapter**
Provides source-level debugging in VS Code and Sublime Text that works with GBDK2020.
<https://marketplace.visualstudio.com/items?itemName=emulicious.emulicious-debugger>
 - If compiler optimization is making the program source hard to step through in the debugger then adding this flag to `lcc` can help. Note that using this flag will likely reduce code performance and increase code size while enabled, so it is best to only use it temporarily.


```
* -Wf--max-allocs-per-node0
```
- **romusage**
Calculate used and free space in banks (ROM/RAM) and warn about errors such as bank overflows.
See [romusage-settings](#)
- **noi file to sym conversion for bgb**
Debug information in .noi files can be converted to a symbol format that **BGB** recognizes using:
 - `lcc` : `-Wm-yS` (with `--debug`, or `-Wl-j` to create the .noi)
 - directly with [makebin](#) : `-yS` (with `-j` passed to the linker)
- **src2sym.pl**
Add line-by-line C source code to the main symbol file in a BGB compatible format. This allows for C source-like debugging in BGB in a limited way. <https://gbdev.gg8.se/forums/viewtopic.php?id=710>

3.13 Optimizing Assembly

- **Optimizing Assembly Code**

Pret has a useful guide to optimizing assembly for the Game Boy for times when asm is used in a project in addition to C. <https://github.com/pret/pokecrystal/wiki/Optimizing-assembly-code>

3.14 Continuous Integration and Deployment

- **GBDK GitHub Action Builder**

A Github Action which provides basic CI/CD for building projects based on GBDK (not for building GBDK itself).

<https://github.com/wujoood/gbdk-2020-github-builder>

4 Using GBDK

4.1 Interrupts

Interrupts allow execution to jump to a different part of your code as soon as an external event occurs - for example the LCD entering the vertical blank period, serial data arriving or the timer reaching its end count. For an example see the `irq.c` sample project.

Interrupts in GBDK are handled using the functions `disable_interrupts()`, `enable_interrupts()`, `set_interrupts(uint8_t ier)` and the interrupt service routine (ISR) linkers `add_VBL()`, `add_TIM`, `add_low_priority_TIM`, `add_LCD`, `add_SIO` and `add_JOY` which add interrupt handlers for the vertical blank, timer, LCD, serial link and joypad interrupts respectively.

Since an interrupt can occur at any time an Interrupt Service Request (ISR) cannot take any arguments or return anything. Its only way of communicating with the greater program is through the global variables. When interacting with those shared ISR global variables from main code outside the interrupt, it is a good idea to wrap them in a `critical {}` section in case the interrupt occurs and modifies the variable while it is being used.

Interrupts should be disabled before adding ISRs. To use multiple interrupts, *logical OR* the relevant IFLAGS together.

ISRs should be kept as small and short as possible, do not write an ISR so long that the Game Boy hardware spends all of its time servicing interrupts and has no time spare for the main code.

For more detail on the Game Boy interrupts consider reading about them in the [Pandocs](#).

4.1.1 Available Interrupts

The GameBoy hardware can generate 5 types of interrupts. Custom Interrupt Service Routines (ISRs) can be added in addition to the built-in ones available in GBDK.

- VBL : LCD Vertical Blanking period start
 - The default VBL ISR is installed automatically.
 - * See `add_VBL()` and `remove_VBL()`
- LCD : LCDC status (such as the start of a horizontal line)
 - See `add_LCD()` and `remove_LCD()`
 - Example project: `lcd_isr_wobble`
- TIM : Timer overflow
 - See `add_TIM()` (or `add_low_priority_TIM()`) and `remove_TIM()`
 - Example project: `tim`
- SIO : Serial Link I/O transfer end
 - The default SIO ISR gets installed automatically if any of the standard SIO calls are used (`send_byte()`, `receive_byte()`).

- Once installed the default SIO ISR cannot be removed. Only secondary chained SIO ISRs (added with [add_SIO\(\)](#)) can be removed.
- See [add_SIO\(\)](#) and [remove_SIO\(\)](#)
- Example project: `comm`
- JOY : Transition from high to low of a joystick button
 - See [add_JOY\(\)](#) and [remove_JOY\(\)](#)

4.1.2 Adding your own interrupt handler

It is possible to install your own interrupt handlers (in C or in assembly) for any of these interrupts. Up to 4 chained handlers may be added, with the last added being called last. If the [remove_VBL\(\)](#) function is to be called, only three may be added for VBL.

Interrupt handlers are called in sequence. To install a new interrupt handler, do the following:

1. Write a function (say `foo()`) that takes no parameters, and that returns nothing. Remember that the code executed in an interrupt handler must be short.
2. Inside a `__critical { ... }` section, install your interrupt handling routines using the `add_XXX()` function, where XXX is the interrupt that you want to handle.
3. Enable interrupts for the IRQ you want to handle, using the [set_interrupts\(\)](#) function. Note that the VBL interrupt is already enabled before the `main()` function is called. If you want to set the interrupts before `main()` is called, you must install an initialization routine.

See the `irq` example project for additional details for a complete example.

4.1.3 Using your own Interrupt Dispatcher

If you want to use your own Interrupt Dispatcher instead of the GBDK chained dispatcher (for improved performance), then don't call the `add_...()` function for the respective interrupt and its dispatcher won't be installed.

- Exception: the VBL dispatcher will always be linked in at compile time.
- For the SIO interrupt, also do not make any standard SIO calls to avoid having its dispatcher installed.

Then, [ISR_VECTOR\(\)](#) or [ISR_NESTED_VECTOR\(\)](#) can be used to install a custom ISR handler.

4.1.4 Returning from Interrupts and STAT mode

By default when an Interrupt handler completes and is ready to exit it will check `STAT_REG` and only return at the BEGINNING of either LCD Mode 0 or Mode 1. This helps prevent graphical glitches caused when an ISR interrupts a graphics operation in one mode but returns in a different mode for which that graphics operation is not allowed. You can change this behavior using [nowait_int_handler\(\)](#) which does not check `STAT_REG` before returning. Also see [wait_int_handler\(\)](#).

4.2 What GBDK does automatically and behind the scenes

4.2.1 NES console

For implementation details on the NES console in GBDK, see the [NES entry](#) in [Supported Consoles & Cross Compiling](#)

4.2.2 OAM (VRAM Sprite Attribute Table)

GBDK sets up a Shadow OAM which gets copied automatically to the hardware OAM by the default V-Blank ISR. The Shadow OAM allows updating sprites without worrying about whether it is safe to write to them or not based on the hardware LCD mode.

4.2.3 Graphics Tile Maps and Data on Startup

By default for the Game Boy GBDK assigns:

- Background and Window Tile data starting at 0x8800
- Background Tile Map starting at 0x9800
- Window Tile Map starting at 0x9C00
- Sprites to 8x8 mode

4.2.4 Font tiles when using stdio.h

Including [stdio.h](#) and using functions such as [printf\(\)](#) will use a large number of the background tiles for font characters. If [stdio.h](#) is not included then that space will be available for use with other tiles instead.

4.2.5 Default Interrupt Service Handlers (ISRs)

- V-Blank: A default V-Blank ISR is installed on startup which copies the Shadow OAM to the hardware OAM and increments the global [sys_time](#) variable once per frame.
- Serial Link I/O: If any of the GBDK serial link functions are used such as [send_byte\(\)](#) and [receive_byte\(\)](#), the default SIO serial link handler will be installed automatically at compile-time.
- APA Graphics Mode: When this mode is used (via [drawing.h](#)) a custom LCD ISR handler will be installed ([drawing_lcd](#)). Changing the mode to `(mode(M_TEXT_OUT) ;)` will cause them to be de-installed. These handlers are used to change the tile data source at start-of-frame and mid-frame so that 384 background tiles can be used instead of the typical 256.

4.2.6 Ensuring Safe Access to Graphics Memory

There are certain times during each video frame when memory and registers relating to graphics are "busy" and should not be read or written to (otherwise there may be corrupt or dropped data). GBDK handles this automatically for most graphics related API calls. It also ensures that ISR handlers return in such a way that if they interrupted a graphics access then it will only resume when access is allowed.

The ISR return behavior [can be turned off](#) using the [nowait_int_handler](#).

For more details see the related Pandocs section: https://gbdev.io/pandocs/Accessing_VRAM_and_OAM.html

4.3 Compression

For programs that would benefit from compression GBDK includes the [gbcompress](#) utility and companion API functions.

In addition to the built-in compression unpack is another option:

- UnPACK aPack decompression by Toxa: <https://github.com/untoxa>
- apultra aPack compression: <https://github.com/emmanuel-marty/apultra>

Another way to save space is using 1 bit-per-pixel (bpp) tile pattern data instead of 2-bpp or 4-bpp data. This can reduce the ROM size for groups of tiles which only require two shades of color.

- See: [set_1bpp_colors\(\)](#), [set_bkg_1bpp_data\(\)](#), [set_win_1bpp_data\(\)](#), [set_sprite_1bpp_data\(\)](#)

Use of 1-bpp tile pattern data may be combined with the compression described above to save even more space, however that approach requires using an intermediary RAM buffer before the tile pattern data can be written to VRAM with the [set_*_1bpp_data\(\)](#) functions.

4.4 Copying Functions to RAM and HIRAM

See the `ram_function` example project included with GBDK which demonstrates copying functions to RAM and HIRAM.

Warning! Copying of functions is generally not safe since they may contain jumps to absolute addresses that will not be converted to match the new location.

It is possible to copy functions to RAM and HIRAM (using the `memcpy()` and `hramcpy()` functions), and execute them from C. Ensure you have enough free space in RAM or HIRAM for copying a function.

There are basically two ways for calling a function located in RAM, HIRAM, or ROM:

- Declare a pointer-to-function variable, and set it to the address of the function to call.
- Declare the function as extern, and set its address at link time using the `-Wl-gXXX=#` flag (where XXX is the name of the function, and # is its address).

The second approach is slightly more efficient. Both approaches are demonstrated in the `ram_function.c` example.

4.5 Mixing C and Assembly

The following is primarily oriented toward the Game Boy and related clones (sm83 devices), other targets such as sms/gg may vary.

You can mix C and assembly (ASM) in two ways as described below.

- For additional detail see the [links_sdcc_docs](#) and [SDCC Calling Conventions](#).

4.5.1 Inline ASM within C source files

- The optional `NAKED` keyword may be used to indicate that the function setup and return should have no handling done by the compiler, and will instead be handled entirely by user code.
- If the entire function preserves some registers the optional `PRESERVES_REGS` keyword may be used as additional hinting for the compiler. For example `PRESERVES_REGS(b, c)`. By default it is assumed by the compiler that no registers are preserved.

Example:

```
__asm__("nop");
```

Another Example:

```
void some_c_function()
{
    // Optionally do something
    __asm
        (ASM code goes here)
    __endasm;
}
```

4.5.2 In Separate ASM files

It is possible to assemble and link files written in ASM alongside files written in C.

- A C identifier `i` will be called `_i` in assembly.
- Parameters will be passed, registers saved and results returned in a manner based on the [SDCC Calling Convention](#) used and how the function is declared.
- Assembly identifiers are exported using the `.globl` directive.
- See `global.s` for examples of hardware register definitions.

Here is an example of how to mix assembly with C:

`main.c`

```

uint16_t add(uint16_t, uint16_t);

main()
{
    uint16_t i;

    i = add(1, 3);
}

add.s

.globl _add

.area _CODE
_add:      ; uint16_t add(uint16_t First, uint16_t Second)
            ;
            ; In this particular example there is no use and modification of the stack
            ; No need to save and restore registers
            ;
            ; For calling convention __sdcccall(1)
            ; - first 16 bit param is passed in DE
            ; - second 16 bit param is passed in BC

; Load Second Parameter ("Second") into HL
ld  l,  c
ld  h,  b

; Add Parameters "Second" + "First"
add hl, de

; Return result in BC
ld  c,  l
ld  b,  h
ret      ; 16 bit values are returned in BC

```

4.6 Including binary files in C source with incbin

Data from binary files can be included in C source files as a const array using the [INCBIN\(\)](#) macro. See the `incbin` example project for a demo of how to use it.

4.7 Known Issues and Limitations

4.7.1 SDCC

- Const arrays declared with `somevar[n] = {x}` will **NOT** get initialized with value `x`. This may change when the SDCC RLE initializer is fixed. Use `memset` for now if you need it.
- SDCC banked calls and [far pointers](#) in GBDK only save one byte for the ROM bank, so for example they are limited to **bank 15** max for MBC1 and **bank 255** max for MBC5. See [banked_calls](#) for more details.
- In SDCC **pre-initializing a variable** assigned to SRAM with `-Wf-ba*` will force that variable to be in WRAM instead.
 - The following is a workaround for initializing a variable in SRAM. It assigns value `0xA5` to a variable in bank 0 and assigned to address `0xA000` using the [AT\(\)](#) directive:

```

// Workaround for initializing variable in SRAM
// (MBC RAM and Bank needs to get enabled during GSINIT loading)
static uint8_t AT(0x0000) __rAMG = 0x0a; // Enable SRAM
static uint8_t AT(0x4000) __rAMB = 0x00; // Set SRAM bank 0
// Now SRAM is enabled so the variable can get initialized
uint8_t AT(0xA000)      initialized_sram_var = 0xA5u;

```

5 Coding Guidelines

5.1 Learning C / C fundamentals

Writing games and other programs with GBDK will be much easier with a basic understanding of the C language. In particular, understanding how to use C on "Embedded Platforms" (small computing systems, such as the Game

Boy) can help you write better code (smaller, faster, less error prone) and avoid common pitfalls.

5.1.1 General C tutorials

- <https://www.learn-c.org/>
- <https://www.tutorialspoint.com/cprogramming/index.htm>
- <https://www.chiark.greenend.org.uk/~sgtatham/cdescent/>

5.1.2 Embedded C introductions

- <http://dsp-book.narod.ru/CPES.pdf>
- <https://www.phaedsys.com/principals/bytecraft/bytecraftdata/bcfirststeps.pdf>

5.1.3 Game Boy games in C

- <https://gbdev.io/resources.html#c>

5.2 Understanding the hardware

In addition to understanding the C language it's important to learn how the Game Boy hardware works. What it is capable of doing, what it isn't able to do, and what resources are available to work with. A good way to do this is by reading the [Pandocs](#) and checking out the [awesome_gb](#) list.

5.3 Writing optimal C code for the Game Boy and SDCC

The following guidelines can result in better code for the Game Boy, even though some of the guidance may be contrary to typical advice for general purpose computers that have more resources and speed.

5.3.1 Tools

5.3.1.1 GBTD / GBMB, Arrays and the "const" keyword Important: The old [GBTD/GBMB](#) fails to include the `const` keyword when exporting to C source files for GBDK. That causes arrays to be created in RAM instead of ROM, which wastes RAM, uses a lot of ROM to initialize the RAM arrays and slows the compiler down a lot.

___Use of [toxa's updated GBTD/GBMB](#) is highly recommended.___

If you wish to use the original tools, you must add the `const` keyword every time the graphics are re-exported to C source files.

5.3.2 Avoid Reading from VRAM

In general avoid reading from VRAM since that memory is not accessible at all times. If GBDK a API function which reads from VRAM (such as [get_bkg_tile_xy\(\)](#)) is called during a video mode when VRAM is not accessible, then that function call will delay until VRAM becomes accessible again. This can cause unnecessary slowdowns when running programs on the Game Boy. It is also not supported by GBDK on the NES platform.

Instead it is better to store things such as map data in general purpose RAM which does not have video mode access limitations.

For more information about video modes and VRAM access see the pan docs:

<https://gbdev.io/pandocs/STAT.html#stat-modes>

5.3.3 Variables

- Use 8-bit values as much as possible. They will be much more efficient and compact than 16 and 32 bit types.
- Prefer unsigned variables to signed ones: the code generated will be generally more efficient, especially when comparing two values.

- Use explicit types so you always know the size of your variables. `int8_t`, `uint8_t`, `int16_t`, `uint16_t`, `int32_t`, `uint32_t` and `bool`. These are standard types defined in `stdint.h` (`#include <stdint.h>`) and `stdbool.h` (`#include <stdbool.h>`).
- Global and local static variables are generally more efficient than local non-static variables (which go on the stack and are slower and can result in slower code).
 - An exception to this when there are a small number of local variables (one or two) and the code is not complex. Then the compiler may allocate those variables to CPU registers instead which may be faster.
 - Functions which use global or static local variables will lose re-entrancy. In most cases it is not a problem, but important to keep in mind.
 - In particular avoid putting big arrays on the stack, consider static local or global.
- Keep the number of arguments passed to functions small (ideally one or two arguments at most). When there are a large number of arguments they get pushed onto the stack and result in more overhead for function calls. See the Calling Conventions in the SDCC compiler manual for details.
- `const` keyword: use `const` for arrays, structs and variables with read-only (constant) data. It will reduce ROM, RAM and CPU usage significantly. Non-`const` values are loaded from ROM into RAM inefficiently, and there is no benefit in loading them into the limited available RAM if they aren't going to be changed.
- Here is how to declare `const` pointers and variables:
 - non-const pointer to a const variable: `const uint8_t * some_pointer;`
 - const pointer to a non-const variable: `uint8_t * const some_pointer;`
 - const pointer to a const variable: `const uint8_t * const some_pointer;`
 - <https://codeforwin.org/2017/11/constant-pointer-and-pointer-to-constant-in-c.html>
 - <https://stackoverflow.com/questions/21476869/constant-pointer-vs-pointer-to-const>
- For calculated values that don't change, pre-compute results once and store the result. Using lookup-tables and similar approaches can improve speed and reduce code size. Macros can sometimes help. It may be beneficial to do the calculations with an outside tool and then include the result as C code in a `const` array.
- Use an advancing pointer (`someStruct->var = x; someStruct++`) to loop through arrays of structs instead of using indexing each time in the loop `someStruct[i].var = x`.
- When modifying variables that are also changed in an Interrupt Service Routine (ISR), wrap them the relevant code block in a `__critical { }` block. See <http://sdcc.sourceforge.net/doc/sdccman.pdf#section.3.9>
- When using constants and literals the `U`, `L` and `UL` postfixes can be used.
 - `U` specifies that the constant is unsigned
 - `L` specifies that the constant is long.
 - NOTE: In SDCC 3.6.0, the default for `char` changed from signed to unsigned. The manual says to use `--fsigned-char` for the old behavior, this option flag is included by default when compiling through `lcc`.
- A fixed point type (`fixed`) is included with GBDK when precision greater than whole numbers is required for 8 bit range values (since floating point is not included in GBDK).

See the "Simple Physics" sub-pixel example project.
Code example:

```
fixed player[2];
...
// Modify player position using its 16 bit representation
player[0].w += player_speed_x;
player[1].w += player_speed_y;
...
// Use only the upper 8 bits for setting the sprite position
move_sprite(0, player[0].h, player[1].h);
```


5.3.4 Code structure

- Do not `#include .c` source files into other `.c` source files. Instead create `.h` header files for them and include those. https://www.tutorialspoint.com/cprogramming/c_header_files.htm
- Instead of using a blocking `delay()` for things such as sprite animations/etc (which can prevent the rest of the game from continuing) many times it's better to use a counter which performs an action once every N frames. `sys_time` may be useful in these cases.
- When processing for a given frame is done and it is time to wait before starting the next frame, `vsync()` can be used. It uses HALT to put the CPU into a low power state until processing resumes. The CPU will wake up and resume processing at the end of the current frame when the Vertical Blanking interrupt is triggered.
- Minimize use of multiplication, modulo with non-powers of 2, and division with non-powers of 2. These operations have no corresponding CPU instructions (software functions), and hence are time costly.
 - SDCC has some optimizations for:
 - * Division by powers of 2. For example `n /= 4u` will be optimized to `n >>= 2`.
 - * Modulo by powers of 2. For example: `(n % 8)` will be optimized to `(n & 0x7)`.
 - If you need decimal numbers to count or display a score, you can use the GBDK BCD (`binary coded decimal`) number functions. See: [bcd.h](#) and the BCD example project included with GBDK.
- Avoid long lists of function parameters. Passing many parameters can add overhead, especially if the function is called often. Globals and local static vars can be used instead when applicable.
- Use inline functions if the function is short (with the `inline` keyword, such as `inline uint8_t myFunction() { ... }`).
- Do not use recursive functions.

5.3.5 GBDK API/Library

- `stdio.h`: If you have other ways of printing text, avoid including `stdio.h` and using functions such as `printf()`. Including it will use a large number of the background tiles for font characters. If `stdio.h` is not included then that space will be available for use with other tiles instead.
- `drawing.h`: The Game Boy graphics hardware is not well suited to frame-buffer style graphics such as the kind provided in `drawing.h`. Due to that, most drawing functions (rectangles, circles, etc) will be slow. When possible it's much faster and more efficient to work with the tiles and tile maps that the Game Boy hardware is built around.
- `waitpad()` and `waitpadup` check for input in a loop that doesn't HALT at all, so the CPU will be maxed out until it returns. One alternative is to write a function with a loop that checks input with `joypad()` and then waits a frame using `vsync()` (which idles the CPU while waiting) before checking input again.
- `joypad()`: When testing for multiple different buttons, it's best to read the joypad state *once* into a variable and then test using that variable (instead of making multiple calls).

5.3.6 Toolchain

- See SDCC optimizations: <http://sdcc.sourceforge.net/doc/sdccman.pdf#section.8.1>
- For details about default Compiler data types, see the SDCC Manual (follow links and scroll down 1 page)
 - <https://sdcc.sourceforge.net/doc/sdccman.pdf#section.1.1>
 - Note: by default GBDK enables `--fsigned-char` (via `lcc`) for SDCC
- Use profiling. Look at the ASM generated by the compiler, write several versions of a function, compare them and choose the faster one.

- Use the SDCC `--max-allocs-per-node` flag with large values, such as 50000. `--opt-code-speed` has a much smaller effect.
 - GBDK-2020 (after v4.0.1) compiles the library with `--max-allocs-per-node 50000`, but it must be turned on for your own code.
(example: `lcc ... -Wf--max-allocs-per-node50000` or `sdcc ... --max-allocs-per-node 50000`).
 - The other code/speed flags are `--opt-code-speed` or `--opt-code-size`.
- Use current SDCC builds from <http://sdcc.sourceforge.net/snap.php>
The minimum required version of SDCC will depend on the GBDK-2020 release. See [GBDK Release Notes](#)
- Learn some ASM and inspect the compiler output to understand what the compiler is doing and how your code gets translated. This can help with writing better C code and with debugging.

5.3.7 Constants, Signed-ness and Overflows

There are some scenarios where the compiler will warn about overflows with constants. They often have to do with mixed signedness between constants and variables. To avoid problems use care about whether or not constants are explicitly defined as unsigned and what type of variables they are used with.

WARNING: overflow in implicit constant conversion

- A constant can be used where the value is too high (or low) for the storage medium causing an value overflow.
 - For example this constant value is too high since the max value for a signed 8 bit char is 127.
- This can also happen when constants are not explicitly declared as unsigned (and so may get treated by the compiler as signed) and then added such that the resulting value exceeds the signed maximum.
 - For example, this results in an warning even though the sum total is 254 which is less than the 255, the max value for a unsigned 8 bit char variable.

```
#define TOO_LARGE_CONST 255
int8_t signed_var = TOO_LARGE_CONST;
```

```
#define CONST_UNSIGNED 127u
#define CONST_SIGNED 127
uint8_t unsigned_var = (CONST_SIGNED + CONST_UNSIGNED);
```

- It can be avoided by always using the unsigned `u` when the constant is intended for unsigned operations.

```
#define CONST_UNSIGNED 127u
#define CONST_ALSO_UNSIGNED 127u // <-- Added "u", now no warning
uint8_t unsigned_var = (CONST_UNSIGNED + CONST_ALSO_UNSIGNED);
```

5.3.8 Chars and vararg functions

Parameters (chars, ints, etc) to `printf` / `sprintf` should always be explicitly cast to avoid type related parameter passing issues.

For example, below will result in the likely unintended output:

```
printf(str_temp, "%u, %d, %x\n", UINT16_MAX, INT16_MIN, UINT16_MAX);
// Will output: "65535, 0, 8000"
```

Instead this will give the intended output:

```
printf(str_temp, "%u, %d, %x\n", (uint16_t)UINT16_MAX, (int16_t)INT16_MIN, (uint16_t)UINT16_MAX);
// Will output: "65535, -32768, FFFF"
```

5.3.8.1 Chars In standard C when `chars` are passed to a function with variadic arguments (varargs, those declared with `...` as a parameter), such as `printf()`, those `chars` get automatically promoted to `ints`. For an 8 bit CPU such as the Game Boy's, this is not as efficient or desirable in most cases. So the default SDCC behavior, which GBDK-2020 expects, is that `chars` will remain `chars` and *not* get promoted to `ints` when **explicitly cast as chars while calling a varargs function**.

- They must be explicitly re-cast when passing them to a varargs function, even though they are already declared as `chars`.

- Discussion in SDCC manual:
<http://sdcc.sourceforge.net/doc/sdccman.pdf#section.1.5>
<http://sdcc.sourceforge.net/doc/sdccman.pdf#subsection.3.5.10>
- If SDCC is invoked with `-std-cxx` (`-std-c89`, `-std-c99`, `-std-c11`, etc) then it will conform to standard C behavior and calling functions such as `printf()` with chars may not work as expected.

For example:

```
unsigned char i = 0x5A;
// NO:
// The char will get promoted to an int, producing incorrect printf output
// The output will be: 5A 00
printf("%hx %hx", i, i);
// YES:
// The char will remain a char and printf output will be as expected
// The output will be: 5A 5A
printf("%hx %hx", (unsigned char)i, (unsigned char)i);
```

Some functions that accept varargs:

- `EMU_printf`, `gprintf()`, `printf()`, `sprintf()`

Also See:

- Other cases of char to int promotion: <http://sdcc.sourceforge.net/doc/sdccman.pdf#chapter.6>

5.4 When C isn't fast enough

For many applications C is fast enough but in intensive functions are sometimes better written in assembly. This section deals with interfacing your core C program with fast assembly sub routines.

5.4.1 Reusable Local Labels and Inline ASM

When functions are written assembly it's generally better to not mix the inline ASM with C code and instead write the whole function in assembly.

If they are mixed then descriptive named labels should not be used for inline ASM. This is due to descriptive labels interfering with the expected scope of the reusable local labels generated from the compiled C code. The compiler will not detect this problem and the resulting code may fail to execute correctly without warning.

Instead use reusable local symbols/labels (for example `1$:`). To learn more about them check the SDAS manual section "1.3.3 Reusable Symbols"

5.4.2 Variables and registers

Getting at C variables is slightly tricky due to how local variables are allocated on the stack. However you shouldn't be using the local variables of a calling function in any case. Global variables can be accessed by name by adding an underscore.

5.4.3 Segments / Areas

The use of segments/areas for code, data and variables is more noticeable in assembler. GBDK and SDCC define a number of default ones. The order they are linked is determined by `crt0.s` and is currently as follows for the Game Boy and related clones.

- ROM (in this order)
 - `__HEADER`: For the Game Boy header
 - `__CODE`: CODE is specified as after BASE, but is placed before it due to how the linker works.
 - `__HOME`
 - `__BASE`
 - `__CODE_0`
 - `__INITIALIZER`: Constant data used to init RAM data
 - `__LIT`

- `_GSINIT`: Code used to init RAM data
- `_GSFINAL`
- Banked ROM
 - `_CODE_x` Places code in ROM other than Bank 0, where x is the 16kB bank number.
- WRAM (in this order)
 - `_DATA`: Uninitialized RAM data
 - `_BSS`
 - `_INITIALIZED`: Initialized RAM data
 - `_HEAP`: placed after `_INITIALIZED` so that all spare memory is available for the malloc routines.
 - `STACK`: at the end of WRAM

5.4.4 Calling convention

The following is primarily oriented toward the Game Boy and related clones (sm83 devices), other targets such as sms/gg may vary.

SDCC in common with almost all C compilers prepends a `_` to any function names. For example the function `printf(...)` begins at the label `_printf::`. Note that all functions are declared global.

Functions can be marked with `OLDCALL` which will cause them to use the `__sdcccall(0)` calling convention (the format used prior to SDCC 4.2 & GBDK-2020 4.1.0).

Starting with SDCC 4.2 and GBDK-2020 4.1.0 the new default calling convention is `__sdcccall(1)`.

For additional details about the calling conventions, see sections SM83 calling conventions and Z80, Z180 and Z80N calling conventions in the SDCC manual.

- <http://sdcc.sourceforge.net/doc/sdccman.pdf>
- Section 4.3.9 isn't specific about it, but gbz80/sm83 generally share this subheading with z80 (Game Boy is partially a sub-port of z80 in SDCC). <https://sdcc.sourceforge.net/doc/sdccman.pdf#subsection.4.3.9>

5.4.4.1 Banked Calling Convention *The following is primarily oriented toward the Game Boy and related clones (sm83 devices), other targets such as sms/gg may vary.*

Key Points:

- Function arguments (if present) are always placed on the stack, right to left without particular alignment
- A fixed stack offset (sm83:+4, z80:+3) is added by the Callee (to skip the pushed Caller Bank and additional Trampoline Return Address)
- Return values follow the calling convention (`__sdcccall(1)`, or `__sdcccall(0)` for `OLDCALL`)

Terminology:

- **Caller**: the code which is calling the requested function
- **Callee**: the function to be called (declared as `BANKED` or `__banked`)
- **Trampoline**: The intermediary which performs the bank switching and does hand-off between Caller and Callee during the call and then return.

Banked Call Trampoline

- Banked calls are performed via a trampoline in the non-banked region 0000-3fff
- The `__sdcc_bcall_ehl` trampoline is used by default
 - With it both calling conventions are supported: `__sdcccall(1)` (default) or `__sdcccall(0)` for `OLDCALL`

- If `--legacy-banking` is specified to SDCC the `__sdcc_bcall` trampoline is used.
 - This may only be used with `__sdcccall(0)`

Process for a banked call (using `__sdcc_bcall_ehl`, the default)

1. The Caller

- Function arguments (if present) are always placed on the stack, right to left without particular alignment
- The Bank of Callee function is placed into register E
- The Address of Callee function is placed in HL
- Calls the bank switch Trampoline (which adds Caller return address to the stack)

2. The Trampoline

- Saves the Current Bank onto the stack (pushed as AF, so 16 bits)
- Switches to the Bank of Callee function (in register E)
- Calls the Callee function address in HL (which adds Trampoline return address to the stack)

3. The Callee Function

- SDCC will use an offset to skip the first N bytes of the stack
 - For `sm83` (GB/AP/DUCK): skip first 4 bytes
 - For `z80` (GG/SMS/etc): skip first 3 bytes
- Return values follow the calling convention (`__sdcccall(1)`, or `__sdcccall(0)` for `OLDCALL`)
- Executes a return to Trampoline

4. The Trampoline

- Switches to the Bank of the Caller saved on the stack (and moves Stack Pointer past it)
- Executes a return to Caller

5. The Caller

- Cleans up the stack and uses return value (if present)

6 ROM/SRAM Banking and MBCs

6.1 ROM/SRAM Banking and MBCs (Memory Bank Controllers)

The standard Game Boy cartridge with no MBC has a fixed 32K bytes of ROM. In order to make cartridges with larger ROM sizes (to store more code and graphics) MBCs can be used. They allow switching between multiple ROM banks that use the same memory region. Only one of the banks can be selected as active at a given time, while all the other banks are inactive (and so, inaccessible).

The majority of this section about banking is focused on the Game Boy since that is the original GBDK platform. Much of it still applies for the Game Gear(GG) and Sega Master System(SMS). For additional details about banking specifically related to these two systems see the [SMS/GG Banking](#) section.

6.1.1 Non-banked cartridges

Cartridges with no MBC controller are non-banked, they have 32K bytes of fixed ROM space and no switchable banks. For these cartridges the ROM space between `0000h` and `7FFFh` can be treated as a single large bank of 32K bytes, or as two contiguous banks of 16K bytes in Bank 0 at `0000h` – `3FFFh` and Bank 1 at `4000h` to `7FFFh`.

6.1.2 MBC Banked cartridges (Memory Bank Controllers)

Cartridges with MBCs allow the the Game Boy to work with ROMs up to 8MB in size and with SRAM up to 128kB. Each ROM bank is 16K Bytes. The following are *usually* true, with some exceptions:

- Bank 0 of the ROM is located in the region at 0000h – 3FFFh. It is fixed (non-banked) and cannot be switched out for another bank.
- Banks 1 . . . N can be switched into the upper region at 4000h – 7FFFh. The upper limit for N is determined by the MBC used and available cartridge space.
- It is not necessary to manually assign Bank 0 for source files, that will happen by default if no bank is specified.

See the [Pandocs](#) for more details about the individual MBCs and their capabilities.

6.1.3 Recommended MBC type

For most projects we recommend **MBC5**.

- The [SWITCH_ROM\(\)](#) / ref [SWITCH_RAM\(\)](#) macros work with MBC5 (up to ROM bank 255, [SWITCH_ROM_MBC5_8M](#) may be used if a larger size is needed).
- **MBC1 is not recommended.** Some banks in it's range are unavailable. See pandocs for more details. <https://gbdev.io/pandocs/MBC1>

6.1.3.1 Bank 0 Size Limit and Overflows When Using MBCs When using MBCs and bank switching the space used in the lower fixed Bank 0 **must be <= 16K bytes**. Otherwise it's data will overflow into Bank 1 and may be overwritten or overwrite other data, and can get switched out when banks are changed.

See the [FAQ entry about bank overflow errors](#).

6.1.3.2 Conserving Bank 0 for Important Functions and Data When using MBCs, Bank 0 is the only bank which is always active and it's code can run regardless of what other banks are active. This means it is a limited resource and should be prioritized for data and functions which must be accessible regardless of which bank is currently active.

6.2 Working with Banks

To assign code and constant data (such as graphics) to a ROM bank and use it:

- Place the code for your ROM bank in one or several source files.
- Specify the ROM bank to use, either in the source file or at compile/link time.
- Specify the number of banks and MBC type during link time.
- When the program is running and wants to use data or call a function that is in a given bank, manually or automatically set the desired bank to active.

6.2.1 Setting the ROM bank for a Source file

The cart ROM bank for a source file can be set in a couple different ways. Multiple different banks cannot be assigned inside the same source file (unless the `__addressmod` method is used), but multiple source files can share the same bank.

If no ROM and SRAM bank are specified for a file then the default `_CODE`, `_BSS` and `_DATA` segments are used.

Ways to set the ROM bank for a Source file:

- `#pragma bank <N>` at the start of a source file. Example (ROM bank 2): `#pragma bank 2`
- The lcc switch for ROM bank `-Wf-bo<N>`. Example (ROM bank 2): `-Wf-bo2`
- Using [rom_autobanking](#)

Note: You can use the `NONBANKED` keyword to define a function as non-banked if it resides in a source file which has been assigned a bank.

6.2.2 Setting the Cart SRAM bank for a Source file

- `#pragma dataseg DATA_<N>` at the start of a source file. Example (Cartridge SRAM bank 3)↔
: `#pragma bank 3`
 - See the cross-platform `sram_banks` example
- Using the `lcc` switch for Cartridge SRAM bank `-Wf-ba<N>`. Example (Cartridge SRAM bank 3): `-Wf-ba3`

6.2.3 Setting the MBC and number of Cart ROM & SRAM banks available

At the link stage this is done with `lcc` using pass-through switches for `makebin`.

- `-Wm-yo<N>` where `<N>` is the number of ROM banks. 2, 4, 8, 16, 32, 64, 128, 256, 512
 - `-Wm-yoA` may be used for automatic bank size.
- `-Wm-ya<N>` where `<N>` is the number of 8K Cart SRAM banks. 0, 1, 4, 16
 - (corresponding to: None, 8K, 32K, 128K)
- `-Wm-yt<N>` where `<N>` is the type of MBC cartridge (see chart below).
 - Example: `Wm-yt0x1A`
- If passing the above arguments to `makebin` directly **without** using `lcc`, then the `-Wm` part should be omitted.
 - Note: Some `makebin` switches (such as `-yo A`) require a space when passed directly. See [makebin-settings](#) for details.

The MBC settings below are available when using the `makebin -Wl-yt<N>` switch.

Source: Pandocs. Additional details available at [Pandocs](#)

See the note about [enabling Cart SRAM and number of banks available for SMS/GG](#)

6.2.4 MBC Type Chart

```
0147: Cartridge type:
0x00: ROM ONLY
0x01: ROM+MBC1
0x02: ROM+MBC1+SRAM
0x03: ROM+MBC1+SRAM+BATT
0x05: ROM+MBC2
0x06: ROM+MBC2+BATTERY
0x08: ROM+SRAM
0x09: ROM+SRAM+BATTERY
0x0B: ROM+MMM01
0x0C: ROM+MMM01+SRAM
0x0D: ROM+MMM01+SRAM+BATT
0x0F: ROM+MBC3+TIMER+BATT
0x10: ROM+MBC3+TIMER+SRAM+BATT
0x11: ROM+MBC3
0x12: ROM+MBC3+SRAM
0x13: ROM+MBC3+SRAM+BATT
0x19: ROM+MBC5
0x1A: ROM+MBC5+SRAM
0x1B: ROM+MBC5+SRAM+BATT
0x1C: ROM+MBC5+RUMBLE
0x1D: ROM+MBC5+RUMBLE+SRAM
0x1E: ROM+MBC5+RUMBLE+SRAM+BATT
0x1F: Pocket Camera
0x22: MBC7+ACCELEROMETER+EEPROM
0xFD: Bandai TAMA5
0xFE: Hudson HuC-3
0xFF: Hudson HuC-1
```

Hex Code	MBC Type	Cart SRAM (7)	Battery Save (8)	RTC	Extra Feature	Max ROM Size (1)	Max SRAM Size
0x00	ROM ONLY					32 K	0
0x01	MBC-1 (2)					2 MB	0
0x02	MBC-1 (2)	SRAM				2 MB	32 K (5)
0x03	MBC-1 (2)	SRAM	BATTERY			2 MB	32 K (5)
0x05	MBC-2					256 K	512 x 4 bits (6)
0x06	MBC-2	SRAM (6)	BATTERY			256 K	512 x 4 bits (6)
0x08	ROM (3)	SRAM				32 K	8 K
0x09	ROM (3)	SRAM	BATTERY			32 K	8 K
0x0B	MMM01					8 MB / N	
0x0C	MMM01	SRAM				8 MB / N	128K / N

Hex Code	MBC Type	Cart SRAM (7)	Battery Save (8)	RTC	Extra Feature	Max ROM Size (1)	Max SRAM Size
0x0D	MMM01	SRAM	BATTERY			8 MB / N	128K / N
0x0F	MBC-3		BATTERY (9)	RTC		2 MB	
0x10	MBC-3 (4)	SRAM	BATTERY	RTC		2 MB	32 K
0x11	MBC-3					2 MB	
0x12	MBC-3 (4)	SRAM				2 MB	32 K
0x13	MBC-3 (4)	SRAM	BATTERY			2 MB	32 K
0x19	MBC-5					8 MB	
0x1A	MBC-5	SRAM				8 MB	128 K
0x1B	MBC-5	SRAM	BATTERY			8 MB	128 K
0x1C	MBC-5				RUMBLE	8 MB	
0x1D	MBC-5	SRAM			RUMBLE	8 MB	128 K
0x1E	MBC-5	SRAM	BATTERY		RUMBLE	8 MB	128 K
0x20	MBC-6					~2MB	
0x22	MBC-7	EEPROM			ACCELEROMETER	2MB	256 byte EEPROM
0xFC	POCKET CAMERA					1MB	128KB RAM
0xFD	BANDAI TAMA5					To Do	To Do
0xFE	HuC3			RTC		To Do	To Do
0xFF	HuC1	SRAM	BATTERY		IR	To Do	To Do

1: Max possible size for MBC is shown. When used with generic [SWITCH_ROM\(\)](#) the max size may be smaller. For example:

- The max for MBC1 becomes **Bank 31** (512K)
- The max for MBC5 becomes **Bank 255** (4MB). To use the full 8MB size of MBC5 see [SWITCH_ROM_MBC5_8M\(\)](#).

2: For MBC1 some banks in it's range are unavailable. See pandocs for more details <https://gbdev.io/pandocs/MBC1>

3: No licensed cartridge makes use of this option. Exact behavior is unknown.

4: MBC-3 with SRAM size 64 KByte refers to MBC30, used only in Pocket Monsters Crystal Version for Japan.

5: For MBC-1 the 32 K SRAM is only available for ROM sizes ≤ 512 K.

6: MBC-2 uses integrated SRAM with 512 x 4 bits, the upper 4 bits of each byte should be disregarded.

7: Additional SRAM on the cartridge in the memory range of 0xA000 – 0xBFFF. Contents do not persist after power-off unless the cart has `Battery Save`.

8: With `Battery Save` the contents of the cartridge SRAM will persist after power-off. The electronic implementation on cart may vary, for example it may use FRAM or RAM backed with a coin cell battery.

9: The battery for MBC-3 type 0x0F is only used for the RTC, there is no cartridge SRAM present.

6.2.5 Getting Bank Numbers

The bank number for a banked function, variable or source file can be stored and retrieved using the following macros:

- [BANKREF\(\)](#): create a reference for retrieving the bank number of a variable or function
- [BANK\(\)](#): retrieve a bank number using a reference created with [BANKREF\(\)](#)
- [BANKREF_EXTERN\(\)](#): Make a [BANKREF\(\)](#) reference residing in another source file accessible in the current file for use with [BANK\(\)](#).

6.2.6 Banking and Functions

6.2.6.1 BANKED/NONBANKED Keywords for Functions

- **BANKED** (is a calling convention):
 - The function will use banked (`far`) sdcc calls (which switch to the function's ROM bank automatically).
 - Placed in the bank selected by its source file (or compiler switches).
 - This keyword only specifies the **calling convention** for the function, it does not set a bank itself.
- **NONBANKED** (is a storage attribute):
 - Placed in the non-banked lower 16K region (bank 0), regardless of the bank selected by its source file.
 - Forces the `.area` to `_HOME`.
- `<not-specified>`:
 - The function does not use sdcc banked calls (`near` instead of `far`/ banked sdcc calls)
 - Placed in the bank selected by its source file (or compiler switches).

6.2.6.2 Banked Function Calls Functions in banks can be called as follows:

- When defined with the **BANKED** keyword. Example: `void my_function() BANKED { do stuff }` in a source file which has had its bank set (see above).
- Using [far_pointers](#)
- When defined with an area set up using the `__addressmod` keyword (see the `banks_new` example project and the SDCC manual for details).
- Using [SWITCH_ROM\(\)](#) (and related functions for other MBCs) to manually switch in the required bank and then call the function.

Non-banked functions (either in fixed Bank 0, or in an non-banked ROM with no MBC):

- May call functions in any bank: **YES**
- May use data in any bank: **YES**

Banked functions (located in a switchable ROM bank)

- May call functions in fixed Bank 0: **YES**
- May call **BANKED** functions in any bank: **YES**
 - The compiler and library will manage the bank switching automatically using the bank switching trampoline.
- May use data in any bank: **NO**
 - May only use data from fixed Bank 0 and the currently active bank.
 - A [NONBANKED wrapper function](#) may be used to access data in other banks.
 - Banks cannot be switched manually from inside a **BANKED** function (otherwise it will switch out it's own function code as it is executing it, likely leading to a crash).

Limitations:

- SDCC banked calls and `far_pointers` in GBDK only save one byte for the ROM bank. So, for example, they are limited to **bank 31** max for MBC1 and **bank 255** max for MBC5. This is due to the bank switching for those MBCs requiring a second, additional write to select the upper bits for more banks (banks 32+ in MBC1 and banks 256+ in MBC5).

Calling Convention:

- For details see [Banked Calling Convention](#)

6.2.7 Const Data (Variables in ROM)

Data declared as `const` (read only) will be stored in ROM in the bank associated with its source file (if none is specified it defaults to Bank 0). If that bank is a switchable bank then the data is only accessible while the given bank is active.

6.2.8 Variables in Cart SRAM

Todo Variables in SRAM

6.2.9 Far Pointers

Far pointers include a segment (bank) selector so they are able to point to addresses (functions or data) outside of the current bank (unlike normal pointers which are not bank-aware). A set of macros is provided by GBDK 2020 for working with far pointers.

Warning: Do not call the far pointer function macros from inside interrupt routines (ISRs). The far pointer function macros use a global variable that would not get restored properly if a function called that way was interrupted by another one called the same way. However, they may be called recursively.

See [FAR_CALL](#), [TO_FAR_PTR](#) and the `banks_farptr` example project.

6.2.10 Bank switching

You can manually switch banks using the [SWITCH_ROM\(\)](#), [SWITCH_RAM\(\)](#), and other related macros. See `banks.c` project for an example.

Note: You can only do a `switch_rom_bank` call from non-banked `__CODE` since otherwise you would switch out the code that was executing. Global routines that will be called without an expectation of bank switching should fit within the limited 16k of non-banked `__CODE`.

6.2.11 Wrapper Function for Accessing Banked Data

In order to load Data in one bank from code running in another bank a `NONBANKED` wrapper function can be used. It can save the current bank, switch to another bank, operate on some data, restore the original bank and then return.

An example function which can :

- Load background data from any bank
- And which can be called from code residing in any bank

```
// This function is NONBANKED so it resides in fixed Bank 0
void set_banked_bkg_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data, uint8_t bank) NONBANKED
{
    uint8_t save = CURRENT_BANK;
    SWITCH_ROM(bank);
    set_bkg_data(first_tile, nb_tiles, data);
    SWITCH_ROM(save);
}
// And then it can be called from any bank:
set_banked_bkg_data(<first tile>, <num tiles>, tile_data, BANK(tile_data));
```

6.2.12 Currently active bank: CURRENT_BANK

The global variable [CURRENT_BANK](#) (a macro for `__current_bank`) is updated automatically when calling [SWITCH_ROM\(\)](#), [SWITCH_ROM_MBC1\(\)](#) and [SWITCH_ROM_MBC5](#), or when a `BANKED` function is called.

Normally banked calls are used and the active bank does not need to be directly managed, but in the case that it does the following shows how to save and restore it.

```
// The current bank can be saved
uint8_t _saved_bank = CURRENT_BANK;
// Call some function which changes the bank but does not restore it
// ...
// And then restored if needed
SWITCH_ROM(_saved_bank);
```

6.3 Auto-Banking

A ROM bank auto-assignment feature was added in GBDK 2020 4.0.2.

Instead of having to manually specify which bank a source file will reside in, the banks can be assigned automatically to make the best use of space. The bank assignment operates on object files, after compiling/assembling and before linking.

To turn on auto-banking, use the `-autobank` argument with `lcc`.

For a source example see the `banks_autobank` project.

In the source files you want auto-banked, do the following:

- Set the source file to be autobanked `#pragma bank 255` (this sets the temporary bank to 255, which `bankpack` then updates when repacking).
- Create a reference to store the bank number for that source file: `BANKREF (<some-bank-reference-name>)`.
 - More than one `BANKREF ()` may be created per file, but they should always have unique names.

In the other source files you want to access the banked data from, do the following:

- Create an extern so the bank reference in another file is accessible: `BANKREF_EXTERN (<some-bank-reference-name>)`.
- Obtain the bank number using `BANK (<some-bank-reference-name>)`.

Example: `level_1_map.c`

```
#pragma bank 255
BANKREF (level_1_map)
...
const uint8_t level_1_map[] = {... some map data here ...};
```

Accessing that data: `main.c`

```
BANKREF_EXTERN (level_1_map)
...
SWITCH_ROM( BANK(level_1_map) );
// Do something with level_1_map[]
```

Features and Notes:

- Fixed banked source files can be used in the same project as auto-banked source files. The `bankpack` tool will attempt to pack the auto-banked source files as efficiently as possible around the fixed-bank ones.

Making sure `bankpack` checks all files:

- In order to correctly calculate the bank for all files every time, it is best to use the `-ext=` flag and save the auto-banked output to a different extension (such as `.rel`) and then pass the modified files to the linker. That way all object files will be processed each time the program is compiled.

Recommended:

```
.c and .s -> (compiler) .o -> (bankpack) -> .rel -> (linker) ... -> .gb
```

- It is important because when `bankpack` assigns a bank for an autobanked (bank=255) object file (.o) it rewrites the bank and will then no longer see the file as one that needs to be auto-banked. That file will then remain in its previously assigned bank until a source change causes the compiler to rebuild it to an object file again which resets its bank to 255.
- For example consider a fixed-bank source file growing too large to share a bank with an auto-banked source file that was previously assigned to it. To avoid a bank overflow it would be important to have the auto-banked file check every time whether it can share that bank or not.
- See [bankpack](#) for more options and settings.

6.4 Errors related to banking (overflow, multiple writes to same location)

A *bank overflow* during compile/link time (in `makebin`) is when more code and data are allocated to a ROM bank than it has capacity for. The address for any overflowed data will be incorrect and the data is potentially unreachable since it now resides at the start of a different bank instead of the end of the expected bank.

See the [FAQ entry about bank overflow errors](#).

The current toolchain can only detect and warn (using `ihxcheck`) when one bank overflows into another bank that has data at its start. It cannot warn if a bank overflows into an empty one. For more complete detection, you can use the `romusage` tool.

6.5 Bank space usage

In order to see how much space is used or remains available in a bank you can use the [romusage](#) tool.

6.5.1 Other important notes

- The [SWITCH_ROM_MBC5](#) macro is not interrupt-safe. If using less than 256 banks you may always use SWITCH_ROM - that is faster. Even if you use mbc5 hardware chip in the cart.

6.6 Banking example projects

There are several projects in the GBDK 2020 examples folder which demonstrate different ways to use banking.

- `Banks`: a basic banking example
- `Banks_new`: examples of using new bank assignment and calling conventions available in GBDK 2020 and its updated SDCC version.
- `Banks_farptr`: using far pointers which have the bank number built into the pointer.
- `Banks_autobank`: shows how to use the bank auto-assignment feature in GBDK 2020 4.0.2 or later, instead of having to manually specify which bank a source file will reside it.

"SMS/GG Banking" section.

6.7 SMS/Game Gear Banking

6.7.1 ROM Banks

The memory banking setup for SMS and Game Gear in GBDK is different than it is for the Game Boy. Instead of a single switchable bank in the `0x4000 - 0x7FFF` range, there are two switchable frames at different address ranges. The configuration is as follows:

- Frame 0: Non-banked, at address `0x0000 - 0x3FFF`
- Frame 1: `CODE_<N>`, at address `0x4000 - 0x7FFF`
 - Use for: Banked Code and Assets
 - Example: `#pragma codeseg CODE_2` or `#pragma codeseg CODE_255` for autobanking (no leading underscore)
 - Select the active bank using: [SWITCH_ROM\(\)](#). The current active bank can be queried using [CURRENT_BANK](#) or `MAP_FRAME1`
- Frame 2: `_LIT_<N>`, at address `0x8000- 0xBFFF`
 - Use for: Assets
 - `_DATA_N` may also be mapped into Frame 2 (RAM)
 - Example: `#pragma codeseg LIT_2` or `#pragma codeseg LIT_255` for autobanking (no leading underscore)
 - Select the active bank using [SWITCH_ROM2\(\)](#). The current active bank can be queried using `MAP_↔FRAME2`

Banked code and any pointers associated with it will only work correctly when active in Frame 1 (at `0x4000`), so it must use `CODE_<N>`. Graphics and other assets may go in either Frame 1 (at `0x4000`) or, if designed for it then Frame 2 (at `0x8000`).

"SMS/GG Cart SRAM info"

6.7.2 Cart SRAM Banks

The maximum supported number of Cart SRAM banks for SMS/GG is 2. The size is 8K.

- Additional details in the SMS Power docs: <https://www.smspower.org/Development/↔Mappers#RAMMapping>

For SMS/GG, the ROM file size must be at least 64K to enable mapper support for Cart SRAM banks in emulators.

- If the generated ROM is too small then `-y0 4` for `makebin` (or `-Wm-y04` for `LCC`) can be used to set the size to 64K.
- If auto-banking is being used and Cart SRAM banks are specified then `makebin` will auto-increase the ROM size to 64k to ensure

6.7.3 Auto-Banking

CODE and LIT cannot share the same bank number. For example, if CODE is assigned to bank 3 then LIT cannot be in bank 3 as well.

`bankpack` is aware of this requirement and will group CODE and LIT separately when packing for autobanking. It's process is as follows:

1. Note: CODE and LIT are not sorted before packing
2. Assign fixed banks first (for both CODE and LIT). An error will be generated if both types assigned to the same bank. Banks are marked exclusive to whichever type is assigned in them first.
3. Then autobanked entries (both CODE and LIT) are assigned to banks, they are only assigned to banks of a matching type or an unused bank. Same as above, the first type to use a bank makes it exclusive to that type.

The `bankpack` option `-banktype=` may be used to set a bank to use specific type (CODE or LIT). This will take effect before `bankpack` tries to perform any bank assignment. For example: `-banktype=2:LIT` (or `-Wb-banktype=2:LIT` when used with `lcc`) sets bank 2 to exclusively use type LIT.

"NES Banking" section.

6.8 NES Banking

6.8.1 ROM Banks

To do

"NES Cart SRAM info"

6.8.2 Cart SRAM Banks

Cart SRAM is not currently supported for the NES

7 GBDK Toolchain

7.1 Overview

GBDK 2020 uses the SDCC compiler along with some custom tools to build Game Boy ROMs.

- All tools are located under `bin/`
- The typical order of tools called is as follows (when using `lcc` these steps are usually performed automatically).
 1. Compile and assemble source files (.c, .s, .asm) with `sdcc` and `sdasgb`
 2. Optional: perform auto banking with `bankpack` on the object files
 3. Link the object files into .ihx file with `sdldgb`
 4. Validate the .ihx file with `ihxcheck`
 5. Convert the .ihx file to a ROM file (.gb, .gbc) with `makebin`

To see individual arguments and options for a tool, run that tool from the command line with either no arguments or with `-h`.

7.2 Data Types

For data types and special C keywords, see [asm/sm83/types.h](#) and [asm/types.h](#).

Also see the SDCC manual (scroll down a little on the linked page): <http://sdcc.sourceforge.net/doc/sdccman.pdf#section.1.1>

7.2.1 Using variables in High RAM on the Game Boy

The High RAM (`_HGRAM`) address space is at `0xFF80 - 0xFFFE`. It is eight bit addressable using the `ldh` opcodes which allow for more efficient read and write access.

There are two main ways variables can be placed into High RAM

- Using the `SFR` keyword
 - For example: `SFR my_hram_variable;`
 - Variables will be allocated in the `_HGRAM` area
 - They will be treated as 8-bit unsigned
 - The compiler will automatically tag the variable as `volatile`
 - The codegen will try to optimize them with `ldh` access
- Using the `dataseg` pragma
 - For example: `#pragma dataseg HGRAM followed by unsigned char my_dataseg_var;`
 - The codegen **will not** try to optimize them with `ldh` access

7.3 Changing Important Addresses

It is possible to change some of the important addresses used by the toolchain at link time using the `-Wl-g XXX=YYY` and `=Wl-b XXX=YYY` flags (where `XXX` is the name of the data, and `YYY` is the new address).

`lcc` will include the following linker defaults for `sdlrgb` if they are not defined by the user.

- `_shadow_OAM`
 - Location of sprite ram (requires 0xA0 bytes).
 - Default `-Wl-g _shadow_OAM=0xC000`
- `.STACK`
 - Initial stack address
 - Default `-Wl-g .STACK=0xE000`
- `.refresh_OAM`
 - Address to which the routine for refreshing OAM will be copied (must be in HIRAM). Default
 - Default `-Wl-g .refresh_OAM=0xFF80`
- `_DATA`
 - Start of RAM section (starts after Shadow OAM)
 - Default `-Wl-b _DATA=0xC0A0`
- `_CODE`
 - Start of ROM section
 - Default `-Wl-b _CODE=0x0200`

7.4 Compiling programs

The `lcc` program is the front end compiler driver for the actual compiler, assembler and linker. It works out what you want to do based on command line options and the extensions of the files you give it, computes the order in which the various programs must be called and then executes them in order. Some examples are:

- Compile the C source 'source.c', assemble and link it producing the Gameboy image 'image.gb'
`lcc -o image.gb source.c`
- Assemble the file 'source.s' and link it producing the Gameboy image 'image.gb'
`lcc -o image.gb source.s`
- Compile the C program 'source1.c' and assemble it producing the object file 'object1.o' for later linking.
`lcc -c -o object1.o source1.c`
- Assemble the file 'source2.s' producing the object file 'object2.o' for later linking
`lcc -c -o object2.o source2.s`
- Link the two object files 'object1.o' and 'object2.o' and produce the Gameboy image 'image.gb'
`lcc -o image.gb object1.o object2.o`
- Do all sorts of clever stuff by compiling then assembling source1.c, assembling source2.s and then linking them together to produce image.gb.
`lcc -o image.gb source1.c source2.s`

Arguments to the assembler, linker, etc can be passed via `lcc` using `-Wp...`, `-Wf...`, `-Wa...` and `-Wl...` to pass options to the pre-processor, compiler, assembler and linker respectively. Some common options are:

- To generate an assembler listing file.
`-Wa-l`
- To generate a linker map file.
`-Wl-m`
- To bind var to address 'addr' at link time.
`-Wl-gvar=addr`

For example, to compile the example in the memory section and to generate a listing and map file you would use the following. Note the leading underscore that C adds to symbol names.

```
lcc -Wa-l -Wl-m -Wl-g_snd_stat=0xff26 -o image.gb hardware.c
```

7.4.1 Makefiles

7.4.2 Using Makefiles

Please see the sample projects included with GBDK-2020 for a couple different examples of how to use Makefiles. You may also want to read a tutorial on Makefiles. For example:

<https://makefiletutorial.com/>
<https://www.tutorialspoint.com/makefile/index.htm>

7.4.3 Linker Files and ROM Auto Banking

When `bankpack` is called through `lcc` it will now always use linkerfile output (`-lkout=`) for passing files to the linker (all input object files and linkerfiles will get consolidated to a single linkerfile).

Bankpack:

- `lkin=<filename>` : Adds a input linkerfile (can specify multiple ones)
- `-lkout=<filename>` : Enables linkerfile output and sets name (only one can be specified). ALL loaded object files, both from the command line and any loaded from linkerfiles will have their names written to this single output.

LCC + Bankpack:

- `lcc` passes all input linkerfiles (from `-Wl-f<name>`) to `bankpack` (`-lkin=`)
- Linkerfile output is always used when `lcc` calls `bankpack` (`-lkout=`)
- A temporary file name is used for `bankpack` linkerfile output.
- `lcc` clears out the linker object file and linkerfile lists, then uses the single linkerfile generated by `bankpack`

Also see the `linkerfile` example project.

7.5 Build Tools

7.5.1 `lcc`

`lcc` is the compiler driver (front end) for the GBDK/sdcc toolchain.

For detailed settings see [lcc-settings](#)

It can be used to invoke all the tools needed for building a rom. If preferred, the individual tools can be called directly.

- the `-v` flag can be used to show the exact steps `lcc` executes for a build
- `lcc` can compile, link and generate a binary in a single pass: `lcc -o somerom.gb somesource.c`
- `lcc` now has a `-debug` flag that will turn on the following recommended flags for debugging
 - `--debug` for `sdcc` (`lcc` equiv: `-Wf-debug`)
 - `-y` enables `.cdb` output for `sdldgb` (`lcc` equiv: `-Wl-y`)
 - `-j` enables `.noi` output for `sdldgb` (`lcc` equiv: `-Wl-j`)

7.5.2 `sdcc`

SDCC C Source compiler.

For detailed settings see [sdcc-settings](#)

- Arguments can be passed to it through `lcc` using `-Wf-<argument>` and `-Wp-<argument>` (pre-processor)

7.5.3 `sdasgb`

SDCC Assembler for the Game Boy.

For detailed settings see [sdasgb-settings](#)

- Arguments can be passed to it through `lcc` using `-Wa-<argument>`

7.5.4 `bankpack`

Automatic Bank packer.

For detailed settings see [bankpack-settings](#)

When enabled, automatically assigns banks for object files where bank has been set to 255, see [rom_autobanking](#). Unless an alternative output is specified the given object files are updated with the new bank numbers.

- Can be enabled by using the `-autobank` argument with `lcc`.
- Must be called after compiling/assembling and before linking.
- Arguments can be passed to it through `lcc` using `-Wb-<argument>`

7.5.5 `sdldgb`

The SDCC linker for the gameboy.

For detailed settings see [sdldgb-settings](#)

Links object files (`.o`) into a `.ihx` file which can be processed by [makebin](#)

- Arguments can be passed to it through `lcc` using `-Wl-<argument>`

7.5.6 ihxcheck

IHX file validator.

For detailed settings see [ihxcheck-settings](#)

Checks .ihx files produced by [sdlrgb](#) for correctness.

- It will warn if there are multiple writes to the same ROM address. This may indicate mistakes in the code or ROM bank overflows
- Arguments can be passed to it through [lcc](#) using `-Wi-<argument>`

7.5.7 makebin

IHX to ROM converter.

- For detailed settings see [makebin-settings](#)
- For makebin `-yt` MBC values see [setting_mbc_and_rom_ram_banks](#)

Converts .ihx files produced by [sdlrgb](#) into ROM files (.gb, .gbc). Also used for setting some ROM header data.

- Arguments can be passed to it through [lcc](#) using `-Wm-<argument>`

7.6 GBDK Utilities

7.6.1 GBCompress

Compression utility.

For detailed settings see [gbcompress-settings](#)

Compresses (and decompresses) binary file data with the gbcompress algorithm (also used in GBTD/GBMB). Decompression support is available in GBDK:

- [gb_decompress\(\)](#), [gb_decompress_bkg_data\(\)](#), [gb_decompress_win_data\(\)](#), [gb_decompress_sprite_data\(\)](#)
- The `cross-platform/gbdecompress` example demonstrates how to use this compression

The utility can also compress (and decompress) using block style RLE encoding with the `--alg=rle` flag. Decompression support is available in GBDK:

- [rle_init\(\)](#), [rle_decompress\(\)](#)
- The `cross-platform/rle_map` example demonstrates how to use this compression

7.6.2 png2asset

Tool for converting PNGs into GBDK format MetaSprites and Tile Maps.

- Convert single or multiple frames of graphics into metasprite structured data for use with the `...metasprite...` functions.
- When `-map` is used, converts images into Tile Maps and matching Tile Sets
- Supports Game Boy / Color, SGB borders, SMS/GG, NES

For detailed settings see [png2asset-settings](#)

For working with sprite properties (including cgb palettes), see [metasprite_and_sprite_properties](#)

For API support see [move_metasprite\(\)](#) and related functions in [metasprites.h](#)

7.6.2.1 Working with png2asset

- The origin (pivot) for the metasprite is not required to be in the upper left-hand corner as with regular hardware sprites. See `-px` and `-py`.
- The conversion process supports using both `SPRITES_8x8` (`-spr8x8`) and `SPRITES_8x16` mode (`-spr8x16`). If 8x16 mode is used then the height of the metasprite must be a multiple of 16.

7.6.2.1.1 Terminology The following abbreviations are used in this section:

- Original Game Boy and Game Boy Pocket style hardware: `DMG`
- Game Boy Color: `CGB`

7.6.2.1.2 Conversion Process `png2asset` accepts any png as input, although that does not mean any image will be valid. The program will follow the next steps:

- The image will be subdivided into tiles of 8x8 or 8x16.
- For each tile a palette will be generated.
- If there are more than 4 colors in the palette it will throw an error.
- The palette will be sorted from darkest to lightest. If there is a transparent color that will be the first one (this will create a palette that will also work with `DMG` devices).
- If there are more than 8 palettes the program will throw an error.

With all this, the program will generate a new indexed image (with palette), where each 4 colors define a palette and all colors within a tile can only have colors from one of these palettes

It is also possible to pass a indexed 8-bit png with the palette properly sorted out, using `-keep_palette_order`

- Palettes will be extracted from the image palette in groups of 4 colors.
- Each tile can only have colors from one of these palettes per tile.
- The maximum number of colors is 32.

For indexed color images, sometimes RGB paint programs mix up indexed colors in tiles if the same color exists in multiple palettes.

- `-repair_indexed_pal` can be used to fix this problem, though tiles must still follow the rule of using only one palette per tile.

Using this image a tileset will be created

- Duplicated tiles will be removed.
- Tiles will be matched without mirror, using vertical mirror, horizontal mirror or both (use `-noflip` to turn off matching mirrored tiles).
- The palette won't be taken into account for matching, only the pixel color order, meaning there will be a match between tiles using different palettes but looking identical on grayscale.

7.6.2.1.3 Maps Passing `-map` the png can be converted to a map that can be used in both the background and the window. In this case, `png2asset` will generate:

- The palettes
- The tileset
- The map
- The color info
 - By default, an array of palette index for each tile. This is not the way the hardware works but it takes less space and will create maps compatibles with both `DMG` and `CGB` devices.
 - Passing `-use_map_attributes` will create an array of map attributes. It will also add mirroring info for each tile and because of that maps created with this won't be compatible with `DMG`.
 - * Use `-noflip` to make background maps which are compatible with `DMG` devices.

7.6.2.1.4 Meta sprites By default the png will be converted to metasprites. The image will be subdivided into meta sprites of `-sw x -sh`. In this case `png2asset` will generate:

- The metasprites, containing an array of:
 - tile index
 - y offset
 - x offset
 - flags, containing the mirror info, the palettes for both DMG and GBC and the sprite priority
- The metasprites array

7.6.2.1.5 Super Game Boy Borders (SGB) Screen border assets for the Super Game Boy can be generated using `png2asset`.

The following flags should be used to perform the conversion:

- `<input_border_file.png> -map -bpp 4 -max_palettes 4 -pack_mode sgb -use_map_attributes -c <output_border_data.c>`
- Where `<input_border_file.png>` is the image of the SGB border (256x224) and `<output_border_data.c>` is the name of the source file to write the assets out to.

See the `sgb_border` example project for more details.

7.6.3 makecom

Converts a binary `.rom` file to `.msxdos com` format, including splitting the banks up into separate files.

- For detailed settings see [makecom-settings](#)

7.6.4 makenes

Converts a binary `.rom` file to a `.nes` file, prepending a 16-byte header.

- For an explanation of the interpretation of header bits see <https://www.nesdev.org/wiki/INES>

7.6.5 png2hicolorgb

An updated version of Glen Cook's Windows GUI "hicolour.exe" 1.2 conversion tool for the Game Boy Color. The starting code base was the 1.2 release.

- For detailed settings see [Hi Color](#) on the Game Boy Color is a technique for displaying backgrounds with thousands of colors instead being limited to 32 colors for the entire screen background. It achieves this by changing ~16 colors of the background palette per scanline. The main tradeoffs are that it uses much of the Game Boy's available cpu processing per frame and requires more ROM space. The tile patterns, map, attributes and per-scanline palettes are pre-calculated using the PC based conversion tool.

For the current GBDK example ISR implementation there is a limit of 6 sprites per line before the hi-color timing breaks down and there start to be background artifacts.

Example: `png2hicolorgb myimage.png --csource -o=my_output_filename` Example with higher quality (slower conversion): `png2hicolorgb myimage.png --csource -o=my_output_filename --type=3 -L=2 -R=2`

```
Historical credits and info:
Original Concept : Icarus Productions
Original Code : Jeff Frohwein
Full Screen Modification : Anon
Adaptive Code : Glen Cook
Windows Interface : Glen Cook
Additional Windows Programming : Rob Jones
Original Quantiser Code : Benny
Quantiser Conversion : Glen Cook
```

7.6.5.1 Additional Details For technical details about the conversion process and rendering, see: <https://github.com/bbbbr/png2hicolorgb>

7.6.6 romusage

A utility for estimating usage of Game Boy and SMS/GG ROMs from .noi and .map files, binary ROMs and more.

- For detailed settings see [romusage-settings](#)

Example: `romusage myprogram.noi -g`

8 Supported Consoles & Cross Compiling

8.1 Consoles Supported by GBDK

As of version 4.2.0 GBDK includes support for other consoles in addition to the Game Boy.

- Game Boy and related clones
 - Nintendo Game Boy / Game Boy Color (GB/GBC)
 - Analogue Pocket (AP)
 - Mega Duck / Cougar Boy (DUCK)
- Sega Consoles
 - Sega Master System (SMS)
 - Sega Game Gear (GG)
- NES/Famicom (NES)
- MSX DOS (MSXDOS) (partial support)

While the GBDK API has many convenience functions that work the same or similar across different consoles, it's important to keep their different capabilities in mind when writing code intended to run on more than one. Some (but not all) of the differences are screen sizes, color capabilities, memory layouts, processor type (z80 vs gbz80/sm83) and speed.

8.2 Cross Compiling for Different Consoles

8.2.1 lcc

When compiling and building through [lcc](#) use the `-m<port>:<plat>` flag to select the desired console via its port and platform combination. See below for available settings.

8.2.2 sdcc

When building directly with the `sdcc` toolchain, the following must be specified manually (when using [lcc](#) it will populate these automatically based on `-m<port>:<plat>`).

When compiling with [sdcc](#):

- `-m<port>`, `-D__PORT_<port>` and `-D__TARGET_<plat>`

When assembling select the appropriate include path: `-I<gbdk-path>lib/<plat>`.

The assemblers used are:

- [sdasgb](#) (for GB/AP)
- [sdasz80](#) (for SMS/GG)
- [sdas6500](#) (for NES)

When linking:

- Select the appropriate include paths: `-k <gbdk-path>lib/<port>, -k <gbdk-path>lib/<plat>`
- Include the appropriate library files `-l <port>.lib, -l <plat>.lib`
- The crt will be under `<gbdk-path>lib/<plat>/crt0.o`

The linkers used are:

- [sdlrgb](#) (for GB/AP)
- [sldz80](#) (for SMS/GG or MSXDOS)
- [sld6808](#) (for NES)

MSXDOS requires an additional build step with [makecom](#) after [makebin](#) to create the final binary:

- `makecom <image.bin> [<image.noi>] <output.com>`

8.2.3 Console Port and Platform Settings

Note: Starting with GBDK-2020 4.1.0 and SDCC 4.2, the Game Boy and related clones use `sm83` for the port instead of `gbz80`

- Nintendo Game Boy / Game Boy Color

```
- lcc : -msm83 : gb
- port : sm83, plat : gb
```

- Analogue Pocket

```
- lcc : -msm83 : ap
- port : sm83, plat : ap
```

- Mega Duck / Cougar Boy

```
- lcc : -msm83 : duck
- port : sm83, plat : duck
```

- Sega Master System

```
- lcc : -mz80 : sms
- port : z80, plat : sms
```

- Sega Game Gear

```
- lcc : -mz80 : gg
- port : z80, plat : gg
```

- NES

```
- lcc : -mmos6502 : nes
- port : mos6502, plat : nes
```

- MSX DOS

```
- lcc : -mz80 : msxdos
- port : z80, plat : msxdos
```

8.3 Cross-Platform Constants

There are several constant `#defines` that can be used to help select console specific code during compile time (with `#ifdef`, `#ifndef`).

8.3.1 Console Identifiers

- When `<gb/gb.h>` is included (either directly or through `<gbdk/platform.h>`)
 - When building for Game Boy:
 - * `NINTENDO` will be `#defined`
 - * `GAMEBOY` will be `#defined`
 - When building for Analogue Pocket
 - * `NINTENDO` will be `#defined`
 - * `ANALOGUEPOCKET` will be `#defined`
 - When building for Mega Duck / Cougar Boy
 - * `NINTENDO` will be `#defined`
 - * `MEGADUCK` will be `#defined`
- When `<sms/sms.h>` is included (either directly or through `<gbdk/platform.h>`)
 - When building for Master System
 - * `SEGA` will be `#defined`
 - * `MASTERSYSTEM` will be `#defined`
 - When building for Game Gear
 - * `SEGA` will be `#defined`
 - * `GAMEGEAR` will be `#defined`
- When `<nes/nes.h>` is included (either directly or through `<gbdk/platform.h>`)
 - `NINTENDO_NES` will be `#defined`
- When `<msx/msx.h>` is included (either directly or through `<gbdk/platform.h>`)
 - `MSXDOS` will be `#defined`

8.3.2 Console Hardware Properties

Constants that describe properties of the console hardware are listed below. Their values will change to reflect the current console target that is being built.

- `DEVICE_SCREEN_X_OFFSET`, `DEVICE_SCREEN_Y_OFFSET`
- `DEVICE_SCREEN_WIDTH`, `DEVICE_SCREEN_HEIGHT`
- `DEVICE_SCREEN_BUFFER_WIDTH`, `DEVICE_SCREEN_BUFFER_HEIGHT`
- `DEVICE_SCREEN_MAP_ENTRY_SIZE`
- `DEVICE_SPRITE_PX_OFFSET_X`, `DEVICE_SPRITE_PX_OFFSET_Y`
- `DEVICE_SCREEN_PX_WIDTH`, `DEVICE_SCREEN_PX_HEIGHT`
- `MAX_HARDWARE_SPRITES`
- `HARDWARE_SPRITE_CAN_FLIP_X`, `HARDWARE_SPRITE_CAN_FLIP_Y`

8.4 Using `<gbdk/...>` headers

Some include files under `<gbdk/. . .>` are cross platform and others allow the build process to auto-select the correct include file for the current target port and platform (console). For example, the following can be used

```
#include <gbdk/platform.h>
#include <gbdk/metasprites.h>
```

Instead of

```
#include <gb/gb.h>
#include <gb/metasprites.h>
```

and

```
#include <sms/sms.h>
#include <sms/metasprites.h>
```

8.5 Cross Platform Example Projects

GBDK includes an number of cross platform example projects. These projects show how to write code that can be compiled and run on multiple different consoles (for example Game Boy and Game Gear) with, in some cases, minimal differences.

They also show how to build for multiple target consoles with a single build command and `Makefile`. The `Makefile.targets` allows selecting different `port` and `plat` settings when calling the build stages.

8.5.1 Cross Platform Asset Example

The cross-platform `Logo` example project shows how assets can be managed for multiple different console targets together.

In the example `utility_png2asset` is used to generate assets in the native format for each console at compile-time from separate source PNG images. The `Makefile` is set to use the source PNG folder which matches the current console being compiled, and the source code uses `set_bkg_native_data()` to load the assets tiles in native format to the tile memory used for background tiles on that platform.

8.6 Hardware Summaries

The specs below reflect the typical configuration of hardware when used with GBDK and is not meant as a complete list of their capabilities.

GB/AP/DUCK

- Sprites:
 - 256 tiles (upper 128 are shared with background) (amount is doubled in CGB mode)
 - tile flipping/mirroring: yes
 - 40 total, max 10 per line
 - 2 x 4 color palette (color 0 transparent). 8 x 4 color palettes in CGB mode
- Background: 256 tiles (typical setup: upper 128 are shared with sprites) (amount is doubled in CGB mode)
 - tile grid size: 8x8
 - tile attribute grid size: 8x8 (CGB mode only)
 - tile flipping/mirroring: no (yes in CGB mode)
 - 1 x 4 color palette. 8 x 4 color palettes in CGB mode
- Window "layer": available
- Screen: 160 x 144
- Hardware Map: 256 x 256

SMS/GG

- Sprites:
 - 256 tiles (a bit less in the default setup)
 - tile flipping/mirroring: no
 - 64 total, max 8 per line
 - 1 x 16 color palette (color 0 transparent)
- Background: 512 tiles (upper 256 are shared with sprites)
 - tile grid size: 8x8
 - tile attribute grid size: 8x8
 - tile flipping/mirroring: yes
 - 2 x 16 color palettes
- Window "layer": not available

- SMS
 - Screen: 256 x 192
 - Hardware Map: 256 x 224
- GG
 - Screen: 160 x 144
 - Hardware Map: 256 x 224

NES/Famicom

- Sprites:
 - 8x8 or 8x16
 - 256 tiles
 - tile flipping/mirroring: yes
 - 64 total, max 8 per line
 - 4 x 4 color palette (color 0 transparent)
- Background: 256 tiles
 - tile grid size: 8x8
 - tile attribute grid size: 16x16 (bit packed into 32x32)
 - tile flipping/mirroring: no
 - 4 x 4 color palette (color 0 same for all sub-palettes)
- Window "layer": not available
- Screen: 256 x 240
- Hardware Map: Depends on mirroring mode
 - 256 x 240 (single-screen mirroring)
 - 512 x 240 (vertical mirroring / horizontal scrolling)
 - 256 x 480 (horizontal mirroring / vertical scrolling)
 - 512 x 480 (4-screen layout. Requires additional RAM on cartridge)

8.6.1 Safe VRAM / Display Controller Access

GB/AP

- VRAM / Display Controller (PPU)
 - VRAM and some other display data / registers should only be written to when the [STATF_B_BUSY](#) bit of [STAT_REG](#) is off. Most GBDK API calls manage this automatically.

SMS/GG

- Display Controller (VDP)
 - Writing to the VDP should not be interrupted while an operation is already in progress (since that will interfere with the internal data pointer causing data to be written to the wrong location).
 - Recommended approach: Avoid writing to the VDP (tiles, map, scrolling, colors, etc) during an interrupt routine (ISR).
 - Alternative (requires careful implementation): Make sure writes to the VDP during an ISR are only performed when the [_shadow_OAM_OFF](#) flag indicates it is safe to do so.

NES/Famicom

- See [NES technical details](#)

8.7 Using Game Boy Color (GBC/CGB) Features

8.7.1 Differences Versus the Regular Game Boy (DMG/GBP/SGB)

These are some of the main hardware differences between the Regular Game Boy and the Game Boy Color.

- CPU: Optional 2x Speed mode
- Serial Link: Additional Speeds 2KB/s, 32KB/s, 64KB/s
- IR Port
- Sprites:
 - 2 banks x 256 tile patterns (2x as many) (typically upper 128 of each bank shared with background)
 - 8 x 4 color palettes in CGB mode (BGR-555 per color, 32768 color choices)
- Background:
 - 2 banks x 256 tile patterns (2x as many) (typically upper 128 of each bank shared with sprites)
 - Second map bank for tile attributes (color, flipping/mirroring, priority, bank)
 - 8 x 4 color palettes in CGB mode (BGR-555 per color, 32,768 color choices)
 - BG and Window master priority
- WRAM: 8 x 4K WRAM banks in the 0xD000 - 0xDFFF region
- LCD VRAM DMA

8.7.2 Game Boy Color features in GBDK

These are some of the main GBDK API features for the CGB. Many of the items listed below link to additional information.

- ROM header settings:
 - See the FAQ entry [How do I set SGB, Color only and Color compatibility in the ROM header?](#)
- Tile and Pattern data:
 - Select VRAM Banks: [VBK_REG](#) (used with `set_bkg/win/sprite_*`)
 - [set_bkg_attributes\(\)](#), [set_bkg_submap_attributes\(\)](#)
- Color:
 - [set_bkg_palette\(\)](#), [set_bkg_palette_entry\(\)](#)
 - [set_sprite_palette\(\)](#), [set_sprite_palette_entry\(\)](#)
 - [set_default_palette\(\)](#)
 - [RGB\(\)](#), [RGB8\(\)](#), [RGBHTML\(\)](#)
- Detect and change CPU speed: if (`_cpu == CGB_TYPE`), [cpu_fast\(\)](#)
- More details in [cgb.h](#) (`#include <gb/cgb.h>`)

8.7.3 CGB Examples

Several examples in GBDK show how to use CGB features, including the following:

- [gb/colorbar](#), [gb/dscan](#), [cross-platform/large_map](#), [cross-platform/logo](#), [cross-platform/meta](#)

8.8 Porting Between Supported Consoles

8.8.1 From Game Boy to Analogue Pocket

The Analogue Pocket operating in `.pocket` mode is (for practical purposes) functionally identical to the Game Boy / Color though it has a couple changes listed below. These are handled automatically in GBDK as long as the practices outlined below are followed.

8.8.1.1 Official differences:

- Altered register flag and address definitions
 - **STAT & LCDC**: Order of register bits is reversed
 - * Example: **LCD on/off** is LCDC.0 instead of .7
 - * Example: **LYC Interrupt enable** is STAT.1 instead of .6
 - **LCDC** address is 0xFF4E instead of 0xFF40
- Different logo data in the header at address 0x0104:
 - 0x01, 0x10, 0xCE, 0xEF, 0x00, 0x00, 0x44, 0xAA, 0x00, 0x74, 0x00, 0x18, 0x11, 0x95, 0x00, 0x34, 0x00, 0x1A, 0x00, 0xD5, 0x00, 0x22, 0x00, 0x69, 0x6F, 0xF6, 0xF7, 0x73, 0x09, 0x90, 0xE1, 0x10, 0x44, 0x40, 0x9A, 0x90, 0xD5, 0xD0, 0x44, 0x30, 0xA9, 0x21, 0x5D, 0x48, 0x22, 0xE0, 0xF8, 0x60

8.8.1.2 Observed differences:

- MBC1 and MBC5 are supported, MBC3 won't save and RTC doesn't progress when game is not running, the HuC3 isn't supported at all (via JoseJX and sg).
- The Serial Link port does not work
- The IR port in CGB mode does not work as reliably as the Game Boy Color

In order for software to be easily ported to the Analogue Pocket, or to run on both, use the following practices.

8.8.1.3 Registers and Flags Use API defined registers and register flags instead of hardwired ones.

- LCDC register: **LCDC_REG** or **rLCDC**
- STAT register: **STAT_REG** or **rSTAT**
- LCDC flags: -> LCDCF_... (example: **LDCF_ON**)
- STAT flags: -> STATF_... (example: **STATF_LYC**)

8.8.1.4 Boot logo As long as the target console is **set during build time** then the correct boot logo will be automatically selected.

8.8.2 From Game Boy to SMS/GG

8.8.2.1 RAM Banks

- The SMS/GG ROM file size must be at least 64K to enable mapper support for RAM banks in emulators.
 - If the generated ROM is too small then `-y 4` for makebin (or `-Wm-y 4` for LCC) can be used to set the size to 64K.

8.8.2.2 Tile Data and Tile Map loading

8.8.2.2.1 Tile and Map Data in 2bpp Game Boy Format

- **set_bkg_data()** and **set_sprite_data()** will load 2bpp tile data in "Game Boy" format on both GB and SMS/GG.
- On the SMS/GG **set_2bpp_palette()** sets 4 colors that will be used when loading 2bpp assets with **set_bkg_data()**. This allows GB assets to be easily colorized without changing the asset format. There is some performance penalty for using the conversion.
- **set_bkg_tiles()** loads 1-byte-per-tile tilemaps both for the GB and SMS/GG.

8.8.2.2.2 Tile and Map Data in Native Format Use the following api calls when assets are available in the native format for each platform.

[set_native_tile_data\(\)](#)

- GB/AP: loads 2bpp tiles data
- SMS/GG: loads 4bpp tile data

[set_tile_map\(\)](#)

- GB/AP: loads 1-byte-per-tile tilemaps
- SMS/GG: loads 2-byte-per-tile tilemaps

There are also bit-depth specific API calls:

- 1bpp: [set_1bpp_colors](#), [set_bkg_1bpp_data](#), [set_sprite_1bpp_data](#)
- 2bpp: [set_2bpp_palette](#), [set_bkg_2bpp_data](#), [set_sprite_2bpp_data](#), [set_tile_2bpp_data](#) (sms/gg only)
- 2bpp: [set_bkg_4bpp_data](#) (sms/gg only), [set_sprite_4bpp_data](#) (sms/gg only)

8.8.2.3 Colors and Palettes The SMS/GG have 2 x 16 color palettes:

- The first (0) is just for the background
- The second (1) is shared between sprites and the background (and for sprites a single color 0 of that palette is transparent)

On the Game Gear

- Each Palette is 32 bytes in size: 16 colors x 2 bytes per palette color entry.
- Each color (16 per palette) is packed as BGR-444 format (x:4:4:4, MSBits [15..12] are unused).
- Each component (R, G, B) may have values from 0 - 15 (4 bits), 15 is brightest.

On the SMS

- On SMS each Palette is 16 bytes in size: 16 colors x 1 byte per palette color entry.
- Each color (16 per palette) is packed as BGR-222 format (x:2:2:2, MSBits [7..6] are unused).
- Each component (R, G, B) may have values from 0 - 3 (2 bits), 3 is brightest.

For setting palette data:

- [set_palette_entry\(\)](#): Will set a single color in a palette
- [set_palette\(\)](#): Can set all the colors for one or both palettes
- [set_bkg_palette\(\)](#): Is just an alias for [set_palette\(\)](#). The full 16 colors can be set using this call.
- [set_sprite_palette\(\)](#): Is also an alias for [set_palette\(\)](#), but it offsets to write to the second 16 color palette.
- Also see: [RGB\(\)](#), [RGB8\(\)](#), [RGBHTML\(\)](#)

8.8.2.3.1 Emulated Game Boy Color map attributes on the SMS/Game Gear On the Game Boy Color, [VBK_REG](#) is used to select between the regular background tile map and the background attribute tile map (for setting tile color palette and other properties).

This behavior is emulated for the SMS/GG when using [set_bkg_tiles\(\)](#) and [VBK_REG](#). It allows writing a 1-byte tile map separately from a 1-byte attributes map.

Note

Tile map attributes on SMS/Game Gear use different control bits than the Game Boy Color, so a modified attribute map must be used.

8.8.3 From Game Boy to NES

The NES graphics architecture is similar to the GB's. However, there are a number of design choices in the NES hardware that make the NES a particularly cumbersome platform to develop for, and that will require special attention.

Most notably:

- PPU memory can only be written in a serial fashion using a data port at 0x2007 (PPUDATA)
- PPU memory can only be written to during vblank, or when manually disabling rendering via PPUMASK. Hblank writes to PPU memory are not possible
- PPU memory write address is re-purposed for scrolling coordinates when rendering is enabled which means PPU memory updates / scrolling must cooperate
- PPU internal palette memory is also mapped to external VRAM area making palette updates during rendering very expensive and error-prone
- The base NES system has no support for any scanline interrupts. And cartridge mappers that add scanline interrupts do so using wildly varying solutions
- There's no easy way to determine the current scanline or CPU-to-PPU alignment meaning timed code is often required on the NES
- The PAL variant of the NES has very different CPU / PPU timings, as do the Dendy clone and other clone systems
- The stock 2 kB CPU RAM is just 1/4th the 8kB CPU RAM on a Game Boy
 - Free RAM after accounting for ZP, stack, OAM page and system variables further cuts this in half
 - This means a lot of GB code will need to be carefully optimized for RAM usage when ported to the NES
 - In particular, make sure to use the "const" modifier for arrays that are read-only, to make sure they don't end up in RAM

To provide an easier experience, gbdk-nes attempts to hide most of these quirks so that in theory the programming experience for gbdk-nes should be as close as possible to that of the GB/GBC. However, to avoid surprises it is recommended to familiarize yourself with the NES-specific quirks and implementation choices mentioned here.

This entire section is written as a guide on porting GB projects to NES. If you are new to GBDK, you may wish to familiarize yourself with using GBDK for GB development first as the topics covered will make a lot more sense after gaining experience with GB development.

8.8.3.1 Mapper Currently the NES support in GBDK uses UNROM-512 (Mapper30) with single-screen mirroring.

8.8.3.2 Buffered mode vs direct mode On the GB, the vblank period serves as an optimal time to write data to PPU memory, and PPU memory can also be written efficiently with VRAM DMA.

On the NES, writing PPU memory during the vblank period is not optional. Whenever rendering is turned on the PPU is in a state where accessing PPU memory results in undefined behavior outside the short vblank period. The NES also has no VRAM DMA hardware to help with data writes. This makes the vblank period not only more precious, but important to never exceed to avoid glitched games.

To deal with this limitation, all functions in gbdk-nes that write to PPU memory can run in either *Buffered* or *Direct* mode.

The good news is that switching between buffered and direct mode in gbdk-nes is usually done behind-the-scenes and normally shouldn't affect your code too much, as long as you use the portable GBDK functions and macros to do this.

- DISPLAY_ON / SHOW_BG / SHOW_SPR will all switch the system into buffered mode, allowing limited amounts of transfers during vblank, without affecting the display of graphics
- DISPLAY_OFF will switch the system into direct mode, allowing much larger/faster transfers while the screen is blanked

The following sections describe how the buffered / direct modes work in more detail. As buffered / direct mode is mostly hidden by the API calls, feel free to skip these sections if you wish.

8.8.3.2.1 Buffered mode implementation details To take maximum advantage of the short vblank period, gbdk-nes implements a popular optimization: An unrolled loop that pulls prepared data bytes from the stack.

```
PLA
STA PPUADDR
...
PLA
STA PPUADDR
RTS
```

The data structure to facilitate this is usually called a vram transfer buffer, often affectionately called a "popslide" buffer after Damian Yerrick's implementation. This buffer essentially forms a list of commands where each command sets up a new PPU address and then writes a sequence of bytes with an auto-increment of either +1 or +32. Each such command is often called a "stripe" in the nesdev community.

The transfer buffer starts at 0x100 and takes around half of the hardware stack page. You can think of the transfer buffer as a software-implemented DMA that allows writing bytes at the optimal rate of 8 cycles / byte. (ignoring the PPU address setup cost)

The buffer supports writing up to 32 continuous bytes at a time. This allows updating a full screen row / column, or two 8x8 tiles worth of tile data in one command / "stripe".

By doing writes to this buffer during game logic, your game will effectively keep writing data transfer commands for the vblank NMI handler to process in the next vblank period, without having to wait until the vblank.

Given that the transfer buffer only has space for around 100 data bytes, it is important to not overfill the buffer, as this will bring code execution to a screeching halt, until the NMI handler empties the old contents of the buffer to free up space and allow new commands to be written.

Buffered mode is typically used for scrolling updates or dynamically animated tiles, where only a small amount of bytes need updating per frame.

8.8.3.2.2 Direct mode implementation details During direct mode, all graphics routines will write directly to the PPUADDR / PPUADDR ports and the transfer buffer limit is never a concern because the transfer buffer is effectively bypassed.

Direct mode is typically used for initializing large amounts of tile data at boot and/or level loading time. Unless you plan to have an animated loading screen and decompress a lot of data, it makes more sense to just fade the screen to black and allow direct mode to write data as fast as possible.

Direct mode also affects how (fake) interrupt handlers are processed. As long as `vsync()` is called on each frame, the VBL and LCD handlers will still be executed in direct mode - but no graphics registers will be written.

The TIM handler will still be executed as normal.

8.8.3.2.3 Caveat: Write appropriate global backdrop before turning display off On the GB, when the display is turned off the LCD will display a whiter-than-white color.

On the NES, calling `DISPLAY_OFF` will turn sprites and BG off and allow VRAM writes in direct mode. But the color displayed will be last global backdrop color (i.e. palette entry 0) that was written before `DISPLAY_OFF` was called. Because all palette writes will be postponed in direct mode, the same also applies to other palette entries: Their values will be maintained in RAM, but never actually reach the PPU's hardware palette registers until buffered mode is re-entered by calling `DISPLAY_ON`.

You should not try to do any palette fades after calling `DISPLAY_OFF`, as they will be delayed in direct mode until the display is turned on again with `DISPLAY_ON`. Once `DISPLAY_ON` is called, the NMI handler will start updating the hardware palette registers with the RAM registers as normal.

8.8.3.3 Shadow PPU registers Like the SMS, the NES hardware is designed to only allow loading the full X/Y scroll on the very first scanline. i.e., under normal operation you are only allowed to change the Y-scroll once.

In contrast to the SMS, this limitation can be circumvented with a specific set of out-of-order writes to the `PPUSCROLL`/`PPUADDR` registers, taking advantage of the quirk that the `PPUADDR` and `PPUSCROLL` share register bits. But this write sequence is very timing-sensitive as the writes need to fall into (a smaller portion of) the hblank period in order to avoid race conditions when the CPU and PPU both try to update the same register during scanline rendering.

To simplify the programming interface, gbdk-nes functions like `move_bkg` / `scroll_bkg` only ever update shadow registers in RAM. The vblank NMI handler will later pick these values up and write them to the actual PPU registers.

8.8.3.4 Implementation of (fake) vbl / lcd handlers GBDK provides an API for installing Interrupt Service Routines that execute on start of vblank (VBL handler), or on a specific scanline (LCD handler). But the base NES system has no suitable scanline interrupts that can provide the exact equivalent functionality. So instead, gbdk-nes API allows *fake* handlers to be installed in the goal of keeping code compatible with other platforms.

- An installed VBL handler will be called immediately when calling vsync. This handler should only update PPU shadow registers. After each invocation, shadow registers are stored into a buffer.
- An installed LCD handler for a specific scanline will then be called repeatedly until the value of `_lcd_scanline` is either set to an earlier scanline or ≥ 240 . After each invocation, shadow registers are stored into a buffer.
- After the built-in vblank NMI handler has finished palette updates, OAM DMA, VRAM updates it will then use the buffered VBL shadow registers to write the real registers. If LCD handlers are enabled it will then manually run a delay loop to reach the particular scanlines that the installed LCD handler was pre-called for, and use the contents of the buffer to update registers.

Because the LCD "ISR" is actually implemented with a delay loop, it will burn a lot of CPU cycles in the frame - the further down the requested scanline is the larger the CPU cycle loss. In practice this makes this faked-LCD-ISR functionality mostly suitable for status bars at the top of the screen. Or for simple parallax cutscenes where the CPU has little else to do.

To make porting between user VBL / LCD handlers written in C easier, gbdk-nes also provides aliases for the shadow registers that correspond to the GB hardware registers.

- `SCX_REG` is an alias for `bkg_scroll_x` shadow register
- `SCY_REG` is an alias for `bkg_scroll_y` shadow register
- `LYC_REG` is an alias for `_lcd_scanline` shadow register

Because these are shadow registers that are interpreted by the GBDK library to mimick GB behaviour, they won't behave exactly how the GB hardware registers do under all conditions. However, for most practical purposes they allow writing portable VBL / LCD handlers in C.

Note

The `bkg_scroll_y` shadow register functions the same as `SCY_REG` GB, with its value added to `_lcd_scanline` to determine the final Y scrolling coordinate. However, its range is different due to tilemaps being 32x30 instead 32x32.

Negative coordinates won't work correctly due to the wrapping from 239 to 0. Instead, they need to be corrected with this wrapping in mind. i.e. a negative coordinate of -1 needs to be converted to 239 before being written to `bkg_scroll_y`.

A portable way to do this is to check for a negative offset, and use the screen height define:

```
SCY_REG = offset < 0 ? (uint8_t)(DEVICE_SCREEN_BUFFER_HEIGHT*8 + offset) : offset;
```

8.8.3.5 Caveat: Make sure to call vsync on every frame On the GB, the call to vsync is an optional call that serves two purposes:

1. It provides a consistent frame timing for your game
2. It allows future register writes to be synchronized to the screen

On gbdk-nes the second point is no longer true, because writes need to be made to the shadow registers either *before* vsync is called, or in a user VBL isr handler.

But the vsync call serves three other very important purposes:

A. It calls the optional VBL handler, where shadow registers can be written (and will later be picked up by the actual vblank NMI handler) B. It repeatedly calls the optional LCD handler up to `MAX_LCD_ISR_CALLS` times. After each call, PPU shadow registers are stored into a buffer that will later be used by timed code in the NMI to handle mid-frame changes for screen splits / sprite hiding / etc. C. It calls `flush_shadow_attributes` so that updates to background attributes actually get written to PPU memory

For these reasons you should always include a call to vsync if you expect to see any graphical updates on the screen.

8.8.3.6 Implementation of timer handler The nature of the deferred handling for fake VBL and LCD handlers in gbdk-nes means that lag frames will cause these handlers to be called at delayed irregular times.

For graphics updates this is the behaviour you usually want. But for non-graphics tasks like music playback it will cause distracting stutter.

The timer overflow handler (TIM) provides an alternative method that is guaranteed to be stutter-free. The TIM handler is always called if timer overflow occurs at the end of the NMI handler in both buffered and direct mode.

The TMA_REG and TAC_REG hardware registers are emulated via RAM variable with the same names. The timer emulation matches the values of these registers for a GBC running in double-speed mode. But there will be a small variation in exact frequency compared to real GBC hardware, and the nature of the timer emulation via vblank means the execution is not as evenly paced as on GBC hardware.

The TIMA_REG should not be written or read in gbdk-nes, as the emulation does not handle its contents exactly as on GB.

At reset, TAC_REG is set to clock rate 00 and timer enabled, and TMA_REG is set to either of two values, depending on the detected system:

- `TIMER_VBLANK_PARITY_MODE_SYSTEM_60HZ` for a Famicom / US NES
- `TIMER_VBLANK_PARITY_MODE_SYSTEM_50Hz` for a PAL NES / PAL Famiclone

These default values ensure that the TIM emulation will always call the TIM handler exactly once for every vblank, resulting in 60Hz vs 50Hz depending on the system.

Changing TMA_REG allows setting a slower or faster frequency for this emulated timer overflow interrupt if your GB game uses the timer overflow hardware for regular events like music that are slower or faster than 60Hz (50Hz for PAL).

If you are porting a GB game to gbdk-nes where the music handler is called in the VBL or LCD handler then it is advisable to move this call to the timer handler, in order to achieve reliable music playback at 60Hz (50Hz for PAL). Keep in mind that you should NOT write any graphics registers in the TIM handler. This will likely not do what you want, and it may result in bad graphical glitches.

Tweaking the playback rate by setting TMA_REG / TAC_REG is a decent way to achieve the same average playback rate as on a GB game that uses a different rate than the vblank tick rate and allow similar music speed for different regions.

However, it is recommended to use the default vblank parity mode whenever remaking the music specifically for 60Hz / 50Hz is an option, as keeping the music tick rate steady will give more pleasant sound playback for rates that are already close to native vblank rate of 60Hz / 50Hz.

Note

Because the TIM handler will be called from the vblank NMI, this function and all functions it calls need to use the `#pragma nooverlay` command. This makes memory for local variables and function parameters unique to this function instead of being shared with other functions' allocations in the reusable overlay segment.

```
#pragma save
#pragma nooverlay
void tim_isr(void)
{
    // Do TIM isr things
}
#pragma restore
```

Without this pragma the calls in your TIM handler could end up overwriting local variables or function parameters that the main program was using when it was interrupted. You should also avoid calls to the standard library, and even multiplications and division / modulo operations in your TIM handlers. These more expensive math operations in SDCC are currently implemented with functions that use the overlay segment and would cause similar conflicts. For more details on overlay segment and interrupts, please see section 3.7 in the SDCC manual.

8.8.3.7 Tile Data and Tile Map loading

8.8.3.7.1 Tile and Map Data in 2bpp Game Boy Format

- `set_bkg_data()` and `set_sprite_data()` will load 2bpp tile data in "Game Boy" format on both GB and NES.
- `set_bkg_tiles()` loads 1-byte-per-tile tilemaps both for the GB and NES.

8.8.3.7.2 Tile and Map Data in Native Format Use the following api calls when assets are available in the native format for each platform.

[set_native_tile_data\(\)](#)

- GB/AP: loads 2bpp tiles data
- NES: loads 2bpp tiles data

[set_tile_map\(\)](#)

- GB/AP: loads 1-byte-per-tile tilemaps
- NES: loads 1-byte-per-tile tilemaps

Bit-depth specific API calls:

- 1bpp: [set_1bpp_colors](#), [set_bkg_1bpp_data](#), [set_sprite_1bpp_data](#)
- 2bpp: [set_2bpp_palette](#), [set_bkg_2bpp_data](#), [set_sprite_2bpp_data](#)

Platform specific API calls:

- [set_bkg_attributes_nes16x16\(\)](#), [set_bkg_submap_attributes_nes16x16\(\)](#), [set_bkg_attribute_xy_nes16x16\(\)](#)

8.8.3.7.3 Game Boy Color map attributes on the NES On the Game Boy Color, [VBK_REG](#) is used to select between the regular background tile map and the background attribute tile map (for setting tile color palette and other properties).

This behavior of setting VBK_REG to specify tile indices/attributes is not supported on the NES platform. Instead the dedicated functions for attribute setting should be used. These will work on other platforms as well and are the preferred way to set attributes.

To maintain API compatibility with other platforms that have attributes on an 8x8 grid specified with a whole byte per attribute, the NES platform supports the dedicated calls for setting attributes on an 8x8 grid:

- [set_bkg_attributes\(\)](#)
- [set_bkg_submap_attributes\(\)](#)
- [set_bkg_attribute_xy\(\)](#)

This allows code to for attribute setting to remain unchanged between platforms. The effect of using these calls is that some attribute setting will be redundant due to the coarser attribute grid. i.e., setting the attribute at coordinates (4, 4), (4,5), (5, 4) and (5, 5) will all set the same attribute.

There is one more platform specific difference to note: While the [set_bkg_attribute_xy\(\)](#) function takes coordinates on a 8x8 grid, the [set_bkg_attributes\(\)](#) and [set_bkg_submap_attributes\(\)](#) functions take a pointer to data in NES packed attribute format, where each byte contains data for 4 16x16 attribute. i.e. a 32x32 region.

While this implementation detail of how the attribute map is encoded is usually hidden by the API functions it does mean that code which manually tries to read the attribute data is *NOT* portable between NES/other platforms, and is not recommended.

Note

Tile map attributes on NES are on a 16x16 grid and use different control bits than the Game Boy Color.

- NES 16x16 Tile Attributes are bit packed into 4 attributes per byte with each 16x16 area of a 32x32 pixel block using the bits as follows:
 - D1-D0: Top-left 16x16 pixels
 - D3-D2: Top-right 16x16 pixels
 - D5-D4: Bottom-left 16x16 pixels
 - D7-D6: Bottom-right 16x16 pixels
 - https://www.nesdev.org/wiki/PPU_attribute_tables

8.8.4 From Game Boy to Mega Duck / Cougar Boy

The Mega Duck is (for practical purposes) functionally identical to the Original Game Boy though it has a couple changes listed below.

8.8.4.1 Summary of Hardware changes:

- Cartridge Boot Logo: not present on Mega Duck
- Cartridge Header data: not present on Mega Duck
- Program Entry Point: 0x0000 (on Game Boy: 0x0100)
- Display registers and flag definitions: Some changed
- Audio registers and flag definitions: Some changed
- MBC ROM bank switching register address: 0x0001 (many Game Boy MBCs use 0x2000 - 0x3FFF)
- Also see [links for megaduck development](#)

8.8.4.2 Best Practices In order for software to be easily ported to the Mega Duck, or to run on both, use these practices. That will allow GBDK to automatically handle *most* of the differences (for the exceptions see [Sound Register Value Changes](#)).

- [Set the target console during build time](#)
- Use the GBDK definitions and macros for:
 - Video Registers and Flags (examples: [LCDC_REG](#), [LCDCF_BG8000](#), etc)
 - Audio Registers and Flags (examples: [NR12_REG](#), [NR43_REG](#), etc)
 - Use the default [SWITCH_ROM](#) macro for changing ROM banks

8.8.4.3 Sound Register Value Changes There are two hardware changes which will not be handled automatically when following the practices mentioned above.

These changes may be required when using existing Sound Effects and Music Drivers written for the Game Boy.

1. Registers [NR12_REG](#), [NR22_REG](#), [NR42_REG](#), and [NR43_REG](#) have their contents nybble swapped.
 - To maintain compatibility the value to write (or the value read) can be converted this way: $((\text{uint8_t})(\text{value} \ll 4) \mid (\text{uint8_t})(\text{value} \gg 4))$
2. Register [NR32_REG](#) has the volume bit values changed.
 - Game Boy: Bits:6..5 : 00 = mute, 01 = 100%, 10 = 50%, 11 = 25%
 - Mega Duck: Bits:6..5 : 00 = mute, 01 = 25%, 10 = 50%, 11 = 100%
 - To maintain compatibility the value to write (or the value read) can be converted this way: $((\sim(\text{uint8_t})\text{value}) + (\text{uint8_t})0x20u) \& (\text{uint8_t})0x60u$

8.8.4.4 Graphics Register Bit Changes These changes are handled automatically when their GBDK definitions are used.

LCDC_REG Flag	Game Boy	Mega Duck		Purpose
LCDCF_B_ON	.7	.7	(same)	Bit for LCD On/Off Select
LCDCF_B_WIN9C00	.6	.3		Bit for Window Tile Map Region Select
LCDCF_B_WINON	.5	.5	(same)	Bit for Window Display On/Off Control
LCDCF_B_BG8000	.4	.4	(same)	Bit for BG & Window Tile Data Region Select
LCDCF_B_BG9C00	.3	.2		Bit for BG Tile Map Region Select
LCDCF_B_OBJ16	.2	.1		Bit for Sprites Size Select
LCDCF_B_OBJON	.1	.0		Bit for Sprites Display Visible/Hidden Select
LCDCF_B_BGON	.0	.6		Bit for Background Display Visible Hidden Select

8.8.4.5 Detailed Register Address Changes These changes are handled automatically when their GBDK definitions are used.

Register	Game Boy	Mega Duck
LCDC_REG	0xFF40	0xFF10
STAT_REG	0xFF41	0xFF11
SCY_REG	0xFF42	0xFF12
SCX_REG	0xFF43	0xFF13
LY_REG	0xFF44	0xFF18
LYC_REG	0xFF45	0xFF19
DMA_REG	0xFF46	0xFF1A
BGP_REG	0xFF47	0xFF1B
OBP0_REG	0xFF48	0xFF14
OBP1_REG	0xFF49	0xFF15
WY_REG	0xFF4A	0xFF16
WX_REG	0xFF4B	0xFF17
-	-	-
NR10_REG	0xFF10	0xFF20
NR11_REG	0xFF11	0xFF22
NR12_REG	0xFF12	0xFF21
NR13_REG	0xFF13	0xFF23
NR14_REG	0xFF14	0xFF24
-	-	-
NR21_REG	0xFF16	0xFF25
NR22_REG	0xFF17	0xFF27
NR23_REG	0xFF18	0xFF28
NR24_REG	0xFF19	0xFF29
-	-	-
NR30_REG	0xFF1A	0xFF2A
NR31_REG	0xFF1B	0xFF2B
NR32_REG	0xFF1C	0xFF2C
NR33_REG	0xFF1D	0xFF2E
NR34_REG	0xFF1E	0xFF2D
-	-	-
NR41_REG	0xFF20	0xFF40
NR42_REG	0xFF21	0xFF42
NR43_REG	0xFF22	0xFF41
NR44_REG	0xFF23	0xFF43
-	-	-
NR50_REG	0xFF24	0xFF44
NR51_REG	0xFF25	0xFF46
NR52_REG	0xFF26	0xFF45
-	-	-

9 Example Programs

GBDK includes several example programs both in C and in assembly. They are located in the examples directory, and in its subdirectories. They can be built by typing `make` in the corresponding directory.

9.1 banks (various projects)

There are several different projects showing how to use ROM banking with GBDK.

9.2 comm

Illustrates how to use communication routines.

9.3 crash

Demonstrates how to use the optional GBDK crash handler which dumps debug info to the Game Boy screen in the event of a program crash.

9.4 colorbar

The colorbar program, written by Mr. N.U. of TeamKNOx, illustrates the use of colors on a Color GameBoy.

9.5 dscan

Deep Scan is a game written by Mr. N.U. of TeamKNOx that supports the Color GameBoy. Your aim is to destroy the submarines from your boat, and to avoid the projectiles that they send to you. The game should be self-explanatory. The following keys are used:

```
RIGHT/LEFT : Move your boat
A/B         : Send a bomb from one side of your boat
START      : Start game or pause game
```

When game is paused:

```
SELECT      : Invert A and B buttons
RIGHT/LEFT  : Change speed
UP/DOWN     : Change level
```

9.6 filltest

Demonstrates various graphics routines.

9.7 fonts

Examples of how to work with the built in font and printing features.

9.8 galaxy

A C translation of the space.s assembly program.

9.9 gb-dtmf

The gb-dtmf, written by Osamu Ohashi, is a Dual Tone Multi-Frequency (DTMF) generator.

9.10 gbdecompress

Demonstrates using gbdecompress to load a compressed tile set into VRAM.

9.11 irq

Illustrates how to install interrupt handlers.

9.12 large map

Shows how to scroll with maps larger than 32 x 32 tiles using [set_bkg_submap\(\)](#). It fills rows and columns at the edges of the visible viewport (of the hardware Background Map) with the desired sub-region of the large map as it scrolls.

9.13 metasprites

Demonstrates using the metasprite features to move and animate a large sprite.

- Press A button to show / hide the metasprite
- Press B button to cycle through the metasprite animations
- Press SELECT button to cycle the metasprite through Normal / Flip-Y / Flip-XY / Flip-X
- Up / Down / Left / Right to move the metasprite

9.14 lcd_isr wobble

An example of how to use the LCD ISR for visual special effects.

9.15 paint

The paint example is a painting program. It supports different painting tools, drawing modes, and colors. At the moment, it only paints individual pixels. This program illustrates the use of the full-screen drawing library. It also illustrates the use of generic structures and big sprites.

```
Arrow keys : Move the cursor
SELECT     : Display/hide the tools palette
A          : Select tool
```

9.16 rand

The rand program, written by Luc Van den Borre, illustrates the use of the GBDK random generator.

9.17 ram_fn

The ram_fn example illustrates how to copy functions to RAM or HIRAM, and how to call them from C.

9.18 rpn

A basic RPN calculator. Try entering expressions like 12 134* and then 1789+.

9.19 samptest

Demonstration of playing a sound sample.

9.20 sgb (various)

A collection of examples showing how to use the Super Game Boy API features.

9.21 sound

The sound example is meant for experimenting with the sound generator of the GameBoy (to use on a real GameBoy). The four different sound modes of the GameBoy are available. It also demonstrates the use of bit fields in C (it's a quick hack, so don't expect too much from the code). The following keys are used:

```
UP/DOWN      : Move the cursor
RIGHT/LEFT   : Increment/decrement the value
RIGHT/LEFT+A : Increment/decrement the value by 10
RIGHT/LEFT+B : Set the value to maximum/minimum
START        : Play the current mode's sound (or all modes if in control screen)
START+A      : Play a little music with the current mode's sound
SELECT       : Change the sound mode (1, 2, 3, 4 and control)
SELECT+A     : Dump the sound registers to the screen
```

9.22 space

The space example is an assembly program that demonstrates the use of sprites, window, background, fixed-point values and more. The following keys are used:

```

Arrow keys      : Change the speed (and direction) of the sprite
Arrow keys + A  : Change the speed (and direction) of the window
Arrow keys + B  : Change the speed (and direction) of the background
START           : Open/close the door
SELECT          : Basic fading effect

```

9.23 templates

Two basic template examples are provided as a starting place for writing your GBDK programs.

10 Frequently Asked Questions (FAQ)

10.1 General

- How can sound effects be made?
 - The simplest way is to use the Game Boy sound hardware directly. See the [Sound Example](#) for a way to test out sounds on the hardware.
 - Further discussion on using the Sound Example rom can be found in the ZGB wiki. Note that some example code there is ZGB specific and not part of the base GBDK API: <https://github.com/Zal0/ZGB/wiki/Sounds>

10.2 Licensing

- What license information is required when distributing the compiled ROM (binary) of my game or program?
 - There is no requirement to include or credit any of the GBDK-2020 licenses or authors, although credit of GBDK-2020 is appreciated.
 - This is different and separate from redistributing the GBDK-2020 dev environment itself (or the GBDK-2020 sources) which does require the licenses.

10.3 Graphics and Resources

- How do I use a tile map when its tiles don't start at index zero?
 - The two main options are:
 - * Use [set_bkg_based_tiles\(\)](#), [set_bkg_based_submap\(\)](#), [set_win_based_tiles\(\)](#), [set_win_based_submap\(\)](#) and provide a tile origin offset.
 - * Use [utility_png2asset](#) with `-tile_origin` to create a map with the tile index offsets built in.
- Is it normal for sprites to disappear when they reach the left border of the screen? (NES/SMS/MSX)
 - You can hide the leftmost column using [HIDE_LEFT_COLUMN](#) to work around this.
 - The behavior is due to NES/SMS/MSX having 8-bit Sprite x coordinates while the screen width is also 256 pixels. GB/GG don't have this problem since their screen is smaller and the x-coordinates are larger than the visible screen.

10.4 ROM Header Settings

- How do I set the ROM's title?
 - Use the [makebin](#) `-yn` flag. For example with `lcc -Wm-yn "MYTITLE"` or with [makebin](#) directly `-yn "MYTITLE"`. The maximum length is up to 15 characters, but may be shorter.
 - See "0134-0143 - Title" in [Pandocs](#) for more details.

- How do I set SGB, Color only and Color compatibility in the ROM header?
 - Use the following [makebin](#) flags. Prefix them with `-Wm` if using [lcc](#).
 - * `-yc` : GameBoy Color compatible
 - * `-yC` : GameBoy Color only
 - * `-ys` : Super GameBoy compatible
- How do I set the ROM [MBC](#) type, and what MBC values are available to use with the `-yt` [makebin](#) flag?
 - See [setting_mbc_and_rom_ram_banks](#)

10.5 Editors

- Why is VSCode flagging some GBDK types or functions as unidentified or giving warnings about them?
 - See [code_editors_hinting](#)
 - GBDK platform constants can be declared so that header files are parsed more completely in VSCode.

10.6 Errors and Warnings

- What does the error `old "gbz80" SDCC PORT name specified (in "-mgbz80:gb"). Use "sm83" instead. You must update your build settings. mean?`
 - The `PORT` name for the Game Boy and related clones changed from `gbz80` to `sm83` in the SDCC version used in GBDK-2020 4.1.0 and later. You must change your Makefile, Build settings, etc to use the new name. Additional details in the [Console Port and Platform Settings](#) section.
- What does the warning `?ASlink-Warning-Conflicting sdcc options: "-msm83" in module "_____" and "-mgbz80" in module "_____". mean?`
 - One object file was compiled with the `PORT` setting as `gbz80` (meaning a version of SDCC / GBDK-2020 **OLDER than GBDK-2020 4.1.0**).
 - The other had the `PORT` setting as `sm83` (meaning **GBDK-2020 4.1.0 or LATER**).
 - You must rebuild the object files using `sm83` with GBDK-2020 4.1.0 or later so that the linker is able to use them with the other object files. Additional details in the [Console Port and Platform Settings](#) section.
- What does the warning `?ASlink-Warning-Undefined Global ... mean?`
 - The linker is unable to find a variable or function that is referenced by some part of the program. This usually means a required source file is not being passed to the linker stage. Make sure all of your project source files are getting compiled and passed to the linker.
- What does `z80instructionSize() failed to parse line node, assuming 999 bytes mean?`
 - This is a known issue with SDCC Peephole Optimizer parsing and can be ignored. A bug report has been filed for it.
- What do these kinds of warnings / errors mean? `WARNING: possibly wrote twice at addr 4000 (93->3E) Warning: Write from one bank spans into the next. 7ff7 -> 8016 (bank 1 -> 2)`

- You may have a overflow in one of your ROM banks. If there is more data allocated to a bank than it can hold it then will spill over into the next bank.

A common problem is when there is too much data in ROM0 (the lower 16K unbanked region) and it spills over into ROM1 (the first upper 16K banked region). Make sure ROM0 has 16K or less in it.

The warnings are generated by [ihxcheck](#) during conversion of an .ihx file into a ROM file.

See the section [ROM/SRAM Banking and MBCs](#) for more details about how banks work and what their size is. You may want to use a tool such as [romusage](#) to calculate the amount of free and used space.

- What do these errors mean? `error: size of the buffer is too small error: ROM is too large for number of banks specified`

- Your program is using more banks than you have configured in the toolchain. Either the MBC type was not set, or the number of banks or MBC type should be changed to provide more banks.

See the section [setting_mbc_and_rom_ram_banks](#) for more details.

- What does `warning 283: function declarator with no prototype mean?`

- Function forward declarations and definitions which have no arguments should be changed from `func()` to `func(void)`.
- In C `func()` and `func(void)` do not mean the same thing. `()` means any number of parameters, `(void)` means no parameters. For example if a function with no arguments is declared with `()` then there may not be an error or warning when mistakenly trying to pass arguments to it.

- What do these warnings mean? `warning 126: unreachable code warning 110↵: conditional flow changed by optimizer: so said EVELYN the modified DOG`

- The compiler is indicating that some branches of the code will never get executed or will have no effect, and so have been removed during optimization. There may be a logical error, unnecessary logic or unused variables.
- If this is due to a variable which gets modified outside of normal execution (such as during an interrupt) then the `volatile` keyword can be added to it's declaration. This removes some optimizations and tells the compiler the variable's value may change unexpectedly and cannot be predicted only based on the surrounding code.

- What does this warning mean? `WARNING: overflow in implicit constant conversion`

- See [Constants, Signed-ness and Overflows](#)

- On macOS, what does `... developer cannot be verified, macOS cannot verify that this app is free from malware mean?`

- It does not mean that GBDK is malware. It just means the GBDK toolchain binaries are not signed by Apple, so it won't run them without an additional step.
- For the workaround, see the [macOS unsigned binary workaround](#) for details.

- What do the following kinds of warnings / errors mean? `info 218: z80instructionSize() failed to parse line node, assuming 999 bytes`

- This is a known issue with SDCC, it should not cause actual problems and you can ignore the warning.

10.7 Debugging / Compiling / Toolchain

- What flags should be enabled for debugging?
 - You can use the [lcc debug flag](#) `-debug` to turn on debug output. It covers most uses and removes the need to specify multiple flags such as `-Wa-l -Wl-m -Wl-j`. Also see [tools_debug](#).
- Is it possible to generate a debug symbol file (`.sym`) compatible with an emulator?
 - Yes, turn on `.noi` output (LCC argument: `-Wl-j` or `-debug` and then use `-Wm-yS` with LCC (or `-yS` with `makebin` directly).
 - Also see additional information about using [debugging tools](#).
- How do I move the start of the `DATA` section and the `Shadow OAM` location?
 - The default locations are: `_shadow_OAM=0xC000` and 240 bytes after it `_DATA=0xC0A0`
 - So, for example, if you wanted to move them both to start 256(0x100) bytes later, use these command line arguments for LCC:
 - * To change the Shadow OAM address: `-Wl-g_shadow_OAM=0xC100`
 - * To change the DATA address (again, 240 bytes after the Shadow OAM): `-Wl-b_DATA=0xc1a0`
- Why is the compiler so slow, or why did it suddenly get much slower?
 - This may happen if you have large initialized arrays declared without the `const` keyword. It's important to use the `const` keyword for read-only data. See [const_gbtd_gbmb](#) and [const_array_data](#)
 - It can also happen if C source files are `#included` into other C source files, or if there is a very large source file.
- Known issue: SDCC may fail on Windows when [run from folder names with spaces on non-C drives](#).

10.8 API / Utilities

- Is there a list of all functions in the API?
 - [Functions](#)
 - [Variables](#)
- Can I use the `float` type to do floating point math?
 - There is no support for 'float' in GBDK-2020.
 - Instead consider some form of `fixed point` math (including the [fixed](#) type included in GBDK).
- Why are 8 bit numbers not printing correctly with `printf()`?
 - To correctly pass `chars/uint8s` for printing, they must be explicitly re-cast as such when calling the function. See [docs_chars_varargs](#) for more details.
- How can maps larger than 32x32 tiles be scrolled? & Why is the map wrapping around to the left side when setting a map wider than 32 tiles with `set_bkg_data()`?
 - The hardware Background map is 32 x 32 tiles. The screen viewport that can be scrolled around that map is 20 x 18 tiles. In order to scroll around within a much larger map, new tiles must be loaded at the edges of the screen viewport in the direction that it is being scrolled. [set_bkg_submap](#) can be used to load those rows and columns of tiles from the desired sub-region of the large map.

- See the "Large Map" example program and [set_bkg_submap\(\)](#).
- Writes that exceed coordinate 31 of the Background tile map on the x or y axis will wrap around to the Left and Top edges.
- When using `gbt_player` with music in banks, how can the current bank be restored after calling `gbt_update()`? (since it changes the currently active bank without restoring it).
 - See [restoring the current bank](#)
- How can CGB palettes and other sprite properties be used with metasprites?
 - See [Metasprites and sprite properties](#)
- Weird things are happening to my sprite colors when I use `png2asset` and metasprites. What's going on and how does it work?
 - See [utility_png2asset](#) for details of how the conversion process works.

11 Migrating to new GBDK Versions

This section contains information that may be useful to know or important when upgrading to a newer GBDK release.

11.1 GBDK-2020 versions

11.1.1 Porting to GBDK-2020 4.5.0

- GBDK now requires ~SDCC 4.5.0 or higher with GBDK-2020 patches for the z80, sm83 and NES
- Build Host:
 - Changed from building on MacOS 13 to MacOS 15 for 64 bit Intel
 - Changed from building on Windows 2019 to 2022 for Intel
- SMS/GG
 - Changed parameter order of [set_tile_submap\(\)](#), [set_tile_submap_compat\(\)](#)
 - Remove broken and unused `FAST_DIV8` macro
 - Removed legacy `__sdcc_bcall_trampoline`
- MegaDuck
 - Renamed `DUCK_IO_CMD_PRINT_INIT_MAYBE_EXT_IO` to [DUCK_IO_CMD_PRINT_INIT_EXT_IO](#)
 - Removed `duck_io_printer_detected()`, `duck_io_printer_type()` and replaced them with [duck_io_printer_query\(\)](#), [duck_io_printer_last_status\(\)](#)
- `png2asset`
 - Fixed missing error for `-bin` requiring `-map`

11.1.2 Porting to GBDK-2020 4.4.0

- GBDK now requires ~SDCC 4.5.0 or higher with GBDK-2020 patches for the z80 and NES
- Build Host:
 - Changed from building on MacOS 11 to MacOS 13 for 64 bit Intel
 - Changed from building on Ubuntu Linux 20.04 to 22.04 for 64 bit Intel
- NES

- LCD `bkg_scroll_y` is now relative to the current scanline
 - * This change creates higher compatibility with the Game Boy `SCY_REG` and makes it easier to re-use Game Boy LCD handlers
 - * This behaves differently to 4.3.0 and affects LCD handlers that change the y scrolling coordinate mid-frame
- Added `makenes` utility for finalizing NES rom headers (called automatically by `lcc`)
- SMS/GG
 - Changed from unsigned (`uint16_t`) to signed `int16` (`int16_t`) for coordinates with the family of `...metasprite...()` functions
 - Changed screen map to be at `0x1800` instead of `0x3800`
 - * To switch to the older configuration use `__WRITE_VDP_REG_UNSAFE(VDP_R2, R2_MAP_0x3800);` and `__WRITE_VDP_REG_UNSAFE(VDP_R5, R5_SAT_0x3F00);` near the start of main
- SDCC
 - Changed to using the `-N` flag with `sdas` since the `-n` flag was removed
- Changed `lcc` to use `--no-optsdcc-in-asm` for building user programs and the GBDK library
 - This removes some "O line" meta-data from object files to avoid false-positive linker errors
 - It is a workaround for SDCC now appending the calling convention to the "O Line" in the object files by default

11.1.3 Porting to GBDK-2020 4.3.0

- GBDK now requires ~SDCC 4.4.0 or higher with GBDK-2020 patches for the z80 and NES
- Changed to new calling convention for `printf()`, `sprintf()`, `abs()`
- Changed to new SDCC calling convention for `set_bkg_tile_xy()`, `set_win_tile_xy()`
- The SDCC object file format (`.o`, `.rel` files) changed from XL3 (24 bit addresses) to XL4 (32 bit addresses)
 - Bankpack now supports both
- Recommend using:
 - `CURRENT_BANK` instead of `_current_bank`
 - `BANKED` macro instead of `__banked`
- NES `set_sprite_palette()` now indexes from `0..3` instead of `4..7`
- `png2asset`:
 - If using either `-bpp 1` or `-pack_mode 1bpp` then the other is auto-enabled
 - Significant bug fixes and changes, check to make sure output is as expected

11.1.4 Porting to GBDK-2020 4.2.0

- GBDK now requires ~SDCC 4.3 or higher with GBDK-2020 patches for the z80 and NES
- The following new functions replace old ones:
 - While the old functions will continue to work for now, migration to new versions is strongly encouraged
 - `vsync()`: replaces `wait_vbl_done()`
 - `set_default_palette()`: replaces `cgb_compatibility()`
 - `move_metasprite_flipy()`: replaces `move_metasprite_hflip()`
 - `move_metasprite_flipx()`: replaces `move_metasprite_vflip()`
 - `move_metasprite_flipxy()`: replaces `move_metasprite_hvflip()`

- `move metasprite_ex()`: replaces `move metasprite()`
- The unused `-DINT_16_BITS` argument was removed from the default SDCC compiler and preprocessor arguments (used in pre-GBDK2020 `gbdk/include-gb/types.h`)
- Removed legacy MBC register definitions `.MBC1_ROM_PAGE` and `.MBC_ROM_PAGE`
- SMS/GG
 - Swapped A and B buttons to match game boy buttons

11.1.5 Porting to GBDK-2020 4.1.1

- No significant changes required

11.1.6 Porting to GBDK-2020 4.1.0

- GBDK now requires SDCC 4.2 or higher with GBDK-2020 patches for the z80 linker
- The default calling convention changed in SDCC 4.2, see [Calling Conventions](#) for more details.
 - If you are linking to libraries compiled with an older version of SDCC / GBDK then you may have to recompile them.
 - If there are existing functions written in ASM which **receive parameters** they should also be reviewed to make sure they work with the new `__sdcccall(1)` calling convention, or have their header declaration changed to use `OLDCALL`.
 - If there are existing functions written in ASM which **call other functions written in C** the callee C function should be declared `OLDCALL`.
 - Function pointer declarations should be checked to see if they need `OLDCALL` added to the declaration.
 - * Example (add `OLDCALL` at the end)
 - * FROM: `typedef void (*someFunc)(uint8_t, uint8_t);`
 - * TO: `typedef void (*someFunc)(uint8_t, uint8_t) OLDCALL;`
 - If you are using tools such as `rgb2sdas` (from hUGETracker/Driver) you may need to edit the resulting `.o` file and replace `-mgbz80` with `-msm83` in addition to using `OLDCALL`
- The SDCC `PORT` name for the Game Boy and related clones changed from `gbz80` to `sm83`.
 - Additional details in the [Console Port and Platform Settings](#) section and [FAQ entry](#). `lcc` will error out if the old `PORT` name is passed in.
- The library base path changed from `lib/small/asxxxx/` to `lib/`.
 - For example `lib/small/asxxxx/gb` becomes `lib/gb`
- Allocations for ISR chain lengths were fixed.
 - Now they are VBL: 4 user handlers, LCD: 3 user handlers, SIO/TIM/JOY: 4 user handlers

11.1.7 Porting to GBDK-2020 4.0.6

- Renamed `bgb_emu.h` to `emu_debug.h` and `BGB_*` functions to `EMU_*`
 - Aliases for the `BGB_*` ones and a `bgb_emu.h` shim are present for backward compatibility, but updating to the new naming is recommended

11.1.8 Porting to GBDK-2020 4.0.5

- GBDK now requires SDCC 12259 or higher with GBDK-2020 patches
- Variables in static storage are now initialized to zero per C standard (but remaining WRAM is not cleared)
- [png2asset](#) is the new name for the `png2mtspr` utility
- `lcc` : Changed default output format when not specified from `.ihx` to `.gb` (or other active rom extension)
- The `_BSS` area is deprecated (use `_DATA` instead)
- The `_BASE` area is renamed to `_HOME`
- Variables in static storage are now initialized to zero per C standard (but remaining WRAM is not cleared)
- `itoa()`, `uitoa()`, `ltoa()`, `ultoa()` all now require a radix value (base) argument to be passed. On the Game Boy and Analogue Pocket the parameter is required but not utilized.
- `set_bkg_1bit_data` has been renamed to [set_bkg_1bpp_data](#)
- The following header files which are now cross platform were moved from `gb/` to `gbdk/`↔
: `bcd.h`, `console.h`, `far_ptr.h`, `font.h`, `gbdecompress.h`, `gbdk-lib.h`, `incbin.h`,
`metasprites.h`, `platform.h`, `version.h`
 - When including them use `#include <gbdk/...>` instead of `#include <gb/>`

11.1.9 Porting to GBDK-2020 4.0.4

- GBDK now requires SDCC 12238 or higher
- Made `sample.h`, `cgb.h` and `sgb.h` independent from `gb.h`

11.1.10 Porting to GBDK-2020 4.0.3

- No significant changes required

11.1.11 Porting to GBDK-2020 4.0.2

- The default font has been reduced from 256 to 96 characters.
 - Code using special characters may need to be updated.
 - The off-by-1 character index offset was removed for fonts. Old fonts with the offset need to be re-adjusted.

11.1.12 Porting to GBDK-2020 4.0.1

- **Important!** : The WRAM memory region is no longer automatically initialized to zeros during startup.
 - Any variables which are declared without being initialized may have **indeterminate values instead of 0** on startup. This might reveal previously hidden bugs in your code.
 - Check your code for variables that are not initialized before use.
 - In BGB you can turn on triggering exceptions (options panel) reading from uninitialized RAM. This allows for some additional runtime detection of uninitialized vars.
- In `.ihx` files, multiple writes to the same ROM address are now warned about using [ihxcheck](#).
- `set_*_tiles()` now wrap maps around horizontal and vertical boundaries correctly. Code relying on it not wrapping correctly may be affected.

11.1.13 Porting to GBDK-2020 4.0

- GBDK now requires SDCC 4.0.3 or higher
- The old linker `link-gbz80` has been REMOVED, the linker `sldlgb` from SDCC is used.
 - Due to the linker change, there are no longer warnings about multiple writes to the same ROM address.
- GBDK now generates `.ihx` files, those are converted to a ROM using `makebin` (lcc can do this automatically in some use cases)
- Setting ROM bytes directly with `-Wl-yp0x<address>=0x<value>` is no longer supported. Instead use `makebin` flags. For example, use `-Wm-yC` instead of `-Wl-yp0x143=0xC0`. See [faq_gb_type_header_setting](#).
- OAM symbol has been renamed to `_shadow_OAM`, that allows accessing shadow OAM directly from C code

11.1.14 Porting to GBDK-2020 3.2

- No significant changes required

11.1.15 Porting to GBDK-2020 3.1.1

- No significant changes required

11.1.16 Porting to GBDK-2020 3.1

- Behavior formerly enabled by `USE_SFR_FOR_REG` is on by default now (no need to specify it, it isn't a tested `#ifdef` anymore). check here why: <https://gbdev.gg8.se/forums/viewtopic.php?id=697>

11.1.17 Porting to GBDK-2020 3.0.1

- LCC was upgraded to use SDCC v4.0. Makefile changes may be required
 - The symbol format changed. To get bgb compatible symbols turn on `.noi` output (LCC argument: `-Wl-j` or `-debug`) and use `-Wm-yS`
 - ?? Suggested: With LCC argument: `-Wa-l(sdasgb:-a All user symbols made global)`
 - In SDCC 3.6.0, the default for char changed from signed to unsigned.
 - * If you want the old behavior use `--fsigned-char`.
 - * lcc includes `--fsigned-char` by default
 - * Explicit declaration of unsigned vars is encouraged (for example, `'15U'` instead of `'15'`)
 - `.init` address has been removed

11.2 Historical GBDK versions

11.2.1 GBDK 1.1 to GBDK 2.0

- Change your int variables to long if they have to be bigger than 255. If they should only contain values between 0 and 255, use an unsigned int.
- If your application uses the delay function, you'll have to adapt your delay values.
- Several functions have new names. In particular some of them have been changed to macros (e.g. `show_↵ bkg()` is now `SHOW_BKG`).
- You will probably have to change the name of the header files that you include.

12 GBDK Release Notes

The GBDK-2020 releases can be found on Github: <https://github.com/gbdk-2020/gbdk-2020/releases>

12.1 GBDK-2020 Release Notes

12.1.1 GBDK-2020 4.5.0

2025/12

- SDCC
 - Added HRAM area access for sm83
 - Added `#pragma dataseg DATA_<N>` support for sm83 and z80 platforms
 - [Patched SDCC Builds](#) with support for all platforms are used.
 - See the [github workflow](#) for details.
- Library
 - Added dynamic HRAM variable allocation for GB/AP/Duck
 - Added zx0 decompression support for sm83, z80 platforms: `zx0_decompress()`
 - Changed NULL definition in `types.h` to match `stddef.h`
 - Fixed escaping % symbol in `printf()` and `sprintf()`
 - GB/AP/Duck
 - * Improved `get_bkg_xy_addr()`, `get_win_xy_addr()`
 - * Improved performance for banked calls
 - SMS/GG
 - * Changed GG "not interrupt" link port control bit constant from `GGEXT_NINIT` to `GGEXT_NINT`
 - * Remove broken and unused `FAST_DIV8` macro
 - * Removed legacy `__sdcc_bcall` trampoline
 - * Fixed parameter order of `set_tile_submap()`, `set_tile_submap_compat()`
 - * Fixed tilemap width not set correctly in `set_tile_submap_compat()`
 - NES
 - * Remove broken and unused `FAST_DIV8` macro
 - * Fixed `fill_bkg_rect()` screen edge wrap, now calls `set_bkg_tile_xy()`
- Examples
 - Added HRAM variable example for GB/AP/Duck
 - Added SRAM bank example for new `#pragma dataseg DATA_<N>`
 - Added MegaDuck and Game Gear support to Game Boy Printer example
 - Added missing compile.bat for some examples
 - Improved the Text Scroller example
 - Improved RLE compress example to convert and compress map at build time
 - Fixed Platformer example incorrectly using `-keep_duplicate_tiles` flag
 - Fixed Text Advanced Dialogue example printing (Rodrigo Card)
 - Game Boy
 - * Added MBC5 Rumble example (Ev3)
 - * Improved Rand example with fast modulo, triangle and bell curve distributions
 - * Fixed Sound example not writing length when length bit enabled and triggering channel
 - MegaDuck
 - * Added MegaDuck Laptop Printer example
 - SMS
 - * Added NMI handler example
- Toolchain / Utilities

- [png2asset](#)
 - * Added binary palette export for `-bin` mode
 - * Added logging of conversion arguments to `.c` and `.h` output files
 - Path is stripped to avoid potential unwanted username/etc disclosure
 - * Added `-use_metafile` option to load arguments from `<inputfile>.meta`
 - * Changed to separate attribute array for `-use_map_attributes` + `-use_structs` with SMS/GG
 - * Fixed broken transparency for SGB borders with more than one 16 color palette
 - * Fixed missing error for `-bin` requiring `-map`
 - * Fixed incorrect line breaks and trailing spaces in output
- [bankpack](#)
 - * Changed Linkerfile output order to match packing order for use with asm `.bndry` alignment in rom banks
 - First non-banked, then fixed bank, then auto-banked. Banknum ascending, size descending
 - * Improved bank assignment printout
- [gbcompress](#)
 - * Added include guards for header file output (Rodrigo Card)
 - * Added `zx0` compression and decompression mode: `--alg=zx0`
- [romusage](#)
 - * Fixed missing error when filename not present
- Docs:
 - Improved html style and mobile web formatting
 - Various updates and improvements

12.1.2 GBDK-2020 4.4.0

2025/05

- Includes SDCC version ~4.5.0 (15267) with GBDK-2020 patches for Z80 and NES
 - [Patched SDCC Builds](#) with support for Sega GG/SMS and the Nintendo NES are used.
 - See the [github workflow](#) for details.
- Building GBDK
 - Added native GBDK build for ARM 64 Linux
 - Changed from building on MacOS 11 to MacOS 13 for 64 bit Intel
 - Changed from building on Ubuntu Linux 20.04 to 22.04 for 64 bit Intel
- SDCC
 - Changed to using the `-N` flag with `sdas` since the `-n` flag was removed
- Library
 - Added export of `_vbl_done` ([VBL_DONE](#)), a flag indicating the VBlank ISR has run
 - Game Boy
 - * Added MBC7 related register and flag definitions
 - [rMBC7_SRAM_ENABLE_1](#), [rMBC7_SRAM_ENABLE_2](#), [rMBC7_LATCH_1](#), [rMBC7_LATCH_2](#), [rMBC7_ACCEL_X_LO](#), [rMBC7_ACCEL_X_HI](#), [rMBC7_ACCEL_Y_LO](#), [rMBC7_ACCEL_Y_HI](#)
 - [MBC7_LATCH_ERASE](#), [MBC7_LATCH_CAPTURE](#), [MBC7_SRAM_ENABLE_KEY_1](#), [MBC7_SRAM_ENABLE_KEY_2](#)
 - * Fixed possible halt bug when using [hblank_copy_vram\(\)](#)
 - NES

- * Added `delay()` and `reset()` functions for compatibility
- * Added Timer interrupt emulation: `add_TIM()` , `remove_TIM()`
- * Improved LCD ISR support for less graphics glitches
- * Improved palette function code size (smaller)
- * Changed LCD `bkg_scroll_y` to be relative to the current scanline for higher compatibility with Game Boy SCY_REG and LCD handlers
 - Align coordinates and scanline counting in LCD ISR implementation with GB, add SCX / SCY / LYC defines
- * Fixed bug with `set_bkg_1bpp_data()` not working in buffered mode
- * Fixed bug with multiple column attribute updates
- * Fixed bugs with non-multiple-of-2 (/4) map width in `set_bkg_submap_attributes()`
- SMS/GG
 - * Added Timer interrupt emulation: `add_TIM()` , `remove_TIM()`
 - * Improved metasprite Y clipping
 - * Changed from unsigned (`uint16_t`) to signed `int16` (`int16_t`) for coordinates with the family of `...metasprite...()` functions
- MegaDuck
 - * Added header files: `duck/laptop_io.h`, `duck/model.h`, `duck/laptop_keycodes.h`
- Toolchain / Utilities
 - Added `makenes` utility for finalizing NES rom headers (called automatically by `lcc`)
 - `lcc`
 - * Changed use `--no-optsdcc-in-asm` for building user programs and the GBDK library
 - This removes some "O line" meta-data from object files to avoid false-positive linker errors
 - It is a workaround for SDCC now appending the calling convention to the "O Line" in the object files by default
 - * Changed to warn about some deprecated flags: `A,b, B, -dn, -g, -n, -O, P, -p, -static, -t, -w`
 - `png2asset`
 - * Added `-area` option to specify linker area name (such as `-area LIT` for SMS/GG)
 - * Added support for 512 tiles via alternate tile bank on GBC
 - * Fixed Palette Generation broken when using `-source_tileset` option
 - `bankpack`
 - * Added support for MBC7
 - * Fixed bugs with large number of object files
 - `romusage`
 - * Added Set hex bytes treated as Empty in ROM files (`.gb/etc`) `-b:HEXVAL:[...]`
 - * Added option to Hide memory regions (ex hide all RAM: `-nMEM:RAM`)
 - * Added support NES (`-p:NES1`) `.noi/.map` files
 - * Changed to allow filename at any location in option arguments
 - * Changed to improve error messaging
 - * Fixed detecting areas with leading underscores for `.noi/.map` files
 - * Fixed Brief/summarized mode counting overlapped header areas multiple times
 - `png2hicolorgb`
 - * Added `--palendbit` for indicating end of data
 - * Added `--addendcolor=N` for appending color to clear the background on non-full height images
 - * Added `--precompiled` mode
 - * Added `-s` to specify variable/symbol name in C output
 - `ihxcheck`

- * Fixed check on max bank to prevent crash
- Examples
 - Added Text and Dialog example
 - Changed irq example to be cross-platform
 - Improved emu_debug example
 - Improved Logo example Readme
 - Game Boy
 - * Added MBC3 Real Time Clock (RTC) example
 - * Added MBC7 Accelerometer example
 - MegaDuck
 - * Added MegaDuck Laptop model examples for keyboard, RTC and speech
- Docs:
 - Added Cart SRAM Max Size to MBC Chart and clarify battery/save meaning
 - Various updates and improvements

12.1.3 GBDK-2020 4.3.0

2024/06

- Includes SDCC version ~4.4.0 (14650) with GBDK-2020 patches for Z80 and NES
 - [Patched SDCC Builds](#) with support for Sega GG/SMS and the Nintendo NES are used.
 - See the [github workflow](#) for details.
- Added native GBDK build for Apple ARM cpus
- Known Issues
 - SDCC may fail on Windows when [run from folder names with spaces on non-C drives](#).
- Library
 - Added [get_system\(\)](#) which indicates system speed
 - * [SYSTEM_60HZ](#), [SYSTEM_50HZ](#), [SYSTEM_BITS_DENDY](#), [SYSTEM_BITS_NTSC](#), [SYSTEM_BITS_PAL](#), [SYSTEM_NTSC](#)
 - Changed to new calling convention for [printf\(\)](#), [sprintf\(\)](#), [abs\(\)](#)
 - Changed [EMU_printf\(\)](#) to remove dependency on stdio.h added similar [EMU_fmtbuf\(\)](#)
 - Fixed [emu_debug.h](#) macros missing a trailing space
 - NES
 - * Added PAL support (detects NTSC/PAL/Dendy on reset to adjust fake LCD ISR timings)
 - * Added BCD support
 - * Added deferred hblank system for fake LCD ISRs, allowing for multiple splits per-frame
 - * NMI optimization: Only call delay routine for remaining-vblank and LCD handler execution if LCD handler present
 - * NMI optimization: Skip OAM DMA and delay routines when display is off, to increase unbuffered performance
 - * NMI optimization: Save 8 cycles of stripe setup cost in vram transfer buffer execution
 - * ZP memory optimization: Make gbdk-nes functions use its own dedicated overlay segment
 - * Fixed [_map_tile_offset](#) not being applied for [set_bkg_based_tiles\(\)](#)
 - * Fixed VRAM transfer buffer bug (ensure stack page cleared on reset)
 - * Fixed support for 4-player controllers using fourscore
 - * Fixed [set_sprite_palette\(\)](#) to index from 0 . . 3 instead of 4 . . 7

- * Fixed `move metasprite` to initialize Y index register to zero
- * Fixed `waitpadup` to wait for button release instead of press
- * Updated libc to latest from sdcc 4.4.0
- SMS/GG
 - * Added `SHOW_SPRITES`, `HIDE_SPRITES` (no hiding mid-frame)
 - * Added `S_BANK` tile attribute
 - * Added 6 button controller support in `joypad()`
 - * Added BCD support
 - * Added ability to move VDP SAT and name table to other locations by writing to VDP R2 and VDP R5.
 - Set name table to 0x1800 and SAT to 0x1F00 by default to free up some sprite tile space
 - Added `R5_SAT_0x1F00` definition for the R5 value controlling SAT position in VRAM
 - * Added `__WRITE_VDP_REG_UNSAFE()` VDP macro while interrupts are disabled (such as in ISR handlers)
 - * Added Game Gear registers and definitions
 - `GG_STATE`: `GGSTATE_STT`, `GGSTATE_NJAP`, `GGSTATE_NNTS`
 - `GG_EXT_7BIT`
 - `GG_EXT_CTL`: `GGEXT_NINIT`
 - `GG_SIO_SEND`
 - `GG_SIO_RECV`
 - `GG_SIO_CTL`: `SIOCTL_TXFL`, `SIOCTL_RXRD`, `SIOCTL_FRER`, `SIOCTL_INT`, `SIOCTL_TON`, `SIOCTL_RON`, `SIOCTL_BS0`, `SIOCTL_BS1`
 - `GG_SOUND_PAN`: `SOUNDPAN_TN1R`, `SOUNDPAN_TN2R`, `SOUNDPAN_TN3R`, `SOUNDPAN_NOSR`, `SOUNDPAN_TN1L`, `SOUNDPAN_TN2L`, `SOUNDPAN_TN3L`, `SOUNDPAN_NOSL`
 - * Optimized native tile data loading routines
 - * Improved `WRITE_VDP_DATA` macro so it does not need `di/ei` protection
 - * Improved palette initialization in `crt0`
 - * Fixed tilemap wrapping over the low bound of the VDP name table
 - * Fixed `scroll_sprite()`
 - * Fixed missing `sms.h` in `sms/metaspites.h`
 - * Fixed scroll glitch due to accessing VDP too fast
- Z80 shared SMS/GG/MSX
 - * Added `DIV_REG` emulation for the Z80 systems, may be useful for seeding RNG
 - * Changed VDP to reduce chances of dangerous ISR nesting (see SDCC issue <https://sourceforge.net/p/sdcc/bugs/3721/>)
 - * Changed to allow nested locking of the shadow SAT copying on VBlank
 - * Optimized `memcpy()` for larger amounts of data
 - * Fixed `waitpad()`
 - * Fixed return result of `"set_tile x, y"` family functions
 - * Fixed to not allow interrupts to fire during `crt0` initialization code
- MSXDOS
 - * Fixed `.VDP_COLORDATA2` assembly definition
- Game Boy
 - * Added HBlank copy routines: `hblank_copy_vram()`, `hblank_cpy_vram()`, `hblank_copy()`
 - * Added `SCF_START`, `SCF_SOURCE`, `SCF_SPEED` aliases for SIO (Serial/Link port) control register control constants
 - * Added clamping and changed to new SDCC calling convention for `set_bkg_tile_xy()`, `set_win_tile_xy()`
 - * Added `S_BANK` tile attribute
 - * Fixed 8-bit signed modulus

- MegaDuck
 - * Fixed ROM bank switching on hardware when trying to enable or switch SRAM banks
- Toolchain / Utilities
 - Added [romusage](#) utility for viewing free/used ROM and RAM in compiled programs
 - [lcc](#)
 - * Changed `-debug` to add the following flags: `-Wa-l -Wl-u -Wl-w`
 - [png2asset](#)
 - * Added `-sprite_no_optimize`: sprite conversion will keep duplicate and empty sprite tiles
 - * Added `-entity_tileset`: mark entity locations on maps during conversion with an entity tileset
 - * Added `-rel_paths`: paths to tilesets are relative to the input file path
 - * Changed `-keep_palette_order` to round up to at least one palette
 - * Changed `-use_structs + -source_tileset` behavior
 - Point tile data to external source tileset
 - Add `extra_tiles` struct member pointing to map tiles not found in source tileset (null if none found)
 - * Changed `-use_structs` to use designated initializers
 - * Changed if using either `-bpp 1` or `-pack_mode 1bpp` then the other is auto-enabled
 - * Fixed garbage in unused colors of palettes (set unused colors to black)
 - * Fixed `-bin` mode not honoring `-tiles_only` and `-maps_only`
 - * Fixed segfault and wrong data size for `-pack_mode sgb + -bin`
 - * Fixed missing Palette ID in attributes for multi-palette SMS/GG backgrounds
 - * Fixed not taking `-bpp` into account when converting metasprites and emitting `<symbol>_↵ tile_pals[]`
 - * Fixed crash when filename for `-o` and `-c` not specified
 - * Fixed some attributes missing for metasprite export
 - [makebin](#)
 - * Fixed crash when using `-yS` (`-Wm-yS` with `lcc`)
 - [bankpack](#)
 - * Added `-banktype=` to allow forcing a bank type to CODE or LIT before packing starts
 - * Added support for XL4 (32 bit addresses) object file format (in addition to existing XL3)
 - * Changed minimum bank for auto packing from 1 to 0 (required for the NES)
- Examples
 - Added HBlank copy example for GB/AP/Duck
 - Added Reading SNES joypads on NES example
 - Added Game Boy Printer example
 - Added Joypad testing example
 - Added Display System example to demonstrate [get_system\(\)](#)
 - Added Platformer example
 - Added GBDK_DEBUG Makefile environment var for turning on/off debug flags
 - Changed wav sample: play waveforms on the SMS/GG PSG
 - Changed Random Number example: only call [initrand\(\)](#) once
 - Changed Large Map: support modified initial camera position
 - Changed all examples: use [BANKED](#) macro instead of `__banked`
 - * Also change some to use [CURRENT_BANK](#) instead of `_current_bank`
 - Fixes for SMS/GG: Fonts, Large Map, gbdecompress

- Fixed NES version of Text Scroller to have multiple splits as other platforms do
- Fixed Simple Physics: joypad input caching was wrong
- Fixed Banks Non-Intrinsic: mismatched SRAM banks for final calculation, improved naming
- Removed Analogue Pocket examples that were just duplicates of Game Boy ones
- Docs:
 - Fixed search where some exact matches didn't return a result
 - Various updates and improvements
 - Added more historical release notes

12.1.4 GBDK-2020 4.2.0

2023/08

- Includes SDCC version ~4.3 with GBDK-2020 patches for Z80 and NES
 - [Patched SDCC Builds](#) with support for Sega GG/SMS and the Nintendo NES are used. See the [github workflow](#) for details
- Known Issues
 - SDCC may fail on Windows when [run from folder names with spaces on non-C drives](#).
- Library
 - Added `NORETURN` definition (for `_Noreturn`)
 - Added: [set_bkg_attributes\(\)](#), [set_bkg_submap_attributes\(\)](#), [set_bkg_attribute_xy\(\)](#)
 - The following new functions replace old ones. The old functions will continue to work for now, but migration to new versions is strongly encouraged.
 - * [vsync\(\)](#): replaces [wait_vbl_done\(\)](#)
 - * [set_default_palette\(\)](#): replaces [cgb_compatibility\(\)](#)
 - metasprites: added metasprite functions which can set base sprite property parameter (`__current←_base_prop`) for GB/GBC and NES
 - * [move_metasprite_flipy\(\)](#): replaces [move_metasprite_hflip\(\)](#)
 - * [move_metasprite_flipx\(\)](#): replaces [move_metasprite_vflip\(\)](#)
 - * [move_metasprite_flipxy\(\)](#): replaces [move_metasprite_hvflip\(\)](#)
 - * [move_metasprite_ex\(\)](#): (replaces [move_metasprite\(\)](#))
 - NES
 - * Added support for much of the GBDK API
 - * Banking support (library and sdcc toolchain)
 - * Added [set_bkg_attributes_nes16x16\(\)](#), [set_bkg_submap_attributes_nes16x16\(\)](#), [set_bkg_attribute_xy_nes16x16\(\)](#)
 - SMS/GG
 - * Swapped A and B buttons to match game boy buttons
 - * X coordinate metasprite clipping on the screen edges
 - Game Boy
 - * Minor crt0 optimizations
 - * Faster [vmemcpy\(\)](#), [set_data\(\)](#), [get_data\(\)](#)
 - * Fixed [hide_sprites_range\(39u, 40u\)](#); overflow shadow OAM
 - * Increased [sgb_transfer\(\)](#) maximum packet length to 7 x 16 bytes
 - * Convert [gb_decompress](#) routines to the new calling convention
 - * Convert [rle_decompress](#) routines to the new calling convention
 - * Removed legacy MBC register definitions `.MBC1_ROM_PAGE` and `.MBC_ROM_PAGE`
 - * Workaround for possible HALT bug in Crash Handler

- Refactored interrupts to use less space
- Toolchain / Utilities
 - Added [png2hicolorgb](#)
 - [lcc](#)
 - * Fixed `--sdccbindir`
 - * Removed the unused `-DINT_16_BITS` from the default SDCC compiler and preprocessor arguments
 - * Improved improved Game Gear header compatibility (change header region code from 4 to 6)
 - [png2asset](#)
 - * Added `-o` as a more standard version of the `-c` argument
 - * Added `-repair_index_pal`: Tries to repair tile palettes for indexed color pngs (such as when RGB paint programs mix up indexed colors if the same color exists in multiple palettes). Implies `-keep_palette_order`
 - * Added `-no_palettes`: Do not export palette data
 - * Fixed support for indexed color pngs with less than 8 bits color depth
 - * Fixed incorrect palettes when different colors have same luma value (use RGB values as less-significant bits)
 - * Fixed `-keep_duplicate_tiles` not working with `-source_tileset`
 - * Changed to use cross-platform constants for metasprite properties (`S_FLIPX`, `S_FLIPY` and `S_↔PAL`)
 - [makebin](#)
 - * Warn if RAM banks specified and file size of ROM is less than the 64K required to enable them with in emulators
 - Added `sdl6808` (for NES)
- Examples
 - Fixed `mkdir` broken in some `compile.bat` files (remove unsupported `-p` flag during bat file conversion)
 - Sound Test: Added MegaDuck support
 - Wav Playback: Improved support on AGB/AGS hardware
 - Metasprites: Added sub-palette switching for GBC and NES, software metasprite flipping for sms/gg
 - Large Map: Added color for supported platforms
 - LCD ISR Wobble: Improved interrupt flag settings
 - Added GB-Type example
 - Added Game Boy Color Hi-Color example using [png2hicolorgb](#)
- Docs:
 - Improved search to do partial matches instead of matching start of string only
 - Added SDAS assembler manual (`asmInk_manual.txt`)
 - Added section on [NES support](#)
 - Added [Using Game Boy Color Features](#)
 - Updated [MegaDuck hardware documentation](#)
 - Added [Banked Calling Convention](#)
 - Added mention of [MAX_HARDWARE_SPRITES](#)

12.1.5 GBDK-2020 4.1.1

2022/11

- Includes SDCC version 13350 with GBDK-2020 patches for Z80
- Library
 - Fixed [RGB\(\)](#) and [RGB8\(\)](#) macros

12.1.6 GBDK-2020 4.1.0

2022/10

- Includes SDCC version 13350 with GBDK-2020 patches for Z80
- Known Issues
 - The `compile.bat` batch files for Windows use the an invalid `-p` option for `mkdir`
- Building GBDK
 - The linux port of SDCC is custom built on Ubuntu 16.04 due to reduced GLIBC compatibility issues in more recent SDCC project builds.
 - Added Windows 32-Bit build
- Platforms
 - SDCC has renamed the `gbz80` port to `sm83` see [faq_gbz80_sm83_old_port_name_error](#)
 - Added experimental support for MSXDOS (`msxdos`) and NES (`nes`). These platforms are not fully functional at this time. See [Supported Consoles & Cross Compiling](#)
- Licensing
 - Clarified licensing status with consent from GBDK original authors, added licensing folder to distribution
- Library
 - SGB: Use longer wait between the SGB packet transfers
 - SMS/GG: less garbage on screen when clearing VRAM in the init code
 - SMS/GG: Added [cgb_compatibility\(\)](#) to set default palette with the four shades of gray
 - Fixed: [get_sprite_data\(\)](#), [get_bkg_data\(\)](#) , [get_win_data\(\)](#) when `LDCDCF_BG8000` bit of `LDCDC_REG` is set
 - Fixed ISR chain lengths. VBL: 4 user handlers, LCD: 3 user handlers, SIO/TIM/JOY: 4 user handlers
 - Added new constants for the Game Boy Color (CGB):
 - * [VBK_BANK_0](#), [VBK_BANK_1](#)
 - * [VBK_TILES](#), [VBK_ATTRIBUTES](#)
 - * [BKGF_PRI](#), [BKGF_YFLIP](#), [BKGF_XFLIP](#), [BKGF_BANK0](#), [BKGF_BANK1](#)
 - * [BKGF_CGB_PAL0](#), [BKGF_CGB_PAL1](#), [BKGF_CGB_PAL2](#), [BKGF_CGB_PAL3](#), [BKGF_CGB_PAL4](#), [BKGF_CGB_PAL5](#), [BKGF_CGB_PAL6](#), [BKGF_CGB_PAL7](#)
 - * [VBK_TILES](#), [VBK_ATTRIBUTES](#)
- Toolchain / Utilities
 - [lcc](#)
 - * Changed to Error out and warn when `gbz80` port is used instead of `sm83`
 - [png2asset](#)
 - * Added `-tiles_only`: Export tile data only
 - * Added `-maps_only`: Export map tilemap only
 - * Added `-metasprites_only`: Export metasprite descriptors only
 - * Added `-source_tileset`: Use source tileset image with common tiles
 - * Added `-keep_duplicate_tiles`: Do not remove duplicate tiles
 - * Added `-bin`: Export to binary format (includes header files)
 - * Added `-transposed`: Export transposed (column-by-column instead of row-by-row)
 - * Added basic MSXDOS support
 - Added 1bpp packing mode (`BPP1`)
 - `-spr16x16msx`
 - * Added basic NES support

- `-use_nes_attributes`
 - `-use_nes_colors`
 - * Changed to only export `_tile_pals[]` arrays when `-use-structs` is set (ZGB specific)
- [gbcompress](#)
 - * Added `--bank=<num>` Add Bank Ref: 1 - 511 (default is none, with `--cout` only)
 - * Fixed failure to flush data at end of compression (uncommitted bytes)
 - * Fixed Warning: File read size didn't match expected
- [lcc](#)
 - * When `-autobank` is specified `lcc` will automatically add `-yoA` for [makebin](#) if no `-yo*` entry is present
 - * Fixed broken `-E` Preprocess only flag
- [makecom](#)
 - * Added `makecom` for post-processing msxdos binaries
- [makebin](#)
 - * Fixed (via `sdcc`) bug with `-yp` not always working correctly
 - <https://sourceforge.net/p/sdcc/code/12975/>
- [bankpack](#)
 - * Added support for the Game Boy Camera MBC
 - * Added `-reserve=<bank>:<size>` option to reserve space during autobank packing
 - Workaround for libraries that contain objects in banks (such as `gbt-player`)
- [ihxcheck](#)
 - * Check and warn for bank overflows under specific conditions
 - A multiple write to the same address must occur. The address where the overlap ends is used as BANK.
 - There must also be a write which spans multiple banks, the ending address of that must match BANK. The starting addresses is the OVERFLOW-FROM BANK.
- Examples
 - Changed Logo example to use new GBDK logo art from user "Digit"
 - Added example for APA image mode with more than 256 tiles
 - Added SGB Sound Effects example
 - Changed to new WAV sound example
- Docs
 - Added improved [MBC Type chart](#)
 - Include SDCC manual in pdf format
 - Various doc updates and improvements

12.1.7 GBDK-2020 4.0.6

2022/02

- Includes SDCC version 12539 with GBDK-2020 patches for Z80
- Building GBDK
 - Changed to target older version of macOS (10.10) when building for better compatibility
- Platforms
 - Added support for MegaDuck / Cougar Boy (`duck`). See [Supported Consoles & Cross Compiling](#)

- Library

- Added [memcmp\(\)](#)
- Added [add_low_priority_TIM\(\)](#) function for timer interrupts which allow nesting for GB/CGB
- Added [set_bkg_based_tiles\(\)](#), [set_bkg_based_submap\(\)](#), [set_win_based_tiles\(\)](#), [set_win_based_submap\(\)](#) for when a map's tiles don't start at VRAM index zero
- Added [clock\(\)](#) for SMS/GG
- Added macro definitions for SDCC features:
 - * `#define SFR __sfr`
 - * `#define AT(A) __at(A)`
- Added check for OAM overflow to metasprite calls for GB/CGB
- Added constant definitions [PSG_LATCH](#), [PSG_CH0](#), [PSG_CH1](#), [PSG_CH2](#), [PSG_CH3](#), [PSG_VOLUME](#) for SMS/GG
- Renamed `bgb_emu.h` to `emu_debug.h` and `BGB_*` functions to `EMU_*`.
 - * Aliases for the `BGB_*` ones and a `bgb_emu.h` shim are present for backward compatibility
- Changed headers to wrap SDCC specific features (such as `NONBANKED`) with `#ifdef __SDCC`
- Changed [rand\(\)](#) and [arand\(\)](#) to return `uint8_t` instead of `int8_t` (closer to the standard)
- Fixed declaration for [PCM_SAMPLE](#) and definition for [AUD3WAVE](#)
- Fixed definition of `size_t` to be `unsigned int` instead of `int`
- Fixed [vmemcpy\(\)](#) and [memmove\(\)](#) for SMS/GG
- Fixed random number generation for SMS/GG
- Fixed letter U appearing as K for min font
- Fixed define name in `crash_handler.h`
- Exposed [__rand_seed](#)

- Toolchain / Utilities

- [png2asset](#)
 - * Added SMS/GG graphics format support
 - * Added 4bpp and SGB borders
 - * Added warning when image size is not an even multiple of tile size
 - * Added `-tile_origin` offset option for when map tiles do not start at tile 0 in VRAM
 - * Added `*_TILE_COUNT` definition to output
 - * Fixed CGB `...s_map_attributes` type definition in output
 - * Fixed values for `num_palettes` in output
 - * Fixed incorrect `TILE_COUNT` value when not `-using_structs`
- [lcc](#)
 - * Changed [makebin](#) flags to turn off Nintendo logo copy for GB/CGB (use version in crt instead)
 - * Fixed lcc handling of `makebin -x*` arguments

- Examples

- Added logo example (cross-platform)
- Added [ISR_VECTOR](#) example of a raw ISR vector with no dispatcher for GB/CGB
- Changed `sgb_border` example to use `png2asset` for graphics
- Changed use of [set_interrupts\(\)](#) in examples so it's outside critical sections (since it disables/enables interrupts)
- Changed cross-platform auto-banks example to use `.h` header files
- Changed SGB border example to also work with SGB on PAL SNES

- Docs

- Added new section: Migrating From Pre-GBDK-2020 Tutorials

12.1.8 GBDK-2020 4.0.5

2021/09

- Includes SDCC version 12539 with GBDK-2020 patches for Z80
- Known Issues
 - SDCC: `z80instructionSize()` failed to parse line node, assuming 999 bytes
 - * This is a known issue with the SDCC Peephole Optimizer parsing and can be ignored.
 - `-bo<n>` and `-ba<n>` are not supported by the Windows build of [sdcc](#)
 - On macOS the cross platform `banks` example has problems parsing the filename based ROM and RAM bank assignments into `-bo<n>` and `-ba<n>`
- Added support for new consoles. See [Supported Consoles & Cross Compiling](#)
 - Analogue Pocket (`ap`)
 - Sega Master System (`sms`) and Game Gear (`gg`)
- Library
 - Fixed error when calling `get_bkg_tile_xy`: '?ASlink-Warning-Undefined Global '.set_tile_xy' referenced by module '?ASlink-Warning-Byte PCR relocation error for symbol .set_tile_xy
 - Variables in static storage are now initialized to zero per C standard (but remaining WRAM is not cleared)
 - Added many new register flag constants and names. For example:
 - * [rLDC](#) is a new alias for [LDC_REG](#)
 - * [LDCF_WINON](#), [LDCF_WINOFF](#), [LDCF_B_WINON](#)
 - Added [BANK\(\)](#), [BANKREF\(\)](#), [BANKREF_EXTERN\(\)](#)
 - Added [INCBIN\(\)](#), [BANK\(\)](#), [INCBIN_SIZE\(\)](#), [INCBIN_EXTERN\(\)](#)
 - Added generic [SWITCH_ROM\(\)](#) and [SWITCH_RAM\(\)](#)
 - Added [BGB_printf\(\)](#) and updated emulator debug output.
 - Added [set_native_tile_data\(\)](#), [set_tile_map\(\)](#), [set_1bpp_colors](#), [set_bkg_1bpp_data](#), [set_sprite_1bpp_data](#), [set_2bpp_palette](#), [set_bkg_2bpp_data](#), [set_sprite_2bpp_data](#), [set_tile_2bpp_data](#) (`sms/gg` only), [set_bkg_4bpp_data](#) (`sms/gg` only), [set_sprite_4bpp_data](#) (`sms/gg` only)
 - Added RLE decompression support: [rle_init\(\)](#), [rle_decompress\(\)](#),
 - Changed [itoa\(\)](#), [uitoa\(\)](#), [ltoa\(\)](#), [ultoa\(\)](#) to now require a radix value (base) argument to be passed. On the Game Boy and Analogue Pocket the parameter is required but not utilized.
- Examples
 - Added cross-platform examples (build for multiple consoles: `gb`, `ap`, `sms`, `gg`)
 - Added `sms`, `gg`, `pocket(ap)` examples
 - Added `incbin` example
 - Added simple physics sub-pixel / fixed point math example
 - Added `rle` decompression example
 - Changed windows `make.bat` files to `compile.bat`
 - Bug fixes and updates for existing examples
- Toolchain / Utilities
 - [png2asset](#)
 - * [png2asset](#) is the new name for the `png2mtspr` utility
 - * Added collision rectangle width and height (`-pw`, `-ph`)
 - * Added option to use the palette from the source png (`-keep_palette_order`)

- * Added option to disable tile flip (`-noflip`)
- * Added export as map: `tileset + bg` (`-map`)
- * Added option to use CGB BG Map attributes (`-use_map_attributes`)
- * Added option to group the exported info into structs (`-use_structs`)
- [lcc](#)
 - * Use `-m` to select target port and platform: `"-m[port]:[plat]"` ports:gbz80,z80 plats↵:ap,gb,sms,gg
 - * Changed default output format when not specified from `.ihx` to `.gb` (or other active rom extension)
 - * Changed lcc to always use the linkerfile `-lkout=` option when calling bankpack
 - * Fixed name generation crash when outfile lacks extension
- [bankpack](#)
 - * Added linkerfile input and output: `-lkin=<file>, -lkout=<file>`
 - * Added selector for platform specific behavior `plat=<plat>` (Default:gb, Available:gb, sms). sms/gg targets prohibits packing `LIT_N` areas in the same banks as `CODE_N` areas
 - * Added randomization for auto-banks (`-random`) for debugging and testing
- [utility_gbcompress](#)
 - * Added C source array format output (`-cout`) (optional variable name argument `-varname=`)
 - * Added C source array format input (`-cin`) (experimental)
 - * Added block style rle compression and decompression mode: `--alg=rle`
 - * Fixed compression errors when input size was larger than 64k
- Docs
 - Added [Supported Consoles & Cross Compiling](#) section
 - Various doc updates and improvements

12.1.9 GBDK-2020 4.0.4

2021/06

- Library
 - Support SDCC INITIALIZER area (SDCC ~12207+)
 - Added [get_vram_byte\(\)](#) / [get_win_tile_xy\(\)](#) / [get_bkg_tile_xy\(\)](#)
 - Added [set_tile_data\(\)](#)
 - Fixed SGB detection
 - Fixed broken [get_tiles\(\)](#) / [set_tiles\(\)](#)
 - Fixed broken token handling in [gb_decompress_sprite_data\(\)](#) / [gb_decompress_bkg_data\(\)](#) / [gb_decompress_win_data\(\)](#)
 - Changed all headers to use standard `stdint.h` types (ex: `uint8_t` instead of `UINT8/UBYTE`)
 - Made `sample.h`, `cgb.h` and `sgb.h` independent from `gb.h`
- Examples
 - Added project using a `.lk` linkerfile
 - Changed all examples to use standard `stdint.h` types
 - Moved `banks_farptr` and `banks_new` examples to "broken" due to SDCC changes
- Toolchain / Utilities
 - `png2mtspr`
 - * Added option to change default value for sprite property/attributes in (allows CGB palette, BG/WIN priority, etc).

- * Improved: Turn off suppression of "blank" metasprite frames (composed of entirely transparent sprites)
 - * Fixed endless loop for png files taller than 255 pixels
- bankpack
 - * Fixed -yt mbc specifier to also accept Decimal
 - * Improved: bank ID can be used in same file it is declared. Requires SDCC 12238+ with -n option to defer symbol resolution to link time.
- gbcompress
 - * Added C source input (experimental) and output
 - * Added size #defines
- lcc
 - * Added -no-libs and -no-crt options
 - * Added support for .lk linker files (useful when number of files on lcc command line exceeds max size on windows)
 - * Added support for converting .ihx to .gb
 - * Added rewrite .o files -> .rel for linking when called with -autobank and -Wb-ext=.rel
 - * Workaround [makebin](#) -Wl-yp formatting segfault
- Docs
 - Improved utility_png2mtspr documentation
 - Various doc updates and improvements

12.1.10 GBDK-2020 4.0.3

2021/03

- Library
 - Added [set_vram_byte\(\)](#)
 - Added [set_bkg_tile_xy\(\)](#) / [set_win_tile_xy\(\)](#)
 - Added [get_bkg_xy_addr\(\)](#) / [get_win_xy_addr\(\)](#)
 - Added [set_bkg_submap\(\)](#) / [set_win_submap\(\)](#)
 - Added metasprite api support
 - Added gb_decompress support
 - Added [calloc](#) / [malloc](#) / [realloc](#) / [free](#) and generic [memmove](#)
 - Improved [printf\(\)](#): ignore %0 padding and %1-9 width specifier instead of not printing, support upper case X
 - Fixed [line\(\)](#): handle drawing when x1 is less than x2
- Examples
 - Added large_map: showing how to use [set_bkg_submap\(\)](#)
 - Added scroller: showing use of [get_bkg_xy_addr\(\)](#), [set_bkg_tile_xy\(\)](#) and [set_vram_byte](#)
 - Added gbdecompress: de-compressing tile data into vram
 - Added metasprites: show creating a large sprite with the new metasprite api
 - Added template projects
 - Fixed build issue with banks_autobank example
 - Improved sgb_border
- Toolchain / Utilities
 - Added [utility_gbcompress](#) utility
 - Added utility_png2mtspr metasprite utility
- Docs
 - Added extensive documentation (some of which is imported and updated from the old gbdk docs)
 - Added PDF version of docs

12.1.11 GBDK-2020 4.0.2

2021/01/17

- Includes SDCC snapshot build version 12016 (has a fix for duplicate debug symbols generated from inlined header functions which GBDK 4.0+ uses)
- Updated documentation
- Library was improved
 - Linking with stdio.h does not require that much ROM now
 - Default font is changed to the smaller one (102 characters), that leaves space for user tiles
 - Fixed broken support for multiplying longs
 - memset/memcpy minor enhancements
 - safer copy-to-VRAM functions
 - loading of 1bit data fixed, also now it is possible to specify pixel color
 - Improved code generation for the GBDK Library with SDCC switch on by default: `--max-allocs-per-node 50000`
 - fixed wrong parameter offsets in `hiramcpy()` (broken `ram_function` example)
 - Multiple minor improvements
- New backpack feature, allows automatic bank allocation for data and code, see `banks_autobank` example, feature is in beta state, use with care
- Lcc improvements
 - Fixed option to specify alternate base addresses for `shadow_OAM`, etc
- Examples: Added `bgb` debug example

12.1.12 GBDK-2020 4.0.1

2020/11/14

- Updated API documentation
- IHX is checked for correctness before the makebin stage. That allows to warn about overwriting the same ROM addresses (SDCC toolchain does not check this anymore).
- Library was improved
 - `set_*_tiles()` now wrap maps around horizontal and vertical boundaries correctly
 - new `fill_*_rect()` functions to clear rectangle areas
 - runtime initialization code now does not initialize whole WRAM with zeros anymore, that allows BGB to raise exceptions when code tries to read WRAM that was not written before.
 - enhanced SGB support
 - * `joypad_init()` / `joypad_ex()` support for multiple joypads
 - * SGB border example
 - `_current_bank` variable is updated when using bank switching macros
 - Reorganized examples: each example is in separate folder now, that simplifies understanding.
 - Lcc improvements
 - * Fix -S flag
 - * Fix default stack location from 0xDEFF to 0xE000 (end of WRAM1)
 - * Fix cleanup of `.adb` files with `-Wf-debug` flag
 - * Fix output not working if target is `-o some_filename.ihx`

12.1.13 GBDK-2020 4.0

2020/10/01

- GBDK now requires SDCC 4.0.3 or higher, that has fully working toolchain. Old link-gbz80 linker is not used anymore, sdldgb and makebin are used to link objects and produce binary roms; maccor tool is no longer needed either
 - SDCC 4.0.3 has much better code generator which produces smaller and faster code. Code is twice faster
 - SOURCE LEVEL DEBUGGING is possible now! Native toolchain produces *.CDB files that contain detailed debug info. Look for EMULICIOUS extension for vs.code. It supports breakpoints, watches, inspection of local variables, and more!
 - SDCC 4.0.4 has fixed RGBDS support; library is not updated to support that in full yet, but it is possible to assemble and link code emitted by SDCC with RGBDS
 - New banked trampolines are used, they are faster and smaller
 - New (old) initialization for non-constant arrays do NOT require 5 times larger rom space than initialized array itself, SDCC even tries to compress the data
- Library was improved
 - itoa/lttoa functions were rewritten, div/mod is not required now which is about 10 times faster
 - sprite functions are inline now, which is faster up to 12 times and produces the same or smaller code; .OAM symbol is renamed into _shadow_OAM that allows accessing shadow OAM directly from C code
 - interrupt handling was revised, it is now possible to make dedicated ISR's, that is important for time-sensitive handlers such as HBlank.
 - printf/sprintf were rewritten and splitted, print functions are twice faster now and also require less rom space if you use [sprintf\(\)](#) only, say, in bgb_emu.h
 - crash_handler.h - crash handler that allows to detect problems with ROMs after they are being released (adapted handler, originally written by ISSOtm)
 - improved and fixed string.h
 - many other improvements and fixes - thanks to all contributors!
- Revised examples
- Improved linux support
- Lcc has been updated
 - it works with the latest version of sdcc
 - quoted paths with spaces are working now

12.1.14 GBDK-2020 3.2

2020/06/05

- Fixed OAM initialization that was causing a bad access to VRAM
- Interrupt handlers now wait for lcd controller mode 0 or 1 by default to prevent access to inaccessible VRAM in several functions (like set_bkg_tiles)
- Several optimizations here and there

12.1.15 GBDK-2020 3.1.1

2020/05/17

- Fixed issues with libgcc_s_dw2-1.dll

12.1.16 GBDK-2020 3.1

2020/05/16

- Banked functions are working! The patcher is fully integrated in link-gbz80, no extra tools are needed. It is based on Toxa's work
 - Check this post for more info
 - Check the examples/gb/banked code for basic usage
- Behavior formerly enabled by `USE_SFR_FOR_REG` is on by default now (no need to specify it, it isn't a tested `#ifdef` anymore). check here why: <https://gbdev.gg8.se/forums/viewtopic.php?id=697>
- Fixed examples that were not compiling in the previous version and some improvements in a few of them. Removed all warnings caused by changing to the new SDCC
- Fixed bug in lcc that was causing some files in the temp folder not being deleted
- Removed as-gbz80 (the lib is now compiled with sdasgb thanks to this workaround) <https://github.com/gbdk-2020/gbdk-2020/commit/d2caafa4a66eb08998a14b258cb66af041a0e5c8>
- Profile support with bgb emulator
 - Basic support including `<gb/bgb_emu.h>` and using the macros `BGB_PROFILE_BEGIN` and `BGB_PROFILE_END`. More info in this post <https://gbdev.gg8.se/forums/viewtopic.php?id=703>
 - For full profiling check this repo and this post https://github.com/untoxa/bgb_profiling_toolkit/blob/master/readme.md <https://gbdev.gg8.se/forums/viewtopic.php?id=710>

12.1.17 GBDK-2020 3.0.1

2020/04/12

- Updated SDCC to v4.0
- Updated LCC to work with the new compiler

12.1.18 GBDK-2020 3.0

2020/04/12

- Initial GBDK-2020 release Updated SDCC to v4.0 The new linker is not working so the old version is still there There is an issue with sdagb compiling drawing.s (the JP in line 32 after ".org .MODE_TABLE+4*.G_MODE" it's writing more than 4 bytes invading some addresses required by input.s:41) Because of this, all .s files in libc have been assembled with the old as-gbz80 and that's why it is still included

12.2 Historical GBDK Release Notes

12.2.1 GBDK 2.96

17 April, 2000 Many changes.

- Code generated is now much more reliable and passes all of sdcc's regression suite.
- Added support for large sets of local variables (>127 bytes).
- Added full 32 bit long support.
- Still no floating pt support.

12.2.2 GBDK 2.95-3

19th August, 2000

- Stopped lcc with sdcc from leaking .cdb files all across /tmp.
- Optimised < and > for 16 bit variables.
- Added a new lexer to sdcc. Compiling files with large initialised arrays takes 31% of the time (well, at least samptest.c does :)

This is an experimental release for those who feel keen. The main change is a new lexer (the first part in the compilation process which recognises words and symbols like '!= ' and 'char' and turns them into a token number) which speeds up compilation of large initialised arrays like tile data by a factor of three. Please report any bugs that show up - this is a big change.

I have also included a 'minimal' release for win32 users which omits the documentation, library sources, and examples. If this is useful I will keep doing it.

12.2.3 GBDK 2.95-2

5th August, 2000 Just a small update. From the README:

- Added model switching support –model-medium uses near (16 bit) pointers for data, and banked calls for anything not declared as 'nonbanked' –model-small uses near (16 bit) pointers for data and calls. Nothing uses banked calls. 'nonbanked' functions are still placed in HOME. Libraries are under lib/medium and lib/small.
- Added the gbdk version to 'sdcc –version'
- Changed the ways globals are exported, reducing the amount of extra junk linked in.
- Turned on the optimisations in flex. Large constant arrays like tile data should compile a bit faster.

12.2.4 GBDK 2.95

22nd July, 2000

- Fixed 'a << c' for c = [9..15]
- no\$gmb doesn't support labels of > 32 chars. The linker now trims all labels to 31 chars long.
- Fixed wait_vbl for the case where you miss a vbl
- Fixed + and - for any type where sizeof == 2 and one of the terms was on the stack. This includes pointers and ints. Fixes the text output bug in the examples. Should be faster now as well. Note that + and - for longs is still broken.
- Fixed the missing */ in gb.h
- Added basic far function support. Currently only works for isas and rgbasm. See examples/gb/far/*
- bc is now only pushed if the function uses it. i.e. something like: int silly(int i) { return i; } will not have the push bc; pop bc around it.
- Better rgbasm support. Basically:
 - o Use "sdcc -mgbz80 --asm=rgbds file.c" for each file.c
 - o Use "sdcc -mgbz80 --asm=rgbds crt0.o gbz80.lib gb.lib file1.o file2.o..."

to link everything together. The .lib files are generated using astorgb.pl and sdcc to turn the gbdk libraries into something rgbds compatible. The libraries are *not* fully tested. Trust nothing. But give it a go :)

- Ran a spell checker across the README and ChangeLog

This is a recommended upgrade. Some of the big features are:

Decent rgbds support. All the libraries and most of the examples can now compile with rgbds as the assembler. Banked function support. It is now easier to break the 32k barrier from within C. Functions can live in and be called transparently from any bank. Only works with rgbds Fixed some decent bugs with RSH, LSH, and a nasty bug with + and - for int's and pointers. Various optimisations in the code generator.

7th July, 2000 Information on float and long support. Someone asked about the state of float/long support recently. Heres my reply:

long support is partly there, as is float support. The compiler will correctly recognise the long and float keywords, and will generate the code for most basic ops (+, -, &, | etc) for longs correctly and will generate the function calls for floats and hard long operations (*, /, %) correctly. However it wont generate float constants in the correct format, nor will it 'return' a long or float - gbdk doesn't yet support returning types of 4 bytes. Unfortunately its not going to make it into 2.95 as there's too much else to do, but I should be able to complete long support for 2.96

12.2.5 GBDK 2.94

7th May, 2000 Many fixes - see the README for more.

7th May - Library documentation up. A good size part of the libraries that go with gbdk have been documented - follow the HTML link above to have a look. Thanks to quang for a good chunk of the gb.h documentation. Please report any errors :)

- Fixed #define BLAH 7 // Unterminated ' error in sdccpp
 - Fixed SCY_REG += 2, SCY_REG -= 5 (add and subtract in indirect space) as they were both quite broken.
 - externs and static's now work as expected.
 - You can now specify which bank code should be put into using a #pragma e.g: #pragma bank=HOME Under rgbds and asxxxx putting code in the HOME bank will force the code into bank 0 - useful for library functions. The most recent #pragma bank= will be the one used for the whole file.
 - Fixed an interesting bug in the caching of lit addresses
 - Added support for accessing high registers directly using the 'sfr' directive. See libc/gb/sfr.s and gb/hardware.h for an example. It should be possible with a bit of work to make high ram directly usable by the compiler; at the moment it is experimental. You can test sfr's by enabling USE_SFR_FOR_↵ REG=1
 - Added remove_VBL etc functions.
 - Documented the libs - see the gbdk-doc tarball distributed seperatly.
 - Two dimensional arrays seem to be broken.

12.2.6 GBDK 2.93

6th April, 2000 From the README

- Added multi-bank support into the compiler - The old -Wf-boxx and -Wf-baxx options now work
- Has preliminary support for generating rgbds and ISAS compatible assembler. Try -W-asm=rgbds or -W-asm=isas. The ISAS code is untested as I dont have access to the real assembler.
- RSH is fixed
- AND is fixed
- The missing parts of 2.1.0's libs are there. Note: They are untested.
- The dscan demo now fully works (with a hack :)
- There is a bug with cached computed values which are later used as pointers. When the value is first used as a BYTE arg, then later as a pointer the pointer fails as the high byte was never computed and is now missing. A temporary fix is to declare something appropriate as 'volatile' to stop the value being cached. See dscan.c/bombs() for an example.

12.2.7 GBDK 2.92-2 for win32

26th March, 2000 This is a maintenance release for win32 which fixes some of the niggly install problems, especially:

- win32 only. Takes care of some of the install bugs, including:
 - Now auto detects where it is installed. This can be overridden using set GBDKDIR=...
 - Problems with the installer (now uses WinZip)
 - Problems with the temp directory Now scans TMP, TEMP, TMPDIR and finally c: tmp
 - cygwin1.dll and 'make' are no longer required gbdk is now built using mingw32 which is win32 native make.bat is automagically generated from the Makefile
 - I've reverted to using WORD for signed 16 bit etc. GBDK_2_COMPAT is no longer required.

WORDS are now back to signed. GBDK_2_COMPAT is no longer needed. Temporary files are created in TMP, TEMP, or TMPDIR instead of c: tmp The installer is no more as it's not needed. There is a WinZip wrapped version for those with the extra bandwidth :). gbdk autodetects where it is installed - no more environment variables. cygwin1.dll and make are no longer required - gbdk is now compiled with mingw32.

See the ChangeLog section in the README for more information.

21st March, 2000 Problems with the installer. It seems that the demo of InstallVISE has an unreasonably short time limit. I had planned to use the demo until the license key came through, but there's no sign of the key yet and the 3 day evaluation is up. If anyone knows of a free Windows installer with the ability to modify environment variables, please contact me. I hear that temporarily setting you clock back to the 15th works...

18th March, 2000 libc5 version available / "Error creating temp file" Thanks to Rodrigo Couto there is now a Linux/libc5 version of gbdk3-2.92 available - follow the download link above. At least it will be there when the main sourceforge site comes back up... Also some people have reported a bug where the compiler reports '*** Error creating temp file'. Try typing "mkdir c: tmp" from a DOS prompt and see if that helps.

12.2.8 GBDK 2.92

8th March, 2000 Better than 2.91 :). Can now be installed anywhere. All the demos work. See the README for more.

- All the examples now work (with a little bit of patching :)
 - Fixed problem with registers being cached instead of being marked volatile.
 - More register packing - should be a bit faster.
 - You can now install somewhere except c: gbdk | /usr/lib/gbdk
 - Arrays initialised with constant addresses a'la galaxy.c now work.
 - Fixed minor bug with 104\$: labels in as.
 - Up to 167d/s...

12.2.9 GBDK 2.91

27th Feb, 2000 Better than 2.90 and includes Linux, win32 and a source tar ball. Some notes:

Read the README first Linux users need libgc-4 or above. Debian users try apt-get install libgc5. All the types have changed. Again, please read the README first. I prefer release early, release often. The idea is to get the bugs out there so that they can be squashed quickly. I've split up the libs so that they can be used on other platforms and so that the libs can be updated without updating the compiler. One side effect is that gb specific files have been shifted into their own directory i.e. gb.h is now gb/gb.h.

23rd Feb, 2000 First release of gbdk/sdcc. This is an early release - the only binary is for Linux and the source is only available through cvs. If your interested in the source, have a look at the cvs repository gbdk-support first, which will download all the rest of the code. Alternatively, look at gbdk-support and gbdk-lib at cvs.gbdk.sourceforge.net and sdcc at cvs.sdcc.sourceforge.net. I will be working on binaries for Win32 and a source tar ball soon. Please report any bugs through the bugs link above.

31st Jan, 2000 Added Dermot's far pointer spec. It's mainly here for comment. If sdcc is ported to the Gameboy then I will be looking for some way to do far calls.

8th Jan, 2000 Moved over to sourceforge.net. Thanks must go to David Pfeffer for gbdk's previous resting place, www.gbdev.org. The transition is not complete, but cvs and web have been shifted. Note that the cvs download instructions are stale - you should now look to cvs.gbdk.sourceforge.net. I am currently working on porting sdcc over to the Z80. David Nathan is looking at porting it to the GB.

6th Jan, 2000 Icehawk wrote "I did write some rumble pack routines. Just make sure to remind people to add -Wl-yt0x1C or -Wl-yt0x1D or -Wl-yt0x1E depending on sram and battery usage. Find the routines on my site (as usual). =)"

18th Oct, 1999 Bug tracking / FAQ up. Try the link on the left to report any bugs with GBDK. It's also the first place to look if your having problems.

12.2.10 GBDK 2.1.5

17th Oct, 1999

The compiler is the same, but some of the libraries have been improved. [memset\(\)](#) and [memcpy\(\)](#) are much faster, [malloc\(\)](#) is fixed, and a high speed fixed block alternative [malloc\(\)](#) was added.

12.2.11 GBDK 2.0b11 (DOS binary only) - 24 November 1997

- Fixed another bug in code generation, that could happen when performing logical operations on 1-byte variables.

12.2.12 GBDK 2.0b10 (DOS binary only) - 6 November 1997

- Fixed a nasty bug in code generation, that could happen when performing arithmetic operations on 1-byte variables.
- Changed the name of some files of the gb-dtmf example so that it compiles on DOS.

12.2.13 GBDK 2.0b9 (DOS binary only)

- Several bug fixes in the compiler and in the libraries.

12.2.14 GBDK 2.0b8 (DOS binary only)

- Limited all file names to 8 characters to solve problems on DOS.
- Added communication routines that enable to send data through the link port of the GameBoy. Unfortunately, these routines do not always work; so use them with care until the next GBDK release.
- Added the comm.c example which illustrates how to use communication routines.
- It is possible to specify the name of the program (to be written in the image header) at link time using the -Wl-yn="XXX" flag (where X is the name of the program, which can contain up to 16 characters in quotes, including spaces; on Unix, depending on your shell, you must add backslashes before quotes and spaces like in -Wl-yn="My\ Game").
- Several bug fixes in the compiler.

12.2.15 GBDK 2.0b7 (DOS binary only)

- GBDK now uses a pre-release of lcc 4.1 (DOS binary only), that fixes a couple of problems in code generation.
- A couple of important points have been documented in the GBDK Programming Guidelines and Known Problems sections.
- Several improvements and optimizations to the code generator.

12.2.16 GBDK 2.0b6

- Added a peephole optimizer (with few rules at the moment).
- Changed the name of the hardware registers to match the "official" names.
- Added support for copying complete functions to RAM or HIRAM ([memcpy\(\)](#) and [hramcpy\(\)](#) functions). The compiler now automatically generates two symbol for the start and the end of each function, named start_X and end_X (where X is the name of the function). This enables to calculate the length of a function when copying it to RAM.
- Added the ram_fn.c example which illustrates how to copy functions to RAM and HIRAM.
- Added support for installing IRQ handlers.
- Added the irq.c example which illustrates how to install IRQ handlers.
- Added RAM banks support (switch_ram_bank() function). The switch_bank() function has been renamed to switch_rom_bank(). The banks.c example has been updated. The flags for generating multiple bank images have been modified.
- It is possible to set the sprite ram location at link time using the -Wl-g.OAM=# flag (where # is the address of the sprite ram). The sprite ram address must begin at an address multiple of 0x100, and is 0xA0 bytes long.

12.2.17 GBDK 2.0b5

- New documentation (not finished yet).
- Fixed a bug that could generate wrong code in switch statements.
- Fixed a bug in int comparison.
- Added a DTMF program written by Osamu Ohashi.
- Added a game (Deep Scan) written by a friend of Osamu.
- Modified the [delay\(\)](#) function so that it takes a long parameter. It can be used to wait between 1 and 65536 milliseconds (0 = 65536). The pause() function has been removed.

12.2.18 GBDK 2.0b4

- Fixed a bug that could generate wrong code when using hexadecimal constants.
- A new example (galaxy.c) has been added. It is the C version of the space.s example. sprite.c has been removed.
- Most of the libraries have been split into small modules for reducing final code size.

12.2.19 GBDK 2.0b3

- GBDK can generate multiple-banks images, i.e. images greater than 32kB (see the banks example).
- It is possible to set the stack pointer at link time using the -Wl-g.STACK=# flag (where # is the address of the stack pointer). Several functions (e.g. show_bkg()) have been changed into macros (e.g. SHOW_BKG). The [delay\(\)](#) function waits exactly 1 millisecond, and the pause() waits 256 milliseconds. Linking with the standard libraries is no more required. The lib/gb.lib (lib\gb.lib on DOS) text file contains a list of modules in which to look for undefined symbols. The linker will parse this file, and link your code with the required modules only. The stdio library has been split in several object files, and only necessary modules will be added to your code, thus reducing its size. The GBDK distribution can be located anywhere in your system if you use the -Wo-lccdir=GBDK-DIR flag when invoking lcc. Bug fixes.

12.2.20 GBDK 2.0b2

- Lots of bug fixes.
- GBDK has to be in the \GBDK-2.0 directory on DOS machines.

12.2.21 GBDK 2.0b1

- The code generator has been completely rewritten with the new version of lcc. It produces much smaller and more efficient code. The size of the code is generally between 20 and 50% smaller. A number of small optimizations are still to be done.
- The size of basic types has been changed:
 - An int is 8 bits.
 - A long is 16 bits.
- This change was required for the code generator to produce better code, because the Z80 is actually an 8-bit processor.
- The linker generates the complement checksum correctly now.
- The libraries and example programs have been modified for the new code generator.

12.2.22 GBDK 1.1

- Removed Xloadimage from the GBDK distribution. It is now available as a separate archive.
- A compiled DOS version is now available (cross-compiled on my Sun Workstation!).
- The libraries and the example programs have been improved.
- The make script has been improved. Compiling on UNIX should be easier.
- Many bugfixes.

12.2.23 GBDK 1.0-1 1996

13 Toolchain settings

13.1 lcc settings

```
./lcc [ option | file ]...
    except for -l, options are processed left-to-right before files
    unrecognized options are taken to be linker options
-c                compile only
-debug           Turns on --debug for compiler, -y (.cdb), -j (.noi), -w (wide .map format) for linker
                  -Wa-l (assembler .lst), -Wl-u (.lst -> .rst address update)
-Dname=def       define the preprocessor symbol 'name'
-E              only run preprocessor on named .c and .h files files -> stdout
--save-preproc   Use with -E for output to *.i files instead of stdout
-help or -?      print this message
-Idir            add 'dir' to the beginning of the list of #include directories
-K              don't run ihxcheck test on linker ihx output
-lx             search library 'x'
-m              select port and platform: "-m[port]:[plat]" ports:sm83,z80,mos6502
                  plats:ap,duck,gb,sms,gg,nes
-N              do not search the standard directories for #include files
-no-crt          do not auto-include the gbdk crt0.o runtime in linker list
-no-libs         do not auto-include the gbdk libs in linker list
-o file          leave the output in 'file'
-S              compile to assembly language
-autobank        auto-assign banks set to 255 (bankpack)
-tempdir=dir     place temporary files in 'dir/'
-Uname           undefine the preprocessor symbol 'name'
-v              show commands as they are executed; 2nd -v suppresses execution
-Woarg           specify system-specific 'arg'
-W[pfablimnc]arg pass 'arg' to the (p)reprocessor, (f)compiler, (a)sembler, (b)bankpack,
                  (l)linker, (i)hxcheck, (m)makebin, (n)makenes, (c)makecom
```

13.2 sdcc settings

```
SDCC : z80/sm83/mos6502/mos65c02 TD- 4.5.1 #15267 (Linux)
published under GNU General Public License (GPL)
Usage : sdcc [options] filename
Options :-
General options:
    --help          Display this help
    -v --version    Display sdcc's version
```

```

--verbose          Trace calls to the preprocessor, assembler, and linker
-V               Execute verbosely. Show sub commands as they are run
-d               Output list of macro definitions in effect. Use with -E
-D               Define macro as in -Dmacro
-I               Add to the include (*.h) path, as in -Ipath
-A
-U               Undefined macro as in -Umacro
-M               Preprocessor option
-W               Pass through options to the pre-processor (p), assembler (a) or linker (l)
--include         Pre-include a file during pre-processing
-E --preprocessonly Preprocess only, do not compile
--syntax-only     Parse and verify syntax only, do not compile
-S               Compile only; do not assemble or link
-c --compile-only  Compile and assemble, but do not link
--c1mode          Act in c1 mode. The standard input is preprocessed code, the output is assembly
                  code.
-o               Place the output into the given path resp. file
-x               Optional file type override (c, c-header or none), valid until the next -x
--print-search-dirs display the directories in the compiler's search path
--vc             messages are compatible with Micro$oft visual studio
--use-stdout      send errors to stdout instead of stderr
--nostdlib        Do not include the standard library directory in the search path
--nostdinc        Do not include the standard include directory in the search path
--less-pedantic   Disable some of the more pedantic warnings
--disable-warning <nnnn> Disable specific warning
--Werror          Treat the warnings as errors
--debug           Enable debugging symbol output
--cyclomatic      Display complexity of compiled functions
--std             Determine the language standard (c90, c99, c11, c23, c2y, sdcc89 etc.)
--fdollars-in-identifiers Permit '$' as an identifier character
--fsigned-char     Make "char" signed by default
--use-non-free     Search / include non-free licensed libraries and header files

Code generation options:
-m               Set the port to use e.g. -mz80.
-p               Select port specific processor e.g. -mpic14 -pl6f84
--stack-auto      Stack automatic variables
--xstack          Use external stack
--int-long-reent  Use reentrant calls on the int and long support functions
--float-reent     Use reentrant calls on the float support functions
--xram-movc       Use movc instead of movx to read xram (xdata)
--callee-saves   <func[,func,...]> Cause the called function to save registers instead of the
                  caller
--fomit-frame-pointer Leave out the frame pointer.
--all-callee-saves callee will always save registers used
--stack-probe     insert call to function __stack_probe at each function prologue
--no-xinit-opt    don't memcpy initialized xram from code
--no-c-code-in-asm don't include c-code as comments in the asm file
--no-peep-comments don't include peephole optimizer comments
--codeseg         <name> use this name for the code segment
--constseg        <name> use this name for the const segment
--dataseg         <name> use this name for the data segment

Optimization options:
--opt-code-speed  Optimize for code speed rather than size
--opt-code-size   Optimize for code size rather than speed
--max-allocs-per-node Maximum number of register assignments considered at each node of the tree
                  decomposition
--no-reg-params   On some ports, disable passing some parameters in registers
--nostdlibcall    Disable optimization of calls to standard library
--nooverlay       Disable overlaying leaf function auto variables
--nogcse          Disable the GCSE optimisation
--nolospre        Disable lospre
--nogenconstprop  Disable generalized constant propagation
--nolabelopt      Disable label optimisation
--noinvariant     Disable optimisation of invariants
--noinduction     Disable loop variable induction
--noloopreverse   Disable the loop reverse optimisation
--no-peep         Disable the peephole assembly file optimisation
--peep-asm        Enable peephole optimization on inline assembly
--peep-return     Enable peephole optimization for return instructions
--no-peep-return  Disable peephole optimization for return instructions
--peep-file       <file> use this extra peephole file
--allow-unsafe-read Allow optimizations to read any memory location anytime

Internal debugging options:
--dump-ast        Dump front-end AST before generating i-code
--dump-i-code     Dump the i-code structure at all stages
--dump-graphs     Dump graphs (control-flow, conflict, etc)
--i-code-in-asm   Include i-code as comments in the asm file
--fverbose-asm    Include code generator comments in the asm output

Linker options:
-l               Include the given library in the link
-L               Add the next field to the library search path
--lib-path        <path> use this path to search for libraries
--out-fmt-ihx     Output in Intel hex format
--out-fmt-s19     Output in S19 hex format
--xram-loc        <nnnn> External Ram start location
--xram-size       <nnnn> External Ram size
--iram-size       <nnnn> Internal Ram size

```

```

--xstack-loc      <nnnn> External Stack start location
--code-loc        <nnnn> Code Segment Location
--code-size       <nnnn> Code Segment size
--stack-loc       <nnnn> Stack pointer initial value
--data-loc        <nnnn> Direct data start location
--idata-loc
--no-optsdcc-in-asm Do not emit .optsdcc in asm
Special options for the z80 port:
--callee-saves-bc   Force a called function to always save BC
--portmode=        Determine PORT I/O mode (z80/z180)
-bo                <num> use code bank <num>
-ba                <num> use data bank <num>
--asm=             Define assembler name (rgbds/asxxxx/isas/z80asm/gas)
--codeseg          <name> use this name for the code segment
--constseg         <name> use this name for the const segment
--dataseg          <name> use this name for the data segment
--no-std-crt0       Do not link default crt0.rel
--reserve-regs-iy   Do not use IY (incompatible with --fomit-frame-pointer)
--fno-omit-frame-pointer Do not omit frame pointer
--emit-externs      Emit externs list in generated asm
--legacy-banking    Use legacy method to call banked functions
--nmios-z80         Generate workaround for NMOS Z80 when saving IFF2
--sdcccall          Set ABI version for default calling convention
--allow-undocumented-instructions Allow use of undocumented instructions
Special options for the sm83 port:
-bo                <num> use code bank <num>
-ba                <num> use data bank <num>
--asm=             Define assembler name (rgbds/asxxxx/isas/z80asm/gas)
--callee-saves-bc   Force a called function to always save BC
--codeseg          <name> use this name for the code segment
--constseg         <name> use this name for the const segment
--dataseg          <name> use this name for the data segment
--no-std-crt0       Do not link default crt0.rel
--legacy-banking    Use legacy method to call banked functions
--sdcccall          Set ABI version for default calling convention
Special options for the mos6502 port:
--model-small       8-bit address space for data
--model-large       16-bit address space for data (default)
--no-zp-spill       place register spills in 16-bit address space
--no-std-crt0       Do not link default crt0.rel
Special options for the mos65c02 port:
--model-small       8-bit address space for data
--model-large       16-bit address space for data (default)
--no-zp-spill       place register spills in 16-bit address space
--no-std-crt0       Do not link default crt0.rel

```

13.3 sdasgb settings

```

sdas Assembler V02.00 + NoICE + SDCC mods (GameBoy)
Copyright (C) 2012 Alan R. Baldwin
This program comes with ABSOLUTELY NO WARRANTY.
Usage: [-Options] [-Option with arg] file
Usage: [-Options] [-Option with arg] outfile file1 [file2 ...]
-h or NO ARGUMENTS Show this help list
Input:
-I Add the named directory to the include file
  search path. This option may be used more than once.
  Directories are searched in the order given.
Output:
-l Create list file/outfile[.lst]
-o Create object file/outfile[.rel]
-s Create symbol file/outfile[.sym]
Listing:
-d Decimal listing
-q Octal listing
-x Hex listing (default)
-b Display .define substitutions in listing
-bb and display without .define substitutions
-c Disable instruction cycle count in listing
-f Flag relocatable references by ' in listing file
-ff Flag relocatable references by mode in listing file
-p Disable automatic listing pagination
-u Disable .list/.nlist processing
-w Wide listing format for symbol table
Assembly:
-v Enable out of range signed / unsigned errors
Symbols:
-a All user symbols made global
-g Undefined symbols made global
-N Don't resolve global assigned value symbols
-z Disable case sensitivity for symbols
Debugging:
-j Enable NoICE Debug Symbols
-y Enable SDCC Debug Symbols

```

13.4 sdasz80 settings

```

sdas Assembler V02.00 + NoICE + SDCC mods  (Zilog Z80 / Hitachi HD64180 / ZX-Next / eZ80 / R800)
Copyright (C) 2012 Alan R. Baldwin
This program comes with ABSOLUTELY NO WARRANTY.
Usage: [-Options] [-Option with arg] file
Usage: [-Options] [-Option with arg] outfile file1 [file2 ...]
-h or NO ARGUMENTS Show this help list

Input:
-I Add the named directory to the include file
  search path. This option may be used more than once.
  Directories are searched in the order given.

Output:
-l Create list file/outfile[.lst]
-o Create object file/outfile[.rel]
-s Create symbol file/outfile[.sym]

Listing:
-d Decimal listing
-q Octal listing
-x Hex listing (default)
-b Display .define substitutions in listing
-bb and display without .define substitutions
-c Disable instruction cycle count in listing
-f Flag relocatable references by ` in listing file
-ff Flag relocatable references by mode in listing file
-p Disable automatic listing pagination
-u Disable .list/.nlist processing
-w Wide listing format for symbol table

Assembly:
-v Enable out of range signed / unsigned errors

Symbols:
-a All user symbols made global
-g Undefined symbols made global
-N Don't resolve global assigned value symbols
-z Disable case sensitivity for symbols

Debugging:
-j Enable NoICE Debug Symbols
-y Enable SDCC Debug Symbols

```

13.5 sdas6500 settings

```

sdas Assembler V02.00 + NoICE + SDCC mods  (Rockwell 6502/6510/65C02)
Copyright (C) 2012 Alan R. Baldwin
This program comes with ABSOLUTELY NO WARRANTY.
Usage: [-Options] [-Option with arg] file
Usage: [-Options] [-Option with arg] outfile file1 [file2 ...]
-h or NO ARGUMENTS Show this help list

Input:
-I Add the named directory to the include file
  search path. This option may be used more than once.
  Directories are searched in the order given.

Output:
-l Create list file/outfile[.lst]
-o Create object file/outfile[.rel]
-s Create symbol file/outfile[.sym]

Listing:
-d Decimal listing
-q Octal listing
-x Hex listing (default)
-b Display .define substitutions in listing
-bb and display without .define substitutions
-c Disable instruction cycle count in listing
-f Flag relocatable references by ` in listing file
-ff Flag relocatable references by mode in listing file
-p Disable automatic listing pagination
-u Disable .list/.nlist processing
-w Wide listing format for symbol table

Assembly:
-v Enable out of range signed / unsigned errors

Symbols:
-a All user symbols made global
-g Undefined symbols made global
-N Don't resolve global assigned value symbols
-z Disable case sensitivity for symbols

Debugging:
-j Enable NoICE Debug Symbols
-y Enable SDCC Debug Symbols

```

13.6 bankpack settings

```

bankalloc [options] objfile1 objfile2 etc
Use: Read .o files and auto-assign areas with bank=255.
Typically called by Lcc compiler driver before linker.
Options

```

```

-h                : Show this help
-lkin=<file>      : Load object files specified in linker file <file>
-lkout=<file>     : Write list of object files out to linker file <file>
-yt<mbctype>     : Set MBC type per ROM byte 149 in Decimal or Hex (0xNN)
                  ([see pandocs](https://gbdev.io/pandocs/The_Cartridge_Header.html#0147---cartridge-type))
-mbc=N           : Similar to -yt, but sets MBC type directly to N instead
                  of by interpreting ROM byte 149
                  mbc1 will exclude banks {0x20,0x40,0x60} max=127,
                  mbc2 max=15, mbc3 max=127, mbc5 max=255 (not 511!)
-min=N           : Min assigned ROM bank is N (default 1)
-max=N           : Max assigned ROM bank is N, error if exceeded
-ext=<.ext>       : Write files out with <.ext> instead of source extension
-path=<path>      : Write files out to <path> (<path> *MUST* already exist)
-sym=<prefix>     : Add symbols starting with <prefix> to match + update list
                  Default entry is "__bank_" (see below)
-cartsiz        : Print min required cart size as "autocartsiz:<NNN>"
-plat=<plat>      : Select platform specific behavior (default:gb) (gb,sms)
-random         : Distribute banks randomly for testing (honors -min/-max)
-reserve=<b:n>    : Reserve N bytes (hex) in bank B (decimal)
                  Ex: -reserve=105:30F reserves 0x30F bytes in bank 105
-banktype=<b:t>   : Set bank B (decimal) to use type T (CODE or LIT). For sms/gg
                  Ex: -banktype=2:LIT sets bank 2 to type LIT
-v              : Verbose output, show assignments
Example: "bankpack -ext=.rel -path=some/newpath/ file1.o file2.o"
Unless -ext or -path specify otherwise, input files are overwritten.
Default MBC type is not set. It *must* be specified by -mbc= or -yt!
The following will have FF and 255 replaced with the assigned bank:
A _CODE_255 size <size> flags <flags> addr <address>
S b_<function name> Def0000FF
S __bank_<const name> Def0000FF
(Above can be made by: const void __at(255) __bank_<const name>;

```

13.7 sldldgb settings

```

sldd Linker V03.00/V05.40 + sldd
Usage: [-Options] [-Option with arg] file
Usage: [-Options] [-Option with arg] outfile file1 [file2 ...]
Startup:
-p      Echo commands to stdout (default)
-n      No echo of commands to stdout
Alternates to Command Line Input:
-c              ASlink » prompt input
-f file[.lk]   Command File input
Libraries:
-k      Library path specification, one per -k
-l      Library file specification, one per -l
Relocation:
-b      area base address = expression
-g      global symbol = expression
-a      (platform) Select platform specific virtual address translation
Map format:
-m      Map output generated as (out)file[.map]
-w      Wide listing format for map file
-x      Hexadecimal (default)
-d      Decimal
-q      Octal
Output:
-i      Intel Hex as (out)file[.ihx]
-s      Motorola S Record as (out)file[.s19]
-j      NoICE Debug output as (out)file[.noi]
-y      SDCDB Debug output as (out)file[.cdb]
List:
-u      Update listing file(s) with link data as file(s)[.rst]
Case Sensitivity:
-z      Disable Case Sensitivity for Symbols
End:
-e      or null line terminates input

```

13.8 sldldz80 settings

```

sldd Linker V03.00/V05.40 + sldd
Usage: [-Options] [-Option with arg] file
Usage: [-Options] [-Option with arg] outfile file1 [file2 ...]
Startup:
-p      Echo commands to stdout (default)
-n      No echo of commands to stdout
Alternates to Command Line Input:
-c              ASlink » prompt input
-f file[.lk]   Command File input
Libraries:
-k      Library path specification, one per -k
-l      Library file specification, one per -l
Relocation:
-b      area base address = expression

```



```

-g    global symbol = expression
-a    (platform) Select platform specific virtual address translation
Map format:
-m    Map output generated as (out)file[.map]
-w    Wide listing format for map file
-x    Hexadecimal (default)
-d    Decimal
-q    Octal
Output:
-i    Intel Hex as (out)file[.ihx]
-s    Motorola S Record as (out)file[.s19]
-j    NoICE Debug output as (out)file[.noi]
-y    SDCDB Debug output as (out)file[.cdb]
List:
-u    Update listing file(s) with link data as file(s)[.rst]
Case Sensitivity:
-z    Disable Case Sensitivity for Symbols
End:
-e    or null line terminates input

```

13.9 sdld6808 settings

```

sdld Linker V03.00/V05.40 + sdld
Usage: [-Options] [-Option with arg] file
Usage: [-Options] [-Option with arg] outfile file1 [file2 ...]
Startup:
-p    Echo commands to stdout (default)
-n    No echo of commands to stdout
Alternates to Command Line Input:
-c    ASlink » prompt input
-f    file[.lk]      Command File input
Libraries:
-k    Library path specification, one per -k
-l    Library file specification, one per -l
Relocation:
-b    area base address = expression
-g    global symbol = expression
-a    (platform) Select platform specific virtual address translation
Map format:
-m    Map output generated as (out)file[.map]
-w    Wide listing format for map file
-x    Hexadecimal (default)
-d    Decimal
-q    Octal
Output:
-i    Intel Hex as (out)file[.ihx]
-s    Motorola S Record as (out)file[.s19]
-j    NoICE Debug output as (out)file[.noi]
-y    SDCDB Debug output as (out)file[.cdb]
List:
-u    Update listing file(s) with link data as file(s)[.rst]
Case Sensitivity:
-z    Disable Case Sensitivity for Symbols
End:
-e    or null line terminates input

```

13.10 ihxcheck settings

```

ihx_check input_file.ihx [options]
Options
-h : Show this help
-e : Treat warnings as errors
Use: Read a .ihx and warn about overlapped areas.
Example: "ihx_check build/MyProject.ihx"

```

13.11 makebin settings

Also see [setting_mbc_and_rom_ram_banks](#)

makebin: convert a Intel IHX file to binary or GameBoy format binary.

Usage: makebin [options] [<in_file> [<out_file>]]

```

Options:
-p    pack mode: the binary file size will be truncated to the last occupied byte
-s romsize    size of the binary file (default: rom banks * 16384)
-Z    generate GameBoy format binary file
-S    generate Sega Master System format binary file
-N    generate Famicom/NES format binary file
-o bytes    skip amount of bytes in binary file
SMS format options (applicable only with -S option):
-xo n    header rom size (0xa-0x2) (default: 0xc)
-xj n    set region code (3-7) (default: 4)
-xv n    version number (0-15) (default: 0)
-yo n    number of rom banks (default: 2) (autosize: A)

```

```

-ya n          number of ram banks (default: 0)
GameBoy format options (applicable only with -Z option):
-yo n          number of rom banks (default: 2) (autosize: A)
-ya n          number of ram banks (default: 0)
-yt n          MBC type (default: no MBC)
-yl n          old licensee code (default: 0x33)
-yk cc         new licensee string (default: 00)
-yn name       cartridge name (default: none)
-yc           GameBoy Color compatible
-yC           GameBoy Color only
-ys           Super GameBoy
-yS           Convert .noi file named like input file to .sym
-yj           set non-Japanese region flag
-yN           do not copy big N validation logo into ROM header
-yp addr=value Set address in ROM to given value (address 0x100-0x1FE)
Arguments:
<in_file>     optional IHX input file, '-' means stdin. (default: stdin)
<out_file>     optional output file, '-' means stdout. (default: stdout)

```

13.12 makecom settings

```

makecom image.rom image.noi output.com
Use: convert a binary .rom file to .msxdos com format.

```

13.13 makesnes settings

```

makesnes: Prepend an iNES header to a binary ROM image file.
Usage: makesnes [options] [<in_file> [<out_file>]]
Options:
-m Mapper number (default: 0)
-n Nametable arrangement:
    0: vertical arrangement / horizontal mirroring
    1: horizontal arrangement / vertical mirroring
-b Battery bit set (default: 0)
-a Alternative nametable layout (default: 0)
Arguments:
<in_file>     optional binary input file, '-' means stdin. (default: stdin)
<out_file>     optional .nes output file, '-' means stdout. (default: stdout)

```

13.14 gbcompress settings

```

gbcompress [options] infile outfile
Use: compress a binary file and write it out.
Options
-h          : Show this help screen
-d          : Decompress (default is compress)
-v          : Verbose output
--cin       : Read input as .c source format (8 bit char ONLY, uses first array found)
--cout      : Write output in .c / .h source format (8 bit char ONLY)
--varname=<NAME> : specify variable name for c source output
--alg=<type>    : specify compression type: 'zx0', 'rle', 'gb' (default)
--bank=<num>    : Add Bank Ref: 1 - 511 (default is none, with --cout only)
Example: "gbcompress binaryfile.bin compressed.bin"
Example: "gbcompress -d compressedfile.bin decompressed.bin"
Example: "gbcompress --alg=rle binaryfile.bin compressed.bin"
The default compression (gb) is the type used by gbtd/gbmb
The rle compression is Amiga IFF style

```

13.15 png2asset settings

```

usage: png2asset <file>.png [options]
-o <filename>      output file (if not used then default is <png file>.c)
-c <filename>      deprecated, same as -o
-sw <width>        metasprites width size (default: png width)
-sh <height>       metasprites height size (default: png height)
-sp <props>        change default for sprite OAM property bytes (in hex) (default: 0x00)
-px <x coord>      metasprites pivot x coordinate (default: metasprites width / 2)
-py <y coord>      metasprites pivot y coordinate (default: metasprites height / 2)
-pw <width>        metasprites collision rect width (default: metasprites width)
-ph <height>       metasprites collision rect height (default: metasprites height)
-spr8x8           for 8x8 hardware sprites (use SPRITES_8x8)
-spr8x16          for 8x16 hardware sprites (use SPRITES_8x16) (the default)
-spr16x16msx      MSX only: for 16x16 hardware sprites (use SPRITES_16x16)
-sprite_no_optimize keep empty sprite tiles, do not remove duplicate tiles
-b <banknum>       Bank number (default: fixed bank)
-area <area name> Area name. Alters "pragma bank ..." output to constseg style. (Ex: -area LIT)
-keep_palette_order use png palette
-repair_indexed_pal try to repair indexed tile palettes (implies "-keep_palette_order")
-noflip           disable tile flip
-map             Export as map (tileset + bg) instead of default metasprite output
-use_map_attributes Use CGB BG Map attributes

```

```

-use_nes_attributes Use NES BG Map attributes
-use_nes_colors      Convert RGB color values to NES PPU colors
-use_structs         Group the exported info into structs (default: false) (used by ZGB Game Engine)
-bpp                 bits per pixel: 1, 2, 4 (default: 2. using 1 auto-enables "-pack_mode 1bpp")
-max_palettes        max number of palettes allowed (default: 8)
                     (note: max colors = max_palettes x num colors per palette)
-pack_mode           gb, nes, sgb, sms, lbpp (default: gb. using lbpp auto-enables "-bpp 1")
-tile_origin         tile index offset for maps (default: 0)
-tiles_only          export tile data only
-maps_only           export map tilemap only
-metasprites_only   export metasprite descriptors only
-source_tileset      use source tileset (image with common tiles)
-entity_tileset      (maps only) mark matching tiles counting from 255 down, entity patterns not exported
-keep_duplicate_tiles do not remove duplicate tiles (default: not enabled)
-no_palettes         do not export palette data
-bin                 export to binary format (requires -map)
-transposed          export transposed (column-by-column instead of row-by-row)
-rel_paths           paths to tilesets are relative to the input file path
-use_metafile        Read extra options from file <inputfile>.meta (file missing not an error)
decoder error empty input buffer given to decoder. Maybe caused by non-existing file?

```

13.16 png2hicolorgb settings

```

png2hicolorgb input_image.png [options]
version 1.4.2: bbbbr. Based on Glen Cook's Windows GUI "hicolour.exe" 1.2
Convert an image to Game Boy Hi-Color format
Options
-h          : Show this help
-v*         : Set log level: "-v" verbose, "-vQ" quiet, "-vE" only errors, "-vD" debug
-o <file>   : Set base output filename (otherwise from input image)
-s <name>   : Set output variable/symbol name (otherwise derived from output filename)
--csource   : Export C source format with incbins for data files
--bank=N    : Set bank number for C source output where N is decimal bank number 1-511
--type=N    : Set conversion type where N is one of below
              1: Median Cut - No Dither (*Default*)
              2: Median Cut - With Dither
              3: Wu Quantiser (best quality)
-p          : Show screen attribute pattern options (no processing)
-L=N        : Set Left side of screen palette arrangement where N is name listed below or decimal entry
-R=N        : Set Right side of screen palette arrangement where N is name listed below or decimal entry
              Named options for N: "adaptive-fast", "adaptive-medium", "adaptive-best" (-p for full options)
--best      : Use highest quality conversion settings (--type=3 -L=adaptive-best -R=adaptive-best)
--vaddridd  : Map uses vram id (128->255->0->127) instead of (*Default*) sequential tile order (0->255)
--nodedupe  : Turn off tile pattern deduplication
--precompiled : Export Palette data as pre-compiled executable loading code
--palendbit  : Set unused bit .15 = 1 for last ul6 entry in palette data indicating end (not in
              precompiled)
--addendcolor=N : Append 32 x color N (hex BGR555) in pal data to clear BG for shorter images (64 bytes)
              (not in precompiled)
Example 1: "png2hicolorgb myimage.png"
Example 2: "png2hicolorgb myimage.png --csource -o my_output_filename"
Example 2: "png2hicolorgb myimage.png --palendbit --addendcolor 0x7FFF -o my_output_filename -s
              my_variable_name"
* Default settings provide good results. Better quality but slower: "--type=3 -L=adaptive-best
              -R=adaptive-best"
Historical credits and info:
  Original Concept : Icarus Productions
  Original Code   : Jeff Frohwein
  Full Screen Modification : Anon
  Adaptive Code   : Glen Cook
  Windows Interface : Glen Cook
  Additional Windows Programming : Rob Jones
  Original Quantiser Code : Benny
  Quantiser Conversion : Glen Cook

```

13.17 romusage settings

```

romusage input_file.[map|noi|ihx|cdb|.gb[c]|.pocket|.duck|.gg|.sms] [options]
version 1.3.2, by bbbbr
Options
-h : Show this help
-p : Set platform (GBDK specific), "-p:SMS_GG" for SMS/Game Gear, "-p:NES1" for NES
-a : Show Areas in each Bank. Optional sort by, address:"-aA" or size:"-aS"
-g : Show a small usage graph per bank (-gA for ascii style)
-G : Show a large usage graph per bank (-GA for ascii style)
-B : Brief (summarized) output for banked regions. Auto scales max bank
      shows [Region]_[Max Used Bank] / [auto-sized Max Bank Num]
-F : Force Max ROM and SRAM bank num for -B. (0 based) -F:ROM:SRAM (ex: -F:255:15)
-m : Manually specify an Area -m:NAME:HEXADDR:HEXLENGTH
-e : Manually specify an Area that should not overlap -e:NAME:HEXADDR:HEXLENGTH
-b : Set hex bytes treated as Empty in ROM files (.gb/etc) -b:HEXVAL[...] (default FF)
-E : All areas are exclusive (except HEADERS), warn for any overlaps
-q : Quiet, no output except warnings and errors
-Q : Suppress output of warnings and errors

```

```

-R : Return error code for Area warnings and errors
-sR : [Rainbow] Color output (-sRe for Row Ends, -sRd for Center Dimmed, -sRp % based)
-sP : Custom Color Palette. Colon separated entries are decimal VT100 color codes
      -sP:DEFAULT:ROM:VRAM:SRAM:WRAM:HRAM (section based color only)
-sC : Show Compact Output, hide non-essential columns
-sH : Show HEADER Areas (normally hidden)
-smROM : Show Merged ROM_0 and ROM_1 output (i.e. bare 32K ROM)
-smWRAM : Show Merged WRAM_0 and WRAM_1 output (i.e. DMG/MGB not CGB)
        -sm* compatible with banked ROM_x or WRAM_x when used with -B
-sJ : Show JSON output. Some options not applicable. When used, -Q recommended
-nB : Hide warning banner (for .cdb output)
-nA : Hide areas (shown by default in .cdb output)
-z : Hide areas smaller than SIZE -z:DECSIZE
-nMEM : Hide banks matching case sensitive substring (ex hide all RAM: -nMEM:RAM)
Use: Read a .map, .noi, .cdb or .ihx file to display area sizes
Example 1: "romusage build/MyProject.map"
Example 2: "romusage build/MyProject.noi -a -e:STACK:DEFF:100 -e:SHADOW_OAM:C000:A0"
Example 3: "romusage build/MyProject.ihx -g"
Example 4: "romusage build/MyProject.map -q -R"
Example 5: "romusage build/MyProject.noi -sR -sP:90:32:90:35:33:36"
Example 6: "romusage build/MyProject.map -sRp -g -B -F:255:15 -smROM -smWRAM"
Example 7: "romusage build/MyProject.gb -g -b:FF:00"
Notes:
* GBDK / RGBDS map file format detection is automatic.
* Estimates are as close as possible, but may not be complete.
  Unless specified with -m/-e they *do not* factor regions lacking
  complete ranges in the Map/Noi/Ihx file, for example Shadow OAM and Stack.
* IHX files can only detect overlaps, not detect memory region overflows.
* CDB file output ONLY counts (most) data from C sources.
  It cannot count functions and data from ASM and LIBs,
  so bank totals may be incorrect/missing.
* GB/GBC/ROM files are just guessing, no promises.

```

14 Todo List

File [far_ptr.h](#)

Add link to a discussion about banking (such as, how to assign code and variables to banks)

Page [ROM/SRAM Banking and MBCs](#)

Variables in SRAM

15 Module Index

15.1 Modules

Here is a list of all modules:

List of gbdk fonts 95

16 Data Structure Index

16.1 Data Structures

Here are the data structures with brief descriptions:

__far_ptr	96
_fixed	96
atomic_flag	97
isr_nested_vector_t	97
isr_vector_t	98
joypads_t	98
metasprite_t	100

OAM_item_t	101
sfont_handle	102

17 File Index

17.1 File List

Here is a list of all files with brief descriptions:

gbdk-lib/include/assert.h	107
gbdk-lib/include/ctype.h	108
gbdk-lib/include/limits.h	237
gbdk-lib/include/rand.h	404
gbdk-lib/include/setjmp.h	406
gbdk-lib/include/stdarg.h	107
gbdk-lib/include/stdatomic.h	441
gbdk-lib/include/stdbool.h	442
gbdk-lib/include/stddef.h	443
gbdk-lib/include/stdint.h	445
gbdk-lib/include/stdio.h	454
gbdk-lib/include/stdlib.h	456
gbdk-lib/include/stdnoreturn.h	460
gbdk-lib/include/string.h	474
gbdk-lib/include/time.h	474
gbdk-lib/include/typeof.h	475
gbdk-lib/include/types.h	485
gbdk-lib/include/asm/types.h	481
gbdk-lib/include/asm/mos6502/provides.h	103
gbdk-lib/include/asm/mos6502/stdarg.h	104
gbdk-lib/include/asm/mos6502/string.h	460
gbdk-lib/include/asm/mos6502/types.h	478
gbdk-lib/include/asm/sm83/provides.h	103
gbdk-lib/include/asm/sm83/stdarg.h	105
gbdk-lib/include/asm/sm83/string.h	465
gbdk-lib/include/asm/sm83/types.h	479

gbdk-lib/include/asm/z80/provides.h	104
gbdk-lib/include/asm/z80/stdarg.h	106
gbdk-lib/include/asm/z80/string.h	470
gbdk-lib/include/asm/z80/types.h	484
gbdk-lib/include/duck/laptop_io.h	110
gbdk-lib/include/duck/laptop_keycodes.h	119
gbdk-lib/include/duck/model.h	130
gbdk-lib/include/gb/bcd.h	131
gbdk-lib/include/gb/bgb_emu.h	138
gbdk-lib/include/gb/cgb.h	139
gbdk-lib/include/gb/crash_handler.h	145
gbdk-lib/include/gb/drawing.h	146
gbdk-lib/include/gb/emu_debug.h	151
gbdk-lib/include/gb/gb.h	156
gbdk-lib/include/gb/gbdecompress.h	213
gbdk-lib/include/gb/hardware.h	239
gbdk-lib/include/gb/hblankcpy.h	217
gbdk-lib/include/gb/isr.h	219
gbdk-lib/include/gb/metasprites.h	300
gbdk-lib/include/gb/sgb.h	221
gbdk-lib/include/gbdk/bcd.h	134
gbdk-lib/include/gbdk/console.h	224
gbdk-lib/include/gbdk/emu_debug.h	151
gbdk-lib/include/gbdk/far_ptr.h	226
gbdk-lib/include/gbdk/font.h	229
gbdk-lib/include/gbdk/gbdecompress.h	216
gbdk-lib/include/gbdk/gbdk-lib.h	232
gbdk-lib/include/gbdk/incbin.h	232
gbdk-lib/include/gbdk/metasprites.h	308
gbdk-lib/include/gbdk/platform.h	234
gbdk-lib/include/gbdk/rledecompress.h	235
gbdk-lib/include/gbdk/version.h	236

gbdk-lib/include/gbdk/zx0decompress.h	236
gbdk-lib/include/msx/hardware.h	269
gbdk-lib/include/msx/metasprites.h	308
gbdk-lib/include/msx/msx.h	324
gbdk-lib/include/nes/bcd.h	134
gbdk-lib/include/nes/hardware.h	277
gbdk-lib/include/nes/metasprites.h	312
gbdk-lib/include/nes/nes.h	358
gbdk-lib/include/nes/rgb_to_nes_macro.h	403
gbdk-lib/include/sms/bcd.h	136
gbdk-lib/include/sms/gbdecompress.h	216
gbdk-lib/include/sms/hardware.h	284
gbdk-lib/include/sms/metasprites.h	318
gbdk-lib/include/sms/sms.h	408

18 Module Documentation

18.1 List of gbdk fonts

Variables

- [uint8_t font_spect\[\]](#)
- [uint8_t font_italic\[\]](#)
- [uint8_t font_ibm\[\]](#)
- [uint8_t font_min\[\]](#)
- [uint8_t font_ibm_fixed\[\]](#)

18.1.1 Detailed Description

18.1.2 Variable Documentation

18.1.2.1 font_spect `uint8_t font_spect[]` [extern]

The default fonts

18.1.2.2 font_italic `uint8_t font_italic[]`

18.1.2.3 font_ibm `uint8_t font_ibm[]`

18.1.2.4 font_min `uint8_t font_min[]`

18.1.2.5 font_ibm_fixed `uint8_t font_ibm_fixed[]` [extern]

Backwards compatible font

19 Data Structure Documentation

19.1 `__far_ptr` Union Reference

```
#include <gbdk-lib/include/gbdk/far_ptr.h>
```

Data Fields

- `FAR_PTR ptr`
- struct {
 void * `ofs`
 uint16_t `seg`
} `segofs`
- struct {
 void(* `fn`)(void)
 uint16_t `seg`
} `segfn`

19.1.1 Detailed Description

Union for working with members of a `FAR_PTR`

19.1.2 Field Documentation

19.1.2.1 ptr `FAR_PTR __far_ptr::ptr`

19.1.2.2 ofs `void* __far_ptr::ofs`

19.1.2.3 seg `uint16_t __far_ptr::seg`

19.1.2.4 `struct { ... } __far_ptr::segofs`

19.1.2.5 fn `void(* __far_ptr::fn) (void)`

19.1.2.6 `struct { ... } __far_ptr::segfn`

The documentation for this union was generated from the following file:

- `gbdk-lib/include/gbdk/far_ptr.h`

19.2 `_fixed` Union Reference

```
#include <gbdk-lib/include/asm/types.h>
```

Data Fields

- struct {
 `UBYTE l`
 `UBYTE h`
};

- struct {
 UBYTE l
 UBYTE h
} b
- UWORD w

19.2.1 Detailed Description

Useful definition for working with 8 bit + 8 bit fixed point values

Use .w to access the variable as unsigned 16 bit type.

Use .b.h and .b.l (or just .h and .l) to directly access it's high and low unsigned 8 bit values.

19.2.2 Field Documentation

19.2.2.1 l UBYTE _fixed::l

19.2.2.2 h UBYTE _fixed::h

19.2.2.3 struct { ... } _fixed::@1

19.2.2.4 struct { ... } _fixed::b

19.2.2.5 w UWORD _fixed::w

The documentation for this union was generated from the following file:

- gbdk-lib/include/asm/types.h

19.3 atomic_flag Struct Reference

```
#include <gbdk-lib/include/stdatomic.h>
```

Data Fields

- unsigned char flag

19.3.1 Field Documentation

19.3.1.1 flag unsigned char atomic_flag::flag

The documentation for this struct was generated from the following file:

- gbdk-lib/include/stdatomic.h

19.4 isr_nested_vector_t Struct Reference

```
#include <gbdk-lib/include/gb/isr.h>
```

Data Fields

- uint8_t opcode [2]
- void * func

19.4.1 Field Documentation

19.4.1.1 opcode `uint8_t` `isr_nested_vector_t::opcode[2]`

19.4.1.2 func `void*` `isr_nested_vector_t::func`

The documentation for this struct was generated from the following file:

- `gbdk-lib/include/gb/isr.h`

19.5 `isr_vector_t` Struct Reference

```
#include <gbdk-lib/include/gb/isr.h>
```

Data Fields

- `uint8_t` `opcode`
- `void *` `func`

19.5.1 Field Documentation

19.5.1.1 opcode `uint8_t` `isr_vector_t::opcode`

19.5.1.2 func `void*` `isr_vector_t::func`

The documentation for this struct was generated from the following file:

- `gbdk-lib/include/gb/isr.h`

19.6 `joypads_t` Struct Reference

```
#include <gbdk-lib/include/gb/gb.h>
```

Data Fields

- `uint8_t` `npads`
 - union {
 - struct {
 - `uint8_t` `joy0`
 - `uint8_t` `joy1`
 - `uint8_t` `joy2`
 - `uint8_t` `joy3`
 - `uint8_t` `joypads` [4]
- };
- union {
 - struct {
 - `uint8_t` `joy0`
 - `uint8_t` `joy1`
 - `uint8_t` `joy2`
 - `uint8_t` `joy3`
- `uint8_t` `joypads` [4]
- };

```

• union {
    struct {
        uint8_t joy0
        uint8_t joy1
        uint8_t joy2
        uint8_t joy3
    }
    uint8_t joypads [4]
};

• union {
    struct {
        uint8_t joy0
        uint8_t joy1
        uint8_t joy2
        uint8_t joy3
    }
    uint8_t joypads [4]
};

```

19.6.1 Detailed Description

Multiplayer joystick structure.

Must be initialized with [joypad_init\(\)](#) first then it may be used to poll all available joypads with [joypad_ex\(\)](#)

19.6.2 Field Documentation

19.6.2.1 npads [uint8_t](#) joypads_t::npads

19.6.2.2 joy0 [uint8_t](#) joypads_t::joy0

19.6.2.3 joy1 [uint8_t](#) joypads_t::joy1

19.6.2.4 joy2 [uint8_t](#) joypads_t::joy2

19.6.2.5 joy3 [uint8_t](#) joypads_t::joy3

19.6.2.6 joypads [uint8_t](#) joypads_t::joypads[4]

19.6.2.7 `union { ... } joypads_t::@4`

19.6.2.8 `union { ... } joypads_t::@10`

19.6.2.9 `union { ... } joypads_t::@14`

19.6.2.10 `union { ... } joypads_t::@18`

The documentation for this struct was generated from the following files:

- [gbdk-lib/include/gb/gb.h](#)
- [gbdk-lib/include/msx/msx.h](#)
- [gbdk-lib/include/nes/nes.h](#)
- [gbdk-lib/include/sms/sms.h](#)

19.7 metasprite_t Struct Reference

```
#include <gbdk-lib/include/gb/metaspikes.h>
```

Data Fields

- [int8_t dy](#)
- [int8_t dx](#)
- [uint8_t dtile](#)
- [uint8_t props](#)

19.7.1 Detailed Description

Metasprite sub-item structure

Parameters

<i>dy</i>	(int8_t) Y coordinate of the sprite relative to the metasprite origin (pivot)
<i>dx</i>	(int8_t) X coordinate of the sprite relative to the metasprite origin (pivot)
<i>dtile</i>	(uint8_t) Start tile relative to the metasprites own set of tiles
<i>props</i>	(uint8_t) Property Flags

Metaspikes are built from multiple [metasprite_t](#) items (one for each sub-sprite) and a pool of tiles they reference. If a metasprite has multiple frames then each frame will be built from some number of [metasprite_t](#) items (which may vary based on how many sprites are required for that particular frame).

A metasprite frame is terminated with a {metasprite_end} entry.

Metasprite sub-item structure

Parameters

<i>dy</i>	(int8_t) Y coordinate of the sprite relative to the metasprite origin (pivot)
<i>dx</i>	(int8_t) X coordinate of the sprite relative to the metasprite origin (pivot)
<i>dtile</i>	(uint8_t) Start tile relative to the metasprites own set of tiles

Metaspikes are built from multiple [metasprite_t](#) items (one for each sub-sprite) and a pool of tiles they reference. If a metasprite has multiple frames then each frame will be built from some number of [metasprite_t](#) items (which may vary based on how many sprites are required for that particular frame).

A metasprite frame is terminated with a {metasprite_end} entry.

19.7.2 Field Documentation

19.7.2.1 `dy` [int8_t](#) metasprite_t::dy

19.7.2.2 `dx` [int8_t](#) metasprite_t::dx

19.7.2.3 dtile `uint8_t metasprite_t::dtile`

19.7.2.4 props `uint8_t metasprite_t::props`

The documentation for this struct was generated from the following files:

- [gbdk-lib/include/gb/metasprites.h](#)
- [gbdk-lib/include/msx/metasprites.h](#)
- [gbdk-lib/include/nes/metasprites.h](#)
- [gbdk-lib/include/sms/metasprites.h](#)

19.8 OAM_item_t Struct Reference

```
#include <gbdk-lib/include/gb/gb.h>
```

Data Fields

- [uint8_t y](#)
- [uint8_t x](#)
- [uint8_t tile](#)
- [uint8_t prop](#)

19.8.1 Detailed Description

Sprite Attributes structure

Parameters

<i>x</i>	X Coordinate of the sprite on screen
<i>y</i>	Y Coordinate of the sprite on screen
<i>tile</i>	Sprite tile number (see set_sprite_tile)
<i>prop</i>	OAM Property Flags (see set_sprite_prop)

Sprite Attributes structure

Parameters

<i>x</i>	X Coordinate of the sprite on screen
<i>y</i>	Y Coordinate of the sprite on screen - 1
<i>tile</i>	Sprite tile number (see set_sprite_tile)
<i>prop</i>	OAM Property Flags (see set_sprite_prop)

19.8.2 Field Documentation

19.8.2.1 y `uint8_t OAM_item_t::y`

19.8.2.2 x `uint8_t OAM_item_t::x`

19.8.2.3 tile `uint8_t OAM_item_t::tile`

19.8.2.4 `prop` `uint8_t` `OAM_item_t::prop`

The documentation for this struct was generated from the following files:

- `gbdk-lib/include/gb/gb.h`
- `gbdk-lib/include/msx/msx.h`
- `gbdk-lib/include/nes/nes.h`

19.9 `sfont_handle` Struct Reference

```
#include <gbdk-lib/include/gbdk/font.h>
```

Data Fields

- `uint8_t` `first_tile`
- `void *` `font`

19.9.1 Detailed Description

Font handle structure

19.9.2 Field Documentation

19.9.2.1 `first_tile` `uint8_t` `sfont_handle::first_tile`

First tile used for font

19.9.2.2 `font` `void*` `sfont_handle::font`

Pointer to the base of the font

The documentation for this struct was generated from the following file:

- `gbdk-lib/include/gbdk/font.h`

20 File Documentation

- 20.1 docs/pages/01_getting_started.md File Reference
- 20.2 docs/pages/02_links_and_tools.md File Reference
- 20.3 docs/pages/03_using_gbdk.md File Reference
- 20.4 docs/pages/04_coding_guidelines.md File Reference
- 20.5 docs/pages/05_banking_mbc5.md File Reference
- 20.6 docs/pages/06_toolchain.md File Reference
- 20.7 docs/pages/06b_supported_consoles.md File Reference
- 20.8 docs/pages/07_sample_programs.md File Reference
- 20.9 docs/pages/08_faq.md File Reference
- 20.10 docs/pages/09_migrating_new_versions.md File Reference
- 20.11 docs/pages/10_release_notes.md File Reference
- 20.12 docs/pages/20_toolchain_settings.md File Reference
- 20.13 docs/pages/docs_index.md File Reference
- 20.14 gbdk-lib/include/asm/mos6502/provides.h File Reference

Macros

- `#define USE_C_MEMCPY 0`
- `#define USE_C_STRCPY 0`
- `#define USE_C_STRCMP 0`

20.14.1 Macro Definition Documentation

20.14.1.1 **USE_C_MEMCPY** `#define USE_C_MEMCPY 0`

20.14.1.2 **USE_C_STRCPY** `#define USE_C_STRCPY 0`

20.14.1.3 **USE_C_STRCMP** `#define USE_C_STRCMP 0`

20.15 provides.h

[Go to the documentation of this file.](#)

```
1 #define USE_C_MEMCPY    0
2 #define USE_C_STRCPY    0
3 #define USE_C_STRCMP    0
4
```

20.16 gbdk-lib/include/asm/sm83/provides.h File Reference

Macros

- `#define USE_C_MEMCPY 0`
- `#define USE_C_STRCPY 0`
- `#define USE_C_STRCMP 0`

20.16.1 Macro Definition Documentation

20.16.1.1 USE_C_MEMCPY `#define USE_C_MEMCPY 0`

20.16.1.2 USE_C_STRCPY `#define USE_C_STRCPY 0`

20.16.1.3 USE_C_STRCMP `#define USE_C_STRCMP 0`

20.17 provides.h

[Go to the documentation of this file.](#)

```
1 #define USE_C_MEMCPY    0
2 #define USE_C_STRCPY    0
3 #define USE_C_STRCMP    0
4
```

20.18 gbdk-lib/include/asm/z80/provides.h File Reference

Macros

- `#define USE_C_MEMCPY 0`
- `#define USE_C_STRCPY 0`
- `#define USE_C_STRCMP 1`

20.18.1 Macro Definition Documentation

20.18.1.1 USE_C_MEMCPY `#define USE_C_MEMCPY 0`

20.18.1.2 USE_C_STRCPY `#define USE_C_STRCPY 0`

20.18.1.3 USE_C_STRCMP `#define USE_C_STRCMP 1`

20.19 provides.h

[Go to the documentation of this file.](#)

```
1 #define USE_C_MEMCPY    0
2 #define USE_C_STRCPY    0
3 #define USE_C_STRCMP    1
4
```

20.20 gbdk-lib/include/asm/mos6502/stdarg.h File Reference

Macros

- `#define va_start(list, last) list = (unsigned char *)&last + sizeof(last)`
- `#define va_arg(list, type) *((type *)((list += sizeof(type)) - sizeof(type)))`
- `#define va_end(list)`

Typedefs

- `typedef unsigned char * va_list`

20.20.1 Macro Definition Documentation

20.20.1.1 va_start #define va_start(
 list,
 last) list = (unsigned char *)&last + sizeof(last)

20.20.1.2 va_arg #define va_arg(
 list,
 type) *((type *)((list += sizeof(type)) - sizeof(type)))

20.20.1.3 va_end #define va_end(
 list)

20.20.2 Typedef Documentation

20.20.2.1 va_list typedef unsigned char* va_list

20.21 stdarg.h

[Go to the documentation of this file.](#)

```
1 #ifndef ASM_MOS6502_STDARG_INCLUDE
2 #define ASM_MOS6502_STDARG_INCLUDE
3
4 /* sdcc pushes right to left with the real sizes, not cast up
5    to an int.
6    so printf(int, char, long)
7    results in push long, push char, push int
8    On the 6502 the stack grows down, so the things seem to be in
9    the correct order.
10 */
11
12 typedef unsigned char * va_list;
13 #define va_start(list, last)    list = (unsigned char *)&last + sizeof(last)
14 #define va_arg(list, type)    *((type *)((list += sizeof(type)) - sizeof(type)))
15
16 #define va_end(list)
17
18 #endif
```

20.22 gbdk-lib/include/asm/sm83/stdarg.h File Reference

Macros

- #define [va_start](#)(list, last) list = (unsigned char *)&last + sizeof(last)
- #define [va_arg](#)(list, type) *((type *)((list += sizeof(type)) - sizeof(type)))
- #define [va_end](#)(list)

Typedefs

- typedef unsigned char * [va_list](#)

20.22.1 Macro Definition Documentation

20.22.1.1 va_start #define va_start(
 list,
 last) list = (unsigned char *)&last + sizeof(last)

```
20.22.1.2 va_arg #define va_arg(  
    list,  
    type ) *((type *) ((list += sizeof(type)) - sizeof(type)))
```

```
20.22.1.3 va_end #define va_end(  
    list )
```

20.22.2 Typedef Documentation

```
20.22.2.1 va_list typedef unsigned char* va_list
```

20.23 stdarg.h

[Go to the documentation of this file.](#)

```
1 #ifndef ASM_SM83_STDARG_INCLUDE  
2 #define ASM_SM83_STDARG_INCLUDE  
3  
4 /* sdcc pushes right to left with the real sizes, not cast up  
5    to an int.  
6    so printf(int, char, long)  
7    results in push long, push char, push int  
8    On the z80 the stack grows down, so the things seem to be in  
9    the correct order.  
10 */  
11  
12 typedef unsigned char * va_list;  
13 #define va_start(list, last) list = (unsigned char *)&last + sizeof(last)  
14 #define va_arg(list, type) *((type *) ((list += sizeof(type)) - sizeof(type)))  
15  
16 #define va_end(list)  
17  
18 #endif
```

20.24 gbdk-lib/include/asm/z80/stdarg.h File Reference

Macros

- #define [va_start](#)(list, last) list = (unsigned char *)&last + sizeof(last)
- #define [va_arg](#)(list, type) *((type *) ((list += sizeof(type)) - sizeof(type)))
- #define [va_end](#)(list)

Typedefs

- typedef unsigned char * [va_list](#)

20.24.1 Macro Definition Documentation

```
20.24.1.1 va_start #define va_start(  
    list,  
    last ) list = (unsigned char *)&last + sizeof(last)
```

```
20.24.1.2 va_arg #define va_arg(  
    list,  
    type ) *((type *) ((list += sizeof(type)) - sizeof(type)))
```

```
20.24.1.3 va_end #define va_end(  
    list )
```

20.24.2 Typedef Documentation

20.24.2.1 `va_list` typedef unsigned char* `va_list`

20.25 stdarg.h

[Go to the documentation of this file.](#)

```
1 #ifndef ASM_Z80_STDARG_INCLUDE
2 #define ASM_Z80_STDARG_INCLUDE
3
4 /* sdcc pushes right to left with the real sizes, not cast up
5    to an int.
6    so printf(int, char, long)
7    results in push long, push char, push int
8    On the z80 the stack grows down, so the things seem to be in
9    the correct order.
10   */
11
12 typedef unsigned char * va_list;
13 #define va_start(list, last)    list = (unsigned char *)&last + sizeof(last)
14 #define va_arg(list, type)    *((type *) ((list += sizeof(type)) - sizeof(type)))
15
16 #define va_end(list)
17
18 #endif
```

20.26 gbdk-lib/include/stdarg.h File Reference

```
#include <asm/sm83/stdarg.h>
```

20.27 stdarg.h

[Go to the documentation of this file.](#)

```
1 #ifndef STDARG_INCLUDE
2 #define STDARG_INCLUDE
3
4 #if defined(__PORT_sm83)
5 #include <asm/sm83/stdarg.h>
6 #elif defined(__PORT_z80)
7 #include <asm/z80/stdarg.h>
8 #elif defined(__PORT_mos6502)
9 #include <asm/mos6502/stdarg.h>
10 #endif
11
12 #endif
```

20.28 gbdk-lib/include/assert.h File Reference

Macros

- #define `assert(x)` ((x) ? (void)0 : `__assert`(#x, __func__, __FILE__, __LINE__))

Functions

- void `__assert` (const char *expression, const char *functionname, const char *filename, unsigned int linenumber)

20.28.1 Macro Definition Documentation

20.28.1.1 `assert` #define `assert` (

```

x ) ((x) ? (void)0 : __assert (#x, __func__, __FILE__, __LINE__))
```

20.28.2 Function Documentation

20.28.2.1 `__assert()` `void __assert (`
`const char * expression,`
`const char * functionname,`
`const char * filename,`
`unsigned int linenumber )`

20.29 `assert.h`

[Go to the documentation of this file.](#)

```
1  /*-----
2      assert.h - header file for assert ANSI routine
3
4      Copyright (C) 2018, Philipp Klaus Krause . pkk@spth.de
5
6      This library is free software; you can redistribute it and/or modify it
7      under the terms of the GNU General Public License as published by the
8      Free Software Foundation; either version 2, or (at your option) any
9      later version.
10
11      This library is distributed in the hope that it will be useful,
12      but WITHOUT ANY WARRANTY; without even the implied warranty of
13      MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14      GNU General Public License for more details.
15
16      You should have received a copy of the GNU General Public License
17      along with this library; see the file COPYING. If not, write to the
18      Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,
19      MA 02110-1301, USA.
20
21      As a special exception, if you link this library with other files,
22      some of which are compiled with SDCC, to produce an executable,
23      this library does not by itself cause the resulting executable to
24      be covered by the GNU General Public License. This exception does
25      not however invalidate any other reasons why the executable file
26      might be covered by the GNU General Public License.
27  -----*/
28
29 #undef assert
30
31 #ifdef NDEBUG
32
33 /* Debugging disabled -- do not evaluate assertions. */
34 #define assert(x) ((void)0)
35
36 #else
37
38 /* Debugging enabled -- verify assertions at run time. */
39 void __assert(const char *expression, const char *functionname, const char *filename, unsigned int
    linenumber);
40 #define assert(x) ((x) ? (void)0 : __assert(#x, __func__, __FILE__, __LINE__))
41
42 #if __STDC_VERSION__ >= 201112L
43 #define static_assert _Static_assert
44 #endif
45
46 #endif
47
```

20.30 `gbdk-lib/include/ctype.h` File Reference

```
#include <types.h>
#include <stdbool.h>
```

Functions

- [bool `isalpha`](#) (char c)
- [bool `isupper`](#) (char c)
- [bool `islower`](#) (char c)
- [bool `isdigit`](#) (char c)

- `bool isspace` (char `c`)
- `char toupper` (char `c`)
- `char tolower` (char `c`)

20.30.1 Detailed Description

Character type functions.

20.30.2 Function Documentation

20.30.2.1 isalpha() `bool isalpha (char c)`

Returns TRUE if the character **c** is a letter (a-z, A-Z), otherwise FALSE

Parameters

c	Character to test
---	-------------------

20.30.2.2 isupper() `bool isupper (char c)`

Returns TRUE if the character **c** is an uppercase letter (A-Z), otherwise FALSE

Parameters

c	Character to test
---	-------------------

20.30.2.3 islower() `bool islower (char c)`

Returns TRUE if the character **c** is a lowercase letter (a-z), otherwise FALSE

Parameters

<i>c</i>	Character to test
----------	-------------------

20.30.2.4 isdigit() `bool isdigit (char c)`

Returns TRUE if the character **c** is a digit (0-9), otherwise FALSE

Parameters

c	Character to test
---	-------------------

20.30.2.5 isspace() `bool isspace (char c)`

Returns TRUE if the character **c** is a space (' '), tab (\t), or newline (\n) character, otherwise FALSE

Parameters

c	Character to test
----------	-------------------

20.30.2.6 toupper() `char toupper (`
`char c )`

Returns uppercase version of character **c** if it is a letter (a-z), otherwise it returns the input value unchanged.

Parameters

c	Character to test
----------	-------------------

20.30.2.7 tolower() `char tolower (`
`char c )`

Returns lowercase version of character **c** if it is a letter (A-Z), otherwise it returns the input value unchanged.

Parameters

c	Character to test
----------	-------------------

20.31 ctype.h

[Go to the documentation of this file.](#)

```
1
4 #ifndef _CTYPE_H
5 #define _CTYPE_H
6
7 #include <types.h>
8 #include <stdbool.h>
9
13 bool isalpha(char c);
14
18 bool isupper(char c);
19
23 bool islower(char c);
24
28 bool isdigit(char c);
29
33 bool isspace(char c);
34
38 char toupper(char c);
39
43 char tolower(char c);
44
45 #endif /* _CTYPE_H */
```

20.32 gbdk-lib/include/duck/laptop_io.h File Reference

```
#include <gbdk/platform.h>
#include <stdint.h>
#include <stdbool.h>
```

Macros

- `#define MEGADUCK_LAPTOP_IO_H`
- `#define DUCK_IO_CMD_INIT_START 0x00u`
- `#define DUCK_IO_CMD_GET_KEYS 0x00u`
- `#define DUCK_IO_CMD_DONE_OR_OK 0x01u`

- #define [DUCK_IO_CMD_ABORT_OR_FAIL](#) 0x04u
- #define [DUCK_IO_CMD_PLAY_SPEECH](#) 0x05u
- #define [DUCK_IO_CMD_RUN_CART_IN_SLOT](#) 0x08u
- #define [DUCK_IO_CMD_PRINT_INIT_EXT_IO](#) 0x09u
- #define [DUCK_IO_CMD_SET_RTC](#) 0x0Bu
- #define [DUCK_IO_CMD_GET_RTC](#) 0x0Cu
- #define [DUCK_IO_CMD_PRINT_SEND_BYTES](#) 0x11u
- #define [DUCK_IO_REPLY_BOOT_UNSET](#) 0x00u
- #define [DUCK_IO_REPLY_BOOT_FAIL](#) 0x01u
- #define [DUCK_IO_REPLY_BUFFER_XFER_OK](#) 0x01u
- #define [DUCK_IO_REPLY_SEND_BUFFER_OK](#) 0x03u
- #define [DUCK_IO_REPLY_BOOT_OK](#) 0x01u
- #define [DUCK_IO_LEN_KBD_GET](#) 2u
- #define [DUCK_IO_LEN_RTC_GET](#) 8u
- #define [DUCK_IO_LEN_RTC_SET](#) 8u
- #define [DUCK_IO_LEN_PLAY_SPEECH](#) 1u
- #define [DUCK_IO_REPLY_NO_CART_IN_SLOT](#) 06u
- #define [DUCK_IO_LEN_RX_MAX](#) 14u
- #define [DUCK_IO_LEN_TX_MAX](#) 14u
- #define [DUCK_IO_TIMEOUT_2_MSEC](#) 2u
- #define [DUCK_IO_TIMEOUT_100_MSEC](#) 100u
- #define [DUCK_IO_TIMEOUT_200_MSEC](#) 200u
- #define [DUCK_IO_SPEECH_CMD_MIN](#) 1
- #define [DUCK_IO_SPEECH_CMD_MAX](#) 6
- #define [DUCK_IO_RTC_YEAR](#) 0u
- #define [DUCK_IO_RTC_MON](#) 1u
- #define [DUCK_IO_RTC_DAY](#) 2u
- #define [DUCK_IO_RTC_WEEKDAY](#) 3u
- #define [DUCK_IO_RTC_AMPM](#) 4u
- #define [DUCK_IO_RTC_HOUR](#) 5u
- #define [DUCK_IO_RTC_MIN](#) 6u
- #define [DUCK_IO_RTC_SEC](#) 7u
- #define [DUCK_IO_KBD_FLAGS](#) 0u
- #define [DUCK_IO_KBD_KEYCODE](#) 1u
- #define [DUCK_IO_PRINTER_FAIL](#) 0x00u
- #define [DUCK_IO_PRINTER_TYPE_2_PASS](#) 0x01u
- #define [DUCK_IO_PRINTER_TYPE_1_PASS](#) 0x02u
- #define [DUCK_IO_PRINTER_MAYBE_BUSY](#) 0x03u

Functions

- void [duck_io_send_byte](#) (uint8_t tx_byte)
- uint8_t [duck_io_read_byte_no_timeout](#) (void)
- void [duck_io_enable_read_byte](#) (void)
- bool [duck_io_laptop_init](#) (void)
- uint8_t [duck_io_printer_last_status](#) (void)
- uint8_t [duck_io_printer_query](#) (void)
- bool [duck_io_read_byte_with_msecs_timeout](#) (uint8_t timeout_len_ms)
- bool [duck_io_send_byte_and_check_ack_msecs_timeout](#) (uint8_t tx_byte, uint8_t timeout_len_ms, uint8_t expected_reply)
- bool [duck_io_send_cmd_and_buffer](#) (uint8_t io_cmd)
- bool [duck_io_send_cmd_and_receive_buffer](#) (uint8_t io_cmd)

Variables

- volatile `bool duck_io_rx_byte_done`
- volatile `uint8_t duck_io_rx_byte`
- `uint8_t duck_io_rx_buf [DUCK_IO_LEN_RX_MAX]`
- `uint8_t duck_io_rx_buf_len`
- `uint8_t duck_io_tx_buf [DUCK_IO_LEN_TX_MAX]`
- `uint8_t duck_io_tx_buf_len`

20.32.1 Detailed Description

20.32.2 MegaDuck Laptop Peripheral IO support

The MegaDuck Laptop models (Spanish and German) have several built-in hardware peripherals which are attached via a controller that is communicated with using the serial link port.

Note

Using the `duck_io_*` functions referenced from this header file will cause the `duck_io_` serial interrupt handler to be automatically installed.

To use any functions here, `duck_io_laptop_init()` must be called first (just once).

For the present time all of the serial operations are blocking, they do not return until completed.

20.32.3 Macro Definition Documentation

20.32.3.1 `_MEGADUCK_LAPTOP_IO_H` `#define _MEGADUCK_LAPTOP_IO_H`

20.32.3.2 `DUCK_IO_CMD_INIT_START` `#define DUCK_IO_CMD_INIT_START 0x00u`

Command to request starting the hardware init counter sequence process

20.32.3.3 `DUCK_IO_CMD_GET_KEYS` `#define DUCK_IO_CMD_GET_KEYS 0x00u`

Command to get hardware keyboard data by receiving a multi-byte packet

20.32.3.4 `DUCK_IO_CMD_DONE_OR_OK` `#define DUCK_IO_CMD_DONE_OR_OK 0x01u`

20.32.3.5 `DUCK_IO_CMD_ABORT_OR_FAIL` `#define DUCK_IO_CMD_ABORT_OR_FAIL 0x04u`

20.32.3.6 `DUCK_IO_CMD_PLAY_SPEECH` `#define DUCK_IO_CMD_PLAY_SPEECH 0x05u`

20.32.3.7 `DUCK_IO_CMD_RUN_CART_IN_SLOT` `#define DUCK_IO_CMD_RUN_CART_IN_SLOT 0x08u`

20.32.3.8 `DUCK_IO_CMD_PRINT_INIT_EXT_IO` `#define DUCK_IO_CMD_PRINT_INIT_EXT_IO 0x09u`

Command to init the printer and return status + model type

20.32.3.9 `DUCK_IO_CMD_SET_RTC` `#define DUCK_IO_CMD_SET_RTC 0x0Bu`

Command to set hardware RTC by sending a multi-byte packet

20.32.3.10 `DUCK_IO_CMD_GET_RTC` `#define DUCK_IO_CMD_GET_RTC 0x0Cu`

Command to get hardware RTC by receiving a a multi-byte packet

20.32.3.11 DUCK_IO_CMD_PRINT_SEND_BYTES `#define DUCK_IO_CMD_PRINT_SEND_BYTES 0x11u`
Send printer data

20.32.3.12 DUCK_IO_REPLY_BOOT_UNSET `#define DUCK_IO_REPLY_BOOT_UNSET 0x00u`

20.32.3.13 DUCK_IO_REPLY_BOOT_FAIL `#define DUCK_IO_REPLY_BOOT_FAIL 0x01u`

20.32.3.14 DUCK_IO_REPLY_BUFFER_XFER_OK `#define DUCK_IO_REPLY_BUFFER_XFER_OK 0x01u`

20.32.3.15 DUCK_IO_REPLY_SEND_BUFFER_OK `#define DUCK_IO_REPLY_SEND_BUFFER_OK 0x03u`

20.32.3.16 DUCK_IO_REPLY_BOOT_OK `#define DUCK_IO_REPLY_BOOT_OK 0x01u`

20.32.3.17 DUCK_IO_LEN_KBD_GET `#define DUCK_IO_LEN_KBD_GET 2u`
Get Keyboard key payload size: 2 bytes Payload (excludes 1 length header byte, 1 byte Checksum)

20.32.3.18 DUCK_IO_LEN_RTC_GET `#define DUCK_IO_LEN_RTC_GET 8u`
Get RTC payload size: 8 bytes Payload (excludes 1 length header byte, 1 byte Checksum)

20.32.3.19 DUCK_IO_LEN_RTC_SET `#define DUCK_IO_LEN_RTC_SET 8u`
Set RTC payload size: 8 bytes Payload (excludes 1 length header byte, 1 byte Checksum)

20.32.3.20 DUCK_IO_LEN_PLAY_SPEECH `#define DUCK_IO_LEN_PLAY_SPEECH 1u`
Play Speech payload size: 1 byte Payload (excludes 1 length header byte, 1 byte Checksum)

20.32.3.21 DUCK_IO_REPLY_NO_CART_IN_SLOT `#define DUCK_IO_REPLY_NO_CART_IN_SLOT 06u`

20.32.3.22 DUCK_IO_LEN_RX_MAX `#define DUCK_IO_LEN_RX_MAX 14u`

20.32.3.23 DUCK_IO_LEN_TX_MAX `#define DUCK_IO_LEN_TX_MAX 14u`

20.32.3.24 DUCK_IO_TIMEOUT_2_MSEC `#define DUCK_IO_TIMEOUT_2_MSEC 2u`

20.32.3.25 DUCK_IO_TIMEOUT_100_MSEC `#define DUCK_IO_TIMEOUT_100_MSEC 100u`

20.32.3.26 DUCK_IO_TIMEOUT_200_MSEC `#define DUCK_IO_TIMEOUT_200_MSEC 200u`

20.32.3.27 DUCK_IO_SPEECH_CMD_MIN `#define DUCK_IO_SPEECH_CMD_MIN 1`

20.32.3.28 DUCK_IO_SPEECH_CMD_MAX `#define DUCK_IO_SPEECH_CMD_MAX 6`

20.32.3.29 DUCK_IO_RTC_YEAR `#define DUCK_IO_RTC_YEAR 0u`

20.32.3.30 DUCK_IO_RTC_MON `#define DUCK_IO_RTC_MON 1u`

20.32.3.31 DUCK_IO_RTC_DAY `#define DUCK_IO_RTC_DAY 2u`

20.32.3.32 DUCK_IO_RTC_WEEKDAY `#define DUCK_IO_RTC_WEEKDAY 3u`

20.32.3.33 DUCK_IO_RTC_AMPM `#define DUCK_IO_RTC_AMPM 4u`

20.32.3.34 DUCK_IO_RTC_HOUR `#define DUCK_IO_RTC_HOUR 5u`

20.32.3.35 DUCK_IO_RTC_MIN `#define DUCK_IO_RTC_MIN 6u`

20.32.3.36 DUCK_IO_RTC_SEC `#define DUCK_IO_RTC_SEC 7u`

20.32.3.37 DUCK_IO_KBD_FLAGS `#define DUCK_IO_KBD_FLAGS 0u`

20.32.3.38 DUCK_IO_KBD_KEYCODE `#define DUCK_IO_KBD_KEYCODE 1u`

20.32.3.39 DUCK_IO_PRINTER_FAIL `#define DUCK_IO_PRINTER_FAIL 0x00u`

20.32.3.40 DUCK_IO_PRINTER_TYPE_2_PASS `#define DUCK_IO_PRINTER_TYPE_2_PASS 0x01u`

20.32.3.41 DUCK_IO_PRINTER_TYPE_1_PASS `#define DUCK_IO_PRINTER_TYPE_1_PASS 0x02u`

20.32.3.42 DUCK_IO_PRINTER_MAYBE_BUSY `#define DUCK_IO_PRINTER_MAYBE_BUSY 0x03u`

20.32.4 Function Documentation

20.32.4.1 duck_io_send_byte() `void duck_io_send_byte (`
`uint8_t tx_byte )`

Sends a byte over serial to the MegaDuck laptop peripheral

Parameters

<code>tx_byte</code>	Byte to send
----------------------	--------------

20.32.4.2 duck_io_read_byte_no_timeout() `uint8_t duck_io_read_byte_no_timeout ( void )`

Reads a byte over serial from the MegaDuck laptop peripheral with NO timeout

Returns: the received byte

If there is no reply then it will hang forever

20.32.4.3 duck_io_enable_read_byte() `void duck_io_enable_read_byte ( void )`

Prepares to receive serial data from the MegaDuck laptop peripheral

- Sets serial IO to external clock and enables ready state.
- Turns on Serial interrupt, clears any pending interrupts and then turns interrupts on (state of [IE_REG](#) should be preserved before calling this and then restored at the end of the serial communication being performed).

20.32.4.4 duck_io_laptop_init() `bool duck_io_laptop_init ( void )`

Performs init sequence over serial with the MegaDuck laptop peripheral

Returns `true` if successful, otherwise `false`

Needs to be done *just once* any time system is powered on or a cartridge is booted.

Sends count up sequence + some commands, then waits for a matching count down sequence in reverse.

20.32.4.5 duck_io_printer_last_status() `uint8_t duck_io_printer_last_status ( void )`

Returns status of MegaDuck Printer as last detected by [duck_io_laptop_init\(\)](#) or [duck_io_printer_query\(\)](#)

Should be called immediately before trying to print

Returned unsigned 8 bit value will have Printer Type and Status in bits 1..0

The resulting value will be cached and used for any subsequent [duck_io_printer_last_status\(\)](#) calls.

[duck_io_laptop_init\(\)](#) must be called first

See also

[DUCK_IO_PRINTER_FAIL](#), [DUCK_IO_PRINTER_TYPE_2_PASS](#), [DUCK_IO_PRINTER_TYPE_1_PASS](#),
[DUCK_IO_PRINTER_MAYBE_BUSY](#)

20.32.4.6 duck_io_printer_query() `uint8_t duck_io_printer_query ( void )`

Performs a 3 x printer query serial command and returns raw system printer reply

Should be called immediately before trying to print

Returned unsigned 8 bit value will have Printer Type and Status in bits 1..0

The resulting value will be cached and used for any subsequent [duck_io_printer_last_status\(\)](#) calls.

[duck_io_laptop_init\(\)](#) must be called first

See also

[duck_io_printer_last_status\(\)](#), [DUCK_IO_PRINTER_FAIL](#), [DUCK_IO_PRINTER_TYPE_2_PASS](#), [DUCK_IO_PRINTER_TYPE_1_PASS](#),
[DUCK_IO_PRINTER_MAYBE_BUSY](#)

20.32.4.7 duck_io_read_byte_with_msecs_timeout() `bool duck_io_read_byte_with_msecs_timeout ( uint8_t timeout_len_ms )`

Waits to receive a byte over serial from the MegaDuck laptop peripheral with a timeout

Parameters

<i>timeout_len_ms</i>	Unit size is in msec (100 is about ~ 103 msec or 6.14 frames)
-----------------------	---

Returns:

- `true`: Success, received byte will be in `duck_io_rx_byte` global
- `false`: Read timed out with no reply

20.32.4.8 duck_io_send_byte_and_check_ack_msecs_timeout() `bool` `duck_io_send_byte_and_check_ack_msecs_timeout` (

```

ack_msecs_timeout (
    uint8_t tx_byte,
    uint8_t timeout_len_ms,
    uint8_t expected_reply )

```

Sends a byte over serial to the MegaDuck laptop peripheral and waits for a reply with a timeout

Parameters

<i>tx_byte</i>	Byte to send
<i>timeout_len_ms</i>	Unit size is in msec (100 is about ~ 103 msec or 6.14 frames)
<i>expected_reply</i>	The expected value of the reply byte

Returns:

- `true`: Success
- `false`: if timed out or reply byte didn't match expected value

20.32.4.9 duck_io_send_cmd_and_buffer() `bool` `duck_io_send_cmd_and_buffer` (

```

uint8_t io_cmd )

```

Sends a command and a multi-byte buffer over serial to the MegaDuck laptop peripheral

Parameters

<i>io_cmd</i>	Command byte to send
---------------	----------------------

The data should be pre-loaded into these globals:

- `duck_io_tx_buf` : Buffer with data to send
- `duck_io_tx_buf_len` : Number of bytes to send

Returns: `true` if succeeded

See also

[DUCK_IO_CMD_GET_KEYS](#), [DUCK_IO_CMD_SET_RTC](#)

20.32.4.10 duck_io_send_cmd_and_receive_buffer() `bool` `duck_io_send_cmd_and_receive_buffer` (

```

uint8_t io_cmd )

```

Sends a command and then receives a multi-byte buffer over serial from the MegaDuck laptop peripheral

Parameters

<code>io_cmd</code>	Command byte to send
---------------------	----------------------

If successful, the received data and length will be in these globals:

- `duck_io_rx_buf` : Buffer with received data
- `duck_io_rx_buf_len` : Number of bytes received

Returns: `true` if succeeded, `false` if failed (could be no reply, failed checksum, etc)

See also

[DUCK_IO_CMD_GET_RTC](#)

20.32.5 Variable Documentation

20.32.5.1 `duck_io_rx_byte_done` `volatile bool` `duck_io_rx_byte_done` [extern]

20.32.5.2 `duck_io_rx_byte` `volatile uint8_t` `duck_io_rx_byte` [extern]

20.32.5.3 `duck_io_rx_buf` `uint8_t` `duck_io_rx_buf[DUCK_IO_LEN_RX_MAX]` [extern]

20.32.5.4 `duck_io_rx_buf_len` `uint8_t` `duck_io_rx_buf_len` [extern]

20.32.5.5 `duck_io_tx_buf` `uint8_t` `duck_io_tx_buf[DUCK_IO_LEN_TX_MAX]` [extern]

20.32.5.6 `duck_io_tx_buf_len` `uint8_t` `duck_io_tx_buf_len` [extern]

20.33 laptop_io.h

[Go to the documentation of this file.](#)

```
1 #include <gbdk/platform.h>
2 #include <stdint.h>
3 #include <stdbool.h>
4
26 #ifndef _MEGADUCK_LAPTOP_IO_H
27 #define _MEGADUCK_LAPTOP_IO_H
28
29
30 // Commands sent via serial IO to the Duck laptop peripheral hardware
31 #define DUCK_IO_CMD_INIT_START          0x00u
32 #define DUCK_IO_CMD_GET_KEYS            0x00u
33 #define DUCK_IO_CMD_DONE_OR_OK         0x01u
34 // #define DUCK_IO_CMD_DONE_OR_OK_AND_SOMETHING 0x81u
35 #define DUCK_IO_CMD_ABORT_OR_FAIL      0x04u
36 #define DUCK_IO_CMD_PLAY_SPEECH        0x05u
37 #define DUCK_IO_CMD_RUN_CART_IN_SLOT    0x08u
38 #define DUCK_IO_CMD_PRINT_INIT_EXT_IO   0x09u
39 #define DUCK_IO_CMD_SET_RTC             0x0Bu
40 #define DUCK_IO_CMD_GET_RTC             0x0Cu
41 #define DUCK_IO_CMD_PRINT_SEND_BYTES    0x11u
44 // #define FF60_REG_BEFORE_XFER         0x00u
45 #define DUCK_IO_REPLY_BOOT_UNSET        0x00u
46 #define DUCK_IO_REPLY_BOOT_FAIL         0x01u
47 #define DUCK_IO_REPLY_BUFFER_XFER_OK    0x01u
48 #define DUCK_IO_REPLY_SEND_BUFFER_OK    0x03u
49 // #define DUCK_IO_REPLY_READ_FAIL_MAYBE 0x00u
50 #define DUCK_IO_REPLY_BOOT_OK           0x01u
51
```

```

52 #define DUCK_IO_LEN_KBD_GET          2u
53 #define DUCK_IO_LEN_RTC_GET          8u
54 #define DUCK_IO_LEN_RTC_SET          8u
55 #define DUCK_IO_LEN_PLAY_SPEECH      1u
57 #define DUCK_IO_REPLY_NO_CART_IN_SLOT 06u
58
59 // #define MEGADUCK_KBD_BYTE_1_EXPECT 0x0Eu
60 // #define MEGADUCK_SIO_BOOT_OK       0x01u
61
62 #define DUCK_IO_LEN_RX_MAX            14u // 13 data bytes + 1 checksum byte max reply length?
63 #define DUCK_IO_LEN_TX_MAX            14u // 13 data bytes + 1 checksum byte max reply length?
64
65 #define DUCK_IO_TIMEOUT_2_MSEC         2u // Used for hardware init counter sequence
66 #define DUCK_IO_TIMEOUT_100_MSEC      100u
67 #define DUCK_IO_TIMEOUT_200_MSEC      200u
68
69
70 // Pre-recorded Speech Samples for playback
71 #define DUCK_IO_SPEECH_CMD_MIN 1
72 #define DUCK_IO_SPEECH_CMD_MAX 6
73
74
75 // RTC packet byte ordering (all in BCD format)
76 #define DUCK_IO_RTC_YEAR              0u
77 #define DUCK_IO_RTC_MON               1u
78 #define DUCK_IO_RTC_DAY               2u
79 #define DUCK_IO_RTC_WEEKDAY           3u
80 #define DUCK_IO_RTC_AMPM              4u
81 #define DUCK_IO_RTC_HOUR              5u
82 #define DUCK_IO_RTC_MIN               6u
83 #define DUCK_IO_RTC_SEC               7u
84
85
86 // Keyboard packet byte ordering (all in BCD format)
87 #define DUCK_IO_KBD_FLAGS              0u
88 #define DUCK_IO_KBD_KEYCODE           1u
89
90
91 // Printer init reply related
92 // Init Reply Bits:1..0
93 #define DUCK_IO_PRINTER_FAIL           0x00u
94 #define DUCK_IO_PRINTER_TYPE_2_PASS 0x01u // Bit.1 = 0 // 13 x 12 byte packets + 1 x 5 or 6 byte packet
// (with CR and/or LF)
95 #define DUCK_IO_PRINTER_TYPE_1_PASS 0x02u // Bit.1 = 1 // 3 x 12 byte packets + 118 non-packet bytes
96 #define DUCK_IO_PRINTER_MAYBE_BUSY 0x03u // Maybe indicating that Printer Type 1 is busy?
97
98
99 extern volatile bool    duck_io_rx_byte_done;
100 extern volatile uint8_t duck_io_rx_byte;
101
102
103 // TODO: change these to user supplied buffers?
104 extern uint8_t duck_io_rx_buf[DUCK_IO_LEN_RX_MAX];
105 extern uint8_t duck_io_rx_buf_len;
106
107 extern uint8_t duck_io_tx_buf[DUCK_IO_LEN_TX_MAX];
108 extern uint8_t duck_io_tx_buf_len;
109
110
111 // ===== Low level helper IO functions =====
112
113
114 // TODO: No longer in use(?)
115 //
116 // Waits for a serial transfer to complete with a timeout
117 //
118 // @param timeout_len_ms Unit size is in msec (100 is about ~ 103 msec or 6.14 frames)
119 //
120 // Serial ISR populates status var if anything was received
121 //
122 // void duck_io_wait_done_with_timeout(uint8_t timeout_len_ms);
123
124
125 void duck_io_send_byte(uint8_t tx_byte);
126
127
128 uint8_t duck_io_read_byte_no_timeout(void);
129
130
131 void duck_io_enable_read_byte(void);
132
133
134 bool duck_io_laptop_init(void);
135
136
137 uint8_t duck_io_printer_last_status(void);
138
139
140

```

```

181
195 uint8_t duck_io_printer_query(void);
196
197
198 // ===== Higher level IO functions =====
199
200
209 bool duck_io_read_byte_with_msecs_timeout(uint8_t timeout_len_ms);
210
211
222 bool duck_io_send_byte_and_check_ack_msecs_timeout(uint8_t tx_byte, uint8_t timeout_len_ms, uint8_t
    expected_reply);
223
224
237 bool duck_io_send_cmd_and_buffer(uint8_t io_cmd);
238
239
252 bool duck_io_send_cmd_and_receive_buffer(uint8_t io_cmd);
253
254 #endif // _MEGADUCK_LAPTOP_IO_H

```

20.34 gbdk-lib/include/duck/laptop_keycodes.h File Reference

```

#include <gbdk/platform.h>
#include <stdint.h>
#include <stdbool.h>

```

Macros

- #define [DUCK_IO_KEY_FLAG_KEY_REPEAT](#) 0x01u
- #define [DUCK_IO_KEY_FLAG_KEY_REPEAT_BIT](#) 0x0u
- #define [DUCK_IO_KEY_FLAG_CAPSLOCK](#) 0x02u
- #define [DUCK_IO_KEY_FLAG_CAPSLOCK_BIT](#) 0x1u
- #define [DUCK_IO_KEY_FLAG_SHIFT](#) 0x04u
- #define [DUCK_IO_KEY_FLAG_SHIFT_BIT](#) 0x2u
- #define [DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT](#) 0x08u
- #define [DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT_BIT](#) 0x3u
- #define [DUCK_IO_KEY_BASE_BIT](#) 0x7u
- #define [DUCK_IO_KEY_BASE](#) 0x80u
- #define [DUCK_IO_KEY_F1](#) 0x80u
- #define [DUCK_IO_KEY_F2](#) 0x84u
- #define [DUCK_IO_KEY_F3](#) 0x88u
- #define [DUCK_IO_KEY_F4](#) 0x8Cu
- #define [DUCK_IO_KEY_F5](#) 0x90u
- #define [DUCK_IO_KEY_F6](#) 0x94u
- #define [DUCK_IO_KEY_F7](#) 0x98u
- #define [DUCK_IO_KEY_F8](#) 0x9Cu
- #define [DUCK_IO_KEY_F9](#) 0xA0u
- #define [DUCK_IO_KEY_F10](#) 0xA4u
- #define [DUCK_IO_KEY_F11](#) 0xA8u
- #define [DUCK_IO_KEY_F12](#) 0xACu
- #define [DUCK_IO_KEY_ESCAPE](#) 0x81u
- #define [DUCK_IO_KEY_1](#) 0x85u
- #define [DUCK_IO_KEY_2](#) 0x89u
- #define [DUCK_IO_KEY_3](#) 0x8Du
- #define [DUCK_IO_KEY_4](#) 0x91u
- #define [DUCK_IO_KEY_5](#) 0x95u
- #define [DUCK_IO_KEY_6](#) 0x99u
- #define [DUCK_IO_KEY_7](#) 0x9Du
- #define [DUCK_IO_KEY_8](#) 0xA1u
- #define [DUCK_IO_KEY_9](#) 0xA5u

- `#define DUCK_IO_KEY_0 0xA9u`
- `#define DUCK_IO_KEY_SINGLE_QUOTE 0xADu`
- `#define DUCK_IO_KEY_EXCLAMATION_FLIPPED 0xB1u`
- `#define DUCK_IO_KEY_BACKSPACE 0xB5u`
- `#define DUCK_IO_KEY_HELP 0x82u`
- `#define DUCK_IO_KEY_Q 0x86u`
- `#define DUCK_IO_KEY_W 0x8Au`
- `#define DUCK_IO_KEY_E 0x8Eu`
- `#define DUCK_IO_KEY_R 0x92u`
- `#define DUCK_IO_KEY_T 0x96u`
- `#define DUCK_IO_KEY_Y 0x9Au`
- `#define DUCK_IO_KEY_U 0x9Eu`
- `#define DUCK_IO_KEY_I 0xA2u`
- `#define DUCK_IO_KEY_O 0xA6u`
- `#define DUCK_IO_KEY_P 0xA Au`
- `#define DUCK_IO_KEY_BACKTICK 0xAEu`
- `#define DUCK_IO_KEY_RIGHT_SQ_BRACKET 0xB2u`
- `#define DUCK_IO_KEY_ENTER 0xB6u`
- `#define DUCK_IO_KEY_A 0x87u`
- `#define DUCK_IO_KEY_S 0x8Bu`
- `#define DUCK_IO_KEY_D 0x8Fu`
- `#define DUCK_IO_KEY_F 0x93u`
- `#define DUCK_IO_KEY_G 0x97u`
- `#define DUCK_IO_KEY_H 0x9Bu`
- `#define DUCK_IO_KEY_J 0x9Fu`
- `#define DUCK_IO_KEY_K 0xA3u`
- `#define DUCK_IO_KEY_L 0xA7u`
- `#define DUCK_IO_KEY_N_TILDE 0xABu`
- `#define DUCK_IO_KEY_U_umlaut 0xAFu`
- `#define DUCK_IO_KEY_O_OVER_LINE 0xB3u`
- `#define DUCK_IO_KEY_Z 0xB8u`
- `#define DUCK_IO_KEY_X 0xBCu`
- `#define DUCK_IO_KEY_C 0xC0u`
- `#define DUCK_IO_KEY_V 0xC4u`
- `#define DUCK_IO_KEY_B 0xC8u`
- `#define DUCK_IO_KEY_N 0xCCu`
- `#define DUCK_IO_KEY_M 0xD0u`
- `#define DUCK_IO_KEY_COMMA 0xD4u`
- `#define DUCK_IO_KEY_PERIOD 0xD8u`
- `#define DUCK_IO_KEY_DASH 0xDCu`
- `#define DUCK_IO_KEY_DELETE 0xE0u`
- `#define DUCK_IO_KEY_SPACE 0xB9u`
- `#define DUCK_IO_KEY_LESS_THAN 0xBDu`
- `#define DUCK_IO_KEY_PAGE_UP 0xC1u`
- `#define DUCK_IO_KEY_PAGE_DOWN 0xC5u`
- `#define DUCK_IO_KEY_MEMORY_MINUS 0xC9u`
- `#define DUCK_IO_KEY_MEMORY_PLUS 0xCDu`
- `#define DUCK_IO_KEY_MEMORY_RECALL 0xD1u`
- `#define DUCK_IO_KEY_SQUAREROOT 0xD5u`
- `#define DUCK_IO_KEY_MULTIPLY 0xD9u`
- `#define DUCK_IO_KEY_ARROW_DOWN 0xDDu`
- `#define DUCK_IO_KEY_MINUS 0xE1u`
- `#define DUCK_IO_KEY_ARROW_LEFT 0xE5u`
- `#define DUCK_IO_KEY_EQUALS 0xE9u`
- `#define DUCK_IO_KEY_ARROW_RIGHT 0xEDu`

- `#define DUCK_IO_KEY_DIVIDE 0xE4u`
- `#define DUCK_IO_KEY_ARROW_UP 0xE8u`
- `#define DUCK_IO_KEY_PLUS 0xECu`
- `#define DUCK_IO_KEY_PIANO_DO_SHARP 0xBAu`
- `#define DUCK_IO_KEY_PIANO_RE_SHARP 0xBEu`
- `#define DUCK_IO_KEY_PIANO_FA_SHARP 0xC6u`
- `#define DUCK_IO_KEY_PIANO_SOL_SHARP 0xCAu`
- `#define DUCK_IO_KEY_PIANO_LA_SHARP 0xCEu`
- `#define DUCK_IO_KEY_PIANO_DO_2_SHARP 0xD6u`
- `#define DUCK_IO_KEY_PIANO_RE_2_SHARP 0xDAu`
- `#define DUCK_IO_KEY_PRINTSCREEN_RIGHT 0xDEu`
- `#define DUCK_IO_KEY_PIANO_FA_2_SHARP 0xE2u`
- `#define DUCK_IO_KEY_PIANO_SOL_2_SHARP 0xE6u`
- `#define DUCK_IO_KEY_PIANO_LA_2_SHARP 0xEAu`
- `#define DUCK_IO_KEY_PIANO_DO 0xBBu`
- `#define DUCK_IO_KEY_PIANO_RE 0xBFu`
- `#define DUCK_IO_KEY_PIANO_MI 0xC3u`
- `#define DUCK_IO_KEY_PIANO_FA 0xC7u`
- `#define DUCK_IO_KEY_PIANO_SOL 0xCB u`
- `#define DUCK_IO_KEY_PIANO_LA 0xCFu`
- `#define DUCK_IO_KEY_PIANO_SI 0xD3u`
- `#define DUCK_IO_KEY_PIANO_DO_2 0xD7u`
- `#define DUCK_IO_KEY_PIANO_RE_2 0xDBu`
- `#define DUCK_IO_KEY_PIANO_MI_2 0xDFu`
- `#define DUCK_IO_KEY_PIANO_FA_2 0xE3u`
- `#define DUCK_IO_KEY_PIANO_SOL_2 0xE7u`
- `#define DUCK_IO_KEY_PIANO_LA_2 0xEBu`
- `#define DUCK_IO_KEY_PIANO_SI_2 0xEFu`
- `#define DUCK_IO_KEY_LAST_KEY (DUCK_IO_KEY_PIANO_SI_2u)`
- `#define DUCK_IO_KEY_MAYBE_SYST_CODES_START 0xF0u`
- `#define DUCK_IO_KEY_MAYBE_RX_NOT_A_KEY 0xF6u`

20.34.1 Macro Definition Documentation

20.34.1.1 DUCK_IO_KEY_FLAG_KEY_REPEAT `#define DUCK_IO_KEY_FLAG_KEY_REPEAT 0x01u`

20.34.1.2 DUCK_IO_KEY_FLAG_KEY_REPEAT_BIT `#define DUCK_IO_KEY_FLAG_KEY_REPEAT_BIT 0x0u`

20.34.1.3 DUCK_IO_KEY_FLAG_CAPSLOCK `#define DUCK_IO_KEY_FLAG_CAPSLOCK 0x02u`

20.34.1.4 DUCK_IO_KEY_FLAG_CAPSLOCK_BIT `#define DUCK_IO_KEY_FLAG_CAPSLOCK_BIT 0x1u`

20.34.1.5 DUCK_IO_KEY_FLAG_SHIFT `#define DUCK_IO_KEY_FLAG_SHIFT 0x04u`

20.34.1.6 DUCK_IO_KEY_FLAG_SHIFT_BIT `#define DUCK_IO_KEY_FLAG_SHIFT_BIT 0x2u`

20.34.1.7 DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT #define DUCK_IO_KEY_FLAG_PRINTSCREEN_↵
LEFT 0x08u

20.34.1.8 DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT_BIT #define DUCK_IO_KEY_FLAG_PRINTSCREEN_↵
LEFT_BIT 0x3u

20.34.1.9 DUCK_IO_KEY_BASE_BIT #define DUCK_IO_KEY_BASE_BIT 0x7u

20.34.1.10 DUCK_IO_KEY_BASE #define DUCK_IO_KEY_BASE 0x80u

20.34.1.11 DUCK_IO_KEY_F1 #define DUCK_IO_KEY_F1 0x80u

20.34.1.12 DUCK_IO_KEY_F2 #define DUCK_IO_KEY_F2 0x84u

20.34.1.13 DUCK_IO_KEY_F3 #define DUCK_IO_KEY_F3 0x88u

20.34.1.14 DUCK_IO_KEY_F4 #define DUCK_IO_KEY_F4 0x8Cu

20.34.1.15 DUCK_IO_KEY_F5 #define DUCK_IO_KEY_F5 0x90u

20.34.1.16 DUCK_IO_KEY_F6 #define DUCK_IO_KEY_F6 0x94u

20.34.1.17 DUCK_IO_KEY_F7 #define DUCK_IO_KEY_F7 0x98u

20.34.1.18 DUCK_IO_KEY_F8 #define DUCK_IO_KEY_F8 0x9Cu

20.34.1.19 DUCK_IO_KEY_F9 #define DUCK_IO_KEY_F9 0xA0u

20.34.1.20 DUCK_IO_KEY_F10 #define DUCK_IO_KEY_F10 0xA4u

20.34.1.21 DUCK_IO_KEY_F11 #define DUCK_IO_KEY_F11 0xA8u

20.34.1.22 DUCK_IO_KEY_F12 #define DUCK_IO_KEY_F12 0xACu

20.34.1.23 DUCK_IO_KEY_ESCAPE #define DUCK_IO_KEY_ESCAPE 0x81u

20.34.1.24 DUCK_IO_KEY_1 `#define DUCK_IO_KEY_1 0x85u`

20.34.1.25 DUCK_IO_KEY_2 `#define DUCK_IO_KEY_2 0x89u`

20.34.1.26 DUCK_IO_KEY_3 `#define DUCK_IO_KEY_3 0x8Du`

20.34.1.27 DUCK_IO_KEY_4 `#define DUCK_IO_KEY_4 0x91u`

20.34.1.28 DUCK_IO_KEY_5 `#define DUCK_IO_KEY_5 0x95u`

20.34.1.29 DUCK_IO_KEY_6 `#define DUCK_IO_KEY_6 0x99u`

20.34.1.30 DUCK_IO_KEY_7 `#define DUCK_IO_KEY_7 0x9Du`

20.34.1.31 DUCK_IO_KEY_8 `#define DUCK_IO_KEY_8 0xA1u`

20.34.1.32 DUCK_IO_KEY_9 `#define DUCK_IO_KEY_9 0xA5u`

20.34.1.33 DUCK_IO_KEY_0 `#define DUCK_IO_KEY_0 0xA9u`

20.34.1.34 DUCK_IO_KEY_SINGLE_QUOTE `#define DUCK_IO_KEY_SINGLE_QUOTE 0xADu`

20.34.1.35 DUCK_IO_KEY_EXCLAMATION_FLIPPED `#define DUCK_IO_KEY_EXCLAMATION_FLIPPED 0xB1u`

20.34.1.36 DUCK_IO_KEY_BACKSPACE `#define DUCK_IO_KEY_BACKSPACE 0xB5u`

20.34.1.37 DUCK_IO_KEY_HELP `#define DUCK_IO_KEY_HELP 0x82u`

20.34.1.38 DUCK_IO_KEY_Q `#define DUCK_IO_KEY_Q 0x86u`

20.34.1.39 DUCK_IO_KEY_W `#define DUCK_IO_KEY_W 0x8Au`

20.34.1.40 DUCK_IO_KEY_E `#define DUCK_IO_KEY_E 0x8Eu`

20.34.1.41 DUCK_IO_KEY_R `#define DUCK_IO_KEY_R 0x92u`

20.34.1.42 DUCK_IO_KEY_T `#define DUCK_IO_KEY_T 0x96u`

20.34.1.43 DUCK_IO_KEY_Y `#define DUCK_IO_KEY_Y 0x9Au`

20.34.1.44 DUCK_IO_KEY_U `#define DUCK_IO_KEY_U 0x9Eu`

20.34.1.45 DUCK_IO_KEY_I `#define DUCK_IO_KEY_I 0xA2u`

20.34.1.46 DUCK_IO_KEY_O `#define DUCK_IO_KEY_O 0xA6u`

20.34.1.47 DUCK_IO_KEY_P `#define DUCK_IO_KEY_P 0xAAu`

20.34.1.48 DUCK_IO_KEY_BACKTICK `#define DUCK_IO_KEY_BACKTICK 0xAEu`

20.34.1.49 DUCK_IO_KEY_RIGHT_SQ_BRACKET `#define DUCK_IO_KEY_RIGHT_SQ_BRACKET 0xB2u`

20.34.1.50 DUCK_IO_KEY_ENTER `#define DUCK_IO_KEY_ENTER 0xB6u`

20.34.1.51 DUCK_IO_KEY_A `#define DUCK_IO_KEY_A 0x87u`

20.34.1.52 DUCK_IO_KEY_S `#define DUCK_IO_KEY_S 0x8Bu`

20.34.1.53 DUCK_IO_KEY_D `#define DUCK_IO_KEY_D 0x8Fu`

20.34.1.54 DUCK_IO_KEY_F `#define DUCK_IO_KEY_F 0x93u`

20.34.1.55 DUCK_IO_KEY_G `#define DUCK_IO_KEY_G 0x97u`

20.34.1.56 DUCK_IO_KEY_H `#define DUCK_IO_KEY_H 0x9Bu`

20.34.1.57 DUCK_IO_KEY_J `#define DUCK_IO_KEY_J 0x9Fu`

20.34.1.58 DUCK_IO_KEY_K `#define DUCK_IO_KEY_K 0xA3u`

20.34.1.59 DUCK_IO_KEY_L `#define DUCK_IO_KEY_L 0xA7u`

20.34.1.60 DUCK_IO_KEY_N_TILDE `#define DUCK_IO_KEY_N_TILDE 0xABu`

20.34.1.61 DUCK_IO_KEY_U_UMLAUT `#define DUCK_IO_KEY_U_UMLAUT 0xAFu`

20.34.1.62 DUCK_IO_KEY_O_OVER_LINE `#define DUCK_IO_KEY_O_OVER_LINE 0xB3u`

20.34.1.63 DUCK_IO_KEY_Z `#define DUCK_IO_KEY_Z 0xB8u`

20.34.1.64 DUCK_IO_KEY_X `#define DUCK_IO_KEY_X 0xBCu`

20.34.1.65 DUCK_IO_KEY_C `#define DUCK_IO_KEY_C 0xC0u`

20.34.1.66 DUCK_IO_KEY_V `#define DUCK_IO_KEY_V 0xC4u`

20.34.1.67 DUCK_IO_KEY_B `#define DUCK_IO_KEY_B 0xC8u`

20.34.1.68 DUCK_IO_KEY_N `#define DUCK_IO_KEY_N 0xCCu`

20.34.1.69 DUCK_IO_KEY_M `#define DUCK_IO_KEY_M 0xD0u`

20.34.1.70 DUCK_IO_KEY_COMMA `#define DUCK_IO_KEY_COMMA 0xD4u`

20.34.1.71 DUCK_IO_KEY_PERIOD `#define DUCK_IO_KEY_PERIOD 0xD8u`

20.34.1.72 DUCK_IO_KEY_DASH `#define DUCK_IO_KEY_DASH 0xDCu`

20.34.1.73 DUCK_IO_KEY_DELETE `#define DUCK_IO_KEY_DELETE 0xE0u`

20.34.1.74 DUCK_IO_KEY_SPACE `#define DUCK_IO_KEY_SPACE 0xB9u`

20.34.1.75 DUCK_IO_KEY_LESS_THAN `#define DUCK_IO_KEY_LESS_THAN 0xBDu`

20.34.1.76 DUCK_IO_KEY_PAGE_UP `#define DUCK_IO_KEY_PAGE_UP 0xC1u`

20.34.1.77 DUCK_IO_KEY_PAGE_DOWN `#define DUCK_IO_KEY_PAGE_DOWN 0xC5u`

20.34.1.78 DUCK_IO_KEY_MEMORY_MINUS #define DUCK_IO_KEY_MEMORY_MINUS 0xC9u

20.34.1.79 DUCK_IO_KEY_MEMORY_PLUS #define DUCK_IO_KEY_MEMORY_PLUS 0xCDu

20.34.1.80 DUCK_IO_KEY_MEMORY_RECALL #define DUCK_IO_KEY_MEMORY_RECALL 0xD1u

20.34.1.81 DUCK_IO_KEY_SQUAREROOT #define DUCK_IO_KEY_SQUAREROOT 0xD5u

20.34.1.82 DUCK_IO_KEY_MULTIPLY #define DUCK_IO_KEY_MULTIPLY 0xD9u

20.34.1.83 DUCK_IO_KEY_ARROW_DOWN #define DUCK_IO_KEY_ARROW_DOWN 0xDDu

20.34.1.84 DUCK_IO_KEY_MINUS #define DUCK_IO_KEY_MINUS 0xE1u

20.34.1.85 DUCK_IO_KEY_ARROW_LEFT #define DUCK_IO_KEY_ARROW_LEFT 0xE5u

20.34.1.86 DUCK_IO_KEY_EQUALS #define DUCK_IO_KEY_EQUALS 0xE9u

20.34.1.87 DUCK_IO_KEY_ARROW_RIGHT #define DUCK_IO_KEY_ARROW_RIGHT 0xEDu

20.34.1.88 DUCK_IO_KEY_DIVIDE #define DUCK_IO_KEY_DIVIDE 0xE4u

20.34.1.89 DUCK_IO_KEY_ARROW_UP #define DUCK_IO_KEY_ARROW_UP 0xE8u

20.34.1.90 DUCK_IO_KEY_PLUS #define DUCK_IO_KEY_PLUS 0xECu

20.34.1.91 DUCK_IO_KEY_PIANO_DO_SHARP #define DUCK_IO_KEY_PIANO_DO_SHARP 0xBAu

20.34.1.92 DUCK_IO_KEY_PIANO_RE_SHARP #define DUCK_IO_KEY_PIANO_RE_SHARP 0xBEu

20.34.1.93 DUCK_IO_KEY_PIANO_FA_SHARP #define DUCK_IO_KEY_PIANO_FA_SHARP 0xC6u

20.34.1.94 DUCK_IO_KEY_PIANO_SOL_SHARP #define DUCK_IO_KEY_PIANO_SOL_SHARP 0xCAu

20.34.1.95 DUCK_IO_KEY_PIANO_LA_SHARP #define DUCK_IO_KEY_PIANO_LA_SHARP 0xCEu

- 20.34.1.96 DUCK_IO_KEY_PIANO_DO_2_SHARP** `#define DUCK_IO_KEY_PIANO_DO_2_SHARP 0xD6u`
- 20.34.1.97 DUCK_IO_KEY_PIANO_RE_2_SHARP** `#define DUCK_IO_KEY_PIANO_RE_2_SHARP 0xDAu`
- 20.34.1.98 DUCK_IO_KEY_PRINTSCREEN_RIGHT** `#define DUCK_IO_KEY_PRINTSCREEN_RIGHT 0xDEu`
- 20.34.1.99 DUCK_IO_KEY_PIANO_FA_2_SHARP** `#define DUCK_IO_KEY_PIANO_FA_2_SHARP 0xE2u`
- 20.34.1.100 DUCK_IO_KEY_PIANO_SOL_2_SHARP** `#define DUCK_IO_KEY_PIANO_SOL_2_SHARP 0xE6u`
- 20.34.1.101 DUCK_IO_KEY_PIANO_LA_2_SHARP** `#define DUCK_IO_KEY_PIANO_LA_2_SHARP 0xEAu`
- 20.34.1.102 DUCK_IO_KEY_PIANO_DO** `#define DUCK_IO_KEY_PIANO_DO 0xBBu`
- 20.34.1.103 DUCK_IO_KEY_PIANO_RE** `#define DUCK_IO_KEY_PIANO_RE 0xBFu`
- 20.34.1.104 DUCK_IO_KEY_PIANO_MI** `#define DUCK_IO_KEY_PIANO_MI 0xC3u`
- 20.34.1.105 DUCK_IO_KEY_PIANO_FA** `#define DUCK_IO_KEY_PIANO_FA 0xC7u`
- 20.34.1.106 DUCK_IO_KEY_PIANO_SOL** `#define DUCK_IO_KEY_PIANO_SOL 0xCBu`
- 20.34.1.107 DUCK_IO_KEY_PIANO_LA** `#define DUCK_IO_KEY_PIANO_LA 0xCFu`
- 20.34.1.108 DUCK_IO_KEY_PIANO_SI** `#define DUCK_IO_KEY_PIANO_SI 0xD3u`
- 20.34.1.109 DUCK_IO_KEY_PIANO_DO_2** `#define DUCK_IO_KEY_PIANO_DO_2 0xD7u`
- 20.34.1.110 DUCK_IO_KEY_PIANO_RE_2** `#define DUCK_IO_KEY_PIANO_RE_2 0xDBu`
- 20.34.1.111 DUCK_IO_KEY_PIANO_MI_2** `#define DUCK_IO_KEY_PIANO_MI_2 0xDFu`
- 20.34.1.112 DUCK_IO_KEY_PIANO_FA_2** `#define DUCK_IO_KEY_PIANO_FA_2 0xE3u`
- 20.34.1.113 DUCK_IO_KEY_PIANO_SOL_2** `#define DUCK_IO_KEY_PIANO_SOL_2 0xE7u`

20.34.1.114 DUCK_IO_KEY_PIANO_LA_2 #define DUCK_IO_KEY_PIANO_LA_2 0xEBu

20.34.1.115 DUCK_IO_KEY_PIANO_SI_2 #define DUCK_IO_KEY_PIANO_SI_2 0xEFu

20.34.1.116 DUCK_IO_KEY_LAST_KEY #define DUCK_IO_KEY_LAST_KEY (DUCK_IO_KEY_PIANO_SI_2u)

20.34.1.117 DUCK_IO_KEY_MAYBE_SYST_CODES_START #define DUCK_IO_KEY_MAYBE_SYST_CODES_↵
START 0xF0u

20.34.1.118 DUCK_IO_KEY_MAYBE_RX_NOT_A_KEY #define DUCK_IO_KEY_MAYBE_RX_NOT_A_KEY 0xF6u

20.35 laptop_keycodes.h

[Go to the documentation of this file.](#)

```
1 #include <gbdk/platform.h>
2 #include <stdint.h>
3 #include <stdbool.h>
4
5 #ifndef _MEGADUCK_LAPTOP_KEYCODES_H
6 #define _MEGADUCK_LAPTOP_KEYCODES_H
7
8
9 // - Left /right shift are shared
10 //
11 // Keyboard serial reply scan codes have different ordering than System ROM character codes
12 // - They go diagonal down from upper left for the first *4* rows
13 // - The bottom 4 rows (including piano keys) are more varied
14
15 // Modifier Keys / Flags
16 //
17 // See input_key_modifier_flags__RAM_D027_
18 #define DUCK_IO_KEY_FLAG_KEY_REPEAT 0x01u
19 #define DUCK_IO_KEY_FLAG_KEY_REPEAT_BIT 0x0u
20 #define DUCK_IO_KEY_FLAG_CAPSLOCK 0x02u
21 #define DUCK_IO_KEY_FLAG_CAPSLOCK_BIT 0x1u
22 #define DUCK_IO_KEY_FLAG_SHIFT 0x04u
23 #define DUCK_IO_KEY_FLAG_SHIFT_BIT 0x2u
24 // Right Print Screen has actual scancode vs Left being in a flag
25 #define DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT 0x08u
26 #define DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT_BIT 0x3u
27
28 // Keyboard scancodes
29 // All valid keys seem to have bit 7 set (0x80+)
30 #define DUCK_IO_KEY_BASE_BIT 0x7u
31 #define DUCK_IO_KEY_BASE 0x80u
32
33
34 // First 4 rows (top of keyboard) ~ 0x80 - 0xB7
35 //
36 // - For each row, most chars are +4 vs char to immediate left
37 //   due to the diagonal down-right scancode ordering
38 //
39 // Starting values
40 // - Row 1: 0x80
41 // - Row 2: 0x81
42 // - Row 3: 0x82
43 // - Row 4: 0x83
44
45 // Row 1
46 #define DUCK_IO_KEY_F1 0x80u
47 #define DUCK_IO_KEY_F2 0x84u
48 #define DUCK_IO_KEY_F3 0x88u
49 #define DUCK_IO_KEY_F4 0x8Cu
50 #define DUCK_IO_KEY_F5 0x90u
51 #define DUCK_IO_KEY_F6 0x94u
52 #define DUCK_IO_KEY_F7 0x98u
53 #define DUCK_IO_KEY_F8 0x9Cu
54 #define DUCK_IO_KEY_F9 0xA0u
55 #define DUCK_IO_KEY_F10 0xA4u
56 #define DUCK_IO_KEY_F11 0xA8u
57 #define DUCK_IO_KEY_F12 0xACu
58 // GAP at 0xB0 maybe Blank spot where F13 would be
59 // GAP at 0xB4 maybe ON Key?
```



```

60
61 // Row 2
62 #define DUCK_IO_KEY_ESCAPE          0x81u // Spanish label: Salida | German label: Esc
63 #define DUCK_IO_KEY_1              0x85u // Shift alt: !
64 #define DUCK_IO_KEY_2              0x89u // Shift alt: "
65 #define DUCK_IO_KEY_3              0x8Du // Shift alt: ` (Spanish, mid-dot) | $ (German, legal
        section)
66 #define DUCK_IO_KEY_4              0x91u // Shift alt: $
67 #define DUCK_IO_KEY_5              0x95u // Shift alt: %
68 #define DUCK_IO_KEY_6              0x99u // Shift alt: &
69 #define DUCK_IO_KEY_7              0x9Du // Shift alt: /
70 #define DUCK_IO_KEY_8              0xA1u // Shift alt: (
71 #define DUCK_IO_KEY_9              0xA5u // Shift alt: )
72 #define DUCK_IO_KEY_0              0xA9u // Shift alt: "\ "
73 #define DUCK_IO_KEY_SINGLE_QUOTE  0xADu // Shift alt: ? | German version: ß (eszett)
74 #define DUCK_IO_KEY_EXCLAMATION_FLIPPED 0xB1u // Shift alt: ¿ (Spanish) | ` (German) // German version:
        ' (single quote?)
75 #define DUCK_IO_KEY_BACKSPACE      0xB5u // German label: Löschen
76 // See Continued Row 2 below
77
78 // Row 3
79 #define DUCK_IO_KEY_HELP            0x82u // Spanish label: Ayuda | German label: Hilfe
80 #define DUCK_IO_KEY_Q              0x86u
81 #define DUCK_IO_KEY_W              0x8Au
82 #define DUCK_IO_KEY_E              0x8Eu
83 #define DUCK_IO_KEY_R              0x92u
84 #define DUCK_IO_KEY_T              0x96u
85 #define DUCK_IO_KEY_Y              0x9Au // German version: z
86 #define DUCK_IO_KEY_U              0x9Eu
87 #define DUCK_IO_KEY_I              0xA2u
88 #define DUCK_IO_KEY_O              0xA6u
89 #define DUCK_IO_KEY_P              0xAAu
90 #define DUCK_IO_KEY_BACKTICK      0xAEu // Shift alt: [ (Spanish, only shift mode works) | German
        version: Ü
91 #define DUCK_IO_KEY_RIGHT_SQ_BRACKET 0xB2u // Shift alt: * | German version: ` (mid-dot)
92 #define DUCK_IO_KEY_ENTER          0xB6u // Spanish label: Entra | German label: Ein-gabe
93 // See Continued Row 3 below
94
95 // Row 4
96 // GAP at 0x83 maybe CAPS LOCK (Spanish label: Mayuscula, German label: Groß)
97 #define DUCK_IO_KEY_A              0x87u
98 #define DUCK_IO_KEY_S              0x8Bu
99 #define DUCK_IO_KEY_D              0x8Fu
100 #define DUCK_IO_KEY_F              0x93u
101 #define DUCK_IO_KEY_G              0x97u
102 #define DUCK_IO_KEY_H              0x9Bu
103 #define DUCK_IO_KEY_J              0x9Fu
104 #define DUCK_IO_KEY_K              0xA3u
105 #define DUCK_IO_KEY_L              0xA7u
106 #define DUCK_IO_KEY_N_TILDE       0xABu // German version: ö
107 #define DUCK_IO_KEY_U_UMLAUT      0xAFu // German version: ä
108 #define DUCK_IO_KEY_O_OVER_LINE   0xB3u // Shift alt: [A over line] (Spanish) | ^ (German) |
        German version: #
109 // ? GAP at 0x87 ?
110
111
112 // Second 4 rows (bottom of keyboard) ~ 0x80 - 0xB7
113 //
114 // - For each row, most chars are +4 vs char to immediate left
115 //
116 // Starting values
117 // - Row 5: 0xB8
118 // - Row 6: 0xB9
119 // - Row 7: 0xBA
120 // - Row 8: 0xBB
121
122 // Row 5
123 #define DUCK_IO_KEY_Z              0xB8u // German version: y
124 #define DUCK_IO_KEY_X              0xBCu
125 #define DUCK_IO_KEY_C              0xC0u
126 #define DUCK_IO_KEY_V              0xC4u
127 #define DUCK_IO_KEY_B              0xC8u
128 #define DUCK_IO_KEY_N              0xCCu
129 #define DUCK_IO_KEY_M              0xD0u
130 #define DUCK_IO_KEY_COMMA         0xD4u
131 #define DUCK_IO_KEY_PERIOD        0xD8u
132 #define DUCK_IO_KEY_DASH          0xDCu // Shift alt: _ | German version: @
133 // See Continued Row 5 below
134 // Row 6 Continued (from below)
135 #define DUCK_IO_KEY_DELETE         0xE0u // * Spanish label: Borrar | German label: Entf.
136
137
138
139 // Encoding is less orderly below
140
141 // Row 6
142 #define DUCK_IO_KEY_SPACE          0xB9u // Spanish label: Espacio | German label (blank)

```

```

143 // Continued Row 5
144 #define DUCK_IO_KEY_LESS_THAN      0xBDu // Shift alt: >
145 // Continued Row 6
146 #define DUCK_IO_KEY_PAGE_UP        0xC1u // Spanish label: Pg Arriba | German label: Zu-rück
147 #define DUCK_IO_KEY_PAGE_DOWN      0xC5u // Spanish label: Pg Abajo | German label: Wei-ter
148 #define DUCK_IO_KEY_MEMORY_MINUS   0xC9u
149 // Continued Row 5
150 #define DUCK_IO_KEY_MEMORY_PLUS     0xCDu
151 #define DUCK_IO_KEY_MEMORY_RECALL   0xD1u
152 #define DUCK_IO_KEY_SQUAREROOT      0xD5u
153 // ** 3x3 Arrow and Math Key area **
154 // Continued Row 6
155 #define DUCK_IO_KEY_MULTIPLY         0xD9u
156 #define DUCK_IO_KEY_ARROW_DOWN      0xDDu
157 #define DUCK_IO_KEY_MINUS           0xE1u
158 // Continued Row 3
159 #define DUCK_IO_KEY_ARROW_LEFT       0xE5u
160 #define DUCK_IO_KEY_EQUALS           0xE9u
161 #define DUCK_IO_KEY_ARROW_RIGHT     0xEDu
162 // Continued Row 2
163 #define DUCK_IO_KEY_DIVIDE           0xE4u // German version: :
164 #define DUCK_IO_KEY_ARROW_UP         0xE8u
165 #define DUCK_IO_KEY_PLUS             0xECu
166
167 // Row 7
168 // Piano Sharp Keys
169 #define DUCK_IO_KEY_PIANO_DO_SHARP   0xBAu
170 #define DUCK_IO_KEY_PIANO_RE_SHARP   0xBEu
171 // GAP at 0xC2 where there is no key
172 #define DUCK_IO_KEY_PIANO_FA_SHARP   0xC6u
173 #define DUCK_IO_KEY_PIANO_SOL_SHARP  0xCAu
174 #define DUCK_IO_KEY_PIANO_LA_SHARP   0xCEu
175 // GAP at 0xD2 where there is no key
176 //
177 // Octave 2 maybe
178 #define DUCK_IO_KEY_PIANO_DO_2_SHARP  0xD6u
179 #define DUCK_IO_KEY_PIANO_RE_2_SHARP  0xDAu
180 // Row 6 Continued
181 #define DUCK_IO_KEY_PRINTSCREEN_RIGHT 0xDEu // German label: Druck (* Mixed in with piano keys)
182 // Row 7 Continued
183 #define DUCK_IO_KEY_PIANO_FA_2_SHARP  0xE2u
184 #define DUCK_IO_KEY_PIANO_SOL_2_SHARP 0xE6u
185 #define DUCK_IO_KEY_PIANO_LA_2_SHARP  0xEAu
186
187 // Row 8
188 // Piano Primary Keys
189 #define DUCK_IO_KEY_PIANO_DO           0xBBu
190 #define DUCK_IO_KEY_PIANO_RE           0xBFu
191 #define DUCK_IO_KEY_PIANO_MI           0xC3u
192 #define DUCK_IO_KEY_PIANO_FA           0xC7u
193 #define DUCK_IO_KEY_PIANO_SOL          0xCBu
194 #define DUCK_IO_KEY_PIANO_LA           0xCFu
195 #define DUCK_IO_KEY_PIANO_SI           0xD3u
196 #define DUCK_IO_KEY_PIANO_DO_2        0xD7u
197 #define DUCK_IO_KEY_PIANO_RE_2        0xDBu
198 #define DUCK_IO_KEY_PIANO_MI_2        0xDFu
199 #define DUCK_IO_KEY_PIANO_FA_2        0xE3u
200 #define DUCK_IO_KEY_PIANO_SOL_2       0xE7u
201 #define DUCK_IO_KEY_PIANO_LA_2        0xEBu
202 #define DUCK_IO_KEY_PIANO_SI_2        0xEFu
203
204 #define DUCK_IO_KEY_LAST_KEY           (DUCK_IO_KEY_PIANO_SI_2u)
205
206 // Special System Codes? 0xF0+
207 #define DUCK_IO_KEY_MAYBE_SYST_CODES_START 0xF0u
208 #define DUCK_IO_KEY_MAYBE_RX_NOT_A_KEY    0xF6u
209
210 #endif // _MEGADUCK_LAPTOP_KEYCODES_H

```

20.36 gbdk-lib/include/duck/model.h File Reference

```

#include <gbdk/platform.h>
#include <stdint.h>

```

Macros

- #define MEGADUCK_HANDHELD_STANDARD 0u
- #define MEGADUCK_LAPTOP_SPANISH 1u
- #define MEGADUCK_LAPTOP_GERMAN 2u

Functions

- [uint8_t duck_check_model](#) (void)

20.36.1 Macro Definition Documentation

20.36.1.1 MEGADUCK_HANDHELD_STANDARD `#define MEGADUCK_HANDHELD_STANDARD 0u`

20.36.1.2 MEGADUCK_LAPTOP_SPANISH `#define MEGADUCK_LAPTOP_SPANISH 1u`

20.36.1.3 MEGADUCK_LAPTOP_GERMAN `#define MEGADUCK_LAPTOP_GERMAN 2u`

20.36.2 Function Documentation

20.36.2.1 duck_check_model() `uint8_t duck_check_model (void )`

Returns which MegaDuck Model the program is being run on

Possible models are:

- Handheld: [MEGADUCK_HANDHELD_STANDARD](#)
- Spanish Laptop "Super QuiQue": [MEGADUCK_LAPTOP_SPANISH](#)
- German Laptop "Super Junior Computer": [MEGADUCK_LAPTOP_GERMAN](#)

This detection should be called immediately at the start of the program for most reliable results, since it relies on inspecting uncleared VRAM contents.

It works by checking for distinct font VRAM Tile Patterns (which aren't cleared before cart program launch) between the Spanish and German Laptop models which have slightly different character sets.

So VRAM *must not* be cleared or modified at program startup until after this function is called (not by the crt0.s, not by the program itself).

Note

This detection may not work in emulators which don't simulate the preloaded Laptop System ROM font tiles in VRAM.

20.37 model.h

[Go to the documentation of this file.](#)

```
1 #include <gbdk/platform.h>
2 #include <stdint.h>
3
4 #ifndef _MEGADUCK_MODEL_H
5 #define _MEGADUCK_MODEL_H
6
7 #define MEGADUCK_HANDHELD_STANDARD 0u
8 #define MEGADUCK_LAPTOP_SPANISH 1u
9 #define MEGADUCK_LAPTOP_GERMAN 2u
10
11
12
13 uint8_t duck_check_model(void);
14
15 #endif // _MEGADUCK_MODEL_H
```

20.38 gbdk-lib/include/gb/bcd.h File Reference

```
#include <types.h>
#include <stdint.h>
```

Macros

- `#define BCD_HEX(v) ((BCD)(v))`
- `#define MAKE_BCD(v) BCD_HEX(0x ## v)`

Typedefs

- `typedef uint32_t BCD`

Functions

- `void uint2bcd (uint16_t i, BCD *value) OLDCALL`
- `void bcd_add (BCD *sour, const BCD *value) OLDCALL`
- `void bcd_sub (BCD *sour, const BCD *value) OLDCALL`
- `uint8_t bcd2text (const BCD *bcd, uint8_t tile_offset, uint8_t *buffer) OLDCALL`

20.38.1 Detailed Description

Support for working with BCD (Binary Coded Decimal)
See the example BCD project for additional details.

20.38.2 Macro Definition Documentation

20.38.2.1 BCD_HEX `#define BCD_HEX(
v) ((BCD)(v))`

20.38.2.2 MAKE_BCD `#define MAKE_BCD(
v) BCD_HEX(0x ## v)`

Converts an integer value into BCD format
A maximum of 8 digits may be used

20.38.3 Typedef Documentation

20.38.3.1 BCD `typedef uint32_t BCD`

20.38.4 Function Documentation

20.38.4.1 uint2bcd() `void uint2bcd (
uint16_t i,
BCD * value)`

Converts integer *i* into BCD format (Binary Coded Decimal)

Parameters

<i>i</i>	Numeric value to convert
<i>value</i>	Pointer to a BCD variable to store the converted result

20.38.4.2 bcd_add() `void bcd_add (
BCD * sour,`

```
const BCD * value )
```

Adds two numbers in BCD format: **sour** += **value**

Parameters

<i>sour</i>	Pointer to a BCD value to add to (and where the result is stored)
<i>value</i>	Pointer to the BCD value to add to sour

```
20.38.4.3 bcd_sub() void bcd_sub (
    BCD * sour,
    const BCD * value )
```

Subtracts two numbers in BCD format: **sour** -= **value**

Parameters

<i>sour</i>	Pointer to a BCD value to subtract from (and where the result is stored)
<i>value</i>	Pointer to the BCD value to subtract from sour

```
20.38.4.4 bcd2text() uint8_t bcd2text (
    const BCD * bcd,
    uint8_t tile_offset,
    uint8_t * buffer )
```

Convert a BCD number into an ascii (null terminated) string and return the length

Parameters

<i>bcd</i>	Pointer to BCD value to convert
<i>tile_offset</i>	Optional per-character offset value to add (use 0 for none)
<i>buffer</i>	Buffer to store the result in

Returns: Length in characters (always 8)

buffer should be large enough to store the converted string (9 bytes: 8 characters + 1 for terminator)

There are a couple different ways to use **tile_offset**. For example:

- It can be the Index of the Font Tile '0' in VRAM to allow the buffer to be used directly with [set_bkg_tiles](#).
- It can also be set to the ascii value for character '0' so that the buffer is a normal string that can be passed to [printf](#).

20.39 bcd.h

[Go to the documentation of this file.](#)

```
1 #ifndef __BCD_H_INCLUDE
2 #define __BCD_H_INCLUDE
3
4 #include <types.h>
5 #include <stdint.h>
6
13 // macro for creating BCD constants
14 #define BCD_HEX(v) ((BCD)(v))
15
20 #define MAKE_BCD(v) BCD_HEX(0x ## v)
21
22 typedef uint32_t BCD;
23
28 void uint2bcd(uint16_t i, BCD * value) OLDCALL;
29
34 void bcd_add(BCD * sour, const BCD * value) OLDCALL;
35
```

```

40 void bcd_sub(BCD * sour, const BCD * value) OLDCALL;
41
59 uint8_t bcd2text(const BCD * bcd, uint8_t tile_offset, uint8_t * buffer) OLDCALL;
60
61 #endif

```

20.40 gbdk-lib/include/gbdk/bcd.h File Reference

```
#include <gb/bcd.h>
```

20.41 bcd.h

[Go to the documentation of this file.](#)

```

1 #ifndef __GBDK_BCD_H_INCLUDE
2 #define __GBDK_BCD_H_INCLUDE
3
4 #if defined(__TARGET_gb) || defined(__TARGET_ap) || defined(__TARGET_duck)
5     #include <gb/bcd.h>
6 #elif defined(__TARGET_sms) || defined(__TARGET_gg) || defined(__TARGET_msxdos)
7     #include <sms/bcd.h>
8 #elif defined(__TARGET_nes)
9     #include <nes/bcd.h>
10 #else
11     #error Unrecognized port
12 #endif
13
14 #endif

```

20.42 gbdk-lib/include/nes/bcd.h File Reference

```
#include <types.h>
#include <stdint.h>
```

Macros

- #define `BCD_HEX(v)` `((BCD)(v))`
- #define `MAKE_BCD(v)` `BCD_HEX(0x ## v)`

Typedefs

- typedef `uint32_t` `BCD`

Functions

- void `uint2bcd` (`uint16_t` i, `BCD` *value) `OLDCALL`
- void `bcd_add` (`BCD` *sour, const `BCD` *value) `OLDCALL`
- void `bcd_sub` (`BCD` *sour, const `BCD` *value) `OLDCALL`
- `uint8_t` `bcd2text` (const `BCD` *bcd, `uint8_t` tile_offset, `uint8_t` *buffer) `OLDCALL`

20.42.1 Detailed Description

Support for working with BCD (Binary Coded Decimal)
See the example BCD project for additional details.

20.42.2 Macro Definition Documentation

20.42.2.1 BCD_HEX #define `BCD_HEX(`
`v ) ((BCD)(v))`

20.42.2.2 MAKE_BCD `#define MAKE_BCD(
 v) BCD_HEX(0x ## v)`

Converts an integer value into BCD format
A maximum of 8 digits may be used

20.42.3 Typedef Documentation

20.42.3.1 BCD `typedef uint32_t BCD`

20.42.4 Function Documentation

20.42.4.1 uint2bcd() `void uint2bcd (
 uint16_t i,
 BCD * value)`

Converts integer *i* into BCD format (Binary Coded Decimal)

Parameters

<i>i</i>	Numeric value to convert
<i>value</i>	Pointer to a BCD variable to store the converted result

20.42.4.2 bcd_add() `void bcd_add (
 BCD * sour,
 const BCD * value)`

Adds two numbers in BCD format: **sour += value**

Parameters

<i>sour</i>	Pointer to a BCD value to add to (and where the result is stored)
<i>value</i>	Pointer to the BCD value to add to sour

20.42.4.3 bcd_sub() `void bcd_sub (
 BCD * sour,
 const BCD * value)`

Subtracts two numbers in BCD format: **sour -= value**

Parameters

<i>sour</i>	Pointer to a BCD value to subtract from (and where the result is stored)
<i>value</i>	Pointer to the BCD value to subtract from sour

20.42.4.4 bcd2text() `uint8_t bcd2text (
 const BCD * bcd,
 uint8_t tile_offset,
 uint8_t * buffer)`

Convert a BCD number into an ascii (null terminated) string and return the length

Parameters

<i>bcd</i>	Pointer to BCD value to convert
<i>tile_offset</i>	Optional per-character offset value to add (use 0 for none)
<i>buffer</i>	Buffer to store the result in

Returns: Length in characters (always 8)

buffer should be large enough to store the converted string (9 bytes: 8 characters + 1 for terminator)

There are a couple different ways to use **tile_offset**. For example:

- It can be the Index of the Font Tile '0' in VRAM to allow the buffer to be used directly with [set_bkg_tiles](#).
- It can also be set to the ascii value for character '0' so that the buffer is a normal string that can be passed to [printf](#).

20.43 bcd.h

[Go to the documentation of this file.](#)

```

1 #ifndef __BCD_H_INCLUDE
2 #define __BCD_H_INCLUDE
3
4 #include <types.h>
5 #include <stdint.h>
6
13 // macro for creating BCD constants
14 #define BCD_HEX(v) ((BCD)(v))
15
20 #define MAKE_BCD(v) BCD_HEX(0x ## v)
21
22 typedef uint32_t BCD;
23
28 void uint2bcd(uint16_t i, BCD * value) OLDCALL;
29
34 void bcd_add(BCD * sour, const BCD * value) OLDCALL;
35
40 void bcd_sub(BCD * sour, const BCD * value) OLDCALL;
41
59 uint8_t bcd2text(const BCD * bcd, uint8_t tile_offset, uint8_t * buffer) OLDCALL;
60
61 #endif

```

20.44 gbdk-lib/include/sms/bcd.h File Reference

```

#include <types.h>
#include <stdint.h>

```

Macros

- #define [BCD_HEX\(v\)](#) ((BCD)(v))
- #define [MAKE_BCD\(v\)](#) [BCD_HEX\(0x ## v\)](#)

Typedefs

- typedef [uint32_t BCD](#)

Functions

- void [uint2bcd](#) ([uint16_t](#) i, [BCD](#) *value)
- void [bcd_add](#) ([BCD](#) *sour, const [BCD](#) *value)
- void [bcd_sub](#) ([BCD](#) *sour, const [BCD](#) *value)
- [uint8_t](#) [bcd2text](#) (const [BCD](#) *bcd, [uint8_t](#) tile_offset, [uint8_t](#) *buffer)

20.44.1 Detailed Description

Support for working with BCD (Binary Coded Decimal)

See the example BCD project for additional details.

20.44.2 Macro Definition Documentation

20.44.2.1 BCD_HEX `#define BCD_HEX(
 v) ((BCD)(v))`

20.44.2.2 MAKE_BCD `#define MAKE_BCD(
 v) BCD_HEX(0x ## v)`

Converts an integer value into BCD format

A maximum of 8 digits may be used

20.44.3 Typedef Documentation

20.44.3.1 BCD `typedef uint32_t BCD`

20.44.4 Function Documentation

20.44.4.1 uint2bcd() `void uint2bcd (
 uint16_t i,
 BCD * value)`

Converts integer *i* into BCD format (Binary Coded Decimal)

Parameters

<i>i</i>	Numeric value to convert
<i>value</i>	Pointer to a BCD variable to store the converted result

20.44.4.2 bcd_add() `void bcd_add (
 BCD * sour,
 const BCD * value)`

Adds two numbers in BCD format: **sour += value**

Parameters

<i>sour</i>	Pointer to a BCD value to add to (and where the result is stored)
<i>value</i>	Pointer to the BCD value to add to sour

20.44.4.3 bcd_sub() `void bcd_sub (
 BCD * sour,
 const BCD * value)`

Subtracts two numbers in BCD format: **sour -= value**

Parameters

<i>sour</i>	Pointer to a BCD value to subtract from (and where the result is stored)
<i>value</i>	Pointer to the BCD value to subtract from sour

20.44.4.4 bcd2text() `uint8_t bcd2text (`
 `const BCD * bcd,`
 `uint8_t tile_offset,`
 `uint8_t * buffer )`

Convert a BCD number into an asciiz (null terminated) string and return the length

Parameters

<i>bcd</i>	Pointer to BCD value to convert
<i>tile_offset</i>	Optional per-character offset value to add (use 0 for none)
<i>buffer</i>	Buffer to store the result in

Returns: Length in characters (always 8)

buffer should be large enough to store the converted string (9 bytes: 8 characters + 1 for terminator)

There are a couple different ways to use **tile_offset**. For example:

- It can be the Index of the Font Tile '0' in VRAM to allow the buffer to be used directly with [set_bkg_tiles](#).
- It can also be set to the ascii value for character '0' so that the buffer is a normal string that can be passed to [printf](#).

20.45 bcd.h

[Go to the documentation of this file.](#)

```
1 #ifndef __BCD_H_INCLUDE
2 #define __BCD_H_INCLUDE
3
4 #include <types.h>
5 #include <stdint.h>
6
13 // macro for creating BCD constants
14 #define BCD_HEX(v) ((BCD)(v))
15
20 #define MAKE_BCD(v) BCD_HEX(0x ## v)
21
22 typedef uint32_t BCD;
23
28 void uint2bcd(uint16_t i, BCD * value);
29
34 void bcd_add(BCD * sour, const BCD * value);
35
40 void bcd_sub(BCD * sour, const BCD * value);
41
59 uint8_t bcd2text(const BCD * bcd, uint8_t tile_offset, uint8_t * buffer);
60
61 #endif
```

20.46 gbdk-lib/include/gb/bgb_emu.h File Reference

```
#include <gbdk/emu_debug.h>
```

20.46.1 Detailed Description

Shim for legacy use of [bgb_emu.h](#) which has been migrated to [emu_debug.h](#)

See the `emu_debug` example project included with gbdk.

20.47 bgb_emu.h

[Go to the documentation of this file.](#)

```
1
8 #ifndef __BGB_EMU_INCLUDE
9 #define __BGB_EMU_INCLUDE
10
11 #include <gbdk/emu_debug.h>
12
13 #endif
```

20.48 gbdk-lib/include/gb/cgb.h File Reference

```
#include <types.h>
#include <stdint.h>
```

Macros

- #define [RGB](#)(r, g, b) (((uint16_t)((b) & 0x1f) << 10) | ((uint16_t)((g) & 0x1f) << 5) | ((r) & 0x1f))
- #define [RGB8](#)(r, g, b) (((uint16_t)((b) >> 3) & 0x1f) << 10) | ((uint16_t)((g) >> 3) & 0x1f) << 5) | (((r) >> 3) & 0x1f)
- #define [RGBHTML](#)(RGB24bit) ([RGB8](#)((RGB24bit) >> 16) & 0xff), ((RGB24bit) >> 8) & 0xff), ((RGB24bit) & 0xff))
- #define [RGB_RED](#) [RGB](#)(31, 0, 0)
- #define [RGB_DARKRED](#) [RGB](#)(15, 0, 0)
- #define [RGB_GREEN](#) [RGB](#)(0, 31, 0)
- #define [RGB_DARKGREEN](#) [RGB](#)(0, 15, 0)
- #define [RGB_BLUE](#) [RGB](#)(0, 0, 31)
- #define [RGB_DARKBLUE](#) [RGB](#)(0, 0, 15)
- #define [RGB_YELLOW](#) [RGB](#)(31, 31, 0)
- #define [RGB_DARKYELLOW](#) [RGB](#)(21, 21, 0)
- #define [RGB_CYAN](#) [RGB](#)(0, 31, 31)
- #define [RGB_AQUA](#) [RGB](#)(28, 5, 22)
- #define [RGB_PINK](#) [RGB](#)(31, 0, 31)
- #define [RGB_PURPLE](#) [RGB](#)(21, 0, 21)
- #define [RGB_BLACK](#) [RGB](#)(0, 0, 0)
- #define [RGB_DARKGRAY](#) [RGB](#)(10, 10, 10)
- #define [RGB_LIGHTGRAY](#) [RGB](#)(21, 21, 21)
- #define [RGB_WHITE](#) [RGB](#)(31, 31, 31)
- #define [RGB_LIGHTFLESH](#) [RGB](#)(30, 20, 15)
- #define [RGB_BROWN](#) [RGB](#)(10, 10, 0)
- #define [RGB_ORANGE](#) [RGB](#)(30, 20, 0)
- #define [RGB_TEAL](#) [RGB](#)(15, 15, 0)

Typedefs

- typedef [uint16_t](#) [palette_color_t](#)

Functions

- void [set_bkg_palette](#) ([uint8_t](#) first_palette, [uint8_t](#) nb_palettes, const [palette_color_t](#) *rgb_data) [OLDCALL](#)
- void [set_sprite_palette](#) ([uint8_t](#) first_palette, [uint8_t](#) nb_palettes, const [palette_color_t](#) *rgb_data) [OLDCALL](#)
- void [set_bkg_palette_entry](#) ([uint8_t](#) palette, [uint8_t](#) entry, [uint16_t](#) rgb_data) [OLDCALL](#)
- void [set_sprite_palette_entry](#) ([uint8_t](#) palette, [uint8_t](#) entry, [uint16_t](#) rgb_data) [OLDCALL](#)
- void [cpu_slow](#) (void)
- void [cpu_fast](#) (void)
- void [set_default_palette](#) (void)
- void [cgb_compatibility](#) (void)

20.48.1 Detailed Description

Support for the Color GameBoy (CGB).

Enabling CGB features

To unlock and use CGB features and registers you need to change byte 0143h in the cartridge header. Otherwise, the CGB will operate in monochrome "Non CGB" compatibility mode.

- Use a value of **80h** for games that support CGB and monochrome gameboys
(with Lcc: **-Wm-yc**, or makebin directly: **-yc**)

- Use a value of **C0h** for CGB only games.
(with Lcc: **-Wm-yC**, or makebin directly: **-yC**)

See the Pan Docs for more information CGB features.

20.48.2 Macro Definition Documentation

20.48.2.1 RGB `#define RGB(`

```

    r,
    g,
    b ) ((uint16_t) (((b) & 0x1f) << 10) | ((uint16_t) (((g) & 0x1f) << 5)) | ((r) &
0x1f)))

```

Macro to create a CGB palette color entry out of 5-bit color components.

Parameters

<i>r</i>	5-bit Red Component, range 0 - 31 (31 brightest)
<i>g</i>	5-bit Green Component, range 0 - 31 (31 brightest)
<i>b</i>	5-bit Blue Component, range 0 - 31 (31 brightest)

The resulting format is bitpacked BGR-555 in a uint16_t.

See also

[set_bkg_palette\(\)](#), [set_sprite_palette\(\)](#), [RGB8\(\)](#), [RGBHTML\(\)](#)

20.48.2.2 RGB8 `#define RGB8(`

```

    r,
    g,
    b ) (((uint16_t) (((b) >> 3) & 0x1f) << 10)) | ((uint16_t) (((g) >> 3) & 0x1f)
<< 5)) | (((r) >> 3) & 0x1f))

```

Macro to create a CGB palette color entry out of 8-bit color components.

Parameters

<i>r</i>	8-bit Red Component, range 0 - 255 (255 brightest)
<i>g</i>	8-bit Green Component, range 0 - 255 (255 brightest)
<i>b</i>	8-bit Blue Component, range 0 - 255 (255 brightest)

The resulting format is bitpacked BGR-555 in a uint16_t.

The lowest 3 bits of each color component are dropped during conversion.

See also

[set_bkg_palette\(\)](#), [set_sprite_palette\(\)](#), [RGB\(\)](#), [RGBHTML\(\)](#)

20.48.2.3 RGBHTML `#define RGBHTML(`

```

    RGB24bit ) (RGB8(((RGB24bit) >> 16) & 0xff), ((RGB24bit) >> 8) & 0xff), ((RGB24bit)
& 0xff)))

```

Macro to convert a 24 Bit RGB color to a CGB palette color entry.

Parameters

<i>RGB24bit</i>	Bit packed RGB-888 color (0-255 for each color component).
-----------------	--

The resulting format is bitpacked BGR-555 in a `uint16_t`.
The lowest 3 bits of each color component are dropped during conversion.

See also

[set_bkg_palette\(\)](#), [set_sprite_palette\(\)](#), [RGB\(\)](#), [RGB8\(\)](#)

20.48.2.4 RGB_RED `#define RGB_RED RGB(31, 0, 0)`
Common colors based on the EGA default palette.

20.48.2.5 RGB_DARKRED `#define RGB_DARKRED RGB(15, 0, 0)`

20.48.2.6 RGB_GREEN `#define RGB_GREEN RGB( 0, 31, 0)`

20.48.2.7 RGB_DARKGREEN `#define RGB_DARKGREEN RGB( 0, 15, 0)`

20.48.2.8 RGB_BLUE `#define RGB_BLUE RGB( 0, 0, 31)`

20.48.2.9 RGB_DARKBLUE `#define RGB_DARKBLUE RGB( 0, 0, 15)`

20.48.2.10 RGB_YELLOW `#define RGB_YELLOW RGB(31, 31, 0)`

20.48.2.11 RGB_DARKYELLOW `#define RGB_DARKYELLOW RGB(21, 21, 0)`

20.48.2.12 RGB_CYAN `#define RGB_CYAN RGB( 0, 31, 31)`

20.48.2.13 RGB_AQUA `#define RGB_AQUA RGB(28, 5, 22)`

20.48.2.14 RGB_PINK `#define RGB_PINK RGB(31, 0, 31)`

20.48.2.15 RGB_PURPLE `#define RGB_PURPLE RGB(21, 0, 21)`

20.48.2.16 RGB_BLACK `#define RGB_BLACK RGB( 0, 0, 0)`

20.48.2.17 RGB_DARKGRAY `#define RGB_DARKGRAY RGB(10, 10, 10)`

20.48.2.18 RGB_LIGHTGRAY `#define RGB_LIGHTGRAY RGB(21, 21, 21)`

20.48.2.19 RGB_WHITE `#define RGB_WHITE RGB(31, 31, 31)`

20.48.2.20 RGB_LIGHTFLESH `#define RGB_LIGHTFLESH RGB(30, 20, 15)`

20.48.2.21 RGB_BROWN `#define RGB_BROWN RGB(10, 10, 0)`

20.48.2.22 RGB_ORANGE `#define RGB_ORANGE RGB(30, 20, 0)`

20.48.2.23 RGB_TEAL `#define RGB_TEAL RGB(15, 15, 0)`

20.48.3 Typedef Documentation

20.48.3.1 palette_color_t `typedef uint16_t palette_color_t`
16 bit color entry

20.48.4 Function Documentation

20.48.4.1 set_bkg_palette() `void set_bkg_palette (`
 `uint8_t first_palette,`
 `uint8_t nb_palettes,`
 `const palette_color_t * rgb_data )`

Set CGB background palette(s).

Parameters

<i>first_palette</i>	Index of the first palette to write (0-7)
<i>nb_palettes</i>	Number of palettes to write (1-8, max depends on first_palette)
<i>rgb_data</i>	Pointer to source palette data

Writes **nb_palettes** to background palette data starting at **first_palette**, Palette data is sourced from **rgb_data**.

- Each Palette is 8 bytes in size: 4 colors x 2 bytes per palette color entry.
- Each color (4 per palette) is packed as BGR-555 format (1:5:5:5, MSBit [15] is unused).
- Each component (R, G, B) may have values from 0 - 31 (5 bits), 31 is brightest.

See also

[RGB\(\)](#), [set_bkg_palette_entry\(\)](#)

[BKGF_CGB_PAL0](#), [BKGF_CGB_PAL1](#), [BKGF_CGB_PAL2](#), [BKGF_CGB_PAL3](#)

[BKGF_CGB_PAL4](#), [BKGF_CGB_PAL5](#), [BKGF_CGB_PAL6](#), [BKGF_CGB_PAL7](#)

20.48.4.2 set_sprite_palette() void set_sprite_palette (
 uint8_t first_palette,
 uint8_t nb_palettes,
 const palette_color_t * rgb_data)

Set CGB sprite palette(s).

Parameters

<i>first_palette</i>	Index of the first palette to write (0-7)
<i>nb_palettes</i>	Number of palettes to write (1-8, max depends on first_palette)
<i>rgb_data</i>	Pointer to source palette data

Writes **nb_palettes** to sprite palette data starting at **first_palette**, Palette data is sourced from **rgb_data**.

- Each Palette is 8 bytes in size: 4 colors x 2 bytes per palette color entry.
- Each color (4 per palette) is packed as BGR-555 format (1:5:5:5, MSBit [15] is unused).
- Each component (R, G, B) may have values from 0 - 31 (5 bits), 31 is brightest.

See also

[RGB\(\)](#), [set_sprite_palette_entry\(\)](#)
[OAMF_CGB_PAL0](#), [OAMF_CGB_PAL1](#), [OAMF_CGB_PAL2](#), [OAMF_CGB_PAL3](#)
[OAMF_CGB_PAL4](#), [OAMF_CGB_PAL5](#), [OAMF_CGB_PAL6](#), [OAMF_CGB_PAL7](#)

20.48.4.3 set_bkg_palette_entry() void set_bkg_palette_entry (
 uint8_t palette,
 uint8_t entry,
 uint16_t rgb_data)

Sets a single color in the specified CGB background palette.

Parameters

<i>palette</i>	Index of the palette to modify (0-7)
<i>entry</i>	Index of color in palette to modify (0-3)
<i>rgb_data</i>	New color data in BGR 15bpp format.

See also

[set_bkg_palette\(\)](#), [RGB\(\)](#)
[BKGF_CGB_PAL0](#), [BKGF_CGB_PAL1](#), [BKGF_CGB_PAL2](#), [BKGF_CGB_PAL3](#)
[BKGF_CGB_PAL4](#), [BKGF_CGB_PAL5](#), [BKGF_CGB_PAL6](#), [BKGF_CGB_PAL7](#)

20.48.4.4 set_sprite_palette_entry() void set_sprite_palette_entry (
 uint8_t palette,
 uint8_t entry,
 uint16_t rgb_data)

Sets a single color in the specified CGB sprite palette.

Parameters

<i>palette</i>	Index of the palette to modify (0-7)
----------------	--------------------------------------

Parameters

<i>entry</i>	Index of color in palette to modify (0-3)
<i>rgb_data</i>	New color data in BGR 15bpp format.

See also

[set_sprite_palette\(\)](#), [RGB\(\)](#)
[OAMF_CGB_PAL0](#), [OAMF_CGB_PAL1](#), [OAMF_CGB_PAL2](#), [OAMF_CGB_PAL3](#)
[OAMF_CGB_PAL4](#), [OAMF_CGB_PAL5](#), [OAMF_CGB_PAL6](#), [OAMF_CGB_PAL7](#)

20.48.4.5 **cpu_slow()** `void cpu_slow (` `void )`

Set CPU speed to slow (Normal Speed) operation.

Interrupts are temporarily disabled and then re-enabled during this call.

In this mode the CGB operates at the same speed as the DMG/Pocket/SGB models.

- You can check to see if `_cpu == CGB_TYPE` before using this function.

See also

[cpu_fast\(\)](#)

20.48.4.6 **cpu_fast()** `void cpu_fast (` `void ) [inline]`

Set CPU speed to fast (CGB Double Speed) operation.

On startup the CGB operates in Normal Speed Mode and can be switched into Double speed mode (faster processing but also higher power consumption). See the Pan Docs for more information about which hardware features operate faster and which remain at Normal Speed.

- Interrupts are temporarily disabled and then re-enabled during this call.
- You can check to see if `_cpu == CGB_TYPE` before using this function.

See also

[cpu_slow\(\)](#), [_cpu](#)

20.48.4.7 **set_default_palette()** `void set_default_palette (` `void )`

Sets CGB palette 0 to be compatible with the DMG/GBP.

The default/first CGB palettes for sprites and backgrounds are set to a similar default appearance as on the DMG/↔ Pocket/SGB models. (White, Light Gray, Dark Gray, Black)

- You can check to see if `_cpu == CGB_TYPE` before using this function.

20.48.4.8 **cgb_compatibility()** `void cgb_compatibility (` `void ) [inline]`

Obsolete. This function has been replaced by [set_default_palette\(\)](#), which has identical behavior.

20.49 cgb.h

[Go to the documentation of this file.](#)

```

1
17 #ifndef _CGB_H
18 #define _CGB_H
19
20 #include <types.h>
21 #include <stdint.h>
22
23 #define RGB(r, g, b) (((uint16_t)((b) & 0x1f) << 10) | ((uint16_t)((g) & 0x1f) << 5) | ((r) & 0x1f))
24
25 #define RGB8(r, g, b) (((uint16_t)((b) >> 3) & 0x1f) << 10) | ((uint16_t)((g) >> 3) & 0x1f) << 5) |
26     (((r) >> 3) & 0x1f)
27
28 #define RGBHTML(RGB24bit) (RGB8(((RGB24bit) >> 16) & 0xff), ((RGB24bit) >> 8) & 0xff, ((RGB24bit) &
29     0xff))
30
31 #define RGB_RED          RGB(31, 0, 0)
32 #define RGB_DARKRED      RGB(15, 0, 0)
33 #define RGB_GREEN        RGB(0, 31, 0)
34 #define RGB_DARKGREEN    RGB(0, 15, 0)
35 #define RGB_BLUE         RGB(0, 0, 31)
36 #define RGB_DARKBLUE     RGB(0, 0, 15)
37 #define RGB_YELLOW       RGB(31, 31, 0)
38 #define RGB_DARKYELLOW   RGB(21, 21, 0)
39 #define RGB_CYAN         RGB(0, 31, 31)
40 #define RGB_AQUA         RGB(28, 5, 22)
41 #define RGB_PINK         RGB(31, 0, 31)
42 #define RGB_PURPLE       RGB(21, 0, 21)
43 #define RGB_BLACK        RGB(0, 0, 0)
44 #define RGB_DARKGRAY     RGB(10, 10, 10)
45 #define RGB_LIGHTGRAY    RGB(21, 21, 21)
46 #define RGB_WHITE        RGB(31, 31, 31)
47
48 #define RGB_LIGHTFLESH    RGB(30, 20, 15)
49 #define RGB_BROWN        RGB(10, 10, 0)
50 #define RGB_ORANGE        RGB(30, 20, 0)
51 #define RGB_TEAL         RGB(15, 15, 0)
52
53 typedef uint16_t palette_color_t;
54 void set_bkg_palette(uint8_t first_palette, uint8_t nb_palettes, const palette_color_t *rgb_data)
55     OLDCALL;
56
57 void set_sprite_palette(uint8_t first_palette, uint8_t nb_palettes, const palette_color_t *rgb_data)
58     OLDCALL;
59
60 void set_bkg_palette_entry(uint8_t palette, uint8_t entry, uint16_t rgb_data) OLDCALL;
61
62 void set_sprite_palette_entry(uint8_t palette, uint8_t entry, uint16_t rgb_data) OLDCALL;
63
64 void cpu_slow(void);
65
66 void cpu_fast(void);
67
68 void set_default_palette(void);
69
70 void cgb_compatibility(void);
71
72 #endif /* _CGB_H */

```

20.50 gbdk-lib/include/gb/crash_handler.h File Reference

Functions

- void [__HandleCrash](#) (void)

20.50.1 Detailed Description

When `crash_handler.h` is included, a crash dump screen will be displayed if the CPU executes uninitialized memory (with a value of `0xFF`, the opcode for `RST 38`). A handler is installed for `RST 38` that calls [__HandleCrash\(\)](#).

`#include <gb/crash_handler.h>`

Also see the `crash` example project included with gbdk.

20.50.2 Function Documentation

20.50.2.1 `__HandleCrash()` `void __HandleCrash (`
`void )`

Display the crash dump screen.

See the intro for this file for more details.

20.51 `crash_handler.h`

[Go to the documentation of this file.](#)

```
1
15 #ifndef __CRASH_HANDLER_INCLUDE
16 #define __CRASH_HANDLER_INCLUDE
17
22 void __HandleCrash(void);
23 static void * __CRASH_HANDLER_INIT = &__HandleCrash;
24
25 #endif
```

20.52 `gbdk-lib/include/gb/drawing.h` File Reference

```
#include <types.h>
#include <stdint.h>
```

Macros

- `#define GRAPHICS_WIDTH` 160
- `#define GRAPHICS_HEIGHT` 144
- `#define SOLID` 0x00 /* Overwrites the existing pixels */
- `#define OR` 0x01 /* Performs a logical OR */
- `#define XOR` 0x02 /* Performs a logical XOR */
- `#define AND` 0x03 /* Performs a logical AND */
- `#define WHITE` 0
- `#define LTGREY` 1
- `#define DKGREY` 2
- `#define BLACK` 3
- `#define M_NOFILL` 0
- `#define M_FILL` 1
- `#define SIGNED` 1
- `#define UNSIGNED` 0

Functions

- void `gprint` (char *str) **NONBANKED**
- void `gprintln` (int16_t number, int8_t radix, int8_t signed_value) **NONBANKED**
- void `gprintrn` (int8_t number, int8_t radix, int8_t signed_value) **NONBANKED**
- int8_t `gprintf` (char *fmt,...) **NONBANKED**
- void `plot` (uint8_t x, uint8_t y, uint8_t colour, uint8_t mode) **OLDCALL**
- void `plot_point` (uint8_t x, uint8_t y) **OLDCALL**
- void `switch_data` (uint8_t x, uint8_t y, uint8_t *src, uint8_t *dst) **OLDCALL**
- void `draw_image` (uint8_t *data)
- void `line` (uint8_t x1, uint8_t y1, uint8_t x2, uint8_t y2) **OLDCALL**
- void `box` (uint8_t x1, uint8_t y1, uint8_t x2, uint8_t y2, uint8_t style) **OLDCALL**
- void `circle` (uint8_t x, uint8_t y, uint8_t radius, uint8_t style) **OLDCALL**
- uint8_t `getpix` (uint8_t x, uint8_t y) **OLDCALL**
- void `wrtchr` (char chr) **OLDCALL**
- void `gotogxy` (uint8_t x, uint8_t y) **OLDCALL**
- void `color` (uint8_t forecolor, uint8_t backcolor, uint8_t mode) **OLDCALL**

20.52.1 Detailed Description

All Points Addressable (APA) mode drawing library.

Drawing routines originally by Pascal Felber Legendary overhaul by Jon Fuge : <https://github.com/jf1452> Commenting by Michael Hope

Note: The standard text `printf()` and `putchar()` cannot be used in APA mode - use `gprintf()` and `wrtchr()` instead.

Note: Using drawing.h will cause it's custom LCD ISR (`drawing_lcd`) to be installed. Changing the mode (`mode(M_TEXT_OUT);`) will cause them to be de-installed.

The valid coordinate ranges are from (x,y) 0,0 to 159,143. There is no built-in clipping, so drawing outside valid coordinates will likely produce undesired results (wrapping/etc).

Important note for the drawing API :

The Game Boy graphics hardware is not well suited to frame-buffer style graphics such as the kind provided in `drawing.h`. Due to that, **most drawing functions (rectangles, circles, etc) will be slow** . When possible it's much faster and more efficient to work with the tiles and tile maps that the Game Boy hardware is built around.

20.52.2 Macro Definition Documentation

20.52.2.1 GRAPHICS_WIDTH `#define GRAPHICS_WIDTH 160`

Size of the screen in pixels

20.52.2.2 GRAPHICS_HEIGHT `#define GRAPHICS_HEIGHT 144`

20.52.2.3 SOLID `#define SOLID 0x00 /* Overwrites the existing pixels */`

20.52.2.4 OR `#define OR 0x01 /* Performs a logical OR */`

20.52.2.5 XOR `#define XOR 0x02 /* Performs a logical XOR */`

20.52.2.6 AND `#define AND 0x03 /* Performs a logical AND */`

20.52.2.7 WHITE `#define WHITE 0`

Possible drawing colours

20.52.2.8 LTGREY `#define LTGREY 1`

20.52.2.9 DKGREY `#define DKGREY 2`

20.52.2.10 BLACK `#define BLACK 3`

20.52.2.11 M_NOFILL `#define M_NOFILL 0`

Possible fill styles for `box()` and `circle()`

20.52.2.12 M_FILL `#define M_FILL 1`

20.52.2.13 SIGNED `#define SIGNED 1`

Possible values for `signed_value` in [gprintln\(\)](#) and [gprintn\(\)](#)

20.52.2.14 UNSIGNED `#define UNSIGNED 0`**20.52.3 Function Documentation****20.52.3.1 gprint()** `void gprint (`
`char * str )`

Print the string 'str' with no interpretation

See also

[gotogxy\(\)](#)

20.52.3.2 gprintln() `void gprintln (`
`int16_t number,`
`int8_t radix,`
`int8_t signed_value )`

Print 16 bit **number** in **radix** (base) in the default font at the current text position.

Parameters

<i>number</i>	number to print
<i>radix</i>	radix (base) to print with
<i>signed_value</i>	should be set to SIGNED or UNSIGNED depending on whether the number is signed or not

The current position is advanced by the number of characters printed.

See also

[gotogxy\(\)](#)

20.52.3.3 gprintn() `void gprintn (`
`int8_t number,`
`int8_t radix,`
`int8_t signed_value )`

Print 8 bit **number** in **radix** (base) in the default font at the current text position.

See also

[gprintln\(\)](#), [gotogxy\(\)](#)

20.52.3.4 gprintf() `int8_t gprintf (`
`char * fmt,`
`... )`

Print the string and arguments given by **fmt** with arguments ____

Parameters

<i>fmt</i>	The format string as per printf
...	params

Currently supported:

- %c (character)
- %u (int)
- %d (int8_t)
- %o (int8_t as octal)
- %x (int8_t as hex)
- %s (string)

Returns

Returns the number of items printed, or -1 if there was an error.

See also

[gotogxy\(\)](#)

20.52.3.5 plot() void plot (
 uint8_t x,
 uint8_t y,
 uint8_t colour,
 uint8_t mode)

Old style plot - try [plot_point\(\)](#)

20.52.3.6 plot_point() void plot_point (
 uint8_t x,
 uint8_t y)

Plot a point in the current drawing mode and colour at x,y

20.52.3.7 switch_data() void switch_data (
 uint8_t x,
 uint8_t y,
 uint8_t * src,
 uint8_t * dst)

Exchanges the tile on screen at x,y with the tile pointed by src, original tile is saved in dst. Both src and dst may be NULL - saving or copying to screen is not performed in this case.

20.52.3.8 draw_image() void draw_image (
 uint8_t * data)

Draw a full screen image at **data**

20.52.3.9 line() void line (
 uint8_t x1,
 uint8_t y1,
 uint8_t x2,
 uint8_t y2)

Draw a line in the current drawing mode and colour from x1,y1 to x2,y2

20.52.3.10 box() void box (
 uint8_t x1,
 uint8_t y1,
 uint8_t x2,
 uint8_t y2,
 uint8_t style)

Draw a box (rectangle) with corners x1,y1 and x2,y2 using fill mode **style** (one of NOFILL or FILL)

20.52.3.11 circle() `void circle (`
 `uint8_t x,`
 `uint8_t y,`
 `uint8_t radius,`
 `uint8_t style )`

Draw a circle with centre at **x,y** and **radius** using fill mode **style** (one of NOFILL or FILL)

20.52.3.12 getpix() `uint8_t getpix (`
 `uint8_t x,`
 `uint8_t y )`

Returns the current colour of the pixel at **x,y**

20.52.3.13 wrtchr() `void wrtchr (`
 `char chr )`

Prints the character **chr** in the default font at the current text position.

The current position is advanced by 1 after the character is printed.

See also

[gotogxy\(\)](#)

20.52.3.14 gotogxy() `void gotogxy (`
 `uint8_t x,`
 `uint8_t y )`

Sets the current text position to **x,y**.

Note: **x** and **y** have units of tiles (8 pixels per unit)

See also

[wrtchr\(\)](#)

20.52.3.15 color() `void color (`
 `uint8_t forecolor,`
 `uint8_t backcolor,`
 `uint8_t mode )`

Set the current **forecolor** colour, **backcolor** colour, and draw **mode**

Parameters

<i>forecolor</i>	The primary drawing color (outlines of rectangles with box() , letter color with gprintf() , etc).
<i>backcolor</i>	Secondary or background color where applicable (fill color of rectangles with box() when <code>M_FILL</code> is specified, background color of text with gprintf() , etc).
<i>mode</i>	Drawing style to use. Several settings are available <code>SOLID</code> , <code>OR</code> , <code>XOR</code> , <code>AND</code> .

In order to completely overwrite existing pixels use `SOLID` for **mode**

20.53 drawing.h

[Go to the documentation of this file.](#)

```
1
30 #ifndef __DRAWING_H
31 #define __DRAWING_H
32
33 #include <types.h>
34 #include <stdint.h>
35
37 #define GRAPHICS_WIDTH 160
38 #define GRAPHICS_HEIGHT 144
```

```

39
40 #define SOLID    0x00        /* Overwrites the existing pixels */
41 #define OR       0x01        /* Performs a logical OR */
42 #define XOR      0x02        /* Performs a logical XOR */
43 #define AND      0x03        /* Performs a logical AND */
44
45 #define WHITE    0
46 #define LTGREY  1
47 #define DKGREY  2
48 #define BLACK   3
49
50
51 #define M_NOFILL  0
52 #define M_FILL    1
53
54
55 #define SIGNED    1
56 #define UNSIGNED 0
57
58
59 #include <types.h>
60
61 void gprint(char *str) NONBANKED;
62
63
64 void gprintln(int16_t number, int8_t radix, int8_t signed_value) NONBANKED;
65
66
67 void gprintn(int8_t number, int8_t radix, int8_t signed_value) NONBANKED;
68
69
70 int8_t gprintf(char *fmt,...) NONBANKED;
71
72
73 void plot(uint8_t x, uint8_t y, uint8_t colour, uint8_t mode) OLDCALL;
74
75 void plot_point(uint8_t x, uint8_t y) OLDCALL;
76
77 void switch_data(uint8_t x, uint8_t y, uint8_t *src, uint8_t *dst) OLDCALL;
78
79 void draw_image(uint8_t *data);
80
81 void line(uint8_t x1, uint8_t y1, uint8_t x2, uint8_t y2) OLDCALL;
82
83 void box(uint8_t x1, uint8_t y1, uint8_t x2, uint8_t y2, uint8_t style) OLDCALL;
84
85 void circle(uint8_t x, uint8_t y, uint8_t radius, uint8_t style) OLDCALL;
86
87 uint8_t getpix(uint8_t x, uint8_t y) OLDCALL;
88
89 void wrtchr(char chr) OLDCALL;
90
91
92 void gotogxy(uint8_t x, uint8_t y) OLDCALL;
93
94 void color(uint8_t forecolor, uint8_t backcolor, uint8_t mode) OLDCALL;
95
96
97 #endif /* __DRAWING_H */

```

20.54 gbdk-lib/include/gb/emu_debug.h File Reference

```
#include <gbdk/emu_debug.h>
```

20.54.1 Detailed Description

Shim for legacy use of [gb/emu_debug.h](#) which has been migrated to [gbdk/emu_debug.h](#)
See the `emu_debug` example project included with gbdk.

20.55 emu_debug.h

[Go to the documentation of this file.](#)

```

1
2 #ifndef __EMU_DEBUG_INCLUDE
3 #define __EMU_DEBUG_INCLUDE
4
5 #include <gbdk/emu_debug.h>
6
7 #endif

```

20.56 gbdk-lib/include/gbdk/emu_debug.h File Reference

```
#include <types.h>
```

Macros

- `#define EMU_MESSAGE(message_text) EMU_MESSAGE1(EMU_MACRONAME(__LINE__), message_text)`
- `#define BGB_MESSAGE(message_text) EMU_MESSAGE(message_text)`
- `#define EMU_PROFILE_BEGIN(MSG) EMU_MESSAGE_SUFFIX(MSG, "%ZEROCLKS%");`
- `#define BGB_PROFILE_BEGIN(MSG) EMU_PROFILE_BEGIN(MSG)`
- `#define EMU_PROFILE_END(MSG) EMU_MESSAGE_SUFFIX(MSG, "%-8+LASTCLKS%");`
- `#define BGB_PROFILE_END(MSG) EMU_PROFILE_END(MSG)`
- `#define EMU_TEXT(MSG) EMU_MESSAGE(MSG)`
- `#define BGB_TEXT(MSG) EMU_TEXT(MSG)`
- `#define BGB_profiler_message EMU_profiler_message()`
- `#define BGB_printf(...) EMU_printf(__VA_ARGS__)`
- `#define EMU_BREAKPOINT __asm__("ld b, b");`
- `#define BGB_BREAKPOINT EMU_BREAKPOINT`

Functions

- void `EMU_profiler_message` (void)
- void `EMU_printf` (const char *format,...) `PRESERVES_REGS`(a
- void `EMU_fmtbuf` (const unsigned char *format, void *data) `PRESERVES_REGS`(a

Variables

- void `b`
- void `c`

20.56.1 Detailed Description

Debug window logging and profiling support for emulators (BGB, Emulicious, etc).

Also see the `emu_debug` example project included with `gbdk`.

See the BGB Manual for more information ("expressions, breakpoint conditions, and debug messages") <http://bgb.bircd.org/manual.html#expressions>

20.56.2 Macro Definition Documentation

20.56.2.1 EMU_MESSAGE `#define EMU_MESSAGE( message_text ) EMU_MESSAGE1(EMU_MACRONAME(__LINE__), message_text)`

Macro to display a message in the emulator debug message window

Parameters

<code>message_text</code>	Quoted text string to display in the debug message window
---------------------------	---

The following special parameters can be used when bracketed with "%" characters.

- CPU registers: AF, BC, DE, HL, SP, PC, B, C, D, E, H, L, A, ZERO, ZF, Z, CARRY, CY, IME, ALLREGS
- Other state values: ROMBANK, XRAMBANK, SRAMBANK, WRAMBANK, VRAMBANK, TOTALCLKS, LASTCLKS, CLKS2VBLANK

Example: print a message along with the currently active ROM bank.

```
EMU_MESSAGE("Current ROM Bank is: %ROMBANK%");
```

See the BGB Manual for more information ("expressions, breakpoint conditions, and debug messages") <http://bgb.bircd.org/manual.html#expressions>

See also

[EMU_PROFILE_BEGIN\(\)](#), [EMU_PROFILE_END\(\)](#)

20.56.2.2 BGB_MESSAGE `#define BGB_MESSAGE(
 message_text) EMU_MESSAGE(message_text)`

20.56.2.3 EMU_PROFILE_BEGIN `#define EMU_PROFILE_BEGIN(
 MSG) EMU_MESSAGE_SUFFIX(MSG, "%ZEROCCLKS%");`

Macro to **Start** a profiling block for the emulator (BGB, Emulicious, etc)

Parameters

<i>MSG</i>	Quoted text string to display in the debug message window along with the result
------------	---

To complete the profiling block and print the result call [EMU_PROFILE_END](#).

See also

[EMU_PROFILE_END\(\)](#), [EMU_MESSAGE\(\)](#)

20.56.2.4 BGB_PROFILE_BEGIN `#define BGB_PROFILE_BEGIN(
 MSG) EMU_PROFILE_BEGIN(MSG)`

20.56.2.5 EMU_PROFILE_END `#define EMU_PROFILE_END(
 MSG) EMU_MESSAGE_SUFFIX(MSG, "%-8+LASTCLKS%");`

Macro to **End** a profiling block and print the results in the emulator debug message window

Parameters

<i>MSG</i>	Quoted text string to display in the debug message window along with the result
------------	---

This should only be called after a previous call to [EMU_PROFILE_BEGIN\(\)](#)

The results are in Emulator clock units, which are "1 nop in [CGB] doublespeed mode".

So when running in Normal Speed mode (i.e. non-CGB doublespeed) the printed result should be **divided by 2** to get the actual ellapsed cycle count.

If running in CB Double Speed mode use the below call instead, it correctly compensates for the speed difference.

In this scenario, the result does **not need to be divided by 2** to get the ellapsed cycle count.

[EMU_MESSAGE](#)("NOP TIME: %-4+LASTCLKS%");

See also

[EMU_PROFILE_BEGIN\(\)](#), [EMU_MESSAGE\(\)](#)

20.56.2.6 BGB_PROFILE_END `#define BGB_PROFILE_END(
 MSG) EMU_PROFILE_END(MSG)`

20.56.2.7 EMU_TEXT `#define EMU_TEXT(
 MSG) EMU_MESSAGE(MSG)`

20.56.2.8 BGB_TEXT `#define BGB_TEXT(
MSG) EMU_TEXT(MSG)`

20.56.2.9 BGB_profiler_message `#define BGB_profiler_message EMU_profiler_message()`

20.56.2.10 BGB_printf `#define BGB_printf(
...) EMU_printf(__VA_ARGS__)`

20.56.2.11 EMU_BREAKPOINT `#define EMU_BREAKPOINT __asm__("ld b, b");`
The Emulator will break into debugger when encounters this line

20.56.2.12 BGB_BREAKPOINT `#define BGB_BREAKPOINT EMU_BREAKPOINT`

20.56.3 Function Documentation

20.56.3.1 EMU_profiler_message() `void EMU_profiler_message (
void)`

Display preset debug information in the Emulator debug messages window.

This function is equivalent to:

`EMU_MESSAGE("PROFILE,%(SP+$0)%,%(SP+$1)%,%A%,%TOTALCLKS%,%ROMBANK%,%WRAMBANK%");`

20.56.3.2 EMU_printf() `void EMU_printf (
const char * format,
...)`

Print the string and arguments given by format to the emulator debug message window

Parameters

<i>format</i>	The format string as per printf
---------------	---------------------------------

Does not return the number of characters printed. Currently supported:

- %hx (char as hex)
- %hu (unsigned char)
- %hd (signed char)
- %c (character)
- %u (unsigned int)
- %d (signed int)
- %x (unsigned int as hex)
- %s (string)

Note

Variables for the following 8-bit formats **MUST** be cast to their type when passed to [EMU_printf\(\)](#)

- %hx (char)
- %hu (unsigned char)
- %hd (signed char)

However variables for the following 8-bit format **MUST NOT** be cast to their type when passed to [EMU_printf\(\)](#)

- %c (char)

This behavior is **different** than for [sprintf\(\)](#), which does require %c format char variables to be explicitly cast.

Currently supported in the Emulicious emulator, may be supported by bgb

20.56.3.3 EMU_fmtbuf() void EMU_fmtbuf (
 const unsigned char * *format*,
 void * *data*)

Print the string and arguments in the buffer buffer by the pointer given by format to the emulator debug message window

Parameters

<i>format</i>	The format string as per printf
<i>data</i>	Buffer containing arguments, for example some struct

See also

[EMU_printf](#) for the format string description

Currently supported in the Emulicious emulator

20.56.4 Variable Documentation

20.56.4.1 b void b

20.56.4.2 c void c

20.57 emu_debug.h

[Go to the documentation of this file.](#)

```

1
13 // Suppress SDCC "info 128" warnings that are a non-issue
14 #pragma disable_warning 218
15
16 #ifndef __GBDK_EMU_DEBUG_H_INCLUDE
17 #define __GBDK_EMU_DEBUG_H_INCLUDE
18
19 #include <types.h>
20
21 #if defined(__TARGET_gb) || defined(__TARGET_ap) || defined(__TARGET_sms) || defined(__TARGET_gg)
22
23 #define EMU_MESSAGE(message_text) EMU_MESSAGE1(EMU_MACRONAME(__LINE__), message_text)
24 #define BGB_MESSAGE(message_text) EMU_MESSAGE(message_text)
25
26 #define EMU_MACRONAME(A) EMU_MACRONAME1(A)
27 #define EMU_MACRONAME1(A) EMULOG##A
28
29 #define EMU_MESSAGE1(name, message_text) \
30     __asm \
31     .MACRO name msg_t, ?11b1 \
32     ld d, d \

```

```

57     jr llbl \
58     .dw 0x6464 \
59     .dw 0x0000 \
60     .ascii msg_t \
61 llbl: \
62 .ENDM \
63 name ^/message_text/ \
64 __endasm
65
66 #define EMU_MESSAGE_SUFFIX(message_text, message_suffix) EMU_MESSAGE3(EMU_MACRONAME(__LINE__),
        message_text, message_suffix)
67 #define EMU_MESSAGE3(name, message_text, message_suffix) \
68 __asm \
69 .MACRO name msg_t, msg_s, ?llbl \
70     ld d, d \
71     jr llbl \
72     .dw 0x6464 \
73     .dw 0x0000 \
74     .ascii msg_t \
75     .ascii msg_s \
76 llbl: \
77 .ENDM \
78 name ^/message_text/, ^/message_suffix/ \
79 __endasm
81
82 #define EMU_PROFILE_BEGIN(MSG) EMU_MESSAGE_SUFFIX(MSG, "%ZEROCCLKS%");
83 #define BGB_PROFILE_BEGIN(MSG) EMU_PROFILE_BEGIN(MSG)
119 #if defined(NINTENDO)
120 #define EMU_PROFILE_END(MSG) EMU_MESSAGE_SUFFIX(MSG, "%-8+LASTCLKS%");
121 #define BGB_PROFILE_END(MSG) EMU_PROFILE_END(MSG)
122 #elif defined(SEGA)
123 #define EMU_PROFILE_END(MSG) EMU_MESSAGE_SUFFIX(MSG, "%-16+LASTCLKS%");
124 #define BGB_PROFILE_END(MSG) EMU_PROFILE_END(MSG)
125 #endif
126
127 #define EMU_TEXT(MSG) EMU_MESSAGE(MSG)
128 #define BGB_TEXT(MSG) EMU_TEXT(MSG)
129
130 #if defined(NINTENDO)
131 void EMU_profiler_message(void);
132 #define BGB_profiler_message EMU_profiler_message()
133 #endif // NINTENDO
134
135 void EMU_printf(const char *format, ...) PRESERVES_REGS(a, b, c);
136 #define BGB_printf(...) EMU_printf(__VA_ARGS__)
137
138 void EMU_fmtbuf(const unsigned char * format, void * data) PRESERVES_REGS(a, b, c);
139
140 #ifdef NINTENDO
141 static void * __EMU_PROFILER_INIT = &EMU_profiler_message;
142 #endif // NINTENDO
143
144 #define EMU_BREAKPOINT __asm__("ld b, b");
145 #define BGB_BREAKPOINT EMU_BREAKPOINT
146
147 #elif defined(__TARGET_duck)
148     #error Not implemented yet
149 #else
150     #error Unrecognized port
151 #endif
152 #endif

```

20.58 gbdk-lib/include/gb/gb.h File Reference

```

#include <types.h>
#include <stdint.h>
#include <gbdk/version.h>
#include <gb/hardware.h>

```

Data Structures

- struct [joypads_t](#)
- struct [OAM_item_t](#)

Macros

- #define [NINTENDO](#)

- #define `SYSTEM_60HZ` 0x00
- #define `SYSTEM_50HZ` 0x01
- #define `GAMEBOY`
- #define `J_UP` 0x04U
- #define `J_DOWN` 0x08U
- #define `J_LEFT` 0x02U
- #define `J_RIGHT` 0x01U
- #define `J_A` 0x10U
- #define `J_B` 0x20U
- #define `J_SELECT` 0x40U
- #define `J_START` 0x80U
- #define `M_DRAWING` 0x01U
- #define `M_TEXT_OUT` 0x02U
- #define `M_TEXT_INOUT` 0x03U
- #define `M_NO_SCROLL` 0x04U
- #define `M_NO_INTERP` 0x08U
- #define `S_BANK` 0x08U
- #define `S_PALETTE` 0x10U
- #define `S_FLIPX` 0x20U
- #define `S_FLIPY` 0x40U
- #define `S_PRIORITY` 0x80U
- #define `S_PAL`(n) n
- #define `EMPTY_IFLAG` 0x00U
- #define `VBL_IFLAG` 0x01U
- #define `LCD_IFLAG` 0x02U
- #define `TIM_IFLAG` 0x04U
- #define `SIO_IFLAG` 0x08U
- #define `JOY_IFLAG` 0x10U
- #define `DMG_BLACK` 0x03
- #define `DMG_DARK_GRAY` 0x02
- #define `DMG_LITE_GRAY` 0x01
- #define `DMG_WHITE` 0x00
- #define `DMG_PALETTE`(C0, C1, C2, C3) (((uint8_t) (((C3) & 0x03) << 6) | (((C2) & 0x03) << 4) | (((C1) & 0x03) << 2) | ((C0) & 0x03)))
- #define `SCREENWIDTH` `DEVICE_SCREEN_PX_WIDTH`
- #define `SCREENHEIGHT` `DEVICE_SCREEN_PX_HEIGHT`
- #define `MINWNDPOSX` `DEVICE_WINDOW_PX_OFFSET_X`
- #define `MINWNDPOSY` `DEVICE_WINDOW_PX_OFFSET_Y`
- #define `MAXWNDPOSX` (`DEVICE_WINDOW_PX_OFFSET_X` + `DEVICE_SCREEN_PX_WIDTH` - 1)
- #define `MAXWNDPOSY` (`DEVICE_WINDOW_PX_OFFSET_Y` + `DEVICE_SCREEN_PX_HEIGHT` - 1)
- #define `DMG_TYPE` 0x01
- #define `MGB_TYPE` 0xFF
- #define `CGB_TYPE` 0x11
- #define `GBA_NOT_DETECTED` 0x00
- #define `GBA_DETECTED` 0x01
- #define `DEVICE_SUPPORTS_COLOR` (`_cpu == CGB_TYPE`)
- #define `VBL_DONE` `_vbl_done`
- #define `IO_IDLE` 0x00U
- #define `IO_SENDING` 0x01U
- #define `IO_RECEIVING` 0x02U
- #define `IO_ERROR` 0x04U
- #define `CURRENT_BANK` `_current_bank`
- #define `BANK`(VARNAME) ((uint8_t) &__bank_## VARNAME)
- #define `BANKREF`(VARNAME)
- #define `BANKREF_EXTERN`(VARNAME) extern const void __bank_## VARNAME;

- `#define SWITCH_ROM(b) (_current_bank = (b), rROMB0 = (b))`
- `#define SWITCH_RAM(b) (rRAMB = (b))`
- `#define ENABLE_RAM (rRAMG = 0x0A)`
- `#define DISABLE_RAM (rRAMG = 0x00)`
- `#define SWITCH_ROM_MEGADUCK(b) SWITCH_ROM(b)`
- `#define SWITCH_ROM_MBC1(b) SWITCH_ROM(b)`
- `#define SWITCH_RAM_MBC1(b) SWITCH_RAM(b)`
- `#define ENABLE_RAM_MBC1 ENABLE_RAM`
- `#define DISABLE_RAM_MBC1 DISABLE_RAM`
- `#define SWITCH_16_8_MODE_MBC1 (*(volatile uint8_t *)0x6000 = 0x00)`
- `#define SWITCH_4_32_MODE_MBC1 (*(volatile uint8_t *)0x6000 = 0x01)`
- `#define SWITCH_ROM_MBC5(b) (_current_bank = (b), rROMB1 = 0, rROMB0 = (b))`
- `#define SWITCH_ROM_MBC5_8M(b) (rROMB1 = ((uint16_t)(b) >> 8), rROMB0 = (b))`
- `#define SWITCH_RAM_MBC5(b) SWITCH_RAM(b)`
- `#define ENABLE_RAM_MBC5 ENABLE_RAM`
- `#define DISABLE_RAM_MBC5 DISABLE_RAM`
- `#define DISPLAY_ON LCDC_REG|=LCDCF_ON`
- `#define DISPLAY_OFF display_off();`
- `#define HIDE_LEFT_COLUMN`
- `#define SHOW_LEFT_COLUMN`
- `#define SET_BORDER_COLOR(C)`
- `#define SHOW_BKG LCDC_REG|=LCDCF_BGON`
- `#define HIDE_BKG LCDC_REG&=~LCDCF_BGON`
- `#define SHOW_WIN LCDC_REG|=LCDCF_WINON`
- `#define HIDE_WIN LCDC_REG&=~LCDCF_WINON`
- `#define SHOW_SPRITES LCDC_REG|=LCDCF_OBJON`
- `#define HIDE_SPRITES LCDC_REG&=~LCDCF_OBJON`
- `#define SPRITES_8x16 LCDC_REG|=LCDCF_OBJ16`
- `#define SPRITES_8x8 LCDC_REG&=~LCDCF_OBJ16`
- `#define COMPAT_PALETTE(C0, C1, C2, C3) (((uint8_t)((C3) << 6) | ((C2) << 4) | ((C1) << 2) | (C0)))`
- `#define set_bkg_2bpp_data set_bkg_data`
- `#define set_tile_map set_bkg_tiles`
- `#define set_tile_submap set_bkg_submap`
- `#define set_tile_xy set_bkg_tile_xy`
- `#define set_attribute_xy set_bkg_attribute_xy`
- `#define set_sprite_2bpp_data set_sprite_data`
- `#define DISABLE_OAM_DMA _shadow_OAM_base = 0`
- `#define DISABLE_VBL_TRANSFER DISABLE_OAM_DMA`
- `#define ENABLE_OAM_DMA _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)`
- `#define ENABLE_VBL_TRANSFER ENABLE_OAM_DMA`
- `#define MAX_HARDWARE_SPRITES 40`
- `#define HARDWARE_SPRITE_CAN_FLIP_X 1`
- `#define HARDWARE_SPRITE_CAN_FLIP_Y 1`
- `#define fill_rect fill_bkg_rect`

Typedefs

- `typedef void(* int_handler) (void) NONBANKED`
- `typedef struct OAM_item_t OAM_item_t`

Functions

- void [remove_VBL](#) (int_handler h)
- void [remove_LCD](#) (int_handler h)
- void [remove_TIM](#) (int_handler h)
- void [remove_SIO](#) (int_handler h)
- void [remove_JOY](#) (int_handler h)
- void [add_VBL](#) (int_handler h)
- void [add_LCD](#) (int_handler h)
- void [add_TIM](#) (int_handler h)
- void [add_low_priority_TIM](#) (int_handler h)
- void [add_SIO](#) (int_handler h)
- void [add_JOY](#) (int_handler h)
- void [nowait_int_handler](#) (void)
- void [wait_int_handler](#) (void)
- [uint8_t](#) [cancel_pending_interrupts](#) (void)
- void [mode](#) ([uint8_t](#) m)
- [uint8_t](#) [get_mode](#) (void) PRESERVES_REGS(b)
- [uint8_t](#) [get_system](#) (void)
- void [send_byte](#) (void)
- void [receive_byte](#) (void)
- void [delay](#) ([uint16_t](#) d) PRESERVES_REGS(h)
- [uint8_t](#) [joypad](#) (void) PRESERVES_REGS(b)
- [uint8_t](#) [waitpad](#) ([uint8_t](#) mask) PRESERVES_REGS(b)
- void [waitpadup](#) (void) PRESERVES_REGS(a)
- [uint8_t](#) [joypad_init](#) ([uint8_t](#) npads, [joypads_t](#) *joypads) OLDSCALL
- void [joypad_ex](#) ([joypads_t](#) *joypads) PRESERVES_REGS(b)
- void [enable_interrupts](#) (void) PRESERVES_REGS(a)
- void [disable_interrupts](#) (void) PRESERVES_REGS(a)
- void [set_interrupts](#) ([uint8_t](#) flags) PRESERVES_REGS(b)
- void [reset](#) (void)
- void [vsync](#) (void) PRESERVES_REGS(b)
- void [wait_vbl_done](#) (void) PRESERVES_REGS(b)
- void [display_off](#) (void) PRESERVES_REGS(b)
- void [refresh_OAM](#) (void) PRESERVES_REGS(b)
- void [hramcpy](#) ([uint8_t](#) dst, const void *src, [uint8_t](#) n) OLDSCALL PRESERVES_REGS(b)
- void [set_vram_byte](#) ([uint8_t](#) *addr, [uint8_t](#) v) PRESERVES_REGS(b)
- [uint8_t](#) [get_vram_byte](#) ([uint8_t](#) *addr) PRESERVES_REGS(b)
- [uint8_t](#) * [get_bkg_xy_addr](#) ([uint8_t](#) x, [uint8_t](#) y) PRESERVES_REGS(h)
- void [set_2bpp_palette](#) ([uint16_t](#) palette)
- void [set_1bpp_colors_ex](#) ([uint8_t](#) fgcolor, [uint8_t](#) bgcolor, [uint8_t](#) mode) OLDSCALL
- void [set_1bpp_colors](#) ([uint8_t](#) fgcolor, [uint8_t](#) bgcolor)
- void [set_bkg_data](#) ([uint8_t](#) first_tile, [uint8_t](#) nb_tiles, const [uint8_t](#) *data) OLDSCALL PRESERVES_REGS(b)
- void [set_bkg_1bpp_data](#) ([uint8_t](#) first_tile, [uint8_t](#) nb_tiles, const [uint8_t](#) *data) OLDSCALL PRESERVES_REGS(b)
- void [get_bkg_data](#) ([uint8_t](#) first_tile, [uint8_t](#) nb_tiles, [uint8_t](#) *data) OLDSCALL PRESERVES_REGS(b)
- void [set_bkg_tiles](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) w, [uint8_t](#) h, const [uint8_t](#) *tiles) OLDSCALL PRESERVES_REGS(b)
- void [set_bkg_based_tiles](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) w, [uint8_t](#) h, const [uint8_t](#) *tiles, [uint8_t](#) base_tile)
- void [set_bkg_attributes](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) w, [uint8_t](#) h, const [uint8_t](#) *tiles)
- void [set_bkg_submap](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) w, [uint8_t](#) h, const [uint8_t](#) *map, [uint8_t](#) map_w) OLDSCALL
- void [set_bkg_based_submap](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) w, [uint8_t](#) h, const [uint8_t](#) *map, [uint8_t](#) map_w, [uint8_t](#) base_tile)
- void [set_bkg_submap_attributes](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) w, [uint8_t](#) h, const [uint8_t](#) *map, [uint8_t](#) map_w)
- void [get_bkg_tiles](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) w, [uint8_t](#) h, [uint8_t](#) *tiles) OLDSCALL PRESERVES_REGS(b)
- [uint8_t](#) * [set_bkg_tile_xy](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) t)
- [uint8_t](#) * [set_bkg_attribute_xy](#) ([uint8_t](#) x, [uint8_t](#) y, [uint8_t](#) a)

- `uint8_t get_bkg_tile_xy (uint8_t x, uint8_t y)` OLDSCALL PRESERVES_REGS(b)
- `void move_bkg (uint8_t x, uint8_t y)`
- `void scroll_bkg (uint8_t x, uint8_t y)`
- `uint8_t * get_win_xy_addr (uint8_t x, uint8_t y)` PRESERVES_REGS(h)
- `void set_win_data (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data)` OLDSCALL PRESERVES_REGS(b)
- `void set_win_1bpp_data (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data)` OLDSCALL PRESERVES_REGS(b)
- `void get_win_data (uint8_t first_tile, uint8_t nb_tiles, uint8_t *data)` OLDSCALL PRESERVES_REGS(b)
- `void set_win_tiles (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles)` OLDSCALL PRESERVES_REGS(b)
- `void set_win_based_tiles (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles, uint8_t base_tile)`
- `void set_win_submap (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)` OLDSCALL
- `void set_win_based_submap (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w, uint8_t base_tile)`
- `void get_win_tiles (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *tiles)` OLDSCALL PRESERVES_REGS(b)
- `uint8_t * set_win_tile_xy (uint8_t x, uint8_t y, uint8_t t)`
- `uint8_t get_win_tile_xy (uint8_t x, uint8_t y)` OLDSCALL PRESERVES_REGS(b)
- `void move_win (uint8_t x, uint8_t y)`
- `void scroll_win (uint8_t x, uint8_t y)`
- `void set_sprite_data (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data)` OLDSCALL PRESERVES_REGS(b)
- `void set_sprite_1bpp_data (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data)` OLDSCALL PRESERVES_REGS(b)
- `void get_sprite_data (uint8_t first_tile, uint8_t nb_tiles, uint8_t *data)` OLDSCALL PRESERVES_REGS(b)
- `void SET_SHADOW_OAM_ADDRESS (void *address)`
- `void set_sprite_tile (uint8_t nb, uint8_t tile)`
- `uint8_t get_sprite_tile (uint8_t nb)`
- `void set_sprite_prop (uint8_t nb, uint8_t prop)`
- `uint8_t get_sprite_prop (uint8_t nb)`
- `void move_sprite (uint8_t nb, uint8_t x, uint8_t y)`
- `void scroll_sprite (uint8_t nb, uint8_t x, uint8_t y)`
- `void hide_sprite (uint8_t nb)`
- `void set_data (uint8_t *vram_addr, const uint8_t *data, uint16_t len)`
- `void get_data (uint8_t *data, uint8_t *vram_addr, uint16_t len)`
- `void vmemcpy (uint8_t *dest, uint8_t *sour, uint16_t len)`
- `void set_tiles (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *vram_addr, const uint8_t *tiles)` OLDSCALL
- `void set_tile_data (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data, uint8_t base)` OLDSCALL PRESERVES_REGS(b)
- `void get_tiles (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *vram_addr, uint8_t *tiles)` OLDSCALL
- `void set_native_tile_data (uint16_t first_tile, uint8_t nb_tiles, const uint8_t *data)`
- `void set_bkg_native_data (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data)`
- `void set_sprite_native_data (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data)`
- `void init_win (uint8_t c)` OLDSCALL PRESERVES_REGS(b)
- `void init_bkg (uint8_t c)` OLDSCALL PRESERVES_REGS(b)
- `void vmemset (void *s, uint8_t c, size_t n)` OLDSCALL PRESERVES_REGS(b)
- `void fill_bkg_rect (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t tile)` OLDSCALL PRESERVES_REGS(b)
- `void fill_win_rect (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t tile)` OLDSCALL PRESERVES_REGS(b)

Variables

- `uint8_t c`
- `uint8_t d`
- `uint8_t e`
- `uint8_t h`
- `uint8_t l`
- `uint8_t cpu`
- `uint8_t is_GBA`
- `volatile uint16_t sys_time`
- `__REG_vbl_done`

- volatile [uint8_t _io_status](#)
- volatile [uint8_t _io_in](#)
- volatile [uint8_t _io_out](#)
- [__REG_current_bank](#)
- void [b](#)
- [uint16_t _current_1bpp_colors](#)
- [uint8_t _map_tile_offset](#)
- [uint8_t _submap_tile_offset](#)
- volatile struct [OAM_item_t shadow_OAM](#) []
- [__REG_shadow_OAM_base](#)

20.58.1 Detailed Description

Gameboy specific functions.

20.58.2 Macro Definition Documentation

20.58.2.1 NINTENDO `#define NINTENDO`

20.58.2.2 SYSTEM_60HZ `#define SYSTEM_60HZ 0x00`

20.58.2.3 SYSTEM_50HZ `#define SYSTEM_50HZ 0x01`

20.58.2.4 GAMEBOY `#define GAMEBOY`

20.58.2.5 J_UP `#define J_UP 0x04U`

Joypad bits. A logical OR of these is used in the `wait_pad` and `joypad` functions. For example, to see if the B button is pressed try

```
uint8_t keys; keys = joypad(); if (keys & J_B) { ... }
```

See also

[joypad](#)

20.58.2.6 J_DOWN `#define J_DOWN 0x08U`

20.58.2.7 J_LEFT `#define J_LEFT 0x02U`

20.58.2.8 J_RIGHT `#define J_RIGHT 0x01U`

20.58.2.9 J_A `#define J_A 0x10U`

20.58.2.10 J_B `#define J_B 0x20U`

20.58.2.11 J_SELECT `#define J_SELECT 0x40U`

20.58.2.12 J_START `#define J_START 0x80U`

20.58.2.13 M_DRAWING `#define M_DRAWING 0x01U`

Screen modes. Normally used by internal functions only.

See also

[mode\(\)](#)

20.58.2.14 M_TEXT_OUT `#define M_TEXT_OUT 0x02U`

20.58.2.15 M_TEXT_INOUT `#define M_TEXT_INOUT 0x03U`

20.58.2.16 M_NO_SCROLL `#define M_NO_SCROLL 0x04U`

Set this in addition to the others to disable scrolling

If scrolling is disabled, the cursor returns to (0,0)

See also

[mode\(\)](#)

20.58.2.17 M_NO_INTERP `#define M_NO_INTERP 0x08U`

Set this to disable interpretation

See also

[mode\(\)](#)

20.58.2.18 S_BANK `#define S_BANK 0x08U`

If this bit set clear, the tile from the second VRAM bank is used

See also

[set_sprite_prop\(\)](#)

20.58.2.19 S_PALETTE `#define S_PALETTE 0x10U`

If this is set, sprite colours come from OBJ1PAL. Else they come from OBJ0PAL

See also

[set_sprite_prop\(\)](#).

20.58.2.20 S_FLIPX `#define S_FLIPX 0x20U`

If set the sprite will be flipped horizontally.

See also

[set_sprite_prop\(\)](#)

20.58.2.21 S_FLIPY `#define S_FLIPY 0x40U`

If set the sprite will be flipped vertically.

See also

[set_sprite_prop\(\)](#)

20.58.2.22 S_PRIORITY `#define S_PRIORITY 0x80U`

If this bit is clear, then the sprite will be displayed on top of the background and window.

See also

[set_sprite_prop\(\)](#)

20.58.2.23 S_PAL `#define S_PAL(
n) n`

Defines how palette number is encoded in OAM. Required for the png2asset tool's metasprite output.

20.58.2.24 EMPTY_IFLAG `#define EMPTY_IFLAG 0x00U`

Disable calling of interrupt service routines

20.58.2.25 VBL_IFLAG `#define VBL_IFLAG 0x01U`

VBlank Interrupt occurs at the start of the vertical blank.

During this period the video ram may be freely accessed.

See also

[set_interrupts\(\)](#),

[add_VBL](#)

20.58.2.26 LCD_IFLAG `#define LCD_IFLAG 0x02U`

LCD Interrupt when triggered by the STAT register.

See also

[set_interrupts\(\)](#),

[add_LCD](#)

20.58.2.27 TIM_IFLAG `#define TIM_IFLAG 0x04U`

Timer Interrupt when the timer [TIMA_REG](#) overflows.

See also

[set_interrupts\(\)](#),

[add_TIM](#)

20.58.2.28 SIO_IFLAG `#define SIO_IFLAG 0x08U`

Serial Link Interrupt occurs when the serial transfer has completed.

See also

[set_interrupts\(\)](#),

[add_SIO](#)

20.58.2.29 JOY_IFLAG `#define JOY_IFLAG 0x10U`
Joypad Interrupt occurs on a transition of the keypad.

See also

[set_interrupts\(\)](#),
[add_JOY](#)

20.58.2.30 DMG_BLACK `#define DMG_BLACK 0x03`

20.58.2.31 DMG_DARK_GRAY `#define DMG_DARK_GRAY 0x02`

20.58.2.32 DMG_LITE_GRAY `#define DMG_LITE_GRAY 0x01`

20.58.2.33 DMG_WHITE `#define DMG_WHITE 0x00`

20.58.2.34 DMG_PALETTE `#define DMG_PALETTE(
 C0,
 C1,
 C2,
 C3) ((uint8_t) (((C3) & 0x03) << 6) | ((C2) & 0x03) << 4) | ((C1) & 0x03) <<
2) | ((C0) & 0x03)))`

Macro to create a DMG palette from 4 colors

Parameters

<i>C0</i>	Color for Index 0
<i>C1</i>	Color for Index 1
<i>C2</i>	Color for Index 2
<i>C3</i>	Color for Index 3

The resulting format is four greyscale colors packed into a single unsigned byte.

Example:

```
BGP_REG = DMG_PALETTE(DMG_BLACK, DMG_DARK_GRAY, DMG_LITE_GRAY, DMG_WHITE);
```

See also

[OBP0_REG](#), [OBP1_REG](#), [BGP_REG](#)
[DMG_BLACK](#), [DMG_DARK_GRAY](#), [DMG_LITE_GRAY](#), [DMG_WHITE](#)

20.58.2.35 SCREENWIDTH `#define SCREENWIDTH DEVICE_SCREEN_PX_WIDTHH`
Width of the visible screen in pixels.

20.58.2.36 SCREENHEIGHT `#define SCREENHEIGHT DEVICE_SCREEN_PX_HEIGHT`
Height of the visible screen in pixels.

20.58.2.37 MINWNDPOSX `#define MINWNDPOSX DEVICE_WINDOW_PX_OFFSET_X`
The Minimum X position of the Window Layer (Left edge of screen)

See also

[move_win\(\)](#)

20.58.2.38 MINWNDPOSY `#define MINWNDPOSY DEVICE_WINDOW_PX_OFFSET_Y`

The Minimum Y position of the Window Layer (Top edge of screen)

See also

[move_win\(\)](#)

20.58.2.39 MAXWNDPOSX `#define MAXWNDPOSX (DEVICE_WINDOW_PX_OFFSET_X + DEVICE_SCREEN_PX_WIDTH - 1)`

The Maximum X position of the Window Layer (Right edge of screen)

See also

[move_win\(\)](#)

20.58.2.40 MAXWNDPOSY `#define MAXWNDPOSY (DEVICE_WINDOW_PX_OFFSET_Y + DEVICE_SCREEN_PX_HEIGHT - 1)`

The Maximum Y position of the Window Layer (Bottom edge of screen)

See also

[move_win\(\)](#)

20.58.2.41 DMG_TYPE `#define DMG_TYPE 0x01`

Hardware Model: Original GB or Super GB.

See also

[_cpu](#)

20.58.2.42 MGB_TYPE `#define MGB_TYPE 0xFF`

Hardware Model: Pocket GB or Super GB 2.

See also

[_cpu](#)

20.58.2.43 CGB_TYPE `#define CGB_TYPE 0x11`

Hardware Model: Color GB.

See also

[_cpu](#)

20.58.2.44 GBA_NOT_DETECTED `#define GBA_NOT_DETECTED 0x00`
 Hardware Model: DMG, CGB or MGB.

See also

[_cpu, _is_GBA](#)

20.58.2.45 GBA_DETECTED `#define GBA_DETECTED 0x01`
 Hardware Model: GBA.

See also

[_cpu, _is_GBA](#)

20.58.2.46 DEVICE_SUPPORTS_COLOR `#define DEVICE_SUPPORTS_COLOR (_cpu == CGB_TYPE)`
 Macro returns TRUE if device supports color

20.58.2.47 VBL_DONE `#define VBL_DONE _vbl_done`

20.58.2.48 IO_IDLE `#define IO_IDLE 0x00U`
 Serial Link IO is completed

20.58.2.49 IO_SENDING `#define IO_SENDING 0x01U`
 Serial Link Sending data

20.58.2.50 IO_RECEIVING `#define IO_RECEIVING 0x02U`
 Serial Link Receiving data

20.58.2.51 IO_ERROR `#define IO_ERROR 0x04U`
 Serial Link Error

20.58.2.52 CURRENT_BANK `#define CURRENT_BANK _current_bank`

20.58.2.53 BANK `#define BANK( VARNAME ) ( (uint8_t) & __bank_ ## VARNAME )`
 Obtains the **bank number** of VARNAME

Parameters

<i>VARNAME</i>	Name of the variable which has a <code>__bank_</code> <i>VARNAME</i> companion symbol which is adjusted by bankpack
----------------	---

Use this to obtain the bank number from a bank reference created with [BANKREF\(\)](#).

See also

[BANKREF_EXTERN\(\)](#), [BANKREF\(\)](#)

20.58.2.54 BANKREF `#define BANKREF( VARNAME )`

Value:

```
void __func_ ## VARNAME(void) __banked __naked { \
__asm \
    .local b__func_ ## VARNAME \
    __bank_ ## VARNAME = b__func_ ## VARNAME \
    .globl __bank_ ## VARNAME \
__endasm; \
}
```

Creates a reference for retrieving the bank number of a variable or function

Parameters

VARNAME	Variable name to use, which may be an existing identifier
----------------	---

See also

[BANK\(\)](#) for obtaining the bank number of the included data.

More than one [BANKREF \(\)](#) may be created per file, but each call should always use a unique VARNAME. Use [BANKREF_EXTERN\(\)](#) within another source file to make the variable and it's data accesible there.

20.58.2.55 BANKREF_EXTERN #define BANKREF_EXTERN(
VARNAME) extern const void __bank_ ## VARNAME;

Creates extern references for accessing a [BANKREF\(\)](#) generated variable.

Parameters

VARNAME	Name of the variable used with BANKREF()
----------------	--

This makes a [BANKREF\(\)](#) reference in another source file accessible in the current file for use with [BANK\(\)](#).

See also

[BANKREF\(\)](#), [BANK\(\)](#)

20.58.2.56 SWITCH_ROM #define SWITCH_ROM(
b) (_current_bank = (b), rROMB0 = (b))

Makes default platform MBC switch the active ROM bank

Parameters

b	ROM bank to switch to (max 255)
----------	---------------------------------

- When used with MBC1 the max bank is Bank 31 (512K).
- When used with MBC5 the max bank is Bank 255 (4MB).
- To use the full 8MB size of MBC5 see [SWITCH_ROM_MBC5_8M\(\)](#).
- For MBC1 some banks in it's range are unavailable (typically 0x20, 0x40, 0x60).

Note

Using [SWITCH_ROM_MBC5_8M\(\)](#) should not be mixed with using [SWITCH_ROM_MBC5\(\)](#) and [SWITCH_ROM\(\)](#).

See also

[SWITCH_ROM_MBC1](#), [SWITCH_ROM_MBC5](#), [SWITCH_ROM_MEGADUCK](#)

20.58.2.57 SWITCH_RAM `#define SWITCH_RAM(
 b) (rRAMB = (b))`

Switches SRAM bank on MBC1 and other compatible MBCs

Parameters

<i>b</i>	SRAM bank to switch to
-----------------	------------------------

Before switching SRAM banks enable it using [ENABLE_RAM](#)

See also

[SWITCH_RAM_MBC1](#), [SWITCH_RAM_MBC5](#)

20.58.2.58 ENABLE_RAM `#define ENABLE_RAM (rRAMG = 0x0A)`
Enables SRAM on MBC1 and other compatible MBCs

20.58.2.59 DISABLE_RAM `#define DISABLE_RAM (rRAMG = 0x00)`
Disables SRAM on MBC1 and other compatible MBCs

20.58.2.60 SWITCH_ROM_MEGADUCK `#define SWITCH_ROM_MEGADUCK(
 b) SWITCH_ROM(b)`

Makes MEGADUCK MBC switch the active ROM bank

Parameters

<i>b</i>	ROM bank to switch to (max 3 for 64K, or 7 for 128K)
-----------------	--

20.58.2.61 SWITCH_ROM_MBC1 `#define SWITCH_ROM_MBC1(
 b) SWITCH_ROM(b)`

Makes MBC1 and other compatible MBCs switch the active ROM bank

Parameters

<i>b</i>	ROM bank to switch to
-----------------	-----------------------

For MBC1 some banks in it's range are unavailable (typically 0x20, 0x40, 0x60).
See pandocs for more details <https://gbdev.io/pandocs/MBC1>

20.58.2.62 SWITCH_RAM_MBC1 `#define SWITCH_RAM_MBC1(
 b) SWITCH_RAM(b)`

Switches SRAM bank on MBC1 and other compatible MBCs

Parameters

<i>b</i>	SRAM bank to switch to
-----------------	------------------------

Before switching SRAM banks enable it using [ENABLE_RAM](#)

See also

[SWITCH_RAM](#), [SWITCH_RAM_MBC5](#)

20.58.2.63 ENABLE_RAM_MBC1 `#define ENABLE_RAM_MBC1 ENABLE\_RAM`
Enables SRAM on MBC1

20.58.2.64 DISABLE_RAM_MBC1 `#define DISABLE_RAM_MBC1 DISABLE\_RAM`
Disables SRAM on MBC1

20.58.2.65 SWITCH_16_8_MODE_MBC1 `#define SWITCH_16_8_MODE_MBC1 (*(volatile uint8\_t *)0x6000 = 0x00)`

20.58.2.66 SWITCH_4_32_MODE_MBC1 `#define SWITCH_4_32_MODE_MBC1 (*(volatile uint8\_t *)0x6000 = 0x01)`

20.58.2.67 SWITCH_ROM_MBC5 `#define SWITCH_ROM_MBC5(
 b) (_current_bank = (b), rROMB1 = 0, rROMB0 = (b))`
Makes MBC5 switch to the active ROM bank

Parameters

<i>b</i>	ROM bank to switch to (max 255)
-----------------	---------------------------------

Supports up to ROM bank 255 (4 MB).

[SWITCH_ROM_MBC5_8M](#) may be used if the full 8MB size is needed.

Note

Using [SWITCH_ROM_MBC5_8M\(\)](#) should not be mixed with using [SWITCH_ROM_MBC5\(\)](#) and [SWITCH_ROM\(\)](#).

Note the order used here. Writing the other way around on a MBC1 always selects bank 1

20.58.2.68 SWITCH_ROM_MBC5_8M `#define SWITCH_ROM_MBC5_8M(
 b) (rROMB1 = ((uint16_t)b) >> 8), rROMB0 = (b))`
Makes MBC5 to switch the active ROM bank using the full 8MB size.

See also

[CURRENT_BANK](#)

Parameters

<i>b</i>	ROM bank to switch to
-----------------	-----------------------

This is an alternate to [SWITCH_ROM_MBC5](#) which is limited to 4MB.

Note:

- Banked SDCC calls are not supported if you use this macro.
- The active bank number is not tracked by [CURRENT_BANK](#) if you use this macro.
- Using [SWITCH_ROM_MBC5_8M\(\)](#) should not be mixed with using [SWITCH_ROM_MBC5\(\)](#) and [SWITCH_ROM\(\)](#).

Note the order used here. Writing the other way around on a MBC1 always selects bank 1

20.58.2.69 SWITCH_RAM_MBC5 `#define SWITCH_RAM_MBC5(
 b) SWITCH_RAM(b)`
Switches SRAM bank on MBC5

Parameters

<i>b</i>	SRAM bank to switch to
----------	------------------------

Before switching SRAM banks enable it using [ENABLE_RAM](#)

20.58.2.70 ENABLE_RAM_MBC5 `#define ENABLE_RAM_MBC5 ENABLE\_RAM`

Enables SRAM on MBC5

20.58.2.71 DISABLE_RAM_MBC5 `#define DISABLE_RAM_MBC5 DISABLE\_RAM`

Disables SRAM on MBC5

20.58.2.72 DISPLAY_ON `#define DISPLAY_ON LCDC\_REG |= LCDCF\_ON`

Turns the display back on.

See also

[display_off](#), [DISPLAY_OFF](#)

20.58.2.73 DISPLAY_OFF `#define DISPLAY_OFF display\_off();`

Turns the display off

Waits until the VBL before turning the display off.

See also

[display_off](#), [DISPLAY_ON](#)

20.58.2.74 HIDE_LEFT_COLUMN `#define HIDE_LEFT_COLUMN`

Does nothing for GB

20.58.2.75 SHOW_LEFT_COLUMN `#define SHOW_LEFT_COLUMN`

Does nothing for GB

20.58.2.76 SET_BORDER_COLOR `#define SET_BORDER_COLOR(
C)`

Does nothing for GB

20.58.2.77 SHOW_BKG `#define SHOW_BKG LCDC\_REG |= LCDCF\_BGON`

Turns on the background layer. Sets bit 0 of the LCDC register to 1.

Doesn't work in CGB mode - the bit is reused to control sprite priority over background and window layers instead.

- If 1 (SHOW_BKG), everything works as usual.
- If 0 (HIDE_BKG), all sprites are always drawn over background and window, ignoring any other priority settings.

20.58.2.78 HIDE_BKG `#define HIDE_BKG LCDC\_REG &= ~LCDCF\_BGON`

Turns off the background layer. Sets bit 0 of the LCDC register to 0.

Doesn't work in CGB mode - the bit is reused to control sprite priority over background and window layers instead.

- If 1 (SHOW_BKG), everything works as usual.
- If 0 (HIDE_BKG), all sprites are always drawn over background and window, ignoring any other priority settings.

20.58.2.79 SHOW_WIN `#define SHOW_WIN LCDC_REG|=LCDCF_WINON`

Turns on the Window layer Sets bit 5 of the LCDC register to 1.

This only controls Window visibility. If either the Background layer (which the window is part of) or the Display are not turned then the Window contents will not be visible. Those can be turned on using [SHOW_BKG](#) and [DISPLAY_ON](#).

20.58.2.80 HIDE_WIN `#define HIDE_WIN LCDC_REG&=~LCDCF_WINON`

Turns off the window layer. Clears bit 5 of the LCDC register to 0.

20.58.2.81 SHOW_SPRITES `#define SHOW_SPRITES LCDC_REG|=LCDCF_OBJON`

Turns on the sprites layer. Sets bit 1 of the LCDC register to 1.

20.58.2.82 HIDE_SPRITES `#define HIDE_SPRITES LCDC_REG&=~LCDCF_OBJON`

Turns off the sprites layer. Clears bit 1 of the LCDC register to 0.

See also

[hide_sprite](#), [hide_sprites_range](#)

20.58.2.83 SPRITES_8x16 `#define SPRITES_8x16 LCDC_REG|=LCDCF_OBJ16`

Sets sprite size to 8x16 pixels, two tiles one above the other. Sets bit 2 of the LCDC register to 1.

20.58.2.84 SPRITES_8x8 `#define SPRITES_8x8 LCDC_REG&=~LCDCF_OBJ16`

Sets sprite size to 8x8 pixels, one tile. Clears bit 2 of the LCDC register to 0.

20.58.2.85 COMPAT_PALETTE `#define COMPAT_PALETTE(`

`C0,`

`C1,`

`C2,`

`C3 ) ((uint8_t)((C3) << 6) | ((C2) << 4) | ((C1) << 2) | (C0)))`

20.58.2.86 set_bkg_2bpp_data `#define set_bkg_2bpp_data set_bkg_data`

20.58.2.87 set_tile_map `#define set_tile_map set_bkg_tiles`

20.58.2.88 set_tile_submap `#define set_tile_submap set_bkg_submap`

20.58.2.89 set_tile_xy `#define set_tile_xy set_bkg_tile_xy`

20.58.2.90 set_attribute_xy `#define set_attribute_xy set_bkg_attribute_xy`

20.58.2.91 set_sprite_2bpp_data `#define set_sprite_2bpp_data set_sprite_data`

20.58.2.92 DISABLE_OAM_DMA `#define DISABLE_OAM_DMA _shadow_OAM_base = 0`

20.58.2.93 DISABLE_VBL_TRANSFER `#define DISABLE_VBL_TRANSFER DISABLE\_OAM\_DMA`
 Disable OAM DMA copy each VBlank

20.58.2.94 ENABLE_OAM_DMA `#define ENABLE_OAM_DMA \_shadow\_OAM\_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)`

20.58.2.95 ENABLE_VBL_TRANSFER `#define ENABLE_VBL_TRANSFER ENABLE\_OAM\_DMA`
 Enable OAM DMA copy each VBlank and set it to transfer default shadow_OAM array

20.58.2.96 MAX_HARDWARE_SPRITES `#define MAX_HARDWARE_SPRITES 40`
 Amount of hardware sprites in OAM

20.58.2.97 HARDWARE_SPRITE_CAN_FLIP_X `#define HARDWARE_SPRITE_CAN_FLIP_X 1`
 True if sprite hardware can flip sprites by X (horizontally)

20.58.2.98 HARDWARE_SPRITE_CAN_FLIP_Y `#define HARDWARE_SPRITE_CAN_FLIP_Y 1`
 True if sprite hardware can flip sprites by Y (vertically)

20.58.2.99 fill_rect `#define fill_rect fill\_bkg\_rect`

20.58.3 Typedef Documentation

20.58.3.1 int_handler `typedef void(* int_handler) (void) NONBANKED`
 Interrupt handlers

20.58.3.2 OAM_item_t `typedef struct OAM\_item\_t OAM\_item\_t`
 Sprite Attributes structure

Parameters

<i>x</i>	X Coordinate of the sprite on screen
<i>y</i>	Y Coordinate of the sprite on screen
<i>tile</i>	Sprite tile number (see set_sprite_tile)
<i>prop</i>	OAM Property Flags (see set_sprite_prop)

20.58.4 Function Documentation

20.58.4.1 remove_VBL() `void remove_VBL ( int\_handler h )`

The remove functions will remove any interrupt handler.
 A handler of NULL will cause bad things to happen if the given interrupt is enabled.
 Removes the VBL interrupt handler.

See also

[add_VBL\(\)](#)

Removes the VBL interrupt handler.

See also

[add_VBL\(\)](#)

20.58.4.2 remove_LCD() `void remove_LCD (`
 `int_handler h )`

Removes the LCD interrupt handler.

See also

[add_LCD\(\)](#), [remove_VBL\(\)](#)

20.58.4.3 remove_TIM() `void remove_TIM (`
 `int_handler h )`

Removes the TIM interrupt handler.

See also

[add_TIM\(\)](#), [remove_VBL\(\)](#)

20.58.4.4 remove_SIO() `void remove_SIO (`
 `int_handler h )`

Removes the Serial Link / SIO interrupt handler.

See also

[add_SIO\(\)](#),
[remove_VBL\(\)](#)

The default SIO ISR gets installed automatically if any of the standard SIO calls are used ([send_byte\(\)](#), [receive_byte\(\)](#)).

Once installed the default SIO ISR cannot be removed. Only secondary chained SIO ISRs (added with [add_SIO\(\)](#)) can be removed.

20.58.4.5 remove_JOY() `void remove_JOY (`
 `int_handler h )`

Removes the JOY interrupt handler.

See also

[add_JOY\(\)](#), [remove_VBL\(\)](#)

20.58.4.6 add_VBL() `void add_VBL (`
 `int_handler h )`

Adds a Vertical Blanking interrupt handler.

Parameters

<i>h</i>	The handler to be called whenever a V-blank interrupt occurs.
----------	---

Up to 4 handlers may be added, with the last added being called last.

Do not use the function definition attributes [CRITICAL](#) and [INTERRUPT](#) when declaring ISR functions added via [add_VBL\(\)](#) (or LCD, etc). Those attributes are only required when constructing a bare jump from the interrupt vector itself (such as with [ISR_VECTOR\(\)](#)).

ISR handlers added using [add_VBL\(\)](#)/etc are instead called via the GBDK ISR dispatcher which makes the extra function attributes unnecessary.

Note

The default GBDK VBL is installed automatically.

See also

[ISR_VECTOR\(\)](#)

Adds a V-blank interrupt handler.

20.58.4.7 add_LCD() `void add_LCD (`
`int_handler h )`

Adds a LCD interrupt handler.

Called when the LCD interrupt occurs.

Up to 3 handlers may be added, with the last added being called last.

There are various sources controlled by the [STAT_REG](#) register (\$FF41) which can trigger this interrupt. Common examples include triggering on specific scanlines using [LY_REG](#) == [LYC_REG](#). Another is applying graphics effects on a per-scanline basis such as modifying the X and Y scroll registers ([SCX_REG](#) / [SCY_REG](#) registers).

Note

LYC may not trigger with scanline 0 in the same way as other scanlines due to particular behavior with scanlines 153 and 0. Instead, using an [add_VBL\(\)](#) interrupt handler for start of frame behavior may be more suitable.

Do not use the function definition attributes [CRITICAL](#) and [INTERRUPT](#) when declaring ISR functions added via [add_VBL\(\)](#) (or LCD, etc). Those attributes are only required when constructing a bare jump from the interrupt vector itself (such as with [ISR_VECTOR\(\)](#)).

ISR handlers added using [add_VBL/LCD/etc](#) are instead called via the GBDK ISR dispatcher which makes the extra function attributes unnecessary.

If this ISR is to be called once per each scanline then make sure that the time it takes to execute is less than the duration of a scanline.

See also

[add_VBL](#), [nowait_int_handler](#), [ISR_VECTOR\(\)](#)

Adds a LCD interrupt handler.

20.58.4.8 add_TIM() `void add_TIM (`
`int_handler h )`

Adds a timer interrupt handler.

Can not be used together with [add_low_priority_TIM](#)

This interrupt occurs when the [TIMA_REG](#) register (\$FF05) changes from \$FF to \$00.

Up to 4 handlers may be added, with the last added being called last.

Note

For NES make sure to wrap TIM interrupt handlers with a `nooverlay` pragma. For more details see [docs_nes_tim_overlay](#)

See also

[add_VBL](#)

[set_interrupts\(\)](#) with [TIM_IFLAG](#), [ISR_VECTOR\(\)](#)

20.58.4.9 add_low_priority_TIM() `void add_low_priority_TIM (`
 `int_handler h )`

Adds a timer interrupt handler, that could be interrupted by the other interrupts, as well as itself, if it runs too slow.

Can not be used together with [add_TIM](#)

This interrupt occurs when the [TIMA_REG](#) register (\$FF05) changes from \$FF to \$00.

Up to 4 handlers may be added, with the last added being called last.

See also

[add_VBL](#)

[set_interrupts\(\)](#) with [TIM_IFLAG](#), [ISR_VECTOR\(\)](#)

20.58.4.10 add_SIO() `void add_SIO (`
 `int_handler h )`

Adds a Serial Link transmit complete interrupt handler.

This interrupt occurs when a serial transfer has completed on the game link port.

Up to 4 handlers may be added, with the last added being called last.

The default SIO ISR gets installed automatically if any of the standard SIO calls are used ([send_byte\(\)](#), [receive_byte\(\)](#)).

See also

[send_byte](#), [receive_byte\(\)](#), [add_VBL\(\)](#)

[set_interrupts\(\)](#) with [SIO_IFLAG](#)

20.58.4.11 add_JOY() `void add_JOY (`
 `int_handler h )`

Adds a joypad button change interrupt handler.

This interrupt occurs on a transition of any of the keypad input lines from high to low, if the relevant [P1_REG](#) bits 4 or 5 are set.

For details about configuring flags or reading the data see: https://gbdev.io/pandocs/Interrupt_Sources.html#int-60-joypad-interrupt https://gbdev.io/pandocs/Joyypad_Input.html#ff00-p1joyp-joypad

Due to the fact that keypad "bounce" is virtually always present, software should expect this interrupt to occur one or more times for every button press and one or more times for every button release.

Up to 4 handlers may be added, with the last added being called last.

An example use of this is allowing the user to trigger an exit from the lower-power STOP cpu state.

See also

[joypad\(\)](#), [add_VBL\(\)](#), [IEF_HILO](#), [P1F_5](#), [P1F_4](#), [P1F_3](#), [P1F_2](#), [P1F_1](#), [P1F_0](#), [P1F_GET_DPAD](#), [P1F_GET_BTN](#), [P1F_GET_NONE](#)

20.58.4.12 nowait_int_handler() `void nowait_int_handler (`
 `void )`

Interrupt handler chain terminator that does **not** wait for .STAT

You must add this handler last in every interrupt handler chain if you want to change the default interrupt handler behaviour that waits for LCD controller mode to become 1 or 0 before return from the interrupt.

Example:

```
CRITICAL {
    add_SIO(nowait_int_handler); // Disable wait on VRAM state before returning from SIO interrupt
}
```

See also

[wait_int_handler\(\)](#)

20.58.4.13 wait_int_handler() `void wait_int_handler (`
`void )`

Default Interrupt handler chain terminator that waits for

See also

[STAT_REG](#) and **only** returns at the BEGINNING of either Mode 0 or Mode 1.

Used by default at the end of interrupt chains to help prevent graphical glitches. The glitches are caused when an ISR interrupts a graphics operation in one mode but returns in a different mode for which that graphics operation is not allowed.

See also

[nowait_int_handler\(\)](#)

20.58.4.14 cancel_pending_interrupts() `uint8_t cancel_pending_interrupts (`
`void ) [inline]`

Cancel pending interrupts

20.58.4.15 mode() `void mode (`
`uint8_t m )`

Set the current screen mode - one of M_* modes

Normally used by internal functions only.

See also

[M_DRAWING](#), [M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.58.4.16 get_mode() `uint8_t get_mode (`
`void )`

Returns the current mode

See also

[M_DRAWING](#), [M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.58.4.17 get_system() `uint8_t get_system (`
`void ) [inline]`

Returns the system gbdk is running on.

See also

[SYSTEM_50HZ](#), [SYSTEM_60HZ](#), [SYSTEM_BITS_DENDY](#), [SYSTEM_BITS_NTSC](#), [SYSTEM_BITS_PAL](#),
[SYSTEM_NTSC](#) [SYSTEM_PAL](#)

20.58.4.18 send_byte() `void send_byte (`
`void )`

Serial Link: Send the byte in [_io_out](#) out through the serial port

Make sure to enable interrupts for the Serial Link before trying to transfer data.

See also

[add_SIO\(\)](#), [remove_SIO\(\)](#)

[set_interrupts\(\)](#) with [SIO_IFLAG](#)

20.58.4.19 receive_byte() `void receive_byte (`
`void )`

Serial Link: Receive a byte from the serial port into `_io_in`

Make sure to enable interrupts for the Serial Link before trying to transfer data.

See also

`add_SIO()`, `remove_SIO()`
`set_interrupts()` with `SIO_IFLAG`

20.58.4.20 delay() `void delay (`
`uint16_t d )`

Delays the given number of milliseconds. Uses no timers or interrupts, and can be called with interrupts disabled

20.58.4.21 joypad() `uint8_t joypad (`
`void )`

Reads and returns the current state of the joypad. Follows Nintendo's guidelines for reading the pad. Return value is an OR of `J_*`

When testing for multiple different buttons, it's best to read the joypad state *once* into a variable and then test using that variable.

See also

`J_START`, `J_SELECT`, `J_A`, `J_B`, `J_UP`, `J_DOWN`, `J_LEFT`, `J_RIGHT`

20.58.4.22 waitpad() `uint8_t waitpad (`
`uint8_t mask )`

Waits until at least one of the buttons given in mask are pressed.

Parameters

<code>mask</code>	Bitmask indicating which buttons to wait for
-------------------	--

Normally only used for checking one key, but it will support many, even `J_LEFT` at the same time as `J_RIGHT`. :)

Note

Checks in a loop that doesn't HALT at all, so the CPU will be maxed out until this call returns.

See also

`joypad`
`J_START`, `J_SELECT`, `J_A`, `J_B`, `J_UP`, `J_DOWN`, `J_LEFT`, `J_RIGHT`

20.58.4.23 waitpadup() `void waitpadup (`
`void )`

Waits for the directional pad and all buttons to be released.

Note

Checks in a loop that doesn't HALT at all, so the CPU will be maxed out until this call returns.

20.58.4.24 `joypad_init()` `uint8_t joypad_init (`
 `uint8_t npads,`
 `joypads_t * joypads )`

Initializes `joypads_t` structure for polling multiple joypads (for the GB and ones connected via SGB)

Parameters

<code>npads</code>	number of joypads requested (1, 2 or 4)
<code>joypads</code>	pointer to <code>joypads_t</code> structure to be initialized

Only required for `joypad_ex`, not required for calls to regular `joypad()`

Returns

number of joypads available

See also

[joypad_ex\(\)](#), [joypads_t](#)

20.58.4.25 `joypad_ex()` `void joypad_ex (`
 `joypads_t * joypads )`

Polls all available joypads (for the GB and ones connected via SGB)

Parameters

<code>joypads</code>	pointer to <code>joypads_t</code> structure to be filled with joypad statuses, must be previously initialized with <code>joypad_init()</code>
----------------------	---

See also

[joypad_init\(\)](#), [joypads_t](#)

20.58.4.26 `enable_interrupts()` `void enable_interrupts (`
 `void ) [inline]`

Enables unmasked interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

See also

[disable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.58.4.27 `disable_interrupts()` `void disable_interrupts (`
 `void ) [inline]`

Disables interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

This function may be called as many times as you like; however the first call to [enable_interrupts](#) will re-enable them.

See also

[enable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.58.4.28 set_interrupts() `void set_interrupts (`
`uint8_t flags )`

Clears any pending interrupts and sets the interrupt mask register IO to flags.

Parameters

<i>flags</i>	A logical OR of *_IFLAGS
--------------	--------------------------

Note

This disables and then re-enables interrupts so it must be used outside of a critical section.

See also

[enable_interrupts\(\)](#), [disable_interrupts\(\)](#)
[VBL_IFLAG](#), [LCD_IFLAG](#), [TIM_IFLAG](#), [SIO_IFLAG](#), [JOY_IFLAG](#)

20.58.4.29 reset() `void reset (`
`void ) [inline]`

Performs a soft reset.

For the Game Boy and related it does this by jumping to address 0x0150 which is in crt0.s (the c-runtime that executes before main() is called).

This performs various startup steps such as resetting the stack, clearing WRAM and OAM, resetting initialized variables and some display registers (scroll, window, LCDC), etc.

This is not the same as a hard power reset.

20.58.4.30 vsync() `void vsync (`
`void )`

HALTs the CPU and waits for the vertical blank interrupt and then returns when all registered VBL ISRs have completed.

This is often used in main loops to idle the CPU at low power until it's time to start the next frame. It's also useful for syncing animation with the screen re-draw.

Warning: If the VBL interrupt is disabled, this function will never return. If the screen is off this function returns immediately.

20.58.4.31 wait_vbl_done() `void wait_vbl_done (`
`void )`

Obsolete. This function has been replaced by [vsync\(\)](#), which has identical behavior.

20.58.4.32 display_off() `void display_off (`
`void )`

Turns the display off.

Waits until the VBL before turning the display off.

See also

[DISPLAY_ON](#)

20.58.4.33 refresh_OAM() `void refresh_OAM (`
`void )`

Copies data from shadow OAM to OAM

20.58.4.34 hiramcpy() `void hiramcpy (`
`uint8_t dst,`
`const void * src,`
`uint8_t n )`

Copies data from somewhere in the lower address space to part of hi-ram.

Parameters

<i>dst</i>	Offset in high ram (0xFF00 and above) to copy to.
<i>src</i>	Area to copy from
<i>n</i>	Number of bytes to copy.

20.58.4.35 set_vram_byte() `void set_vram_byte (`
`uint8_t * addr,`
`uint8_t v )`

Set byte in vram at given memory location

Parameters

<i>addr</i>	address to write to
<i>v</i>	value

20.58.4.36 get_vram_byte() `uint8_t get_vram_byte (`
`uint8_t * addr )`

Get byte from vram at given memory location

Parameters

<i>addr</i>	address to read from
-------------	----------------------

Returns

read value

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

20.58.4.37 get_bkg_xy_addr() `uint8_t * get_bkg_xy_addr (`
`uint8_t x,`
`uint8_t y )`

Get address of X,Y tile of background map

20.58.4.38 set_2bpp_palette() `void set_2bpp_palette (`
`uint16_t palette ) [inline]`

Sets palette for 2bpp color translation for GG/SMS, does nothing on GB

20.58.4.39 set_1bpp_colors_ex() void set_1bpp_colors_ex (
 uint8_t fgcolor,
 uint8_t bgcolor,
 uint8_t mode)

Sets the Foreground and Background colors used by the set_*_1bpp_*() functions

Parameters

<i>fgcolor</i>	Foreground color
<i>bgcolor</i>	Background color
<i>mode</i>	Draw Mode

See [set_1bpp_colors](#) for details.

20.58.4.40 set_1bpp_colors() void set_1bpp_colors (
 uint8_t fgcolor,
 uint8_t bgcolor) [inline]

Sets the Foreground and Background colors used by the set_*_1bpp_*() functions

Parameters

<i>fgcolor</i>	Foreground color to use
<i>bgcolor</i>	Background color to use

The default colors are:

- Foreground: DMG_BLACK
- Background: DMG_WHITE

Example:

```
// Use DMG_BLACK as the Foreground color and DMG_LITE_GRAY  
// as the Background color when loading 1bpp tile data.  
set_1bpp_colors(DMG_BLACK, DMG_LITE_GRAY);
```

See also

[DMG_BLACK](#), [DMG_DARK_GRAY](#), [DMG_LITE_GRAY](#), [DMG_WHITE](#)
[set_bkg_1bpp_data](#), [set_win_1bpp_data](#), [set_sprite_1bpp_data](#)

20.58.4.41 set_bkg_data() void set_bkg_data (
 uint8_t first_tile,
 uint8_t nb_tiles,
 const uint8_t * data)

Sets VRAM Tile Pattern data for the Background / Window

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (2 bpp) source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**. Each Tile is 16 bytes in size (8x8 pixels, 2 bits-per-pixel).

Note

Sprite Tiles 128-255 share the same memory region as Background Tiles 128-255.

GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- `VBK_REG = VBK_BANK_0` indicates the first bank
- `VBK_REG = VBK_BANK_1` indicates the second

See also

[set_win_data](#), [set_tile_data](#)

20.58.4.42 `set_bkg_1bpp_data()` `void set_bkg_1bpp_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data )`

Sets VRAM Tile Pattern data for the Background / Window using 1bpp source data

Parameters

<i>first_tile</i>	Index of the first Tile to write
<i>nb_tiles</i>	Number of Tiles to write
<i>data</i>	Pointer to (1bpp) source Tile Pattern data

Similar to [set_bkg_data](#), except source data is 1 bit-per-pixel which gets expanded into 2 bits-per-pixel.
For a given bit that represent a pixel:

- 0 will be expanded into the Background color
- 1 will be expanded into the Foreground color

See [set_1bpp_colors](#) for details about setting the Foreground and Background colors.

See also

[SHOW_BKG](#), [HIDE_BKG](#), [set_bkg_tiles](#)
[set_win_1bpp_data](#), [set_sprite_1bpp_data](#)

20.58.4.43 `get_bkg_data()` `void get_bkg_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `uint8_t * data )`

Copies from Background / Window VRAM Tile Pattern data into a buffer

Parameters

<i>first_tile</i>	Index of the first Tile to read from
<i>nb_tiles</i>	Number of Tiles to read
<i>data</i>	Pointer to destination buffer for Tile Pattern data

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

Copies **nb_tiles** tiles from VRAM starting at **first_tile**, Tile data is copied into **data**.

Each Tile is 16 bytes, so the buffer pointed to by **data** should be at least **nb_tiles** x 16 bytes in size.

See also

[get_win_data](#), [get_data](#)

20.58.4.44 set_bkg_tiles() `void set_bkg_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles )`

Sets a rectangular region of Background Tile Map.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data

Entries are copied from map at **tiles** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

Use [set_bkg_submap\(\)](#) instead when:

- Source map is wider than 32 tiles.
- Writing a width that does not match the source map width **and** more than one row high at a time.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

Note

Patterns 128-255 overlap with patterns 128-255 of the sprite Tile Pattern table.

GBC only: [VBK_REG](#) determines whether Tile Numbers or Tile Attributes get set.

- [VBK_REG](#) = [VBK_TILES](#) Tile Numbers are written
- [VBK_REG](#) = [VBK_ATTRIBUTES](#) Tile Attributes are written

GBC Tile Attributes are defined as:

- Bit 7 - Priority flag. When this is set, it puts the tile above the sprites with colour 0 being transparent.
0: Below sprites
1: Above sprites
Note: [SHOW_BKG](#) needs to be set for these priorities to take place.
- Bit 6 - Vertical flip. Dictates which way up the tile is drawn vertically.
0: Normal
1: Flipped Vertically
- Bit 5 - Horizontal flip. Dictates which way up the tile is drawn horizontally.
0: Normal
1: Flipped Horizontally

- Bit 4 - Not used
- Bit 3 - Character Bank specification. Dictates from which bank of Background Tile Patterns the tile is taken.
0: Bank 0
1: Bank 1
- Bit 2 - See bit 0.
- Bit 1 - See bit 0.
- Bit 0 - Bits 0-2 indicate which of the 7 BKG colour palettes the tile is assigned.

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_submap](#), [set_win_tiles](#), [set_tiles](#)

20.58.4.45 set_bkg_based_tiles() `void set_bkg_based_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles,`
`uint8_t base_tile ) [inline]`

Sets a rectangular region of Background Tile Map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_tiles\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_tiles](#) for more details

20.58.4.46 set_bkg_attributes() `void set_bkg_attributes (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles ) [inline]`

Sets a rectangular region of Background Tile Map Attributes.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31

Parameters

<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map attribute data

Entries are copied from map at **tiles** to the Background Tile Map starting at **x, y** writing across for **w** tiles and down for **h** tiles.

Use [set_bkg_submap_attributes\(\)](#) instead when:

- Source map is wider than 32 tiles.
- Writing a width that does not match the source map width **and** more than one row high at a time.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

GBC Tile Attributes are defined as:

- Bit 7 - Priority flag. When this is set, it puts the tile above the sprites with colour 0 being transparent.
0: Below sprites
1: Above sprites
Note: [SHOW_BKG](#) needs to be set for these priorities to take place.
- Bit 6 - Vertical flip. Dictates which way up the tile is drawn vertically.
0: Normal
1: Flipped Vertically
- Bit 5 - Horizontal flip. Dictates which way up the tile is drawn horizontally.
0: Normal
1: Flipped Horizontally
- Bit 4 - Not used
- Bit 3 - Character Bank specification. Dictates from which bank of Background Tile Patterns the tile is taken.
0: Bank 0
1: Bank 1
- Bit 2 - See bit 0.
- Bit 1 - See bit 0.
- Bit 0 - Bits 0-2 indicate which of the 7 BKG colour palettes the tile is assigned.

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_submap_attributes](#), [set_win_tiles](#), [set_tiles](#)

Note

On the Game Boy this is only usable in Game Boy Color mode

```
20.58.4.47 set_bkg_submap() void set_bkg_submap (
    uint8_t x,
    uint8_t y,
    uint8_t w,
    uint8_t h,
    const uint8_t * map,
    uint8_t map_w ) [inline]
```

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. Useful for scrolling implementations of maps larger than 32 x 32 tiles.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map</i> ↔ <i>_w</i>	Width of source tile map in tiles. Range 1 - 255

Entries are copied from **map** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles, using **map_w** as the rowstride for the source tile map.

The **x** and **y** parameters are in Source Tile Map tile coordinates. The location tiles will be written to on the hardware Background Map is derived from those, but only uses the lower 5 bits of each axis, for range of 0-31 (they are bit-masked: $x \& 0x1F$ and $y \& 0x1F$). As a result the two coordinate systems are aligned together.

In order to transfer tile map data in a way where the coordinate systems are not aligned, an offset from the Source Tile Map pointer can be passed in: $(map_ptr + x + (y * map_width))$.

For example, if you want the tile id at 1, 2 from the source map to show up at 0, 0 on the hardware Background Map (instead of at 1, 2) then modify the pointer address that is passed in: $map_ptr + 1 + (2 * map_width)$

Use this instead of [set_bkg_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

See [set_bkg_tiles](#) for setting CGB attribute maps with [VBK_REG](#).

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_tiles](#), [set_win_submap](#), [set_tiles](#)

```
20.58.4.48 set_bkg_based_submap() void set_bkg_based_submap (
    uint8_t x,
    uint8_t y,
    uint8_t w,
    uint8_t h,
    const uint8_t * map,
    uint8_t map_w,
    uint8_t base_tile ) [inline]
```

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_submap](#) for more details

20.58.4.49 set_bkg_submap_attributes() `void set_bkg_submap_attributes (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * map,`
`uint8_t map_w ) [inline]`

Sets a rectangular area of the Background Tile Map Attributes using a sub-region from a source tile attribute map. Useful for scrolling implementations of maps larger than 32 x 32 tiles.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map attribute data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255

Entries are copied from **map** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles, using **map_w** as the rowstride for the source tile map.

The **x** and **y** parameters are in Source Tile Map tile coordinates. The location tiles will be written to on the hardware Background Map is derived from those, but only uses the lower 5 bits of each axis, for range of 0-31 (they are bit-masked: $x \& 0x1F$ and $y \& 0x1F$). As a result the two coordinate systems are aligned together.

In order to transfer tile map data in a way where the coordinate systems are not aligned, an offset from the Source Tile Map pointer can be passed in: $(map_ptr + x + (y * map_width))$.

For example, if you want the tile id at 1, 2 from the source map to show up at 0, 0 on the hardware Background Map (instead of at 1, 2) then modify the pointer address that is passed in: $map_ptr + 1 + (2 * map_width)$

Use this instead of [set_bkg_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

See [set_bkg_tiles](#) for setting CGB attribute maps with [VBK_REG](#).

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_attributes](#), [set_win_submap](#), [set_tiles](#)

Note

On the Game Boy this is only usable in Game Boy Color mode

20.58.4.50 get_bkg_tiles() `void get_bkg_tiles (`
`uint8_t x,`

```
uint8_t y,
uint8_t w,
uint8_t h,
uint8_t * tiles )
```

Copies a rectangular region of Background Tile Map entries into a buffer.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to copy in tiles. Range 0 - 31
<i>h</i>	Height of area to copy in tiles. Range 0 - 31
<i>tiles</i>	Pointer to destination buffer for Tile Map data

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

Entries are copied into **tiles** from the Background Tile Map starting at **x**, **y** reading across for **w** tiles and down for **h** tiles.

One byte per tile.

The buffer pointed to by **tiles** should be at least **x** x **y** bytes in size.

See also

[get_win_tiles](#), [get_bkg_tile_xy](#), [get_tiles](#), [get_vram_byte](#)

20.58.4.51 set_bkg_tile_xy() `uint8_t * set_bkg_tile_xy (`
`uint8_t x,`
`uint8_t y,`
`uint8_t t )`

Set single tile t on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>t</i>	tile index

Returns

returns the address of tile, so you may use faster [set_vram_byte\(\)](#) later

20.58.4.52 set_bkg_attribute_xy() `uint8_t * set_bkg_attribute_xy (`
`uint8_t x,`
`uint8_t y,`
`uint8_t a ) [inline]`

Set single attribute data a on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>a</i>	tile attributes

Returns

returns the address of tile attribute, so you may use faster [set_vram_byte\(\)](#) later

Note

On the Game Boy this is only usable in Game Boy Color mode

20.58.4.53 [get_bkg_tile_xy\(\)](#) `uint8_t get_bkg_tile_xy (`
 `uint8_t x,`
 `uint8_t y )`

Get single tile t on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate

Returns

returns tile index

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

20.58.4.54 [move_bkg\(\)](#) `void move_bkg (`
 `uint8_t x,`
 `uint8_t y ) [inline]`

Moves the Background Layer to the position specified in **x** and **y** in pixels.

Parameters

<i>x</i>	X axis screen coordinate for Left edge of the Background
<i>y</i>	Y axis screen coordinate for Top edge of the Background

0,0 is the top left corner of the GB screen. The Background Layer wraps around the screen, so when part of it goes off the screen it appears on the opposite side (factoring in the larger size of the Background Layer versus the screen size).

The background layer is always under the Window Layer.

See also

[SHOW_BKG](#), [HIDE_BKG](#)

20.58.4.55 [scroll_bkg\(\)](#) `void scroll_bkg (`
 `int8_t x,`
 `int8_t y ) [inline]`

Moves the Background relative to it's current position.

Parameters

<i>x</i>	Number of pixels to move the Background on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the Background on the Y axis Range: -128 - 127

See also

[move_bkg](#)

20.58.4.56 `get_win_xy_addr()` `uint8_t * get_win_xy_addr (`
 `uint8_t x,`
 `uint8_t y )`

Get address of X,Y tile of window map

20.58.4.57 `set_win_data()` `void set_win_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data )`

Sets VRAM Tile Pattern data for the Window / Background

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (2 bpp) source Tile Pattern data.

This is the same as [set_bkg_data](#), since the Window Layer and Background Layer share the same Tile pattern data.

See also

[set_bkg_data](#)

[set_win_tiles](#), [set_bkg_data](#), [set_data](#)

[SHOW_WIN](#), [HIDE_WIN](#)

20.58.4.58 `set_win_1bpp_data()` `void set_win_1bpp_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data )`

Sets VRAM Tile Pattern data for the Window / Background using 1bpp source data

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (1bpp) source Tile Pattern data

This is the same as [set_bkg_1bpp_data](#), since the Window Layer and Background Layer share the same Tile pattern data.

For a given bit that represent a pixel:

- 0 will be expanded into the Background color

- 1 will be expanded into the Foreground color

See [set_1bpp_colors](#) for details about setting the Foreground and Background colors.

See also

[set_bkg_data](#), [set_win_data](#), [set_1bpp_colors](#)
[set_bkg_1bpp_data](#), [set_sprite_1bpp_data](#)

20.58.4.59 get_win_data() `void get_win_data (`
`uint8_t first_tile,`
`uint8_t nb_tiles,`
`uint8_t * data )`

Copies from Window / Background VRAM Tile Pattern data into a buffer

Parameters

<i>first_tile</i>	Index of the first Tile to read from
<i>nb_tiles</i>	Number of Tiles to read
<i>data</i>	Pointer to destination buffer for Tile Pattern Data

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

This is the same as [get_bkg_data](#), since the Window Layer and Background Layer share the same Tile pattern data.

See also

[get_bkg_data](#), [get_data](#)

20.58.4.60 set_win_tiles() `void set_win_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles )`

Sets a rectangular region of the Window Tile Map.

Parameters

<i>x</i>	X Start position in Window Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Window Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data

Entries are copied from map at **tiles** to the Window Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

Use [set_win_submap\(\)](#) instead when:

- Source map is wider than 32 tiles.
- Writing a width that does not match the source map width **and** more than one row high at a time.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

Note

Patterns 128-255 overlap with patterns 128-255 of the sprite Tile Pattern table.

GBC only: [VBK_REG](#) determines whether Tile Numbers or Tile Attributes get set.

- [VBK_REG](#) = [VBK_TILES](#) Tile Numbers are written
- [VBK_REG](#) = [VBK_ATTRIBUTES](#) Tile Attributes are written

For more details about GBC Tile Attributes see [set_bkg_tiles](#).

See also

[SHOW_WIN](#), [HIDE_WIN](#), [set_win_submap](#), [set_bkg_tiles](#), [set_bkg_data](#), [set_tiles](#)

20.58.4.61 set_win_based_tiles() `void set_win_based_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles,`
`uint8_t base_tile ) [inline]`

Sets a rectangular region of the Window Tile Map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Window Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Window Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_win_tiles\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_win_tiles](#) for more details

20.58.4.62 set_win_submap() `void set_win_submap (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * map,`
`uint8_t map_w ) [inline]`

Sets a rectangular area of the Window Tile Map using a sub-region from a source tile map.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Window Map tile coordinates. Range 0 - 255
----------	--

Parameters

<i>y</i>	Y Start position in both the Source Tile Map and hardware Window Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255

Entries are copied from **map** to the Window Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles, using **map_w** as the rowstride for the source tile map.

The **x** and **y** parameters are in Source Tile Map tile coordinates. The location tiles will be written to on the hardware Background Map is derived from those, but only uses the lower 5 bits of each axis, for range of 0-31 (they are bit-masked: $x \& 0x1F$ and $y \& 0x1F$). As a result the two coordinate systems are aligned together.

In order to transfer tile map data in a way where the coordinate systems are not aligned, an offset from the Source Tile Map pointer can be passed in: $(map_ptr + x + (y * map_width))$.

For example, if you want the tile id at 1, 2 from the source map to show up at 0, 0 on the hardware Background Map (instead of at 1, 2) then modify the pointer address that is passed in: $map_ptr + 1 + (2 * map_width)$

Use this instead of [set_win_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

GBC only: [VBK_REG](#) determines whether Tile Numbers or Tile Attributes get set.

- [VBK_REG](#) = [VBK_TILES](#) Tile Numbers are written
- [VBK_REG](#) = [VBK_ATTRIBUTES](#) Tile Attributes are written

See [set_bkg_tiles](#) for details about CGB attribute maps with [VBK_REG](#).

See also

[SHOW_WIN](#), [HIDE_WIN](#), [set_win_tiles](#), [set_bkg_submap](#), [set_bkg_tiles](#), [set_bkg_data](#), [set_tiles](#)

20.58.4.63 [set_win_based_submap\(\)](#) `void set_win_based_submap (`

```

uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * map,
uint8_t map_w,
uint8_t base_tile ) [inline]
```

Sets a rectangular area of the Window Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Window Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Window Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_win_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_win_submap](#) for more details

20.58.4.64 get_win_tiles() `void get_win_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`uint8_t * tiles )`

Copies a rectangular region of Window Tile Map entries into a buffer.

Parameters

<i>x</i>	X Start position in Window Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Window Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to copy in tiles. Range 0 - 31
<i>h</i>	Height of area to copy in tiles. Range 0 - 31
<i>tiles</i>	Pointer to destination buffer for Tile Map data

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

Entries are copied into **tiles** from the Window Tile Map starting at **x**, **y** reading across for **w** tiles and down for **h** tiles.

One byte per tile.

The buffer pointed to by **tiles** should be at least **x x y** bytes in size.

See also

[get_bkg_tiles](#), [get_bkg_tile_xy](#), [get_tiles](#), [get_vram_byte](#)

20.58.4.65 set_win_tile_xy() `uint8_t * set_win_tile_xy (`
`uint8_t x,`
`uint8_t y,`
`uint8_t t )`

Set single tile t on window layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>t</i>	tile index

Returns

returns the address of tile, so you may use faster [set_vram_byte\(\)](#) later

20.58.4.66 get_win_tile_xy() `uint8_t get_win_tile_xy (`
 `uint8_t x,`
 `uint8_t y )`

Get single tile t on window layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate

Returns

returns the tile index

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

20.58.4.67 move_win() `void move_win (`
 `uint8_t x,`
 `uint8_t y ) [inline]`

Moves the Window to the **x, y** position on the screen.

Parameters

<i>x</i>	X coordinate for Left edge of the Window (actual displayed location will be X - 7)
<i>y</i>	Y coordinate for Top edge of the Window

7,0 is the top left corner of the screen in Window coordinates. The Window is locked to the bottom right corner. The Window is always over the Background layer.

See also

[SHOW_WIN](#), [HIDE_WIN](#)

20.58.4.68 scroll_win() `void scroll_win (`
 `int8_t x,`
 `int8_t y ) [inline]`

Move the Window relative to its current position.

Parameters

<i>x</i>	Number of pixels to move the window on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the window on the Y axis Range: -128 - 127

See also

[move_win](#)

20.58.4.69 set_sprite_data() void set_sprite_data (
 uint8_t first_tile,
 uint8_t nb_tiles,
 const uint8_t * data)

Sets VRAM Tile Pattern data for Sprites

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (2 bpp) source Tile Pattern data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**. Each Tile is 16 bytes in size (8x8 pixels, 2 bits-per-pixel).

Note

Sprite Tiles 128-255 share the same memory region as Background Tiles 128-255.

GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- VBK_REG = [VBK_BANK_0](#) indicates the first bank
- VBK_REG = [VBK_BANK_1](#) indicates the second

20.58.4.70 set_sprite_1bpp_data() void set_sprite_1bpp_data (
 uint8_t first_tile,
 uint8_t nb_tiles,
 const uint8_t * data)

Sets VRAM Tile Pattern data for Sprites using 1bpp source data

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (1bpp) source Tile Pattern data

Similar to [set_sprite_data](#), except source data is 1 bit-per-pixel which gets expanded into 2 bits-per-pixel. For a given bit that represent a pixel:

- 0 will be expanded into the Background color
- 1 will be expanded into the Foreground color

See [set_1bpp_colors](#) for details about setting the Foreground and Background colors.

See also

[SHOW_SPRITES](#), [HIDE_SPRITES](#), [set_sprite_tile](#)
[set_bkg_1bpp_data](#), [set_win_1bpp_data](#)

20.58.4.71 get_sprite_data() void get_sprite_data (
 uint8_t first_tile,
 uint8_t nb_tiles,
 uint8_t * data)

Copies from Sprite VRAM Tile Pattern data into a buffer

Parameters

<i>first_tile</i>	Index of the first tile to read from
<i>nb_tiles</i>	Number of tiles to read
<i>data</i>	Pointer to destination buffer for Tile Pattern data

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

Copies **nb_tiles** tiles from VRAM starting at **first_tile**, tile data is copied into **data**.

Each Tile is 16 bytes, so the buffer pointed to by **data** should be at least **nb_tiles** x 16 bytes in size.

20.58.4.72 SET_SHADOW_OAM_ADDRESS() `void SET_SHADOW_OAM_ADDRESS ( void * address ) [inline]`

Enable OAM DMA copy each VBlank and set it to transfer any 256-byte aligned array

20.58.4.73 set_sprite_tile() `void set_sprite_tile ( uint8_t nb, uint8_t tile ) [inline]`

Sets sprite number **nb** in the OAM to display tile number **tile**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>tile</i>	Selects a tile (0 - 255) from memory at 8000h - 8FFFh In CGB Mode this could be either in VRAM Bank 0 or 1, depending on Bit 3 of the OAM Attribute Flag (see set_sprite_prop)

In 8x16 mode:

- The sprite will also display the next tile (**tile** + 1) directly below (y + 8) the first tile.
- The lower bit of the tile number is ignored: the upper 8x8 tile is (**tile** & 0xFE), and the lower 8x8 tile is (**tile** | 0x01).
- See: [SPRITES_8x16](#)

20.58.4.74 get_sprite_tile() `uint8_t get_sprite_tile ( uint8_t nb ) [inline]`

Returns the tile number of sprite number **nb** in the OAM.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_tile](#) for more details

20.58.4.75 set_sprite_prop() `void set_sprite_prop ( uint8_t nb, uint8_t prop ) [inline]`

Sets the OAM Property Flags of sprite number **nb** to those defined in **prop**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>prop</i>	Property setting (see bitfield description)

The bits in **prop** represent:

- Bit 7 - Priority flag. When this is set the sprites appear behind the background and window layer.
0: infront
1: behind
- Bit 6 - Vertical flip. Dictates which way up the sprite is drawn vertically.
0: normal
1:upside down
- Bit 5 - Horizontal flip. Dictates which way up the sprite is drawn horizontally.
0: normal
1:back to front
- Bit 4 - DMG/Non-CGB Mode Only. Assigns either one of the two b/w palettes to the sprite.
0: OBJ palette 0
1: OBJ palette 1
- Bit 3 - GBC only. Dictates from which bank of Sprite Tile Patterns the tile is taken.
0: Bank 0
1: Bank 1
- Bit 2 - See bit 0.
- Bit 1 - See bit 0.
- Bit 0 - GBC only. Bits 0-2 indicate which of the 7 OBJ colour palettes the sprite is assigned.

It's recommended to use GBDK constants (eg: S_FLIPY) to configure sprite properties as these are crossplatform.

```
// Load palette data into the first palette
set_sprite_palette(4, 1, exampleSprite_palettes)
// Set the OAM value for the sprite
// These flags tell the sprite to flip both vertically and horizontally.
set_sprite_prop(0, S_FLIPY | S_FLIPX);
```

See also

[S_PALETTE](#), [S_FLIPX](#), [S_FLIPY](#), [S_PRIORITY](#)

20.58.4.76 `get_sprite_prop()` `uint8_t get_sprite_prop (`
`uint8_t nb ) [inline]`

Returns the OAM Property Flags of sprite number **nb**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_prop](#) for property bitfield settings

20.58.4.77 `move_sprite()` `void move_sprite (`
`uint8_t nb,`

```
uint8_t x,
uint8_t y ) [inline]
```

Moves sprite number **nb** to the **x**, **y** position on the screen.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	X Position. Specifies the sprites horizontal position on the screen (minus 8). An offscreen value (X=0 or X>=168) hides the sprite, but the sprite still affects the priority ordering - a better way to hide a sprite is to set its Y-coordinate offscreen.
<i>y</i>	Y Position. Specifies the sprites vertical position on the screen (minus 16). An offscreen value (for example, Y=0 or Y>=160) hides the sprite.

Moving the sprite to 0,0 (or similar off-screen location) will hide it.

20.58.4.78 scroll_sprite() void scroll_sprite (

```
uint8_t nb,
int8_t x,
int8_t y ) [inline]
```

Moves sprite number **nb** relative to its current position.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	Number of pixels to move the sprite on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the sprite on the Y axis Range: -128 - 127

See also

[move_sprite](#) for more details about the X and Y position

20.58.4.79 hide_sprite() void hide_sprite (

```
uint8_t nb ) [inline]
```

Hides sprite number **nb** by moving it to zero position by Y.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[hide_sprites_range](#), [HIDE_SPRITES](#)

20.58.4.80 set_data() void set_data (

```
uint8_t * vram_addr,
const uint8_t * data,
uint16_t len )
```

Copies arbitrary data to an address in VRAM without taking into account the state of LCDC bits 3 or 4.

Parameters

<i>vram_addr</i>	Pointer to destination VRAM Address
<i>data</i>	Pointer to source buffer
<i>len</i>	Number of bytes to copy

Copies **len** bytes from a buffer at **data** to VRAM starting at **vram_addr**.

GBC only: **VBK_REG** determines which bank of tile patterns are written to.

- **VBK_REG** = **VBK_BANK_0** indicates the first bank
- **VBK_REG** = **VBK_BANK_1** indicates the second

See also

[set_bkg_data](#), [set_win_data](#), [set_bkg_tiles](#), [set_win_tiles](#), [set_tile_data](#), [set_tiles](#)

20.58.4.81 **get_data()** `void get_data (`
 `uint8_t * data,`
 `uint8_t * vram_addr,`
 `uint16_t len )`

Copies arbitrary data from an address in VRAM into a buffer without taking into account the state of LCDC bits 3 or 4.

Parameters

<i>vram_addr</i>	Pointer to source VRAM Address
<i>data</i>	Pointer to destination buffer
<i>len</i>	Number of bytes to copy

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

Copies **len** bytes from VRAM starting at **vram_addr** into a buffer at **data**.

GBC only: **VBK_REG** determines which bank of tile patterns are written to.

- **VBK_REG** = **VBK_BANK_0** indicates the first bank
- **VBK_REG** = **VBK_BANK_1** indicates the second

See also

[get_bkg_data](#), [get_win_data](#), [get_bkg_tiles](#), [get_win_tiles](#), [get_tiles](#)

20.58.4.82 **vmemcpy()** `void vmemcpy (`
 `uint8_t * dest,`
 `uint8_t * sour,`
 `uint16_t len )`

Copies arbitrary data from an address in VRAM into a buffer

Parameters

<i>dest</i>	Pointer to destination buffer (may be in VRAM)
<i>sour</i>	Pointer to source buffer (may be in VRAM)
<i>len</i>	Number of bytes to copy

Copies **len** bytes from or to VRAM starting at **sour** into a buffer or to VRAM at **dest**.
 GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- `VBK_REG = VBK_BANK_0` indicates the first bank
- `VBK_REG = VBK_BANK_1` indicates the second

20.58.4.83 set_tiles() `void set_tiles (`
 `uint8_t x,`
 `uint8_t y,`
 `uint8_t w,`
 `uint8_t h,`
 `uint8_t * vram_addr,`
 `const uint8_t * tiles )`

Sets a rectangular region of Tile Map entries at a given VRAM Address without taking into account the state of LCDC bit 3.

Parameters

<i>x</i>	X Start position in Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>vram_addr</i>	Pointer to destination VRAM Address
<i>tiles</i>	Pointer to source Tile Map data

Entries are copied from **tiles** to Tile Map at address `vram_addr` starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

One byte per source tile map entry.

There are two 32x32 Tile Maps in VRAM at addresses 9800h-9BFFh and 9C00h-9FFFh.

GBC only: [VBK_REG](#) determines whether Tile Numbers or Tile Attributes get set.

- `VBK_REG = VBK_TILES` Tile Numbers are written
- `VBK_REG = VBK_ATTRIBUTES` Tile Attributes are written

See also

[set_bkg_tiles](#), [set_win_tiles](#)

20.58.4.84 set_tile_data() `void set_tile_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data,`
 `uint8_t base )`

Sets VRAM Tile Pattern data starting from given base address without taking into account the state of LCDC bit 4.

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (2 bpp) source Tile Pattern data.
<i>base</i>	MSB of the destination address in VRAM (usually 0x80 or 0x90 which gives 0x8000 or 0x9000)

See also

[set_bkg_data](#), [set_win_data](#), [set_data](#)

20.58.4.85 **get_tiles()** `void get_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`uint8_t * vram_addr,`
`uint8_t * tiles )`

Copies a rectangular region of Tile Map entries from a given VRAM Address into a buffer without taking into account the state of LCDC bit 3.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to copy in tiles. Range 0 - 31
<i>h</i>	Height of area to copy in tiles. Range 0 - 31
<i>vram_addr</i>	Pointer to source VRAM Address
<i>tiles</i>	Pointer to destination buffer for Tile Map data

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

Entries are copied into **tiles** from the Background Tile Map starting at **x**, **y** reading across for **w** tiles and down for **h** tiles.

One byte per tile.

There are two 32x32 Tile Maps in VRAM at addresses 9800h - 9BFFh and 9C00h - 9FFFh.

The buffer pointed to by **tiles** should be at least **x** x **y** bytes in size.

See also

[get_bkg_tiles](#), [get_win_tiles](#)

20.58.4.86 **set_native_tile_data()** `void set_native_tile_data (`
`uint16_t first_tile,`
`uint8_t nb_tiles,`
`const uint8_t * data ) [inline]`

Sets VRAM Tile Pattern data in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write (0 - 511)
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source Tile Pattern data.

When *first_tile* is larger than 256 on the GB/AP, it will write to sprite data instead of background data.

The bit depth of the source Tile Pattern data depends on which console is being used:

- Game Boy/Analogue Pocket: loads 2bpp tiles data
- SMS/GG: loads 4bpp tile data

20.58.4.87 set_bkg_native_data() `void set_bkg_native_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data ) [inline]`

Sets VRAM Tile Pattern data for the Background / Window in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**.
GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- [VBK_REG](#) = [VBK_BANK_0](#) indicates the first bank
- [VBK_REG](#) = [VBK_BANK_1](#) indicates the second

See also

[set_win_data](#), [set_tile_data](#)

20.58.4.88 set_sprite_native_data() `void set_sprite_native_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data ) [inline]`

Sets VRAM Tile Pattern data for Sprites in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**.
GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- [VBK_REG](#) = [VBK_BANK_0](#) indicates the first bank
- [VBK_REG](#) = [VBK_BANK_1](#) indicates the second

20.58.4.89 init_win() `void init_win (`
 `uint8_t c )`

Initializes the entire Window Tile Map with Tile Number **c**

Parameters

<i>c</i>	Tile number to fill with
----------	--------------------------

Note

This function avoids writes during modes 2 & 3

20.58.4.90 init_bkg() `void init_bkg (`
`uint8_t c )`

Initializes the entire Background Tile Map with Tile Number **c**

Parameters

<i>c</i>	Tile number to fill with
----------	--------------------------

Note

This function avoids writes during modes 2 & 3

20.58.4.91 vmemset() `void vmemset (`
`void * s,`
`uint8_t c,`
`size_t n )`

Fills the VRAM memory region **s** of size **n** with Tile Number **c**

Parameters

<i>s</i>	Start address in VRAM
<i>c</i>	Tile number to fill with
<i>n</i>	Size of memory region (in bytes) to fill

Note

This function avoids writes during modes 2 & 3

20.58.4.92 fill_bkg_rect() `void fill_bkg_rect (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`uint8_t tile )`

Fills a rectangular region of Tile Map entries for the Background layer with tile.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tile</i>	Fill value

20.58.4.93 fill_win_rect() `void fill_win_rect (`

```
uint8_t x,  
uint8_t y,  
uint8_t w,  
uint8_t h,  
uint8_t tile )
```

Fills a rectangular region of Tile Map entries for the Window layer with tile.

Parameters

<i>x</i>	X Start position in Window Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Window Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tile</i>	Fill value

20.58.5 Variable Documentation

20.58.5.1 **c** void c

20.58.5.2 **d** void d

20.58.5.3 **e** void e

20.58.5.4 **h** void h

20.58.5.5 **l** void l

Initial value:

```
{  
    __asm__("ei")
```

20.58.5.6 **_cpu** uint8_t _cpu [extern]
GB CPU type

See also

```
DMG_TYPE, MGB_TYPE, CGB_TYPE, cpu_fast(), cpu_slow(), _is_GBA
```

20.58.5.7 **_is_GBA** uint8_t _is_GBA [extern]
GBA detection

See also

```
GBA_DETECTED, GBA_NOT_DETECTED, _cpu
```

20.58.5.8 **sys_time** volatile uint16_t sys_time [extern]
Global Time Counter in VBL periods (60Hz)
Increments once per Frame
Will wrap around every ~18 minutes (unsigned 16 bits = 65535 / 60 / 60 = 18.2)

20.58.5.9 `_vbl_done` `__REG` `_vbl_done`

Flag indicating the VBlank ISR has run

Flag gets cleared at the start of `vsync()` / `wait_vbl_done()` and set in the default VBlank ISR handler.

20.58.5.10 `_io_status` `volatile` `uint8_t` `_io_status` `[extern]`

Serial Link: Current IO Status. An OR of `IO_*`

20.58.5.11 `_io_in` `volatile` `uint8_t` `_io_in` `[extern]`

Serial Link: Byte just read after calling `receive_byte()`

20.58.5.12 `_io_out` `volatile` `uint8_t` `_io_out` `[extern]`

Serial Link: Write byte to send here before calling `send_byte()`

20.58.5.13 `_current_bank` `__REG` `_current_bank`

Tracks current active ROM bank

In most cases the `CURRENT_BANK` macro for this variable is recommended for use instead of the variable itself.

The active bank number is not tracked by `_current_bank` when `SWITCH_ROM_MBC5_8M` is used.

This variable is updated automatically when you call `SWITCH_ROM_MBC1` or `SWITCH_ROM_MBC5`, `SWITCH_ROM()`, or call a BANKED function.

See also

[SWITCH_ROM_MBC1\(\)](#), [SWITCH_ROM_MBC5\(\)](#), [SWITCH_ROM\(\)](#)

20.58.5.14 `b` `void` `b`**20.58.5.15** `_current_1bpp_colors` `uint16_t` `_current_1bpp_colors` `[extern]`**20.58.5.16** `_map_tile_offset` `uint8_t` `_map_tile_offset` `[extern]`**20.58.5.17** `_submap_tile_offset` `uint8_t` `_submap_tile_offset` `[extern]`**20.58.5.18** `shadow_OAM` `volatile` `struct` `OAM_item_t` `shadow_OAM[]` `[extern]`

Shadow OAM array in WRAM, that is DMA-transferred into the real OAM each VBlank

20.58.5.19 `_shadow_OAM_base` `__REG` `_shadow_OAM_base`

MSB of `shadow_OAM` address is used by OAM DMA copying routine

20.59 `gb.h`

[Go to the documentation of this file.](#)

```
1
4 #ifndef _GB_H
5 #define _GB_H
6
7 #include <types.h>
8 #include <stdint.h>
9 #include <gbdk/version.h>
10 #include <gb/hardware.h>
11
12 // Here NINTENDO means Game Boy & related clones
13 #define NINTENDO
14
15 #ifdef SEGA
16 #undef SEGA
17 #endif
```

```

18
19 #ifdef NINTENDO_NES
20 #undef NINTENDO_NES
21 #endif
22
23 #ifdef MSX
24 #undef MSX
25 #endif
26
27 #define SYSTEM_60HZ      0x00
28 #define SYSTEM_50HZ      0x01
29
30 #if defined(__TARGET_ap)
31 #define ANALOGUEPOCKET
32 #elif defined(__TARGET_gb)
33 #define GAMEBOY
34 #elif defined(__TARGET_duck)
35 #define MEGADUCK
36 #endif
37
38
52 #define J_UP      0x04U
53 #define J_DOWN    0x08U
54 #define J_LEFT    0x02U
55 #define J_RIGHT   0x01U
56 #define J_A       0x10U
57 #define J_B       0x20U
58 #define J_SELECT  0x40U
59 #define J_START   0x80U
60
65 #define M_DRAWING  0x01U
66 #define M_TEXT_OUT 0x02U
67 #define M_TEXT_INOUT 0x03U
73 #define M_NO_SCROLL 0x04U
77 #define M_NO_INTERP 0x08U
78
83 #define S_BANK      0x08U
88 #define S_PALETTE   0x10U
92 #define S_FLIPX     0x20U
96 #define S_FLIPY     0x40U
101 #define S_PRIORITY 0x80U
105 #define S_PAL(n)    n
106
107 /* Interrupt flags */
110 #define EMPTY_IFLAG 0x00U
116 #define VBL_IFLAG  0x01U
120 #define LCD_IFLAG  0x02U
124 #define TIM_IFLAG  0x04U
128 #define SIO_IFLAG  0x08U
132 #define JOY_IFLAG  0x10U
133
134
135 /* DMG Palettes */
136 #define DMG_BLACK      0x03
137 #define DMG_DARK_GRAY 0x02
138 #define DMG_LITE_GRAY 0x01
139 #define DMG_WHITE      0x00
159 #define DMG_PALETTE(C0, C1, C2, C3) ((uint8_t)((C3 & 0x03) << 6) | ((C2 & 0x03) << 4) | ((C1) &
    0x03) << 2) | ((C0) & 0x03))
160
161 /* Limits */
164 #define SCREENWIDTH  DEVICE_SCREEN_PX_WIDTH
167 #define SCREENHEIGHT DEVICE_SCREEN_PX_HEIGHT
170 #define MINWNDPOSX   DEVICE_WINDOW_PX_OFFSET_X
173 #define MINWNDPOSY   DEVICE_WINDOW_PX_OFFSET_Y
176 #define MAXWNDPOSX   (DEVICE_WINDOW_PX_OFFSET_X + DEVICE_SCREEN_PX_WIDTH - 1)
179 #define MAXWNDPOSY   (DEVICE_WINDOW_PX_OFFSET_Y + DEVICE_SCREEN_PX_HEIGHT - 1)
180
181
184 typedef void (*int_handler)(void) NONBANKED;
185
193 void remove_VBL(int_handler h);
194
198 void remove_LCD(int_handler h);
199
203 void remove_TIM(int_handler h);
204
216 void remove_SIO(int_handler h);
217
221 void remove_JOY(int_handler h);
222
246 void add_VBL(int_handler h);
247
286 void add_LCD(int_handler h);
287
306 void add_TIM(int_handler h);
307

```

```

323 void add_low_priority_TIM(int_handler h);
324
340 void add_SIO(int_handler h);
341
342
366 void add_JOY(int_handler h);
367
368
384 void nowait_int_handler(void);
385
386
399 void wait_int_handler(void);
400
403 inline uint8_t cancel_pending_interrupts(void) {
404     return IF_REG = 0;
405 }
406
413 void mode(uint8_t m);
414
419 uint8_t get_mode(void) PRESERVES_REGS(b, c, d, e, h, l);
420
426 inline uint8_t get_system(void) {
427     return SYSTEM_60HZ;
428 }
429
434 extern uint8_t _cpu;
435
438 #define DMG_TYPE 0x01
441 #define MGB_TYPE 0xFF
444 #define CGB_TYPE 0x11
445
450 extern uint8_t _is_GBA;
451
454 #define GBA_NOT_DETECTED 0x00
457 #define GBA_DETECTED 0x01
458
461 #define DEVICE_SUPPORTS_COLOR (_cpu == CGB_TYPE)
462
469 extern volatile uint16_t sys_time;
470
476 __REG _vbl_done;
477 #define VBL_DONE _vbl_done
478
479
487 void send_byte(void);
488
496 void receive_byte(void);
497
499 extern volatile uint8_t _io_status;
500
503 extern volatile uint8_t _io_in;
504
507 extern volatile uint8_t _io_out;
508
509 /* Status codes */
511 #define IO_IDLE 0x00U
513 #define IO_SENDING 0x01U
515 #define IO_RECEIVING 0x02U
517 #define IO_ERROR 0x04U
518
519
520
534 __REG _current_bank;
535 #define CURRENT_BANK _current_bank
536
546 #ifndef BANK
547 #define BANK(VARNAME) ( (uint8_t) & __bank_ ## VARNAME )
548 #endif
549
562 #define BANKREF(VARNAME) void __func_ ## VARNAME(void) __banked __naked { \
563     __asm \
564         .local b__func_ ## VARNAME \
565         __bank_ ## VARNAME = b__func_ ## VARNAME \
566         .globl __bank_ ## VARNAME \
567     __endasm; \
568 }
569
579 #define BANKREF_EXTERN(VARNAME) extern const void __bank_ ## VARNAME;
580
596 #define SWITCH_ROM(b) (_current_bank = (b), rROMB0 = (b))
597
598 #if defined(__TARGET_duck)
599
600 #define SWITCH_RAM(b) (0)
601
602 #define ENABLE_RAM
603

```



```

604 #define DISABLE_RAM
605
606 #else
607
615 #define SWITCH_RAM(b) (rRAMB = (b))
616
619 #define ENABLE_RAM (rRAMG = 0x0A)
620
623 #define DISABLE_RAM (rRAMG = 0x00)
624
625 #endif
626
630 #define SWITCH_ROM_MEGADUCK(b) SWITCH_ROM(b)
631
640 #define SWITCH_ROM_MBC1(b) SWITCH_ROM(b)
641
649 #define SWITCH_RAM_MBC1(b) SWITCH_RAM(b)
650
653 #define ENABLE_RAM_MBC1 ENABLE_RAM
654
657 #define DISABLE_RAM_MBC1 DISABLE_RAM
658
659 #define SWITCH_16_8_MODE_MBC1 (*(volatile uint8_t *)0x6000 = 0x00)
660
661 #define SWITCH_4_32_MODE_MBC1 (*(volatile uint8_t *)0x6000 = 0x01)
662
675 #define SWITCH_ROM_MBC5(b) (_current_bank = (b), rROMB1 = 0, rROMB0 = (b))
676
690 #define SWITCH_ROM_MBC5_8M(b) (rROMB1 = ((uint16_t)(b) >> 8), rROMB0 = (b))
691
697 #define SWITCH_RAM_MBC5(b) SWITCH_RAM(b)
698
701 #define ENABLE_RAM_MBC5 ENABLE_RAM
702
705 #define DISABLE_RAM_MBC5 DISABLE_RAM
706
707
708
713 void delay(uint16_t d) PRESERVES_REGS(h, l);
714
715
716
727 uint8_t joypad(void) PRESERVES_REGS(b, c, h, l);
728
741 uint8_t waitpad(uint8_t mask) PRESERVES_REGS(b, c, h, l);
742
748 void waitpadup(void) PRESERVES_REGS(a, b, c, d, e, h, l);
749
755 typedef struct {
756     uint8_t npads;
757     union {
758         struct {
759             uint8_t joy0, joy1, joy2, joy3;
760         };
761         uint8_t joypads[4];
762     };
763 } joypads_t;
764
774 uint8_t joypad_init(uint8_t npads, joypads_t * joypads) OLDDCALL;
775
782 void joypad_ex(joypads_t * joypads) PRESERVES_REGS(b, c);
783
784
785
794 inline void enable_interrupts(void) PRESERVES_REGS(a, b, c, d, e, h, l) {
795     __asm__("ei");
796 }
797
810 inline void disable_interrupts(void) PRESERVES_REGS(a, b, c, d, e, h, l) {
811     __asm__("di");
812 }
813
824 void set_interrupts(uint8_t flags) PRESERVES_REGS(b, c, d, e, h, l);
825
837 void reset(void);
838
850 void vsync(void) PRESERVES_REGS(b, c, d, e, h, l);
851
855 void wait_vbl_done(void) PRESERVES_REGS(b, c, d, e, h, l);
856
862 void display_off(void) PRESERVES_REGS(b, c, d, e, h, l);
863
866 void refresh_OAM(void) PRESERVES_REGS(b, c, d, e, h, l);
867
868
874 void hiramcpy(uint8_t dst, const void *src, uint8_t n) OLDDCALL PRESERVES_REGS(b, c);
875

```

```

876
880 #define DISPLAY_ON \
881     LCDC_REG|=LCDCF_ON
882
888 #define DISPLAY_OFF \
889     display_off();
890
893 #define HIDE_LEFT_COLUMN
894
897 #define SHOW_LEFT_COLUMN
898
901 #define SET_BORDER_COLOR(C)
902
912 #define SHOW_BKG \
913     LCDC_REG|=LCDCF_BGON
914
924 #define HIDE_BKG \
925     LCDC_REG&=~LCDCF_BGON
926
936 #define SHOW_WIN \
937     LCDC_REG|=LCDCF_WINON
938
942 #define HIDE_WIN \
943     LCDC_REG&=~LCDCF_WINON
944
948 #define SHOW_SPRITES \
949     LCDC_REG|=LCDCF_OBJON
950
956 #define HIDE_SPRITES \
957     LCDC_REG&=~LCDCF_OBJON
958
962 #define SPRITES_8x16 \
963     LCDC_REG|=LCDCF_OBJ16
964
968 #define SPRITES_8x8 \
969     LCDC_REG&=~LCDCF_OBJ16
970
971
972
979 void set_vram_byte(uint8_t * addr, uint8_t v) PRESERVES_REGS(b, c);
980
992 uint8_t get_vram_byte(uint8_t * addr) PRESERVES_REGS(b, c, h, l);
993
994
998 uint8_t * get_bkg_xy_addr(uint8_t x, uint8_t y) PRESERVES_REGS(h, l);
999
1000 #define COMPAT_PALETTE(C0,C1,C2,C3) ((uint8_t)(((C3) << 6) | ((C2) << 4) | ((C1) << 2) | (C0)))
1001
1004 inline void set_2bpp_palette(uint16_t palette) {
1005     palette;
1006 }
1007
1008 extern uint16_t _current_lbpp_colors;
1009
1017 void set_lbpp_colors_ex(uint8_t fgcolor, uint8_t bgcolor, uint8_t mode) OLDCALL;
1018
1038 inline void set_lbpp_colors(uint8_t fgcolor, uint8_t bgcolor) {
1039     set_lbpp_colors_ex(fgcolor, bgcolor, 0);
1040 }
1041
1059 void set_bkg_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) OLDCALL PRESERVES_REGS(b,
c);
1060 #define set_bkg_2bpp_data set_bkg_data
1061
1080 void set_bkg_lbpp_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) OLDCALL
PRESERVES_REGS(b, c);
1081
1102 void get_bkg_data(uint8_t first_tile, uint8_t nb_tiles, uint8_t *data) OLDCALL PRESERVES_REGS(b, c);
1103
1104
1157 void set_bkg_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) OLDCALL
PRESERVES_REGS(b, c);
1158 #define set_tile_map set_bkg_tiles
1159
1160
1161 extern uint8_t _map_tile_offset;
1162
1181 inline void set_bkg_based_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles,
uint8_t base_tile) {
1182     _map_tile_offset = base_tile;
1183     set_bkg_tiles(x, y, w, h, tiles);
1184     _map_tile_offset = 0;
1185 }
1186
1187
1236 inline void set_bkg_attributes(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles)
1237 {

```

```

1238     VBK_REG = VBK_ATTRIBUTES;
1239     set_bkg_tiles(x, y, w, h, tiles);
1240     VBK_REG = VBK_TILES;
1241 }
1242
1243
1244 void set_bkg_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)
1245     OLDCALL;
1246 #define set_tile_submap set_bkg_submap
1247
1248 extern uint8_t _submap_tile_offset;
1249
1250 inline void set_bkg_based_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map,
1251     uint8_t map_w, uint8_t base_tile) {
1252     _submap_tile_offset = base_tile;
1253     set_bkg_submap(x, y, w, h, map, map_w);
1254     _submap_tile_offset = 0;
1255 }
1256
1257 inline void set_bkg_submap_attributes(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map,
1258     uint8_t map_w)
1259 {
1260     VBK_REG = VBK_ATTRIBUTES;
1261     set_bkg_submap(x, y, w, h, map, map_w);
1262     VBK_REG = VBK_TILES;
1263 }
1264
1265 void get_bkg_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *tiles) OLDCALL
1266     PRESERVES_REGS(b, c);
1267
1268 uint8_t * set_bkg_tile_xy(uint8_t x, uint8_t y, uint8_t t);
1269 #define set_tile_xy set_bkg_tile_xy
1270
1271 inline uint8_t * set_bkg_attribute_xy(uint8_t x, uint8_t y, uint8_t a)
1272 {
1273     uint8_t* addr;
1274     VBK_REG = VBK_ATTRIBUTES;
1275     addr = set_bkg_tile_xy(x, y, a);
1276     VBK_REG = VBK_TILES;
1277     return addr;
1278 }
1279 #define set_attribute_xy set_bkg_attribute_xy
1280
1281 uint8_t get_bkg_tile_xy(uint8_t x, uint8_t y) OLDCALL PRESERVES_REGS(b, c);
1282
1283 inline void move_bkg(uint8_t x, uint8_t y) {
1284     SCX_REG=x, SCY_REG=y;
1285 }
1286
1287 inline void scroll_bkg(int8_t x, int8_t y) {
1288     SCX_REG+=x, SCY_REG+=y;
1289 }
1290
1291 uint8_t * get_win_xy_addr(uint8_t x, uint8_t y) PRESERVES_REGS(h, l);
1292
1293 void set_win_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) OLDCALL PRESERVES_REGS(b,
1294     c);
1295
1296 void set_win_lbpp_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) OLDCALL
1297     PRESERVES_REGS(b, c);
1298
1299 void get_win_data(uint8_t first_tile, uint8_t nb_tiles, uint8_t *data) OLDCALL PRESERVES_REGS(b, c);
1300
1301 void set_win_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) OLDCALL
1302     PRESERVES_REGS(b, c);
1303
1304 inline void set_win_based_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles,
1305     uint8_t base_tile) {
1306     _map_tile_offset = base_tile;
1307     set_win_tiles(x, y, w, h, tiles);
1308     _map_tile_offset = 0;
1309 }
1310
1311 void set_win_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)
1312     OLDCALL;

```

```

1648
1649
1669 inline void set_win_based_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map,
    uint8_t map_w, uint8_t base_tile) {
1670     _submap_tile_offset = base_tile;
1671     set_win_submap(x, y, w, h, map, map_w);
1672     _submap_tile_offset = 0;
1673 }
1674
1675
1699 void get_win_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *tiles) OLDCALL
    PRESERVES_REGS(b, c);
1700
1701
1709 uint8_t * set_win_tile_xy(uint8_t x, uint8_t y, uint8_t t);
1710
1711
1724 uint8_t get_win_tile_xy(uint8_t x, uint8_t y) OLDCALL PRESERVES_REGS(b, c);
1725
1726
1738 inline void move_win(uint8_t x, uint8_t y) {
1739     WX_REG=x, WY_REG=y;
1740 }
1741
1742
1752 inline void scroll_win(int8_t x, int8_t y) {
1753     WX_REG+=x, WY_REG+=y;
1754 }
1755
1756
1757
1773 void set_sprite_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) OLDCALL
    PRESERVES_REGS(b, c);
1774 #define set_sprite_2bpp_data set_sprite_data
1775
1794 void set_sprite_1bpp_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) OLDCALL
    PRESERVES_REGS(b, c);
1795
1814 void get_sprite_data(uint8_t first_tile, uint8_t nb_tiles, uint8_t *data) OLDCALL PRESERVES_REGS(b, c);
1815
1816
1823 typedef struct OAM_item_t {
1824     uint8_t y, x; ///< X, Y Coordinates of the sprite on screen
1825     uint8_t tile; ///< Sprite tile number
1826     uint8_t prop; ///< OAM Property Flags
1827 } OAM_item_t;
1828
1829
1832 extern volatile struct OAM_item_t shadow_OAM[];
1833
1836 __REG __shadow_OAM_base;
1837
1838 #define DISABLE_OAM_DMA \
1839     __shadow_OAM_base = 0
1840
1843 #define DISABLE_VBL_TRANSFER DISABLE_OAM_DMA
1844
1845 #define ENABLE_OAM_DMA \
1846     __shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM » 8)
1847
1850 #define ENABLE_VBL_TRANSFER ENABLE_OAM_DMA
1851
1854 #define MAX_HARDWARE_SPRITES 40
1855
1858 #define HARDWARE_SPRITE_CAN_FLIP_X 1
1859
1862 #define HARDWARE_SPRITE_CAN_FLIP_Y 1
1863
1866 inline void SET_SHADOW_OAM_ADDRESS(void * address) {
1867     __shadow_OAM_base = (uint8_t)((uint16_t)address » 8);
1868 }
1869
1886 inline void set_sprite_tile(uint8_t nb, uint8_t tile) {
1887     shadow_OAM[nb].tile=tile;
1888 }
1889
1890
1897 inline uint8_t get_sprite_tile(uint8_t nb) {
1898     return shadow_OAM[nb].tile;
1899 }
1900
1901
1945 inline void set_sprite_prop(uint8_t nb, uint8_t prop) {
1946     shadow_OAM[nb].prop=prop;
1947 }
1948
1949

```

```

1955 inline uint8_t get_sprite_prop(uint8_t nb) {
1956     return shadow_OAM[nb].prop;
1957 }
1958
1959
1972 inline void move_sprite(uint8_t nb, uint8_t x, uint8_t y) {
1973     OAM_item_t * itm = &shadow_OAM[nb];
1974     itm->y=y, itm->x=x;
1975 }
1976
1977
1988 inline void scroll_sprite(uint8_t nb, int8_t x, int8_t y) {
1989     OAM_item_t * itm = &shadow_OAM[nb];
1990     itm->y+=y, itm->x+=x;
1991 }
1992
1993
2000 inline void hide_sprite(uint8_t nb) {
2001     shadow_OAM[nb].y = 0;
2002 }
2003
2004
2005
2021 void set_data(uint8_t *vram_addr, const uint8_t *data, uint16_t len);
2022
2023
2045 void get_data(uint8_t *data, uint8_t *vram_addr, uint16_t len);
2046
2059 void vmemcpy(uint8_t *dest, uint8_t *sour, uint16_t len);
2060
2061
2062
2086 void set_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *vram_addr, const uint8_t *tiles)
    OLDCALL;
2087
2098 void set_tile_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data, uint8_t base) OLDCALL
    PRESERVES_REGS(b, c);
2099
2127 void get_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *vram_addr, uint8_t *tiles) OLDCALL;
2128
2129
2144 inline void set_native_tile_data(uint16_t first_tile, uint8_t nb_tiles, const uint8_t *data) {
2145     if (first_tile < 256) {
2146         set_bkg_data(first_tile, nb_tiles, data);
2147     } else {
2148         set_sprite_data(first_tile - 256, nb_tiles, data);
2149     }
2150 }
2151
2167 inline void set_bkg_native_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) {
2168     set_bkg_data(first_tile, nb_tiles, data);
2169 }
2170
2184 inline void set_sprite_native_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) {
2185     set_sprite_data(first_tile, nb_tiles, data);
2186 }
2187
2193 void init_win(uint8_t c) OLDCALL PRESERVES_REGS(b, c);
2194
2200 void init_bkg(uint8_t c) OLDCALL PRESERVES_REGS(b, c);
2201
2209 void vmemset (void *s, uint8_t c, size_t n) OLDCALL PRESERVES_REGS(b, c);
2210
2211
2212
2221 void fill_bkg_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t tile) OLDCALL PRESERVES_REGS(b,
    c);
2222 #define fill_rect fill_bkg_rect
2223
2232 void fill_win_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t tile) OLDCALL PRESERVES_REGS(b,
    c);
2233
2234 #endif /* _GB_H */

```

20.60 gbdk-lib/include/gb/gbdecompress.h File Reference

```

#include <types.h>
#include <stdint.h>

```

Functions

- [uint16_t gb_decompress](#) (const [uint8_t](#) *sour, [uint8_t](#) *dest)
- void [gb_decompress_bkg_data](#) ([uint8_t](#) first_tile, const [uint8_t](#) *sour)
- void [gb_decompress_win_data](#) ([uint8_t](#) first_tile, const [uint8_t](#) *sour)
- void [gb_decompress_sprite_data](#) ([uint8_t](#) first_tile, const [uint8_t](#) *sour)

20.60.1 Detailed Description

GB-Compress decompressor Compatible with the compression used in GBTD

See also

[utility_gbcompress](#) "gbcompress"

GB-Compress decompressor Compatible with the compression used in GBTD

20.60.2 Function Documentation

20.60.2.1 [gb_decompress\(\)](#) [uint16_t](#) [gb_decompress](#) (

```

    const uint8\_t * sour,
    uint8\_t * dest )

```

gb-decompress data from sour into dest

Parameters

<i>sour</i>	Pointer to source gb-compressed data
<i>dest</i>	Pointer to destination buffer/address

Will decompress **all** of it's data to destination without stopping until the end of compressed data is reached. It is not possible to set a limit, so ensure the destination buffer has sufficient space to avoid an overflow.

See also

[gb_decompress_bkg_data](#), [gb_decompress_win_data](#), [gb_decompress_sprite_data](#), [rle_decompress](#)

gb-decompress data from sour into dest

Parameters

<i>sour</i>	Pointer to source gb-compressed data
<i>dest</i>	Pointer to destination buffer/address

Returns

Return value is number of bytes decompressed

See also

[gb_decompress_bkg_data](#), [gb_decompress_win_data](#), [gb_decompress_sprite_data](#)

20.60.2.2 [gb_decompress_bkg_data\(\)](#) void [gb_decompress_bkg_data](#) (

```

    uint8\_t first_tile,
    const uint8\_t * sour )

```

gb-decompress background tiles into VRAM

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>sour</i>	Pointer to (gb-compressed 2 bpp) source Tile Pattern data.

Note: This function avoids writes during modes 2 & 3

Will decompress **all** of it's data to destination without stopping until the end of compressed data is reached. It is not possible to set a limit, so ensure the destination buffer has sufficient space to avoid an overflow.

See also

[gb_decompress_bkg_data](#), [gb_decompress_win_data](#), [gb_decompress_sprite_data](#)

20.60.2.3 gb_decompress_win_data() void gb_decompress_win_data (
 uint8_t first_tile,
 const uint8_t * sour)

gb-decompress window tiles into VRAM

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>sour</i>	Pointer to (gb-compressed 2 bpp) source Tile Pattern data.

This is the same as [gb_decompress_bkg_data](#), since the Window Layer and Background Layer share the same Tile pattern data.

Note: This function avoids writes during modes 2 & 3

Will decompress **all** of it's data to destination without stopping until the end of compressed data is reached. It is not possible to set a limit, so ensure the destination buffer has sufficient space to avoid an overflow.

See also

[gb_decompress](#), [gb_decompress_bkg_data](#), [gb_decompress_sprite_data](#)

20.60.2.4 gb_decompress_sprite_data() void gb_decompress_sprite_data (
 uint8_t first_tile,
 const uint8_t * sour)

gb-decompress sprite tiles into VRAM

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>sour</i>	Pointer to source compressed data

Note: This function avoids writes during modes 2 & 3

Will decompress **all** of it's data to destination without stopping until the end of compressed data is reached. It is not possible to set a limit, so ensure the destination buffer has sufficient space to avoid an overflow.

See also

[gb_decompress](#), [gb_decompress_bkg_data](#), [gb_decompress_win_data](#)

20.61 gbdecompress.h

[Go to the documentation of this file.](#)

```
1
8 #ifndef __GBDECOMPRESS_H_INCLUDE
```

```

9 #define __GBDECOMPRESS_H_INCLUDE
10
11 #include <types.h>
12 #include <stdint.h>
13
26 uint16_t gb_decompress(const uint8_t * sour, uint8_t * dest);
27
28
43 void gb_decompress_bkg_data(uint8_t first_tile, const uint8_t * sour);
44
45
63 void gb_decompress_win_data(uint8_t first_tile, const uint8_t * sour);
64
65
80 void gb_decompress_sprite_data(uint8_t first_tile, const uint8_t * sour);
81
82 #endif

```

20.62 gbdk-lib/include/gbdk/gbdecompress.h File Reference

```
#include <gb/gbdecompress.h>
```

20.63 gbdecompress.h

[Go to the documentation of this file.](#)

```

1 #ifndef __GB_DECOMPRESS_H_INCLUDE
2 #define __GB_DECOMPRESS_H_INCLUDE
3
4 #if defined(__TARGET_gb) || defined(__TARGET_ap) || defined(__TARGET_duck)
5 #include <gb/gbdecompress.h>
6 #elif defined(__TARGET_sms) || defined(__TARGET_gg) || defined(__TARGET_msx)
7 #include <sms/gbdecompress.h>
8 #else
9 #error Unrecognized port
10 #endif
11
12 #endif

```

20.64 gbdk-lib/include/sms/gbdecompress.h File Reference

```
#include <types.h>
#include <stdint.h>
```

Functions

- [uint16_t gb_decompress](#) (const [uint8_t](#) *sour, [uint8_t](#) *dest) [Z88DK_CALLEE PRESERVES_REGS\(b](#)

Variables

- [uint16_t c](#)

20.64.1 Function Documentation

20.64.1.1 gb_decompress() [uint16_t](#) gb_decompress (

```

    const uint8\_t * sour,
    uint8\_t * dest )

```

gb-decompress data from sour into dest

Parameters

<i>sour</i>	Pointer to source gb-compressed data
<i>dest</i>	Pointer to destination buffer/address

Returns

Return value is number of bytes decompressed

See also

[gb_decompress_bkg_data](#), [gb_decompress_win_data](#), [gb_decompress_sprite_data](#)

20.64.2 Variable Documentation

20.64.2.1 `c uint16_t c`

20.65 gbdecompress.h

[Go to the documentation of this file.](#)

```

1
7 #ifndef __GBDECOMPRESS_H_INCLUDE
8 #define __GBDECOMPRESS_H_INCLUDE
9
10 #include <types.h>
11 #include <stdint.h>
12
22 uint16_t gb_decompress(const uint8_t * sour, uint8_t * dest) Z88DK_CALLEE PRESERVES_REGS(b, c);
23
24 #endif

```

20.66 gbdk-lib/include/gb/hblankcpy.h File Reference

```
#include <stdint.h>
```

Functions

- void [hblank_copy_vram](#) (const [uint8_t](#) *sour, [uint8_t](#) count)
- void [hblank_cpy_vram](#) (const [uint8_t](#) *sour, [uint8_t](#) count)
- void [hblank_copy](#) ([uint8_t](#) *dest, const [uint8_t](#) *sour, [uint16_t](#) size)

Variables

- [uint8_t](#) * [hblank_copy_destination](#)

20.66.1 Function Documentation

20.66.1.1 [hblank_copy_vram\(\)](#) void [hblank_copy_vram](#) (
const [uint8_t](#) * *sour*,
[uint8_t](#) *count*)

HBlank stack copy routine

Parameters

<i>sour</i>	Source address to copy from
<i>count</i>	Number of 16 byte chunks to copy

Performs the required STAT_REG, IE_REG, IF_REG manipulation when called and restores STAT_REG and IE_REG on exit (unlike [hblank_cpy_vram\(\)](#)).

Before calling:

- Set the destination using [hblank_copy_destination](#)

- Interrupts must be disabled

See also

[hblank_cpy_vram](#), [hblank_copy_destination](#), [hblank_copy](#)

20.66.1.2 hblank_cpy_vram() `void hblank_cpy_vram (`
 `const uint8_t * sour,`
 `uint8_t count )`

HBlank stack copy routine

Parameters

<i>sour</i>	Source address to copy from
<i>count</i>	Number of 16 byte chunks to copy

Unlike [hblank_copy_vram\(\)](#) does not perform the required STAT_REG, IE_REG, IF_REG manipulation, nor does it restore STAT_REG and IE_REG on exit.

Before calling:

- Set the destination using [hblank_copy_destination](#)
- Interrupts must be properly configured
- Interrupts must be disabled

See also

[hblank_copy_vram](#), [hblank_copy_destination](#), [hblank_copy](#)

20.66.1.3 hblank_copy() `void hblank_copy (`
 `uint8_t * dest,`
 `const uint8_t * sour,`
 `uint16_t size ) [inline]`

HBlank stack copy routine (must be called with interrupts disabled!)

Parameters

<i>dest</i>	destination pointer
<i>sour</i>	source pointer
<i>size</i>	number of bytes to copy (rounded to 16-byte chunks)

Performs a fast vram safe copy of data during HBlank.

20.66.2 Variable Documentation

20.66.2.1 hblank_copy_destination `uint8_t* hblank_copy_destination [extern]`
Destination address for hblank copy routine

20.67 hblankcpy.h

[Go to the documentation of this file.](#)

```

1 #ifndef __HBLANKCPY_H_INCLUDE__
2 #define __HBLANKCPY_H_INCLUDE__
3
4 #include <stdint.h>
5
20 void hblank_copy_vram(const uint8_t * sour, uint8_t count);
21
37 void hblank_cpy_vram(const uint8_t * sour, uint8_t count);
38
39 extern uint8_t * hblank_copy_destination;
48 inline void hblank_copy(uint8_t * dest, const uint8_t * sour, uint16_t size) {
49     hblank_copy_destination = dest;
50     hblank_copy_vram(sour, size >> 4);
51 }
52
53 #endif

```

20.68 gbdk-lib/include/gb/isr.h File Reference

```

#include <stdint.h>
#include <types.h>

```

Data Structures

- struct [isr_vector_t](#)
- struct [isr_nested_vector_t](#)

Macros

- #define [VECTOR_STAT](#) 0x48
- #define [VECTOR_TIMER](#) 0x50
- #define [VECTOR_SERIAL](#) 0x58
- #define [VECTOR_JOYPAD](#) 0x60
- #define [ISR_VECTOR](#)(ADDR, FUNC) static const [isr_vector_t](#) [AT](#)((ADDR)) __ISR_ ## ADDR = {0xc3, (void *)&(FUNC)};
- #define [ISR_NESTED_VECTOR](#)(ADDR, FUNC) static const [isr_nested_vector_t](#) [AT](#)((ADDR)) __ISR_ ## ADDR = {{0xfb, 0xc3}, (void *)&(FUNC)};

Typedefs

- typedef struct [isr_vector_t](#) [isr_vector_t](#)
- typedef struct [isr_nested_vector_t](#) [isr_nested_vector_t](#)

20.68.1 Detailed Description

Macros for creating raw interrupt service routines (ISRs) which do not use the default GBDK ISR dispatcher. Handlers installed this way will have less overhead than ones which use the GBDK ISR dispatcher.

20.68.2 Macro Definition Documentation

20.68.2.1 VECTOR_STAT #define VECTOR_STAT 0x48
Address for the STAT interrupt vector

20.68.2.2 VECTOR_TIMER #define VECTOR_TIMER 0x50
Address for the TIMER interrupt vector

20.68.2.3 VECTOR_SERIAL #define VECTOR_SERIAL 0x58
Address for the SERIAL interrupt vector

20.68.2.4 VECTOR_JOYPAD `#define VECTOR_JOYPAD 0x60`

Address for the JOYPAD interrupt vector

20.68.2.5 ISR_VECTOR `#define ISR_VECTOR(`

`ADDR,`

`FUNC ) static const isr_vector_t AT((ADDR)) __ISR_ ## ADDR = {0xc3, (void *)&(FUNC)};`

Creates an interrupt vector at the given address for a raw interrupt service routine (which does not use the GBDK ISR dispatcher)

Parameters

<i>ADDR</i>	Address of the interrupt vector, any of: VECTOR_STAT , VECTOR_TIMER , VECTOR_SERIAL , VECTOR_JOYPAD
<i>FUNC</i>	ISR function supplied by the user

This cannot be used with the VBLANK interrupt.

Do not use this in combination with interrupt installers that rely on the default GBDK ISR dispatcher such as [add_TIM\(\)](#), [remove_TIM\(\)](#) (and the same for all other interrupts).

Example:

```
#include <gb/isr.h>
void TimerISR() __critical __interrupt {
    // some ISR code here
}
ISR_VECTOR(VECTOR_TIMER, TimerISR)
```

See also

[ISR_NESTED_VECTOR](#), [set_interrupts](#)

20.68.2.6 ISR_NESTED_VECTOR `#define ISR_NESTED_VECTOR(`

`ADDR,`

`FUNC ) static const isr_nested_vector_t AT((ADDR)) __ISR_ ## ADDR = {{0xfb, 0xc3}, (void *)&(FUNC)};`

Creates an interrupt vector at the given address for a raw interrupt service routine allowing nested interrupts

Parameters

<i>ADDR</i>	Address of the interrupt vector, any of: VECTOR_STAT , VECTOR_TIMER , VECTOR_SERIAL , VECTOR_JOYPAD
<i>FUNC</i>	ISR function

This cannot be used with the VBLANK interrupt

The LCD STAT vector ([VECTOR_STAT](#)) cannot be used in the same program as `stdio.h` since they install an ISR vector to the same location.

See also

[ISR_VECTOR](#)

20.68.3 Typedef Documentation

20.68.3.1 `isr_vector_t` `typedef struct isr_vector_t isr_vector_t`

20.68.3.2 `isr_nested_vector_t` `typedef struct isr_nested_vector_t isr_nested_vector_t`

20.69 isr.h

[Go to the documentation of this file.](#)

```

1
9 #ifndef __ISR_H_INCLUDE__
10 #define __ISR_H_INCLUDE__
11
12 #include <stdint.h>
13 #include <types.h>
14
15 // #define VECTOR_VBL      0x40 // you can not define raw vector for VBlank interrupt
16 #define VECTOR_STAT      0x48
17 #define VECTOR_TIMER     0x50
18 #define VECTOR_SERIAL    0x58
19 #define VECTOR_JOYPAD    0x60
21 typedef struct isr_vector_t {
22     uint8_t opcode;
23     void * func;
24 } isr_vector_t;
25
52 #define ISR_VECTOR(ADDR, FUNC) \
53 static const isr_vector_t AT((ADDR)) __ISR_ ## ADDR = {0xc3, (void *)&(FUNC)};
54
55 typedef struct isr_nested_vector_t {
56     uint8_t opcode[2];
57     void * func;
58 } isr_nested_vector_t;
59
74 #define ISR_NESTED_VECTOR(ADDR, FUNC) \
75 static const isr_nested_vector_t AT((ADDR)) __ISR_ ## ADDR = {{0xfb, 0xc3}, (void *)&(FUNC)};
76
77
78 #endif // __ISR_H_INCLUDE__

```

20.70 gbdk-lib/include/gb/sgb.h File Reference

```

#include <types.h>
#include <stdint.h>

```

Macros

- #define [SGB_PAL_01](#) 0x00U
- #define [SGB_PAL_23](#) 0x01U
- #define [SGB_PAL_03](#) 0x02U
- #define [SGB_PAL_12](#) 0x03U
- #define [SGB_ATTR_BLK](#) 0x04U
- #define [SGB_ATTR_LIN](#) 0x05U
- #define [SGB_ATTR_DIV](#) 0x06U
- #define [SGB_ATTR_CHR](#) 0x07U
- #define [SGB_SOUND](#) 0x08U
- #define [SGB_SOU_TRN](#) 0x09U
- #define [SGB_PAL_SET](#) 0x0AU
- #define [SGB_PAL_TRN](#) 0x0BU
- #define [SGB_ATTRC_EN](#) 0x0CU
- #define [SGB_TEST_EN](#) 0x0DU
- #define [SGB_ICON_EN](#) 0x0EU
- #define [SGB_DATA_SND](#) 0x0FU
- #define [SGB_DATA_TRN](#) 0x10U
- #define [SGB_MLT_REQ](#) 0x11U
- #define [SGB_JUMP](#) 0x12U
- #define [SGB_CHR_TRN](#) 0x13U
- #define [SGB_PCT_TRN](#) 0x14U
- #define [SGB_ATTR_TRN](#) 0x15U
- #define [SGB_ATTR_SET](#) 0x16U
- #define [SGB_MASK_EN](#) 0x17U
- #define [SGB_OBJ_TRN](#) 0x18U

Functions

- `uint8_t sgb_check` (void) [OLDCALL PRESERVES_REGS\(b](#)
- void `sgb_transfer` (`uint8_t *packet`) [OLDCALL PRESERVES_REGS\(b](#)

Variables

- `uint8_t c`

20.70.1 Detailed Description

Super Gameboy definitions.

See the example SGB project for additional details.

20.70.2 Macro Definition Documentation

20.70.2.1 **SGB_PAL_01** `#define SGB_PAL_01 0x00U`

SGB Command: Set SGB Palettes 0 & 1

20.70.2.2 **SGB_PAL_23** `#define SGB_PAL_23 0x01U`

SGB Command: Set SGB Palettes 2 & 3

20.70.2.3 **SGB_PAL_03** `#define SGB_PAL_03 0x02U`

SGB Command: Set SGB Palettes 0 & 3

20.70.2.4 **SGB_PAL_12** `#define SGB_PAL_12 0x03U`

SGB Command: Set SGB Palettes 1 & 2

20.70.2.5 **SGB_ATTR_BLK** `#define SGB_ATTR_BLK 0x04U`

SGB Command: Set color attributes for rectangular regions

20.70.2.6 **SGB_ATTR_LIN** `#define SGB_ATTR_LIN 0x05U`

SGB Command: Set color attributes for horizontal or vertical character lines

20.70.2.7 **SGB_ATTR_DIV** `#define SGB_ATTR_DIV 0x06U`

SGB Command: Split screen in half and assign separate color attribes to each side and the divider

20.70.2.8 **SGB_ATTR_CHR** `#define SGB_ATTR_CHR 0x07U`

SGB Command: Set color attributes for separate charactersSet SGB Palette 0,1 Data

20.70.2.9 **SGB_SOUND** `#define SGB_SOUND 0x08U`

SGB Command: Start and stop a internal sound effect, and sounds using internal tone data

20.70.2.10 **SGB_SOU_TRN** `#define SGB_SOU_TRN 0x09U`

SGB Command: Transfer sound code or data to the SNES APU RAM

20.70.2.11 **SGB_PAL_SET** `#define SGB_PAL_SET 0x0AU`

SGB Command: Apply (previously transferred) SGB system color palettes to actual SNES palettes

20.70.2.12 **SGB_PAL_TRN** `#define SGB_PAL_TRN 0x0BU`

SGB Command: Transfer palette data into SGB system color palettes

20.70.2.13 **SGB_ATTRC_EN** `#define SGB_ATTRC_EN 0x0CU`

SGB Command: Enable/disable Attraction mode. It is enabled by default

20.70.2.14 SGB_TEST_EN `#define SGB_TEST_EN 0x0DU`

SGB Command: Enable/disable test mode for "SGB-CPU variable clock speed function"

20.70.2.15 SGB_ICON_EN `#define SGB_ICON_EN 0x0EU`

SGB Command: Enable/disable ICON functionality

20.70.2.16 SGB_DATA_SND `#define SGB_DATA_SND 0x0FU`

SGB Command: Write one or more bytes into SNES Work RAM

20.70.2.17 SGB_DATA_TRN `#define SGB_DATA_TRN 0x10U`

SGB Command: Transfer code or data into SNES RAM

20.70.2.18 SGB_MLT_REQ `#define SGB_MLT_REQ 0x11U`

SGB Command: Request multiplayer mode (input from more than one joystick)

20.70.2.19 SGB_JUMP `#define SGB_JUMP 0x12U`

SGB Command: Set the SNES program counter and NMI (vblank interrupt) handler to specific addresses

20.70.2.20 SGB_CHR_TRN `#define SGB_CHR_TRN 0x13U`

SGB Command: Transfer tile data (characters) to SNES Tile memory

20.70.2.21 SGB_PCT_TRN `#define SGB_PCT_TRN 0x14U`

SGB Command: Transfer tile map and palette data to SNES BG Map memory

20.70.2.22 SGB_ATTR_TRN `#define SGB_ATTR_TRN 0x15U`

SGB Command: Transfer data to (color) Attribute Files (ATFs) in SNES RAM

20.70.2.23 SGB_ATTR_SET `#define SGB_ATTR_SET 0x16U`

SGB Command: Transfer attributes from (color) Attribute Files (ATF) to the Game Boy window

20.70.2.24 SGB_MASK_EN `#define SGB_MASK_EN 0x17U`

SGB Command: Modify Game Boy window mask settings

20.70.2.25 SGB_OBJ_TRN `#define SGB_OBJ_TRN 0x18U`

SGB Command: Transfer OBJ attributes to SNES OAM memory

20.70.3 Function Documentation

20.70.3.1 sgb_check() `uint8_t sgb_check (`
`void )`

Returns a non-zero value if running on a Super GameBoy

Since `sgb_check()` uses `sgb_transfer()`, the same delay at startup requirement applies to ensure correct operation on PAL SNES. See `sgb_transfer()` for details.

20.70.3.2 sgb_transfer() `void sgb_transfer (`
`uint8_t * packet )`

Transfer a SGB packet

Parameters

<code>packet</code>	Pointer to buffer with SGB packet data.
---------------------	---

The first byte of **packet** should be a SGB command, then up to 15 bytes of command parameter data.

See the `sgb_border` GBDK example project for a demo of how to use these the `sgb` functions.

When using the SGB with a PAL SNES, a delay should be added just after program startup such as:

```
// Wait 4 frames
// For PAL SNES this delay is required on startup
for (uint8_t i = 4; i != 0; i--) wait_vbl_done();
```

See also

[sgb_check\(\)](#)

20.70.4 Variable Documentation

20.70.4.1 c void c

20.71 sgb.h

[Go to the documentation of this file.](#)

```
1
2 // #ifndef _SGB_H
3 // #define _SGB_H
4
5 #include <types.h>
6 #include <stdint.h>
7
8 #define SGB_PAL_01 0x00U
9 #define SGB_PAL_23 0x01U
10 #define SGB_PAL_03 0x02U
11 #define SGB_PAL_12 0x03U
12 #define SGB_ATTR_BLK 0x04U
13 #define SGB_ATTR_LIN 0x05U
14 #define SGB_ATTR_DIV 0x06U
15 #define SGB_ATTR_CHR 0x07U
16 #define SGB_SOUND 0x08U
17 #define SGB_SOU_TRN 0x09U
18 #define SGB_PAL_SET 0x0AU
19 #define SGB_PAL_TRN 0x0BU
20 #define SGB_ATTRC_EN 0x0CU
21 #define SGB_TEST_EN 0x0DU
22 #define SGB_ICON_EN 0x0EU
23 #define SGB_DATA_SND 0x0FU
24 #define SGB_DATA_TRN 0x10U
25 #define SGB_MLT_REQ 0x11U
26 #define SGB_JUMP 0x12U
27 #define SGB_CHR_TRN 0x13U
28 #define SGB_PCT_TRN 0x14U
29 #define SGB_ATTR_TRN 0x15U
30 #define SGB_ATTR_SET 0x16U
31 #define SGB_MASK_EN 0x17U
32 #define SGB_OBJ_TRN 0x18U
33
34 uint8_t sgb_check(void) OLDCALL PRESERVES_REGS(b, c);
35
36 void sgb_transfer(uint8_t * packet) OLDCALL PRESERVES_REGS(b, c);
37
38 #endif /* _SGB_H */
```

20.72 gbdk-lib/include/gbdk/console.h File Reference

```
#include <types.h>
#include <stdint.h>
```

Functions

- void [gotoxy](#) (uint8_t x, uint8_t y) OLDCALL
- uint8_t [posx](#) (void) OLDCALL
- uint8_t [posy](#) (void) OLDCALL
- void [setchar](#) (char c) OLDCALL
- void [cls](#) (void)

20.72.1 Detailed Description

Console functions that work like Turbo C's.
The font is 8x8, making the screen 20x18 characters.

20.72.2 Function Documentation

20.72.2.1 gotoxy() `void gotoxy (`
 `uint8_t x,`
 `uint8_t y )`

Move the cursor to an absolute position at **x**, **y**.
x and **y** have units of tiles (8 pixels per unit)

See also

[setchar\(\)](#)

20.72.2.2 posx() `uint8_t posx (`
 `void )`

Returns the current X position of the cursor.

See also

[gotoxy\(\)](#)

20.72.2.3 posy() `uint8_t posy (`
 `void )`

Returns the current Y position of the cursor.

See also

[gotoxy\(\)](#)

20.72.2.4 setchar() `void setchar (`
 `char c )`

Writes out a single character at the current cursor position.
Does not update the cursor or interpret the character.

See also

[gotoxy\(\)](#)

20.72.2.5 cls() `void cls (`
 `void )`

Clears the screen

20.73 console.h

[Go to the documentation of this file.](#)

```
1
6 #ifndef _CONSOLE_H
7 #define _CONSOLE_H
8
9 #include <types.h>
```

```

10 #include <stdint.h>
11
17 void gotoxy(uint8_t x, uint8_t y) OLDCALL;
18
23 uint8_t posx(void) OLDCALL;
24
29 uint8_t posy(void) OLDCALL;
30
38 void setchar(char c) OLDCALL;
39
42 void cls(void);
43
44
45 #endif /* _CONSOLE_H */

```

20.74 gbdk-lib/include/gbdk/far_ptr.h File Reference

```

#include <types.h>
#include <stdint.h>

```

Data Structures

- union [__far_ptr](#)

Macros

- #define [TO_FAR_PTR](#)(ofs, seg) ((([FAR_PTR](#))seg << 16) | ([FAR_PTR](#))ofs)
- #define [FAR_SEG](#)(ptr) (((union [__far_ptr](#) *)&ptr)->segofs.seg)
- #define [FAR_OFS](#)(ptr) (((union [__far_ptr](#) *)&ptr)->segofs.ofs)
- #define [FAR_FUNC](#)(ptr, typ) ((typ)(((union [__far_ptr](#) *)&ptr)->segfn.fn))
- #define [FAR_CALL](#)(ptr, typ, ...) ([__call_banked_ptr](#)=ptr,((typ)([__call_banked](#)))(__VA_ARGS__))

Typedefs

- typedef [uint32_t](#) [FAR_PTR](#)

Functions

- void [__call_banked](#) (void)
- [uint32_t to_far_ptr](#) (void *ofs, [uint16_t](#) seg)

Variables

- volatile [FAR_PTR](#) [__call_banked_ptr](#)
- volatile void * [__call_banked_addr](#)
- volatile [uint8_t](#) [__call_banked_bank](#)

20.74.1 Detailed Description

Far pointers include a segment (bank) selector so they are able to point to addresses (functions or data) outside of the current bank (unlike normal pointers which are not bank-aware).

See the `banks_farptr` example project included with gbdk.

Todo Add link to a discussion about banking (such as, how to assign code and variables to banks)

20.74.2 Macro Definition Documentation

20.74.2.1 TO_FAR_PTR #define TO_FAR_PTR(
 ofs,
 seg) ((([FAR_PTR](#))seg << 16) | ([FAR_PTR](#))ofs)

Macro to obtain a far pointer at compile-time

Parameters

<i>ofs</i>	Memory address within the given Segment (Bank)
<i>seg</i>	Segment (Bank) number

Returns

A far pointer (type [FAR_PTR](#))

20.74.2.2 FAR_SEG `#define FAR_SEG(
ptr) (((union __far_ptr *)&ptr)->segofs.seg)`

Macro to get the Segment (Bank) number of a far pointer

Parameters

<i>ptr</i>	A far pointer (type FAR_PTR)
------------	---

Returns

Segment (Bank) of the far pointer (type `uint16_t`)

20.74.2.3 FAR_OFS `#define FAR_OFS(
ptr) (((union __far_ptr *)&ptr)->segofs.ofs)`

Macro to get the Offset (address) of a far pointer

Parameters

<i>ptr</i>	A far pointer (type FAR_PTR)
------------	---

Returns

Offset (address) of the far pointer (type `void *`)

20.74.2.4 FAR_FUNC `#define FAR_FUNC(
ptr,
typ) ((typ) (((union __far_ptr *)&ptr)->segfn.fn))`

20.74.2.5 FAR_CALL `#define FAR_CALL(
ptr,
typ,
...) (__call_banked_ptr=ptr, ((typ) (&__call_banked)) (__VA_ARGS__))`

Macro to call a function at far pointer **ptr** of type **typ**

Parameters

<i>ptr</i>	Far pointer of a function to call (type FAR_PTR)
<i>typ</i>	Type to cast the function far pointer to.
...	VA Args list of parameters for the function

type should match the definition of the function being called. For example:

```
// A function in bank 2
#pragma bank 2
uint16_t some_function(uint16_t param1, uint16_t param2) __banked { return 1; };
...
// Code elsewhere, such as unbanked main()
// This type declaration should match the above function
typedef uint16_t (*some_function_t)(uint16_t, uint16_t) __banked;
// Using FAR_CALL() with the above as *ptr*, *typ*, and two parameters.
result = FAR_CALL(some_function, some_function_t, 100, 50);
```

Returns

Value returned by the function (if present)

20.74.3 Typedef Documentation

20.74.3.1 FAR_PTR typedef uint32_t FAR_PTR

Type for storing a FAR_PTR

20.74.4 Function Documentation

20.74.4.1 __call_banked() void __call_banked (void)

20.74.4.2 to_far_ptr() uint32_t to_far_ptr (void * ofs, uint16_t seg)

Obtain a far pointer at runtime

Parameters

<i>ofs</i>	Memory address within the given Segment (Bank)
<i>seg</i>	Segment (Bank) number

Returns

A far pointer (type [FAR_PTR](#))

20.74.5 Variable Documentation

20.74.5.1 __call_banked_ptr volatile FAR_PTR __call_banked_ptr [extern]

20.74.5.2 __call_banked_addr volatile void* __call_banked_addr [extern]

20.74.5.3 __call_banked_bank volatile uint8_t __call_banked_bank [extern]

20.75 far_ptr.h

[Go to the documentation of this file.](#)

```
1
13 #ifndef __FAR_PTR_H_INCLUDE
14 #define __FAR_PTR_H_INCLUDE
```

```

15
16 #include <types.h>
17 #include <stdint.h>
18
25 #define TO_FAR_PTR(ofs, seg) (((FAR_PTR)seg << 16) | (FAR_PTR)ofs)
26
32 #define FAR_SEG(ptr) (((union __far_ptr *)&ptr)->segofs.seg)
33
39 #define FAR_OFS(ptr) (((union __far_ptr *)&ptr)->segofs.ofs)
40
41 #define FAR_FUNC(ptr, typ) ((typ)(((union __far_ptr *)&ptr)->segfn.fn))
42
65 #define FAR_CALL(ptr, typ, ...) (__call_banked_ptr=ptr, ((typ)(&__call_banked))(__VA_ARGS__))
66
69 typedef uint32_t FAR_PTR;
70
73 union __far_ptr {
74     FAR_PTR ptr;
75     struct {
76         void * ofs;
77         uint16_t seg;
78     } segofs;
79     struct {
80         void (*fn)(void);
81         uint16_t seg;
82     } segfn;
83 };
84
85 extern volatile FAR_PTR __call_banked_ptr;
86 extern volatile void * __call_banked_addr;
87 extern volatile uint8_t __call_banked_bank;
88
89 void __call_banked(void);
90
97 uint32_t to_far_ptr(void* ofs, uint16_t seg);
98
99 #endif

```

20.76 gbdk-lib/include/gbdk/font.h File Reference

```

#include <types.h>
#include <stdint.h>

```

Data Structures

- struct [sfont_handle](#)

Macros

- #define [FONT_256ENCODING](#) 0
- #define [FONT_128ENCODING](#) 1
- #define [FONT_NOENCODING](#) 2
- #define [FONT_COMPRESSED](#) 4

Typedefs

- typedef [uint16_t](#) [font_t](#)
- typedef struct [sfont_handle](#) [mfont_handle](#)
- typedef struct [sfont_handle](#) * [pmfont_handle](#)

Functions

- void [font_init](#) (void)
- [font_t](#) [font_load](#) (void *font) [OLDCALL](#)
- [font_t](#) [font_set](#) ([font_t](#) font_handle) [OLDCALL](#)
- void [font_color](#) ([uint8_t](#) forecolor, [uint8_t](#) backcolor) [OLDCALL](#)

Variables

- `uint8_t font_spect []`
- `uint8_t font_italic []`
- `uint8_t font_ibm []`
- `uint8_t font_min []`
- `uint8_t font_ibm_fixed []`

20.76.1 Detailed Description

Multiple font support for the GameBoy Michael Hope, 1999 michaelh@earthling.net

20.76.2 Macro Definition Documentation

20.76.2.1 FONT_256ENCODING `#define FONT_256ENCODING 0`

Various flags in the font header.

20.76.2.2 FONT_128ENCODING `#define FONT_128ENCODING 1`

20.76.2.3 FONT_NOENCODING `#define FONT_NOENCODING 2`

20.76.2.4 FONT_COMPRESSED `#define FONT_COMPRESSED 4`

20.76.3 Typedef Documentation

20.76.3.1 font_t `typedef uint16_t font_t`

`font_t` is a handle to a font loaded by `font_load()`. It can be used with `font_set()`

20.76.3.2 mfont_handle `typedef struct sfont_handle mfont_handle`

Internal representation of a font. What a `font_t` really is

20.76.3.3 pmfont_handle `typedef struct sfont_handle* pmfont_handle`

20.76.4 Function Documentation

20.76.4.1 font_init() `void font_init (`
`void )`

Initializes the font system. Should be called before other font functions.

20.76.4.2 font_load() `font_t font_load (`
`void * font )`

Load a font and set it as the current font.

Parameters

<code>font</code>	Pointer to a font to load (usually a gbdk font)
-------------------	---

Returns

Handle to the loaded font, which can be used with [font_set\(\)](#)

See also

[font_init\(\)](#), [font_set\(\)](#), [List of gbdk fonts](#)

20.76.4.3 font_set() `font_t font_set (`
`font_t font_handle )`

Set the current font.

Parameters

<code>font_handle</code>	handle of a font returned by font_load()
--------------------------	--

Returns

The previously used font handle.

See also

[font_init\(\)](#), [font_load\(\)](#)

20.76.4.4 font_color() `void font_color (`
`uint8_t forecolor,`
`uint8_t backcolor )`

Set the current **foreground** colour (for pixels), **background** colour

20.77 font.h

[Go to the documentation of this file.](#)

```

1
2
3
4
5
6 #ifndef __FONT_H
7 #define __FONT_H
8
9 #include <types.h>
10 #include <stdint.h>
11
12
13
14 #define FONT_256ENCODING    0
15 #define FONT_128ENCODING    1
16 #define FONT_NOENCODING     2
17
18 #define FONT_COMPRESSED     4
19
20 /* See gb.h/M_NO_SCROLL and gb.h/M_NO_INTERP */
21
22
23
24 typedef uint16_t font_t;
25
26
27
28
29
30
31
32 extern uint8_t font_spect[], font_italic[], font_ibm[], font_min[];
33
34
35 extern uint8_t font_ibm_fixed[];
36
37
38
39
40
41
42
43 void font_init(void);
44
45
46
47
48
49
50
51 font_t font_load(void *font) OLDCALL;
52
53
54
55
56
57
58
59 font_t font_set(font_t font_handle) OLDCALL;
60
61
62
63
64
65
66
67
68
69
70 struct sfont_handle {
71     uint8_t first_tile;

```

```
72     void *font;
73 };
74
76 void font_color(uint8_t forecolor, uint8_t backcolor) OLDCALL;
77
78 #endif /* __FONT_H */
```

20.78 gbdk-lib/include/gbdk/gbdk-lib.h File Reference

#include <asm/sm83/provides.h>

20.78.1 Detailed Description

Settings for the greater library system.

20.79 gbdk-lib.h

[Go to the documentation of this file.](#)

```
1
4 #ifndef GBDK_LIB_INCLUDE
5 #define GBDK_LIB_INCLUDE
6
7 #if defined(__PORT_sm83)
8     #include <asm/sm83/provides.h>
9 #elif defined(__PORT_z80)
10    #include <asm/z80/provides.h>
11 #elif defined(__PORT_mos6502)
12    #include <asm/mos6502/provides.h>
13 #else
14    #error Unrecognized port
15 #endif
16
17
18 #ifndef USE_C_MEMCPY
19 #define USE_C_MEMCPY      1
20 #endif
21
22 #ifndef USE_C_STRCPY
23 #define USE_C_STRCPY      1
24 #endif
25
26 #ifndef USE_C_STRCMP
27 #define USE_C_STRCMP      1
28 #endif
29 #endif
```

20.80 gbdk-lib/include/gbdk/incbin.h File Reference

#include <stdint.h>

Macros

- #define **INCBIN_EXTERN**(VARNAME)
- #define **INCBIN_SIZE**(VARNAME) ((uint16_t) & __size_ ## VARNAME)
- #define **BANK**(VARNAME) ((uint8_t) & __bank_ ## VARNAME)
- #define **INCBIN**(VARNAME, FILEPATH)

20.80.1 Detailed Description

Allows binary data from other files to be included into a C source file.

It is implemented using asm .incbin and macros.

See the `incbin` example project for a demo of how to use it.

20.80.2 Macro Definition Documentation

20.80.2.1 INCBIN_EXTERN `#define INCBIN_EXTERN(
VARNAME)`

Value:

```
extern const uint8_t VARNAME[]; \
extern const void __size_ ## VARNAME; \
extern const void __bank_ ## VARNAME;
```

Creates extern entries for accessing a [INCBIN\(\)](#) generated variable and it's size in another source file.

Parameters

<i>VARNAME</i>	Name of the variable used with INCBIN
----------------	---------------------------------------

An entry is created for the variable and it's size variable.

[INCBIN\(\)](#), [INCBIN_SIZE\(\)](#)

20.80.2.2 INCBIN_SIZE `#define INCBIN_SIZE(
VARNAME) ((uint16_t) & __size_ ## VARNAME)`

Obtains the **size in bytes** of the [INCBIN\(\)](#) generated data

Parameters

<i>VARNAME</i>	Name of the variable used with INCBIN
----------------	---------------------------------------

Requires [INCBIN_EXTERN\(\)](#) to have been called earlier in the source file

[INCBIN\(\)](#), [INCBIN_EXTERN\(\)](#)

20.80.2.3 BANK `#define BANK(
VARNAME) ((uint8_t) & __bank_ ## VARNAME)`

Obtains the **bank number** of the [INCBIN\(\)](#) generated data

Parameters

<i>VARNAME</i>	Name of the variable used with INCBIN
----------------	---------------------------------------

Requires [INCBIN_EXTERN\(\)](#) to have been called earlier in the source file

[INCBIN\(\)](#), [INCBIN_EXTERN\(\)](#)

20.80.2.4 INCBIN `#define INCBIN(
VARNAME,
FILEPATH)`

Value:

```
void __func_ ## VARNAME(void) __banked __naked { \
__asm \
_ ## VARNAME:: \
1$: \
.incbn FILEPATH \
2$: \
__size_ ## VARNAME = (2$-1$) \
.globl __size_ ## VARNAME \
.local b__func_ ## VARNAME \
__bank_ ## VARNAME = b__func_ ## VARNAME \
.globl __bank_ ## VARNAME \
__endasm; \
}
```

Includes binary data into a C source file

Parameters

<i>VARNAME</i>	Variable name to use
<i>FILEPATH</i>	Path to the file which will be binary included into the C source file

filepath is relative to the working directory of the tool that is calling it (often a makefile's working directory), **NOT** to

the file it's being included into.

The variable name is not modified and can be used as-is.

The `INCBIN()` macro will declare the `BANK()` and `INCBIN_SIZE()` helper symbols. Then if `INCBIN_EXTERN()` is used in the header then those helper macros can be used in the application code.

- `INCBIN_SIZE()` for obtaining the size of the included data.
- `BANK()` for obtaining the bank number of the included data.

Use `INCBIN_EXTERN()` within another source file to make the variable and it's data accessible there.

20.81 incbin.h

[Go to the documentation of this file.](#)

```

1
10 #ifndef __INCBIN_H
11 #define __INCBIN_H
12
13 #include <stdint.h>
14
15
25 #define INCBIN_EXTERN(VARNAME) extern const uint8_t VARNAME[]; \
26 extern const void __size_ ## VARNAME; \
27 extern const void __bank_ ## VARNAME;
28
37 #define INCBIN_SIZE(VARNAME) ( (uint16_t) & __size_ ## VARNAME )
38
47 #ifndef BANK
48 #define BANK(VARNAME) ( (uint8_t) & __bank_ ## VARNAME )
49 #endif
50
74 #define INCBIN(VARNAME, FILEPATH) void __func_ ## VARNAME(void) __banked __naked { \
75 __asm \
76 _ ## VARNAME:: \
77 1$: \
78 .incbin FILEPATH \
79 2$: \
80 __size_ ## VARNAME = (2$-1$) \
81 .globl __size_ ## VARNAME \
82 .local b__func_ ## VARNAME \
83 __bank_ ## VARNAME = b__func_ ## VARNAME \
84 .globl __bank_ ## VARNAME \
85 __endasm; \
86 }
87
88 #endif // __INCBIN_H

```

20.82 gbdk-lib/include/gbdk/platform.h File Reference

```

#include <gb/gb.h>
#include <gb/cgb.h>
#include <gb/sgb.h>

```

20.83 platform.h

[Go to the documentation of this file.](#)

```

1 #ifndef __PLATFORM_H_INCLUDE
2 #define __PLATFORM_H_INCLUDE
3
4 #if defined(__TARGET_gb) || defined(__TARGET_ap) || defined(__TARGET_duck)
5 #include <gb/gb.h>
6 #include <gb/cgb.h>
7 #include <gb/sgb.h>
8 #elif defined(__TARGET_sms) || defined(__TARGET_gg)
9 #include <sms/sms.h>
10 #elif defined(__TARGET_msxdos)
11 #include <msx/msx.h>
12 #elif defined(__TARGET_nes)
13 #include <nes/nes.h>
14 #else
15 #error Unrecognized port
16 #endif
17
18 #endif

```

20.84 gbdk-lib/include/gbdk/rledecompress.h File Reference

```
#include <types.h>
#include <stdint.h>
```

Macros

- `#define RLE_STOP 0`

Functions

- `uint8_t rle_init` (void *data)
- `uint8_t rle_decompress` (void *dest, uint8_t len)

20.84.1 Detailed Description

Decompressor for RLE encoded data

Decompresses data which has been compressed with `gbccompress` using the `--alg=rle` argument.

20.84.2 Macro Definition Documentation

20.84.2.1 RLE_STOP `#define RLE_STOP 0`

20.84.3 Function Documentation

20.84.3.1 rle_init() `uint8_t rle_init (`
 `void * data )`

Initialize the RLE decompressor with RLE data at address **data**

Parameters

<i>data</i>	Pointer to start of RLE compressed data
-------------	---

See also

[rle_decompress](#)

20.84.3.2 rle_decompress() `uint8_t rle_decompress (`
 `void * dest,`
 `uint8_t len )`

Decompress RLE compressed data into **dest** for length **len** bytes

Parameters

<i>dest</i>	Pointer to destination buffer/address
<i>len</i>	Number of bytes to decompress

Returns

Returns 0 if compression is complete, 1 if there is more data to decompress

Before calling this function `rle_init` must be called one time to initialize the RLE decompressor.
Decompresses data which has been compressed with `gbccompress` using the `--alg=rle` argument.

See also

[rle_init](#)

20.85 rledecompress.h

[Go to the documentation of this file.](#)

```
1
9 #ifndef __RLEDECOMPRESS_H_INCLUDE
10 #define __RLEDECOMPRESS_H_INCLUDE
11
12 #include <types.h>
13 #include <stdint.h>
14
15 #define RLE_STOP 0
16
17 #if defined(__TARGET_gb) || defined(__TARGET_ap) || defined(__TARGET_duck) || defined(__TARGET_nes)
24 uint8_t rle_init(void * data);
25
40 uint8_t rle_decompress(void * dest, uint8_t len);
41 #elif defined(__TARGET_sms) || defined(__TARGET_gg) || defined(__TARGET_msx)
42 uint8_t rle_init(void * data) Z88DK_FASTCALL;
43 uint8_t rle_decompress(void * dest, uint8_t len) Z88DK_CALLEE;
44 #else
45 #error Unrecognized port
46 #endif
47
48 #endif
```

20.86 gbdk-lib/include/gbdk/version.h File Reference

Macros

- `#define __GBDK_VERSION 450`

20.86.1 Macro Definition Documentation

20.86.1.1 `__GBDK_VERSION` `#define __GBDK_VERSION 450`

20.87 version.h

[Go to the documentation of this file.](#)

```
1 #ifndef __VERSION_H_INCLUDE__
2 #define __VERSION_H_INCLUDE__
3
4 #define __GBDK_VERSION 450
5
6 #endif
```

20.88 gbdk-lib/include/gbdk/zx0decompress.h File Reference

Functions

- void `zx0_decompress` (void *sour, void *dest)

20.88.1 Function Documentation

20.88.1.1 zx0_decompress() `void zx0_decompress (`
 `void * sour,`
 `void * dest )`

Decompress zx0 compressed data from sour into dest

Parameters

<i>sour</i>	Pointer to source zx0 compressed data
<i>dest</i>	Pointer to destination buffer/address

Will decompress **all** of it's data to destination without stopping until the end of compressed data is reached. It is not possible to set a limit, so ensure the destination buffer has sufficient space to avoid an overflow.

Decompresses data which has been compressed with [gbcompress](#) using the `--alg=zx0` argument.

20.89 zx0decompress.h

[Go to the documentation of this file.](#)

```
1 #ifndef __ZX0DECOMPRESS_H_INCLUDE
2 #define __ZX0DECOMPRESS_H_INCLUDE
3
4 #if defined(__TARGET_gb) || defined(__TARGET_ap) || defined(__TARGET_duck) || defined(__TARGET_sms) ||
   defined(__TARGET_gg) || defined(__TARGET_msx)
5
18 void zx0_decompress(void * sour, void * dest);
19
20 #endif
21
22 #endif
```

20.90 gbdk-lib/include/limits.h File Reference

Macros

- #define [CHAR_BIT](#) 8 /* bits in a char */
- #define [SCHAR_MAX](#) 127
- #define [SCHAR_MIN](#) -128
- #define [UCHAR_MAX](#) 0xff
- #define [CHAR_MAX](#) [SCHAR_MAX](#)
- #define [CHAR_MIN](#) [SCHAR_MIN](#)
- #define [INT_MIN](#) (-32767 - 1)
- #define [INT_MAX](#) 32767
- #define [SHRT_MAX](#) [INT_MAX](#)
- #define [SHRT_MIN](#) [INT_MIN](#)
- #define [UINT_MAX](#) 0xffff
- #define [UINT_MIN](#) 0
- #define [USHRT_MAX](#) [UINT_MAX](#)
- #define [USHRT_MIN](#) [UINT_MIN](#)
- #define [LONG_MIN](#) (-2147483647L-1)
- #define [LONG_MAX](#) 2147483647L
- #define [ULONG_MAX](#) 0xffffffff
- #define [ULONG_MIN](#) 0

20.90.1 Macro Definition Documentation

20.90.1.1 CHAR_BIT `#define CHAR_BIT 8 /* bits in a char */`

20.90.1.2 SCHAR_MAX `#define SCHAR_MAX 127`

20.90.1.3 SCHAR_MIN `#define SCHAR_MIN -128`

20.90.1.4 UCHAR_MAX `#define UCHAR_MAX 0xff`

20.90.1.5 CHAR_MAX `#define CHAR_MAX SCHAR\_MAX`

20.90.1.6 CHAR_MIN `#define CHAR_MIN SCHAR\_MIN`

20.90.1.7 INT_MIN `#define INT_MIN (-32767 - 1)`

20.90.1.8 INT_MAX `#define INT_MAX 32767`

20.90.1.9 SHRT_MAX `#define SHRT_MAX INT\_MAX`

20.90.1.10 SHRT_MIN `#define SHRT_MIN INT\_MIN`

20.90.1.11 UINT_MAX `#define UINT_MAX 0xffff`

20.90.1.12 UINT_MIN `#define UINT_MIN 0`

20.90.1.13 USHRT_MAX `#define USHRT_MAX UINT\_MAX`

20.90.1.14 USHRT_MIN `#define USHRT_MIN UINT\_MIN`

20.90.1.15 LONG_MIN `#define LONG_MIN (-2147483647L-1)`

20.90.1.16 LONG_MAX `#define LONG_MAX 2147483647L`

20.90.1.17 ULONG_MAX `#define ULONG_MAX 0xffffffff`

20.90.1.18 ULONG_MIN `#define ULONG_MIN 0`

20.91 limits.h

[Go to the documentation of this file.](#)

```
1 /*-----  
2 limits.h - ANSI defines constants for sizes of integral types  
3  
4 Copyright (C) 1999, Sandeep Dutta . sandeep.dutta@usa.net  
5  
6 This library is free software; you can redistribute it and/or modify it  
7 under the terms of the GNU General Public License as published by the
```

```

8   Free Software Foundation; either version 2, or (at your option) any
9   later version.
10
11   This library is distributed in the hope that it will be useful,
12   but WITHOUT ANY WARRANTY; without even the implied warranty of
13   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14   GNU General Public License for more details.
15
16   You should have received a copy of the GNU General Public License
17   along with this library; see the file COPYING. If not, write to the
18   Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,
19   MA 02110-1301, USA.
20
21   As a special exception, if you link this library with other files,
22   some of which are compiled with SDCC, to produce an executable,
23   this library does not by itself cause the resulting executable to
24   be covered by the GNU General Public License. This exception does
25   not however invalidate any other reasons why the executable file
26   might be covered by the GNU General Public License.
27   -----*/
28
29 #ifndef __SDC51_LIMITS_H
30 #define __SDC51_LIMITS_H 1
31
32 #define CHAR_BIT      8      /* bits in a char */
33 #define SCHAR_MAX     127
34 #define SCHAR_MIN    -128
35 #define UCHAR_MAX    0xff
36
37 #ifdef __SDCC_CHAR_UNSIGNED
38 #define CHAR_MAX     UCHAR_MAX
39 #define CHAR_MIN     0
40 #else
41 #define CHAR_MAX     SCHAR_MAX
42 #define CHAR_MIN     SCHAR_MIN
43 #endif
44
45 #if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199409L
46 #define MB_LEN_MAX   4
47 #endif
48
49 #define INT_MIN      (-32767 - 1)
50 #define INT_MAX      32767
51 #define SHRT_MAX     INT_MAX
52 #define SHRT_MIN     INT_MIN
53 #define UINT_MAX     0xffff
54 #define UINT_MIN     0
55 #define USHRT_MAX    UINT_MAX
56 #define USHRT_MIN    UINT_MIN
57 #define LONG_MIN     (-2147483647L-1)
58 #define LONG_MAX     2147483647L
59 #define ULONG_MAX    0xffffffff
60 #define ULONG_MIN    0
61
62 #if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L
63 #define LLONG_MIN    (-9223372036854775807LL-1)
64 #define LLONG_MAX    9223372036854775807LL
65 #define ULLONG_MAX   18446744073709551615ULL
66 #endif
67
68 #endif
69

```

20.92 gbdk-lib/include/gb/hardware.h File Reference

```
#include <types.h>
```

Macros

- `#define __BYTES` extern `UBYTE`
- `#define __BYTE_REG` extern volatile `UBYTE`
- `#define __REG` extern volatile `SFR`
- `#define MBC7_LATCH_ERASE` `0x55u`
- `#define MBC7_LATCH_CAPTURE` `0xAAu`
- `#define MBC7_SRAM_ENABLE_KEY_1` `0x0Au`
- `#define MBC7_SRAM_ENABLE_KEY_2` `0x40u`
- `#define rP1` `P1_REG`

- #define P1F_5 0b00100000
- #define P1F_4 0b00010000
- #define P1F_3 0b00001000
- #define P1F_2 0b00000100
- #define P1F_1 0b00000010
- #define P1F_0 0b00000001
- #define P1F_GET_DPAD P1F_5
- #define P1F_GET_BTN P1F_4
- #define P1F_GET_NONE (P1F_4 | P1F_5)
- #define rSB SB_REG
- #define rSC SC_REG
- #define SIOF_XFER_START 0b10000000
- #define SIOF_CLOCK_INT 0b00000001
- #define SIOF_CLOCK_EXT 0b00000000
- #define SIOF_SPEED_1X 0b00000000
- #define SIOF_SPEED_32X 0b00000010
- #define SIOF_B_CLOCK 0
- #define SIOF_B_SPEED 1
- #define SIOF_B_XFER_START 7
- #define SCF_START SIOF_XFER_START
- #define SCF_SOURCE SIOF_CLOCK_INT
- #define SCF_SPEED SIOF_SPEED_32X
- #define rDIV DIV_REG
- #define rTIMA TIMA_REG
- #define rTMA TMA_REG
- #define rTAC TAC_REG
- #define TACF_START 0b00000100
- #define TACF_STOP 0b00000000
- #define TACF_4KHZ 0b00000000
- #define TACF_16KHZ 0b00000011
- #define TACF_65KHZ 0b00000010
- #define TACF_262KHZ 0b00000001
- #define rIF IF_REG
- #define rAUD1SWEEP NR10_REG
- #define AUD1SWEEP_UP 0b00000000
- #define AUD1SWEEP_DOWN 0b00001000
- #define AUD1SWEEP_TIME(x) ((x) << 4)
- #define AUD1SWEEP_LENGTH(x) (x)
- #define rAUD1LEN NR11_REG
- #define rAUD1ENV NR12_REG
- #define rAUD1LOW NR13_REG
- #define rAUD1HIGH NR14_REG
- #define rAUD2LEN NR21_REG
- #define rAUD2ENV NR22_REG
- #define rAUD2LOW NR23_REG
- #define rAUD2HIGH NR24_REG
- #define rAUD3ENA NR30_REG
- #define rAUD3LEN NR31_REG
- #define rAUD3LEVEL NR32_REG
- #define rAUD3LOW NR33_REG
- #define rAUD3HIGH NR34_REG
- #define rAUD4LEN NR41_REG
- #define rAUD4ENV NR42_REG
- #define rAUD4POLY NR43_REG
- #define AUD4POLY_WIDTH_15BIT 0x00

- #define AUD4POLY_WIDTH_7BIT 0x08
- #define rAUD4GO NR44_REG
- #define rAUDVOL NR50_REG
- #define AUDVOL_VOL_LEFT(x) ((x) < 4)
- #define AUDVOL_VOL_RIGHT(x) ((x))
- #define AUDVOL_VIN_LEFT 0b10000000
- #define AUDVOL_VIN_RIGHT 0b00001000
- #define rAUDTERM NR51_REG
- #define AUDTERM_4_LEFT 0b10000000
- #define AUDTERM_3_LEFT 0b01000000
- #define AUDTERM_2_LEFT 0b00100000
- #define AUDTERM_1_LEFT 0b00010000
- #define AUDTERM_4_RIGHT 0b00001000
- #define AUDTERM_3_RIGHT 0b00000100
- #define AUDTERM_2_RIGHT 0b00000010
- #define AUDTERM_1_RIGHT 0b00000001
- #define rAUDENA NR52_REG
- #define AUDENA_ON 0b10000000
- #define AUDENA_OFF 0b00000000
- #define rLCDC LCDC_REG
- #define LCDCF_OFF 0b00000000
- #define LCDCF_ON 0b10000000
- #define LCDCF_WIN9800 0b00000000
- #define LCDCF_WIN9C00 0b01000000
- #define LCDCF_WINOFF 0b00000000
- #define LCDCF_WINON 0b00100000
- #define LCDCF_BG8800 0b00000000
- #define LCDCF_BG8000 0b00010000
- #define LCDCF_BG9800 0b00000000
- #define LCDCF_BG9C00 0b00001000
- #define LCDCF_OBJ8 0b00000000
- #define LCDCF_OBJ16 0b00000100
- #define LCDCF_OBJOFF 0b00000000
- #define LCDCF_OBJON 0b00000010
- #define LCDCF_BG0FF 0b00000000
- #define LCDCF_BGON 0b00000001
- #define LCDCF_B_ON 7
- #define LCDCF_B_WIN9C00 6
- #define LCDCF_B_WINON 5
- #define LCDCF_B_BG8000 4
- #define LCDCF_B_BG9C00 3
- #define LCDCF_B_OBJ16 2
- #define LCDCF_B_OBJON 1
- #define LCDCF_B_BGON 0
- #define rSTAT STAT_REG
- #define STATF_LYC 0b01000000
- #define STATF_MODE10 0b00100000
- #define STATF_MODE01 0b00010000
- #define STATF_MODE00 0b00001000
- #define STATF_LYCF 0b00000100
- #define STATF_HBL 0b00000000
- #define STATF_VBL 0b00000001
- #define STATF_OAM 0b00000010
- #define STATF_LCD 0b00000011
- #define STATF_BUSY 0b00000010

- #define STATF_B_LYC 6
- #define STATF_B_MODE10 5
- #define STATF_B_MODE01 4
- #define STATF_B_MODE00 3
- #define STATF_B_LYCF 2
- #define STATF_B_VBL 0
- #define STATF_B_OAM 1
- #define STATF_B_BUSY 1
- #define rSCY
- #define rSCX SCX_REG
- #define rLY LY_REG
- #define rLYC LYC_REG
- #define rDMA DMA_REG
- #define rBGP BGP_REG
- #define rOBP0 OBP0_REG
- #define rOBP1 OBP1_REG
- #define rWY WY_REG
- #define rWX WX_REG
- #define rKEY1 KEY1_REG
- #define rSPD KEY1_REG
- #define KEY1F_DBLSPD 0b10000000
- #define KEY1F_PREPARE 0b00000001
- #define rVBK VBK_REG
- #define VBK_BANK_0 0
- #define VBK_TILES 0
- #define VBK_BANK_1 1
- #define VBK_ATTRIBUTES 1
- #define BKGF_PRI 0b10000000
- #define BKGF_YFLIP 0b01000000
- #define BKGF_XFLIP 0b00100000
- #define BKGF_BANK0 0b00000000
- #define BKGF_BANK1 0b00001000
- #define BKGF_CGB_PAL0 0b00000000
- #define BKGF_CGB_PAL1 0b00000001
- #define BKGF_CGB_PAL2 0b00000010
- #define BKGF_CGB_PAL3 0b00000011
- #define BKGF_CGB_PAL4 0b00000100
- #define BKGF_CGB_PAL5 0b00000101
- #define BKGF_CGB_PAL6 0b00000110
- #define BKGF_CGB_PAL7 0b00000111
- #define rHDMA1 HDMA1_REG
- #define rHDMA2 HDMA2_REG
- #define rHDMA3 HDMA3_REG
- #define rHDMA4 HDMA4_REG
- #define rHDMA5 HDMA5_REG
- #define HDMA5F_MODE_GP 0b00000000
- #define HDMA5F_MODE_HBL 0b10000000
- #define HDMA5F_BUSY 0b10000000
- #define rRP RP_REG
- #define RPF_ENREAD 0b11000000
- #define RPF_DATAIN 0b00000010
- #define RPF_WRITE_HI 0b00000001
- #define RPF_WRITE_LO 0b00000000
- #define rBCPS BCPS_REG
- #define BCPSF_AUTOINC 0b10000000

- #define `rBCPD` `BCPD_REG`
- #define `rOCPS` `OCPS_REG`
- #define `OCPSF_AUTOINC` `0b10000000`
- #define `rOCPD` `OCPD_REG`
- #define `rSVBK` `SVBK_REG`
- #define `rSMBK` `SVBK_REG`
- #define `rPCM12` `PCM12_REG`
- #define `rPCM34` `PCM34_REG`
- #define `rIE` `IE_REG`
- #define `IEF_HILO` `0b00010000`
- #define `IEF_SERIAL` `0b00001000`
- #define `IEF_TIMER` `0b00000100`
- #define `IEF_STAT` `0b00000010`
- #define `IEF_VBLANK` `0b00000001`
- #define `AUDLEN_DUTY_12_5` `0b00000000`
- #define `AUDLEN_DUTY_25` `0b01000000`
- #define `AUDLEN_DUTY_50` `0b10000000`
- #define `AUDLEN_DUTY_75` `0b11000000`
- #define `AUDLEN_LENGTH(x)` `(x)`
- #define `AUDENV_VOL(x)` `((x) << 4)`
- #define `AUDENV_UP` `0b00001000`
- #define `AUDENV_DOWN` `0b00000000`
- #define `AUDENV_LENGTH(x)` `(x)`
- #define `AUDHIGH_RESTART` `0b10000000`
- #define `AUDHIGH_LENGTH_ON` `0b01000000`
- #define `AUDHIGH_LENGTH_OFF` `0b00000000`
- #define `OAMF_PRI` `0b10000000`
- #define `OAMF_YFLIP` `0b01000000`
- #define `OAMF_XFLIP` `0b00100000`
- #define `OAMF_PAL0` `0b00000000`
- #define `OAMF_PAL1` `0b00010000`
- #define `OAMF_BANK0` `0b00000000`
- #define `OAMF_BANK1` `0b00001000`
- #define `OAMF_CGB_PAL0` `0b00000000`
- #define `OAMF_CGB_PAL1` `0b00000001`
- #define `OAMF_CGB_PAL2` `0b00000010`
- #define `OAMF_CGB_PAL3` `0b00000011`
- #define `OAMF_CGB_PAL4` `0b00000100`
- #define `OAMF_CGB_PAL5` `0b00000101`
- #define `OAMF_CGB_PAL6` `0b00000110`
- #define `OAMF_CGB_PAL7` `0b00000111`
- #define `OAMF_PALMASK` `0b00000111`
- #define `DEVICE_SCREEN_X_OFFSET` `0`
- #define `DEVICE_SCREEN_Y_OFFSET` `0`
- #define `DEVICE_SCREEN_WIDTH` `20`
- #define `DEVICE_SCREEN_HEIGHT` `18`
- #define `DEVICE_SCREEN_BUFFER_WIDTH` `32`
- #define `DEVICE_SCREEN_BUFFER_HEIGHT` `32`
- #define `DEVICE_SCREEN_MAP_ENTRY_SIZE` `1`
- #define `DEVICE_SPRITE_PX_OFFSET_X` `8`
- #define `DEVICE_SPRITE_PX_OFFSET_Y` `16`
- #define `DEVICE_WINDOW_PX_OFFSET_X` `7`
- #define `DEVICE_WINDOW_PX_OFFSET_Y` `0`
- #define `DEVICE_SCREEN_PX_WIDTH` `(DEVICE_SCREEN_WIDTH * 8)`
- #define `DEVICE_SCREEN_PX_HEIGHT` `(DEVICE_SCREEN_HEIGHT * 8)`

Variables

- [__BYTES_VRAM \[\]](#)
- [__BYTES_VRAM8000 \[\]](#)
- [__BYTES_VRAM8800 \[\]](#)
- [__BYTES_VRAM9000 \[\]](#)
- [__BYTES_SCRN0 \[\]](#)
- [__BYTES_SCRN1 \[\]](#)
- [__BYTES_SRAM \[\]](#)
- [__BYTES_RAM \[\]](#)
- [__BYTES_RAMBANK \[\]](#)
- [__BYTES_OAMRAM \[\]](#)
- [__BYTE_REG_IO \[\]](#)
- [__BYTE_REG_AUD3WAVERAM \[\]](#)
- [__BYTE_REG_HRAM \[\]](#)
- [__BYTE_REG rRAMG](#)
- [__BYTE_REG rROMB0](#)
- [__BYTE_REG rROMB1](#)
- [__BYTE_REG rRAMB](#)
- [__BYTE_REG rMBC7_SRAM_ENABLE_1](#)
- [__BYTE_REG rMBC7_SRAM_ENABLE_2](#)
- [__BYTE_REG rMBC7_LATCH_1](#)
- [__BYTE_REG rMBC7_LATCH_2](#)
- [__BYTE_REG rMBC7_ACCEL_X_LO](#)
- [__BYTE_REG rMBC7_ACCEL_X_HI](#)
- [__BYTE_REG rMBC7_ACCEL_Y_LO](#)
- [__BYTE_REG rMBC7_ACCEL_Y_HI](#)
- [__REG P1_REG](#)
- [__REG SB_REG](#)
- [__REG SC_REG](#)
- [__REG DIV_REG](#)
- [__REG TIMA_REG](#)
- [__REG TMA_REG](#)
- [__REG TAC_REG](#)
- [__REG IF_REG](#)
- [__REG NR10_REG](#)
- [__REG NR11_REG](#)
- [__REG NR12_REG](#)
- [__REG NR13_REG](#)
- [__REG NR14_REG](#)
- [__REG NR21_REG](#)
- [__REG NR22_REG](#)
- [__REG NR23_REG](#)
- [__REG NR24_REG](#)
- [__REG NR30_REG](#)
- [__REG NR31_REG](#)
- [__REG NR32_REG](#)
- [__REG NR33_REG](#)
- [__REG NR34_REG](#)
- [__REG NR41_REG](#)
- [__REG NR42_REG](#)
- [__REG NR43_REG](#)
- [__REG NR44_REG](#)
- [__REG NR50_REG](#)
- [__REG NR51_REG](#)

- [__REG NR52_REG](#)
- [__BYTE_REG AUD3WAVE](#) [16]
- [__BYTE_REG PCM_SAMPLE](#) [16]
- [__REG LCDC_REG](#)
- [__REG STAT_REG](#)
- [__REG SCY_REG](#)
- [__REG SCX_REG](#)
- [__REG LY_REG](#)
- [__REG LYC_REG](#)
- [__REG DMA_REG](#)
- [__REG BGP_REG](#)
- [__REG OBP0_REG](#)
- [__REG OBP1_REG](#)
- [__REG WY_REG](#)
- [__REG WX_REG](#)
- [__REG KEY1_REG](#)
- [__REG VBK_REG](#)
- [__REG HDMA1_REG](#)
- [__REG HDMA2_REG](#)
- [__REG HDMA3_REG](#)
- [__REG HDMA4_REG](#)
- [__REG HDMA5_REG](#)
- [__REG RP_REG](#)
- [__REG BCPS_REG](#)
- [__REG BCPD_REG](#)
- [__REG OCPS_REG](#)
- [__REG OCPD_REG](#)
- [__REG SVBK_REG](#)
- [__REG PCM12_REG](#)
- [__REG PCM34_REG](#)
- [__REG IE_REG](#)

20.92.1 Detailed Description

Defines that let the GB's hardware registers be accessed from C.
See the [Pandocs](#) for more details on each register.

20.92.2 Macro Definition Documentation

20.92.2.1 [__BYTES](#) `#define __BYTES extern UBYTE`

20.92.2.2 [__BYTE_REG](#) `#define __BYTE_REG extern volatile UBYTE`

20.92.2.3 [__REG](#) `#define __REG extern volatile SFR`

20.92.2.4 [MBC7_LATCH_ERASE](#) `#define MBC7_LATCH_ERASE 0x55u`

20.92.2.5 [MBC7_LATCH_CAPTURE](#) `#define MBC7_LATCH_CAPTURE 0xAAu`

20.92.2.6 MBC7_SRAM_ENABLE_KEY_1 `#define MBC7_SRAM_ENABLE_KEY_1 0x0Au`

20.92.2.7 MBC7_SRAM_ENABLE_KEY_2 `#define MBC7_SRAM_ENABLE_KEY_2 0x40u`

20.92.2.8 rP1 `#define rP1 P1_REG`

20.92.2.9 P1F_5 `#define P1F_5 0b00100000`

20.92.2.10 P1F_4 `#define P1F_4 0b00010000`

20.92.2.11 P1F_3 `#define P1F_3 0b00001000`

20.92.2.12 P1F_2 `#define P1F_2 0b00000100`

20.92.2.13 P1F_1 `#define P1F_1 0b00000010`

20.92.2.14 P1F_0 `#define P1F_0 0b00000001`

20.92.2.15 P1F_GET_DPAD `#define P1F_GET_DPAD P1F_5`

20.92.2.16 P1F_GET_BTN `#define P1F_GET_BTN P1F_4`

20.92.2.17 P1F_GET_NONE `#define P1F_GET_NONE (P1F_4 | P1F_5)`

20.92.2.18 rSB `#define rSB SB_REG`

20.92.2.19 rSC `#define rSC SC_REG`

20.92.2.20 SIOF_XFER_START `#define SIOF_XFER_START 0b10000000`
Serial IO: Start Transfer. Automatically cleared at the end of transfer

20.92.2.21 SIOF_CLOCK_INT `#define SIOF_CLOCK_INT 0b00000001`
Serial IO: Use Internal clock

20.92.2.22 SIOF_CLOCK_EXT `#define SIOF_CLOCK_EXT 0b00000000`
Serial IO: Use External clock

20.92.2.23 SIOF_SPEED_1X `#define SIOF_SPEED_1X 0b00000000`
Serial IO: If internal clock then 8KHz mode, 1KB/s (16KHz in CGB high-speed mode, 2KB/s)

20.92.2.24 SIOF_SPEED_32X `#define SIOF_SPEED_32X 0b00000010`

Serial IO: **CGB-Mode ONLY** If internal clock then 256KHz mode, 32KB/s (512KHz in CGB high-speed mode, 64KB/s)

20.92.2.25 SIOF_B_CLOCK `#define SIOF_B_CLOCK 0`

20.92.2.26 SIOF_B_SPEED `#define SIOF_B_SPEED 1`

20.92.2.27 SIOF_B_XFER_START `#define SIOF_B_XFER_START 7`

20.92.2.28 SCF_START `#define SCF_START SIOF_XFER_START`

20.92.2.29 SCF_SOURCE `#define SCF_SOURCE SIOF_CLOCK_INT`

20.92.2.30 SCF_SPEED `#define SCF_SPEED SIOF_SPEED_32X`

20.92.2.31 rDIV `#define rDIV DIV_REG`

20.92.2.32 rTIMA `#define rTIMA TIMA_REG`

20.92.2.33 rTMA `#define rTMA TMA_REG`

20.92.2.34 rTAC `#define rTAC TAC_REG`

20.92.2.35 TACF_START `#define TACF_START 0b00000100`

20.92.2.36 TACF_STOP `#define TACF_STOP 0b00000000`

20.92.2.37 TACF_4KHZ `#define TACF_4KHZ 0b00000000`

20.92.2.38 TACF_16KHZ `#define TACF_16KHZ 0b00000011`

20.92.2.39 TACF_65KHZ `#define TACF_65KHZ 0b00000010`

20.92.2.40 TACF_262KHZ `#define TACF_262KHZ 0b00000001`

20.92.2.41 rIF `#define rIF IF_REG`

20.92.2.42 rAUD1SWEEP #define rAUD1SWEEP NR10_REG
Sound Channel 1, NR10: Sweep

20.92.2.43 AUD1SWEEP_UP #define AUD1SWEEP_UP 0b00000000
For Sound Channel 1, NR10: Sweep Addition, period increases

20.92.2.44 AUD1SWEEP_DOWN #define AUD1SWEEP_DOWN 0b00001000
For Sound Channel 1, NR10: Sweep Subtraction, period decreases

20.92.2.45 AUD1SWEEP_TIME #define AUD1SWEEP_TIME(
x) ((x) << 4)
For Sound Channel 1, NR10: Sweep Time/Pace, Range: 0-7

20.92.2.46 AUD1SWEEP_LENGTH #define AUD1SWEEP_LENGTH(
x) (x)
For Sound Channel 1, NR10: Sweep Length/Individual step, Range: 0-7

20.92.2.47 rAUD1LEN #define rAUD1LEN NR11_REG
Sound Channel 1, NR11: Sound length/Wave pattern duty

20.92.2.48 rAUD1ENV #define rAUD1ENV NR12_REG
Sound Channel 1, NR12: Volume Envelope

20.92.2.49 rAUD1LOW #define rAUD1LOW NR13_REG
Sound Channel 1, NR13: Frequency Low

20.92.2.50 rAUD1HIGH #define rAUD1HIGH NR14_REG
Sound Channel 1, NR14: Frequency High

20.92.2.51 rAUD2LEN #define rAUD2LEN NR21_REG
Sound Channel 2, NR21_REG: Tone

20.92.2.52 rAUD2ENV #define rAUD2ENV NR22_REG
Sound Channel 2, NR22_REG: Volume Envelope

20.92.2.53 rAUD2LOW #define rAUD2LOW NR23_REG
Sound Channel 2, NR23_REG: Frequency data Low

20.92.2.54 rAUD2HIGH #define rAUD2HIGH NR24_REG
Sound Channel 2, NR24_REG: Frequency data High

20.92.2.55 rAUD3ENA #define rAUD3ENA NR30_REG
Sound Channel 3, NR30_REG: Sound on/off

20.92.2.56 rAUD3LEN #define rAUD3LEN NR31_REG
Sound Channel 3, NR31_REG: Sound Length

20.92.2.57 rAUD3LEVEL #define rAUD3LEVEL NR32_REG
Sound Channel 3, NR32_REG: Select output level

20.92.2.58 rAUD3LOW #define rAUD3LOW NR33_REG
Sound Channel 3, NR33_REG: Frequency data Low

20.92.2.59 rAUD3HIGH `#define rAUD3HIGH NR34_REG`

Sound Channel 3, NR34_REG: Frequency data High

20.92.2.60 rAUD4LEN `#define rAUD4LEN NR41_REG`

Sound Channel 4, NR41_REG: Sound Length

20.92.2.61 rAUD4ENV `#define rAUD4ENV NR42_REG`

Sound Channel 4, NR42_REG: Volume Envelope

20.92.2.62 rAUD4POLY `#define rAUD4POLY NR43_REG`

Sound Channel 4, NR43_REG: Polynomial Counter

20.92.2.63 AUD4POLY_WIDTH_15BIT `#define AUD4POLY_WIDTH_15BIT 0x00`

For Sound Channel 4, NR43_REG: Polynomial counter use 15 steps

20.92.2.64 AUD4POLY_WIDTH_7BIT `#define AUD4POLY_WIDTH_7BIT 0x08`

For Sound Channel 4, NR43_REG: Polynomial counter use 7 steps

20.92.2.65 rAUD4GO `#define rAUD4GO NR44_REG`

Sound Channel 4, NR44_REG: Counter / Consecutive and Initial

20.92.2.66 rAUDVOL `#define rAUDVOL NR50_REG`

Sound Master Volume, NR50: Volume and Cart external sound input (VIN)

20.92.2.67 AUDVOL_VOL_LEFT `#define AUDVOL_VOL_LEFT(`
`x) ((x) << 4)`

For Sound Master Volume, NR50: Left Volume, Range: 0-7

20.92.2.68 AUDVOL_VOL_RIGHT `#define AUDVOL_VOL_RIGHT(`
`x) ((x))`

For Sound Master Volume, NR50: Right Volume, Range: 0-7

20.92.2.69 AUDVOL_VIN_LEFT `#define AUDVOL_VIN_LEFT 0b10000000`

For Sound Master Volume, NR50: Cart external sound input (VIN) Left bit, 1 = ON, 0 = OFF

20.92.2.70 AUDVOL_VIN_RIGHT `#define AUDVOL_VIN_RIGHT 0b00001000`

For Sound Master Volume, NR50: Cart external sound input (VIN) Right bit, 1 = ON, 0 = OFF

20.92.2.71 rAUDTERM `#define rAUDTERM NR51_REG`

Sound Panning, NR51: Enable/disable left and right output for sound channels

20.92.2.72 AUDTERM_4_LEFT `#define AUDTERM_4_LEFT 0b10000000`

For Sound Panning, NR51: Channel 4 Left bit, 1 = ON, 0 = OFF

20.92.2.73 AUDTERM_3_LEFT `#define AUDTERM_3_LEFT 0b01000000`

For Sound Panning, NR51: Channel 3 Left bit, 1 = ON, 0 = OFF

20.92.2.74 AUDTERM_2_LEFT `#define AUDTERM_2_LEFT 0b00100000`

For Sound Panning, NR51: Channel 2 Left bit, 1 = ON, 0 = OFF

20.92.2.75 AUDTERM_1_LEFT `#define AUDTERM_1_LEFT 0b00010000`
For Sound Panning, NR51: Channel 1 Left bit, 1 = ON, 0 = OFF

20.92.2.76 AUDTERM_4_RIGHT `#define AUDTERM_4_RIGHT 0b00001000`
For Sound Panning, NR51: Channel 4 Right bit, 1 = ON, 0 = OFF

20.92.2.77 AUDTERM_3_RIGHT `#define AUDTERM_3_RIGHT 0b00000100`
For Sound Panning, NR51: Channel 4 Right bit, 1 = ON, 0 = OFF

20.92.2.78 AUDTERM_2_RIGHT `#define AUDTERM_2_RIGHT 0b00000010`
For Sound Panning, NR51: Channel 4 Right bit, 1 = ON, 0 = OFF

20.92.2.79 AUDTERM_1_RIGHT `#define AUDTERM_1_RIGHT 0b00000001`
For Sound Panning, NR51: Channel 4 Right bit, 1 = ON, 0 = OFF

20.92.2.80 rAUDENA `#define rAUDENA NR52_REG`
Sound Master Control, NR52: ON / OFF

20.92.2.81 AUDENA_ON `#define AUDENA_ON 0b10000000`
For Sound Master Control, NR52: Sound ON

20.92.2.82 AUDENA_OFF `#define AUDENA_OFF 0b00000000`
For Sound Master Control, NR52: Sound OFF

20.92.2.83 rLCDC `#define rLCDC LCDC_REG`

20.92.2.84 LCDCF_OFF `#define LCDCF_OFF 0b00000000`
LCD Control: Off

20.92.2.85 LCDCF_ON `#define LCDCF_ON 0b10000000`
LCD Control: On

20.92.2.86 LCDCF_WIN9800 `#define LCDCF_WIN9800 0b00000000`
Window Tile Map: Use 9800 Region

20.92.2.87 LCDCF_WIN9C00 `#define LCDCF_WIN9C00 0b01000000`
Window Tile Map: Use 9C00 Region

20.92.2.88 LCDCF_WINOFF `#define LCDCF_WINOFF 0b00000000`
Window Display: Hidden

20.92.2.89 LCDCF_WINON `#define LCDCF_WINON 0b00100000`
Window Display: Visible

20.92.2.90 LCDCF_BG8800 `#define LCDCF_BG8800 0b00000000`
BG & Window Tile Data: Use 8800 Region

20.92.2.91 LCDCF_BG8000 `#define LCDCF_BG8000 0b00010000`
BG & Window Tile Data: Use 8000 Region

20.92.2.92 LCDCF_BG9800 `#define LCDCF_BG9800 0b00000000`
BG Tile Map: use 9800 Region

20.92.2.93 LCDCF_BG9C00 `#define LCDCF_BG9C00 0b00001000`
BG Tile Map: use 9C00 Region

20.92.2.94 LCDCF_OBJ8 `#define LCDCF_OBJ8 0b00000000`
Sprites Size: 8x8 pixels

20.92.2.95 LCDCF_OBJ16 `#define LCDCF_OBJ16 0b00000100`
Sprites Size: 8x16 pixels

20.92.2.96 LCDCF_OBJOFF `#define LCDCF_OBJOFF 0b00000000`
Sprites Display: Hidden

20.92.2.97 LCDCF_OBJON `#define LCDCF_OBJON 0b00000010`
Sprites Display: Visible

20.92.2.98 LCDCF_BG0FF `#define LCDCF_BG0FF 0b00000000`
Background Display: Hidden

20.92.2.99 LCDCF_BGON `#define LCDCF_BGON 0b00000001`
Background Display: Visible

20.92.2.100 LCDCF_B_ON `#define LCDCF_B_ON 7`
Bit for LCD On/Off Select

20.92.2.101 LCDCF_B_WIN9C00 `#define LCDCF_B_WIN9C00 6`
Bit for Window Tile Map Region Select

20.92.2.102 LCDCF_B_WINON `#define LCDCF_B_WINON 5`
Bit for Window Display On/Off Control

20.92.2.103 LCDCF_B_BG8000 `#define LCDCF_B_BG8000 4`
Bit for BG & Window Tile Data Region Select

20.92.2.104 LCDCF_B_BG9C00 `#define LCDCF_B_BG9C00 3`
Bit for BG Tile Map Region Select

20.92.2.105 LCDCF_B_OBJ16 `#define LCDCF_B_OBJ16 2`
Bit for Sprites Size Select

20.92.2.106 LCDCF_B_OBJON `#define LCDCF_B_OBJON 1`
Bit for Sprites Display Visible/Hidden Select

20.92.2.107 LCDCF_B_BGON `#define LCDCF_B_BGON 0`
Bit for Background Display Visible/Hidden Select

20.92.2.108 rSTAT `#define rSTAT STAT_REG`

20.92.2.109 STATF_LYC `#define STATF_LYC 0b01000000`
STAT Interrupt: LYC=LY Coincidence Source Enable

20.92.2.110 STATF_MODE10 `#define STATF_MODE10 0b00100000`
STAT Interrupt: Mode 2 OAM Source Enable

20.92.2.111 STATF_MODE01 `#define STATF_MODE01 0b00010000`
STAT Interrupt: Mode 1 VBlank Source Enable

20.92.2.112 STATF_MODE00 `#define STATF_MODE00 0b00001000`
STAT Interrupt: Mode 0 HBlank Source Enable

20.92.2.113 STATF_LYCF `#define STATF_LYCF 0b00000100`
LYC=LY Coincidence Status Flag, Set when LY contains the same value as LYC

20.92.2.114 STATF_HBL `#define STATF_HBL 0b00000000`
Current LCD Mode is: 0, in H-Blank

20.92.2.115 STATF_VBL `#define STATF_VBL 0b00000001`
Current LCD Mode is: 1, in V-Blank

20.92.2.116 STATF_OAM `#define STATF_OAM 0b00000010`
Current LCD Mode is: 2, in OAM-RAM is used by system (Searching OAM)

20.92.2.117 STATF_LCD `#define STATF_LCD 0b00000011`
Current LCD Mode is: 3, both OAM and VRAM used by system (Transferring Data to LCD Controller)

20.92.2.118 STATF_BUSY `#define STATF_BUSY 0b00000010`
When set, VRAM access is unsafe

20.92.2.119 STATF_B_LYC `#define STATF_B_LYC 6`
Bit for STAT Interrupt: LYC=LY Coincidence Source Enable

20.92.2.120 STATF_B_MODE10 `#define STATF_B_MODE10 5`
Bit for STAT Interrupt: Mode 2 OAM Source Enable

20.92.2.121 STATF_B_MODE01 `#define STATF_B_MODE01 4`
Bit for STAT Interrupt: Mode 1 VBlank Source Enable

20.92.2.122 STATF_B_MODE00 `#define STATF_B_MODE00 3`
Bit for STAT Interrupt: Mode 0 HBlank Source Enable

20.92.2.123 STATF_B_LYCF `#define STATF_B_LYCF 2`
Bit for LYC=LY Coincidence Status Flag

20.92.2.124 STATF_B_VBL `#define STATF_B_VBL 0`

20.92.2.125 STATF_B_OAM `#define STATF_B_OAM 1`

20.92.2.126 STATF_B_BUSY `#define STATF_B_BUSY 1`
Bit for when VRAM access is unsafe

20.92.2.127 rSCY `#define rSCY`

20.92.2.128 rSCX `#define rSCX SCX_REG`

20.92.2.129 rLY `#define rLY LY_REG`

20.92.2.130 rLYC `#define rLYC LYC_REG`

20.92.2.131 rDMA `#define rDMA DMA_REG`

20.92.2.132 rBGP `#define rBGP BGP_REG`

20.92.2.133 rOBP0 `#define rOBP0 OBP0_REG`

20.92.2.134 rOBP1 `#define rOBP1 OBP1_REG`

20.92.2.135 rWY `#define rWY WY_REG`

20.92.2.136 rWX `#define rWX WX_REG`

20.92.2.137 rKEY1 `#define rKEY1 KEY1_REG`

20.92.2.138 rSPD `#define rSPD KEY1_REG`

20.92.2.139 KEY1F_DBLSPD `#define KEY1F_DBLSPD 0b10000000`

20.92.2.140 KEY1F_PREPARE `#define KEY1F_PREPARE 0b00000001`

20.92.2.141 rVBK `#define rVBK VBK_REG`

20.92.2.142 VBK_BANK_0 `#define VBK_BANK_0 0`
Select Regular Map and Normal Tiles (CGB Mode Only)

20.92.2.143 VBK_TILES `#define VBK_TILES 0`
Select Regular Map and Normal Tiles (CGB Mode Only)

20.92.2.144 VBK_BANK_1 `#define VBK_BANK_1 1`
Select Map Attributes and Extra Tile Bank (CGB Mode Only)

20.92.2.145 VBK_ATTRIBUTES `#define VBK_ATTRIBUTES 1`
Select Map Attributes and Extra Tile Bank (CGB Mode Only)

20.92.2.146 BKGF_PRI `#define BKGF_PRI 0b10000000`
Background CGB BG and Window over Sprite priority Enabled

20.92.2.147 BKGF_YFLIP `#define BKGF_YFLIP 0b01000000`
Background CGB Y axis flip: Vertically mirrored

20.92.2.148 BKGF_XFLIP `#define BKGF_XFLIP 0b00100000`
Background CGB X axis flip: Horizontally mirrored

20.92.2.149 BKGF_BANK0 `#define BKGF_BANK0 0b00000000`
Background CGB Tile VRAM-Bank: Use Bank 0 (CGB Mode Only)

20.92.2.150 BKGF_BANK1 `#define BKGF_BANK1 0b00001000`
Background CGB Tile VRAM-Bank: Use Bank 1 (CGB Mode Only)

20.92.2.151 BKGF_CGB_PAL0 `#define BKGF_CGB_PAL0 0b00000000`
Background CGB Palette number (CGB Mode Only)

20.92.2.152 BKGF_CGB_PAL1 `#define BKGF_CGB_PAL1 0b00000001`
Background CGB Palette number (CGB Mode Only)

20.92.2.153 BKGF_CGB_PAL2 `#define BKGF_CGB_PAL2 0b00000010`
Background CGB Palette number (CGB Mode Only)

20.92.2.154 BKGF_CGB_PAL3 `#define BKGF_CGB_PAL3 0b00000011`
Background CGB Palette number (CGB Mode Only)

20.92.2.155 BKGF_CGB_PAL4 `#define BKGF_CGB_PAL4 0b00000100`
Background CGB Palette number (CGB Mode Only)

20.92.2.156 BKGF_CGB_PAL5 `#define BKGF_CGB_PAL5 0b00000101`
Background CGB Palette number (CGB Mode Only)

20.92.2.157 BKGF_CGB_PAL6 `#define BKGF_CGB_PAL6 0b00000110`
Background CGB Palette number (CGB Mode Only)

20.92.2.158 BKGF_CGB_PAL7 `#define BKGF_CGB_PAL7 0b00000111`
Background CGB Palette number (CGB Mode Only)

20.92.2.159 rHDMA1 `#define rHDMA1 HDMA1\_REG`

20.92.2.160 rHDMA2 `#define rHDMA2 HDMA2\_REG`

20.92.2.161 rHDMA3 `#define rHDMA3 HDMA3\_REG`

20.92.2.162 rHDMA4 `#define rHDMA4 HDMA4\_REG`

20.92.2.163 rHDMA5 `#define rHDMA5 HDMA5\_REG`

20.92.2.164 HDMA5F_MODE_GP `#define HDMA5F_MODE_GP 0b00000000`

20.92.2.165 HDMA5F_MODE_HBL `#define HDMA5F_MODE_HBL 0b10000000`

20.92.2.166 HDMA5F_BUSY `#define HDMA5F_BUSY 0b10000000`

20.92.2.167 rRP `#define rRP RP_REG`

20.92.2.168 RPF_ENREAD `#define RPF_ENREAD 0b11000000`

20.92.2.169 RPF_DATAIN `#define RPF_DATAIN 0b00000010`

20.92.2.170 RPF_WRITE_HI `#define RPF_WRITE_HI 0b00000001`

20.92.2.171 RPF_WRITE_LO `#define RPF_WRITE_LO 0b00000000`

20.92.2.172 rBCPS `#define rBCPS BCPS_REG`

20.92.2.173 BCPSF_AUTOINC `#define BCPSF_AUTOINC 0b10000000`

20.92.2.174 rBCPD `#define rBCPD BCPD_REG`

20.92.2.175 rOCPS `#define rOCPS OCPS_REG`

20.92.2.176 OCPSF_AUTOINC `#define OCPSF_AUTOINC 0b10000000`

20.92.2.177 rOCPD `#define rOCPD OCPD_REG`

20.92.2.178 rSVBK `#define rSVBK SVBK_REG`

20.92.2.179 rSMBK `#define rSMBK SVBK_REG`

20.92.2.180 rPCM12 `#define rPCM12 PCM12_REG`

20.92.2.181 rPCM34 `#define rPCM34 PCM34_REG`

20.92.2.182 **rIE** `#define rIE IE_REG`

20.92.2.183 **IEF_HILO** `#define IEF_HILO 0b00010000`
Joypad interrupt enable flag

20.92.2.184 **IEF_SERIAL** `#define IEF_SERIAL 0b00001000`
Serial interrupt enable flag

20.92.2.185 **IEF_TIMER** `#define IEF_TIMER 0b00000100`
Timer interrupt enable flag

20.92.2.186 **IEF_STAT** `#define IEF_STAT 0b00000010`
Stat interrupt enable flag

20.92.2.187 **IEF_VBLANK** `#define IEF_VBLANK 0b00000001`
VBlank interrupt enable flag

20.92.2.188 **AUDLEN_DUTY_12_5** `#define AUDLEN_DUTY_12_5 0b00000000`

20.92.2.189 **AUDLEN_DUTY_25** `#define AUDLEN_DUTY_25 0b01000000`

20.92.2.190 **AUDLEN_DUTY_50** `#define AUDLEN_DUTY_50 0b10000000`

20.92.2.191 **AUDLEN_DUTY_75** `#define AUDLEN_DUTY_75 0b11000000`

20.92.2.192 **AUDLEN_LENGTH** `#define AUDLEN_LENGTH(
x) (x)`

20.92.2.193 **AUDENV_VOL** `#define AUDENV_VOL(
x) ((x) << 4)`

20.92.2.194 **AUDENV_UP** `#define AUDENV_UP 0b00001000`

20.92.2.195 **AUDENV_DOWN** `#define AUDENV_DOWN 0b00000000`

20.92.2.196 **AUDENV_LENGTH** `#define AUDENV_LENGTH(
x) (x)`

20.92.2.197 **AUDHIGH_RESTART** `#define AUDHIGH_RESTART 0b10000000`

20.92.2.198 **AUDHIGH_LENGTH_ON** `#define AUDHIGH_LENGTH_ON 0b01000000`

20.92.2.199 AUDHIGH_LENGTH_OFF `#define AUDHIGH_LENGTH_OFF 0b00000000`

20.92.2.200 OAMF_PRI `#define OAMF_PRI 0b10000000`

BG and Window over Sprite Enabled

20.92.2.201 OAMF_YFLIP `#define OAMF_YFLIP 0b01000000`

Sprite Y axis flip: Vertically mirrored

20.92.2.202 OAMF_XFLIP `#define OAMF_XFLIP 0b00100000`

Sprite X axis flip: Horizontally mirrored

20.92.2.203 OAMF_PAL0 `#define OAMF_PAL0 0b00000000`

Sprite Palette number: use OBP0 (Non-CGB Mode Only)

20.92.2.204 OAMF_PAL1 `#define OAMF_PAL1 0b00010000`

Sprite Palette number: use OBP1 (Non-CGB Mode Only)

20.92.2.205 OAMF_BANK0 `#define OAMF_BANK0 0b00000000`

Sprite Tile VRAM-Bank: Use Bank 0 (CGB Mode Only)

20.92.2.206 OAMF_BANK1 `#define OAMF_BANK1 0b00001000`

Sprite Tile VRAM-Bank: Use Bank 1 (CGB Mode Only)

20.92.2.207 OAMF_CGB_PAL0 `#define OAMF_CGB_PAL0 0b00000000`

Sprite CGB Palette number: use OCP0 (CGB Mode Only)

20.92.2.208 OAMF_CGB_PAL1 `#define OAMF_CGB_PAL1 0b00000001`

Sprite CGB Palette number: use OCP1 (CGB Mode Only)

20.92.2.209 OAMF_CGB_PAL2 `#define OAMF_CGB_PAL2 0b00000010`

Sprite CGB Palette number: use OCP2 (CGB Mode Only)

20.92.2.210 OAMF_CGB_PAL3 `#define OAMF_CGB_PAL3 0b00000011`

Sprite CGB Palette number: use OCP3 (CGB Mode Only)

20.92.2.211 OAMF_CGB_PAL4 `#define OAMF_CGB_PAL4 0b00000100`

Sprite CGB Palette number: use OCP4 (CGB Mode Only)

20.92.2.212 OAMF_CGB_PAL5 `#define OAMF_CGB_PAL5 0b00000101`

Sprite CGB Palette number: use OCP5 (CGB Mode Only)

20.92.2.213 OAMF_CGB_PAL6 `#define OAMF_CGB_PAL6 0b00000110`

Sprite CGB Palette number: use OCP6 (CGB Mode Only)

20.92.2.214 OAMF_CGB_PAL7 `#define OAMF_CGB_PAL7 0b00000111`

Sprite CGB Palette number: use OCP7 (CGB Mode Only)

20.92.2.215 OAMF_PALMASK `#define OAMF_PALMASK 0b00000111`

Mask for Sprite CGB Palette number (CGB Mode Only)

20.92.2.216 DEVICE_SCREEN_X_OFFSET `#define DEVICE_SCREEN_X_OFFSET 0`

Offset of visible screen (in tile units) from left edge of hardware map

20.92.2.217 DEVICE_SCREEN_Y_OFFSET `#define DEVICE_SCREEN_Y_OFFSET 0`
Offset of visible screen (in tile units) from top edge of hardware map

20.92.2.218 DEVICE_SCREEN_WIDTH `#define DEVICE_SCREEN_WIDTH 20`
Width of visible screen in tile units

20.92.2.219 DEVICE_SCREEN_HEIGHT `#define DEVICE_SCREEN_HEIGHT 18`
Height of visible screen in tile units

20.92.2.220 DEVICE_SCREEN_BUFFER_WIDTH `#define DEVICE_SCREEN_BUFFER_WIDTH 32`
Width of hardware map buffer in tile units

20.92.2.221 DEVICE_SCREEN_BUFFER_HEIGHT `#define DEVICE_SCREEN_BUFFER_HEIGHT 32`
Height of hardware map buffer in tile units

20.92.2.222 DEVICE_SCREEN_MAP_ENTRY_SIZE `#define DEVICE_SCREEN_MAP_ENTRY_SIZE 1`
Number of bytes per hardware map entry

20.92.2.223 DEVICE_SPRITE_PX_OFFSET_X `#define DEVICE_SPRITE_PX_OFFSET_X 8`
Offset of sprite X coordinate origin (in pixels) from left edge of visible screen

20.92.2.224 DEVICE_SPRITE_PX_OFFSET_Y `#define DEVICE_SPRITE_PX_OFFSET_Y 16`
Offset of sprite Y coordinate origin (in pixels) from top edge of visible screen

20.92.2.225 DEVICE_WINDOW_PX_OFFSET_X `#define DEVICE_WINDOW_PX_OFFSET_X 7`
Minimal X coordinate of the window layer

20.92.2.226 DEVICE_WINDOW_PX_OFFSET_Y `#define DEVICE_WINDOW_PX_OFFSET_Y 0`
Minimal Y coordinate of the window layer

20.92.2.227 DEVICE_SCREEN_PX_WIDTH `#define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)`
Width of visible screen in pixels

20.92.2.228 DEVICE_SCREEN_PX_HEIGHT `#define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)`
Height of visible screen in pixels

20.92.3 Variable Documentation

20.92.3.1 _VRAM `__BYTES _VRAM[ ]`
Memory map

20.92.3.2 _VRAM8000 `__BYTES _VRAM8000[ ]`

20.92.3.3 _VRAM8800 `__BYTES _VRAM8800[ ]`

20.92.3.4 _VRAM9000 `__BYTES _VRAM9000[ ]`

20.92.3.5 `_SCRN0` `__BYTES` `_SCRN0 [ ]`

20.92.3.6 `_SCRN1` `__BYTES` `_SCRN1 [ ]`

20.92.3.7 `_SRAM` `__BYTES` `_SRAM [ ]`

20.92.3.8 `_RAM` `__BYTES` `_RAM [ ]`

20.92.3.9 `_RAMBANK` `__BYTES` `_RAMBANK [ ]`

20.92.3.10 `_OAMRAM` `__BYTES` `_OAMRAM [ ]`

20.92.3.11 `_IO` `__BYTE_REG` `_IO [ ]`

20.92.3.12 `_AUD3WAVERAM` `__BYTE_REG` `_AUD3WAVERAM [ ]`

20.92.3.13 `_HRAM` `__BYTE_REG` `_HRAM [ ]`

20.92.3.14 `rRAMG` `__BYTE_REG` `rRAMG`
MBC5 registers

20.92.3.15 `rROMB0` `__BYTE_REG` `rROMB0`

20.92.3.16 `rROMB1` `__BYTE_REG` `rROMB1`

20.92.3.17 `rRAMB` `__BYTE_REG` `rRAMB`

20.92.3.18 `rMBC7_SRAM_ENABLE_1` `__BYTE_REG` `rMBC7_SRAM_ENABLE_1`
MBC7 registers

20.92.3.19 `rMBC7_SRAM_ENABLE_2` `__BYTE_REG` `rMBC7_SRAM_ENABLE_2`

20.92.3.20 `rMBC7_LATCH_1` `__BYTE_REG` `rMBC7_LATCH_1`

20.92.3.21 `rMBC7_LATCH_2` `__BYTE_REG` `rMBC7_LATCH_2`

20.92.3.22 `rMBC7_ACCEL_X_LO` `__BYTE_REG` `rMBC7_ACCEL_X_LO`

20.92.3.23 **rMBC7_ACCEL_X_HI** [__BYTE_REG](#) rMBC7_ACCEL_X_HI

20.92.3.24 **rMBC7_ACCEL_Y_LO** [__BYTE_REG](#) rMBC7_ACCEL_Y_LO

20.92.3.25 **rMBC7_ACCEL_Y_HI** [__BYTE_REG](#) rMBC7_ACCEL_Y_HI

20.92.3.26 **P1_REG** [__REG](#) P1_REG

IO Registers Joystick register

See also

[joypad\(\)](#), [add_JOY\(\)](#), [IEF_HILO](#), [P1F_5](#), [P1F_4](#), [P1F_3](#), [P1F_2](#), [P1F_1](#), [P1F_0](#), [P1F_GET_DPAD](#),
[P1F_GET_BTN](#), [P1F_GET_NONE](#)

20.92.3.27 **SB_REG** [__REG](#) SB_REG

Serial IO data buffer

20.92.3.28 **SC_REG** [__REG](#) SC_REG

Serial IO control register

20.92.3.29 **DIV_REG** [__REG](#) DIV_REG

Divider register

20.92.3.30 **TIMA_REG** [__REG](#) TIMA_REG

Timer counter

20.92.3.31 **TMA_REG** [__REG](#) TMA_REG

Timer modulo

20.92.3.32 **TAC_REG** [__REG](#) TAC_REG

Timer control

20.92.3.33 **IF_REG** [__REG](#) IF_REG

Interrupt flags: [IEF_HILO](#), [IEF_SERIAL](#), [IEF_TIMER](#), [IEF_STAT](#), [IEF_VBLANK](#)

20.92.3.34 **NR10_REG** [__REG](#) NR10_REG

Sound Channel 1, NR10: Sweep

20.92.3.35 **NR11_REG** [__REG](#) NR11_REG

Sound Channel 1, NR11: Sound length/Wave pattern duty

20.92.3.36 **NR12_REG** [__REG](#) NR12_REG

Sound Channel 1, NR12: Volume Envelope

20.92.3.37 **NR13_REG** [__REG](#) NR13_REG

Sound Channel 1, NR13: Frequency Low

20.92.3.38 **NR14_REG** [__REG](#) NR14_REG

Sound Channel 1, NR14: Frequency High

20.92.3.39 NR21_REG `__REG` NR21_REG

Sound Channel 2, NR21_REG: Tone

20.92.3.40 NR22_REG `__REG` NR22_REG

Sound Channel 2, NR22_REG: Volume Envelope

20.92.3.41 NR23_REG `__REG` NR23_REG

Sound Channel 2, NR23_REG: Frequency data Low

20.92.3.42 NR24_REG `__REG` NR24_REG

Sound Channel 2, NR24_REG: Frequency data High

20.92.3.43 NR30_REG `__REG` NR30_REG

Sound Channel 3, NR30_REG: Sound on/off

20.92.3.44 NR31_REG `__REG` NR31_REG

Sound Channel 3, NR31_REG: Sound Length

20.92.3.45 NR32_REG `__REG` NR32_REG

Sound Channel 3, NR32_REG: Select output level

20.92.3.46 NR33_REG `__REG` NR33_REG

Sound Channel 3, NR33_REG: Frequency data Low

20.92.3.47 NR34_REG `__REG` NR34_REG

Sound Channel 3, NR34_REG: Frequency data High

20.92.3.48 NR41_REG `__REG` NR41_REG

Sound Channel 4, NR41_REG: Sound Length

20.92.3.49 NR42_REG `__REG` NR42_REG

Sound Channel 4, NR42_REG: Volume Envelope

20.92.3.50 NR43_REG `__REG` NR43_REG

Sound Channel 4, NR43_REG: Polynomial Counter

20.92.3.51 NR44_REG `__REG` NR44_REG

Sound Channel 4, NR44_REG: Counter / Consecutive and Initial

20.92.3.52 NR50_REG `__REG` NR50_REG

Sound Master Volume, NR50: Volume and Cart external sound input (VIN)

20.92.3.53 NR51_REG `__REG` NR51_REG

Sound Panning, NR51: Enable/disable left and right output for sound channels

20.92.3.54 NR52_REG `__REG` NR52_REG

Sound Master Control, NR52: ON / OFF

20.92.3.55 AUD3WAVE `__BYTE_REG` AUD3WAVE[16]**20.92.3.56 PCM_SAMPLE** `__BYTE_REG` PCM_SAMPLE[16]

20.92.3.57 LCDC_REG [__REG](#) LCDC_REG
LCD control

20.92.3.58 STAT_REG [__REG](#) STAT_REG
LCD status

20.92.3.59 SCY_REG [__REG](#) SCY_REG
Scroll Y

20.92.3.60 SCX_REG [__REG](#) SCX_REG
Scroll X

20.92.3.61 LY_REG [__REG](#) LY_REG
LCDC Y-coordinate

20.92.3.62 LYC_REG [__REG](#) LYC_REG
LY compare

20.92.3.63 DMA_REG [__REG](#) DMA_REG
DMA transfer

20.92.3.64 BGP_REG [__REG](#) BGP_REG
Set and Read the Background palette.

Example with the [DMG_PALETTE\(\)](#) helper function and constants:
BGP_REG = [DMG_PALETTE\(DMG_BLACK, DMG_DARK_GRAY, DMG_LITE_GRAY, DMG_WHITE\)](#);

20.92.3.65 OBP0_REG [__REG](#) OBP0_REG
Set and Read the OBJ (Sprite) palette 0.

The first color entry is always transparent.

Example with the [DMG_PALETTE\(\)](#) helper function and constants:
OBP0_REG = [DMG_PALETTE\(DMG_BLACK, DMG_DARK_GRAY, DMG_LITE_GRAY, DMG_WHITE\)](#);

20.92.3.66 OBP1_REG [__REG](#) OBP1_REG
Set and Read the OBJ (Sprite) palette 1.

The first color entry is always transparent.

Example with the [DMG_PALETTE\(\)](#) helper function and constants:
OBP1_REG = [DMG_PALETTE\(DMG_BLACK, DMG_DARK_GRAY, DMG_LITE_GRAY, DMG_WHITE\)](#);

20.92.3.67 WY_REG [__REG](#) WY_REG
Window Y coordinate

20.92.3.68 WX_REG [__REG](#) WX_REG
Window X coordinate

20.92.3.69 KEY1_REG [__REG](#) KEY1_REG
CPU speed

20.92.3.70 VBK_REG [__REG](#) VBK_REG
VRAM bank select (CGB only)

See also

[VBK_BANK_0](#), [VBK_TILES](#), [VBK_BANK_1](#), [VBK_ATTRIBUTES](#)

20.92.3.71 HDMA1_REG [__REG](#) HDMA1_REG
DMA control 1

20.92.3.72 HDMA2_REG [__REG](#) HDMA2_REG
DMA control 2

20.92.3.73 HDMA3_REG [__REG](#) HDMA3_REG
DMA control 3

20.92.3.74 HDMA4_REG [__REG](#) HDMA4_REG
DMA control 4

20.92.3.75 HDMA5_REG [__REG](#) HDMA5_REG
DMA control 5

20.92.3.76 RP_REG [__REG](#) RP_REG
IR port

20.92.3.77 BCPS_REG [__REG](#) BCPS_REG
BG color palette specification

20.92.3.78 BCPD_REG [__REG](#) BCPD_REG
BG color palette data

20.92.3.79 OCPS_REG [__REG](#) OCPS_REG
OBJ color palette specification

20.92.3.80 OCPD_REG [__REG](#) OCPD_REG
OBJ color palette data

20.92.3.81 SVBK_REG [__REG](#) SVBK_REG
Selects the WRAM upper region bank (CGB Only). WRAM Banking is NOT officially supported in GBDK and SDCC.
The stack must be moved and other special care taken.

20.92.3.82 PCM12_REG [__REG](#) PCM12_REG
Sound channel 1&2 PCM amplitude (R)

20.92.3.83 PCM34_REG [__REG](#) PCM34_REG
Sound channel 3&4 PCM amplitude (R)

20.92.3.84 IE_REG [__REG](#) IE_REG
Interrupt enable

20.93 hardware.h

[Go to the documentation of this file.](#)

```

1
2 #ifndef _HARDWARE_H
3 #define _HARDWARE_H
4
5 #include <types.h>
6
7 #define __BYTES extern UBYTE
8 #define __BYTE_REG extern volatile UBYTE
9 #define __REG extern volatile SFR
10
11
12 __BYTES _VRAM[];
13 __BYTES _VRAM8000[];
14 __BYTES _VRAM8800[];
15 __BYTES _VRAM9000[];
16 __BYTES _SCRN0[];
17 __BYTES _SCRN1[];
18 __BYTES _SRAM[];
19 __BYTES _RAM[];
20 __BYTES _RAMBANK[];
21 __BYTES _OAMRAM[];
22 __BYTE_REG _IO[];
23 __BYTE_REG _AUD3WAVERAM[];
24 __BYTE_REG _HRAM[];
25
26 __BYTE_REG rRAMG;
27 __BYTE_REG rROMB0;
28 __BYTE_REG rROMB1;
29 __BYTE_REG rRAMB;
30
31 __BYTE_REG rMBC7_SRAM_ENABLE_1;
32 __BYTE_REG rMBC7_SRAM_ENABLE_2;
33 __BYTE_REG rMBC7_LATCH_1;
34 __BYTE_REG rMBC7_LATCH_2;
35 __BYTE_REG rMBC7_ACCEL_X_LO;
36 __BYTE_REG rMBC7_ACCEL_X_HI;
37 __BYTE_REG rMBC7_ACCEL_Y_LO;
38 __BYTE_REG rMBC7_ACCEL_Y_HI;
39
40 #define MBC7_LATCH_ERASE 0x55u
41 #define MBC7_LATCH_CAPTURE 0xAAu
42 #define MBC7_SRAM_ENABLE_KEY_1 0x0Au
43 #define MBC7_SRAM_ENABLE_KEY_2 0x40u
44
45 __REG P1_REG;
46 #define rP1 P1_REG
47
48 #define P1F_5 0b00100000
49 #define P1F_4 0b00010000
50 #define P1F_3 0b00001000
51 #define P1F_2 0b00000100
52 #define P1F_1 0b00000010
53 #define P1F_0 0b00000001
54
55 #define P1F_GET_DPAD P1F_5
56 #define P1F_GET_BTN P1F_4
57 #define P1F_GET_NONE (P1F_4 | P1F_5)
58
59 __REG SB_REG;
60 #define rSB SB_REG
61 __REG SC_REG;
62 #define rSC SC_REG
63
64 #define SIOF_XFER_START 0b10000000
65 #define SIOF_CLOCK_INT 0b00000001
66 #define SIOF_CLOCK_EXT 0b00000000
67 #define SIOF_SPEED_1X 0b00000000
68 #define SIOF_SPEED_32X 0b00000010
69 #define SIOF_B_CLOCK 0
70 #define SIOF_B_SPEED 1
71 #define SIOF_B_XFER_START 7
72 #define SCF_START SIOF_XFER_START
73 #define SCF_SOURCE SIOF_CLOCK_INT
74 #define SCF_SPEED SIOF_SPEED_32X
75
76 __REG DIV_REG;
77 #define rDIV DIV_REG
78 __REG TMA_REG;
79 #define rTMA TMA_REG
80 __REG TAC_REG;
81 #define rTAC TAC_REG
82
83 #define TACF_START 0b00000100

```



```

98 #define TACF_STOP    0b00000000
99 #define TACF_4KHZ    0b00000000
100 #define TACF_16KHZ   0b00000011
101 #define TACF_65KHZ   0b00000010
102 #define TACF_262KHZ  0b00000001
103
104 __REG IF_REG;
105 #define rIF IF_REG
106
107 __REG NR10_REG;
108 #define rAUD1SWEEP NR10_REG
109 #define AUD1SWEEP_UP    0b00000000
110 #define AUD1SWEEP_DOWN  0b00001000
111 #define AUD1SWEEP_TIME(x) ((x) < 4)
112 #define AUD1SWEEP_LENGTH(x) (x)
113 __REG NR11_REG;
114 #define rAUD1LEN NR11_REG
115 __REG NR12_REG;
116 #define rAUD1ENV NR12_REG
117 __REG NR13_REG;
118 #define rAUD1LOW NR13_REG
119 __REG NR14_REG;
120 #define rAUD1HIGH NR14_REG
122 __REG NR21_REG;
123 #define rAUD2LEN NR21_REG
124 __REG NR22_REG;
125 #define rAUD2ENV NR22_REG
126 __REG NR23_REG;
127 #define rAUD2LOW NR23_REG
128 __REG NR24_REG;
129 #define rAUD2HIGH NR24_REG
131 __REG NR30_REG;
132 #define rAUD3ENA NR30_REG
133 __REG NR31_REG;
134 #define rAUD3LEN NR31_REG
135 __REG NR32_REG;
136 #define rAUD3LEVEL NR32_REG
137 __REG NR33_REG;
138 #define rAUD3LOW NR33_REG
139 __REG NR34_REG;
140 #define rAUD3HIGH NR34_REG
142 __REG NR41_REG;
143 #define rAUD4LEN NR41_REG
144 __REG NR42_REG;
145 #define rAUD4ENV NR42_REG
146 __REG NR43_REG;
147 #define rAUD4POLY NR43_REG
148 #define AUD4POLY_WIDTH_15BIT 0x00
149 #define AUD4POLY_WIDTH_7BIT 0x08
150 __REG NR44_REG;
151 #define rAUD4GO NR44_REG
153 __REG NR50_REG;
154 #define rAUDVOL NR50_REG
156 #define AUDVOL_VOL_LEFT(x) ((x) < 4)
157 #define AUDVOL_VOL_RIGHT(x) ((x))
158 #define AUDVOL_VIN_LEFT    0b10000000
159 #define AUDVOL_VIN_RIGHT   0b00001000
161 __REG NR51_REG;
162 #define rAUDTERM NR51_REG
164 #define AUDTERM_4_LEFT    0b10000000
165 #define AUDTERM_3_LEFT    0b01000000
166 #define AUDTERM_2_LEFT    0b00100000
167 #define AUDTERM_1_LEFT    0b00010000
168 #define AUDTERM_4_RIGHT   0b00001000
169 #define AUDTERM_3_RIGHT   0b00000100
170 #define AUDTERM_2_RIGHT   0b00000010
171 #define AUDTERM_1_RIGHT   0b00000001
173 __REG NR52_REG;
174 #define rAUDENA NR52_REG
176 #define AUDENA_ON    0b10000000
177 #define AUDENA_OFF   0b00000000
179 __BYTE_REG AUD3WAVE[16];
180 __BYTE_REG PCM_SAMPLE[16];
181
182 __REG LCDC_REG;
183 #define rLCDC LCDC_REG
184
185 #if defined(__TARGET_ap)
186 #define LCDCF_OFF    0b00000000
187 #define LCDCF_ON     0b00000001
188 #define LCDCF_WIN9800 0b00000000
189 #define LCDCF_WIN9C00 0b00000010
190 #define LCDCF_WINOFF  0b00000000
191 #define LCDCF_WINON   0b00000100
192 #define LCDCF_BG8800  0b00000000
193 #define LCDCF_BG8000  0b00001000
194 #define LCDCF_BG9800  0b00000000

```

```
195 #define LCDCF_BG9C00      0b00010000
196 #define LCDCF_OBJ8        0b00000000
197 #define LCDCF_OBJ16       0b00100000
198 #define LCDCF_OBJOFF      0b00000000
199 #define LCDCF_OBJJON      0b01000000
200 #define LCDCF_BGOFF       0b00000000
201 #define LCDCF_BGON        0b10000000
202 #define LCDCF_B_ON        0
203 #define LCDCF_B_WIN9C00   1
204 #define LCDCF_B_WINON     2
205 #define LCDCF_B_BG8000    3
206 #define LCDCF_B_BG9C00   4
207 #define LCDCF_B_OBJ16     5
208 #define LCDCF_B_OBJJON    6
209 #define LCDCF_B_BGON      7
210 #elif defined(__TARGET_duck)
211 #define LCDCF_OFF         0b00000000
212 #define LCDCF_ON          0b10000000
213 #define LCDCF_WIN9800     0b00000000
214 #define LCDCF_WIN9C00     0b00001000
215 #define LCDCF_WINOFF      0b00000000
216 #define LCDCF_WINON       0b00100000
217 #define LCDCF_BG8800      0b00000000
218 #define LCDCF_BG8000      0b00010000
219 #define LCDCF_BG9800      0b00000000
220 #define LCDCF_BG9C00      0b00000100
221 #define LCDCF_OBJ8        0b00000000
222 #define LCDCF_OBJ16       0b00000010
223 #define LCDCF_OBJJOFF     0b00000000
224 #define LCDCF_OBJJON      0b00000001
225 #define LCDCF_BGOFF       0b00000000
226 #define LCDCF_BGON        0b01000000
227 #define LCDCF_B_ON        7
228 #define LCDCF_B_WIN9C00   3
229 #define LCDCF_B_WINON     5
230 #define LCDCF_B_BG8000    4
231 #define LCDCF_B_BG9C00    2
232 #define LCDCF_B_OBJ16     1
233 #define LCDCF_B_OBJJON    0
234 #define LCDCF_B_BGON      6
235 #else
236 #define LCDCF_OFF         0b00000000
237 #define LCDCF_ON          0b10000000
238 #define LCDCF_WIN9800     0b00000000
239 #define LCDCF_WIN9C00     0b01000000
240 #define LCDCF_WINOFF      0b00000000
241 #define LCDCF_WINON       0b00100000
242 #define LCDCF_BG8800      0b00000000
243 #define LCDCF_BG8000      0b00010000
244 #define LCDCF_BG9800      0b00000000
245 #define LCDCF_BG9C00      0b00001000
246 #define LCDCF_OBJ8        0b00000000
247 #define LCDCF_OBJ16       0b00000100
248 #define LCDCF_OBJJOFF     0b00000000
249 #define LCDCF_OBJJON      0b00000010
250 #define LCDCF_BGOFF       0b00000000
251 #define LCDCF_BGON        0b00000001
252 #define LCDCF_B_ON        7
253 #define LCDCF_B_WIN9C00    6
254 #define LCDCF_B_WINON     5
255 #define LCDCF_B_BG8000    4
256 #define LCDCF_B_BG9C00    3
257 #define LCDCF_B_OBJ16     2
258 #define LCDCF_B_OBJJON    1
259 #define LCDCF_B_BGON      0
260 #endif
261
262 __REG STAT_REG;
263 #define rSTAT STAT_REG
264
265 #if defined(__TARGET_ap)
266 #define STATF_LYC          0b00000010
267 #define STATF_MODE10       0b00000100
268 #define STATF_MODE01       0b00001000
269 #define STATF_MODE00       0b00010000
270 #define STATF_LYCF         0b00100000
271 #define STATF_HBL          0b00000000
272 #define STATF_VBL          0b10000000
273 #define STATF_OAM           0b01000000
274 #define STATF_LCD           0b11000000
275 #define STATF_BUSY         0b01000000
276 #define STATF_B_LYC        1
277 #define STATF_B_MODE10     2
278 #define STATF_B_MODE01     3
279 #define STATF_B_MODE00     4
280 #define STATF_B_LYCF       5
281 #define STATF_B_VBL        7
```

```
282 #define STATF_B_OAM      6
283 #define STATF_B_BUSY     6
284 #else
285 #define STATF_LYC         0b01000000
286 #define STATF_MODE10      0b00100000
287 #define STATF_MODE01      0b00010000
288 #define STATF_MODE00      0b00001000
289 #define STATF_LYCF        0b00000100
290 #define STATF_HBL         0b00000000
291 #define STATF_VBL         0b00000001
292 #define STATF_OAM         0b00000010
293 #define STATF_LCD         0b00000011
294 #define STATF_BUSY        0b00000010
295 #define STATF_B_LYC       6
296 #define STATF_B_MODE10    5
297 #define STATF_B_MODE01    4
298 #define STATF_B_MODE00    3
299 #define STATF_B_LYCF      2
300 #define STATF_B_VBL       0
301 #define STATF_B_OAM       1
302 #define STATF_B_BUSY      1
303 #endif
304
305 __REG SCY_REG;
306 #define rSCY
307 __REG SCX_REG;
308 #define rSCX SCX_REG
309 __REG LY_REG;
310 #define rLY LY_REG
311 __REG LYC_REG;
312 #define rLYC LYC_REG
313 __REG DMA_REG;
314 #define rDMA DMA_REG
315 __REG BGP_REG;
316 #define rBGP BGP_REG
317 __REG OBP0_REG;
318 #define rOBP0 OBP0_REG
319 __REG OBP1_REG;
320 #define rOBP1 OBP1_REG
321 __REG WY_REG;
322 #define rWY WY_REG
323 __REG WX_REG;
324 #define rWX WX_REG
325 __REG KEY1_REG;
326 #define rKEY1 KEY1_REG
327 #define rSPD KEY1_REG
328
329 #define KEY1F_DBLSPD      0b10000000
330 #define KEY1F_PREPARE    0b00000001
331
332 __REG VBK_REG;
333 #define rVBK VBK_REG
334
335 #define VBK_BANK_0        0
336 #define VBK_TILES         0
337 #define VBK_BANK_1        1
338 #define VBK_ATTRIBUTES    1
339
340 #define BKGF_PRI           0b10000000
341 #define BKGF_YFLIP        0b01000000
342 #define BKGF_XFLIP        0b00100000
343 #define BKGF_BANK0        0b00000000
344 #define BKGF_BANK1        0b00001000
345 #define BKGF_CGB_PAL0     0b00000000
346 #define BKGF_CGB_PAL1     0b00000001
347 #define BKGF_CGB_PAL2     0b00000010
348 #define BKGF_CGB_PAL3     0b00000011
349 #define BKGF_CGB_PAL4     0b00000100
350 #define BKGF_CGB_PAL5     0b00000101
351 #define BKGF_CGB_PAL6     0b00000110
352 #define BKGF_CGB_PAL7     0b00000111
353
354 __REG HDMA1_REG;
355 #define rHDMA1 HDMA1_REG
356 __REG HDMA2_REG;
357 #define rHDMA2 HDMA2_REG
358 __REG HDMA3_REG;
359 #define rHDMA3 HDMA3_REG
360 __REG HDMA4_REG;
361 #define rHDMA4 HDMA4_REG
362 __REG HDMA5_REG;
363 #define rHDMA5 HDMA5_REG
364
365
366 #define HDMA5F_MODE_GP     0b00000000
367 #define HDMA5F_MODE_HBL   0b10000000
368
369 #define HDMA5F_BUSY        0b10000000
370
371 __REG RP_REG;
```

```
372 #define rRP RP_REG
373
374 #define RPF_ENREAD    0b11000000
375 #define RPF_DATAIN    0b00000010
376 #define RPF_WRITE_HI  0b00000001
377 #define RPF_WRITE_LO  0b00000000
378
379 __REG BCPS_REG;
380 #define rBCPS BCPS_REG
381
382 #define BCPSF_AUTOINC 0b10000000
383 __REG BCPD_REG;
384 #define rBCPD BCPD_REG
385
386 __REG OCPS_REG;
387 #define rOCPS OCPS_REG
388
389 #define OCPSF_AUTOINC 0b10000000
390 __REG OCPD_REG;
391 #define rOCPD OCPD_REG
392 __REG SVBK_REG;
393 #define rSVBK SVBK_REG
394 #define rSMBK SVBK_REG
395
396 __REG PCM12_REG;
397 #define rPCM12 PCM12_REG
398
399 __REG PCM34_REG;
400 #define rPCM34 PCM34_REG
401
402 __REG IE_REG;
403 #define rIE IE_REG
404
405 #define IEF_HILO    0b00010000
406 #define IEF_SERIAL 0b00001000
407 #define IEF_TIMER   0b00000100
408 #define IEF_STAT    0b00000010
409 #define IEF_VBLANK  0b00000001
412 /* Square wave duty cycle */
413 #define AUDLEN_DUTY_12_5 0b00000000
414 #define AUDLEN_DUTY_25   0b01000000
415 #define AUDLEN_DUTY_50   0b10000000
416 #define AUDLEN_DUTY_75   0b11000000
417 #define AUDLEN_LENGTH(x) (x)
418
419 /* Audio envelope flags */
420 #define AUDENV_VOL(x)    ((x) << 4)
421 #define AUDENV_UP        0b00001000
422 #define AUDENV_DOWN      0b00000000
423 #define AUDENV_LENGTH(x) (x)
424
425 /* Audio trigger flags */
426 #define AUDHIGH_RESTART  0b10000000
427 #define AUDHIGH_LENGTH_ON 0b01000000
428 #define AUDHIGH_LENGTH_OFF 0b00000000
429
430 /* OAM attributes flags */
431 #define OAMF_PRI          0b10000000
432 #define OAMF_YFLIP        0b01000000
433 #define OAMF_XFLIP        0b00100000
434 #define OAMF_PAL0         0b00000000
435 #define OAMF_PAL1         0b00010000
436 #define OAMF_BANK0        0b00000000
437 #define OAMF_BANK1        0b00001000
438 #define OAMF_CGB_PAL0     0b00000000
439 #define OAMF_CGB_PAL1     0b00000001
440 #define OAMF_CGB_PAL2     0b00000010
441 #define OAMF_CGB_PAL3     0b00000011
442 #define OAMF_CGB_PAL4     0b00000100
443 #define OAMF_CGB_PAL5     0b00000101
444 #define OAMF_CGB_PAL6     0b00000110
445 #define OAMF_CGB_PAL7     0b00000111
446 #define OAMF_PALMASK      0b00000111
448 #define DEVICE_SCREEN_X_OFFSET 0
449 #define DEVICE_SCREEN_Y_OFFSET 0
450 #define DEVICE_SCREEN_WIDTH 20
451 #define DEVICE_SCREEN_HEIGHT 18
452 #define DEVICE_SCREEN_BUFFER_WIDTH 32
453 #define DEVICE_SCREEN_BUFFER_HEIGHT 32
454 #define DEVICE_SCREEN_MAP_ENTRY_SIZE 1
455 #define DEVICE_SPRITE_PX_OFFSET_X 8
456 #define DEVICE_SPRITE_PX_OFFSET_Y 16
457 #define DEVICE_WINDOW_PX_OFFSET_X 7
458 #define DEVICE_WINDOW_PX_OFFSET_Y 0
459 #define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)
460 #define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)
461 #define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)
462 #define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)
464 #endif
```

20.94 gbdk-lib/include/msx/hardware.h File Reference

```
#include <types.h>
```

Macros

- #define [__BYTES](#) extern [UBYTE](#)
- #define [__BYTE_REG](#) extern volatile [UBYTE](#)
- #define [PSG_LATCH](#) 0x80
- #define [PSG_CH0](#) 0b00000000
- #define [PSG_CH1](#) 0b00100000
- #define [PSG_CH2](#) 0b01000000
- #define [PSG_CH3](#) 0b01100000
- #define [PSG_VOLUME](#) 0b00010000
- #define [STATF_INT_VBL](#) 0b10000000
- #define [STATF_9_SPR](#) 0b01000000
- #define [STATF_SPR_COLL](#) 0b00100000
- #define [VDP_REG_MASK](#) 0b10000000
- #define [VDP_R0](#) 0b10000000
- #define [R0_DEFAULT](#) 0b00000000
- #define [R0_CB_OUTPUT](#) 0b00000000
- #define [R0_CB_INPUT](#) 0b01000000
- #define [R0_IE2_OFF](#) 0b00000000
- #define [R0_IE2](#) 0b00100000
- #define [R0_IE1_OFF](#) 0b00000000
- #define [R0_IE1](#) 0b00010000
- #define [R0_SCR_MODE1](#) 0b00000000
- #define [R0_SCR_MODE2](#) 0b00000010
- #define [R0_SCR_MODE3](#) 0b00000100
- #define [R0_ES_OFF](#) 0b00000000
- #define [R0_ES](#) 0b00000001
- #define [VDP_R1](#) 0b10000001
- #define [R1_DEFAULT](#) 0b10000000
- #define [R1_DISP_OFF](#) 0b00000000
- #define [R1_DISP_ON](#) 0b01000000
- #define [R1_IE_OFF](#) 0b00000000
- #define [R1_IE](#) 0b00100000
- #define [R1_SCR_MODE1](#) 0b00010000
- #define [R1_SCR_MODE2](#) 0b00000000
- #define [R1_SCR_MODE3](#) 0b00000000
- #define [R1_SPR_8X8](#) 0b00000000
- #define [R1_SPR_16X16](#) 0b00000010
- #define [R1_SPR_MAG](#) 0b00000001
- #define [R1_SPR_MAG_OFF](#) 0b00000000
- #define [VDP_R2](#) 0b10000010
- #define [R2_MAP_0x3800](#) 0xFF
- #define [R2_MAP_0x3000](#) 0xFD
- #define [R2_MAP_0x2800](#) 0xFB
- #define [R2_MAP_0x2000](#) 0xF9
- #define [R2_MAP_0x1800](#) 0xF7
- #define [R2_MAP_0x1000](#) 0xF5
- #define [R2_MAP_0x0800](#) 0xF3
- #define [R2_MAP_0x0000](#) 0xF1
- #define [VDP_R3](#) 0b10000011

- `#define VDP_R4 0b10000100`
- `#define VDP_R5 0b10000101`
- `#define R5_SAT_0x3F00 0xFF`
- `#define R5_SAT_MASK 0b10000001`
- `#define VDP_R6 0b10000110`
- `#define R6_BANK0 0xFB`
- `#define R6_DATA_0x0000 0xFB`
- `#define R6_BANK1 0xFF`
- `#define R6_DATA_0x2000 0xFF`
- `#define VDP_R7 0b10000111`
- `#define VDP_RBORDER 0b10000111`
- `#define R7_COLOR_MASK 0b11110000`
- `#define VDP_R8 0b10001000`
- `#define VDP_RSCX 0b10001000`
- `#define VDP_R9 0b10001001`
- `#define VDP_RSCY 0b10001001`
- `#define VDP_R10 0b10001010`
- `#define R10_INT_OFF 0xFF`
- `#define R10_INT_EVERY 0x00`
- `#define SYSTEM_PAL 0x00`
- `#define SYSTEM_NTSC 0x01`
- `#define VBK_TILES 0`
- `#define VBK_ATTRIBUTES 1`
- `#define VDP_SAT_TERM 0xD0`
- `#define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)`
- `#define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)`

Variables

- `UBYTE shadow_VDP_R0`
- `UBYTE shadow_VDP_R1`
- `UBYTE shadow_VDP_R2`
- `UBYTE shadow_VDP_R3`
- `UBYTE shadow_VDP_R4`
- `UBYTE shadow_VDP_R5`
- `UBYTE shadow_VDP_R6`
- `UBYTE shadow_VDP_R7`
- `UBYTE shadow_VDP_RBORDER`
- `UBYTE shadow_VDP_R8`
- `UBYTE shadow_VDP_RSCX`
- `UBYTE shadow_VDP_R9`
- `UBYTE shadow_VDP_RSCY`
- `UBYTE shadow_VDP_R10`
- `const UBYTE _SYSTEM`
- `volatile UBYTE VDP_ATTR_SHIFT`

20.94.1 Detailed Description

Defines that let the MSX hardware registers be accessed from C.

20.94.2 Macro Definition Documentation

20.94.2.1 `__BYTES` `#define __BYTES extern UBYTE`

20.94.2.2 `__BYTE_REG` `#define __BYTE_REG extern volatile UBYTE`

20.94.2.3 `PSG_LATCH` `#define PSG_LATCH 0x80`

20.94.2.4 `PSG_CH0` `#define PSG_CH0 0b00000000`

20.94.2.5 `PSG_CH1` `#define PSG_CH1 0b00100000`

20.94.2.6 `PSG_CH2` `#define PSG_CH2 0b01000000`

20.94.2.7 `PSG_CH3` `#define PSG_CH3 0b01100000`

20.94.2.8 `PSG_VOLUME` `#define PSG_VOLUME 0b00010000`

20.94.2.9 `STATF_INT_VBL` `#define STATF_INT_VBL 0b10000000`

20.94.2.10 `STATF_9_SPR` `#define STATF_9_SPR 0b01000000`

20.94.2.11 `STATF_SPR_COLL` `#define STATF_SPR_COLL 0b00100000`

20.94.2.12 `VDP_REG_MASK` `#define VDP_REG_MASK 0b10000000`

20.94.2.13 `VDP_R0` `#define VDP_R0 0b10000000`

20.94.2.14 `R0_DEFAULT` `#define R0_DEFAULT 0b00000000`

20.94.2.15 `R0_CB_OUTPUT` `#define R0_CB_OUTPUT 0b00000000`

20.94.2.16 `R0_CB_INPUT` `#define R0_CB_INPUT 0b01000000`

20.94.2.17 `R0_IE2_OFF` `#define R0_IE2_OFF 0b00000000`

20.94.2.18 `R0_IE2` `#define R0_IE2 0b00100000`

20.94.2.19 `R0_IE1_OFF` `#define R0_IE1_OFF 0b00000000`

20.94.2.20 R0_IE1 #define R0_IE1 0b00010000

20.94.2.21 R0_SCR_MODE1 #define R0_SCR_MODE1 0b00000000

20.94.2.22 R0_SCR_MODE2 #define R0_SCR_MODE2 0b00000010

20.94.2.23 R0_SCR_MODE3 #define R0_SCR_MODE3 0b00000100

20.94.2.24 R0_ES_OFF #define R0_ES_OFF 0b00000000

20.94.2.25 R0_ES #define R0_ES 0b00000001

20.94.2.26 VDP_R1 #define VDP_R1 0b10000001

20.94.2.27 R1_DEFAULT #define R1_DEFAULT 0b10000000

20.94.2.28 R1_DISP_OFF #define R1_DISP_OFF 0b00000000

20.94.2.29 R1_DISP_ON #define R1_DISP_ON 0b01000000

20.94.2.30 R1_IE_OFF #define R1_IE_OFF 0b00000000

20.94.2.31 R1_IE #define R1_IE 0b00100000

20.94.2.32 R1_SCR_MODE1 #define R1_SCR_MODE1 0b00010000

20.94.2.33 R1_SCR_MODE2 #define R1_SCR_MODE2 0b00000000

20.94.2.34 R1_SCR_MODE3 #define R1_SCR_MODE3 0b00000000

20.94.2.35 R1_SPR_8X8 #define R1_SPR_8X8 0b00000000

20.94.2.36 R1_SPR_16X16 #define R1_SPR_16X16 0b00000010

20.94.2.37 R1_SPR_MAG #define R1_SPR_MAG 0b00000001

20.94.2.38 R1_SPR_MAG_OFF `#define R1_SPR_MAG_OFF 0b00000000`

20.94.2.39 VDP_R2 `#define VDP_R2 0b10000010`

20.94.2.40 R2_MAP_0x3800 `#define R2_MAP_0x3800 0xFF`

20.94.2.41 R2_MAP_0x3000 `#define R2_MAP_0x3000 0xFD`

20.94.2.42 R2_MAP_0x2800 `#define R2_MAP_0x2800 0xFB`

20.94.2.43 R2_MAP_0x2000 `#define R2_MAP_0x2000 0xF9`

20.94.2.44 R2_MAP_0x1800 `#define R2_MAP_0x1800 0xF7`

20.94.2.45 R2_MAP_0x1000 `#define R2_MAP_0x1000 0xF5`

20.94.2.46 R2_MAP_0x0800 `#define R2_MAP_0x0800 0xF3`

20.94.2.47 R2_MAP_0x0000 `#define R2_MAP_0x0000 0xF1`

20.94.2.48 VDP_R3 `#define VDP_R3 0b10000011`

20.94.2.49 VDP_R4 `#define VDP_R4 0b10000100`

20.94.2.50 VDP_R5 `#define VDP_R5 0b10000101`

20.94.2.51 R5_SAT_0x3F00 `#define R5_SAT_0x3F00 0xFF`

20.94.2.52 R5_SAT_MASK `#define R5_SAT_MASK 0b10000001`

20.94.2.53 VDP_R6 `#define VDP_R6 0b10000110`

20.94.2.54 R6_BANK0 `#define R6_BANK0 0xFB`

20.94.2.55 R6_DATA_0x0000 `#define R6_DATA_0x0000 0xFB`

20.94.2.56 R6_BANK1 `#define R6_BANK1 0xFF`

20.94.2.57 R6_DATA_0x2000 `#define R6_DATA_0x2000 0xFF`

20.94.2.58 VDP_R7 `#define VDP_R7 0b10000111`

20.94.2.59 VDP_RBORDER `#define VDP_RBORDER 0b10000111`

20.94.2.60 R7_COLOR_MASK `#define R7_COLOR_MASK 0b11110000`

20.94.2.61 VDP_R8 `#define VDP_R8 0b10001000`

20.94.2.62 VDP_RSCX `#define VDP_RSCX 0b10001000`

20.94.2.63 VDP_R9 `#define VDP_R9 0b10001001`

20.94.2.64 VDP_RSCY `#define VDP_RSCY 0b10001001`

20.94.2.65 VDP_R10 `#define VDP_R10 0b10001010`

20.94.2.66 R10_INT_OFF `#define R10_INT_OFF 0xFF`

20.94.2.67 R10_INT EVERY `#define R10_INT EVERY 0x00`

20.94.2.68 SYSTEM_PAL `#define SYSTEM_PAL 0x00`

20.94.2.69 SYSTEM_NTSC `#define SYSTEM_NTSC 0x01`

20.94.2.70 VBK_TILES `#define VBK_TILES 0`

20.94.2.71 VBK_ATTRIBUTES `#define VBK_ATTRIBUTES 1`

20.94.2.72 VDP_SAT_TERM `#define VDP_SAT_TERM 0xD0`

20.94.2.73 DEVICE_SCREEN_PX_WIDTH `#define DEVICE_SCREEN_PX_WIDTH (DEVICE\_SCREEN\_WIDTH * 8)`

20.94.2.74 **DEVICE_SCREEN_PX_HEIGHT** `#define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)`

20.94.3 Variable Documentation

20.94.3.1 **shadow_VDP_R0** `UBYTE shadow_VDP_R0 [extern]`

20.94.3.2 **shadow_VDP_R1** `UBYTE shadow_VDP_R1 [extern]`

20.94.3.3 **shadow_VDP_R2** `UBYTE shadow_VDP_R2 [extern]`

20.94.3.4 **shadow_VDP_R3** `UBYTE shadow_VDP_R3 [extern]`

20.94.3.5 **shadow_VDP_R4** `UBYTE shadow_VDP_R4 [extern]`

20.94.3.6 **shadow_VDP_R5** `UBYTE shadow_VDP_R5 [extern]`

20.94.3.7 **shadow_VDP_R6** `UBYTE shadow_VDP_R6 [extern]`

20.94.3.8 **shadow_VDP_R7** `UBYTE shadow_VDP_R7 [extern]`

20.94.3.9 **shadow_VDP_RBORDER** `UBYTE shadow_VDP_RBORDER [extern]`

20.94.3.10 **shadow_VDP_R8** `UBYTE shadow_VDP_R8 [extern]`

20.94.3.11 **shadow_VDP_RSCX** `UBYTE shadow_VDP_RSCX [extern]`

20.94.3.12 **shadow_VDP_R9** `UBYTE shadow_VDP_R9 [extern]`

20.94.3.13 **shadow_VDP_RSCY** `UBYTE shadow_VDP_RSCY [extern]`

20.94.3.14 **shadow_VDP_R10** `UBYTE shadow_VDP_R10 [extern]`

20.94.3.15 **_SYSTEM** `const UBYTE _SYSTEM [extern]`

20.94.3.16 **VDP_ATTR_SHIFT** `volatile UBYTE VDP_ATTR_SHIFT [extern]`

20.95 hardware.h

[Go to the documentation of this file.](#)

```

1
5 #ifndef _HARDWARE_H
6 #define _HARDWARE_H
7
8 #include <types.h>
9
10 #define __BYTES extern UBYTE
11 #define __BYTE_REG extern volatile UBYTE
12
13 static volatile SFR AT(0x7E) VCOUNTER;
14
15 static volatile SFR AT(0x7F) PSG;
16
17 #define PSG_LATCH      0x80
18
19 #define PSG_CH0        0b00000000
20 #define PSG_CH1        0b00100000
21 #define PSG_CH2        0b01000000
22 #define PSG_CH3        0b01100000
23
24 #define PSG_VOLUME     0b00010000
25
26 static volatile SFR AT(0x7F) HCOUNT;
27
28 static volatile SFR AT(0x98) VDP_DATA;
29 static volatile SFR AT(0x99) VDP_CMD;
30 static volatile SFR AT(0x99) VDP_STATUS;
31
32 #define STATF_INT_VBL   0b10000000
33 #define STATF_9_SPR     0b01000000
34 #define STATF_SPR_COLL  0b00100000
35
36 #define VDP_REG_MASK    0b10000000
37 #define VDP_R0          0b10000000
38 extern UBYTE shadow_VDP_R0;
39
40 #define R0_DEFAULT      0b00000000
41 #define R0_CB_OUTPUT    0b00000000
42 #define R0_CB_INPUT     0b01000000
43 #define R0_IE2_OFF      0b00000000
44 #define R0_IE2          0b00100000
45 #define R0_IE1_OFF      0b00000000
46 #define R0_IE1          0b00010000
47 #define R0_SCR_MODE1    0b00000000
48 #define R0_SCR_MODE2    0b00000010
49 #define R0_SCR_MODE3    0b00000100
50 #define R0_ES_OFF       0b00000000
51 #define R0_ES           0b00000001
52
53 #define VDP_R1          0b10000001
54 extern UBYTE shadow_VDP_R1;
55
56 #define R1_DEFAULT      0b10000000
57 #define R1_DISP_OFF     0b00000000
58 #define R1_DISP_ON      0b01000000
59 #define R1_IE_OFF       0b00000000
60 #define R1_IE           0b00100000
61 #define R1_SCR_MODE1    0b00010000
62 #define R1_SCR_MODE2    0b00000000
63 #define R1_SCR_MODE3    0b00000000
64 #define R1_SPR_8X8      0b00000000
65 #define R1_SPR_16X16    0b00000010
66 #define R1_SPR_MAG      0b00000001
67 #define R1_SPR_MAG_OFF  0b00000000
68
69 #define VDP_R2          0b10000010
70 extern UBYTE shadow_VDP_R2;
71
72 #define R2_MAP_0x3800    0xFF
73 #define R2_MAP_0x3000    0xFD
74 #define R2_MAP_0x2800    0xFB
75 #define R2_MAP_0x2000    0xF9
76 #define R2_MAP_0x1800    0xF7
77 #define R2_MAP_0x1000    0xF5
78 #define R2_MAP_0x0800    0xF3
79 #define R2_MAP_0x0000    0xF1
80
81 #define VDP_R3          0b10000011
82 extern UBYTE shadow_VDP_R3;
83 #define VDP_R4          0b10000100
84 extern UBYTE shadow_VDP_R4;
85 #define VDP_R5          0b10000101
86 extern UBYTE shadow_VDP_R5;
87

```

```

88 #define R5_SAT_0x3F00  0xFF
89 #define R5_SAT_MASK    0b10000001
90
91 #define VDP_R6          0b10000110
92 extern UBYTE shadow_VDP_R6;
93
94 #define R6_BANK0        0xFB
95 #define R6_DATA_0x0000  0xFB
96 #define R6_BANK1        0xFF
97 #define R6_DATA_0x2000  0xFF
98
99 #define VDP_R7          0b10000111
100 extern UBYTE shadow_VDP_R7;
101 #define VDP_RBORDER     0b10000111
102 extern UBYTE shadow_VDP_RBORDER;
103
104 #define R7_COLOR_MASK   0b11110000
105
106 #define VDP_R8          0b10001000
107 extern UBYTE shadow_VDP_R8;
108 #define VDP_RSCX        0b10001000
109 extern UBYTE shadow_VDP_RSCX;
110
111 #define VDP_R9          0b10001001
112 extern UBYTE shadow_VDP_R9;
113 #define VDP_RSCY        0b10001001
114 extern UBYTE shadow_VDP_RSCY;
115
116 #define VDP_R10         0b10001010
117 extern UBYTE shadow_VDP_R10;
118
119 #define R10_INT_OFF     0xFF
120 #define R10_INT_EVERY   0x00
121
122 static volatile SFR AT(0xF0) FMADDRESS;
123 static volatile SFR AT(0xF1) FMDATA;
124 static volatile SFR AT(0xF2) AUDIOCTRL;
125
126 static volatile SFR AT(0xfc) MAP_FRAME0;
127 static volatile SFR AT(0xfd) MAP_FRAME1;
128 static volatile SFR AT(0xfe) MAP_FRAME2;
129 static volatile SFR AT(0xff) MAP_FRAME3;
130
131 extern const UBYTE _SYSTEM;
132
133 #define SYSTEM_PAL      0x00
134 #define SYSTEM_NTSC     0x01
135
136 extern volatile UBYTE VDP_ATTR_SHIFT;
137
138 #define VBK_TILES        0
139 #define VBK_ATTRIBUTES   1
140
141 #define VDP_SAT_TERM     0xD0
142
143 #if defined(__TARGET_msxdos)
144 #define DEVICE_SCREEN_X_OFFSET 0
145 #define DEVICE_SCREEN_Y_OFFSET 0
146 #define DEVICE_SCREEN_WIDTH 32
147 #define DEVICE_SCREEN_HEIGHT 24
148 #define DEVICE_SCREEN_BUFFER_WIDTH 32
149 #define DEVICE_SCREEN_BUFFER_HEIGHT 28
150 #define DEVICE_SCREEN_MAP_ENTRY_SIZE 2
151 #define DEVICE_SPRITE_PX_OFFSET_X 0
152 #define DEVICE_SPRITE_PX_OFFSET_Y -1
153 #define DEVICE_WINDOW_PX_OFFSET_X 0
154 #define DEVICE_WINDOW_PX_OFFSET_Y 0
155 #else
156 #error Unrecognized port
157 #endif
158 #define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)
159 #define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)
160
161 #endif

```

20.96 gbdk-lib/include/nes/hardware.h File Reference

#include <types.h>

Macros

- #define [__SHADOW_REG](#) extern volatile uint8_t

- `#define __REG(addr) volatile __at (addr) uint8_t`
- `#define PPUCTRL_NMI 0b10000000`
- `#define PPUCTRL_SPR_8X8 0b00000000`
- `#define PPUCTRL_SPR_8X16 0b00100000`
- `#define PPUCTRL_BG_CHR 0b00010000`
- `#define PPUCTRL_SPR_CHR 0b00001000`
- `#define PPUCTRL_INC32 0b00000100`
- `#define PPUMASK_BLUE 0b10000000`
- `#define PPUMASK_RED 0b01000000`
- `#define PPUMASK_GREEN 0b00100000`
- `#define PPUMASK_SHOW_SPR 0b00010000`
- `#define PPUMASK_SHOW_BG 0b00001000`
- `#define PPUMASK_SHOW_SPR_LC 0b00000100`
- `#define PPUMASK_SHOW_BG_LC 0b00000010`
- `#define PPUMASK_MONOCHROME 0b00000001`
- `#define DEVICE_SCREEN_X_OFFSET 0`
- `#define DEVICE_SCREEN_Y_OFFSET 0`
- `#define DEVICE_SCREEN_WIDTH 32`
- `#define DEVICE_SCREEN_HEIGHT 30`
- `#define DEVICE_SCREEN_BUFFER_WIDTH 32`
- `#define DEVICE_SCREEN_BUFFER_HEIGHT 30`
- `#define DEVICE_SCREEN_MAP_ENTRY_SIZE 1`
- `#define DEVICE_SPRITE_PX_OFFSET_X 0`
- `#define DEVICE_SPRITE_PX_OFFSET_Y -1`
- `#define DEVICE_WINDOW_PX_OFFSET_X 0`
- `#define DEVICE_WINDOW_PX_OFFSET_Y 0`
- `#define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)`
- `#define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)`
- `#define SCY_REG bkg_scroll_y`
- `#define rSCY SCY_REG`
- `#define SCX_REG bkg_scroll_x`
- `#define rSCX SCX_REG`
- `#define LY_REG _lcd_scanline`
- `#define rLY LY_REG`
- `#define LYC_REG _lcd_scanline`
- `#define rLYC LYC_REG`

Typedefs

- `typedef uint8_t scroll_x_t`
- `typedef uint8_t scroll_y_t`

Functions

- `__REG (0x2000) PPUCTRL`
- `__REG (0x2001) PPUMASK`
- `__REG (0x2002) PPUSTATUS`
- `__REG (0x2003) OAMADDR`
- `__REG (0x2004) OAMDATA`
- `__REG (0x2005) PPUSCROLL`
- `__REG (0x2006) PPUADDR`
- `__REG (0x2007) PPUDATA`
- `__REG (0x4014) OAMDMA`

Variables

- [__SHADOW_REG shadow_PPUCTRL](#)
- [__SHADOW_REG shadow_PPUMASK](#)
- [__SHADOW_REG bkg_scroll_x](#)
- [__SHADOW_REG bkg_scroll_y](#)
- [__SHADOW_REG _lcd_scanline](#)
- volatile [UBYTE TIMA_REG](#)
- volatile [UBYTE TMA_REG](#)
- volatile [UBYTE TAC_REG](#)

20.96.1 Detailed Description

Defines that let the NES hardware registers be accessed from C.

20.96.2 Macro Definition Documentation

20.96.2.1 [__SHADOW_REG](#) `#define __SHADOW_REG extern volatile uint8_t`

20.96.2.2 [__REG](#) `#define __REG(
addr) volatile __at (addr) uint8_t`

20.96.2.3 [PPUCTRL_NMI](#) `#define PPUCTRL_NMI 0b10000000`

20.96.2.4 [PPUCTRL_SPR_8X8](#) `#define PPUCTRL_SPR_8X8 0b00000000`

20.96.2.5 [PPUCTRL_SPR_8X16](#) `#define PPUCTRL_SPR_8X16 0b00100000`

20.96.2.6 [PPUCTRL_BG_CHR](#) `#define PPUCTRL_BG_CHR 0b00010000`

20.96.2.7 [PPUCTRL_SPR_CHR](#) `#define PPUCTRL_SPR_CHR 0b00001000`

20.96.2.8 [PPUCTRL_INC32](#) `#define PPUCTRL_INC32 0b00000100`

20.96.2.9 [PPUMASK_BLUE](#) `#define PPUMASK_BLUE 0b10000000`

20.96.2.10 [PPUMASK_RED](#) `#define PPUMASK_RED 0b01000000`

20.96.2.11 [PPUMASK_GREEN](#) `#define PPUMASK_GREEN 0b00100000`

20.96.2.12 [PPUMASK_SHOW_SPR](#) `#define PPUMASK_SHOW_SPR 0b00010000`

20.96.2.13 PPUMASK_SHOW_BG `#define PPUMASK_SHOW_BG 0b00001000`

20.96.2.14 PPUMASK_SHOW_SPR_LC `#define PPUMASK_SHOW_SPR_LC 0b00000100`

20.96.2.15 PPUMASK_SHOW_BG_LC `#define PPUMASK_SHOW_BG_LC 0b00000010`

20.96.2.16 PPUMASK_MONOCHROME `#define PPUMASK_MONOCHROME 0b00000001`

20.96.2.17 DEVICE_SCREEN_X_OFFSET `#define DEVICE_SCREEN_X_OFFSET 0`

20.96.2.18 DEVICE_SCREEN_Y_OFFSET `#define DEVICE_SCREEN_Y_OFFSET 0`

20.96.2.19 DEVICE_SCREEN_WIDTH `#define DEVICE_SCREEN_WIDTH 32`

20.96.2.20 DEVICE_SCREEN_HEIGHT `#define DEVICE_SCREEN_HEIGHT 30`

20.96.2.21 DEVICE_SCREEN_BUFFER_WIDTH `#define DEVICE_SCREEN_BUFFER_WIDTH 32`

20.96.2.22 DEVICE_SCREEN_BUFFER_HEIGHT `#define DEVICE_SCREEN_BUFFER_HEIGHT 30`

20.96.2.23 DEVICE_SCREEN_MAP_ENTRY_SIZE `#define DEVICE_SCREEN_MAP_ENTRY_SIZE 1`

20.96.2.24 DEVICE_SPRITE_PX_OFFSET_X `#define DEVICE_SPRITE_PX_OFFSET_X 0`

20.96.2.25 DEVICE_SPRITE_PX_OFFSET_Y `#define DEVICE_SPRITE_PX_OFFSET_Y -1`

20.96.2.26 DEVICE_WINDOW_PX_OFFSET_X `#define DEVICE_WINDOW_PX_OFFSET_X 0`

20.96.2.27 DEVICE_WINDOW_PX_OFFSET_Y `#define DEVICE_WINDOW_PX_OFFSET_Y 0`

20.96.2.28 DEVICE_SCREEN_PX_WIDTH `#define DEVICE_SCREEN_PX_WIDTH (DEVICE\_SCREEN\_WIDTH * 8)`

20.96.2.29 DEVICE_SCREEN_PX_HEIGHT `#define DEVICE_SCREEN_PX_HEIGHT (DEVICE\_SCREEN\_HEIGHT * 8)`

20.96.2.30 SCY_REG #define SCY_REG `bkg_scroll_y`
Scroll Y

20.96.2.31 rSCY #define rSCY `SCY_REG`

20.96.2.32 SCX_REG #define SCX_REG `bkg_scroll_x`
Scroll X

20.96.2.33 rSCX #define rSCX `SCX_REG`

20.96.2.34 LY_REG #define LY_REG `_lcd_scanline`
LCDC Y-coordinate

20.96.2.35 rLY #define rLY `LY_REG`

20.96.2.36 LYC_REG #define LYC_REG `_lcd_scanline`
LY compare

20.96.2.37 rLYC #define rLYC `LYC_REG`

20.96.3 Typedef Documentation

20.96.3.1 scroll_x_t typedef `uint8_t scroll_x_t`

20.96.3.2 scroll_y_t typedef `uint8_t scroll_y_t`

20.96.4 Function Documentation

20.96.4.1 __REG() [1/9] __REG (
0x2000)

20.96.4.2 __REG() [2/9] __REG (
0x2001)

20.96.4.3 __REG() [3/9] __REG (
0x2002)

20.96.4.4 __REG() [4/9] __REG (
0x2003)

20.96.4.5 __REG() [5/9] __REG (
0x2004)

20.96.4.6 `__REG()` [6/9] `__REG` (
0x2005)

20.96.4.7 `__REG()` [7/9] `__REG` (
0x2006)

20.96.4.8 `__REG()` [8/9] `__REG` (
0x2007)

20.96.4.9 `__REG()` [9/9] `__REG` (
0x4014)

20.96.5 Variable Documentation

20.96.5.1 `shadow_PPUCTRL` `__SHADOW_REG` `shadow_PPUCTRL`

20.96.5.2 `shadow_PPUMASK` `__SHADOW_REG` `shadow_PPUMASK`

20.96.5.3 `bkg_scroll_x` `__SHADOW_REG` `bkg_scroll_x`

20.96.5.4 `bkg_scroll_y` `__SHADOW_REG` `bkg_scroll_y`

20.96.5.5 `_lcd_scanline` `__SHADOW_REG` `_lcd_scanline`

20.96.5.6 `TIMA_REG` `volatile UBYTE` `TIMA_REG` [extern]
Timer counter

20.96.5.7 `TMA_REG` `volatile UBYTE` `TMA_REG` [extern]
Timer modulo

20.96.5.8 `TAC_REG` `volatile UBYTE` `TAC_REG` [extern]
Timer control

20.97 hardware.h

[Go to the documentation of this file.](#)

```
1
5 #ifndef _HARDWARE_H
6 #define _HARDWARE_H
7
8 #include <types.h>
9
10 #define __SHADOW_REG extern volatile uint8_t
11 #define __REG(addr) volatile __at (addr) uint8_t
12
13 __REG(0x2000) PPUCTRL;
14 #define PPUCTRL_NMI          0b10000000
15 #define PPUCTRL_SPR_8X8     0b00000000
16 #define PPUCTRL_SPR_8X16    0b00100000
17 #define PPUCTRL_BG_CHR      0b00010000
```

```

18 #define PPUCTRL_SPR_CHR      0b00001000
19 #define PPUCTRL_INC32        0b00000100
20 __SHADOW_REG shadow_PPUCTRL;
21
22 __REG(0x2001) PPUMASK;
23 #define PPUMASK_BLUE          0b10000000
24 #define PPUMASK_RED           0b01000000
25 #define PPUMASK_GREEN         0b00100000
26 #define PPUMASK_SHOW_SPR     0b00010000
27 #define PPUMASK_SHOW_BG      0b00001000
28 #define PPUMASK_SHOW_SPR_LC  0b00000100
29 #define PPUMASK_SHOW_BG_LC   0b00000010
30 #define PPUMASK_MONOCHROME    0b00000001
31 __SHADOW_REG shadow_PPUMASK;
32
33 __REG(0x2002) PPUSTATUS;
34 __REG(0x2003) OAMADDR;
35 __REG(0x2004) OAMDATA;
36 __REG(0x2005) PPUSCROLL;
37 __REG(0x2006) PPUADDR;
38 __REG(0x2007) PPUDATA;
39 __REG(0x4014) OAMDMA;
40
41 #define DEVICE_SCREEN_X_OFFSET 0
42 #define DEVICE_SCREEN_Y_OFFSET 0
43 #define DEVICE_SCREEN_WIDTH 32
44 #define DEVICE_SCREEN_HEIGHT 30
45
46 #if defined(NES_TILEMAP_F)
47 // Full tilemap
48 #define DEVICE_SCREEN_BUFFER_WIDTH 64
49 #define DEVICE_SCREEN_BUFFER_HEIGHT 60
50 typedef uint16_t scroll_x_t;
51 typedef uint16_t scroll_y_t;
52 #elif defined(NES_TILEMAP_H)
53 // Horizontally arranged tilemap
54 #define DEVICE_SCREEN_BUFFER_WIDTH 64
55 #define DEVICE_SCREEN_BUFFER_HEIGHT 30
56 typedef uint16_t scroll_x_t;
57 typedef uint8_t scroll_y_t;
58 #elif defined(NES_TILEMAP_V)
59 // Vertically arranged tilemap
60 #define DEVICE_SCREEN_BUFFER_WIDTH 32
61 #define DEVICE_SCREEN_BUFFER_HEIGHT 60
62 typedef uint8_t scroll_x_t;
63 typedef uint16_t scroll_y_t;
64 #else
65 // Single-screen tilemap
66 #define DEVICE_SCREEN_BUFFER_WIDTH 32
67 #define DEVICE_SCREEN_BUFFER_HEIGHT 30
68 typedef uint8_t scroll_x_t;
69 typedef uint8_t scroll_y_t;
70 #endif
71
72 #define DEVICE_SCREEN_MAP_ENTRY_SIZE 1
73 #define DEVICE_SPRITE_PX_OFFSET_X 0
74 #define DEVICE_SPRITE_PX_OFFSET_Y -1
75 #define DEVICE_WINDOW_PX_OFFSET_X 0
76 #define DEVICE_WINDOW_PX_OFFSET_Y 0
77 #define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)
78 #define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)
79
80 // Scrolling coordinates (will be written to PPUSCROLL at end-of-vblank by NMI handler)
81 __SHADOW_REG bkg_scroll_x;
82 __SHADOW_REG bkg_scroll_y;
83 // LCD scanline - a software-driven version of GB's increasing 'LY' scanline counter
84 __SHADOW_REG lcd_scanline;
85
86 extern volatile UBYTE TIMA_REG;
87 extern volatile UBYTE TMA_REG;
88 extern volatile UBYTE TAC_REG;
89
90 // Compatibility defines for GB LY / LYC registers, to allow easier LCD ISR porting
91 #define SCY_REG bkg_scroll_y
92 #define rSCY SCY_REG
93 #define SCX_REG bkg_scroll_x
94 #define rSCX SCX_REG
95 #define LY_REG lcd_scanline
96 #define rLY LY_REG
97 #define LYC_REG lcd_scanline
98 #define rLYC LYC_REG
99
100 #endif

```

20.98 gbdk-lib/include/sms/hardware.h File Reference

```
#include <types.h>
```

Macros

- #define [__BYTES](#) extern [UBYTE](#)
- #define [__BYTE_REG](#) extern volatile [UBYTE](#)
- #define [GGSTATE_STT](#) 0b10000000
- #define [GGSTATE_NJAP](#) 0b01000000
- #define [GGSTATE_NNTS](#) 0b00100000
- #define [GGEXT_NINT](#) 0b10000000
- #define [SIOCTL_TXFL](#) 0b00000001
- #define [SIOCTL_RXRD](#) 0b00000010
- #define [SIOCTL_FRER](#) 0b00000100
- #define [SIOCTL_INT](#) 0b00001000
- #define [SIOCTL_TON](#) 0b00010000
- #define [SIOCTL_RON](#) 0b00100000
- #define [SIOCTL_BS0](#) 0b01000000
- #define [SIOCTL_BS1](#) 0b10000000
- #define [SOUNDPAN_TN1R](#) 0b00000001
- #define [SOUNDPAN_TN2R](#) 0b00000010
- #define [SOUNDPAN_TN3R](#) 0b00000100
- #define [SOUNDPAN_NOSR](#) 0b00001000
- #define [SOUNDPAN_TN1L](#) 0b00010000
- #define [SOUNDPAN_TN2L](#) 0b00100000
- #define [SOUNDPAN_TN3L](#) 0b01000000
- #define [SOUNDPAN_NOSL](#) 0b10000000
- #define [MEMCTL_JOYON](#) 0b00000000
- #define [MEMCTL_JOYOFF](#) 0b00000100
- #define [MEMCTL_BASEON](#) 0b00000000
- #define [MEMCTL_BASEOFF](#) 0b00001000
- #define [MEMCTL_RAMON](#) 0b00000000
- #define [MEMCTL_RAMOFF](#) 0b00010000
- #define [MEMCTL_CROMON](#) 0b00000000
- #define [MEMCTL_CROMOFF](#) 0b00100000
- #define [MEMCTL_ROMON](#) 0b00000000
- #define [MEMCTL_ROMOFF](#) 0b01000000
- #define [MEMCTL_EXTON](#) 0b00000000
- #define [MEMCTL_EXTOFF](#) 0b10000000
- #define [JOY_P1_TR_DIR_IN](#) 0b00000001
- #define [JOY_P1_TR_DIR_OUT](#) 0b00000000
- #define [JOY_P1_TH_DIR_IN](#) 0b00000010
- #define [GUN_P1_LATCH](#) [JOY_P1_TH_DIR_IN](#)
- #define [JOY_P1_TH_DIR_OUT](#) 0b00000000
- #define [JOY_P2_TR_DIR_IN](#) 0b00000100
- #define [JOY_P2_TR_DIR_OUT](#) 0b00000000
- #define [JOY_P2_TH_DIR_IN](#) 0b00001000
- #define [GUN_P2_LATCH](#) [JOY_P2_TH_DIR_IN](#)
- #define [JOY_P2_TH_DIR_OUT](#) 0b00000000
- #define [JOY_P1_TR_OUT_HI](#) 0b00010000
- #define [JOY_P1_TR_OUT_LO](#) 0b00000000
- #define [JOY_P1_TH_OUT_HI](#) 0b00100000
- #define [JOY_P1_TH_OUT_LO](#) 0b00000000

- `#define JOY_P2_TR_OUT_HI 0b01000000`
- `#define JOY_P2_TR_OUT_LO 0b00000000`
- `#define JOY_P2_TH_OUT_HI 0b10000000`
- `#define JOY_P2_TH_OUT_LO 0b00000000`
- `#define JOY_TH_HI (JOY_P1_TR_DIR_IN | JOY_P1_TH_DIR_OUT | JOY_P2_TR_DIR_IN | JOY_P2_TH_DIR_OUT | JOY_P1_TR_OUT_HI | JOY_P1_TH_OUT_HI | JOY_P2_TR_OUT_HI | JOY_P2_TH_OUT_HI)`
- `#define JOY_TH_LO (JOY_P1_TR_DIR_IN | JOY_P1_TH_DIR_OUT | JOY_P2_TR_DIR_IN | JOY_P2_TH_DIR_OUT | JOY_P1_TR_OUT_HI | JOY_P1_TH_OUT_LO | JOY_P2_TR_OUT_HI | JOY_P2_TH_OUT_LO)`
- `#define PSG_LATCH 0b10000000`
- `#define PSG_CH0 0b00000000`
- `#define PSG_CH1 0b00100000`
- `#define PSG_CH2 0b01000000`
- `#define PSG_CH3 0b01100000`
- `#define PSG_VOLUME 0b00010000`
- `#define STATF_INT_VBL 0b10000000`
- `#define STATF_9_SPR 0b01000000`
- `#define STATF_SPR_COLL 0b00100000`
- `#define VDP_REG_MASK 0b10000000`
- `#define VDP_R0 0b10000000`
- `#define R0_VSCRL 0b00000000`
- `#define R0_VSCRL_INH 0b10000000`
- `#define R0_HSCRL 0b00000000`
- `#define R0_HSCRL_INH 0b01000000`
- `#define R0_NO_LCB 0b00000000`
- `#define R0_LCB 0b00100000`
- `#define R0_IE1_OFF 0b00000000`
- `#define R0_IE1 0b00010000`
- `#define R0_SS_OFF 0b00000000`
- `#define R0_SS 0b00001000`
- `#define R0_DEFAULT 0b00000110`
- `#define R0_ES_OFF 0b00000000`
- `#define R0_ES 0b00000001`
- `#define VDP_R1 0b10000001`
- `#define R1_DEFAULT 0b10000000`
- `#define R1_DISP_OFF 0b00000000`
- `#define R1_DISP_ON 0b01000000`
- `#define R1_IE_OFF 0b00000000`
- `#define R1_IE 0b00100000`
- `#define R1_SPR_8X8 0b00000000`
- `#define R1_SPR_8X16 0b00000010`
- `#define VDP_R2 0b10000010`
- `#define R2_MAP_0x3800 0xFF`
- `#define R2_MAP_0x3000 0xFD`
- `#define R2_MAP_0x2800 0xFB`
- `#define R2_MAP_0x2000 0xF9`
- `#define R2_MAP_0x1800 0xF7`
- `#define R2_MAP_0x1000 0xF5`
- `#define R2_MAP_0x0800 0xF3`
- `#define R2_MAP_0x0000 0xF1`
- `#define VDP_R3 0b10000011`
- `#define VDP_R4 0b10000100`
- `#define VDP_R5 0b10000101`
- `#define R5_SAT_0x3F00 0xFF`
- `#define R5_SAT_0x1F00 0xBF`
- `#define R5_SAT_MASK 0b10000001`

- #define VDP_R6 0b10000110
- #define R6_BANK0 0xFB
- #define R6_DATA_0x0000 0xFB
- #define R6_BANK1 0xFF
- #define R6_DATA_0x2000 0xFF
- #define VDP_R7 0b10000111
- #define VDP_RBORDER 0b10000111
- #define R7_COLOR_MASK 0b11110000
- #define VDP_R8 0b10001000
- #define VDP_RSCX 0b10001000
- #define VDP_R9 0b10001001
- #define VDP_RSCY 0b10001001
- #define VDP_R10 0b10001010
- #define R10_INT_OFF 0xFF
- #define R10_INT_EVERY 0x00
- #define JOY_P1_UP 0b00000001
- #define JOY_P1_MD_Z JOY_P1_UP
- #define JOY_P1_DOWN 0b00000010
- #define JOY_P1_MD_Y JOY_P1_DOWN
- #define JOY_P1_LEFT 0b00000100
- #define JOY_P1_MD_X JOY_P1_LEFT
- #define JOY_P1_RIGHT 0b00001000
- #define JOY_P1_MD_MODE JOY_P1_RIGHT
- #define JOY_P1_SW1 0b00010000
- #define JOY_P1_TRIGGER JOY_P1_SW1
- #define JOY_P1_MD_A JOY_P1_SW1
- #define JOY_P1_SW2 0b00100000
- #define JOY_P1_MD_START JOY_P1_SW2
- #define JOY_P2_UP 0b01000000
- #define JOY_P2_MD_Z JOY_P2_UP
- #define JOY_P2_DOWN 0b10000000
- #define JOY_P2_MD_Y JOY_P2_DOWN
- #define JOY_P2_LEFT 0b00000001
- #define JOY_P2_MD_X JOY_P2_LEFT
- #define JOY_P2_RIGHT 0b00000010
- #define JOY_P2_MD_MODE JOY_P2_RIGHT
- #define JOY_P2_SW1 0b00000100
- #define JOY_P2_TRIGGER JOY_P2_SW1
- #define JOY_P2_MD_A JOY_P2_SW1
- #define JOY_P2_SW2 0b00001000
- #define JOY_P2_MD_START JOY_P2_SW2
- #define JOY_RESET 0b00010000
- #define JOY_P1_LIGHT 0b01000000
- #define JOY_P2_LIGHT 0b10000000
- #define RAMCTL_BANK 0b00000100
- #define RAMCTL_ROM 0b00000000
- #define RAMCTL_RAM 0b00001000
- #define RAMCTL_RO 0b00010000
- #define RAMCTL_PROT 0b10000000
- #define VBK_TILES 0
- #define VBK_ATTRIBUTES 1
- #define VDP_SAT_TERM 0xD0
- #define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)
- #define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)

Variables

- [UBYTE shadow_VDP_R0](#)
- [UBYTE shadow_VDP_R1](#)
- [UBYTE shadow_VDP_R2](#)
- [UBYTE shadow_VDP_R3](#)
- [UBYTE shadow_VDP_R4](#)
- [UBYTE shadow_VDP_R5](#)
- [UBYTE shadow_VDP_R6](#)
- [UBYTE shadow_VDP_R7](#)
- [UBYTE shadow_VDP_RBORDER](#)
- [UBYTE shadow_VDP_R8](#)
- [UBYTE shadow_VDP_RSCX](#)
- [UBYTE shadow_VDP_R9](#)
- [UBYTE shadow_VDP_RSCY](#)
- [UBYTE shadow_VDP_R10](#)
- volatile [UBYTE TIMA_REG](#)
- volatile [UBYTE TMA_REG](#)
- volatile [UBYTE TAC_REG](#)
- volatile [UBYTE VDP_ATTR_SHIFT](#)

20.98.1 Detailed Description

Defines that let the SMS/GG hardware registers be accessed from C.

20.98.2 Macro Definition Documentation

20.98.2.1 `__BYTES` `#define __BYTES extern UBYTE`

20.98.2.2 `__BYTE_REG` `#define __BYTE_REG extern volatile UBYTE`

20.98.2.3 `GGSTATE_STT` `#define GGSTATE_STT 0b10000000`

20.98.2.4 `GGSTATE_NJAP` `#define GGSTATE_NJAP 0b01000000`

20.98.2.5 `GGSTATE_NNTS` `#define GGSTATE_NNTS 0b00100000`

20.98.2.6 `GGEXT_NINT` `#define GGEXT_NINT 0b10000000`

20.98.2.7 `SIOCTL_TXFL` `#define SIOCTL_TXFL 0b00000001`

20.98.2.8 `SIOCTL_RXRD` `#define SIOCTL_RXRD 0b00000010`

20.98.2.9 `SIOCTL_FRER` `#define SIOCTL_FRER 0b00000100`

20.98.2.10 SIOCTL_INT #define SIOCTL_INT 0b00001000

20.98.2.11 SIOCTL_TON #define SIOCTL_TON 0b00010000

20.98.2.12 SIOCTL_RON #define SIOCTL_RON 0b00100000

20.98.2.13 SIOCTL_BS0 #define SIOCTL_BS0 0b01000000

20.98.2.14 SIOCTL_BS1 #define SIOCTL_BS1 0b10000000

20.98.2.15 SOUNDPAN_TN1R #define SOUNDPAN_TN1R 0b00000001

20.98.2.16 SOUNDPAN_TN2R #define SOUNDPAN_TN2R 0b00000010

20.98.2.17 SOUNDPAN_TN3R #define SOUNDPAN_TN3R 0b00000100

20.98.2.18 SOUNDPAN_NOSR #define SOUNDPAN_NOSR 0b00001000

20.98.2.19 SOUNDPAN_TN1L #define SOUNDPAN_TN1L 0b00010000

20.98.2.20 SOUNDPAN_TN2L #define SOUNDPAN_TN2L 0b00100000

20.98.2.21 SOUNDPAN_TN3L #define SOUNDPAN_TN3L 0b01000000

20.98.2.22 SOUNDPAN_NOSL #define SOUNDPAN_NOSL 0b10000000

20.98.2.23 MEMCTL_JOYON #define MEMCTL_JOYON 0b00000000

20.98.2.24 MEMCTL_JOYOFF #define MEMCTL_JOYOFF 0b00000100

20.98.2.25 MEMCTL_BASEON #define MEMCTL_BASEON 0b00000000

20.98.2.26 MEMCTL_BASEOFF #define MEMCTL_BASEOFF 0b00001000

20.98.2.27 MEMCTL_RAMON #define MEMCTL_RAMON 0b00000000

20.98.2.28 MEMCTL_RAMOFF `#define MEMCTL_RAMOFF 0b00010000`

20.98.2.29 MEMCTL_CROMON `#define MEMCTL_CROMON 0b00000000`

20.98.2.30 MEMCTL_CROMOFF `#define MEMCTL_CROMOFF 0b00100000`

20.98.2.31 MEMCTL_ROMON `#define MEMCTL_ROMON 0b00000000`

20.98.2.32 MEMCTL_ROMOFF `#define MEMCTL_ROMOFF 0b01000000`

20.98.2.33 MEMCTL_EXTON `#define MEMCTL_EXTON 0b00000000`

20.98.2.34 MEMCTL_EXTOFF `#define MEMCTL_EXTOFF 0b10000000`

20.98.2.35 JOY_P1_TR_DIR_IN `#define JOY_P1_TR_DIR_IN 0b00000001`

20.98.2.36 JOY_P1_TR_DIR_OUT `#define JOY_P1_TR_DIR_OUT 0b00000000`

20.98.2.37 JOY_P1_TH_DIR_IN `#define JOY_P1_TH_DIR_IN 0b00000010`

20.98.2.38 GUN_P1_LATCH `#define GUN_P1_LATCH JOY_P1_TH_DIR_IN`

20.98.2.39 JOY_P1_TH_DIR_OUT `#define JOY_P1_TH_DIR_OUT 0b00000000`

20.98.2.40 JOY_P2_TR_DIR_IN `#define JOY_P2_TR_DIR_IN 0b00000100`

20.98.2.41 JOY_P2_TR_DIR_OUT `#define JOY_P2_TR_DIR_OUT 0b00000000`

20.98.2.42 JOY_P2_TH_DIR_IN `#define JOY_P2_TH_DIR_IN 0b00001000`

20.98.2.43 GUN_P2_LATCH `#define GUN_P2_LATCH JOY_P2_TH_DIR_IN`

20.98.2.44 JOY_P2_TH_DIR_OUT `#define JOY_P2_TH_DIR_OUT 0b00000000`

20.98.2.45 JOY_P1_TR_OUT_HI `#define JOY_P1_TR_OUT_HI 0b00010000`

20.98.2.46 JOY_P1_TR_OUT_LO `#define JOY_P1_TR_OUT_LO 0b00000000`

20.98.2.47 JOY_P1_TH_OUT_HI `#define JOY_P1_TH_OUT_HI 0b00100000`

20.98.2.48 JOY_P1_TH_OUT_LO `#define JOY_P1_TH_OUT_LO 0b00000000`

20.98.2.49 JOY_P2_TR_OUT_HI `#define JOY_P2_TR_OUT_HI 0b01000000`

20.98.2.50 JOY_P2_TR_OUT_LO `#define JOY_P2_TR_OUT_LO 0b00000000`

20.98.2.51 JOY_P2_TH_OUT_HI `#define JOY_P2_TH_OUT_HI 0b10000000`

20.98.2.52 JOY_P2_TH_OUT_LO `#define JOY_P2_TH_OUT_LO 0b00000000`

20.98.2.53 JOY_TH_HI `#define JOY_TH_HI (JOY_P1_TR_DIR_IN | JOY_P1_TH_DIR_OUT | JOY_P2_TR_DIR_IN | JOY_P2_TH_DIR_OUT | JOY_P1_TR_OUT_HI | JOY_P1_TH_OUT_HI | JOY_P2_TR_OUT_HI | JOY_P2_TH_OUT_HI)`

20.98.2.54 JOY_TH_LO `#define JOY_TH_LO (JOY_P1_TR_DIR_IN | JOY_P1_TH_DIR_OUT | JOY_P2_TR_DIR_IN | JOY_P2_TH_DIR_OUT | JOY_P1_TR_OUT_HI | JOY_P1_TH_OUT_LO | JOY_P2_TR_OUT_HI | JOY_P2_TH_OUT_LO)`

20.98.2.55 PSG_LATCH `#define PSG_LATCH 0b10000000`

20.98.2.56 PSG_CH0 `#define PSG_CH0 0b00000000`

20.98.2.57 PSG_CH1 `#define PSG_CH1 0b00100000`

20.98.2.58 PSG_CH2 `#define PSG_CH2 0b01000000`

20.98.2.59 PSG_CH3 `#define PSG_CH3 0b01100000`

20.98.2.60 PSG_VOLUME `#define PSG_VOLUME 0b00010000`

20.98.2.61 STATF_INT_VBL `#define STATF_INT_VBL 0b10000000`

20.98.2.62 STATF_9_SPR `#define STATF_9_SPR 0b01000000`

20.98.2.63 STATF_SPR_COLL `#define STATF_SPR_COLL 0b00100000`

20.98.2.64 VDP_REG_MASK `#define VDP_REG_MASK 0b10000000`

20.98.2.65 VDP_R0 `#define VDP_R0 0b10000000`

20.98.2.66 R0_VSCRL `#define R0_VSCRL 0b00000000`

20.98.2.67 R0_VSCRL_INH `#define R0_VSCRL_INH 0b10000000`

20.98.2.68 R0_HSCRL `#define R0_HSCRL 0b00000000`

20.98.2.69 R0_HSCRL_INH `#define R0_HSCRL_INH 0b01000000`

20.98.2.70 R0_NO_LCB `#define R0_NO_LCB 0b00000000`

20.98.2.71 R0_LCB `#define R0_LCB 0b00100000`

20.98.2.72 R0_IE1_OFF `#define R0_IE1_OFF 0b00000000`

20.98.2.73 R0_IE1 `#define R0_IE1 0b00010000`

20.98.2.74 R0_SS_OFF `#define R0_SS_OFF 0b00000000`

20.98.2.75 R0_SS `#define R0_SS 0b00001000`

20.98.2.76 R0_DEFAULT `#define R0_DEFAULT 0b00000110`

20.98.2.77 R0_ES_OFF `#define R0_ES_OFF 0b00000000`

20.98.2.78 R0_ES `#define R0_ES 0b00000001`

20.98.2.79 VDP_R1 `#define VDP_R1 0b10000001`

20.98.2.80 R1_DEFAULT `#define R1_DEFAULT 0b10000000`

20.98.2.81 R1_DISP_OFF `#define R1_DISP_OFF 0b00000000`

20.98.2.82 R1_DISP_ON `#define R1_DISP_ON 0b01000000`

20.98.2.83 R1_IE_OFF `#define R1_IE_OFF 0b00000000`

20.98.2.84 R1_IE `#define R1_IE 0b00100000`

20.98.2.85 R1_SPR_8X8 `#define R1_SPR_8X8 0b00000000`

20.98.2.86 R1_SPR_8X16 `#define R1_SPR_8X16 0b00000010`

20.98.2.87 VDP_R2 `#define VDP_R2 0b10000010`

20.98.2.88 R2_MAP_0x3800 `#define R2_MAP_0x3800 0xFF`

20.98.2.89 R2_MAP_0x3000 `#define R2_MAP_0x3000 0xFD`

20.98.2.90 R2_MAP_0x2800 `#define R2_MAP_0x2800 0xFB`

20.98.2.91 R2_MAP_0x2000 `#define R2_MAP_0x2000 0xF9`

20.98.2.92 R2_MAP_0x1800 `#define R2_MAP_0x1800 0xF7`

20.98.2.93 R2_MAP_0x1000 `#define R2_MAP_0x1000 0xF5`

20.98.2.94 R2_MAP_0x0800 `#define R2_MAP_0x0800 0xF3`

20.98.2.95 R2_MAP_0x0000 `#define R2_MAP_0x0000 0xF1`

20.98.2.96 VDP_R3 `#define VDP_R3 0b10000011`

20.98.2.97 VDP_R4 `#define VDP_R4 0b10000100`

20.98.2.98 VDP_R5 `#define VDP_R5 0b10000101`

20.98.2.99 R5_SAT_0x3F00 `#define R5_SAT_0x3F00 0xFF`

20.98.2.100 R5_SAT_0x1F00 `#define R5_SAT_0x1F00 0xBF`

20.98.2.101 R5_SAT_MASK `#define R5_SAT_MASK 0b10000001`

20.98.2.102 VDP_R6 `#define VDP_R6 0b10000110`

20.98.2.103 R6_BANK0 `#define R6_BANK0 0xFB`

20.98.2.104 R6_DATA_0x0000 `#define R6_DATA_0x0000 0xFB`

20.98.2.105 R6_BANK1 `#define R6_BANK1 0xFF`

20.98.2.106 R6_DATA_0x2000 `#define R6_DATA_0x2000 0xFF`

20.98.2.107 VDP_R7 `#define VDP_R7 0b10000111`

20.98.2.108 VDP_RBORDER `#define VDP_RBORDER 0b10000111`

20.98.2.109 R7_COLOR_MASK `#define R7_COLOR_MASK 0b11110000`

20.98.2.110 VDP_R8 `#define VDP_R8 0b10001000`

20.98.2.111 VDP_RSCX `#define VDP_RSCX 0b10001000`

20.98.2.112 VDP_R9 `#define VDP_R9 0b10001001`

20.98.2.113 VDP_RSCY `#define VDP_RSCY 0b10001001`

20.98.2.114 VDP_R10 `#define VDP_R10 0b10001010`

20.98.2.115 R10_INT_OFF `#define R10_INT_OFF 0xFF`

20.98.2.116 R10_INT EVERY `#define R10_INT EVERY 0x00`

20.98.2.117 JOY_P1_UP `#define JOY_P1_UP 0b00000001`

20.98.2.118 JOY_P1_MD_Z `#define JOY_P1_MD_Z JOY_P1_UP`

20.98.2.119 JOY_P1_DOWN `#define JOY_P1_DOWN 0b00000010`

20.98.2.120 JOY_P1_MD_Y `#define JOY_P1_MD_Y JOY_P1_DOWN`

20.98.2.121 JOY_P1_LEFT `#define JOY_P1_LEFT 0b00000100`

20.98.2.122 JOY_P1_MD_X `#define JOY_P1_MD_X JOY_P1_LEFT`

20.98.2.123 JOY_P1_RIGHT `#define JOY_P1_RIGHT 0b00001000`

20.98.2.124 JOY_P1_MD_MODE `#define JOY_P1_MD_MODE JOY_P1_RIGHT`

20.98.2.125 JOY_P1_SW1 `#define JOY_P1_SW1 0b00010000`

20.98.2.126 JOY_P1_TRIGGER `#define JOY_P1_TRIGGER JOY_P1_SW1`

20.98.2.127 JOY_P1_MD_A `#define JOY_P1_MD_A JOY_P1_SW1`

20.98.2.128 JOY_P1_SW2 `#define JOY_P1_SW2 0b00100000`

20.98.2.129 JOY_P1_MD_START `#define JOY_P1_MD_START JOY_P1_SW2`

20.98.2.130 JOY_P2_UP `#define JOY_P2_UP 0b01000000`

20.98.2.131 JOY_P2_MD_Z `#define JOY_P2_MD_Z JOY_P2_UP`

20.98.2.132 JOY_P2_DOWN `#define JOY_P2_DOWN 0b10000000`

20.98.2.133 JOY_P2_MD_Y `#define JOY_P2_MD_Y JOY_P2_DOWN`

20.98.2.134 JOY_P2_LEFT `#define JOY_P2_LEFT 0b00000001`

- 20.98.2.135 JOY_P2_MD_X** `#define JOY_P2_MD_X JOY_P2_LEFT`
- 20.98.2.136 JOY_P2_RIGHT** `#define JOY_P2_RIGHT 0b00000010`
- 20.98.2.137 JOY_P2_MD_MODE** `#define JOY_P2_MD_MODE JOY_P2_RIGHT`
- 20.98.2.138 JOY_P2_SW1** `#define JOY_P2_SW1 0b00000100`
- 20.98.2.139 JOY_P2_TRIGGER** `#define JOY_P2_TRIGGER JOY_P2_SW1`
- 20.98.2.140 JOY_P2_MD_A** `#define JOY_P2_MD_A JOY_P2_SW1`
- 20.98.2.141 JOY_P2_SW2** `#define JOY_P2_SW2 0b00001000`
- 20.98.2.142 JOY_P2_MD_START** `#define JOY_P2_MD_START JOY_P2_SW2`
- 20.98.2.143 JOY_RESET** `#define JOY_RESET 0b00010000`
- 20.98.2.144 JOY_P1_LIGHT** `#define JOY_P1_LIGHT 0b01000000`
- 20.98.2.145 JOY_P2_LIGHT** `#define JOY_P2_LIGHT 0b10000000`
- 20.98.2.146 RAMCTL_BANK** `#define RAMCTL_BANK 0b00000100`
- 20.98.2.147 RAMCTL_ROM** `#define RAMCTL_ROM 0b00000000`
- 20.98.2.148 RAMCTL_RAM** `#define RAMCTL_RAM 0b00001000`
- 20.98.2.149 RAMCTL_RO** `#define RAMCTL_RO 0b00010000`
- 20.98.2.150 RAMCTL_PROT** `#define RAMCTL_PROT 0b10000000`
- 20.98.2.151 VBK_TILES** `#define VBK_TILES 0`
- 20.98.2.152 VBK_ATTRIBUTES** `#define VBK_ATTRIBUTES 1`

20.98.2.153 VDP_SAT_TERM `#define VDP_SAT_TERM 0xD0`

20.98.2.154 DEVICE_SCREEN_PX_WIDTH `#define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)`

20.98.2.155 DEVICE_SCREEN_PX_HEIGHT `#define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)`

20.98.3 Variable Documentation

20.98.3.1 shadow_VDP_R0 `UBYTE shadow_VDP_R0 [extern]`

20.98.3.2 shadow_VDP_R1 `UBYTE shadow_VDP_R1 [extern]`

20.98.3.3 shadow_VDP_R2 `UBYTE shadow_VDP_R2 [extern]`

20.98.3.4 shadow_VDP_R3 `UBYTE shadow_VDP_R3 [extern]`

20.98.3.5 shadow_VDP_R4 `UBYTE shadow_VDP_R4 [extern]`

20.98.3.6 shadow_VDP_R5 `UBYTE shadow_VDP_R5 [extern]`

20.98.3.7 shadow_VDP_R6 `UBYTE shadow_VDP_R6 [extern]`

20.98.3.8 shadow_VDP_R7 `UBYTE shadow_VDP_R7 [extern]`

20.98.3.9 shadow_VDP_RBORDER `UBYTE shadow_VDP_RBORDER [extern]`

20.98.3.10 shadow_VDP_R8 `UBYTE shadow_VDP_R8 [extern]`

20.98.3.11 shadow_VDP_RSCX `UBYTE shadow_VDP_RSCX [extern]`

20.98.3.12 shadow_VDP_R9 `UBYTE shadow_VDP_R9 [extern]`

20.98.3.13 shadow_VDP_RSCY `UBYTE shadow_VDP_RSCY [extern]`

20.98.3.14 shadow_VDP_R10 `UBYTE shadow_VDP_R10 [extern]`

20.98.3.15 TIMA_REG volatile UBYTE TIMA_REG [extern]
Timer counter

20.98.3.16 TMA_REG volatile UBYTE TMA_REG [extern]
Timer modulo

20.98.3.17 TAC_REG volatile UBYTE TAC_REG [extern]
Timer control

20.98.3.18 VDP_ATTR_SHIFT volatile UBYTE VDP_ATTR_SHIFT [extern]

20.99 hardware.h

[Go to the documentation of this file.](#)

```

1
2
3
4
5 #ifndef _HARDWARE_H
6 #define _HARDWARE_H
7
8 #include <types.h>
9
10 #define __BYTES extern UBYTE
11 #define __BYTE_REG extern volatile UBYTE
12
13 static volatile SFR AT(0x00) GG_STATE;
14 #define GGSTATE_STT 0b10000000
15 #define GGSTATE_NJAP 0b01000000
16 #define GGSTATE_NNTS 0b00100000
17
18
19 static volatile SFR AT(0x01) GG_EXT_7BIT;
20 static volatile SFR AT(0x02) GG_EXT_CTL;
21 #define GGEXT_NINT 0b10000000
22
23
24
25 static volatile SFR AT(0x03) GG_SIO_SEND;
26 static volatile SFR AT(0x04) GG_SIO_RECV;
27 static volatile SFR AT(0x05) GG_SIO_CTL;
28
29 #define SIOCTL_TXFL 0b00000001
30 #define SIOCTL_RXRD 0b00000010
31 #define SIOCTL_FRER 0b00000100
32 #define SIOCTL_INT 0b00001000
33 #define SIOCTL_TON 0b00010000
34 #define SIOCTL_RON 0b00100000
35 #define SIOCTL_BS0 0b01000000
36 #define SIOCTL_BS1 0b10000000
37
38 static volatile SFR AT(0x06) GG_SOUND_PAN;
39
40 #define SOUNDPAN_TN1R 0b00000001
41 #define SOUNDPAN_TN2R 0b00000010
42 #define SOUNDPAN_TN3R 0b00000100
43 #define SOUNDPAN_NOSR 0b00001000
44 #define SOUNDPAN_TN1L 0b00010000
45 #define SOUNDPAN_TN2L 0b00100000
46 #define SOUNDPAN_TN3L 0b01000000
47 #define SOUNDPAN_NOSL 0b10000000
48
49 static volatile SFR AT(0x3E) MEMORY_CTL;
50
51 #define MEMCTL_JOYON 0b00000000
52 #define MEMCTL_JOYOFF 0b00000100
53 #define MEMCTL_BASEON 0b00000000
54 #define MEMCTL_BASEOFF 0b00001000
55 #define MEMCTL_RAMON 0b00000000
56 #define MEMCTL_RAMOFF 0b00010000
57 #define MEMCTL_CROMON 0b00000000
58 #define MEMCTL_CROMOFF 0b00100000
59 #define MEMCTL_ROMON 0b00000000
60 #define MEMCTL_ROMOFF 0b01000000
61 #define MEMCTL_EXTON 0b00000000
62 #define MEMCTL_EXTOFF 0b10000000
63
64 static volatile SFR AT(0x3F) JOY_CTL;
65
66 #define JOY_P1_TR_DIR_IN 0b00000001
67 #define JOY_P1_TR_DIR_OUT 0b00000000
68 #define JOY_P1_TH_DIR_IN 0b00000010
69 #define GUN_P1_LATCH JOY_P1_TH_DIR_IN
70 #define JOY_P1_TH_DIR_OUT 0b00000000
71 #define JOY_P2_TR_DIR_IN 0b00000100
72 #define JOY_P2_TR_DIR_OUT 0b00000000
73 #define JOY_P2_TH_DIR_IN 0b00001000

```

```

74 #define GUN_P2_LATCH      JOY_P2_TH_DIR_IN
75 #define JOY_P2_TH_DIR_OUT 0b00000000
76 #define JOY_P1_TR_OUT_HI  0b00010000
77 #define JOY_P1_TR_OUT_LO  0b00000000
78 #define JOY_P1_TH_OUT_HI  0b00100000
79 #define JOY_P1_TH_OUT_LO  0b00000000
80 #define JOY_P2_TR_OUT_HI  0b01000000
81 #define JOY_P2_TR_OUT_LO  0b00000000
82 #define JOY_P2_TH_OUT_HI  0b10000000
83 #define JOY_P2_TH_OUT_LO  0b00000000
84
85 #define JOY_TH_HI (JOY_P1_TR_DIR_IN | JOY_P1_TH_DIR_OUT | JOY_P2_TR_DIR_IN | JOY_P2_TH_DIR_OUT |
    JOY_P1_TR_OUT_HI | JOY_P1_TH_OUT_HI | JOY_P2_TR_OUT_HI | JOY_P2_TH_OUT_HI)
86 #define JOY_TH_LO (JOY_P1_TR_DIR_IN | JOY_P1_TH_DIR_OUT | JOY_P2_TR_DIR_IN | JOY_P2_TH_DIR_OUT |
    JOY_P1_TR_OUT_HI | JOY_P1_TH_OUT_LO | JOY_P2_TR_OUT_HI | JOY_P2_TH_OUT_LO)
87
88 static volatile SFR AT(0x7E) VCOUNTER;
89
90 static volatile SFR AT(0x7F) PSG;
91
92 #define PSG_LATCH      0b10000000
93
94 #define PSG_CH0      0b00000000
95 #define PSG_CH1      0b00100000
96 #define PSG_CH2      0b01000000
97 #define PSG_CH3      0b01100000
98
99 #define PSG_VOLUME    0b00010000
100
101 static volatile SFR AT(0x7F) HCOUNT;
102
103 static volatile SFR AT(0xBE) VDP_DATA;
104 static volatile SFR AT(0xBF) VDP_CMD;
105 static volatile SFR AT(0xBF) VDP_STATUS;
106
107 #define STATF_INT_VBL  0b10000000
108 #define STATF_9_SPR    0b01000000
109 #define STATF_SPR_COLL 0b00100000
110
111 #define VDP_REG_MASK   0b10000000
112 #define VDP_R0         0b10000000
113 extern UBYTE shadow_VDP_R0;
114
115 #define R0_VSCRL       0b00000000
116 #define R0_VSCRL_INH   0b10000000
117 #define R0_HSCRL       0b00000000
118 #define R0_HSCRL_INH   0b01000000
119 #define R0_NO_LCB      0b00000000
120 #define R0_LCB         0b00100000
121 #define R0_IE1_OFF     0b00000000
122 #define R0_IE1         0b00010000
123 #define R0_SS_OFF      0b00000000
124 #define R0_SS          0b00001000
125 #define R0_DEFAULT     0b00000110
126 #define R0_ES_OFF      0b00000000
127 #define R0_ES          0b00000001
128
129 #define VDP_R1         0b10000001
130 extern UBYTE shadow_VDP_R1;
131
132 #define R1_DEFAULT     0b10000000
133 #define R1_DISP_OFF    0b00000000
134 #define R1_DISP_ON     0b01000000
135 #define R1_IE_OFF      0b00000000
136 #define R1_IE          0b00100000
137 #define R1_SPR_8X8     0b00000000
138 #define R1_SPR_8X16    0b00000010
139
140 #define VDP_R2         0b10000010
141 extern UBYTE shadow_VDP_R2;
142
143 #define R2_MAP_0x3800  0xFF
144 #define R2_MAP_0x3000  0xFD
145 #define R2_MAP_0x2800  0xFB
146 #define R2_MAP_0x2000  0xF9
147 #define R2_MAP_0x1800  0xF7
148 #define R2_MAP_0x1000  0xF5
149 #define R2_MAP_0x0800  0xF3
150 #define R2_MAP_0x0000  0xF1
151
152 #define VDP_R3         0b10000011
153 extern UBYTE shadow_VDP_R3;
154 #define VDP_R4         0b10000100
155 extern UBYTE shadow_VDP_R4;
156 #define VDP_R5         0b10000101
157 extern UBYTE shadow_VDP_R5;
158

```

```
159 #define R5_SAT_0x3F00 0xFF
160 #define R5_SAT_0x1F00 0xBF
161 #define R5_SAT_MASK 0b10000001
162
163 #define VDP_R6 0b10000110
164 extern UBYTE shadow_VDP_R6;
165
166 #define R6_BANK0 0xFB
167 #define R6_DATA_0x0000 0xFB
168 #define R6_BANK1 0xFF
169 #define R6_DATA_0x2000 0xFF
170
171 #define VDP_R7 0b10000111
172 extern UBYTE shadow_VDP_R7;
173 #define VDP_RBORDER 0b10000111
174 extern UBYTE shadow_VDP_RBORDER;
175
176 #define R7_COLOR_MASK 0b11110000
177
178 #define VDP_R8 0b10001000
179 extern UBYTE shadow_VDP_R8;
180 #define VDP_RSCX 0b10001000
181 extern UBYTE shadow_VDP_RSCX;
182
183 #define VDP_R9 0b10001001
184 extern UBYTE shadow_VDP_R9;
185 #define VDP_RSCY 0b10001001
186 extern UBYTE shadow_VDP_RSCY;
187
188 #define VDP_R10 0b10001010
189 extern UBYTE shadow_VDP_R10;
190
191 #define R10_INT_OFF 0xFF
192 #define R10_INT_EVERY 0x00
193
194 static volatile SFR AT(0xDC) JOY_PORT1;
195
196 #define JOY_P1_UP 0b00000001
197 #define JOY_P1_MD_Z JOY_P1_UP
198 #define JOY_P1_DOWN 0b00000010
199 #define JOY_P1_MD_Y JOY_P1_DOWN
200 #define JOY_P1_LEFT 0b00000100
201 #define JOY_P1_MD_X JOY_P1_LEFT
202 #define JOY_P1_RIGHT 0b00001000
203 #define JOY_P1_MD_MODE JOY_P1_RIGHT
204 #define JOY_P1_SW1 0b00010000
205 #define JOY_P1_TRIGGER JOY_P1_SW1
206 #define JOY_P1_MD_A JOY_P1_SW1
207 #define JOY_P1_SW2 0b00100000
208 #define JOY_P1_MD_START JOY_P1_SW2
209 #define JOY_P2_UP 0b01000000
210 #define JOY_P2_MD_Z JOY_P2_UP
211 #define JOY_P2_DOWN 0b10000000
212 #define JOY_P2_MD_Y JOY_P2_DOWN
213
214 static volatile SFR AT(0xDD) JOY_PORT2;
215
216 #define JOY_P2_LEFT 0b00000001
217 #define JOY_P2_MD_X JOY_P2_LEFT
218 #define JOY_P2_RIGHT 0b00000010
219 #define JOY_P2_MD_MODE JOY_P2_RIGHT
220 #define JOY_P2_SW1 0b00000100
221 #define JOY_P2_TRIGGER JOY_P2_SW1
222 #define JOY_P2_MD_A JOY_P2_SW1
223 #define JOY_P2_SW2 0b00001000
224 #define JOY_P2_MD_START JOY_P2_SW2
225 #define JOY_RESET 0b00010000
226 #define JOY_P1_LIGHT 0b01000000
227 #define JOY_P2_LIGHT 0b10000000
228
229 static volatile SFR AT(0xF0) FMADDRESS;
230 static volatile SFR AT(0xF1) FMDATA;
231 static volatile SFR AT(0xF2) AUDIOCTRL;
232
233 static volatile UBYTE AT(0xfffc) RAM_CONTROL;
234
235 #define RAMCTL_BANK 0b00000100
236 #define RAMCTL_ROM 0b00000000
237 #define RAMCTL_RAM 0b00001000
238 #define RAMCTL_RO 0b00010000
239 #define RAMCTL_PROT 0b10000000
240
241 static volatile UBYTE AT(0xffff8) GLASSES_3D;
242
243 static volatile UBYTE AT(0xffffd) MAP_FRAME0;
244 static volatile UBYTE AT(0xffffe) MAP_FRAME1;
245 static volatile UBYTE AT(0xfffff) MAP_FRAME2;
```

```

246
247 extern volatile UBYTE TIMA_REG;
248 extern volatile UBYTE TMA_REG;
249 extern volatile UBYTE TAC_REG;
250
251 extern volatile UBYTE VDP_ATTR_SHIFT;
252
253 #define VBK_TILES      0
254 #define VBK_ATTRIBUTES 1
255
256 #define VDP_SAT_TERM    0xD0
257
258 #if defined(__TARGET_sms)
259 #define DEVICE_SCREEN_X_OFFSET 0
260 #define DEVICE_SCREEN_Y_OFFSET 0
261 #define DEVICE_SCREEN_WIDTH 32
262 #define DEVICE_SCREEN_HEIGHT 24
263 #define DEVICE_SCREEN_BUFFER_WIDTH 32
264 #define DEVICE_SCREEN_BUFFER_HEIGHT 28
265 #define DEVICE_SCREEN_MAP_ENTRY_SIZE 2
266 #define DEVICE_SPRITE_PX_OFFSET_X 0
267 #define DEVICE_SPRITE_PX_OFFSET_Y -1
268 #define DEVICE_WINDOW_PX_OFFSET_X 0
269 #define DEVICE_WINDOW_PX_OFFSET_Y 0
270 #elif defined(__TARGET_gg)
271 #define DEVICE_SCREEN_X_OFFSET 6
272 #define DEVICE_SCREEN_Y_OFFSET 3
273 #define DEVICE_SCREEN_WIDTH 20
274 #define DEVICE_SCREEN_HEIGHT 18
275 #define DEVICE_SCREEN_BUFFER_WIDTH 32
276 #define DEVICE_SCREEN_BUFFER_HEIGHT 28
277 #define DEVICE_SCREEN_MAP_ENTRY_SIZE 2
278 #define DEVICE_SPRITE_PX_OFFSET_X 48
279 #define DEVICE_SPRITE_PX_OFFSET_Y 23
280 #define DEVICE_WINDOW_PX_OFFSET_X 0
281 #define DEVICE_WINDOW_PX_OFFSET_Y 0
282 #else
283 #error Unrecognized port
284 #endif
285 #define DEVICE_SCREEN_PX_WIDTH (DEVICE_SCREEN_WIDTH * 8)
286 #define DEVICE_SCREEN_PX_HEIGHT (DEVICE_SCREEN_HEIGHT * 8)
287
288 #endif

```

20.100 gbdk-lib/include/gb/metasprites.h File Reference

```

#include <gb/hardware.h>
#include <types.h>
#include <stdint.h>

```

Data Structures

- struct [metasprite_t](#)

Macros

- #define [metasprite_end](#) -128
- #define [METASPR_ITEM](#)(dy, dx, dt, a) {(dy),(dx),(dt),(a)}
- #define [METASPR_TERM](#) {metasprite_end}

Typedefs

- typedef struct [metasprite_t](#) [metasprite_t](#)

Functions

- void [hide_sprites_range](#) (uint8_t from, uint8_t to)
- uint8_t [move_metasprite_ex](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, uint8_t x, uint8_t y)
- uint8_t [move_metasprite](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_sprite, uint8_t x, uint8_t y)

- `uint8_t move_metasprite_flipx` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_prop, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- `uint8_t move_metasprite_vflip` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- `uint8_t move_metasprite_flipy` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_prop, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- `uint8_t move_metasprite_hflip` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- `uint8_t move_metasprite_flipxy` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_prop, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- `uint8_t move_metasprite_hvflip` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- `void hide_metasprite` (const `metasprite_t` *metasprite, `uint8_t` base_sprite)

Variables

- `const void * __current_metasprite`
- `uint8_t __current_base_tile`
- `uint8_t __current_base_prop`
- `uint8_t __render_shadow_OAM`

20.100.1 Detailed Description

20.100.2 Metasprite support

A metasprite is a larger sprite made up from a collection of smaller individual hardware sprites. Different frames of the same metasprites can share tile data.

The api supports metasprites in both `SPRITES_8x8` and `SPRITES_8x16` mode. If 8x16 mode is used then the height of the metasprite must be a multiple of 16.

The origin (pivot) for the metasprite is not required to be in the upper left-hand corner as with regular hardware sprites.

Use the `utility_png2asset` tool to convert single or multiple frames of graphics into metasprite structured data for use with the `...metasprite...()` functions.

20.100.3 Metasprites composed of variable numbers of sprites

When using `png2asset`, it's common for the output of different frames to be composed of different numbers of hardware sprites (since it's trying to create each frame as efficiently as possible). Due to that, it's good practice to clear out (hide) unused sprites in the `shadow_OAM` that have been set by previous frames.

```
// Example:
// Hide rest of the hardware sprites, because amount
// of sprites differ between animation frames.
// (where hiwater == last hardware sprite used + 1)
hide_sprites_range(hiwater, MAX_HARDWARE_SPRITES);
```

20.100.4 Metasprites and sprite properties (including cgb palette)

When the `move_metasprite_*`() functions are called they update all properties for the affected sprites in the `Shadow OAM`. This means any existing property flags set for a sprite (CGB palette, BG/WIN priority, Tile VRAM Bank) will get overwritten.

How to use sprite property flags with metasprites:

- Primary method: Use the `base_prop` parameter for the `move_metasprite_*`() functions.
 - For more details about the properties on the Game Boy see: <https://gbdev.io/pandocs/OAM.html#byte-3-attributesflags>
 - This can be left at zero for defaults
 - Various `OAMF_*` flags can be used depending on the platform:
 - * `OAMF_BANK0`, `OAMF_BANK1`

- * [OAMF_CGB_PAL0](#), [OAMF_CGB_PAL1](#), [OAMF_CGB_PAL2](#), [OAMF_CGB_PAL3](#), [OAMF_CGB_PAL4](#), [OAMF_CGB_PAL5](#), [OAMF_CGB_PAL6](#), [OAMF_CGB_PAL7](#),
- * [OAMF_PAL0](#), [OAMF_PAL1](#),
- * [OAMF_PALMASK](#), [OAMF_PRI](#), [OAMF_XFLIP](#), [OAMF_YFLIP](#)

- Alternate method: The metasprite structures can have the property flags modified before compilation (such as with `-sp <props>` in the [png2asset](#) tool).

The following functions only support hardware sprite flipping on the Game Boy / Mega Duck and NES. For other consoles which do not have hardware sprite flipping see the cross-platform metasprite example for a workaround (with some performance penalty).

- [move metasprite flipx\(\)](#)
- [move metasprite flipy\(\)](#)
- [move metasprite flipxy\(\)](#)

To test for hardware support see [HARDWARE_SPRITE_CAN_FLIP_X](#) and [HARDWARE_SPRITE_CAN_FLIP_Y](#). Also see [docs_consoles_supported_list](#) for a brief summary of console capabilities.

20.100.5 Macro Definition Documentation

20.100.5.1 `metasprite_end` `#define metasprite_end -128`

20.100.5.2 `METASPR_ITEM` `#define METASPR_ITEM(`
`dy,`
`dx,`
`dt,`
`a ) { (dy), (dx), (dt), (a) }`

20.100.5.3 `METASPR_TERM` `#define METASPR_TERM {metasprite_end}`

20.100.6 Typedef Documentation

20.100.6.1 `metasprite_t` `typedef struct metasprite_t metasprite_t`
 Metasprite sub-item structure

Parameters

<i>dy</i>	(int8_t) Y coordinate of the sprite relative to the metasprite origin (pivot)
<i>dx</i>	(int8_t) X coordinate of the sprite relative to the metasprite origin (pivot)
<i>dtile</i>	(uint8_t) Start tile relative to the metasprites own set of tiles
<i>props</i>	(uint8_t) Property Flags

Metasprites are built from multiple [metasprite_t](#) items (one for each sub-sprite) and a pool of tiles they reference. If a metasprite has multiple frames then each frame will be built from some number of [metasprite_t](#) items (which may vary based on how many sprites are required for that particular frame).

A metasprite frame is terminated with a `{metasprite_end}` entry.

20.100.7 Function Documentation

20.100.7.1 hide_sprites_range() `void hide_sprites_range (`
 `uint8_t from,`
 `uint8_t to )`

Hides all hardware sprites in range from $\leq X < to$

Parameters

<i>from</i>	start OAM index
<i>to</i>	finish OAM index (must be $\leq \text{MAX_HARDWARE_SPRITES}$)

See also

[hide_sprite](#), [MAX_HARDWARE_SPRITES](#)

Hides all hardware sprites in range from $\leq X < to$

Parameters

<i>from</i>	start OAM index
<i>to</i>	finish OAM index

20.100.7.2 move_metasprite_ex() `uint8_t move_metasprite_ex (`
 `const metasprite_t * metasprite,`
 `uint8_t base_tile,`
 `uint8_t base_prop,`
 `uint8_t base_sprite,`
 `uint8_t x,`
 `uint8_t y ) [inline]`

Moves metasprite to the absolute position **x** and **y**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (can be used to set palette, etc)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Moves **metasprite** to the absolute position **x** and **y** (with **no flip** on the X or Y axis). Hardware sprites are allocated starting from **base_sprite**, using tiles starting from **base_tile**.

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as CGB Palette), see [Metaspprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

20.100.7.3 `move metasprite()` `uint8_t` `move metasprite` (
 const `metasprite_t` * `metasprite`,
 `uint8_t` `base_tile`,
 `uint8_t` `base_sprite`,
 `uint8_t` `x`,
 `uint8_t` `y`) [inline]

Obsolete. This function has been replaced by [move metasprite_ex\(\)](#)

20.100.7.4 `move metasprite_flipx()` `uint8_t` `move metasprite_flipx` (
 const `metasprite_t` * `metasprite`,
 `uint8_t` `base_tile`,
 `uint8_t` `base_prop`,
 `uint8_t` `base_sprite`,
 `uint8_t` `x`,
 `uint8_t` `y`) [inline]

Moves metasprite to the absolute position `x` and `y`, **flipped by X (horizontally)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (can be used to set palette, etc)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move metasprite\(\)](#), but with the metasprite flipped by X (horizontally).

Sets:

- `__current_metasprite` = `metasprite`;
- `__current_base_tile` = `base_tile`;

Note: Overwrites OAM sprite properties (such as CGB palette), see [Metasprites and sprite properties](#).

This function is only available on Game Boy and related clone consoles.

Returns

Number of hardware sprites used to draw this metasprite

See also

[move metasprite\(\)](#)

20.100.7.5 `move metasprite_vflip()` `uint8_t` `move metasprite_vflip` (
 const `metasprite_t` * `metasprite`,
 `uint8_t` `base_tile`,
 `uint8_t` `base_sprite`,
 `uint8_t` `x`,
 `uint8_t` `y`) [inline]

Obsolete. This function has been replaced by [move metasprite_flipx\(\)](#)

20.100.7.6 `move metasprite_flipy()` `uint8_t` `move metasprite_flipy` (
 const `metasprite_t` * `metasprite`,
 `uint8_t` `base_tile`,
 `uint8_t` `base_prop`,
 `uint8_t` `base_sprite`,


```
uint8_t x,
uint8_t y ) [inline]
```

Moves metasprite to the absolute position x and y, **flipped by Y (vertically)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (can be used to set palette, etc)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move metasprite\(\)](#), but with the metasprite flipped by Y (vertically).

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as CGB palette), see [Metasp_sprites and sprite properties](#). This function is only available on Game Boy and related clone consoles.

Returns

Number of hardware sprites used to draw this metasprite

See also

[move metasprite\(\)](#)

20.100.7.7 move metasprite_hflip() `uint8_t move metasprite_hflip (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_sprite,`
`uint8_t x,`
`uint8_t y ) [inline]`

Obsolete. This function has been replaced by [move metasprite_flipy\(\)](#)

20.100.7.8 move metasprite_flipxy() `uint8_t move metasprite_flipxy (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_prop,`
`uint8_t base_sprite,`
`uint8_t x,`
`uint8_t y ) [inline]`

Moves metasprite to the absolute position x and y, **flipped by X and Y (horizontally and vertically)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (can be used to set palette, etc)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move metasprite\(\)](#), but with the metasprite flipped by X and Y (horizontally and vertically).
Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as CGB palette), see [Metasprites and sprite properties](#).
This function is only available on Game Boy and related clone consoles.

Returns

Number of hardware sprites used to draw this metasprite

See also

[move metasprite\(\)](#)

20.100.7.9 move_metasprite_hvflip() `uint8_t move_metasprite_hvflip (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_sprite,`
`uint8_t x,`
`uint8_t y ) [inline]`

Obsolete. This function has been replaced by [move_metasprite_flipxy\(\)](#)

20.100.7.10 hide_metasprite() `void hide_metasprite (`
`const metasprite_t * metasprite,`
`uint8_t base_sprite ) [inline]`

Hides a metasprite from the screen

Parameters

<i>metasprite</i>	Pointer to first struct of the desired metasprite frame
<i>base_sprite</i>	Number of hardware sprite to start with

Sets:

- `__current_metasprite = metasprite;`

20.100.8 Variable Documentation

20.100.8.1 __current_metasprite `const void* __current_metasprite [extern]`

20.100.8.2 __current_base_tile `uint8_t __current_base_tile [extern]`

20.100.8.3 __current_base_prop `uint8_t __current_base_prop [extern]`

20.100.8.4 __render_shadow_OAM `uint8_t __render_shadow_OAM [extern]`

20.101 metasprites.h

[Go to the documentation of this file.](#)

```

1
82 #ifndef _METASPRITES_H_INCLUDE
83 #define _METASPRITES_H_INCLUDE
84
85 #include <gb/hardware.h>
86 #include <types.h>
87 #include <stdint.h>
88
102 typedef struct metasprite_t {
103     int8_t dy, dx;
104     uint8_t dtile;
105     uint8_t props;
106 } metasprite_t;
107
108 #define metasprite_end -128
109 #define METASPR_ITEM(dy,dx,dt,a) {(dy),(dx),(dt),(a)}
110 #define METASPR_TERM {metasprite_end}
111
112 extern const void * __current_metasprite;
113 extern uint8_t __current_base_tile;
114 extern uint8_t __current_base_prop;
115 extern uint8_t __render_shadow_OAM;
116
117
118 static uint8_t __move_metasprite(uint8_t id, uint16_t yx);
119 static uint8_t __move_metasprite_flipx(uint8_t id, uint16_t yx);
120 static uint8_t __move_metasprite_flipy(uint8_t id, uint16_t yx);
121 static uint8_t __move_metasprite_flipxy(uint8_t id, uint16_t yx);
122 static uint8_t __move_metasprite_vflip(uint8_t id, uint16_t yx);
123 static uint8_t __move_metasprite_hflip(uint8_t id, uint16_t yx);
124 static uint8_t __move_metasprite_hvflip(uint8_t id, uint16_t yx);
125 static void __hide_metasprite(uint8_t id);
126
134 void hide_sprites_range(uint8_t from, uint8_t to);
135
159 inline uint8_t move_metasprite_ex(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_prop,
    uint8_t base_sprite, uint8_t x, uint8_t y) {
160     __current_metasprite = metasprite;
161     __current_base_tile = base_tile;
162     __current_base_prop = base_prop;
163     return __move_metasprite(base_sprite, (y << 8) | (uint8_t)x);
164 }
165
168 inline uint8_t move_metasprite(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_sprite,
    uint8_t x, uint8_t y) {
169     __current_metasprite = metasprite;
170     __current_base_tile = base_tile;
171     __current_base_prop = 0;
172     return __move_metasprite(base_sprite, (y << 8) | (uint8_t)x);
173 }
174
199 inline uint8_t move_metasprite_flipx(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_prop, uint8_t base_sprite, uint8_t x, uint8_t y) {
200     __current_metasprite = metasprite;
201     __current_base_tile = base_tile;
202     __current_base_prop = base_prop;
203     return __move_metasprite_flipx(base_sprite, (y << 8) | (uint8_t)(x - 8u));
204 }
205
208 inline uint8_t move_metasprite_vflip(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, uint8_t x, uint8_t y) {
209     __current_metasprite = metasprite;
210     __current_base_tile = base_tile;
211     __current_base_prop = 0;
212     return __move_metasprite_vflip(base_sprite, (y << 8) | (uint8_t)(x - 8u));
213 }
214
215
240 inline uint8_t move_metasprite_flipy(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_prop, uint8_t base_sprite, uint8_t x, uint8_t y) {
241     __current_metasprite = metasprite;
242     __current_base_tile = base_tile;
243     __current_base_prop = base_prop;
244     return __move_metasprite_flipy(base_sprite, ((y - ((LCDC_REG & LCDCF_OBJ16) ? 16u : 8u)) << 8) | x);
245 }
246
249 inline uint8_t move_metasprite_hflip(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, uint8_t x, uint8_t y) {
250     __current_metasprite = metasprite;
251     __current_base_tile = base_tile;
252     __current_base_prop = 0;
253     return __move_metasprite_hflip(base_sprite, ((y - ((LCDC_REG & LCDCF_OBJ16) ? 16u : 8u)) << 8) | x);
254 }
255

```

```

280 inline uint8_t move metasprite_flipxy(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_prop, uint8_t base_sprite, uint8_t x, uint8_t y) {
281     __current_metasprite = metasprite;
282     __current_base_tile = base_tile;
283     __current_base_prop = base_prop;
284     return __move_metasprite_flipxy(base_sprite, ((y - ((LCDREG & LCDCF_OBJ16) ? 16u : 8u)) < 8) |
        (uint8_t)(x - 8));
285 }
286
289 inline uint8_t move_metasprite_hvflip(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, uint8_t x, uint8_t y) {
290     __current_metasprite = metasprite;
291     __current_base_tile = base_tile;
292     __current_base_prop = 0;
293     return __move_metasprite_hvflip(base_sprite, ((y - ((LCDREG & LCDCF_OBJ16) ? 16u : 8u)) < 8) |
        (uint8_t)(x - 8));
294 }
295
305 inline void hide_metasprite(const metasprite_t * metasprite, uint8_t base_sprite) {
306     __current_metasprite = metasprite;
307     __hide_metasprite(base_sprite);
308 }
309
310 #endif

```

20.102 gbdk-lib/include/gbdk/metasprites.h File Reference

```
#include <gb/metasprites.h>
```

20.103 metasprites.h

[Go to the documentation of this file.](#)

```

1 #ifndef __PLAT_METASPRITES_H_INVCLUE
2 #define __PLAT_METASPRITES_H_INVCLUE
3
4 #if defined(__TARGET_gb) || defined(__TARGET_ap) || defined(__TARGET_duck)
5     #include <gb/metasprites.h>
6 #elif defined(__TARGET_sms) || defined(__TARGET_gg)
7     #include <sms/metasprites.h>
8 #elif defined(__TARGET_msxdos)
9     #include <msx/metasprites.h>
10 #elif defined(__TARGET_nes)
11     #include <nes/metasprites.h>
12 #else
13     #error Unrecognized port
14 #endif
15
16 #endif

```

20.104 gbdk-lib/include/msx/metasprites.h File Reference

```

#include <msx/hardware.h>
#include <types.h>
#include <stdint.h>

```

Data Structures

- struct [metasprite_t](#)

Macros

- #define [metasprite_end](#) -128
- #define [METASPR_ITEM](#)(dy, dx, dt, a) {(dy),(dx),(dt),(a)}
- #define [METASPR_TERM](#) {metasprite_end}

Typedefs

- typedef struct [metasprite_t](#) metasprite_t

Functions

- void `hide_sprites_range` (`uint8_t` from, `uint8_t` to) `Z88DK_CALLEE PRESERVES_REGS(iyh`
- `uint8_t` `move metasprite_ex` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_prop, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- `uint8_t` `move metasprite` (const `metasprite_t` *metasprite, `uint8_t` base_tile, `uint8_t` base_sprite, `uint8_t` x, `uint8_t` y)
- void `hide metasprite` (const `metasprite_t` *metasprite, `uint8_t` base_sprite)

Variables

- const void * `__current metasprite`
- `uint8_t` `__current_base_tile`
- `uint8_t` `__render_shadow_OAM`
- static `uint8_t` `iyi`

20.104.1 Macro Definition Documentation

20.104.1.1 metasprite_end `#define metasprite_end -128`

20.104.1.2 METASPR_ITEM `#define METASPR_ITEM(`
`dy,`
`dx,`
`dt,`
`a ) { (dy), (dx), (dt), (a) }`

20.104.1.3 METASPR_TERM `#define METASPR_TERM {metasprite_end}`

20.104.2 Typedef Documentation

20.104.2.1 metasprite_t `typedef struct metasprite_t metasprite_t`
Metasprite sub-item structure

Parameters

<i>dy</i>	(<code>int8_t</code>) Y coordinate of the sprite relative to the metasprite origin (pivot)
<i>dx</i>	(<code>int8_t</code>) X coordinate of the sprite relative to the metasprite origin (pivot)
<i>dtile</i>	(<code>uint8_t</code>) Start tile relative to the metasprites own set of tiles
<i>props</i>	(<code>uint8_t</code>) Property Flags

Metasprites are built from multiple `metasprite_t` items (one for each sub-sprite) and a pool of tiles they reference. If a metasprite has multiple frames then each frame will be built from some number of `metasprite_t` items (which may vary based on how many sprites are required for that particular frame). A metasprite frame is terminated with a `{metasprite_end}` entry.

20.104.3 Function Documentation

20.104.3.1 hide_sprites_range() `void hide_sprites_range (`
`uint8_t` from,

```
uint8_t to )
```

Hides all hardware sprites in range from $\leq X < to$

Parameters

<i>from</i>	start OAM index
<i>to</i>	finish OAM index

20.104.3.2 move metasprite_ex() `uint8_t move metasprite_ex (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_prop,`
`uint8_t base_sprite,`
`uint8_t x,`
`uint8_t y ) [inline]`

Moves metasprite to the absolute position **x** and **y**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (unused on this platform)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Moves **metasprite** to the absolute position **x** and **y** (with **no flip** on the X or Y axis). Hardware sprites are allocated starting from **base_sprite**, using tiles starting from **base_tile**.

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Returns

Number of hardware sprites used to draw this metasprite

20.104.3.3 move metasprite() `uint8_t move metasprite (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_sprite,`
`uint8_t x,`
`uint8_t y ) [inline]`

Obsolete

20.104.3.4 hide metasprite() `void hide metasprite (`
`const metasprite_t * metasprite,`
`uint8_t base_sprite ) [inline]`

Hides a metasprite from the screen

Parameters

<i>metasprite</i>	Pointer to first struct of the desired metasprite frame
<i>base_sprite</i>	Number of hardware sprite to start with

Sets:

- `__current metasprite = metasprite;`

20.104.4 Variable Documentation

20.104.4.1 `__current metasprite` `const void* __current metasprite` [extern]

20.104.4.2 `__current base tile` `uint8_t __current base tile` [extern]

20.104.4.3 `__render shadow_OAM` `uint8_t __render shadow_OAM` [extern]

20.104.4.4 `iy1` `uint8_t iy1`

Initial value:

```
{
    __asm__("ei")
}
```

20.105 metasprites.h

[Go to the documentation of this file.](#)

```
1
14 #ifndef _METASPRITES_H_INCLUDE
15 #define _METASPRITES_H_INCLUDE
16
17 #include <msx/hardware.h>
18 #include <types.h>
19 #include <stdint.h>
20
34 typedef struct metasprite_t {
35     int8_t dy, dx;
36     uint8_t dtile;
37     uint8_t props;
38 } metasprite_t;
39
40 #define metasprite_end -128
41 #define METASPR_ITEM(dy,dx,dt,a) {(dy),(dx),(dt),(a)}
42 #define METASPR_TERM {metasprite_end}
43
44 extern const void * __current metasprite;
45 extern uint8_t __current base tile;
46 extern uint8_t __render shadow_OAM;
47
48
49 static uint8_t __move metasprite(uint8_t id, uint8_t x, uint8_t y) Z88DK_CALLEE PRESERVES_REGS(iyh, iy1);
50 static void __hide metasprite(uint8_t id) Z88DK_FASTCALL PRESERVES_REGS(iyh, iy1);
51
57 void hide_sprites_range(uint8_t from, uint8_t to) Z88DK_CALLEE PRESERVES_REGS(iyh, iy1);
58
79 inline uint8_t move metasprite_ex(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_prop,
    uint8_t base_sprite, uint8_t x, uint8_t y) {
80     base_prop;
81     __current metasprite = metasprite;
82     __current base tile = base_tile;
83     return __move metasprite(base_sprite, x, y);
84 }
85
88 inline uint8_t move metasprite(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_sprite,
    uint8_t x, uint8_t y) {
89     __current metasprite = metasprite;
90     __current base tile = base_tile;
91     return __move metasprite(base_sprite, x, y);
92 }
93
94
104 inline void hide metasprite(const metasprite_t * metasprite, uint8_t base_sprite) {
105     __current metasprite = metasprite;
106     __hide metasprite(base_sprite);
107 }
108
109 #endif
```

20.106 gbdk-lib/include/nes/metasprites.h File Reference

```
#include <nes/hardware.h>
#include <types.h>
#include <stdint.h>
```

Data Structures

- struct [metasprite_t](#)

Macros

- #define [metasprite_end](#) -128
- #define [METASPR_ITEM](#)(dy, dx, dt, a) {(dy),(dx),(dt),(a)}
- #define [METASPR_TERM](#) {metasprite_end}

Typedefs

- typedef struct [metasprite_t](#) [metasprite_t](#)

Functions

- void [hide_sprites_range](#) (uint8_t from, uint8_t to) [OLDCALL](#)
- uint8_t [move_metasprite_ex](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_flipx](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_vflip](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_flipy](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_hflip](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_flipxy](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_hvflip](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_sprite, int16_t x, int16_t y)
- void [hide_metasprite](#) (const [metasprite_t](#) *metasprite, uint8_t base_sprite)

Variables

- const void * [__current_metasprite](#)
- uint8_t [__current_base_tile](#)
- uint8_t [__current_base_prop](#)
- uint8_t [__render_shadow_OAM](#)

20.106.1 Detailed Description

20.106.2 Metasprite support

A metasprite is a larger sprite made up from a collection of smaller individual hardware sprites. Different frames of the same metasprites can share tile data.

See the main [metasprite docs](#) under the game Boy platform for additional details.

20.106.3 Macro Definition Documentation

20.106.3.1 metasprite_end `#define metasprite_end -128`

20.106.3.2 METASPR_ITEM `#define METASPR_ITEM(
 dy,
 dx,
 dt,
 a) { (dy), (dx), (dt), (a) }`

20.106.3.3 METASPR_TERM `#define METASPR_TERM {metasprite_end}`

20.106.4 Typedef Documentation

20.106.4.1 metasprite_t `typedef struct metasprite_t metasprite_t`
 Metasprite sub-item structure

Parameters

<i>dy</i>	(int8_t) Y coordinate of the sprite relative to the metasprite origin (pivot)
<i>dx</i>	(int8_t) X coordinate of the sprite relative to the metasprite origin (pivot)
<i>dtile</i>	(uint8_t) Start tile relative to the metasprites own set of tiles
<i>props</i>	(uint8_t) Property Flags

Metasprites are built from multiple [metasprite_t](#) items (one for each sub-sprite) and a pool of tiles they reference. If a metasprite has multiple frames then each frame will be built from some number of [metasprite_t](#) items (which may vary based on how many sprites are required for that particular frame). A metasprite frame is terminated with a {metasprite_end} entry.

20.106.5 Function Documentation

20.106.5.1 hide_sprites_range() `void hide_sprites_range (
 uint8_t from,
 uint8_t to)`

Hides all hardware sprites in range from <= X < to

Parameters

<i>from</i>	start OAM index
<i>to</i>	finish OAM index

20.106.5.2 move_metasprite_ex() `uint8_t move_metasprite_ex (
 const metasprite_t * metasprite,
 uint8_t base_tile,
 uint8_t base_prop,
 uint8_t base_sprite,
 int16_t x,
 int16_t y) [inline]`

Moves metasprite to the absolute position x and y

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Moves **metasprite** to the absolute position **x** and **y** (with **no flip** on the X or Y axis). Hardware sprites are allocated starting from **base_sprite**, using tiles starting from **base_tile**.

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as palette), see [Metasprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

20.106.5.3 `move_metasprite()` `uint8_t` `move_metasprite` (

```

    const metasprite_t * metasprite,
    uint8_t base_tile,
    uint8_t base_sprite,
    int16_t x,
    int16_t y ) [inline]
```

Obsolete. Replaced by [move_metasprite_ex\(\)](#)

20.106.5.4 `move_metasprite_flipx()` `uint8_t` `move_metasprite_flipx` (

```

    const metasprite_t * metasprite,
    uint8_t base_tile,
    uint8_t base_prop,
    uint8_t base_sprite,
    int16_t x,
    int16_t y ) [inline]
```

Moves metasprite to the absolute position x and y, **flipped by X (horizontally)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move_metasprite\(\)](#), but with the metasprite flipped by X (horizontally).

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as palette), see [Metasprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

See also

[move_metasprite\(\)](#)

20.106.5.5 move_metasprite_vflip() `uint8_t move_metasprite_vflip (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Obsolete. Replaced by [move_metasprite_flipx\(\)](#)

20.106.5.6 move_metasprite_flipy() `uint8_t move_metasprite_flipy (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_prop,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Moves metasprite to the absolute position x and y, **flipped by Y (vertically)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move_metasprite\(\)](#), but with the metasprite flipped by Y (vertically).

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as palette), see [Metasprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

See also

[move_metasprite\(\)](#)

20.106.5.7 move_metasprite_hflip() `uint8_t move_metasprite_hflip (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Obsolete. Replaced by [move_metasprite_flipy\(\)](#)

20.106.5.8 move metasprite_flipxy() `uint8_t move metasprite_flipxy (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_prop,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Moves metasprite to the absolute position x and y, **flipped by X and Y (horizontally and vertically)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move metasprite\(\)](#), but with the metasprite flipped by X and Y (horizontally and vertically).
 Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as palette), see [Metasprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

See also

[move metasprite\(\)](#)

20.106.5.9 move metasprite_hvflip() `uint8_t move metasprite_hvflip (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Obsolete. Replaced by [move metasprite_flipxy\(\)](#)

20.106.5.10 hide metasprite() `void hide metasprite (`
`const metasprite_t * metasprite,`
`uint8_t base_sprite ) [inline]`

Hides a metasprite from the screen

Parameters

<i>metasprite</i>	Pointer to first struct of the desired metasprite frame
<i>base_sprite</i>	Number of hardware sprite to start with

Sets:

- `__current_metasprite = metasprite;`

20.106.6 Variable Documentation

20.106.6.1 `__current_metasprite` `const void* __current_metasprite` [extern]

20.106.6.2 `__current_base_tile` `uint8_t __current_base_tile` [extern]

20.106.6.3 `__current_base_prop` `uint8_t __current_base_prop` [extern]

20.106.6.4 `__render_shadow_OAM` `uint8_t __render_shadow_OAM` [extern]

20.107 metasprites.h

[Go to the documentation of this file.](#)

```

1
14 #ifndef _METASPRITES_H_INCLUDE
15 #define _METASPRITES_H_INCLUDE
16
17 #include <nes/hardware.h>
18 #include <types.h>
19 #include <stdint.h>
20
34 typedef struct metasprite_t {
35     int8_t dy, dx;
36     uint8_t dtile;
37     uint8_t props;
38 } metasprite_t;
39
40 #define metasprite_end -128
41 #define METASPR_ITEM(dy,dx,dt,a) {(dy),(dx),(dt),(a)}
42 #define METASPR_TERM {metasprite_end}
43
44 extern const void * __current_metasprite;
45 extern uint8_t __current_base_tile;
46 extern uint8_t __current_base_prop;
47 extern uint8_t __render_shadow_OAM;
48
49
50 static uint8_t __move_metasprite(uint8_t id, int16_t x, int16_t y) OLDCALL;
51 static uint8_t __move_metasprite_flipx(uint8_t id, int16_t x, int16_t y) OLDCALL;
52 static uint8_t __move_metasprite_flipy(uint8_t id, int16_t x, int16_t y) OLDCALL;
53 static uint8_t __move_metasprite_flipxy(uint8_t id, int16_t x, int16_t y) OLDCALL;
54 static uint8_t __move_metasprite_vflip(uint8_t id, int16_t x, int16_t y) OLDCALL;
55 static uint8_t __move_metasprite_hflip(uint8_t id, int16_t x, int16_t y) OLDCALL;
56 static uint8_t __move_metasprite_hvflip(uint8_t id, int16_t x, int16_t y) OLDCALL;
57 static void __hide_metasprite(uint8_t id) OLDCALL;
58
64 void hide_sprites_range(uint8_t from, uint8_t to) OLDCALL;
65
89 inline uint8_t move_metasprite_ex(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_prop,
    uint8_t base_sprite, int16_t x, int16_t y) {
90     base_prop;
91     __current_metasprite = metasprite;
92     __current_base_tile = base_tile;
93     __current_base_prop = base_prop;
94     return __move_metasprite(base_sprite, x, y);
95 }
96
99 inline uint8_t move_metasprite(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_sprite,
    int16_t x, int16_t y) {
100     __current_metasprite = metasprite;
101     __current_base_tile = base_tile;
102     __current_base_prop = 0;
103     return __move_metasprite(base_sprite, x, y);
104 }
105
128 inline uint8_t move_metasprite_flipx(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_prop, uint8_t base_sprite, int16_t x, int16_t y) {
129     base_prop;
130     __current_metasprite = metasprite;
131     __current_base_tile = base_tile;
132     __current_base_prop = base_prop;
133     return __move_metasprite_flipx(base_sprite, x - 8, y);
134 }

```

```

135
138 inline uint8_t move metasprite_vflip(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, int16_t x, int16_t y) {
139     __current_metasprite = metasprite;
140     __current_base_tile = base_tile;
141     __current_base_prop = 0;
142     return __move_metasprite_vflip(base_sprite, x - 8, y);
143 }
144
145
168 inline uint8_t move metasprite_flipy(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, uint8_t base_prop, int16_t x, int16_t y) {
169     base_prop;
170     __current_metasprite = metasprite;
171     __current_base_tile = base_tile;
172     __current_base_prop = base_prop;
173     return __move_metasprite_flipy(base_sprite, x, y - ((shadow_PPUCTRL & PPUCTRL_SPR_8X16) ? 16 : 8) );
174 }
175
178 inline uint8_t move metasprite_hflip(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, int16_t x, int16_t y) {
179     __current_metasprite = metasprite;
180     __current_base_tile = base_tile;
181     __current_base_prop = 0;
182     return __move_metasprite_hflip(base_sprite, x, y - ((shadow_PPUCTRL & PPUCTRL_SPR_8X16) ? 16 : 8) );
183 }
184
207 inline uint8_t move metasprite_flipxy(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, uint8_t base_prop, int16_t x, int16_t y) {
208     base_prop;
209     __current_metasprite = metasprite;
210     __current_base_tile = base_tile;
211     __current_base_prop = base_prop;
212     return __move_metasprite_flipxy(base_sprite, x - 8, y - ((shadow_PPUCTRL & PPUCTRL_SPR_8X16) ? 16 :
    8));
213 }
214
217 inline uint8_t move metasprite_hvflip(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_sprite, int16_t x, int16_t y) {
218     __current_metasprite = metasprite;
219     __current_base_tile = base_tile;
220     __current_base_prop = 0;
221     return __move_metasprite_hvflip(base_sprite, x - 8, y - ((shadow_PPUCTRL & PPUCTRL_SPR_8X16) ? 16 :
    8));
222 }
223
233 inline void hide_metasprite(const metasprite_t * metasprite, uint8_t base_sprite) {
234     __current_metasprite = metasprite;
235     __hide_metasprite(base_sprite);
236 }
237
238 #endif

```

20.108 gbdk-lib/include/sms/metasprires.h File Reference

```

#include <sms/sms.h>
#include <sms/hardware.h>
#include <types.h>
#include <stdint.h>

```

Data Structures

- struct [metasprite_t](#)

Macros

- #define [metasprite_end](#) -128
- #define [METASPR_ITEM](#)(dy, dx, dt, a) {(dy),(dx),(dt)}
- #define [METASPR_TERM](#) {metasprite_end}

Typedefs

- typedef struct [metasprite_t](#) [metasprite_t](#)

Functions

- void [hide_sprites_range](#) (uint8_t from, uint8_t to) [PRESERVES_REGS](#)(iyh
- uint8_t [move_metasprite_ex](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_flipx](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_flipy](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- uint8_t [move_metasprite_flipxy](#) (const [metasprite_t](#) *metasprite, uint8_t base_tile, uint8_t base_prop, uint8_t base_sprite, int16_t x, int16_t y)
- void [hide_metasprite](#) (const [metasprite_t](#) *metasprite, uint8_t base_sprite)

Variables

- const void * [__current_metasprite](#)
- uint8_t [__current_base_tile](#)
- uint8_t [__render_shadow_OAM](#)
- static void [iyl](#)

20.108.1 Detailed Description

20.108.2 Metasprite support

A metasprite is a larger sprite made up from a collection of smaller individual hardware sprites. Different frames of the same metasprites can share tile data.

See the main [metasprite docs](#) under the game Boy platform for additional details.

20.108.3 Metasprite support

A metasprite is a larger sprite made up from a collection of smaller individual hardware sprites. Different frames of the same metasprites can share tile data.

See the main [metasprite docs](#) under the game Boy platform for additional details.

20.108.4 Macro Definition Documentation

20.108.4.1 [metasprite_end](#) `#define metasprite_end -128`

20.108.4.2 [METASPR_ITEM](#) `#define METASPR_ITEM(
 dy,
 dx,
 dt,
 a) { (dy), (dx), (dt) }`

20.108.4.3 [METASPR_TERM](#) `#define METASPR_TERM {metasprite_end}`

20.108.5 Typedef Documentation

20.108.5.1 [metasprite_t](#) `typedef struct metasprite\_t metasprite_t`
 Metasprite sub-item structure

Parameters

<i>dy</i>	(int8_t) Y coordinate of the sprite relative to the metasprite origin (pivot)
<i>dx</i>	(int8_t) X coordinate of the sprite relative to the metasprite origin (pivot)
<i>dtile</i>	(uint8_t) Start tile relative to the metasprites own set of tiles

Metasprites are built from multiple [metasprite_t](#) items (one for each sub-sprite) and a pool of tiles they reference. If a metasprite has multiple frames then each frame will be built from some number of [metasprite_t](#) items (which may vary based on how many sprites are required for that particular frame).

A metasprite frame is terminated with a {metasprite_end} entry.

20.108.6 Function Documentation

20.108.6.1 hide_sprites_range() void hide_sprites_range (
 uint8_t from,
 uint8_t to)

Hides all hardware sprites in range from $\leq X < to$

Parameters

<i>from</i>	start OAM index
<i>to</i>	finish OAM index

20.108.6.2 move_metasprite_ex() uint8_t move_metasprite_ex (
 const metasprite_t * metasprite,
 uint8_t base_tile,
 uint8_t base_prop,
 uint8_t base_sprite,
 int16_t x,
 int16_t y) [inline]

Moves metasprite to the absolute position x and y

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (unused on this platform)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Moves **metasprite** to the absolute position **x** and **y** (with **no flip** on the X or Y axis). Hardware sprites are allocated starting from **base_sprite**, using tiles starting from **base_tile**.

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Returns

Number of hardware sprites used to draw this metasprite

20.108.6.3 move_metasprite() `uint8_t move_metasprite (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Obsolete. This function has been replaced by [move_metasprite_ex\(\)](#)

20.108.6.4 move_metasprite_flipx() `uint8_t move_metasprite_flipx (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_prop,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Moves metasprite to the absolute position x and y, **flipped by X (horizontally)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (unused on this platform)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move_metasprite\(\)](#), but with the metasprite flipped by X (horizontally).

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as palette), see [Metaspprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

See also

[move_metasprite\(\)](#)

20.108.6.5 move_metasprite_flipy() `uint8_t move_metasprite_flipy (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_prop,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Moves metasprite to the absolute position x and y, **flipped by Y (vertically)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (unused on this platform)

Parameters

<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move metasprite\(\)](#), but with the metasprite flipped by Y (vertically).

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as palette), see [Metasprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

See also

[move metasprite\(\)](#)

20.108.6.6 move_metasprite_flipxy() `uint8_t move_metasprite_flipxy (`
`const metasprite_t * metasprite,`
`uint8_t base_tile,`
`uint8_t base_prop,`
`uint8_t base_sprite,`
`int16_t x,`
`int16_t y ) [inline]`

Moves metasprite to the absolute position x and y, **flipped by X and Y (horizontally and vertically)**

Parameters

<i>metasprite</i>	Pointer to the first struct of the metasprite (for the desired frame)
<i>base_tile</i>	Number of the first tile where the metasprite's tiles start
<i>base_prop</i>	Base sprite property flags (unused on this platform)
<i>base_sprite</i>	Number of the first hardware sprite to be used by the metasprite
<i>x</i>	Absolute x coordinate of the sprite
<i>y</i>	Absolute y coordinate of the sprite

Same as [move metasprite\(\)](#), but with the metasprite flipped by X and Y (horizontally and vertically).

Sets:

- `__current_metasprite = metasprite;`
- `__current_base_tile = base_tile;`

Note: Overwrites OAM sprite properties (such as palette), see [Metasprites and sprite properties](#).

Returns

Number of hardware sprites used to draw this metasprite

See also

[move metasprite\(\)](#)

20.108.6.7 hide_metasprite() void hide_metasprite (
 const metasprite_t * metasprite,
 uint8_t base_sprite) [inline]

Hides a metasprite from the screen

Parameters

<i>metasprite</i>	Pointer to first struct of the desired metasprite frame
<i>base_sprite</i>	Number of hardware sprite to start with

Sets:

- __current_metasprite = metasprite;

20.108.7 Variable Documentation

20.108.7.1 __current_metasprite const void* __current_metasprite [extern]

20.108.7.2 __current_base_tile uint8_t __current_base_tile [extern]

20.108.7.3 __render_shadow_OAM uint8_t __render_shadow_OAM [extern]

20.108.7.4 iyl void iyl

20.109 metasprites.h

[Go to the documentation of this file.](#)

```

1
14 #ifndef _METASPRITES_H_INCLUDE
15 #define _METASPRITES_H_INCLUDE
16
17 #include <sms/sms.h>
18 #include <sms/hardware.h>
19 #include <types.h>
20 #include <stdint.h>
21
22 typedef struct metasprite_t {
23     int8_t dy, dx;
24     uint8_t dtile;
25 } metasprite_t;
26
27 #define metasprite_end -128
28 #define METASPR_ITEM(dy,dx,dt,a) {(dy),(dx),(dt)}
29 #define METASPR_TERM {metasprite_end}
30
31 extern const void * __current_metasprite;
32 extern uint8_t __current_base_tile;
33 extern uint8_t __render_shadow_OAM;
34
35
36 static uint8_t __move_metasprite(uint8_t id, int16_t x, int16_t y);
37 static uint8_t __move_metasprite_flipx(uint8_t id, int16_t x, int16_t y);
38 static uint8_t __move_metasprite_flipy(uint8_t id, int16_t x, int16_t y);
39 static uint8_t __move_metasprite_flipxy(uint8_t id, int16_t x, int16_t y);
40 static void __hide_metasprite(uint8_t id) Z88DK_FASTCALL PRESERVES_REGS(iyh, iyl);
41
42 void hide_sprites_range(uint8_t from, uint8_t to) PRESERVES_REGS(iyh, iyl);
43
44 inline uint8_t move_metasprite_ex(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_prop,
45     uint8_t base_sprite, int16_t x, int16_t y) {
46     base_prop;
47     __current_metasprite = metasprite;
48     __current_base_tile = base_tile;
49     return __move_metasprite(base_sprite, x, y);
50 }

```

```

87
90 inline uint8_t move metasprite(const metasprite_t * metasprite, uint8_t base_tile, uint8_t base_sprite,
    int16_t x, int16_t y) {
91     __current_metasprite = metasprite;
92     __current_base_tile = base_tile;
93     return __move_metasprite(base_sprite, x, y);
94 }
95
118 inline uint8_t move_metasprite_flipx(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_prop, uint8_t base_sprite, int16_t x, int16_t y) {
119     base_prop;
120     __current_metasprite = metasprite;
121     __current_base_tile = base_tile;
122     return __move_metasprite_flipx(base_sprite, x - 8, y);
123 }
124
147 inline uint8_t move_metasprite_flipy(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_prop, uint8_t base_sprite, int16_t x, int16_t y) {
148     base_prop;
149     __current_metasprite = metasprite;
150     __current_base_tile = base_tile;
151     return __move_metasprite_flipy(base_sprite, x, y - ((__READ_VDP_REG(VDP_R1) & R1_SPR_8X16) ? 16 : 8)
    );
152 }
153
176 inline uint8_t move_metasprite_flipxy(const metasprite_t * metasprite, uint8_t base_tile, uint8_t
    base_prop, uint8_t base_sprite, int16_t x, int16_t y) {
177     base_prop;
178     __current_metasprite = metasprite;
179     __current_base_tile = base_tile;
180     return __move_metasprite_flipxy(base_sprite, x - 8, y - ((__READ_VDP_REG(VDP_R1) & R1_SPR_8X16) ? 16
    : 8));
181 }
182
192 inline void hide_metasprite(const metasprite_t * metasprite, uint8_t base_sprite) {
193     __current_metasprite = metasprite;
194     __hide_metasprite(base_sprite);
195 }
196
197 #endif

```

20.110 gbdk-lib/include/msx/msx.h File Reference

```

#include <types.h>
#include <stdint.h>
#include <gbdk/version.h>
#include <msx/hardware.h>

```

Data Structures

- struct [joypads_t](#)
- struct [OAM_item_t](#)

Macros

- #define [MSX](#)
- #define [SYSTEM_60HZ](#) 0x00
- #define [SYSTEM_50HZ](#) 0x01
- #define [VBK_REG_VDP_ATTR_SHIFT](#)
- #define [J_UP](#) 0b00100000
- #define [J_DOWN](#) 0b01000000
- #define [J_LEFT](#) 0b00010000
- #define [J_RIGHT](#) 0b10000000
- #define [J_A](#) 0b00000001
- #define [J_B](#) 0b00000100
- #define [J_SELECT](#) 0b00001000
- #define [J_START](#) 0b00000010
- #define [M_TEXT_OUT](#) 0x02U
- #define [M_TEXT_INOUT](#) 0x03U
- #define [M_NO_SCROLL](#) 0x04U

- #define `M_NO_INTERP` 0x08U
- #define `S_BANK` 0x01U
- #define `S_FLIPX` 0x02U
- #define `S_FLIPY` 0x04U
- #define `S_PALETTE` 0x08U
- #define `S_PRIORITY` 0x10U
- #define `S_PAL`(n) (((n) & 0x01U) << 3)
- #define `__WRITE_VDP_REG_UNSAFE`(REG, v) shadow_##REG=(v),VDP_CMD=(shadow_##REG),VDP←
_CMD=REG
- #define `__WRITE_VDP_REG`(REG, v) shadow_##REG=(v);__asm__("di");VDP_CMD=(shadow_←
_##REG);VDP_CMD=REG;__asm__("ei")
- #define `__READ_VDP_REG`(REG) shadow_##REG
- #define `EMPTY_IFLAG` 0x00U
- #define `VBL_IFLAG` 0x01U
- #define `LCD_IFLAG` 0x02U
- #define `TIM_IFLAG` 0x04U
- #define `SIO_IFLAG` 0x08U
- #define `JOY_IFLAG` 0x10U
- #define `SCREENWIDTH` `DEVICE_SCREEN_PX_WIDTH`
- #define `SCREENHEIGHT` `DEVICE_SCREEN_PX_HEIGHT`
- #define `MINWNDPOSX` 0x00U
- #define `MINWNDPOSY` 0x00U
- #define `MAXWNDPOSX` 0x00U
- #define `MAXWNDPOSY` 0x00U
- #define `DISPLAY_ON` `__WRITE_VDP_REG`(VDP_R1, `__READ_VDP_REG`(VDP_R1) |= R1_DISP_ON)
- #define `DISPLAY_OFF` `display_off`();
- #define `HIDE_LEFT_COLUMN` `__WRITE_VDP_REG`(VDP_R0, `__READ_VDP_REG`(VDP_R0) |= R0_LCB)
- #define `SHOW_LEFT_COLUMN` `__WRITE_VDP_REG`(VDP_R0, `__READ_VDP_REG`(VDP_R0) &=
- `(~R0_LCB))`
- #define `SET_BORDER_COLOR`(C) `__WRITE_VDP_REG`(VDP_R7, ((C) | 0xf0u))
- #define `SHOW_BKG`
- #define `HIDE_BKG`
- #define `SHOW_WIN`
- #define `HIDE_WIN`
- #define `SHOW_SPRITES`
- #define `HIDE_SPRITES`
- #define `SPRITES_16x16` `__WRITE_VDP_REG`(VDP_R1, `__READ_VDP_REG`(VDP_R1) |= R1_SPR_16X16)
- #define `SPRITES_8x8` `__WRITE_VDP_REG`(VDP_R1, `__READ_VDP_REG`(VDP_R1) &= (~R1_SPR_16X16))
- #define `DEVICE_SUPPORTS_COLOR` (TRUE)
- #define `VBL_DONE` `_vbl_done`
- #define `DIV_REG` `get_r_reg`()
- #define `CURRENT_BANK` `_current_bank`
- #define `BANK`(VARNAME) ((uint8_t) & __bank_ ## VARNAME)
- #define `BANKREF`(VARNAME)
- #define `BANKREF_EXTERN`(VARNAME) extern const void __bank_ ## VARNAME;
- #define `SWITCH_ROM1` `SWITCH_ROM`
- #define `SWITCH_ROM2`(b) `MAP_FRAME2`=(b)
- #define `SWITCH_RAM`(b) `RAM_CONTROL`=(((b)&1)?`RAM_CONTROL`|`RAMCTL_BANK`:`RAM_CONTROL`&(~`RAMCTL_BANK`))
- #define `ENABLE_RAM` `RAM_CONTROL`|=`RAMCTL_RAM`
- #define `DISABLE_RAM` `RAM_CONTROL`&=(~`RAMCTL_RAM`)
- #define `set_bkg_palette_entry` `set_palette_entry`
- #define `set_sprite_palette_entry`(palette, entry, rgb_data) `set_palette_entry`(1,entry,rgb_data)
- #define `set_bkg_palette` `set_palette`
- #define `set_sprite_palette`(first_palette, nb_palettes, rgb_data) `set_palette`(1,1,rgb_data)

- `#define COMPAT_PALETTE(C0, C1, C2, C3) (((uint16_t)(C3) << 12) | ((uint16_t)(C2) << 8) | ((uint16_t)(C1) << 4) | (uint16_t)(C0))`
- `#define set_bkg_tiles set_tile_map`
- `#define set_win_tiles set_tile_map`
- `#define fill_bkg_rect fill_rect`
- `#define fill_win_rect fill_rect`
- `#define DISABLE_VBL_TRANSFER _shadow_OAM_base = 0`
- `#define ENABLE_VBL_TRANSFER _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)`
- `#define MAX_HARDWARE_SPRITES 32`
- `#define HARDWARE_SPRITE_CAN_FLIP_X 0`
- `#define HARDWARE_SPRITE_CAN_FLIP_Y 0`
- `#define set_bkg_tile_xy set_tile_xy`
- `#define set_win_tile_xy set_tile_xy`
- `#define get_win_xy_addr get_bkg_xy_addr`

Typedefs

- `typedef void(* int_handler) (void) NONBANKED`
- `typedef struct OAM_item_t OAM_item_t`

Functions

- `void WRITE_VDP_CMD (uint16_t cmd) Z88DK_FASTCALL PRESERVES_REGS(b)`
- `void WRITE_VDP_DATA (uint16_t data) Z88DK_FASTCALL PRESERVES_REGS(b)`
- `void mode (uint8_t m) OLDCALL`
- `uint8_t get_mode (void) OLDCALL`
- `uint8_t get_system (void)`
- `void set_interrupts (uint8_t flags) Z88DK_FASTCALL`
- `void remove_VBL (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(iyh)`
- `void remove_LCD (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b)`
- `void remove_TIM (int_handler h) Z88DK_FASTCALL`
- `void remove_SIO (int_handler h) Z88DK_FASTCALL`
- `void remove_JOY (int_handler h) Z88DK_FASTCALL`
- `void add_VBL (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(d)`
- `void add_LCD (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b)`
- `void add_TIM (int_handler h) Z88DK_FASTCALL`
- `void add_SIO (int_handler h) Z88DK_FASTCALL`
- `void add_JOY (int_handler h) Z88DK_FASTCALL`
- `uint8_t cancel_pending_interrupts (void)`
- `void move_bkg (uint8_t x, uint8_t y)`
- `void scroll_bkg (int8_t x, int8_t y)`
- `void vsync (void) PRESERVES_REGS(b)`
- `void wait_vbl_done (void) PRESERVES_REGS(b)`
- `void display_off (void)`
- `void refresh_OAM (void)`
- `uint8_t get_r_reg (void) PRESERVES_REGS(b)`
- `void SWITCH_ROM (uint8_t bank) Z88DK_FASTCALL PRESERVES_REGS(b)`
- `void delay (uint16_t d) Z88DK_FASTCALL`
- `uint8_t joypad (void) OLDCALL PRESERVES_REGS(b)`
- `uint8_t waitpad (uint8_t mask) Z88DK_FASTCALL PRESERVES_REGS(b)`
- `void waitpadup (void) PRESERVES_REGS(b)`
- `uint8_t joypad_init (uint8_t npads, joypads_t *joypads) Z88DK_CALLEE`
- `void joypad_ex (joypads_t *joypads) Z88DK_FASTCALL PRESERVES_REGS(iyh)`
- `void enable_interrupts (void) PRESERVES_REGS(a)`
- `void disable_interrupts (void) PRESERVES_REGS(a)`

- void [set_default_palette](#) (void)
- void [cpu_fast](#) (void)
- void [set_palette_entry](#) (uint8_t palette, uint8_t entry, uint16_t rgb_data) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [set_palette](#) (uint8_t first_palette, uint8_t nb_palettes, const [palette_color_t](#) *rgb_data) [Z88DK_CALLEE](#)
- void [set_native_tile_data](#) (uint16_t start, uint16_t ntiles, const void *src) [Z88DK_CALLEE](#)
- void [set_bkg_4bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_sprite_1bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src) [Z88DK_CALLEE](#)
- void [set_native_sprite_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_2bpp_palette](#) (uint16_t palette)
- void [set_bkg_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_sprite_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_1bpp_colors](#) (uint8_t fgcolor, uint8_t bgcolor)
- void [set_tile_1bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src, uint16_t colors) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [set_bkg_1bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_data](#) (uint16_t dst, const void *src, uint16_t size) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [vmemcpy](#) (uint16_t dst, const void *src, uint16_t size) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [set_tile_map](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [set_bkg_based_tiles](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles, uint8_t base_tile)
- void [set_win_based_tiles](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles, uint8_t base_tile)
- void [set_tile_submap](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t map_w, const uint8_t *map) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [set_tile_submap_compat](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t map_w, const uint8_t *map) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [set_bkg_submap](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)
- void [set_win_submap](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)
- void [set_bkg_based_submap](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w, uint8_t base_tile)
- void [set_win_based_submap](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w, uint8_t base_tile)
- void [fill_rect](#) (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint16_t tile) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- void [SET_SHADOW_OAM_ADDRESS](#) (void *address)
- void [set_sprite_tile](#) (uint8_t nb, uint8_t tile)
- [uint8_t](#) [get_sprite_tile](#) (uint8_t nb)
- void [set_sprite_prop](#) (uint8_t nb, uint8_t prop)
- [uint8_t](#) [get_sprite_prop](#) (uint8_t nb)
- void [move_sprite](#) (uint8_t nb, uint8_t x, uint8_t y)
- void [scroll_sprite](#) (uint8_t nb, int8_t x, int8_t y)
- void [hide_sprite](#) (uint8_t nb)
- void [set_vram_byte](#) (uint8_t *addr, uint8_t v) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- [uint8_t](#) * [set_attributed_tile_xy](#) (uint8_t x, uint8_t y, uint16_t t) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- [uint8_t](#) * [set_tile_xy](#) (uint8_t x, uint8_t y, uint8_t t) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)
- [uint8_t](#) * [get_bkg_xy_addr](#) (uint8_t x, uint8_t y) [Z88DK_CALLEE PRESERVES_REGS\(iyh](#)

Variables

- const [uint8_t](#) [_SYSTEM](#)
- void [c](#)
- void [d](#)
- void [e](#)
- void [iyh](#)
- void [iyl](#)
- void [h](#)
- void [l](#)

- volatile [uint16_t sys_time](#)
- volatile [uint8_t _vbl_done](#)
- volatile [uint8_t _current_bank](#)
- void [b](#)
- [uint16_t _current_2bpp_palette](#)
- [uint16_t _current_1bpp_colors](#)
- [uint8_t _map_tile_offset](#)
- [uint8_t _submap_tile_offset](#)
- volatile struct [OAM_item_t shadow_OAM](#) []
- volatile [uint8_t _shadow_OAM_base](#)
- volatile [uint8_t _shadow_OAM_OFF](#)

20.110.1 Detailed Description

MSX specific functions.

20.110.2 Macro Definition Documentation

20.110.2.1 MSX `#define MSX`

20.110.2.2 SYSTEM_60HZ `#define SYSTEM_60HZ 0x00`

20.110.2.3 SYSTEM_50HZ `#define SYSTEM_50HZ 0x01`

20.110.2.4 VBK_REG `#define VBK_REG VDP\_ATTR\_SHIFT`

20.110.2.5 J_UP `#define J_UP 0b00100000`

Joypad bits. A logical OR of these is used in the `wait_pad` and `joypad` functions. For example, to see if the B button is pressed try

```
uint8_t keys; keys = joypad\(\); if (keys & J_B) { ... }
```

See also

[joypad](#)

20.110.2.6 J_DOWN `#define J_DOWN 0b01000000`

20.110.2.7 J_LEFT `#define J_LEFT 0b00010000`

20.110.2.8 J_RIGHT `#define J_RIGHT 0b10000000`

20.110.2.9 J_A `#define J_A 0b00000001`

20.110.2.10 J_B `#define J_B 0b00000100`

20.110.2.11 J_SELECT `#define J_SELECT 0b00001000`

20.110.2.12 J_START `#define J_START 0b00000010`

20.110.2.13 M_TEXT_OUT `#define M_TEXT_OUT 0x02U`

Screen modes. Normally used by internal functions only.

See also

[mode\(\)](#)

20.110.2.14 M_TEXT_INOUT `#define M_TEXT_INOUT 0x03U`

20.110.2.15 M_NO_SCROLL `#define M_NO_SCROLL 0x04U`

Set this in addition to the others to disable scrolling

If scrolling is disabled, the cursor returns to (0,0)

See also

[mode\(\)](#)

20.110.2.16 M_NO_INTERP `#define M_NO_INTERP 0x08U`

Set this to disable interpretation

See also

[mode\(\)](#)

20.110.2.17 S_BANK `#define S_BANK 0x01U`

The ninth bit of the tile id

20.110.2.18 S_FLIPX `#define S_FLIPX 0x02U`

If set the background tile will be flipped horizontally.

20.110.2.19 S_FLIPY `#define S_FLIPY 0x04U`

If set the background tile will be flipped vertically.

20.110.2.20 S_PALETTE `#define S_PALETTE 0x08U`

If set the background tile palette.

20.110.2.21 S_PRIORITY `#define S_PRIORITY 0x10U`

If set the background tile priority.

20.110.2.22 S_PAL `#define S_PAL(
n) ((n) & 0x01U) << 3)`

Defines how palette number is encoded in OAM. Required for the png2asset tool's metasprite output.

20.110.2.23 __WRITE_VDP_REG_UNSAFE `#define __WRITE_VDP_REG_UNSAFE(
REG,`

`v ) shadow_##REG=(v), VDP_CMD=(shadow_##REG), VDP_CMD=REG`

20.110.2.24 **__WRITE_VDP_REG** `#define __WRITE_VDP_REG(
 REG,
 v) shadow_##REG=(v);__asm__("di");VDP_CMD=(shadow_##REG);VDP_CMD=REG;__asm__↵
 ("ei")`

20.110.2.25 **__READ_VDP_REG** `#define __READ_VDP_REG(
 REG) shadow_##REG`

20.110.2.26 **EMPTY_IFLAG** `#define EMPTY_IFLAG 0x00U`
Disable calling of interrupt service routines

20.110.2.27 **VBL_IFLAG** `#define VBL_IFLAG 0x01U`
VBlank Interrupt occurs at the start of the vertical blank.
During this period the video ram may be freely accessed.

See also

[set_interrupts\(\)](#),
[add_VBL](#)

20.110.2.28 **LCD_IFLAG** `#define LCD_IFLAG 0x02U`
LCD Interrupt when triggered by the STAT register.

See also

[set_interrupts\(\)](#),
[add_LCD](#)

20.110.2.29 **TIM_IFLAG** `#define TIM_IFLAG 0x04U`
Does nothing on MSX

20.110.2.30 **SIO_IFLAG** `#define SIO_IFLAG 0x08U`
Does nothing on MSX

20.110.2.31 **JOY_IFLAG** `#define JOY_IFLAG 0x10U`
Does nothing on MSX

20.110.2.32 **SCREENWIDTH** `#define SCREENWIDTH DEVICE\_SCREEN\_PX\_WIDTH`
Width of the visible screen in pixels.

20.110.2.33 **SCREENHEIGHT** `#define SCREENHEIGHT DEVICE\_SCREEN\_PX\_HEIGHT`
Height of the visible screen in pixels.

20.110.2.34 **MINWNDPOSX** `#define MINWNDPOSX 0x00U`
The Minimum X position of the Window Layer (Left edge of screen)

See also

[move_win\(\)](#)

20.110.2.35 MINWNDPOSY `#define MINWNDPOSY 0x00U`

The Minimum Y position of the Window Layer (Top edge of screen)

See also

[move_win\(\)](#)

20.110.2.36 MAXWNDPOSX `#define MAXWNDPOSX 0x00U`

The Maximum X position of the Window Layer (Right edge of screen)

See also

[move_win\(\)](#)

20.110.2.37 MAXWNDPOSY `#define MAXWNDPOSY 0x00U`

The Maximum Y position of the Window Layer (Bottom edge of screen)

See also

[move_win\(\)](#)

20.110.2.38 DISPLAY_ON `#define DISPLAY_ON __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) | R1_DISP_ON)`

Turns the display back on.

See also

[display_off](#), [DISPLAY_OFF](#)

20.110.2.39 DISPLAY_OFF `#define DISPLAY_OFF display_off();`

Turns the display off immediately.

See also

[display_off](#), [DISPLAY_ON](#)

20.110.2.40 HIDE_LEFT_COLUMN `#define HIDE_LEFT_COLUMN __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) | R0_LCB)`

Blanks leftmost column, so it is not garbaged when you use horizontal scroll

See also

[SHOW_LEFT_COLUMN](#)

20.110.2.41 SHOW_LEFT_COLUMN `#define SHOW_LEFT_COLUMN __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) &= (~R0_LCB))`

Shows leftmost column

See also

[HIDE_LEFT_COLUMN](#)

20.110.2.42 SET_BORDER_COLOR `#define SET_BORDER_COLOR(
C) __WRITE_VDP_REG(VDP_R7, ((C) | 0xf0u))`

Sets border color

20.110.2.43 SHOW_BKG `#define SHOW_BKG`

Turns on the background layer. Not yet implemented

20.110.2.44 HIDE_BKG `#define HIDE_BKG`

Turns off the background layer. Not yet implemented

20.110.2.45 SHOW_WIN `#define SHOW_WIN`

Turns on the window layer Not yet implemented

20.110.2.46 HIDE_WIN `#define HIDE_WIN`

Turns off the window layer. Not yet implemented

20.110.2.47 SHOW_SPRITES `#define SHOW_SPRITES`

Turns on the sprites layer. Not yet implemented

20.110.2.48 HIDE_SPRITES `#define HIDE_SPRITES`

Turns off the sprites layer. Not yet implemented

20.110.2.49 SPRITES_16x16 `#define SPRITES_16x16 __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1)
|= R1_SPR_16X16)`

Sets sprite size to 8x16 pixels, two tiles one above the other.

20.110.2.50 SPRITES_8x8 `#define SPRITES_8x8 __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1)
&= (~R1_SPR_16X16))`

Sets sprite size to 8x8 pixels, one tile.

20.110.2.51 DEVICE_SUPPORTS_COLOR `#define DEVICE_SUPPORTS_COLOR (TRUE)`

Macro returns TRUE if device supports color (it always does on MSX)

20.110.2.52 VBL_DONE `#define VBL_DONE _vbl_done`

20.110.2.53 DIV_REG `#define DIV_REG get_r_reg()`

20.110.2.54 CURRENT_BANK `#define CURRENT_BANK _current_bank`

20.110.2.55 BANK `#define BANK(
VARNAME) ((uint8_t) & __bank_ ## VARNAME)`

Obtains the **bank number** of VARNAME

Parameters

VARNAME	Name of the variable which has a <code>__bank_VARNAME</code> companion symbol which is adjusted by bankpack
----------------	---

Use this to obtain the bank number from a bank reference created with [BANKREF\(\)](#).

See also

[BANKREF_EXTERN\(\)](#), [BANKREF\(\)](#)

20.110.2.56 BANKREF `#define BANKREF(`
`VARNAME )`

Value:

```
void __func_ ## VARNAME(void) __banked __naked { \
__asm \
    .local b__func_ ## VARNAME \
    __bank_ ## VARNAME = b__func_ ## VARNAME \
    .globl __bank_ ## VARNAME \
__endasm; \
}
```

Creates a reference for retrieving the bank number of a variable or function

Parameters

<i>VARNAME</i>	Variable name to use, which may be an existing identifier
----------------	---

See also

[BANK\(\)](#) for obtaining the bank number of the included data.

More than one [BANKREF \(\)](#) may be created per file, but each call should always use a unique VARNAME. Use [BANKREF_EXTERN\(\)](#) within another source file to make the variable and it's data accesible there.

20.110.2.57 BANKREF_EXTERN `#define BANKREF_EXTERN(`
`VARNAME ) extern const void __bank_ ## VARNAME;`

Creates extern references for accessing a [BANKREF\(\)](#) generated variable.

Parameters

<i>VARNAME</i>	Name of the variable used with BANKREF()
----------------	--

This makes a [BANKREF\(\)](#) reference in another source file accessible in the current file for use with [BANK\(\)](#).

See also

[BANKREF\(\)](#), [BANK\(\)](#)

20.110.2.58 SWITCH_ROM1 `#define SWITCH_ROM1 SWITCH_ROM`

20.110.2.59 SWITCH_ROM2 `#define SWITCH_ROM2(`
`b ) MAP_FRAME2=(b)`

Makes switch the active ROM bank in frame 2

Parameters

<i>b</i>	ROM bank to switch to
----------	-----------------------

20.110.2.60 SWITCH_RAM `#define SWITCH_RAM(`
`b ) RAM_CONTROL=( (b) &1) ? RAM_CONTROL|RAMCTL_BANK:RAM_CONTROL& (~RAMCTL_BANK)`

Switches RAM bank

Parameters

<i>b</i>	SRAM bank to switch to
----------	------------------------

20.110.2.61 ENABLE_RAM `#define ENABLE_RAM RAM_CONTROL|=RAMCTL_RAM`

Enables RAM

20.110.2.62 DISABLE_RAM `#define DISABLE_RAM RAM_CONTROL&=(~RAMCTL_RAM)`

Disables RAM

20.110.2.63 set_bkg_palette_entry `#define set_bkg_palette_entry set_palette_entry`

20.110.2.64 set_sprite_palette_entry `#define set_sprite_palette_entry(
 palette,
 entry,
 rgb_data) set_palette_entry(1,entry,rgb_data)`

20.110.2.65 set_bkg_palette `#define set_bkg_palette set_palette`

20.110.2.66 set_sprite_palette `#define set_sprite_palette(
 first_palette,
 nb_palettes,
 rgb_data) set_palette(1,1,rgb_data)`

20.110.2.67 COMPAT_PALETTE `#define COMPAT_PALETTE(
 C0,
 C1,
 C2,
 C3) (((uint16_t)(C3) << 12) | ((uint16_t)(C2) << 8) | ((uint16_t)(C1) << 4) |
 (uint16_t)(C0))`

20.110.2.68 set_bkg_tiles `#define set_bkg_tiles set_tile_map`

20.110.2.69 set_win_tiles `#define set_win_tiles set_tile_map`

20.110.2.70 fill_bkg_rect `#define fill_bkg_rect fill_rect`

20.110.2.71 fill_win_rect `#define fill_win_rect fill_rect`

20.110.2.72 DISABLE_VBL_TRANSFER `#define DISABLE_VBL_TRANSFER _shadow_OAM_base = 0`

Disable shadow OAM to VRAM copy on each VBlank

20.110.2.73 ENABLE_VBL_TRANSFER `#define ENABLE_VBL_TRANSFER _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM_base >> 8)`

Enable shadow OAM to VRAM copy on each VBlank

20.110.2.74 MAX_HARDWARE_SPRITES `#define MAX_HARDWARE_SPRITES 32`

Amount of hardware sprites in OAM

20.110.2.75 HARDWARE_SPRITE_CAN_FLIP_X `#define HARDWARE_SPRITE_CAN_FLIP_X 0`

True if sprite hardware can flip sprites by X (horizontally)

20.110.2.76 HARDWARE_SPRITE_CAN_FLIP_Y `#define HARDWARE_SPRITE_CAN_FLIP_Y 0`

True if sprite hardware can flip sprites by Y (vertically)

20.110.2.77 set_bkg_tile_xy `#define set_bkg_tile_xy set_tile_xy`

20.110.2.78 set_win_tile_xy `#define set_win_tile_xy set_tile_xy`

20.110.2.79 get_win_xy_addr `#define get_win_xy_addr get_bkg_xy_addr`

20.110.3 Typedef Documentation

20.110.3.1 int_handler `typedef void(* int_handler) (void) NONBANKED`

Interrupt handlers

20.110.3.2 OAM_item_t `typedef struct OAM_item_t OAM_item_t`

Sprite Attributes structure

Parameters

<i>x</i>	X Coordinate of the sprite on screen
<i>y</i>	Y Coordinate of the sprite on screen
<i>tile</i>	Sprite tile number (see set_sprite_tile)
<i>prop</i>	OAM Property Flags (see set_sprite_prop)

20.110.4 Function Documentation

20.110.4.1 WRITE_VDP_CMD() `void WRITE_VDP_CMD (uint16_t cmd)`

20.110.4.2 WRITE_VDP_DATA() `void WRITE_VDP_DATA (uint16_t data)`

20.110.4.3 mode() `void mode (uint8_t m)`

Set the current screen mode - one of M_* modes

Normally used by internal functions only.

See also

[M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.110.4.4 `get_mode()` `uint8_t get_mode (`
`void )`

Returns the current mode

See also

[M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

Returns the current mode

See also

[M_DRAWING](#), [M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.110.4.5 `get_system()` `uint8_t get_system (`
`void ) [inline]`

Returns the system gbdk is running on.

20.110.4.6 `set_interrupts()` `void set_interrupts (`
`uint8_t flags )`

Clears any pending interrupts and sets the interrupt mask register IO to flags.

Parameters

<i>flags</i>	A logical OR of *_IFLAGS
--------------	--------------------------

Note

This disables and then re-enables interrupts so it must be used outside of a critical section.

See also

[enable_interrupts\(\)](#), [disable_interrupts\(\)](#)

[VBL_IFLAG](#), [LCD_IFLAG](#), [TIM_IFLAG](#), [SIO_IFLAG](#), [JOY_IFLAG](#)

20.110.4.7 `remove_VBL()` `void remove_VBL (`
`int_handler h )`

Removes the VBL interrupt handler.

See also

[add_VBL\(\)](#)

20.110.4.8 `remove_LCD()` `void remove_LCD (`
`int_handler h )`

Removes the LCD interrupt handler.

See also

[add_LCD\(\)](#), [remove_VBL\(\)](#)

20.110.4.9 remove_TIM() `void remove_TIM (`
 `int_handler h )`

20.110.4.10 remove_SIO() `void remove_SIO (`
 `int_handler h )`

20.110.4.11 remove_JOY() `void remove_JOY (`
 `int_handler h )`

20.110.4.12 add_VBL() `void add_VBL (`
 `int_handler h )`

Adds a V-blank interrupt handler.

20.110.4.13 add_LCD() `void add_LCD (`
 `int_handler h )`

Adds a LCD interrupt handler.

20.110.4.14 add_TIM() `void add_TIM (`
 `int_handler h )`

Does nothing on MSX

20.110.4.15 add_SIO() `void add_SIO (`
 `int_handler h )`

Does nothing on MSX

20.110.4.16 add_JOY() `void add_JOY (`
 `int_handler h )`

Does nothing on MSX

20.110.4.17 cancel_pending_interrupts() `uint8_t cancel_pending_interrupts (`
 `void ) [inline]`

Cancel pending interrupts

20.110.4.18 move_bkg() `void move_bkg (`
 `uint8_t x,`
 `uint8_t y ) [inline]`

20.110.4.19 scroll_bkg() `void scroll_bkg (`
 `int8_t x,`
 `int8_t y ) [inline]`

20.110.4.20 vsync() `void vsync (`
 `void )`

HALTs the CPU and waits for the vertical blank interrupt.

This is often used in main loops to idle the CPU at low power until it's time to start the next frame. It's also useful for syncing animation with the screen re-draw.

Warning: If the VBL interrupt is disabled, this function will never return. If the screen is off this function returns immediately.

20.110.4.21 wait_vbl_done() `void wait_vbl_done (`
`void )`

Obsolete. This function has been replaced by [vsync\(\)](#), which has identical behavior.

20.110.4.22 display_off() `void display_off (`
`void ) [inline]`

Turns the display off.

See also

[DISPLAY_ON](#)

20.110.4.23 refresh_OAM() `void refresh_OAM (`
`void )`

Copies data from shadow OAM to OAM

20.110.4.24 get_r_reg() `uint8_t get_r_reg (`
`void )`

Return R register for the DIV_REG emulation

Increments once per CPU instruction (fetches the Z80 CPU R register)

20.110.4.25 SWITCH_ROM() `void SWITCH_ROM (`
`uint8_t bank )`

Makes switch the active ROM bank in frame 1

Parameters

<i>bank</i>	ROM bank to switch to
-------------	-----------------------

20.110.4.26 delay() `void delay (`
`uint16_t d )`

Delays the given number of milliseconds. Uses no timers or interrupts, and can be called with interrupts disabled

20.110.4.27 joypad() `uint8_t joypad (`
`void )`

Reads and returns the current state of the joypad.

20.110.4.28 waitpad() `uint8_t waitpad (`
`uint8_t mask )`

Waits until at least one of the buttons given in mask are pressed.

20.110.4.29 waitpadup() `void waitpadup (`
`void )`

Waits for the directional pad and all buttons to be released.

Note: Checks in a loop that doesn't HALT at all, so the CPU will be maxed out until this call returns.

20.110.4.30 joypad_init() `uint8_t joypad_init (`
`uint8_t npads,`
`joypads_t * joypads )`

Initializes [joypads_t](#) structure for polling multiple joypads

Parameters

<i>npads</i>	number of joypads requested (1, 2 or 4)
--------------	---

Parameters

<i>joypads</i>	pointer to joypads_t structure to be initialized
----------------	--

Only required for [joypad_ex](#), not required for calls to regular [joypad\(\)](#)

Returns

number of joypads available

See also

[joypad_ex\(\)](#), [joypads_t](#)

20.110.4.31 joypad_ex() `void joypad_ex (`
 [joypads_t](#) * *joypads*)

Polls all available joypads

Parameters

<i>joypads</i>	pointer to joypads_t structure to be filled with joypad statuses, must be previously initialized with joypad_init()
----------------	---

See also

[joypad_init\(\)](#), [joypads_t](#)

20.110.4.32 enable_interrupts() `void enable_interrupts (`
 void) [inline]

Enables unmasked interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

See also

[disable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.110.4.33 disable_interrupts() `void disable_interrupts (`
 void) [inline]

Disables interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

This function may be called as many times as you like; however the first call to [enable_interrupts](#) will re-enable them.

See also

[enable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.110.4.34 set_default_palette() `void set_default_palette (`
`void )`

20.110.4.35 cpu_fast() `void cpu_fast (`
`void ) [inline]`

Set CPU speed to fast (CGB Double Speed) operation.

On startup the CGB operates in Normal Speed Mode and can be switched into Double speed mode (faster processing but also higher power consumption). See the Pan Docs for more information about which hardware features operate faster and which remain at Normal Speed.

- Interrupts are temporarily disabled and then re-enabled during this call.
- You can check to see if `_cpu == CGB_TYPE` before using this function.

See also

[cpu_slow\(\)](#), [_cpu](#)

20.110.4.36 set_palette_entry() `void set_palette_entry (`
`uint8_t palette,`
`uint8_t entry,`
`uint16_t rgb_data )`

20.110.4.37 set_palette() `void set_palette (`
`uint8_t first_palette,`
`uint8_t nb_palettes,`
`const palette_color_t * rgb_data )`

20.110.4.38 set_native_tile_data() `void set_native_tile_data (`
`uint16_t start,`
`uint16_t ntiles,`
`const void * src )`

20.110.4.39 set_bkg_4bpp_data() `void set_bkg_4bpp_data (`
`uint16_t start,`
`uint16_t ntiles,`
`const void * src ) [inline]`

20.110.4.40 set_sprite_1bpp_data() `void set_sprite_1bpp_data (`
`uint16_t start,`
`uint16_t ntiles,`
`const void * src ) [inline]`

20.110.4.41 set_native_sprite_data() `void set_native_sprite_data (`
`uint16_t start,`
`uint16_t ntiles,`
`const void * src ) [inline]`

20.110.4.42 set_2bpp_palette() void set_2bpp_palette (
 uint16_t palette) [inline]

20.110.4.43 set_bkg_data() void set_bkg_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.110.4.44 set_sprite_data() void set_sprite_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.110.4.45 set_1bpp_colors() void set_1bpp_colors (
 uint8_t fgcolor,
 uint8_t bgcolor) [inline]

20.110.4.46 set_tile_1bpp_data() void set_tile_1bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src,
 uint16_t colors)

20.110.4.47 set_bkg_1bpp_data() void set_bkg_1bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.110.4.48 set_data() void set_data (
 uint16_t dst,
 const void * src,
 uint16_t size)

Copies arbitrary data to an address in VRAM

Parameters

<i>dst</i>	destination VRAM Address
<i>src</i>	Pointer to source buffer
<i>size</i>	Number of bytes to copy

Copies **size** bytes from a buffer at **_src__** to VRAM starting at **dst**.

20.110.4.49 vmemcpy() void vmemcpy (
 uint16_t dst,
 const void * src,
 uint16_t size)

20.110.4.50 set_tile_map() void set_tile_map (
 uint8_t x,

```

uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * tiles )

```

20.110.4.51 **set_bkg_based_tiles()** void set_bkg_based_tiles (

```

uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * tiles,
uint8_t base_tile ) [inline]

```

Sets a rectangular region of Background Tile Map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_tiles\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_tiles](#) for more details

20.110.4.52 **set_win_based_tiles()** void set_win_based_tiles (

```

uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * tiles,
uint8_t base_tile ) [inline]

```

20.110.4.53 **set_tile_submap()** void set_tile_submap (

```

uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
uint8_t map_w,
const uint8_t * map )

```

20.110.4.54 **set_tile_submap_compat()** void set_tile_submap_compat (

```

uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,

```

```
uint8_t map_w,
const uint8_t * map )
```

20.110.4.55 set_bkg_submap() void set_bkg_submap (

```
uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * map,
uint8_t map_w ) [inline]
```

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. Useful for scrolling implementations of maps larger than 32 x 32 tiles.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map</i> ↔ <i>_w</i>	Width of source tile map in tiles. Range 1 - 255

Entries are copied from **map** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles, using **map_w** as the rowstride for the source tile map.

The **x** and **y** parameters are in Source Tile Map tile coordinates. The location tiles will be written to on the hardware Background Map is derived from those, but only uses the lower 5 bits of each axis, for range of 0-31 (they are bit-masked: $x \& 0x1F$ and $y \& 0x1F$). As a result the two coordinate systems are aligned together.

In order to transfer tile map data in a way where the coordinate systems are not aligned, an offset from the Source Tile Map pointer can be passed in: $(map_ptr + x + (y * map_width))$.

For example, if you want the tile id at 1, 2 from the source map to show up at 0, 0 on the hardware Background Map (instead of at 1, 2) then modify the pointer address that is passed in: $map_ptr + 1 + (2 * map_width)$

Use this instead of [set_bkg_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

See [set_bkg_tiles](#) for setting CGB attribute maps with [VBK_REG](#).

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_tiles](#), [set_win_submap](#), [set_tiles](#)

20.110.4.56 set_win_submap() void set_win_submap (

```
uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * map,
uint8_t map_w ) [inline]
```

Sets a rectangular area of the Window Tile Map using a sub-region from a source tile map.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Window Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Window Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map</i> _↔ <i>_w</i>	Width of source tile map in tiles. Range 1 - 255

Entries are copied from **map** to the Window Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles, using **map_w** as the rowstride for the source tile map.

The **x** and **y** parameters are in Source Tile Map tile coordinates. The location tiles will be written to on the hardware Background Map is derived from those, but only uses the lower 5 bits of each axis, for range of 0-31 (they are bit-masked: $x \& 0x1F$ and $y \& 0x1F$). As a result the two coordinate systems are aligned together.

In order to transfer tile map data in a way where the coordinate systems are not aligned, an offset from the Source Tile Map pointer can be passed in: $(map_ptr + x + (y * map_width))$.

For example, if you want the tile id at 1, 2 from the source map to show up at 0, 0 on the hardware Background Map (instead of at 1, 2) then modify the pointer address that is passed in: $map_ptr + 1 + (2 * map_width)$

Use this instead of [set_win_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

GBC only: [VBK_REG](#) determines whether Tile Numbers or Tile Attributes get set.

- [VBK_REG](#) = [VBK_TILES](#) Tile Numbers are written
- [VBK_REG](#) = [VBK_ATTRIBUTES](#) Tile Attributes are written

See [set_bkg_tiles](#) for details about CGB attribute maps with [VBK_REG](#).

See also

[SHOW_WIN](#), [HIDE_WIN](#), [set_win_tiles](#), [set_bkg_submap](#), [set_bkg_tiles](#), [set_bkg_data](#), [set_tiles](#)

```
20.110.4.57 set_bkg_based_submap() void set_bkg_based_submap (
    uint8_t x,
    uint8_t y,
    uint8_t w,
    uint8_t h,
    const uint8_t * map,
    uint8_t map_w,
    uint8_t base_tile ) [inline]
```

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_submap](#) for more details

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_submap](#) for more details

20.110.4.58 set_win_based_submap() `void set_win_based_submap (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * map,`
`uint8_t map_w,`
`uint8_t base_tile ) [inline]`

20.110.4.59 fill_rect() `void fill_rect (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint16_t tile )`

20.110.4.60 SET_SHADOW_OAM_ADDRESS() `void SET_SHADOW_OAM_ADDRESS (`
`void * address ) [inline]`

Sets address of 256-byte aligned array of shadow OAM to be transferred on each VBlank

20.110.4.61 set_sprite_tile() `void set_sprite_tile (`
`uint8_t nb,`
`uint8_t tile ) [inline]`

Sets sprite number **nb** in the OAM to display tile number **__tile**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>tile</i>	Selects a tile (0 - 255) from memory at 8000h - 8FFFh In CGB Mode this could be either in VRAM Bank 0 or 1, depending on Bit 3 of the OAM Attribute Flag (see set_sprite_prop)

In 8x16 mode:

- The sprite will also display the next tile (**tile** + 1) directly below (y + 8) the first tile.
- The lower bit of the tile number is ignored: the upper 8x8 tile is (**tile** & 0xFE), and the lower 8x8 tile is (**tile** | 0x01).
- See: [SPRITES_8x16](#)

20.110.4.62 get_sprite_tile() `uint8_t get_sprite_tile (`
`uint8_t nb ) [inline]`

Returns the tile number of sprite number **nb** in the OAM.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_tile](#) for more details

20.110.4.63 set_sprite_prop() `void set_sprite_prop (`
`uint8_t nb,`
`uint8_t prop ) [inline]`

Sets the OAM Property Flags of sprite number **nb** to those defined in **prop**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>prop</i>	Property setting (see bitfield description)

The bits in **prop** represent:

- Bit 7 - Vertical flip. Dictates which way up the sprite is drawn vertically.
0: normal
1: upside down
- Bit 6 - Horizontal flip. Dictates which way up the sprite is drawn horizontally.
0: normal
1: back to front
- Bit 5 - Priority flag. When this is set, the sprites appear behind the background and window layer.
0: infront
1: behind
- Bit 4 - Unimplemented
- Bit 3 - Unimplemented

- Bit 2 - Unimplemented
- Bit 1 - See bit 0.
- Bit 0 - Bits 0-1 indicate which color palette the sprite should use. Note: only palettes 4 to 7 will be available for NES sprites.

It's recommended to use GBDK constants (eg: `S_FLIPY`) to configure sprite properties as these are crossplatform.

```
// Load palette data into the first palette
set_sprite_palette(4, 1, exampleSprite_palettes)
// Set the OAM value for the sprite
// These flags tell the sprite to use the first sprite palette (palette 4) and to flip the sprite both
// vertically and horizontally.
set_sprite_prop(0, S_FLIPY | S_FLIPX);
```

See also

[S_PALETTE](#), [S_FLIPX](#), [S_FLIPY](#), [S_PRIORITY](#)

Sets the OAM Property Flags of sprite number **nb** to those defined in **prop**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>prop</i>	Property setting (see bitfield description)

The bits in **prop** represent:

- Bit 7 - Priority flag. When this is set the sprites appear behind the background and window layer.
0: infront
1: behind
- Bit 6 - Vertical flip. Dictates which way up the sprite is drawn vertically.
0: normal
1:upside down
- Bit 5 - Horizontal flip. Dictates which way up the sprite is drawn horizontally.
0: normal
1:back to front
- Bit 4 - DMG/Non-CGB Mode Only. Assigns either one of the two b/w palettes to the sprite.
0: OBJ palette 0
1: OBJ palette 1
- Bit 3 - GBC only. Dictates from which bank of Sprite Tile Patterns the tile is taken.
0: Bank 0
1: Bank 1
- Bit 2 - See bit 0.
- Bit 1 - See bit 0.
- Bit 0 - GBC only. Bits 0-2 indicate which of the 7 OBJ colour palettes the sprite is assigned.

It's recommended to use GBDK constants (eg: `S_FLIPY`) to configure sprite properties as these are crossplatform.

```
// Load palette data into the first palette
set_sprite_palette(4, 1, exampleSprite_palettes)
// Set the OAM value for the sprite
// These flags tell the sprite to flip both vertically and horizontally.
set_sprite_prop(0, S_FLIPY | S_FLIPX);
```

See also

[S_PALETTE](#), [S_FLIPX](#), [S_FLIPY](#), [S_PRIORITY](#)

20.110.4.64 `get_sprite_prop()` `uint8_t` `get_sprite_prop` (
 `uint8_t` *nb*) `[inline]`

Returns the OAM Property Flags of sprite number **nb**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_prop](#) for property bitfield settings

20.110.4.65 move_sprite() `void move_sprite (`
 `uint8_t nb,`
 `uint8_t x,`
 `uint8_t y ) [inline]`

Moves sprite number **nb** to the **x**, **y** position on the screen.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	X Position. Specifies the sprites horizontal position on the screen (minus 8). An offscreen value (X=0 or X>=168) hides the sprite, but the sprite still affects the priority ordering - a better way to hide a sprite is to set its Y-coordinate offscreen.
<i>y</i>	Y Position. Specifies the sprites vertical position on the screen (minus 16). An offscreen value (for example, Y=0 or Y>=160) hides the sprite.

Moving the sprite to 0,0 (or similar off-screen location) will hide it.

20.110.4.66 scroll_sprite() `void scroll_sprite (`
 `uint8_t nb,`
 `int8_t x,`
 `int8_t y ) [inline]`

Moves sprite number **nb** relative to its current position.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	Number of pixels to move the sprite on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the sprite on the Y axis Range: -128 - 127

See also

[move_sprite](#) for more details about the X and Y position

20.110.4.67 hide_sprite() `void hide_sprite (`
 `uint8_t nb ) [inline]`

Hides sprite number **nb** by moving it to zero position by Y.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

20.110.4.68 set_vram_byte() `void set_vram_byte (`
 `uint8_t * addr,`
 `uint8_t v )`

Set byte in vram at given memory location

Parameters

<i>addr</i>	address to write to
<i>v</i>	value

20.110.4.69 set_attributed_tile_xy() `uint8_t * set_attributed_tile_xy (`
 `uint8_t x,`
 `uint8_t y,`
 `uint16_t t )`

Set single tile t with attributes on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>t</i>	tile index

Returns

returns the address of tile, so you may use faster [set_vram_byte\(\)](#) later

20.110.4.70 set_tile_xy() `uint8_t * set_tile_xy (`
 `uint8_t x,`
 `uint8_t y,`
 `uint8_t t )`

Set single tile t on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>t</i>	tile index

Returns

returns the address of tile, so you may use faster [set_vram_byte\(\)](#) later

20.110.4.71 get_bkg_xy_addr() `uint8_t * get_bkg_xy_addr (`
 `uint8_t x,`
 `uint8_t y )`

Get address of X,Y tile of background map

20.110.5 Variable Documentation

20.110.5.1 `_SYSTEM` `const uint8_t _SYSTEM` [extern]

20.110.5.2 `c` `void c`

20.110.5.3 `d` `void d`

20.110.5.4 `e` `void e`

20.110.5.5 `iyh` `void iyh`

20.110.5.6 `iy1` `uint8_t iy1`

Initial value:

```
{  
    __asm__("ei")  
}
```

20.110.5.7 `h` `void h`

20.110.5.8 `l` `void l`

20.110.5.9 `sys_time` `volatile uint16_t sys_time` [extern]

Global Time Counter in VBL periods (60Hz)

Increments once per Frame

Will wrap around every ~18 minutes (unsigned 16 bits = 65535 / 60 / 60 = 18.2)

20.110.5.10 `_vbl_done` `volatile uint8_t _vbl_done` [extern]

Flag indicating the VBlank ISR has run

Flag gets cleared at the start of `vsync()` / `wait_vbl_done()` and set in the default VBlank ISR handler.

20.110.5.11 `_current_bank` `volatile uint8_t _current_bank` [extern]

Tracks current active ROM bank in frame 1

Tracks current active ROM bank

In most cases the `CURRENT_BANK` macro for this variable is recommended for use instead of the variable itself.

The active bank number is not tracked by `_current_bank` when `SWITCH_ROM_MBC5_8M` is used.

This variable is updated automatically when you call `SWITCH_ROM_MBC1` or `SWITCH_ROM_MBC5`, `SWITCH_ROM()`, or call a BANKED function.

See also

[SWITCH_ROM_MBC1\(\)](#), [SWITCH_ROM_MBC5\(\)](#), [SWITCH_ROM\(\)](#)

20.110.5.12 `b` `void b`

20.110.5.13 `_current_2bpp_palette` `uint16_t _current_2bpp_palette` [extern]

20.110.5.14 `_current_1bpp_colors` `uint16_t` `_current_1bpp_colors` [extern]

20.110.5.15 `_map_tile_offset` `uint8_t` `_map_tile_offset` [extern]

20.110.5.16 `_submap_tile_offset` `uint8_t` `_submap_tile_offset` [extern]

20.110.5.17 `shadow_OAM` `volatile struct OAM_item_t` `shadow_OAM[]` [extern]

Shadow OAM array in WRAM, that is DMA-transferred into the real OAM each VBlank

20.110.5.18 `_shadow_OAM_base` `volatile uint8_t` `_shadow_OAM_base` [extern]

MSB of shadow_OAM address is used by OAM copying routine

MSB of shadow_OAM address is used by OAM DMA copying routine

20.110.5.19 `_shadow_OAM_OFF` `volatile uint8_t` `_shadow_OAM_OFF` [extern]

Flag for disabling of OAM copying routine

Values:

- 1: OAM copy routine is disabled (non-isr VDP operation may be in progress)
- 0: OAM copy routine is enabled

This flag is modified by all MSX GBDK API calls that write to the VDP. It is set to DISABLED when they start and ENABLED when they complete.

Note

It is recommended to avoid writing to the Video Display Processor (VDP) during an interrupt service routine (ISR) since it can corrupt the VDP pointer of an VDP operation already in progress.

If it is necessary, this flag can be used during an ISR to determine whether a VDP operation is already in progress.

If the value is 1 then avoid writing to the VDP (tiles, map, scrolling, colors, etc).

```
// at the beginning of and ISR that would write to the VDP
if (_shadow_OAM_OFF) return;
```

See also

[docs_consoles_safe_display_controller_access](#)

20.111 msx.h

[Go to the documentation of this file.](#)

```
1
2
3
4 #ifndef _MSX_H
5 #define _MSX_H
6
7 #include <types.h>
8 #include <stdint.h>
9 #include <gbdk/version.h>
10 #include <msx/hardware.h>
11
12 #define MSX
13
14 // Here NINTENDO means Game Boy & related clones
15 #ifdef NINTENDO
16 #undef NINTENDO
17 #endif
18
19 #ifdef NINTENDO_NES
20 #undef NINTENDO_NES
21 #endif
22
23 #ifdef SEGA
24 #undef SEGA
25 #endif
26
27 #if defined(__TARGET_msxdos)
28 #define MSXDOS
```



```

29 #endif
30
31 extern const uint8_t _SYSTEM;
32
33 #define SYSTEM_60HZ 0x00
34 #define SYSTEM_50HZ 0x01
35
36 #define VBK_REG VDP_ATTR_SHIFT
37
51 #define J_UP 0b00100000
52 #define J_DOWN 0b01000000
53 #define J_LEFT 0b00010000
54 #define J_RIGHT 0b10000000
55 #define J_A 0b00000001
56 #define J_B 0b00000100
57 #define J_SELECT 0b00001000
58 #define J_START 0b00000010
59
64 #define M_TEXT_OUT 0x02U
65 #define M_TEXT_INOUT 0x03U
71 #define M_NO_SCROLL 0x04U
75 #define M_NO_INTERP 0x08U
76
79 #define S_BANK 0x01U
82 #define S_FLIPX 0x02U
85 #define S_FLIPY 0x04U
88 #define S_PALETTE 0x08U
91 #define S_PRIORITY 0x10U
95 #define S_PAL(n) ((n) & 0x01U) << 3)
96
97 // VDP helper macros
98 #define __WRITE_VDP_REG_UNSAFE(REG, v) shadow_##REG=(v),VDP_CMD=(shadow_##REG),VDP_CMD=REG
99 #define __WRITE_VDP_REG(REG, v)
100     shadow_##REG=(v);__asm__("di");VDP_CMD=(shadow_##REG);VDP_CMD=REG;__asm__("ei")
101 #define __READ_VDP_REG(REG) shadow_##REG
102
102 void WRITE_VDP_CMD(uint16_t cmd) Z88DK_FASTCALL PRESERVES_REGS(b, c, d, e, iyh, iyl);
103 void WRITE_VDP_DATA(uint16_t data) Z88DK_FASTCALL PRESERVES_REGS(b, c, d, e, iyh, iyl);
104
111 void mode(uint8_t m) OLDCALL;
112
117 uint8_t get_mode(void) OLDCALL;
118
122 inline uint8_t get_system(void) {
123     return _SYSTEM;
124 }
125
126 /* Interrupt flags */
129 #define EMPTY_IFLAG 0x00U
135 #define VBL_IFLAG 0x01U
139 #define LCD_IFLAG 0x02U
142 #define TIM_IFLAG 0x04U
145 #define SIO_IFLAG 0x08U
148 #define JOY_IFLAG 0x10U
149
150 void set_interrupts(uint8_t flags) Z88DK_FASTCALL;
151
152 /* Limits */
155 #define SCREENWIDTH DEVICE_SCREEN_PX_WIDTH
158 #define SCREENHEIGHT DEVICE_SCREEN_PX_HEIGHT
161 #define MINWNDPOSX 0x00U
164 #define MINWNDPOSY 0x00U
167 #define MAXWNDPOSX 0x00U
170 #define MAXWNDPOSY 0x00U
171
172
175 typedef void (*int_handler)(void) NONBANKED;
176
180 void remove_VBL(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(iyh, iyl);
181
185 void remove_LCD(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b, c, iyh, iyl);
186
187 void remove_TIM(int_handler h) Z88DK_FASTCALL;
188 void remove_SIO(int_handler h) Z88DK_FASTCALL;
189 void remove_JOY(int_handler h) Z88DK_FASTCALL;
190
193 void add_VBL(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(d, e, iyh, iyl);
194
197 void add_LCD(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b, c, iyh, iyl);
198
201 void add_TIM(int_handler h) Z88DK_FASTCALL;
202
205 void add_SIO(int_handler h) Z88DK_FASTCALL;
206
209 void add_JOY(int_handler h) Z88DK_FASTCALL;
210
213 inline uint8_t cancel_pending_interrupts(void) {

```

```

214     return VDP_STATUS;
215 }
216
217 inline void move_bkg(uint8_t x, uint8_t y) {
218     __WRITE_VDP_REG(VDP_RSCX, -x);
219     __WRITE_VDP_REG(VDP_RSCY, y);
220 }
221
222 inline void scroll_bkg(int8_t x, int8_t y) {
223     __WRITE_VDP_REG(VDP_RSCX, __READ_VDP_REG(VDP_RSCX) - x);
224     int16_t tmp = __READ_VDP_REG(VDP_RSCY) + y;
225     __WRITE_VDP_REG(VDP_RSCY, (tmp < 0) ? 224 + tmp : tmp % 224u);
226 }
227
228 void vsync(void) PRESERVES_REGS(b, c, d, e, h, l, iyh, iyl);
229
230 void wait_vbl_done(void) PRESERVES_REGS(b, c, d, e, h, l, iyh, iyl);
231
232 inline void display_off(void) {
233     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) &= (~R1_DISP_ON));
234 }
235
236 #define DISPLAY_ON \
237     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) |= R1_DISP_ON)
238
239 #define DISPLAY_OFF \
240     display_off();
241
242 void refresh_OAM(void);
243
244 #define HIDE_LEFT_COLUMN \
245     __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) |= R0_LCB)
246
247 #define SHOW_LEFT_COLUMN \
248     __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) &= (~R0_LCB))
249
250 #define SET_BORDER_COLOR(C) __WRITE_VDP_REG(VDP_R7, ((C) | 0xf0u))
251
252 #define SHOW_BKG
253
254 #define HIDE_BKG
255
256 #define SHOW_WIN
257
258 #define HIDE_WIN
259
260 #define SHOW_SPRITES
261
262 #define HIDE_SPRITES
263
264 #define SPRITES_16x16 \
265     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) |= R1_SPR_16X16)
266
267 #define SPRITES_8x8 \
268     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) &= (~R1_SPR_16X16))
269
270 #define DEVICE_SUPPORTS_COLOR (TRUE)
271
272 extern volatile uint16_t sys_time;
273
274 extern volatile uint8_t _vbl_done;
275 #define VBL_DONE _vbl_done
276
277 uint8_t get_r_reg(void) PRESERVES_REGS(b, c, d, e, h, l, iyh, iyl);
278
279 #define DIV_REG get_r_reg()
280
281 extern volatile uint8_t _current_bank;
282 #define CURRENT_BANK _current_bank
283
284 #ifndef BANK
285 #define BANK(VARNAME) ( (uint8_t) &__bank_ ## VARNAME )
286 #endif
287
288 #define BANKREF(VARNAME) void __func_ ## VARNAME(void) __banked __naked { \
289     __asm \
290     .local b__func_ ## VARNAME \
291     __bank_ ## VARNAME = b__func_ ## VARNAME \
292     .globl __bank_ ## VARNAME \
293     __endasm; \
294 }
295
296 #define BANKREF_EXTERN(VARNAME) extern const void __bank_ ## VARNAME;
297
298 void SWITCH_ROM(uint8_t bank) Z88DK_FASTCALL PRESERVES_REGS(b, c, d, e, iyh, iyl);
299 #define SWITCH_ROM1 SWITCH_ROM

```

```

410
415 #define SWITCH_ROM2(b) MAP_FRAME2=(b)
416
421 #define SWITCH_RAM(b) RAM_CONTROL=(b)&1?RAM_CONTROL|RAMCTL_BANK:RAM_CONTROL&(~RAMCTL_BANK)
422
426 #define ENABLE_RAM RAM_CONTROL|=RAMCTL_RAM
427
431 #define DISABLE_RAM RAM_CONTROL&=~(RAMCTL_RAM)
432
433
438 void delay(uint16_t d) Z88DK_FASTCALL;
439
440
443 uint8_t joypad(void) OLDCALL PRESERVES_REGS(b, c, d, e, h, iyh, iyl);
444
447 uint8_t waitpad(uint8_t mask) Z88DK_FASTCALL PRESERVES_REGS(b, c, d, e, iyh, iyl);
448
454 void waitpadup(void) PRESERVES_REGS(b, c, d, e, iyh, iyl);
455
461 typedef struct {
462     uint8_t npads;
463     union {
464         struct {
465             uint8_t joy0, joy1, joy2, joy3;
466         };
467         uint8_t joypads[4];
468     };
469 } joypads_t;
470
479 uint8_t joypad_init(uint8_t npads, joypads_t * joypads) Z88DK_CALLEE;
480
487 void joypad_ex(joypads_t * joypads) Z88DK_FASTCALL PRESERVES_REGS(iyh, iyl);
488
489
498 inline void enable_interrupts(void) PRESERVES_REGS(a, b, c, d, e, h, l, iyh, iyl) {
499     __asm__("ei");
500 }
501
514 inline void disable_interrupts(void) PRESERVES_REGS(a, b, c, d, e, h, l, iyh, iyl) {
515     __asm__("di");
516 }
517
518
519 #if defined(__TARGET_msxdos)
520
521 #define RGB(r,g,b) ((r) | ((g) << 2) | ((b) << 4))
522 #define RGB8(r,g,b) (((r) >> 6) | (((g) >> 6) << 2) | (((b) >> 6) << 4))
523 #define RGBHTML(RGB24bit) (((RGB24bit) >> 22) | (((RGB24bit) & 0xFFFF) >> 14) << 2) | (((RGB24bit) & 0xFF) >> 6) << 4)
524
527 #define RGB_RED RGB( 3, 0, 0)
528 #define RGB_DARKRED RGB( 2, 0, 0)
529 #define RGB_GREEN RGB( 0, 3, 0)
530 #define RGB_DARKGREEN RGB( 0, 2, 0)
531 #define RGB_BLUE RGB( 0, 0, 3)
532 #define RGB_DARKBLUE RGB( 0, 0, 2)
533 #define RGB_YELLOW RGB( 3, 3, 0)
534 #define RGB_DARKYELLOW RGB( 2, 2, 0)
535 #define RGB_CYAN RGB( 0, 3, 3)
536 #define RGB_AQUA RGB( 3, 1, 2)
537 #define RGB_PINK RGB( 3, 0, 3)
538 #define RGB_PURPLE RGB( 2, 0, 2)
539 #define RGB_BLACK RGB( 0, 0, 0)
540 #define RGB_DARKGRAY RGB( 1, 1, 1)
541 #define RGB_LIGHTGRAY RGB( 2, 2, 2)
542 #define RGB_WHITE RGB( 3, 3, 3)
543
544 typedef uint8_t palette_color_t;
545
546 #else
547 #error Unrecognized port
548 #endif
549
550 void set_default_palette(void);
551 inline void cpu_fast(void) {}
552
553 void set_palette_entry(uint8_t palette, uint8_t entry, uint16_t rgb_data) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
554 #define set_bkg_palette_entry set_palette_entry
555 #define set_sprite_palette_entry(palette,entry,rgb_data) set_palette_entry(l,entry,rgb_data)
556 void set_palette(uint8_t first_palette, uint8_t nb_palettes, const palette_color_t *rgb_data) Z88DK_CALLEE;
557 #define set_bkg_palette set_palette
558 #define set_sprite_palette(first_palette,nb_palettes,rgb_data) set_palette(l,l,rgb_data)
559
560 void set_native_tile_data(uint16_t start, uint16_t ntiles, const void *src) Z88DK_CALLEE;
561 inline void set_bkg_4bpp_data(uint16_t start, uint16_t ntiles, const void *src) {

```

```

562     set_native_tile_data(start, ntiles, src);
563 }
564 void set_sprite_lbpp_data(uint16_t start, uint16_t ntiles, const void *src) Z88DK_CALLEE;
565 inline void set_native_sprite_data(uint16_t start, uint16_t ntiles, const void *src) {
566     set_sprite_lbpp_data(start, ntiles, src);
567 }
568
569 #define COMPAT_PALETTE(C0,C1,C2,C3) (((uint16_t)(C3) < 12) | ((uint16_t)(C2) < 8) | ((uint16_t)(C1) < 4)
    | (uint16_t)(C0))
570 extern uint16_t _current_2bpp_palette;
571 inline void set_2bpp_palette(uint16_t palette) {
572     _current_2bpp_palette = palette;
573 }
574 //void set_tile_2bpp_data(uint16_t start, uint16_t ntiles, const void *src, uint16_t palette)
    Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
575 inline void set_bkg_data(uint16_t start, uint16_t ntiles, const void *src) {
576     set_native_tile_data(start, ntiles, src);
577 }
578 inline void set_sprite_data(uint16_t start, uint16_t ntiles, const void *src) {
579     set_sprite_lbpp_data(start, ntiles, src);
580 }
581 //inline void set_bkg_2bpp_data(uint16_t start, uint16_t ntiles, const void *src) {
582 //    set_tile_2bpp_data(start, ntiles, src, _current_2bpp_palette);
583 //}
584 //inline void set_sprite_2bpp_data(uint16_t start, uint16_t ntiles, const void *src) {
585 //    set_tile_2bpp_data((uint8_t)(start) + 0x100u, ntiles, src, _current_2bpp_palette);
586 //}
587
588 extern uint16_t _current_lbpp_colors;
589 inline void set_lbpp_colors(uint8_t fgcolor, uint8_t bgcolor) {
590     _current_lbpp_colors = ((uint16_t)bgcolor < 8) | fgcolor;
591 }
592 void set_tile_lbpp_data(uint16_t start, uint16_t ntiles, const void *src, uint16_t colors) Z88DK_CALLEE
    PRESERVES_REGS(iyh, iyl);
593 inline void set_bkg_lbpp_data(uint16_t start, uint16_t ntiles, const void *src) {
594     set_tile_lbpp_data(start, ntiles, src, _current_lbpp_colors);
595 }
596
597
606 void set_data(uint16_t dst, const void *src, uint16_t size) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
607 void vmemcpy(uint16_t dst, const void *src, uint16_t size) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
608
609 void set_tile_map(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) Z88DK_CALLEE
    PRESERVES_REGS(iyh, iyl);
610 #define set_bkg_tiles set_tile_map
611 #define set_win_tiles set_tile_map
612
613 extern uint8_t _map_tile_offset;
614 inline void set_bkg_based_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles,
    uint8_t base_tile) {
615     _map_tile_offset = base_tile;
616     set_tile_map(x, y, w, h, tiles);
617     _map_tile_offset = 0;
618 }
619 inline void set_win_based_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles,
    uint8_t base_tile) {
620     _map_tile_offset = base_tile;
621     set_tile_map(x, y, w, h, tiles);
622     _map_tile_offset = 0;
623 }
624
625 void set_tile_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t map_w, const uint8_t *map)
    Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
626 void set_tile_submap_compat(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t map_w, const uint8_t
    *map) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
627 inline void set_bkg_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
    map_w) {
628     set_tile_submap_compat(x, y, w, h, map_w, map);
629 }
630 inline void set_win_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
    map_w) {
631     set_tile_submap_compat(x, y, w, h, map_w, map);
632 }
633
634 extern uint8_t _submap_tile_offset;
635 inline void set_bkg_based_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
    map_w, uint8_t base_tile) {
636     _submap_tile_offset = base_tile;
637     set_tile_submap_compat(x, y, w, h, map_w, map);
638     _submap_tile_offset = 0;
639 }
640 inline void set_win_based_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
    map_w, uint8_t base_tile) {
641     _submap_tile_offset = base_tile;
642     set_tile_submap_compat(x, y, w, h, map_w, map);
643     _submap_tile_offset = 0;
644 }

```

```

645
646 void fill_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint16_t tile) Z88DK_CALLEE
    PRESERVES_REGS(iyh, iyl);
647 #define fill_bkg_rect fill_rect
648 #define fill_win_rect fill_rect
649
650 typedef struct OAM_item_t {
651     uint8_t y, x;    /*< X, Y Coordinates of the sprite on screen
652     uint8_t tile;    /*< Sprite tile number
653     uint8_t prop;    /*< OAM Property Flags
654 } OAM_item_t;
655
656 extern volatile struct OAM_item_t shadow_OAM[];
657
658 extern volatile uint8_t _shadow_OAM_base;
659
660 extern volatile uint8_t _shadow_OAM_OFF;
661
662 #define DISABLE_VBL_TRANSFER \
    _shadow_OAM_base = 0
663
664 #define ENABLE_VBL_TRANSFER \
    _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)
665
666 #define MAX_HARDWARE_SPRITES 32
667
668 #define HARDWARE_SPRITE_CAN_FLIP_X 0
669
670 #define HARDWARE_SPRITE_CAN_FLIP_Y 0
671
672 inline void SET_SHADOW_OAM_ADDRESS(void * address) {
673     _shadow_OAM_base = (uint8_t)((uint16_t)address >> 8);
674 }
675
676 inline void set_sprite_tile(uint8_t nb, uint8_t tile) {
677     shadow_OAM[nb].tile=tile;
678 }
679
680 inline uint8_t get_sprite_tile(uint8_t nb) {
681     return shadow_OAM[nb].tile;
682 }
683
684 inline void set_sprite_prop(uint8_t nb, uint8_t prop) {
685     shadow_OAM[nb].prop = prop;
686 }
687
688 inline uint8_t get_sprite_prop(uint8_t nb) {
689     return shadow_OAM[nb].prop;
690 }
691
692 inline void move_sprite(uint8_t nb, uint8_t x, uint8_t y) {
693     OAM_item_t * itm = &shadow_OAM[nb];
694     itm->y=y, itm->x=x;
695 }
696
697 inline void scroll_sprite(uint8_t nb, int8_t x, int8_t y) {
698     OAM_item_t * itm = &shadow_OAM[nb];
699     itm->y+=y, itm->x+=x;
700 }
701
702 inline void hide_sprite(uint8_t nb) {
703     shadow_OAM[nb].y = 0xC0;
704 }
705
706 void set_vram_byte(uint8_t * addr, uint8_t v) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
707
708 uint8_t * set_attributed_tile_xy(uint8_t x, uint8_t y, uint16_t t) Z88DK_CALLEE PRESERVES_REGS(iyh,
    iyl);
709
710 uint8_t * set_tile_xy(uint8_t x, uint8_t y, uint8_t t) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
711 #define set_bkg_tile_xy set_tile_xy
712 #define set_win_tile_xy set_tile_xy
713
714 uint8_t * get_bkg_xy_addr(uint8_t x, uint8_t y) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
715 #define get_win_xy_addr get_bkg_xy_addr
716
717 #endif /* _MSX_H */

```

20.112 gbdk-lib/include/nes/nes.h File Reference

```
#include <types.h>
#include <stdint.h>
#include <gbdk/version.h>
#include <nes/hardware.h>
#include <nes/rgb_to_nes_macro.h>
```

Data Structures

- struct [joypads_t](#)
- struct [OAM_item_t](#)

Macros

- #define [NINTENDO_NES](#)
- #define [SYSTEM_BITS_NTSC](#) 0x00
- #define [SYSTEM_BITS_PAL](#) 0x40
- #define [SYSTEM_BITS_DENDY](#) 0x80
- #define [SYSTEM_60HZ](#) 0x00
- #define [SYSTEM_50HZ](#) 0x01
- #define [TIMER_VBLANK_PARITY_MODE_SYSTEM_60HZ](#) 0x78
- #define [TIMER_VBLANK_PARITY_MODE_SYSTEM_50HZ](#) 0x5D
- #define [RGB](#)(r, g, b) [RGB_TO_NES](#)((((r) | ((g) << 2) | ((b) << 4)))
- #define [RGB8](#)(r, g, b) [RGB_TO_NES](#)(((((r) >> 6) | (((g) >> 6) << 2) | (((b) >> 6) << 4)))
- #define [RGBHTML](#)(RGB24bit) [RGB_TO_NES](#)(((((RGB24bit) >> 22) | (((RGB24bit) & 0xFFFF) >> 14) << 2) | (((RGB24bit) & 0xFF) >> 6) << 4)))
- #define [RGB_RED](#) 0x16
- #define [RGB_DARKRED](#) 0x06
- #define [RGB_GREEN](#) 0x2A
- #define [RGB_DARKGREEN](#) 0x1A
- #define [RGB_BLUE](#) 0x12
- #define [RGB_DARKBLUE](#) 0x02
- #define [RGB_YELLOW](#) 0x28
- #define [RGB_DARKYELLOW](#) 0x18
- #define [RGB_CYAN](#) 0x2C
- #define [RGB_AQUA](#) 0x1C
- #define [RGB_PINK](#) 0x24
- #define [RGB_PURPLE](#) 0x14
- #define [RGB_BLACK](#) 0x0F
- #define [RGB_DARKGRAY](#) 0x00
- #define [RGB_LIGHTGRAY](#) 0x10
- #define [RGB_WHITE](#) 0x30
- #define [J_UP](#) 0x08U
- #define [J_DOWN](#) 0x04U
- #define [J_LEFT](#) 0x02U
- #define [J_RIGHT](#) 0x01U
- #define [J_A](#) 0x80U
- #define [J_B](#) 0x40U
- #define [J_SELECT](#) 0x20U
- #define [J_START](#) 0x10U
- #define [M_DRAWING](#) 0x01U
- #define [M_TEXT_OUT](#) 0x02U
- #define [M_TEXT_INOUT](#) 0x03U
- #define [M_NO_SCROLL](#) 0x04U

- `#define M_NO_INTERP 0x08U`
- `#define S_PALETTE 0x10U`
- `#define S_FLIPX 0x40U`
- `#define S_FLIPY 0x80U`
- `#define S_PRIORITY 0x20U`
- `#define S_PAL(n) n`
- `#define EMPTY_IFLAG 0x00U`
- `#define VBL_IFLAG 0x01U`
- `#define LCD_IFLAG 0x02U`
- `#define TIM_IFLAG 0x04U`
- `#define DMG_BLACK 0x03`
- `#define DMG_DARK_GRAY 0x02`
- `#define DMG_LITE_GRAY 0x01`
- `#define DMG_WHITE 0x00`
- `#define DMG_PALETTE(C0, C1, C2, C3) (((uint8_t) (((C3) & 0x03) << 6) | (((C2) & 0x03) << 4) | (((C1) & 0x03) << 2) | ((C0) & 0x03)))`
- `#define SCREENWIDTH DEVICE_SCREEN_PX_WIDTH`
- `#define SCREENHEIGHT DEVICE_SCREEN_PX_HEIGHT`
- `#define MAX_LCD_ISR_CALLS 4`
- `#define CURRENT_BANK _current_bank`
- `#define BANK(VARNAME) ( (uint8_t) & __bank_ ## VARNAME )`
- `#define BANKREF(VARNAME)`
- `#define BANKREF_EXTERN(VARNAME) extern const void __bank_ ## VARNAME;`
- `#define SWITCH_ROM_DUMMY(b)`
- `#define SWITCH_ROM_UNROM(b) _switch_prg0(b)`
- `#define SWITCH_ROM SWITCH_ROM_UNROM`
- `#define SWITCH_RAM(b) 0`
- `#define ENABLE_RAM`
- `#define DISABLE_RAM`
- `#define DISPLAY_ON display_on();`
- `#define DISPLAY_OFF display_off();`
- `#define HIDE_LEFT_COLUMN shadow_PPUMASK &= ~(PPUMASK_SHOW_BG_LC | PPUMASK_SHOW_SPR_LC);`
`\`
- `#define SHOW_LEFT_COLUMN shadow_PPUMASK |= (PPUMASK_SHOW_BG_LC | PPUMASK_SHOW_SPR_LC);`
- `#define SET_BORDER_COLOR(C)`
- `#define SHOW_BKG shadow_PPUMASK |= PPUMASK_SHOW_BG;`
- `#define HIDE_BKG shadow_PPUMASK &= ~PPUMASK_SHOW_BG;`
- `#define SHOW_SPRITES shadow_PPUMASK |= PPUMASK_SHOW_SPR;`
- `#define HIDE_SPRITES shadow_PPUMASK &= ~PPUMASK_SHOW_SPR;`
- `#define SPRITES_8x16 shadow_PPUCTRL |= PPUCTRL_SPR_8x16;`
- `#define SPRITES_8x8 shadow_PPUCTRL &= ~PPUCTRL_SPR_8x16;`
- `#define COMPAT_PALETTE(C0, C1, C2, C3) (((uint8_t) (((C3) << 6) | ((C2) << 4) | ((C1) << 2) | (C0)))`
- `#define set_bkg_2bpp_data set_bkg_data`
- `#define set_tile_map set_bkg_tiles`
- `#define set_tile_submap set_bkg_submap`
- `#define set_tile_xy set_bkg_tile_xy`
- `#define set_attribute_xy set_bkg_attribute_xy`
- `#define set_sprite_2bpp_data set_sprite_data`
- `#define DISABLE_OAM_DMA _shadow_OAM_base = 0`
- `#define DISABLE_VBL_TRANSFER DISABLE_OAM_DMA`
- `#define ENABLE_OAM_DMA _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)`
- `#define ENABLE_VBL_TRANSFER ENABLE_OAM_DMA`
- `#define MAX_HARDWARE_SPRITES 64`
- `#define HARDWARE_SPRITE_CAN_FLIP_X 1`
- `#define HARDWARE_SPRITE_CAN_FLIP_Y 1`
- `#define fill_rect fill_bkg_rect`

Typedefs

- typedef `uint8_t` `palette_color_t`
- typedef `void(* int_handler)` `(void)` `NONBANKED`
- typedef struct `OAM_item_t` `OAM_item_t`

Functions

- void `set_bkg_palette` (`uint8_t` first_palette, `uint8_t` nb_palettes, const `palette_color_t` *rgb_data) `NO_OVERLAY_LOCALS`
- void `set_sprite_palette` (`uint8_t` first_palette, `uint8_t` nb_palettes, const `palette_color_t` *rgb_data) `NO_OVERLAY_LOCALS`
- void `set_bkg_palette_entry` (`uint8_t` palette, `uint8_t` entry, `palette_color_t` rgb_data) `NO_OVERLAY_LOCALS`
- void `set_sprite_palette_entry` (`uint8_t` palette, `uint8_t` entry, `palette_color_t` rgb_data) `NO_OVERLAY_LOCALS`
- void `remove_VBL` (`int_handler` h) `NO_OVERLAY_LOCALS`
- void `remove_LCD` (`int_handler` h) `NO_OVERLAY_LOCALS`
- void `remove_TIM` (`int_handler` h) `NO_OVERLAY_LOCALS`
- void `add_VBL` (`int_handler` h) `NO_OVERLAY_LOCALS`
- void `add_LCD` (`int_handler` h) `NO_OVERLAY_LOCALS`
- void `add_TIM` (`int_handler` h) `NO_OVERLAY_LOCALS`
- void `mode` (`uint8_t` m) `NO_OVERLAY_LOCALS`
- `uint8_t` `get_mode` (`void`) `NO_OVERLAY_LOCALS`
- `uint8_t` `get_system` (`void`)
- void `delay` (`uint16_t` d) `NO_OVERLAY_LOCALS`
- `uint8_t` `joypad` (`void`) `NO_OVERLAY_LOCALS`
- `uint8_t` `waitpad` (`uint8_t` mask) `NO_OVERLAY_LOCALS`
- void `waitpadup` (`void`) `NO_OVERLAY_LOCALS`
- `uint8_t` `joypad_init` (`uint8_t` npads, `joypads_t` *joypads) `NO_OVERLAY_LOCALS`
- void `joypad_ex` (`joypads_t` *joypads) `NO_OVERLAY_LOCALS`
- void `enable_interrupts` (`void`)
- void `disable_interrupts` (`void`)
- void `set_interrupts` (`uint8_t` flags) `NO_OVERLAY_LOCALS`
- void `reset` (`void`)
- void `vsync` (`void`) `NO_OVERLAY_LOCALS`
- void `wait_vbl_done` (`void`) `NO_OVERLAY_LOCALS`
- void `display_on` (`void`) `NO_OVERLAY_LOCALS`
- void `display_off` (`void`) `NO_OVERLAY_LOCALS`
- void `refresh_OAM` (`void`) `NO_OVERLAY_LOCALS`
- void `set_vram_byte` (`uint8_t` *addr, `uint8_t` v) `NO_OVERLAY_LOCALS`
- `uint8_t` * `get_bkg_xy_addr` (`uint8_t` x, `uint8_t` y) `NO_OVERLAY_LOCALS`
- void `set_2bpp_palette` (`uint16_t` palette)
- void `set_1bpp_colors_ex` (`uint8_t` fgcolor, `uint8_t` bgcolor, `uint8_t` mode) `NO_OVERLAY_LOCALS`
- void `set_1bpp_colors` (`uint8_t` fgcolor, `uint8_t` bgcolor)
- void `set_bkg_data` (`uint8_t` first_tile, `uint8_t` nb_tiles, const `uint8_t` *data) `NO_OVERLAY_LOCALS`
- void `set_bkg_1bpp_data` (`uint8_t` first_tile, `uint8_t` nb_tiles, const `uint8_t` *data) `NO_OVERLAY_LOCALS`
- void `set_bkg_tiles` (`uint8_t` x, `uint8_t` y, `uint8_t` w, `uint8_t` h, const `uint8_t` *tiles) `NO_OVERLAY_LOCALS`
- void `set_bkg_attributes_nes16x16` (`uint8_t` x, `uint8_t` y, `uint8_t` w, `uint8_t` h, const `uint8_t` *attributes) `NO_OVERLAY_LOCALS`
- void `set_bkg_attributes` (`uint8_t` x, `uint8_t` y, `uint8_t` w, `uint8_t` h, const `uint8_t` *attributes)
- void `set_bkg_submap_attributes_nes16x16` (`uint8_t` x, `uint8_t` y, `uint8_t` w, `uint8_t` h, const `uint8_t` *map, `uint8_t` map_w) `NO_OVERLAY_LOCALS`
- void `set_bkg_submap_attributes` (`uint8_t` x, `uint8_t` y, `uint8_t` w, `uint8_t` h, const `uint8_t` *attributes, `uint8_t` map_w)
- void `set_bkg_based_tiles` (`uint8_t` x, `uint8_t` y, `uint8_t` w, `uint8_t` h, const `uint8_t` *tiles, `uint8_t` base_tile)

- void `set_bkg_submap` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w) NO_OVERLAY_LOCALS
- void `set_bkg_based_submap` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w, uint8_t base_tile)
- void `get_bkg_tiles` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *tiles) NO_OVERLAY_LOCALS
- uint8_t * `set_bkg_tile_xy` (uint8_t x, uint8_t y, uint8_t t) NO_OVERLAY_LOCALS
- void `set_bkg_attribute_xy_nes16x16` (uint8_t x, uint8_t y, uint8_t a) NO_OVERLAY_LOCALS
- void `set_bkg_attribute_xy` (uint8_t x, uint8_t y, uint8_t a)
- uint8_t `get_bkg_tile_xy` (uint8_t x, uint8_t y) NO_OVERLAY_LOCALS
- void `move_bkg` (scroll_x_t x, scroll_y_t y)
- void `scroll_bkg` (int8_t x, int8_t y)
- void `set_sprite_data` (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS
- void `set_sprite_1bpp_data` (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS
- void `SET_SHADOW_OAM_ADDRESS` (void *address)
- void `set_sprite_tile` (uint8_t nb, uint8_t tile) NO_OVERLAY_LOCALS
- uint8_t `get_sprite_tile` (uint8_t nb) NO_OVERLAY_LOCALS
- void `set_sprite_prop` (uint8_t nb, uint8_t prop) NO_OVERLAY_LOCALS
- uint8_t `get_sprite_prop` (uint8_t nb) NO_OVERLAY_LOCALS
- void `move_sprite` (uint8_t nb, uint8_t x, uint8_t y) NO_OVERLAY_LOCALS
- void `scroll_sprite` (uint8_t nb, int8_t x, int8_t y) NO_OVERLAY_LOCALS
- void `hide_sprite` (uint8_t nb) NO_OVERLAY_LOCALS
- void `set_data` (uint8_t *vram_addr, const uint8_t *data, uint16_t len) NO_OVERLAY_LOCALS
- void `set_tiles` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *vram_addr, const uint8_t *tiles) NO_OVERLAY_LOCALS
- void `set_tile_data` (uint16_t first_tile, uint8_t nb_tiles, const uint8_t *data)
- void `set_bkg_native_data` (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS
- void `set_sprite_native_data` (uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS
- void `set_native_tile_data` (uint16_t first_tile, uint8_t nb_tiles, const uint8_t *data)
- void `init_bkg` (uint8_t c) NO_OVERLAY_LOCALS
- void `vmemset` (void *s, uint8_t c, size_t n) NO_OVERLAY_LOCALS
- void `fill_bkg_rect` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t tile) NO_OVERLAY_LOCALS
- void `flush_shadow_attributes` (void) NO_OVERLAY_LOCALS
- uint8_t `switch_prg0` (uint8_t bank) NO_OVERLAY_LOCALS

Variables

- const uint8_t `_SYSTEM`
- volatile uint16_t `sys_time`
- volatile uint8_t `current_bank`
- uint16_t `current_1bpp_colors`
- volatile struct `OAM_item_t shadow_OAM` []
- uint8_t `shadow_OAM_base`

20.112.1 Detailed Description

NES specific functions.

20.112.2 Macro Definition Documentation

20.112.2.1 NINTENDO_NES `#define NINTENDO_NES`

20.112.2.2 SYSTEM_BITS_NTSC `#define SYSTEM_BITS_NTSC 0x00`

20.112.2.3 SYSTEM_BITS_PAL `#define SYSTEM_BITS_PAL 0x40`

20.112.2.4 SYSTEM_BITS_DENDY `#define SYSTEM_BITS_DENDY 0x80`

20.112.2.5 SYSTEM_60HZ `#define SYSTEM_60HZ 0x00`

20.112.2.6 SYSTEM_50HZ `#define SYSTEM_50HZ 0x01`

20.112.2.7 TIMER_VBLANK_PARITY_MODE_SYSTEM_60HZ `#define TIMER_VBLANK_PARITY_MODE_↵
SYSTEM_60HZ 0x78`

20.112.2.8 TIMER_VBLANK_PARITY_MODE_SYSTEM_50HZ `#define TIMER_VBLANK_PARITY_MODE_↵
SYSTEM_50HZ 0x5D`

20.112.2.9 RGB `#define RGB(
 r,
 g,
 b) RGB_TO_NES(((r) | ((g) << 2) | ((b) << 4)))`

20.112.2.10 RGB8 `#define RGB8(
 r,
 g,
 b) RGB_TO_NES((((r) >> 6) | (((g) >> 6) << 2) | (((b) >> 6) << 4)))`

20.112.2.11 RGBHTML `#define RGBHTML(
 RGB24bit) RGB_TO_NES((((RGB24bit) >> 22) | (((RGB24bit) & 0xFFFF) >> 14) <<
2) | (((RGB24bit) & 0xFF) >> 6) << 4)))`

20.112.2.12 RGB_RED `#define RGB_RED 0x16`
Common colors based on the EGA default palette.
Manually entered from https://www.nesdev.org/wiki/PPU_palettes#RGBI

20.112.2.13 RGB_DARKRED `#define RGB_DARKRED 0x06`

20.112.2.14 RGB_GREEN `#define RGB_GREEN 0x2A`

20.112.2.15 RGB_DARKGREEN `#define RGB_DARKGREEN 0x1A`

20.112.2.16 RGB_BLUE `#define RGB_BLUE 0x12`

20.112.2.17 RGB_DARKBLUE `#define RGB_DARKBLUE 0x02`

20.112.2.18 RGB_YELLOW `#define RGB_YELLOW 0x28`

20.112.2.19 RGB_DARKYELLOW `#define RGB_DARKYELLOW 0x18`

20.112.2.20 RGB_CYAN `#define RGB_CYAN 0x2C`

20.112.2.21 RGB_AQUA `#define RGB_AQUA 0x1C`

20.112.2.22 RGB_PINK `#define RGB_PINK 0x24`

20.112.2.23 RGB_PURPLE `#define RGB_PURPLE 0x14`

20.112.2.24 RGB_BLACK `#define RGB_BLACK 0x0F`

20.112.2.25 RGB_DARKGRAY `#define RGB_DARKGRAY 0x00`

20.112.2.26 RGB_LIGHTGRAY `#define RGB_LIGHTGRAY 0x10`

20.112.2.27 RGB_WHITE `#define RGB_WHITE 0x30`

20.112.2.28 J_UP `#define J_UP 0x08U`

Joypad bits. A logical OR of these is used in the `wait_pad` and `joypad` functions. For example, to see if the B button is pressed try

```
uint8_t keys; keys = joypad(); if (keys & J_B) { ... }
```

See also

[joypad](#)

20.112.2.29 J_DOWN `#define J_DOWN 0x04U`

20.112.2.30 J_LEFT `#define J_LEFT 0x02U`

20.112.2.31 J_RIGHT `#define J_RIGHT 0x01U`

20.112.2.32 J_A `#define J_A 0x80U`

20.112.2.33 J_B `#define J_B 0x40U`

20.112.2.34 J_SELECT `#define J_SELECT 0x20U`

20.112.2.35 J_START `#define J_START 0x10U`

20.112.2.36 M_DRAWING `#define M_DRAWING 0x01U`

Screen modes. Normally used by internal functions only.

See also

[mode\(\)](#)

20.112.2.37 M_TEXT_OUT `#define M_TEXT_OUT 0x02U`

20.112.2.38 M_TEXT_INOUT `#define M_TEXT_INOUT 0x03U`

20.112.2.39 M_NO_SCROLL `#define M_NO_SCROLL 0x04U`

Set this in addition to the others to disable scrolling

If scrolling is disabled, the cursor returns to (0,0)

See also

[mode\(\)](#)

20.112.2.40 M_NO_INTERP `#define M_NO_INTERP 0x08U`

Set this to disable interpretation

See also

[mode\(\)](#)

20.112.2.41 S_PALETTE `#define S_PALETTE 0x10U`

If this is set, sprite colours come from OBJ1PAL. Else they come from OBJ0PAL

See also

[set_sprite_prop\(\)](#).

20.112.2.42 S_FLIPX `#define S_FLIPX 0x40U`

If set the sprite will be flipped horizontally.

See also

[set_sprite_prop\(\)](#)

20.112.2.43 S_FLIPY `#define S_FLIPY 0x80U`

If set the sprite will be flipped vertically.

See also

[set_sprite_prop\(\)](#)

20.112.2.44 S_PRIORITY `#define S_PRIORITY 0x20U`

If this bit is clear, then the sprite will be displayed on top of the background and window.

See also

[set_sprite_prop\(\)](#)

20.112.2.45 S_PAL `#define S_PAL(
n) n`

Defines how palette number is encoded in OAM. Required for the png2asset tool's metasprite output.

20.112.2.46 EMPTY_IFLAG `#define EMPTY_IFLAG 0x00U`

Disable calling of interrupt service routines

20.112.2.47 VBL_IFLAG `#define VBL_IFLAG 0x01U`

VBlank Interrupt occurs at the start of the vertical blank.

During this period the video ram may be freely accessed.

See also

[set_interrupts\(\)](#),

[add_VBL](#)

20.112.2.48 LCD_IFLAG `#define LCD_IFLAG 0x02U`

LCD Interrupt when triggered by the STAT register.

See also

[set_interrupts\(\)](#),

[add_LCD](#)

20.112.2.49 TIM_IFLAG `#define TIM_IFLAG 0x04U`

Timer Interrupt when the timer [TIMA_REG](#) overflows.

See also

[set_interrupts\(\)](#),

[add_TIM](#)

20.112.2.50 DMG_BLACK `#define DMG_BLACK 0x03`**20.112.2.51 DMG_DARK_GRAY** `#define DMG_DARK_GRAY 0x02`**20.112.2.52 DMG_LITE_GRAY** `#define DMG_LITE_GRAY 0x01`**20.112.2.53 DMG_WHITE** `#define DMG_WHITE 0x00`

20.112.2.54 DMG_PALETTE `#define DMG_PALETTE(`
`C0,`
`C1,`
`C2,`
`C3 ) ((uint8_t) (((C3) & 0x03) << 6) | (((C2) & 0x03) << 4) | (((C1) & 0x03) <<`
`2) | (((C0) & 0x03)))`

Macro to create a DMG palette from 4 colors

Parameters

<i>C0</i>	Color for Index 0
<i>C1</i>	Color for Index 1
<i>C2</i>	Color for Index 2
<i>C3</i>	Color for Index 3

The resulting format is four greyscale colors packed into a single unsigned byte.

Example:

```
REG_BGP = DMG_PALETTE(DMG_BLACK, DMG_DARK_GRAY, DMG_LITE_GRAY, DMG_WHITE);
```

See also

[OBP0_REG](#), [OBP1_REG](#), [BGP_REG](#)

[DMG_BLACK](#), [DMG_DARK_GRAY](#), [DMG_LITE_GRAY](#), [DMG_WHITE](#)

20.112.2.55 SCREENWIDTH `#define SCREENWIDTH DEVICE_SCREEN_PX_WIDTH`

Width of the visible screen in pixels.

20.112.2.56 SCREENHEIGHT `#define SCREENHEIGHT DEVICE_SCREEN_PX_HEIGHT`

Height of the visible screen in pixels.

20.112.2.57 MAX_LCD_ISR_CALLS `#define MAX_LCD_ISR_CALLS 4`

The maximum number of times the LCD handler will be called per frame.

20.112.2.58 CURRENT_BANK `#define CURRENT_BANK __current_bank`

20.112.2.59 BANK `#define BANK(`
`VARNAME ) ( (uint8_t) & __bank_ ## VARNAME )`

Obtains the **bank number** of VARNAME

Parameters

<i>VARNAME</i>	Name of the variable which has a <code>__bank_VARNAME</code> companion symbol which is adjusted by bankpack
----------------	---

Use this to obtain the bank number from a bank reference created with [BANKREF\(\)](#).

See also

[BANKREF_EXTERN\(\)](#), [BANKREF\(\)](#)

20.112.2.60 BANKREF `#define BANKREF(`
`VARNAME )`

Value:

```
void __func_ ## VARNAME(void) __banked __naked { \
```

```
__asm \
    .local b__func_ ## VARNAME \
    __bank_ ## VARNAME = b__func_ ## VARNAME \
    .globl __bank_ ## VARNAME \
__endasm; \
}
```

Creates a reference for retrieving the bank number of a variable or function

Parameters

<i>VARNAME</i>	Variable name to use, which may be an existing identifier
----------------	---

See also

[BANK\(\)](#) for obtaining the bank number of the included data.

More than one [BANKREF \(\)](#) may be created per file, but each call should always use a unique VARNAME. Use [BANKREF_EXTERN\(\)](#) within another source file to make the variable and it's data accesible there.

20.112.2.61 BANKREF_EXTERN #define BANKREF_EXTERN(
 VARNAME) extern const void __bank_ ## VARNAME;

Creates extern references for accessing a [BANKREF\(\)](#) generated variable.

Parameters

<i>VARNAME</i>	Name of the variable used with BANKREF()
----------------	--

This makes a [BANKREF\(\)](#) reference in another source file accessible in the current file for use with [BANK\(\)](#).

See also

[BANKREF\(\)](#), [BANK\(\)](#)

20.112.2.62 SWITCH_ROM_DUMMY #define SWITCH_ROM_DUMMY(
 b)

Dummy macro for no-bank-switching WIP prototype

Parameters

<i>b</i>	ROM bank to switch to
----------	-----------------------

20.112.2.63 SWITCH_ROM_UNROM #define SWITCH_ROM_UNROM(
 b) __switch_prg0(*b*)

Macro for simple UNROM-like switching (write bank# to single 8-bit register)

Parameters

<i>b</i>	ROM bank to switch to
----------	-----------------------

20.112.2.64 SWITCH_ROM #define SWITCH_ROM [SWITCH_ROM_UNROM](#)

Makes default mapper switch the active ROM bank

Parameters

<i>b</i>	ROM bank to switch to (max 255)
----------	---------------------------------

See also

[SWITCH_ROM_UNROM](#)

20.112.2.65 SWITCH_RAM `#define SWITCH_RAM(
 b) 0`

No-op at the moment. Placeholder for future mappers / test compatibility.

Parameters

<i>b</i>	SRAM bank to switch to
----------	------------------------

20.112.2.66 ENABLE_RAM `#define ENABLE_RAM`

No-op at the moment. Placeholder for future mappers / test compatibility.

20.112.2.67 DISABLE_RAM `#define DISABLE_RAM`

No-op at the moment. Placeholder for future mappers / test compatibility.

20.112.2.68 DISPLAY_ON `#define DISPLAY_ON display\_on\(\);`

Turns the display back on.

See also

[display_off](#), [DISPLAY_OFF](#)

20.112.2.69 DISPLAY_OFF `#define DISPLAY_OFF display\_off\(\);`

Turns the display off immediately.

See also

[display_off](#), [DISPLAY_ON](#)

20.112.2.70 HIDE_LEFT_COLUMN `#define HIDE_LEFT_COLUMN shadow_PPUMASK &= ~(PPUMASK_SHOW_BG_LC
| PPUMASK_SHOW_SPR_LC); \`

Blanks leftmost column, so it is not garbaged when you use horizontal scroll

See also

[SHOW_LEFT_COLUMN](#)

20.112.2.71 SHOW_LEFT_COLUMN `#define SHOW_LEFT_COLUMN shadow_PPUMASK |= (PPUMASK_SHOW_BG_LC
| PPUMASK_SHOW_SPR_LC);`

Shows leftmost column

See also

[HIDE_LEFT_COLUMN](#)

20.112.2.72 SET_BORDER_COLOR `#define SET_BORDER_COLOR(
C)`

Does nothing for NES not implemented yet

20.112.2.73 SHOW_BKG `#define SHOW_BKG shadow_PPUMASK |= PPUMASK_SHOW_BG;`
Turns on the background layer. Sets bit 0 of the LCDC register to 1.

20.112.2.74 HIDE_BKG `#define HIDE_BKG shadow_PPUMASK &= ~PPUMASK_SHOW_BG;`
Turns off the background layer. Sets bit 0 of the LCDC register to 0.

20.112.2.75 SHOW_SPRITES `#define SHOW_SPRITES shadow_PPUMASK |= PPUMASK_SHOW_SPR;`
Turns on the sprites layer. Sets bit 1 of the LCDC register to 1.

20.112.2.76 HIDE_SPRITES `#define HIDE_SPRITES shadow_PPUMASK &= ~PPUMASK_SHOW_SPR;`
Turns off the sprites layer. Clears bit 1 of the LCDC register to 0.

20.112.2.77 SPRITES_8x16 `#define SPRITES_8x16 shadow_PPUCTRL |= PPUCTRL_SPR_8X16;`
Sets sprite size to 8x16 pixels, two tiles one above the other. Sets bit 2 of the LCDC register to 1.

20.112.2.78 SPRITES_8x8 `#define SPRITES_8x8 shadow_PPUCTRL &= ~PPUCTRL_SPR_8X16;`
Sets sprite size to 8x8 pixels, one tile. Clears bit 2 of the LCDC register to 0.

20.112.2.79 COMPAT_PALETTE `#define COMPAT_PALETTE(
C0,
C1,
C2,
C3) ((uint8_t)((C3) << 6) | ((C2) << 4) | ((C1) << 2) | (C0))`

20.112.2.80 set_bkg_2bpp_data `#define set_bkg_2bpp_data set_bkg_data`

20.112.2.81 set_tile_map `#define set_tile_map set_bkg_tiles`

20.112.2.82 set_tile_submap `#define set_tile_submap set_bkg_submap`

20.112.2.83 set_tile_xy `#define set_tile_xy set_bkg_tile_xy`

20.112.2.84 set_attribute_xy `#define set_attribute_xy set_bkg_attribute_xy`

20.112.2.85 set_sprite_2bpp_data `#define set_sprite_2bpp_data set_sprite_data`

20.112.2.86 DISABLE_OAM_DMA `#define DISABLE_OAM_DMA _shadow_OAM_base = 0`

20.112.2.87 DISABLE_VBL_TRANSFER `#define DISABLE_VBL_TRANSFER DISABLE_OAM_DMA`
Disable OAM DMA copy each VBlank

20.112.2.88 ENABLE_OAM_DMA `#define ENABLE_OAM_DMA _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)`

20.112.2.89 ENABLE_VBL_TRANSFER `#define ENABLE_VBL_TRANSFER ENABLE_OAM_DMA`
Enable OAM DMA copy each VBlank and set it to transfer default shadow_OAM array

20.112.2.90 MAX_HARDWARE_SPRITES `#define MAX_HARDWARE_SPRITES 64`
Amount of hardware sprites in OAM

20.112.2.91 HARDWARE_SPRITE_CAN_FLIP_X `#define HARDWARE_SPRITE_CAN_FLIP_X 1`
True if sprite hardware can flip sprites by X (horizontally)

20.112.2.92 HARDWARE_SPRITE_CAN_FLIP_Y `#define HARDWARE_SPRITE_CAN_FLIP_Y 1`
True if sprite hardware can flip sprites by Y (vertically)

20.112.2.93 fill_rect `#define fill_rect fill_bkg_rect`

20.112.3 Typedef Documentation

20.112.3.1 palette_color_t `typedef uint8_t palette_color_t`

20.112.3.2 int_handler `typedef void(* int_handler) (void) NONBANKED`
Interrupt handlers

20.112.3.3 OAM_item_t `typedef struct OAM_item_t OAM_item_t`
Sprite Attributes structure

Parameters

<i>x</i>	X Coordinate of the sprite on screen
<i>y</i>	Y Coordinate of the sprite on screen - 1
<i>tile</i>	Sprite tile number (see set_sprite_tile)
<i>prop</i>	OAM Property Flags (see set_sprite_prop)

20.112.4 Function Documentation

20.112.4.1 set_bkg_palette() `void set_bkg_palette (`
`uint8_t first_palette,`
`uint8_t nb_palettes,`
`const palette_color_t * rgb_data )`

20.112.4.2 set_sprite_palette() `void set_sprite_palette (`
`uint8_t first_palette,`
`uint8_t nb_palettes,`
`const palette_color_t * rgb_data )`

20.112.4.3 set_bkg_palette_entry() void set_bkg_palette_entry (
 uint8_t palette,
 uint8_t entry,
 palette_color_t rgb_data)

20.112.4.4 set_sprite_palette_entry() void set_sprite_palette_entry (
 uint8_t palette,
 uint8_t entry,
 palette_color_t rgb_data)

20.112.4.5 remove_VBL() void remove_VBL (
 int_handler h)

The remove functions will remove any interrupt handler.

A handler of NULL will cause bad things to happen if the given interrupt is enabled.

Removes the VBL interrupt handler.

See also

[add_VBL\(\)](#)

Removes the VBL interrupt handler.

See also

[add_VBL\(\)](#)

20.112.4.6 remove_LCD() void remove_LCD (
 int_handler h)

Removes the LCD interrupt handler.

See also

[add_LCD\(\)](#), [remove_VBL\(\)](#)

20.112.4.7 remove_TIM() void remove_TIM (
 int_handler h)

Removes the TIM interrupt handler.

See also

[add_TIM\(\)](#), [remove_VBL\(\)](#)

20.112.4.8 add_VBL() void add_VBL (
 int_handler h)

Adds a Vertical Blanking interrupt handler.

Parameters

<i>h</i>	The handler to be called whenever a V-blank interrupt occurs.
----------	---

Only a single handler is currently supported for NES.

Do not use the function definition attributes [CRITICAL](#) and [INTERRUPT](#) when declaring ISR functions added via [add_VBL\(\)](#) (or LCD, etc). Those attributes are only required when constructing a bare jump from the interrupt vector

itself (such as with [ISR_VECTOR\(\)](#)).

ISR handlers added using [add_VBL\(\)](#)/etc are instead called via the GBDK ISR dispatcher which makes the extra function attributes unnecessary.

Note

The default GBDK VBL is installed automatically.

On the current NES implementation, this handler is actually faked, and called before vblank occurs, by [vsync\(\)](#). Writes to PPU registers should be done to the shadow_ versions, so they are updated by the default VBL handler only when vblank actually occurs.

See also

[ISR_VECTOR\(\)](#)

Adds a V-blank interrupt handler.

20.112.4.9 add_LCD() `void add_LCD (`
`int_handler h )`

Adds a LCD interrupt handler.

Called when the scanline matches the `_lcd_scanline` variables.

Only a single handler is currently supported for NES.

The use-case is to indicate to the user when the video hardware is about to redraw a given LCD line. This can be useful for dynamically controlling the scrolling registers to perform special video effects.

Do not use the function definition attributes [CRITICAL](#) and [INTERRUPT](#) when declaring ISR functions added via [add_VBL\(\)](#) (or LCD, etc). Those attributes are only required when constructing a bare jump from the interrupt vector itself (such as with [ISR_VECTOR\(\)](#)).

ISR handlers added using [add_VBL\(\)](#)/etc are instead called via the GBDK ISR dispatcher which makes the extra function attributes unnecessary.

Note

On the current NES implementation, this handler is actually faked, and called by the default VBL handler after a manual delay loop. Only one such faked "interrupt" is possible per frame. This means the CPU cycles wasted in the delay loop increase with higher values of `_lcd_scanline`. In practice, it makes this functionality mostly suited for a top status bar.

See also

[add_VBL](#), [nowait_int_handler](#), [ISR_VECTOR\(\)](#)

Adds a LCD interrupt handler.

20.112.4.10 add_TIM() `void add_TIM (`
`int_handler h )`

Adds a timer interrupt handler.

Can not be used together with [add_low_priority_TIM](#)

This interrupt handler is invoked at the end of the NMI handler for gbdk-nes, after first processing the registers writes done by the VBL and and LCD handlers. It is therefore currently limited to 60Hz / 50Hz (depending on system).

Note

Make sure to wrap TIM interrupt handlers with a nooverlay pragma. For more details see [docs_nes_tim_overlay](#)

See also

[add_VBL](#)

[set_interrupts\(\)](#) with [TIM_IFLAG](#), [ISR_VECTOR\(\)](#)

20.112.4.11 mode() `void mode (`
`uint8_t m )`

Set the current screen mode - one of M_* modes
 Normally used by internal functions only.

See also

[M_DRAWING](#), [M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.112.4.12 get_mode() `uint8_t get_mode (`
`void )`

Returns the current mode

See also

[M_DRAWING](#), [M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.112.4.13 get_system() `uint8_t get_system (`
`void ) [inline]`

Returns the system gbdk is running on.

20.112.4.14 delay() `void delay (`
`uint16_t d )`

Delays the given number of milliseconds. Uses no timers or interrupts, and can be called with interrupts disabled

20.112.4.15 joypad() `uint8_t joypad (`
`void )`

Reads and returns the current state of the joypad. Return value is an OR of J_*
 When testing for multiple different buttons, it's best to read the joypad state *once* into a variable and then test using that variable.

See also

[J_START](#), [J_SELECT](#), [J_A](#), [J_B](#), [J_UP](#), [J_DOWN](#), [J_LEFT](#), [J_RIGHT](#)

Reads and returns the current state of the joypad. Follows Nintendo's guidelines for reading the pad. Return value is an OR of J_*

When testing for multiple different buttons, it's best to read the joypad state *once* into a variable and then test using that variable.

See also

[J_START](#), [J_SELECT](#), [J_A](#), [J_B](#), [J_UP](#), [J_DOWN](#), [J_LEFT](#), [J_RIGHT](#)

Reads and returns the current state of the joypad.

20.112.4.16 waitpad() `uint8_t waitpad (`
`uint8_t mask )`

Waits until at least one of the buttons given in mask are pressed.

Normally only used for checking one key, but it will support many, even J_LEFT at the same time as J_RIGHT. :)

See also

[joypad](#)

[J_START](#), [J_SELECT](#), [J_A](#), [J_B](#), [J_UP](#), [J_DOWN](#), [J_LEFT](#), [J_RIGHT](#)

Waits until at least one of the buttons given in mask are pressed.

Parameters

<i>mask</i>	Bitmask indicating which buttons to wait for
-------------	--

Normally only used for checking one key, but it will support many, even J_LEFT at the same time as J_RIGHT. :)

Note

Checks in a loop that doesn't HALT at all, so the CPU will be maxed out until this call returns.

See also

[joypad](#)

[J_START](#), [J_SELECT](#), [J_A](#), [J_B](#), [J_UP](#), [J_DOWN](#), [J_LEFT](#), [J_RIGHT](#)

Waits until at least one of the buttons given in mask are pressed.

20.112.4.17 waitpadup() `void waitpadup (`
`void )`

Waits for the directional pad and all buttons to be released.

Waits for the directional pad and all buttons to be released.

Note

Checks in a loop that doesn't HALT at all, so the CPU will be maxed out until this call returns.

Waits for the directional pad and all buttons to be released.

Note: Checks in a loop that doesn't HALT at all, so the CPU will be maxed out until this call returns.

20.112.4.18 joypad_init() `uint8_t joypad_init (`
`uint8_t npads,`
`joypads_t * joypads )`

Initializes [joypads_t](#) structure for polling multiple joypads

Parameters

<i>npads</i>	number of joypads requested (1, 2 or 4)
<i>joypads</i>	pointer to joypads_t structure to be initialized

Only required for [joypad_ex](#), not required for calls to regular [joypad\(\)](#)

Returns

number of joypads available

See also

[joypad_ex\(\)](#), [joypads_t](#)

20.112.4.19 joypad_ex() `void joypad_ex (`
`joypads_t * joypads )`

Polls all available joypads

See also

[joypad_init\(\)](#), [joypads_t](#)

Polls all available joypads (for the GB and ones connected via SGB)

Parameters

<i>joypads</i>	pointer to joypads_t structure to be filled with joypad statuses, must be previously initialized with joypad_init()
----------------	---

See also

[joypad_init\(\)](#), [joypads_t](#)

Polls all available joypads

Parameters

<i>joypads</i>	pointer to joypads_t structure to be filled with joypad statuses, must be previously initialized with joypad_init()
----------------	---

See also

[joypad_init\(\)](#), [joypads_t](#)

20.112.4.20 [enable_interrupts\(\)](#) `void enable_interrupts (`
`void ) [inline]`

Enables unmasked interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

See also

[disable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.112.4.21 [disable_interrupts\(\)](#) `void disable_interrupts (`
`void ) [inline]`

Disables interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

This function may be called as many times as you like; however the first call to [enable_interrupts](#) will re-enable them.

See also

[enable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.112.4.22 [set_interrupts\(\)](#) `void set_interrupts (`
`uint8_t flags )`

Sets the interrupt mask to flags.

Parameters

<i>flags</i>	A logical OR of *_IFLAGS
--------------	--------------------------

See also

[VBL_IFLAG](#), [LCD_IFLAG](#), [TIM_IFLAG](#)

Clears any pending interrupts and sets the interrupt mask register IO to flags.

Parameters

<i>flags</i>	A logical OR of *_IFLAGS
--------------	--------------------------

Note

This disables and then re-enables interrupts so it must be used outside of a critical section.

See also

[enable_interrupts\(\)](#), [disable_interrupts\(\)](#)

[VBL_IFLAG](#), [LCD_IFLAG](#), [TIM_IFLAG](#), [SIO_IFLAG](#), [JOY_IFLAG](#)

20.112.4.23 reset() `void reset (`
`void ) [inline]`

Performs a soft reset.

For the Game Boy and related it does this by jumping to address 0x0150 which is in crt0.s (the c-runtime that executes before main() is called).

This performs various startup steps such as resetting the stack, clearing WRAM and OAM, resetting initialized variables and some display registers (scroll, window, LCDC), etc.

This is not the same a hard power reset.

20.112.4.24 vsync() `void vsync (`
`void )`

Waits for the vertical blank interrupt.

This is often used in main loops to idle the CPU until it's time to start the next frame. It's also useful for syncing animation with the screen re-draw.

Warning: If the VBL interrupt is disabled, this function will never return.

HALTs the CPU and waits for the vertical blank interrupt and then returns when all registered VBL ISRs have completed.

This is often used in main loops to idle the CPU at low power until it's time to start the next frame. It's also useful for syncing animation with the screen re-draw.

Warning: If the VBL interrupt is disabled, this function will never return. If the screen is off this function returns immediately.

HALTs the CPU and waits for the vertical blank interrupt.

This is often used in main loops to idle the CPU at low power until it's time to start the next frame. It's also useful for syncing animation with the screen re-draw.

Warning: If the VBL interrupt is disabled, this function will never return. If the screen is off this function returns immediately.

20.112.4.25 wait_vbl_done() `void wait_vbl_done (`
`void )`

Obsolete. This function has been replaced by [vsync\(\)](#), which has identical behavior.

20.112.4.26 display_on() `void display_on (`
`void )`

Turns the display on.

See also

[DISPLAY_ON](#)

20.112.4.27 display_off() `void display_off (`
`void ) [inline]`

Turns the display off immediately.

See also

[DISPLAY_ON](#)

Turns the display off.

Waits until the VBL before turning the display off.

See also

[DISPLAY_ON](#)

Turns the display off.

See also

[DISPLAY_ON](#)

20.112.4.28 refresh_OAM() `void refresh_OAM (`
`void )`

Copies data from shadow OAM to OAM

20.112.4.29 set_vram_byte() `void set_vram_byte (`
`uint8_t * addr,`
`uint8_t v )`

Set byte in vram at given memory location

Parameters

<i>addr</i>	address to write to
<i>v</i>	value

20.112.4.30 get_bkg_xy_addr() `uint8_t * get_bkg_xy_addr (`
`uint8_t x,`
`uint8_t y )`

Get address of X,Y tile of background map

20.112.4.31 set_2bpp_palette() `void set_2bpp_palette (`
`uint16_t palette ) [inline]`

Sets palette for 2bpp color translation for GG/SMS, does nothing on GB

20.112.4.32 set_1bpp_colors_ex() `void set_1bpp_colors_ex (`
`uint8_t fgcolor,`
`uint8_t bgcolor,`
`uint8_t mode )`

20.112.4.33 set_1bpp_colors() `void set_1bpp_colors (`
`uint8_t fgcolor,`
`uint8_t bgcolor ) [inline]`

20.112.4.34 set_bkg_data() `void set_bkg_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data )`

Sets VRAM Tile Pattern data for the Background

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**. Each Tile is 16 bytes in size (8x8 pixels, 2 bits-per-pixel).

Note: Sprite Tiles 128-255 share the same memory region as Background Tiles 128-255.

See also

[set_tile_data](#)

Sets VRAM Tile Pattern data for the Background / Window

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (2 bpp) source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**. Each Tile is 16 bytes in size (8x8 pixels, 2 bits-per-pixel).

Note

Sprite Tiles 128-255 share the same memory region as Background Tiles 128-255.

GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- `VBK_REG = VBK_BANK_0` indicates the first bank
- `VBK_REG = VBK_BANK_1` indicates the second

See also

[set_win_data](#), [set_tile_data](#)

20.112.4.35 set_bkg_1bpp_data() `void set_bkg_1bpp_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data )`

Sets VRAM Tile Pattern data for the Background using 1bpp source data

Similar to [set_bkg_data](#), except source data is 1 bit-per-pixel which gets expanded into 2 bits-per-pixel.

For a given bit that represent a pixel:

- 0 will be expanded into color 0
- 1 will be expanded into color 1, 2 or 3 depending on color argument

See also

[SHOW_BKG](#), [HIDE_BKG](#), [set_bkg_tiles](#)

Sets VRAM Tile Pattern data for the Background / Window using 1bpp source data

Parameters

<i>first_tile</i>	Index of the first Tile to write
<i>nb_tiles</i>	Number of Tiles to write
<i>data</i>	Pointer to (1bpp) source Tile Pattern data

Similar to [set_bkg_data](#), except source data is 1 bit-per-pixel which gets expanded into 2 bits-per-pixel. For a given bit that represent a pixel:

- 0 will be expanded into the Background color
- 1 will be expanded into the Foreground color

See [set_1bpp_colors](#) for details about setting the Foreground and Background colors.

See also

[SHOW_BKG](#), [HIDE_BKG](#), [set_bkg_tiles](#)
[set_win_1bpp_data](#), [set_sprite_1bpp_data](#)

20.112.4.36 set_bkg_tiles() `void set_bkg_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles )`

Sets a rectangular region of Background Tile Map.

Entries are copied from map at **tiles** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

Use [set_bkg_submap\(\)](#) instead when:

- Source map is wider than 32 tiles.
- Writing a width that does not match the source map width **and** more than one row high at a time.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

See also

[SHOW_BKG](#)
[set_bkg_data](#), [set_bkg_submap](#), [set_win_tiles](#), [set_tiles](#)

Sets a rectangular region of Background Tile Map.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data

Entries are copied from map at **tiles** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

Use [set_bkg_submap\(\)](#) instead when:

- Source map is wider than 32 tiles.
- Writing a width that does not match the source map width **and** more than one row high at a time.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

Note

Patterns 128-255 overlap with patterns 128-255 of the sprite Tile Pattern table.

GBC only: [VBK_REG](#) determines whether Tile Numbers or Tile Attributes get set.

- `VBK_REG = VBK_TILES` Tile Numbers are written
- `VBK_REG = VBK_ATTRIBUTES` Tile Attributes are written

GBC Tile Attributes are defined as:

- Bit 7 - Priority flag. When this is set, it puts the tile above the sprites with colour 0 being transparent.
0: Below sprites
1: Above sprites
Note: [SHOW_BKG](#) needs to be set for these priorities to take place.
- Bit 6 - Vertical flip. Dictates which way up the tile is drawn vertically.
0: Normal
1: Flipped Vertically
- Bit 5 - Horizontal flip. Dictates which way up the tile is drawn horizontally.
0: Normal
1: Flipped Horizontally
- Bit 4 - Not used
- Bit 3 - Character Bank specification. Dictates from which bank of Background Tile Patterns the tile is taken.
0: Bank 0
1: Bank 1
- Bit 2 - See bit 0.
- Bit 1 - See bit 0.
- Bit 0 - Bits 0-2 indicate which of the 7 BKG colour palettes the tile is assigned.

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_submap](#), [set_win_tiles](#), [set_tiles](#)

20.112.4.37 `set_bkg_attributes_nes16x16()` `void set_bkg_attributes_nes16x16 (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * attributes )`

Sets a rectangular region of Background Tile Map Attributes.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 15
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 14
<i>w</i>	Width of area to set in tiles. Range 1 - 16
<i>h</i>	Height of area to set in tiles. Range 1 - 15
<i>attributes</i>	Pointer to source tile map attribute data

Entries are copied from map at **tiles** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

NES 16x16 Tile Attributes are tightly packed into 4 attributes per byte, with each 16x16 area of a 32x32 pixel block using the bits as follows: D1-D0: Top-left 16x16 pixels D3-D2: Top-right 16x16 pixels D5-D4: Bottom-left 16x16 pixels D7-D6: Bottom-right 16x16 pixels

https://www.nesdev.org/wiki/PPU_attribute_tables

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_submap_attributes](#), [set_win_tiles](#), [set_tiles](#)

20.112.4.38 set_bkg_attributes() void set_bkg_attributes (
 uint8_t x,
 uint8_t y,
 uint8_t w,
 uint8_t h,
 const uint8_t * attributes) [inline]

Sets a rectangular region of Background Tile Map Attributes.

Entries are copied from map at **tiles** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

Use [set_bkg_submap_attributes\(\)](#) instead when:

- Source map is wider than 32 tiles.
- Writing a width that does not match the source map width **and** more than one row high at a time.

One byte per source tile map attribute entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

Please note that this is just a wrapper function for [set_bkg_attributes_nes16x16\(\)](#) and divides the coordinates and dimensions by 2 to achieve this. It is intended to make code more portable by using the same coordinate system that systems with the much more common 8x8 attribute resolution would use.

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_submap_attributes](#), [set_win_tiles](#), [set_tiles](#)

20.112.4.39 set_bkg_submap_attributes_nes16x16() void set_bkg_submap_attributes_nes16x16 (
 uint8_t x,
 uint8_t y,
 uint8_t w,
 uint8_t h,
 const uint8_t * map,
 uint8_t map_w)

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. Useful for scrolling implementations of maps larger than 32 x 30 tiles / 16x15 attributes.

Parameters

<i>x</i>	X Start position in both the Source Attribute Map and hardware Background Map attribute coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Attribute Map and hardware Background Map attribute coordinates. Range 0 - 255
<i>w</i>	Width of area to set in Attributes. Range 1 - 127
<i>h</i>	Height of area to set in Attributes. Range 1 - 127
<i>map</i>	Pointer to source tile map data
<i>map</i> ↔ <i>_w</i>	Width of source tile map in tiles. Range 1 - 127

Entries are copied from **map** to the Background Attribute Map starting at **x**, **y** writing across for **w** tiles and down for **h** attributes, using **map_w** as the rowstride for the source attribute map.

The **x** and **y** parameters are in Source Attribute Map Attribute coordinates. The location tiles will be written to on the hardware Background Map is derived from those, but only uses the lower 5 bits of each axis, for range of 0-15 (they are bit-masked: $x \& 0xF$ and $y \& 0xF$). As a result the two coordinate systems are aligned together.

In order to transfer tile map data in a way where the coordinate systems are not aligned, an offset from the Source Attribute Map pointer can be passed in: $(map_ptr + x + (y * map_width))$.

For example, if you want the tile id at 1, 2 from the source map to show up at 0, 0 on the hardware Background Map (instead of at 1, 2) then modify the pointer address that is passed in: $map_ptr + 1 + (2 * map_width)$

Use this instead of [set_bkg_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source attribute map entry.

Writes that exceed coordinate 15/14 on the x / y axis will wrap around to the Left and Top edges.

See [set_bkg_tiles](#) for setting CGB attribute maps with **VBK_REG**.

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_tiles](#), [set_win_submap](#), [set_tiles](#)

20.112.4.40 **set_bkg_submap_attributes()** `void set_bkg_submap_attributes (`

```
uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * attributes,
uint8_t map_w ) [inline]
```

Sets a rectangular area of the Background Tile Map attributes using a sub-region from a source tile map. Useful for scrolling implementations of maps larger than 32 x 30 tiles.

Please note that this is just a wrapper function for [set_bkg_submap_attributes_nes16x16\(\)](#) and divides the coordinates and dimensions by 2 to achieve this. It is intended to make code more portable by using the same coordinate system that systems with the much more common 8x8 attribute resolution would use.

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_tiles](#), [set_win_submap](#), [set_tiles](#)

20.112.4.41 **set_bkg_based_tiles()** `void set_bkg_based_tiles (`

```
uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * tiles,
uint8_t base_tile ) [inline]
```

Sets a rectangular region of Background Tile Map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_tiles\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_tiles](#) for more details

20.112.4.42 set_bkg_submap() `void set_bkg_submap (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * map,`
`uint8_t map_w ) [inline]`

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. Useful for scrolling implementations of maps larger than 32 x 32 tiles.

@ param **x** X Start position in Background Map tile coordinates. Range 0 - 31 @ param **y** Y Start position in Background Map tile coordinates. Range 0 - 31 @ param **w** Width of area to set in tiles. Range 1 - 255 @ param **h** Height of area to set in tiles. Range 1 - 255 @ param **map** Pointer to source tile map data @ param **map_w** Width of source tile map in tiles. Range 1 - 255

Entries are copied from **map** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles, using **map_w** as the rowstride for the source tile map.

Use this instead of [set_bkg_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

See [set_bkg_tiles](#) for setting CGB attribute maps with [VBK_REG](#).

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_tiles](#), [set_win_submap](#), [set_tiles](#)

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. Useful for scrolling implementations of maps larger than 32 x 32 tiles.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map</i> ↔ <i>_w</i>	Width of source tile map in tiles. Range 1 - 255

Entries are copied from **map** to the Background Tile Map starting at **x**, **y** writing across for **w** tiles and down for **h** tiles, using **map_w** as the rowstride for the source tile map.

The **x** and **y** parameters are in Source Tile Map coordinates. The location tiles will be written to on the hardware Background Map is derived from those, but only uses the lower 5 bits of each axis, for range of 0-31 (they are bit-masked: **x** & 0x1F and **y** & 0x1F). As a result the two coordinate systems are aligned together.

In order to transfer tile map data in a way where the coordinate systems are not aligned, an offset from the Source Tile Map pointer can be passed in: (**map_ptr** + **x** + (**y** * **map_width**)).

For example, if you want the tile id at 1, 2 from the source map to show up at 0, 0 on the hardware Background Map (instead of at 1, 2) then modify the pointer address that is passed in: **map_ptr** + 1 + (2 * **map_width**)

Use this instead of [set_bkg_tiles](#) when the source map is wider than 32 tiles or when writing a width that does not match the source map width.

One byte per source tile map entry.

Writes that exceed coordinate 31 on the x or y axis will wrap around to the Left and Top edges.

See [set_bkg_tiles](#) for setting CGB attribute maps with [VBK_REG](#).

See also

[SHOW_BKG](#)

[set_bkg_data](#), [set_bkg_tiles](#), [set_win_submap](#), [set_tiles](#)

```
20.112.4.43 set_bkg_based_submap() void set_bkg_based_submap (
    uint8_t x,
    uint8_t y,
    uint8_t w,
    uint8_t h,
    const uint8_t * map,
    uint8_t map_w,
    uint8_t base_tile ) [inline]
```

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_submap](#) for more details

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't

start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_submap](#) for more details

20.112.4.44 get_bkg_tiles() `void get_bkg_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`uint8_t * tiles )`

Copies a rectangular region of Background Tile Map entries into a buffer.

Entries are copied into **tiles** from the Background Tile Map starting at **x**, **y** reading across for **w** tiles and down for **h** tiles.

One byte per tile.

The buffer pointed to by **tiles** should be at least **x** x **y** bytes in size.

See also

[get_bkg_tile_xy](#), [get_tiles](#)

Copies a rectangular region of Background Tile Map entries into a buffer.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to copy in tiles. Range 0 - 31
<i>h</i>	Height of area to copy in tiles. Range 0 - 31
<i>tiles</i>	Pointer to destination buffer for Tile Map data

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

Entries are copied into **tiles** from the Background Tile Map starting at **x**, **y** reading across for **w** tiles and down for **h** tiles.

One byte per tile.

The buffer pointed to by **tiles** should be at least **x** x **y** bytes in size.

See also

[get_win_tiles](#), [get_bkg_tile_xy](#), [get_tiles](#), [get_vram_byte](#)

20.112.4.45 set_bkg_tile_xy() `uint8_t * set_bkg_tile_xy (`
`uint8_t x,`
`uint8_t y,`
`uint8_t t )`

Set single tile **t** on background layer at **x**,**y**

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>t</i>	tile index

Returns

returns the address of tile, so you may use faster [set_vram_byte\(\)](#) later

20.112.4.46 [set_bkg_attribute_xy_nes16x16\(\)](#) `void set_bkg_attribute_xy_nes16x16 (`
 [uint8_t](#) *x*,
 [uint8_t](#) *y*,
 [uint8_t](#) *a*)

Set single attribute data *a* on background layer at *x*,*y*

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>a</i>	tile attributes

20.112.4.47 [set_bkg_attribute_xy\(\)](#) `void set_bkg_attribute_xy (`
 [uint8_t](#) *x*,
 [uint8_t](#) *y*,
 [uint8_t](#) *a*) [inline]

Set single attribute data *a* on background layer at *x*,*y*

Please note that this is just a wrapper function for [set_bkg_submap_attributes_nes16x16\(\)](#) and divides the coordinates and dimensions by 2 to achieve this. It is intended to make code more portable by using the same coordinate system that systems with the much more common 8x8 attribute resolution would use.

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>a</i>	tile attributes

20.112.4.48 [get_bkg_tile_xy\(\)](#) `uint8_t get_bkg_tile_xy (`
 [uint8_t](#) *x*,
 [uint8_t](#) *y*)

Get single tile *t* on background layer at *x*,*y*

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate

Returns

returns tile index

Get single tile *t* on background layer at *x*,*y*

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate

Returns

returns tile index

Note

In general **avoid reading from VRAM** since that memory is not accessible at all times. It is also not supported by GBDK on the NES platform. See [coding guidelines](#) for more details.

20.112.4.49 move_bkg() `void move_bkg (`
 `scroll_x_t x,`
 `scroll_y_t y ) [inline]`

Moves the Background Layer to the position specified in **x** and **y** in pixels.

Parameters

<i>x</i>	X axis screen coordinate for Left edge of the Background
<i>y</i>	Y axis screen coordinate for Top edge of the Background

0,0 is the top left corner of the GB screen. The Background Layer wraps around the screen, so when part of it goes off the screen it appears on the opposite side (factoring in the larger size of the Background Layer versus the screen size).

The background layer is always under the Window Layer.

See also

[SHOW_BKG](#), [HIDE_BKG](#)

20.112.4.50 scroll_bkg() `void scroll_bkg (`
 `int8_t x,`
 `int8_t y ) [inline]`

Moves the Background relative to it's current position.

Parameters

<i>x</i>	Number of pixels to move the Background on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the Background on the Y axis Range: -128 - 127

See also

[move_bkg](#)

20.112.4.51 set_sprite_data() `void set_sprite_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data )`

Sets VRAM Tile Pattern data for Sprites

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**. Each Tile is 16 bytes in size (8x8 pixels, 2 bits-per-pixel).

Note: Sprite Tiles 128-255 share the same memory region as Background Tiles 128-255.

GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- VBK_REG=0 indicates the first bank
- VBK_REG=1 indicates the second

Sets VRAM Tile Pattern data for Sprites

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (2 bpp) source Tile Pattern data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**. Each Tile is 16 bytes in size (8x8 pixels, 2 bits-per-pixel).

Note

Sprite Tiles 128-255 share the same memory region as Background Tiles 128-255.

GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- VBK_REG = [VBK_BANK_0](#) indicates the first bank
- VBK_REG = [VBK_BANK_1](#) indicates the second

20.112.4.52 set_sprite_1bpp_data() `void set_sprite_1bpp_data (`
 `uint8_t first_tile,`
 `uint8_t nb_tiles,`
 `const uint8_t * data )`

Sets VRAM Tile Pattern data for Sprites using 1bpp source data

Similar to [set_sprite_data](#), except source data is 1 bit-per-pixel which gets expanded into 2 bits-per-pixel.

For a given bit that represent a pixel:

- 0 will be expanded into color 0
- 1 will be expanded into color 3

See also

[SHOW_SPRITES](#), [HIDE_SPRITES](#), [set_sprite_tile](#)

Sets VRAM Tile Pattern data for Sprites using 1bpp source data

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to (1bpp) source Tile Pattern data

Similar to [set_sprite_data](#), except source data is 1 bit-per-pixel which gets expanded into 2 bits-per-pixel.

For a given bit that represent a pixel:

- 0 will be expanded into the Background color
- 1 will be expanded into the Foreground color

See [set_1bpp_colors](#) for details about setting the Foreground and Background colors.

See also

[SHOW_SPRITES](#), [HIDE_SPRITES](#), [set_sprite_tile](#)
[set_bkg_1bpp_data](#), [set_win_1bpp_data](#)

20.112.4.53 SET_SHADOW_OAM_ADDRESS() `void SET_SHADOW_OAM_ADDRESS ( void * address ) [inline]`

Enable OAM DMA copy each VBlank and set it to transfer any 256-byte aligned array

20.112.4.54 set_sprite_tile() `void set_sprite_tile ( uint8_t nb, uint8_t tile ) [inline]`

Sets sprite number **nb** in the OAM to display tile number **tile**.

Parameters

<i>nb</i>	Sprite number, range 0 - 63
<i>tile</i>	Selects a tile (0 - 255) from PPU memory at 0000h - 0FFFh / 1000h - 1FFFh

In 8x16 mode:

- The sprite will also display the next tile (**tile** + 1) directly below (y + 8) the first tile.
- The lower bit of the tile number is ignored: the upper 8x8 tile is (**tile** & 0xFE), and the lower 8x8 tile is (**tile** | 0x01).
- See: [SPRITES_8x16](#)

Sets sprite number **nb** in the OAM to display tile number **tile**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>tile</i>	Selects a tile (0 - 255) from memory at 8000h - 8FFFh In CGB Mode this could be either in VRAM Bank 0 or 1, depending on Bit 3 of the OAM Attribute Flag (see set_sprite_prop)

In 8x16 mode:

- The sprite will also display the next tile (**tile** + 1) directly below (y + 8) the first tile.
- The lower bit of the tile number is ignored: the upper 8x8 tile is (**tile** & 0xFE), and the lower 8x8 tile is (**tile** | 0x01).
- See: [SPRITES_8x16](#)

20.112.4.55 get_sprite_tile() `uint8_t get_sprite_tile ( uint8_t nb ) [inline]`

Returns the tile number of sprite number **nb** in the OAM.

Parameters

<i>nb</i>	Sprite number, range 0 - 63
-----------	-----------------------------

See also

[set_sprite_tile](#) for more details

Returns the tile number of sprite number **nb** in the OAM.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_tile](#) for more details

20.112.4.56 set_sprite_prop() `void set_sprite_prop (`
 `uint8_t nb,`
 `uint8_t prop ) [inline]`

Sets the OAM Property Flags of sprite number **nb** to those defined in **prop**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>prop</i>	Property setting (see bitfield description)

The bits in **prop** represent:

- Bit 7 - Vertical flip. Dictates which way up the sprite is drawn vertically.
0: normal
1: upside down
- Bit 6 - Horizontal flip. Dictates which way up the sprite is drawn horizontally.
0: normal
1: back to front
- Bit 5 - Priority flag. When this is set, the sprites appear behind the background and window layer.
0: infront
1: behind
- Bit 4 - Unimplemented
- Bit 3 - Unimplemented
- Bit 2 - Unimplemented
- Bit 1 - See bit 0.
- Bit 0 - Bits 0-1 indicate which color palette the sprite should use. Note: only palettes 4 to 7 will be available for NES sprites.

It's recommended to use GBDK constants (eg: `S_FLIPY`) to configure sprite properties as these are crossplatform.

```
// Load palette data into the first palette
set_sprite_palette(4, 1, exampleSprite_palettes)
// Set the OAM value for the sprite
// These flags tell the sprite to use the first sprite palette (palette 4) and to flip the sprite both
// vertically and horizontally.
set_sprite_prop(0, S_FLIPY | S_FLIPX);
```

See also

[S_PALETTE](#), [S_FLIPX](#), [S_FLIPY](#), [S_PRIORITY](#)

Sets the OAM Property Flags of sprite number **nb** to those defined in **prop**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>prop</i>	Property setting (see bitfield description)

The bits in **prop** represent:

- Bit 7 - Priority flag. When this is set the sprites appear behind the background and window layer.
0: infront
1: behind
- Bit 6 - Vertical flip. Dictates which way up the sprite is drawn vertically.
0: normal
1:upside down
- Bit 5 - Horizontal flip. Dictates which way up the sprite is drawn horizontally.
0: normal
1:back to front
- Bit 4 - DMG/Non-CGB Mode Only. Assigns either one of the two b/w palettes to the sprite.
0: OBJ palette 0
1: OBJ palette 1
- Bit 3 - GBC only. Dictates from which bank of Sprite Tile Patterns the tile is taken.
0: Bank 0
1: Bank 1
- Bit 2 - See bit 0.
- Bit 1 - See bit 0.
- Bit 0 - GBC only. Bits 0-2 indicate which of the 7 OBJ colour palettes the sprite is assigned.

It's recommended to use GBDK constants (eg: S_FLIPY) to configure sprite properties as these are crossplatform.

```
// Load palette data into the first palette
set_sprite_palette(4, 1, exampleSprite_palettes)
// Set the OAM value for the sprite
// These flags tell the sprite to flip both vertically and horizontally.
set_sprite_prop(0, S_FLIPY | S_FLIPX);
```

See also

[S_PALETTE](#), [S_FLIPX](#), [S_FLIPY](#), [S_PRIORITY](#)

Function has no affect on sms.

This function is only here to enable game portability

20.112.4.57 `get_sprite_prop()` `uint8_t` get_sprite_prop (
 `uint8_t nb`) [inline]

Returns the OAM Property Flags of sprite number **nb**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_prop](#) for property bitfield settings

20.112.4.58 `move_sprite()` `void` move_sprite (
 `uint8_t nb`,

```
uint8_t x,
uint8_t y ) [inline]
```

Moves sprite number **nb** to the **x, y** position on the screen.

Parameters

<i>nb</i>	Sprite number, range 0 - 63
<i>x</i>	X Position. Specifies the sprites horizontal position on the screen (minus 8).
<i>y</i>	Y Position. Specifies the sprites vertical position on the screen (minus 16). An offscreen value ($Y \geq 240$) hides the sprite.

Moving the sprite to 0,0 (or similar off-screen location) will hide it.

Moves sprite number **nb** to the **x, y** position on the screen.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	X Position. Specifies the sprites horizontal position on the screen (minus 8). An offscreen value ($X=0$ or $X \geq 168$) hides the sprite, but the sprite still affects the priority ordering - a better way to hide a sprite is to set its Y-coordinate offscreen.
<i>y</i>	Y Position. Specifies the sprites vertical position on the screen (minus 16). An offscreen value (for example, $Y=0$ or $Y \geq 160$) hides the sprite.

Moving the sprite to 0,0 (or similar off-screen location) will hide it.

20.112.4.59 scroll_sprite() void scroll_sprite (

```
uint8_t nb,
int8_t x,
int8_t y ) [inline]
```

Moves sprite number **nb** relative to its current position.

Parameters

<i>nb</i>	Sprite number, range 0 - 63
<i>x</i>	Number of pixels to move the sprite on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the sprite on the Y axis Range: -128 - 127

See also

[move_sprite](#) for more details about the X and Y position

Moves sprite number **nb** relative to its current position.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	Number of pixels to move the sprite on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the sprite on the Y axis Range: -128 - 127

See also

[move_sprite](#) for more details about the X and Y position

20.112.4.60 hide_sprite() `void hide_sprite (`
`uint8_t nb ) [inline]`

Hides sprite number **nb** by moving it to Y position 240.

Parameters

<i>nb</i>	Sprite number, range 0 - 63
-----------	-----------------------------

Hides sprite number **nb** by moving it to zero position by Y.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[hide_sprites_range](#), [HIDE_SPRITES](#)

Hides sprite number **nb** by moving it to zero position by Y.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

20.112.4.61 set_data() `void set_data (`
`uint8_t * vram_addr,`
`const uint8_t * data,`
`uint16_t len )`

Copies arbitrary data to an address in VRAM without taking into account the state of LCDC bits 3 or 4.
 Copies **len** bytes from a buffer at **data** to VRAM starting at **vram_addr**.

See also

[set_bkg_data](#), [set_win_data](#), [set_bkg_tiles](#), [set_win_tiles](#), [set_tile_data](#), [set_tiles](#)

20.112.4.62 set_tiles() `void set_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`uint8_t * vram_addr,`
`const uint8_t * tiles )`

Sets a rectangular region of Tile Map entries at a given VRAM Address.

Parameters

<i>x</i>	X Start position in Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32

Parameters

<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>vram_addr</i>	Pointer to destination VRAM Address
<i>tiles</i>	Pointer to source Tile Map data

Entries are copied from **tiles** to Tile Map at address *vram_addr* starting at **x**, **y** writing across for **w** tiles and down for **h** tiles.

One byte per source tile map entry.

There are two 32x30 Tile Maps in VRAM at addresses 2000h-23FFh and 2400h-27FFh.

See also

[set_bkg_tiles](#)

20.112.4.63 set_tile_data() `void set_tile_data (`
`uint16_t first_tile,`
`uint8_t nb_tiles,`
`const uint8_t * data ) [inline]`

Sets VRAM Tile Pattern data starting from given base address without taking into account the state of PPUMASK.

See also

[set_bkg_data](#), [set_data](#)

20.112.4.64 set_bkg_native_data() `void set_bkg_native_data (`
`uint8_t first_tile,`
`uint8_t nb_tiles,`
`const uint8_t * data ) [inline]`

Sets VRAM Tile Pattern data for the Background in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**.

See also

[set_tile_data](#)

Sets VRAM Tile Pattern data for the Background / Window in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**.

GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- [VBK_REG](#) = [VBK_BANK_0](#) indicates the first bank

- VBK_REG = [VBK_BANK_1](#) indicates the second

See also

[set_win_data](#), [set_tile_data](#)

20.112.4.65 set_sprite_native_data() `void set_sprite_native_data (`
 [uint8_t](#) *first_tile*,
 [uint8_t](#) *nb_tiles*,
 const [uint8_t](#) * *data*) [inline]

Sets VRAM Tile Pattern data for Sprites in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**.

Sets VRAM Tile Pattern data for Sprites in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source tile data

Writes **nb_tiles** tiles to VRAM starting at **first_tile**, tile data is sourced from **data**.

GBC only: [VBK_REG](#) determines which bank of tile patterns are written to.

- VBK_REG = [VBK_BANK_0](#) indicates the first bank
- VBK_REG = [VBK_BANK_1](#) indicates the second

20.112.4.66 set_native_tile_data() `void set_native_tile_data (`
 [uint16_t](#) *first_tile*,
 [uint8_t](#) *nb_tiles*,
 const [uint8_t](#) * *data*) [inline]

Sets VRAM Tile Pattern data in the native format

Parameters

<i>first_tile</i>	Index of the first tile to write (0 - 511)
<i>nb_tiles</i>	Number of tiles to write
<i>data</i>	Pointer to source Tile Pattern data.

When *first_tile* is larger than 256 on the GB/AP, it will write to sprite data instead of background data.

The bit depth of the source Tile Pattern data depends on which console is being used:

- NES: loads 2bpp tiles data

20.112.4.67 init_bkg() `void init_bkg (`
 [uint8_t](#) *c*)

Initializes the entire Background Tile Map with Tile Number **c**

Parameters

c	Tile number to fill with
----------	--------------------------

Note: This function avoids writes during modes 2 & 3

Initializes the entire Background Tile Map with Tile Number **c**

Parameters

c	Tile number to fill with
----------	--------------------------

Note

This function avoids writes during modes 2 & 3

20.112.4.68 vmemset() `void vmemset (`
`void * s,`
`uint8_t c,`
`size_t n )`

Fills the VRAM memory region **s** of size **n** with Tile Number **c**

Parameters

s	Start address in VRAM
c	Tile number to fill with
n	Size of memory region (in bytes) to fill

Note: This function avoids writes during modes 2 & 3

Fills the VRAM memory region **s** of size **n** with Tile Number **c**

Parameters

s	Start address in VRAM
c	Tile number to fill with
n	Size of memory region (in bytes) to fill

Note

This function avoids writes during modes 2 & 3

20.112.4.69 fill_bkg_rect() `void fill_bkg_rect (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`uint8_t tile )`

Fills a rectangular region of Tile Map entries for the Background layer with tile.

Parameters

x	X Start position in Background Map tile coordinates. Range 0 - 31
----------	---

Parameters

<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tile</i>	Fill value

20.112.4.70 flush_shadow_attributes() `void flush_shadow_attributes ( void )`

"Flushes" the updates to the shadow attributes so they are written to the transfer buffer, and then written to PPU memory on next vblank.

This function must be called to see visible changes to attributes on the NES target. But it will automatically be called by [vsync\(\)](#), so the use-cases for calling it manually are rare in practice.

20.112.4.71 _switch_prg0() `uint8_t _switch_prg0 ( uint8_t bank )`

20.112.5 Variable Documentation

20.112.5.1 _SYSTEM `const uint8_t _SYSTEM [extern]`

20.112.5.2 sys_time `volatile uint16_t sys_time [extern]`

Global Time Counter in VBL periods (60Hz)

Increments once per Frame

Will wrap around every ~18 minutes (unsigned 16 bits = 65535 / 60 / 60 = 18.2)

20.112.5.3 _current_bank `volatile uint8_t _current_bank [extern]`

Tracks current active ROM bank

The active bank number is not tracked by [_current_bank](#) when [SWITCH_ROM_MBC5_8M](#) is used.

This variable is updated automatically when you call [SWITCH_ROM_MBC1](#) or [SWITCH_ROM_MBC5](#), [SWITCH_ROM\(\)](#), or call a BANKED function.

See also

[SWITCH_ROM_MBC1\(\)](#), [SWITCH_ROM_MBC5\(\)](#), [SWITCH_ROM\(\)](#)

Tracks current active ROM bank

In most cases the [CURRENT_BANK](#) macro for this variable is recommended for use instead of the variable itself.

The active bank number is not tracked by [_current_bank](#) when [SWITCH_ROM_MBC5_8M](#) is used.

This variable is updated automatically when you call [SWITCH_ROM_MBC1](#) or [SWITCH_ROM_MBC5](#), [SWITCH_ROM\(\)](#), or call a BANKED function.

See also

[SWITCH_ROM_MBC1\(\)](#), [SWITCH_ROM_MBC5\(\)](#), [SWITCH_ROM\(\)](#)

20.112.5.4 _current_1bpp_colors `uint16_t _current_1bpp_colors [extern]`

20.112.5.5 shadow_OAM `volatile struct OAM_item_t shadow_OAM[] [extern]`

Shadow OAM array in WRAM, that is DMA-transferred into the real OAM each VBlank

20.112.5.6 _shadow_OAM_base uint8_t _shadow_OAM_base [extern]

MSB of shadow_OAM address is used by OAM DMA copying routine

20.113 nes.h

[Go to the documentation of this file.](#)

```

1
2
3
4 #ifndef _NES_H
5 #define _NES_H
6
7 #include <types.h>
8 #include <stdint.h>
9 #include <gbdk/version.h>
10 #include <nes/hardware.h>
11 #include <nes/rgb_to_nes_macro.h>
12
13 #define NINTENDO_NES
14
15 // Here NINTENDO means Game Boy & related clones
16 #ifdef NINTENDO
17 #undef NINTENDO
18 #endif
19
20 #ifdef SEGA
21 #undef SEGA
22 #endif
23
24 #ifdef MSX
25 #undef MSX
26 #endif
27
28 #define SYSTEM_BITS_NTSC      0x00
29 #define SYSTEM_BITS_PAL      0x40
30 #define SYSTEM_BITS_DENDY    0x80
31 extern const uint8_t _SYSTEM;
32
33 #define SYSTEM_60HZ      0x00
34 #define SYSTEM_50HZ      0x01
35
36 #define TIMER_VBLANK_PARITY_MODE_SYSTEM_60HZ      0x78
37 #define TIMER_VBLANK_PARITY_MODE_SYSTEM_50HZ      0x5D
38
39 #define RGB(r,g,b)      RGB_TO_NES(((r) | ((g) << 2) | ((b) << 4)))
40 #define RGB8(r,g,b)      RGB_TO_NES((((r) >> 6) | (((g) >> 6) << 2) | (((b) >> 6) << 4)))
41 #define RGBHTML(RGB24bit) RGB_TO_NES((((RGB24bit) >> 22) | (((RGB24bit) & 0xFFFF) >> 14) << 2) |
42     (((RGB24bit) & 0xFF) >> 6) << 4)))
43
44 #define RGB_RED      0x16      // EGA12
45 #define RGB_DARKRED   0x06      // EGA4
46 #define RGB_GREEN     0x2A      // EGA10
47 #define RGB_DARKGREEN 0x1A      // EGA2
48 #define RGB_BLUE      0x12      // EGA9
49 #define RGB_DARKBLUE  0x02      // EGA1
50 #define RGB_YELLOW    0x28      // EGA14
51 #define RGB_DARKYELLOW 0x18      // EGA6
52 #define RGB_CYAN      0x2C      // EGA11
53 #define RGB_AQUA      0x1C      // EGA3
54 #define RGB_PINK       0x24      // EGA13
55 #define RGB_PURPLE     0x14      // EGA5
56 #define RGB_BLACK      0x0F      // EGA0
57 #define RGB_DARKGRAY   0x00      // EGA8
58 #define RGB_LIGHTGRAY  0x10      // EGA7
59 #define RGB_WHITE      0x30      // EGA15
60
61 typedef uint8_t palette_color_t;
62
63 void set_bkg_palette(uint8_t first_palette, uint8_t nb_palettes, const palette_color_t *rgb_data)
64     NO_OVERLAY_LOCALS;
65
66 void set_sprite_palette(uint8_t first_palette, uint8_t nb_palettes, const palette_color_t *rgb_data)
67     NO_OVERLAY_LOCALS;
68
69 void set_bkg_palette_entry(uint8_t palette, uint8_t entry, palette_color_t rgb_data) NO_OVERLAY_LOCALS;
70
71 void set_sprite_palette_entry(uint8_t palette, uint8_t entry, palette_color_t rgb_data)
72     NO_OVERLAY_LOCALS;
73
74 #define J_UP      0x08U
75 #define J_DOWN    0x04U
76 #define J_LEFT    0x02U
77 #define J_RIGHT   0x01U
78 #define J_A       0x80U
79 #define J_B       0x40U
80 #define J_SELECT  0x20U
81 #define J_START   0x10U

```

```

96
101 #define M_DRAWING      0x01U
102 #define M_TEXT_OUT     0x02U
103 #define M_TEXT_INOUT   0x03U
109 #define M_NO_SCROLL    0x04U
113 #define M_NO_INTERP    0x08U
114
119 #define S_PALETTE      0x10U
123 #define S_FLIPX       0x40U
127 #define S_FLIPY       0x80U
132 #define S_PRIORITY    0x20U
136 #define S_PAL(n)      n
137
138 /* Interrupt flags */
141 #define EMPTY_IFLAG    0x00U
147 #define VBL_IFLAG     0x01U
151 #define LCD_IFLAG     0x02U
155 #define TIM_IFLAG     0x04U
156
157 /* DMG Palettes */
158 #define DMG_BLACK      0x03
159 #define DMG_DARK_GRAY 0x02
160 #define DMG_LITE_GRAY 0x01
161 #define DMG_WHITE      0x00
181 #define DMG_PALETTE(C0, C1, C2, C3) (((uint8_t)((C3 & 0x03) << 6) | ((C2 & 0x03) << 4) | ((C1 &
    0x03) << 2) | ((C0 & 0x03)))
182
183 /* Limits */
186 #define SCREENWIDTH    DEVICE_SCREEN_PX_WIDTH
189 #define SCREENHEIGHT   DEVICE_SCREEN_PX_HEIGHT
190
193 typedef void (*int_handler)(void) NONBANKED;
194
202 void remove_VBL(int_handler h) NO_OVERLAY_LOCALS;
203
207 void remove_LCD(int_handler h) NO_OVERLAY_LOCALS;
208
212 void remove_TIM(int_handler h) NO_OVERLAY_LOCALS;
213
242 void add_VBL(int_handler h) NO_OVERLAY_LOCALS;
243
276 void add_LCD(int_handler h) NO_OVERLAY_LOCALS;
277
295 void add_TIM(int_handler h) NO_OVERLAY_LOCALS;
296
299 #define MAX_LCD_ISR_CALLS 4
300
307 void mode(uint8_t m) NO_OVERLAY_LOCALS;
308
313 uint8_t get_mode(void) NO_OVERLAY_LOCALS;
314
318 inline uint8_t get_system(void) {
319     if(_SYSTEM == SYSTEM_BITS_NTSC)
320         return SYSTEM_60HZ;
321     else
322         return SYSTEM_50HZ;
323 }
324
331 extern volatile uint16_t sys_time;
332
343 extern volatile uint8_t _current_bank;
344 #define CURRENT_BANK _current_bank
345
355 #ifndef BANK
356 #define BANK(VARNAME) ( (uint8_t) &__bank_ ## VARNAME )
357 #endif
358
371 #define BANKREF(VARNAME) void __func_ ## VARNAME(void) __banked __naked { \
372     __asm \
373     .local b__func_ ## VARNAME \
374     __bank_ ## VARNAME = b__func_ ## VARNAME \
375     .globl __bank_ ## VARNAME \
376     __endasm; \
377 }
378
388 #define BANKREF_EXTERN(VARNAME) extern const void __bank_ ## VARNAME;
389
393 #define SWITCH_ROM_DUMMY(b)
394
398 #define SWITCH_ROM_UNROM(b) _switch_prg0(b)
399
405 #define SWITCH_ROM SWITCH_ROM_UNROM
406
411 #define SWITCH_RAM(b) 0
412
416 #define ENABLE_RAM
417

```

```

421 #define DISABLE_RAM
422
427 void delay(uint16_t d) NO_OVERLAY_LOCALS;
428
438 uint8_t joypad(void) NO_OVERLAY_LOCALS;
439
448 uint8_t waitpad(uint8_t mask) NO_OVERLAY_LOCALS;
449
453 void waitpadup(void) NO_OVERLAY_LOCALS;
454
460 typedef struct {
461     uint8_t npads;
462     union {
463         struct {
464             uint8_t joy0, joy1, joy2, joy3;
465         };
466         uint8_t joypads[4];
467     };
468 } joypads_t;
469
478 uint8_t joypad_init(uint8_t npads, joypads_t * joypads) NO_OVERLAY_LOCALS;
479
484 void joypad_ex(joypads_t * joypads) NO_OVERLAY_LOCALS;
485
486
487
496 inline void enable_interrupts(void) {
497     __asm__("cli");
498 }
499
512 inline void disable_interrupts(void) {
513     __asm__("sei");
514 }
515
521 void set_interrupts(uint8_t flags) NO_OVERLAY_LOCALS;
522
534 inline void reset(void) {
535     __asm__("jmp [0xFFFC]");
536 }
537
547 void vsync(void) NO_OVERLAY_LOCALS;
548
551 void wait_vbl_done(void) NO_OVERLAY_LOCALS;
552
557 void display_on(void) NO_OVERLAY_LOCALS;
558
562 void display_off(void) NO_OVERLAY_LOCALS;
563
566 void refresh_OAM(void) NO_OVERLAY_LOCALS;
567
571 #define DISPLAY_ON \
572     display_on();
573
577 #define DISPLAY_OFF \
578     display_off();
579
583 #define HIDE_LEFT_COLUMN \
584     shadow_PPUMASK &= ~(PPUMASK_SHOW_BG_LC | PPUMASK_SHOW_SPR_LC); \
585
589 #define SHOW_LEFT_COLUMN \
590     shadow_PPUMASK |= (PPUMASK_SHOW_BG_LC | PPUMASK_SHOW_SPR_LC);
591
595 #define SET_BORDER_COLOR(C)
596
600 #define SHOW_BKG \
601     shadow_PPUMASK |= PPUMASK_SHOW_BG;
602
606 #define HIDE_BKG \
607     shadow_PPUMASK &= ~PPUMASK_SHOW_BG;
608
612 #define SHOW_SPRITES \
613     shadow_PPUMASK |= PPUMASK_SHOW_SPR;
614
618 #define HIDE_SPRITES \
619     shadow_PPUMASK &= ~PPUMASK_SHOW_SPR;
620
624 #define SPRITES_8x16 \
625     shadow_PPUCTRL |= PPUCTRL_SPR_8X16;
626
630 #define SPRITES_8x8 \
631     shadow_PPUCTRL &= ~PPUCTRL_SPR_8X16;
632
633
634
641 void set_vram_byte(uint8_t * addr, uint8_t v) NO_OVERLAY_LOCALS;
642
646 uint8_t * get_bkg_xy_addr(uint8_t x, uint8_t y) NO_OVERLAY_LOCALS;

```



```

647
648 #define COMPAT_PALETTE(C0,C1,C2,C3) ((uint8_t)((C3) << 6) | ((C2) << 4) | ((C1) << 2) | (C0))
649
652 inline void set_2bpp_palette(uint16_t palette) {
653     palette;
654 }
655
656 extern uint16_t _current_lbpp_colors;
657 void set_lbpp_colors_ex(uint8_t fgcolor, uint8_t bgcolor, uint8_t mode) NO_OVERLAY_LOCALS;
658 inline void set_lbpp_colors(uint8_t fgcolor, uint8_t bgcolor) {
659     set_lbpp_colors_ex(fgcolor, bgcolor, 0);
660 }
661
662 void set_bkg_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS;
663 #define set_bkg_2bpp_data set_bkg_data
664
665 void set_bkg_lbpp_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS;
666
667 void set_bkg_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) NO_OVERLAY_LOCALS;
668 #define set_tile_map set_bkg_tiles
669
670 void set_bkg_attributes_nes16x16(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *attributes)
671     NO_OVERLAY_LOCALS;
672
673 inline void set_bkg_attributes(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *attributes)
674 {
675     set_bkg_attributes_nes16x16(x >> 1, y >> 1, (w + 1) >> 1, (h + 1) >> 1, attributes);
676 }
677
678 void set_bkg_submap_attributes_nes16x16(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map,
679     uint8_t map_w) NO_OVERLAY_LOCALS;
680
681 inline void set_bkg_submap_attributes(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t
682     *attributes, uint8_t map_w)
683 {
684     set_bkg_submap_attributes_nes16x16(x >> 1, y >> 1, (w + 1) >> 1, (h + 1) >> 1, attributes, (map_w + 1) >>
685     1);
686 }
687
688
689
690 inline void set_bkg_based_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles,
691     uint8_t base_tile);
692
693
694
695 void set_bkg_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)
696     NO_OVERLAY_LOCALS;
697 #define set_tile_submap set_bkg_submap
698
699
700 inline void set_bkg_based_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
701     map_w, uint8_t base_tile);
702
703
704
705 void get_bkg_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *tiles) NO_OVERLAY_LOCALS;
706
707
708
709 uint8_t * set_bkg_tile_xy(uint8_t x, uint8_t y, uint8_t t) NO_OVERLAY_LOCALS;
710 #define set_tile_xy set_bkg_tile_xy
711
712
713 void set_bkg_attribute_xy_nes16x16(uint8_t x, uint8_t y, uint8_t a) NO_OVERLAY_LOCALS;
714
715 inline void set_bkg_attribute_xy(uint8_t x, uint8_t y, uint8_t a)
716 {
717     set_bkg_attribute_xy_nes16x16(x >> 1, y >> 1, a);
718 }
719 #define set_attribute_xy set_bkg_attribute_xy
720
721
722 uint8_t get_bkg_tile_xy(uint8_t x, uint8_t y) NO_OVERLAY_LOCALS;
723
724
725
726 inline void move_bkg(scroll_x_t x, scroll_y_t y) {
727     // store low 8 bits to shadow scroll registers
728     bkg_scroll_x = (uint8_t)x;
729     bkg_scroll_y = (uint8_t)(y >= 240 ? (y - 240) : y);
730     // store 9th bit of x and y in shadow PPUCTRL register
731     #if DEVICE_SCREEN_BUFFER_WIDTH > 32 && DEVICE_SCREEN_BUFFER_HEIGHT > 30
732         uint8_t msb_x = (uint8_t)((x >> 8) & 1);
733         uint8_t msb_y = (uint8_t)(y >= 240 ? 1 : 0);
734         shadow_PPUCTRL = (shadow_PPUCTRL & 0xFC) | (msb_y << 1) | msb_x;
735     #elif DEVICE_SCREEN_BUFFER_WIDTH > 32
736         uint8_t msb_x = (uint8_t)((x >> 8) & 1);
737         shadow_PPUCTRL = (shadow_PPUCTRL & 0xFC) | msb_x;
738     #elif DEVICE_SCREEN_BUFFER_HEIGHT > 30
739         uint8_t msb_y = (uint8_t)(y >= 240 ? 1 : 0);
740         shadow_PPUCTRL = (shadow_PPUCTRL & 0xFC) | (msb_y << 1);
741     #endif
742 }
743
744
745

```

```

991
1001 inline void scroll_bkg(int8_t x, int8_t y) {
1002     move_bkg(bkg_scroll_x + x, bkg_scroll_y + y);
1003 }
1004
1005
1017 void set_sprite_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS;
1018 #define set_sprite_2bpp_data set_sprite_data
1019
1031 void set_sprite_1bpp_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS;
1032
1039 typedef struct OAM_item_t {
1040     uint8_t y;        ///< Y coordinate of the sprite on screen - 1
1041     uint8_t tile;     ///< Sprite tile number
1042     uint8_t prop;     ///< OAM Property Flags
1043     uint8_t x;        ///< X coordinate of the sprite on screen
1044 } OAM_item_t;
1045
1046
1049 extern volatile struct OAM_item_t shadow_OAM[];
1050
1053 extern uint8_t _shadow_OAM_base;
1054
1055 #define DISABLE_OAM_DMA \
1056     _shadow_OAM_base = 0
1057
1060 #define DISABLE_VBL_TRANSFER DISABLE_OAM_DMA
1061
1062 #define ENABLE_OAM_DMA \
1063     _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM » 8)
1064
1067 #define ENABLE_VBL_TRANSFER ENABLE_OAM_DMA
1068
1071 #define MAX_HARDWARE_SPRITES 64
1072
1075 #define HARDWARE_SPRITE_CAN_FLIP_X 1
1076
1079 #define HARDWARE_SPRITE_CAN_FLIP_Y 1
1080
1083 inline void SET_SHADOW_OAM_ADDRESS(void * address) {
1084     _shadow_OAM_base = (uint8_t)((uint16_t)address » 8);
1085 }
1086
1100 void set_sprite_tile(uint8_t nb, uint8_t tile) NO_OVERLAY_LOCALS;
1101
1102
1109 uint8_t get_sprite_tile(uint8_t nb) NO_OVERLAY_LOCALS;
1110
1111
1149 void set_sprite_prop(uint8_t nb, uint8_t prop) NO_OVERLAY_LOCALS;
1150
1151
1157 uint8_t get_sprite_prop(uint8_t nb) NO_OVERLAY_LOCALS;
1158
1159
1169 void move_sprite(uint8_t nb, uint8_t x, uint8_t y) NO_OVERLAY_LOCALS;
1170
1171
1182 void scroll_sprite(uint8_t nb, int8_t x, int8_t y) NO_OVERLAY_LOCALS;
1183
1184
1189 void hide_sprite(uint8_t nb) NO_OVERLAY_LOCALS;
1190
1191
1192
1200 void set_data(uint8_t *vram_addr, const uint8_t *data, uint16_t len) NO_OVERLAY_LOCALS;
1201
1202
1221 void set_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t *vram_addr, const uint8_t *tiles)
    NO_OVERLAY_LOCALS;
1222
1228 inline void set_tile_data(uint16_t first_tile, uint8_t nb_tiles, const uint8_t *data) {
1229     if (first_tile < 256) {
1230         set_bkg_data(first_tile, nb_tiles, data);
1231         if(first_tile + nb_tiles > 256)
1232             set_sprite_data(first_tile - 256, nb_tiles, data);
1233     } else {
1234         set_sprite_data(first_tile - 256, nb_tiles, data);
1235     }
1236 }
1237
1249 void set_bkg_native_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data) NO_OVERLAY_LOCALS;
1250
1260 void set_sprite_native_data(uint8_t first_tile, uint8_t nb_tiles, const uint8_t *data)
    NO_OVERLAY_LOCALS;
1261
1275 inline void set_native_tile_data(uint16_t first_tile, uint8_t nb_tiles, const uint8_t *data) {

```

```

1276     if (first_tile < 256) {
1277         set_bkg_native_data(first_tile, nb_tiles, data);
1278         if(first_tile + nb_tiles > 256)
1279             set_sprite_native_data(first_tile - 256, nb_tiles, data);
1280     } else {
1281         set_sprite_native_data(first_tile - 256, nb_tiles, data);
1282     }
1283 }
1284
1290 void init_bkg(uint8_t c) NO_OVERLAY_LOCALS;
1291
1299 void vmemset (void *s, uint8_t c, size_t n) NO_OVERLAY_LOCALS;
1300
1309 void fill_bkg_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t tile) NO_OVERLAY_LOCALS;
1310 #define fill_rect fill_bkg_rect
1311
1319 void flush_shadow_attributes(void) NO_OVERLAY_LOCALS;
1320
1321 uint8_t _switch_prg0(uint8_t bank) NO_OVERLAY_LOCALS;
1322
1323 #endif /* _NES_H */

```

20.114 gbdk-lib/include/nes/rgb_to_nes_macro.h File Reference

Macros

- #define [RGB_TO_NES\(c\)](#)

20.114.1 Macro Definition Documentation

20.114.1.1 RGB_TO_NES #define RGB_TO_NES(
 c)

20.115 rgb_to_nes_macro.h

[Go to the documentation of this file.](#)

```

1 // File auto-generated file by nespal.py
2 #ifndef __RGB_TO_NES_MACRO_H__
3 #define __RGB_TO_NES_MACRO_H__
4 #define RGB_TO_NES(c) \
5     (c == 0x00) ? 0x1D : \
6     (c == 0x01) ? 0x06 : \
7     (c == 0x02) ? 0x17 : \
8     (c == 0x03) ? 0x16 : \
9     (c == 0x04) ? 0x19 : \
10    (c == 0x05) ? 0x18 : \
11    (c == 0x06) ? 0x17 : \
12    (c == 0x07) ? 0x27 : \
13    (c == 0x08) ? 0x2A : \
14    (c == 0x09) ? 0x29 : \
15    (c == 0x0A) ? 0x28 : \
16    (c == 0x0B) ? 0x27 : \
17    (c == 0x0C) ? 0x2A : \
18    (c == 0x0D) ? 0x29 : \
19    (c == 0x0E) ? 0x29 : \
20    (c == 0x0F) ? 0x28 : \
21    (c == 0x10) ? 0x01 : \
22    (c == 0x11) ? 0x04 : \
23    (c == 0x12) ? 0x15 : \
24    (c == 0x13) ? 0x15 : \
25    (c == 0x14) ? 0x1C : \
26    (c == 0x15) ? 0x00 : \
27    (c == 0x16) ? 0x15 : \
28    (c == 0x17) ? 0x26 : \
29    (c == 0x18) ? 0x2B : \
30    (c == 0x19) ? 0x2A : \
31    (c == 0x1A) ? 0x10 : \
32    (c == 0x1B) ? 0x26 : \
33    (c == 0x1C) ? 0x2B : \
34    (c == 0x1D) ? 0x2A : \
35    (c == 0x1E) ? 0x39 : \
36    (c == 0x1F) ? 0x38 : \
37    (c == 0x20) ? 0x02 : \
38    (c == 0x21) ? 0x13 : \
39    (c == 0x22) ? 0x14 : \
40    (c == 0x23) ? 0x14 : \
41    (c == 0x24) ? 0x11 : \

```

```
42      (c == 0x25) ? 0x13 : \  
43      (c == 0x26) ? 0x10 : \  
44      (c == 0x27) ? 0x25 : \  
45      (c == 0x28) ? 0x2C : \  
46      (c == 0x29) ? 0x10 : \  
47      (c == 0x2A) ? 0x3D : \  
48      (c == 0x2B) ? 0x36 : \  
49      (c == 0x2C) ? 0x2C : \  
50      (c == 0x2D) ? 0x3B : \  
51      (c == 0x2E) ? 0x3A : \  
52      (c == 0x2F) ? 0x37 : \  
53      (c == 0x30) ? 0x12 : \  
54      (c == 0x31) ? 0x13 : \  
55      (c == 0x32) ? 0x14 : \  
56      (c == 0x33) ? 0x24 : \  
57      (c == 0x34) ? 0x12 : \  
58      (c == 0x35) ? 0x22 : \  
59      (c == 0x36) ? 0x23 : \  
60      (c == 0x37) ? 0x24 : \  
61      (c == 0x38) ? 0x21 : \  
62      (c == 0x39) ? 0x22 : \  
63      (c == 0x3A) ? 0x32 : \  
64      (c == 0x3B) ? 0x34 : \  
65      (c == 0x3C) ? 0x2C : \  
66      (c == 0x3D) ? 0x3C : \  
67      (c == 0x3E) ? 0x3C : \  
68      (c == 0x3F) ? 0x20 : \  
69      0xFF // out-of-range value - set to 0xFF  
70 #endif
```

20.116 gbdk-lib/include/rand.h File Reference

```
#include <types.h>  
#include <stdint.h>
```

Macros

- `#define RAND_MAX 255`
- `#define RANDW_MAX 65535`

Functions

- `void initrand (uint16_t seed) OLDCALL`
- `uint8_t rand (void) OLDCALL PRESERVES_REGS(b`
- `uint16_t randw (void) OLDCALL PRESERVES_REGS(b`
- `void initarand (uint16_t seed) OLDCALL`
- `uint8_t arand (void) OLDCALL PRESERVES_REGS(b`

Variables

- `uint16_t __rand_seed`
- `uint8_t c`

20.116.1 Detailed Description

Random generator using the linear congruential method

Author

Luc Van den Borre

20.116.2 Macro Definition Documentation

20.116.2.1 RAND_MAX `#define RAND_MAX 255`

20.116.2.2 RANDW_MAX `#define RANDW_MAX 65535`**20.116.3 Function Documentation****20.116.3.1 initrand()** `void initrand (`
`uint16_t seed )`

Initialise the pseudo-random number generator.

Parameters

<code>seed</code>	The value for initializing the random number generator.
-------------------	---

The seed should be different each time, otherwise the same pseudo-random sequence will be generated. One way to do this is sampling ([DIV_REG](#)) up to 2 times (high byte of seed value then the low byte) at variable, non-deterministic points in time (such as when the player presses buttons on the title screen or in a menu). It only needs to be called once to be initialized.

See also

[rand\(\)](#), [randw\(\)](#)

20.116.3.2 rand() `uint8_t rand (`
`void )`

Returns a random byte (8 bit) value.

[initrand\(\)](#) should be used to initialize the random number generator before using [rand\(\)](#)

20.116.3.3 randw() `uint16_t randw (`
`void )`

Returns a random word (16 bit) value.

[initrand\(\)](#) should be used to initialize the random number generator before using [rand\(\)](#)

20.116.3.4 initarand() `void initarand (`
`uint16_t seed )`

Random generator using the linear lagged additive method

Parameters

<code>seed</code>	The value for initializing the random number generator.
-------------------	---

Note: [initarand\(\)](#) calls [initrand\(\)](#) with the same seed value, and uses [rand\(\)](#) to initialize the random generator.

See also

[initrand\(\)](#) for suggestions about seed values, [arand\(\)](#)

20.116.3.5 arand() `uint8_t arand (`
`void )`

Returns a random number generated with the linear lagged additive method.

[initarand\(\)](#) should be used to initialize the random number generator before using [arand\(\)](#)

20.116.4 Variable Documentation

20.116.4.1 `__rand_seed` `uint16_t` `__rand_seed` [extern]

The random number seed is stored in `__rand_seed` and can be saved and restored if needed.

```
// Save
some_uint16 = __rand_seed;
...
// Restore
__rand_seed = some_uint16;
```

20.116.4.2 `c` `uint8_t` `c`**20.117** `rand.h`

[Go to the documentation of this file.](#)

```
1
6 #ifndef RAND_INCLUDE
7 #define RAND_INCLUDE
8
9 #include <types.h>
10 #include <stdint.h>
11
27 #if defined(__PORT_sm83) || defined(__PORT_mos6502)
28 void initrand(uint16_t seed) OLDCALL;
29 #elif defined(__PORT_z80)
30 void initrand(uint16_t seed) Z88DK_FASTCALL;
31 #endif
32
33 #define RAND_MAX 255
34 #define RANDW_MAX 65535
35
47 extern uint16_t __rand_seed;
48
53 #if defined(__PORT_sm83)
54 uint8_t rand(void) OLDCALL PRESERVES_REGS(b, c);
55 #elif defined(__PORT_z80)
56 uint8_t rand(void) OLDCALL PRESERVES_REGS(b, c, iyh, iyl);
57 #elif defined(__PORT_mos6502)
58 uint8_t rand(void);
59 #endif
60
65 #if defined(__PORT_sm83)
66 uint16_t randw(void) OLDCALL PRESERVES_REGS(b, c);
67 #elif defined(__PORT_z80)
68 uint16_t randw(void) OLDCALL PRESERVES_REGS(b, c, iyh, iyl);
69 #elif defined(__PORT_mos6502)
70 uint16_t randw(void);
71 #endif
72
82 #if defined(__PORT_sm83) || defined(__PORT_mos6502)
83 void initarand(uint16_t seed) OLDCALL;
84 #elif defined(__PORT_z80)
85 void initarand(uint16_t seed) Z88DK_FASTCALL;
86 #endif
87
92 #if defined(__PORT_sm83)
93 uint8_t arand(void) OLDCALL PRESERVES_REGS(b, c);
94 #elif defined(__PORT_z80)
95 uint8_t arand(void) OLDCALL PRESERVES_REGS(b, c, iyh, iyl);
96 #elif defined(__PORT_mos6502)
97 uint8_t arand(void);
98 #endif
99
100 #endif
```

20.118 `gbdk-lib/include/setjmp.h` File Reference**Macros**

- `#define SP_SIZE 1`
- `#define BP_SIZE 0`
- `#define SPX_SIZE 0`
- `#define BPX_SIZE SPX_SIZE`
- `#define RET_SIZE 2`
- `#define setjmp(jump_buf) __setjmp(jump_buf)`

Typedefs

- typedef unsigned char `jmp_buf`[`RET_SIZE+SP_SIZE+BP_SIZE+SPX_SIZE+BPX_SIZE`]

Functions

- int `__setjmp` (`jmp_buf`) `OLDCALL`
- `_Noreturn` void `longjmp` (`jmp_buf`, int) `OLDCALL`

20.118.1 Macro Definition Documentation

20.118.1.1 SP_SIZE `#define SP_SIZE 1`

20.118.1.2 BP_SIZE `#define BP_SIZE 0`

20.118.1.3 SPX_SIZE `#define SPX_SIZE 0`

20.118.1.4 BPX_SIZE `#define BPX_SIZE SPX_SIZE`

20.118.1.5 RET_SIZE `#define RET_SIZE 2`

20.118.1.6 setjmp `#define setjmp(
 jump_buf) __setjmp(jump_buf)`

20.118.2 Typedef Documentation

20.118.2.1 jmp_buf `typedef unsigned char jmp_buf[RET_SIZE+SP_SIZE+BP_SIZE+SPX_SIZE+BPX_SIZE]`

20.118.3 Function Documentation

20.118.3.1 __setjmp() `int __setjmp (
 jmp_buf)`

20.118.3.2 longjmp() `_Noreturn void longjmp (
 jmp_buf ,
 int)`

20.119 setjmp.h

[Go to the documentation of this file.](#)

```
1 /*-----
2  setjmp.h - header file for setjmp & longjmp ANSI routines
3
4  Copyright (C) 1999, Sandeep Dutta . sandeep.dutta@usa.net
5
6  This library is free software; you can redistribute it and/or modify it
7  under the terms of the GNU General Public License as published by the
```

```

8   Free Software Foundation; either version 2, or (at your option) any
9   later version.
10
11   This library is distributed in the hope that it will be useful,
12   but WITHOUT ANY WARRANTY; without even the implied warranty of
13   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14   GNU General Public License for more details.
15
16   You should have received a copy of the GNU General Public License
17   along with this library; see the file COPYING. If not, write to the
18   Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,
19   MA 02110-1301, USA.
20
21   As a special exception, if you link this library with other files,
22   some of which are compiled with SDCC, to produce an executable,
23   this library does not by itself cause the resulting executable to
24   be covered by the GNU General Public License. This exception does
25   not however invalidate any other reasons why the executable file
26   might be covered by the GNU General Public License.
27 -----*/
28
29 #ifndef __SDCC_SETJMP_H
30 #define __SDCC_SETJMP_H
31
32 #define SP_SIZE      1
33
34 #ifdef __SDCC_STACK_AUTO
35 #define BP_SIZE      SP_SIZE
36 #else
37 #define BP_SIZE      0
38 #endif
39
40 #ifdef __SDCC_USE_XSTACK
41 #define SPX_SIZE      1
42 #else
43 #define SPX_SIZE      0
44 #endif
45
46 #define BPX_SIZE      SPX_SIZE
47
48 #ifdef __SDCC_MODEL_HUGE
49 #define RET_SIZE      3
50 #else
51 #define RET_SIZE      2
52 #endif
53
54 #if defined (__SDCC_z80) || defined (__SDCC_z180) || defined (__SDCC_r2k) || defined (__SDCC_r3ka) ||
    defined (__SDCC_tlcs90) || defined (__SDCC_ez80_z80) || defined (__SDCC_z80n)
55 typedef unsigned char jmp_buf[6]; /* 2 for the stack pointer, 2 for the return address, 2 for the frame
    pointer. */
56 #elif defined (__SDCC_ds390) || defined (__SDCC_stm8) && defined (__SDCC_MODEL_LARGE)
57 typedef unsigned char jmp_buf[5]; /* 2 for the stack pointer, 3 for the return address. */
58 #elif defined (__SDCC_stm8) || defined (__SDCC_sm83) || defined (__SDCC_hc08) || defined (__SDCC_s08)
59 typedef unsigned char jmp_buf[4]; /* 2 for the stack pointer, 2 for the return address. */
60 #elif defined (__SDCC_pdk13) || defined (__SDCC_pdk14) || defined (__SDCC_pdk15)
61 typedef unsigned char jmp_buf[3]; /* 1 for the stack pointer, 2 for the return address. */
62 #else
63 typedef unsigned char jmp_buf[RET_SIZE + SP_SIZE + BP_SIZE + SPX_SIZE + BPX_SIZE];
64 #endif
65
66 int __setjmp (jmp_buf) OLDCALL;
67
68 /* C99 might require setjmp to be a macro. The standard seems self-contradicting on this issue. */
69 /* However, it is clear that the standards allow setjmp to be a macro. */
70 #define setjmp(jump_buf) __setjmp(jump_buf)
71
72 #ifndef __SDCC_HIDE_LONGJMP
73 _Noreturn void longjmp(jmp_buf, int) OLDCALL;
74 #endif
75
76 #undef RET_SIZE
77 #undef SP_SIZE
78 #undef BP_SIZE
79 #undef SPX_SIZE
80 #undef BPX_SIZE
81
82 #endif
83

```

20.120 gbdk-lib/include/sms/sms.h File Reference

```

#include <types.h>
#include <stdint.h>
#include <gbdk/version.h>

```



```
#include <sms/hardware.h>
```

Data Structures

- struct [joypads_t](#)

Macros

- #define [SEGA](#)
- #define [SYSTEM_60HZ](#) 0x00
- #define [SYSTEM_50HZ](#) 0x01
- #define [VBK_REG_VDP_ATTR_SHIFT](#)
- #define [J_UP](#) 0b00000001
- #define [J_DOWN](#) 0b00000010
- #define [J_LEFT](#) 0b00000100
- #define [J_RIGHT](#) 0b00001000
- #define [J_B](#) 0b00010000
- #define [J_A](#) 0b00100000
- #define [J_START](#) 0b01000000
- #define [J_SELECT](#) 0b10000000
- #define [M_TEXT_OUT](#) 0x02U
- #define [M_TEXT_INOUT](#) 0x03U
- #define [M_NO_SCROLL](#) 0x04U
- #define [M_NO_INTERP](#) 0x08U
- #define [S_BANK](#) 0x01U
- #define [S_FLIPX](#) 0x02U
- #define [S_FLIPY](#) 0x04U
- #define [S_PALETTE](#) 0x08U
- #define [S_PRIORITY](#) 0x10U
- #define [S_PAL\(n\)](#) (((n) & 0x01U) << 3)
- #define [__WRITE_VDP_REG_UNSAFE](#)(REG, v) shadow_##REG=(v),VDP_CMD=(shadow_##REG),VDP←_CMD=REG
- #define [__WRITE_VDP_REG](#)(REG, v) shadow_##REG=(v);__asm__("di");VDP_CMD=(shadow_←_##REG);VDP_CMD=REG;__asm__("ei")
- #define [__READ_VDP_REG](#)(REG) shadow_##REG
- #define [EMPTY_IFLAG](#) 0x00U
- #define [VBL_IFLAG](#) 0x01U
- #define [LCD_IFLAG](#) 0x02U
- #define [TIM_IFLAG](#) 0x04U
- #define [SIO_IFLAG](#) 0x08U
- #define [JOY_IFLAG](#) 0x10U
- #define [SCREENWIDTH](#) [DEVICE_SCREEN_PX_WIDTH](#)
- #define [SCREENHEIGHT](#) [DEVICE_SCREEN_PX_HEIGHT](#)
- #define [MINWNDPOSX](#) 0x00U
- #define [MINWNDPOSY](#) 0x00U
- #define [MAXWNDPOSX](#) 0x00U
- #define [MAXWNDPOSY](#) 0x00U
- #define [DISPLAY_ON](#) [__WRITE_VDP_REG](#)(VDP_R1, [__READ_VDP_REG](#)(VDP_R1) |= R1_DISP_ON)
- #define [DISPLAY_OFF](#) [display_off](#)();
- #define [HIDE_LEFT_COLUMN](#) [__WRITE_VDP_REG](#)(VDP_R0, [__READ_VDP_REG](#)(VDP_R0) |= R0_LCB)
- #define [SHOW_LEFT_COLUMN](#) [__WRITE_VDP_REG](#)(VDP_R0, [__READ_VDP_REG](#)(VDP_R0) &= (~R0_LCB))
- #define [SET_BORDER_COLOR](#)(C) [__WRITE_VDP_REG](#)(VDP_R7, ((C) | 0xf0u))
- #define [SHOW_BKG](#)
- #define [HIDE_BKG](#)

- `#define SHOW_WIN`
- `#define HIDE_WIN`
- `#define SHOW_SPRITES ( _sprites_OFF = 0)`
- `#define HIDE_SPRITES ( _sprites_OFF = 1)`
- `#define SPRITES_8x16 __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) |= R1_SPR_8X16)`
- `#define SPRITES_8x8 __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) &= (~R1_SPR_8X16))`
- `#define DEVICE_SUPPORTS_COLOR (TRUE)`
- `#define VBL_DONE _vbl_done`
- `#define DIV_REG get_r_reg()`
- `#define _current_bank MAP_FRAME1`
- `#define CURRENT_BANK MAP_FRAME1`
- `#define BANK(VARNAME) ( (uint8_t) & __bank_ ## VARNAME )`
- `#define BANKREF(VARNAME)`
- `#define BANKREF_EXTERN(VARNAME) extern const void __bank_ ## VARNAME;`
- `#define SWITCH_ROM(b) MAP_FRAME1=(b)`
- `#define SWITCH_ROM1 SWITCH_ROM`
- `#define SWITCH_ROM2(b) MAP_FRAME2=(b)`
- `#define SWITCH_RAM(b) RAM_CONTROL=((b)&1)?RAM_CONTROL|RAMCTL_BANK:RAM_CONTROL&(~RAMCTL_BANK)`
- `#define ENABLE_RAM RAM_CONTROL|=RAMCTL_RAM`
- `#define DISABLE_RAM RAM_CONTROL&=(~RAMCTL_RAM)`
- `#define set_bkg_palette_entry set_palette_entry`
- `#define set_sprite_palette_entry(palette, entry, rgb_data) set_palette_entry(1,entry,rgb_data)`
- `#define set_bkg_palette set_palette`
- `#define set_sprite_palette(first_palette, nb_palettes, rgb_data) set_palette(1,1,rgb_data)`
- `#define COMPAT_PALETTE(C0, C1, C2, C3) (((uint16_t)(C3) << 12) | ((uint16_t)(C2) << 8) | ((uint16_t)(C1) << 4) | (uint16_t)(C0))`
- `#define set_bkg_tiles set_tile_map_compat`
- `#define set_win_tiles set_tile_map_compat`
- `#define set_bkg_submap set_tile_submap_compat`
- `#define set_win_submap set_tile_submap_compat`
- `#define fill_bkg_rect fill_rect_compat`
- `#define fill_win_rect fill_rect_compat`
- `#define DISABLE_VBL_TRANSFER _shadow_OAM_base = 0`
- `#define ENABLE_VBL_TRANSFER _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)`
- `#define MAX_HARDWARE_SPRITES 64`
- `#define HARDWARE_SPRITE_CAN_FLIP_X 0`
- `#define HARDWARE_SPRITE_CAN_FLIP_Y 0`
- `#define set_bkg_tile_xy set_tile_xy`
- `#define set_win_tile_xy set_tile_xy`
- `#define set_bkg_attribute_xy set_attribute_xy`
- `#define set_win_attribute_xy set_attribute_xy`
- `#define get_win_xy_addr get_bkg_xy_addr`

Typedefs

- `typedef void(* int_handler) (void) NONBANKED`

Functions

- void [WRITE_VDP_CMD](#) (uint16_t cmd) Z88DK_FASTCALL PRESERVES_REGS(b)
- void [WRITE_VDP_DATA](#) (uint16_t data) Z88DK_FASTCALL PRESERVES_REGS(b)
- void [mode](#) (uint8_t m) OLDCALL
- [uint8_t get_mode](#) (void) OLDCALL
- [uint8_t get_system](#) (void)
- void [set_interrupts](#) (uint8_t flags) Z88DK_FASTCALL
- void [remove_VBL](#) (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(iyh)
- void [remove_LCD](#) (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b)
- void [remove_TIM](#) (int_handler h) Z88DK_FASTCALL
- void [remove_SIO](#) (int_handler h) Z88DK_FASTCALL
- void [remove_JOY](#) (int_handler h) Z88DK_FASTCALL
- void [add_VBL](#) (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(d)
- void [add_LCD](#) (int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b)
- void [add_TIM](#) (int_handler h) Z88DK_FASTCALL
- void [add_SIO](#) (int_handler h) Z88DK_FASTCALL
- void [add_JOY](#) (int_handler h) Z88DK_FASTCALL
- [uint8_t cancel_pending_interrupts](#) (void)
- void [move_bkg](#) (uint8_t x, uint8_t y)
- void [scroll_bkg](#) (int8_t x, int8_t y)
- void [vsync](#) (void) PRESERVES_REGS(b)
- void [wait_vbl_done](#) (void) PRESERVES_REGS(b)
- void [display_off](#) (void)
- void [refresh_OAM](#) (void)
- [uint8_t get_r_reg](#) (void) PRESERVES_REGS(b)
- void [delay](#) (uint16_t d) Z88DK_FASTCALL
- [uint8_t joypad](#) (void) OLDCALL PRESERVES_REGS(b)
- [uint8_t waitpad](#) (uint8_t mask) Z88DK_FASTCALL PRESERVES_REGS(d)
- void [waitpadup](#) (void) PRESERVES_REGS(d)
- [uint8_t joypad_init](#) (uint8_t npads, joypads_t *joypads) Z88DK_CALLEE
- void [joypad_ex](#) (joypads_t *joypads) Z88DK_FASTCALL PRESERVES_REGS(iyh)
- void [enable_interrupts](#) (void) PRESERVES_REGS(a)
- void [disable_interrupts](#) (void) PRESERVES_REGS(a)
- void [set_default_palette](#) (void)
- void [cgb_compatibility](#) (void)
- void [cpu_fast](#) (void)
- void [set_palette_entry](#) (uint8_t palette, uint8_t entry, uint16_t rgb_data) Z88DK_CALLEE PRESERVES_REGS(iyh)
- void [set_palette](#) (uint8_t first_palette, uint8_t nb_palettes, const palette_color_t *rgb_data) Z88DK_CALLEE
- void [set_native_tile_data](#) (uint16_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh)
- void [set_bkg_4bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh)
- void [set_bkg_native_data](#) (uint16_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh)
- void [set_sprite_4bpp_data](#) (uint8_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh)
- void [set_sprite_native_data](#) (uint8_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh)
- void [set_2bpp_palette](#) (uint16_t palette)
- void [set_tile_2bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src, uint16_t palette) Z88DK_CALLEE PRESERVES_REGS(iyh)
- void [set_bkg_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_sprite_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_bkg_2bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_sprite_2bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src)
- void [set_1bpp_colors](#) (uint8_t fgcolor, uint8_t bgcolor)
- void [set_tile_1bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src, uint16_t colors) Z88DK_CALLEE PRESERVES_REGS(iyh)
- void [set_bkg_1bpp_data](#) (uint16_t start, uint16_t ntiles, const void *src)

- void `set_sprite_1bpp_data` (uint16_t start, uint16_t ntiles, const void *src)
- void `set_data` (uint16_t dst, const void *src, uint16_t size) Z88DK_CALLEE PRESERVES_REGS(iyh)
- void `vmemcpy` (uint16_t dst, const void *src, uint16_t size) Z88DK_CALLEE PRESERVES_REGS(iyh)
- void `set_tile_map` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) Z88DK_CALLEE
- void `set_tile_map_compat` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) Z88DK_CALLEE
- void `set_bkg_based_tiles` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles, uint8_t base_tile)
- void `set_win_based_tiles` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles, uint8_t base_tile)
- void `set_bkg_attributes` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles)
- void `set_tile_submap` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w) Z88DK_CALLEE
- void `set_tile_submap_compat` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w) Z88DK_CALLEE
- void `set_bkg_based_submap` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w, uint8_t base_tile)
- void `set_win_based_submap` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w, uint8_t base_tile)
- void `set_bkg_submap_attributes` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)
- void `fill_rect` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint16_t tile) Z88DK_CALLEE
- void `fill_rect_compat` (uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint16_t tile) Z88DK_CALLEE
- void `SET_SHADOW_OAM_ADDRESS` (void *address)
- void `set_sprite_tile` (uint8_t nb, uint8_t tile)
- uint8_t `get_sprite_tile` (uint8_t nb)
- void `set_sprite_prop` (uint8_t nb, uint8_t prop)
- uint8_t `get_sprite_prop` (uint8_t nb)
- void `move_sprite` (uint8_t nb, uint8_t x, uint8_t y)
- void `scroll_sprite` (uint8_t nb, int8_t x, int8_t y)
- void `hide_sprite` (uint8_t nb)
- void `set_vram_byte` (uint8_t *addr, uint8_t v) Z88DK_CALLEE PRESERVES_REGS(iyh)
- uint8_t * `set_attributed_tile_xy` (uint8_t x, uint8_t y, uint16_t t) Z88DK_CALLEE PRESERVES_REGS(iyh)
- uint8_t * `set_tile_xy` (uint8_t x, uint8_t y, uint8_t t) Z88DK_CALLEE PRESERVES_REGS(iyh)
- uint8_t * `set_attribute_xy` (uint8_t x, uint8_t y, uint8_t a) Z88DK_CALLEE PRESERVES_REGS(iyh)
- uint8_t * `get_bkg_xy_addr` (uint8_t x, uint8_t y) Z88DK_CALLEE PRESERVES_REGS(iyh)

Variables

- const `UBYTE_BIOS`
- const `uint8_t_SYSTEM`
- void `c`
- void `d`
- void `e`
- void `iyh`
- void `iyi`
- void `h`
- void `l`
- volatile `uint16_t_sys_time`
- volatile `uint8_t_vbl_done`
- void `b`
- `uint16_t_current_2bpp_palette`
- `uint16_t_current_1bpp_colors`
- `uint8_t_map_tile_offset`
- `uint8_t_submap_tile_offset`
- volatile `uint8_t_shadow_OAM[]`
- volatile `uint8_t_shadow_OAM_base`
- volatile `uint8_t_shadow_OAM_OFF`
- volatile `uint8_t_sprites_OFF`

20.120.1 Detailed Description

SMS/GG specific functions.

20.120.2 Macro Definition Documentation

20.120.2.1 SEGA `#define SEGA`

20.120.2.2 SYSTEM_60HZ `#define SYSTEM_60HZ 0x00`

20.120.2.3 SYSTEM_50HZ `#define SYSTEM_50HZ 0x01`

20.120.2.4 VBK_REG `#define VBK_REG VDP_ATTR_SHIFT`

20.120.2.5 J_UP `#define J_UP 0b00000001`

Joypad bits. A logical OR of these is used in the `wait_pad` and `joypad` functions. For example, to see if the B button is pressed try

```
uint8_t keys; keys = joypad(); if (keys & J_B) { ... }
```

See also

[joypad](#)

20.120.2.6 J_DOWN `#define J_DOWN 0b00000010`

20.120.2.7 J_LEFT `#define J_LEFT 0b00000100`

20.120.2.8 J_RIGHT `#define J_RIGHT 0b00001000`

20.120.2.9 J_B `#define J_B 0b00010000`

20.120.2.10 J_A `#define J_A 0b00100000`

20.120.2.11 J_START `#define J_START 0b01000000`

20.120.2.12 J_SELECT `#define J_SELECT 0b10000000`

20.120.2.13 M_TEXT_OUT `#define M_TEXT_OUT 0x02U`

Screen modes. Normally used by internal functions only.

See also

[mode\(\)](#)

20.120.2.14 M_TEXT_INOUT `#define M_TEXT_INOUT 0x03U`

20.120.2.15 M_NO_SCROLL `#define M_NO_SCROLL 0x04U`

Set this in addition to the others to disable scrolling

If scrolling is disabled, the cursor returns to (0,0)

See also

[mode\(\)](#)

20.120.2.16 M_NO_INTERP `#define M_NO_INTERP 0x08U`

Set this to disable interpretation

See also

[mode\(\)](#)

20.120.2.17 S_BANK `#define S_BANK 0x01U`

The ninth bit of the tile id

20.120.2.18 S_FLIPX `#define S_FLIPX 0x02U`

If set the background tile will be flipped horizontally.

20.120.2.19 S_FLIPY `#define S_FLIPY 0x04U`

If set the background tile will be flipped vertically.

20.120.2.20 S_PALETTE `#define S_PALETTE 0x08U`

If set the background tile palette.

20.120.2.21 S_PRIORITY `#define S_PRIORITY 0x10U`

If set the background tile priority.

20.120.2.22 S_PAL `#define S_PAL(
n) ((n) & 0x01U) << 3)`

Dummy function used by other platforms. Required for the png2asset tool's metasprite output.

20.120.2.23 __WRITE_VDP_REG_UNSAFE `#define __WRITE_VDP_REG_UNSAFE(
REG,
v) shadow_##REG=(v),VDP_CMD=(shadow_##REG),VDP_CMD=REG`

20.120.2.24 __WRITE_VDP_REG `#define __WRITE_VDP_REG(
REG,
v) shadow_##REG=(v);__asm__("di");VDP_CMD=(shadow_##REG);VDP_CMD=REG;__asm__↵
("ei")`

20.120.2.25 __READ_VDP_REG `#define __READ_VDP_REG(
REG) shadow_##REG`

20.120.2.26 EMPTY_IFLAG `#define EMPTY_IFLAG 0x00U`

Disable calling of interrupt service routines

20.120.2.27 VBL_IFLAG `#define VBL_IFLAG 0x01U`

VBlank Interrupt occurs at the start of the vertical blank.
During this period the video ram may be freely accessed.

See also

[set_interrupts\(\)](#),
[add_VBL](#)

20.120.2.28 LCD_IFLAG `#define LCD_IFLAG 0x02U`

LCD Interrupt when triggered by the STAT register.

See also

[set_interrupts\(\)](#),
[add_LCD](#)

20.120.2.29 TIM_IFLAG `#define TIM_IFLAG 0x04U`

Does nothing on SMS/GG

20.120.2.30 SIO_IFLAG `#define SIO_IFLAG 0x08U`

Does nothing on SMS/GG

20.120.2.31 JOY_IFLAG `#define JOY_IFLAG 0x10U`

Does nothing on SMS/GG

20.120.2.32 SCREENWIDTH `#define SCREENWIDTH DEVICE_SCREEN_PX_WIDTH`

Width of the visible screen in pixels.

20.120.2.33 SCREENHEIGHT `#define SCREENHEIGHT DEVICE_SCREEN_PX_HEIGHT`

Height of the visible screen in pixels.

20.120.2.34 MINWNDPOSX `#define MINWNDPOSX 0x00U`

The Minimum X position of the Window Layer (Left edge of screen)

See also

[move_win\(\)](#)

20.120.2.35 MINWNDPOSY `#define MINWNDPOSY 0x00U`

The Minimum Y position of the Window Layer (Top edge of screen)

See also

[move_win\(\)](#)

20.120.2.36 MAXWNDPOSX `#define MAXWNDPOSX 0x00U`

The Maximum X position of the Window Layer (Right edge of screen)

See also

[move_win\(\)](#)

20.120.2.37 MAXWNDPOSY `#define MAXWNDPOSY 0x00U`

The Maximum Y position of the Window Layer (Bottom edge of screen)

See also

[move_win\(\)](#)

20.120.2.38 DISPLAY_ON `#define DISPLAY_ON __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) |= R1_DISP_ON)`

Turns the display back on.

See also

[display_off](#), [DISPLAY_OFF](#)

20.120.2.39 DISPLAY_OFF `#define DISPLAY_OFF display_off();`

Turns the display off immediately.

See also

[display_off](#), [DISPLAY_ON](#)

20.120.2.40 HIDE_LEFT_COLUMN `#define HIDE_LEFT_COLUMN __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) |= R0_LCB)`

Blanks leftmost column, so it is not garbaged when you use horizontal scroll

See also

[SHOW_LEFT_COLUMN](#)

20.120.2.41 SHOW_LEFT_COLUMN `#define SHOW_LEFT_COLUMN __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) &= (~R0_LCB))`

Shows leftmost column

See also

[HIDE_LEFT_COLUMN](#)

20.120.2.42 SET_BORDER_COLOR `#define SET_BORDER_COLOR(C) __WRITE_VDP_REG(VDP_R7, ((C) | 0xf0u))`

Sets border color

20.120.2.43 SHOW_BKG `#define SHOW_BKG`

Turns on the background layer. Not yet implemented

20.120.2.44 HIDE_BKG `#define HIDE_BKG`

Turns off the background layer. Not yet implemented

20.120.2.45 SHOW_WIN `#define SHOW_WIN`

Turns on the window layer Not yet implemented

20.120.2.46 HIDE_WIN `#define HIDE_WIN`

Turns off the window layer. Not yet implemented

20.120.2.47 SHOW_SPRITES `#define SHOW_SPRITES ( _sprites_OFF = 0 )`
Turns on the sprites layer.

20.120.2.48 HIDE_SPRITES `#define HIDE_SPRITES ( _sprites_OFF = 1 )`
Turns off the sprites layer.

20.120.2.49 SPRITES_8x16 `#define SPRITES_8x16 __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) | R1_SPR_8X16)`
Sets sprite size to 8x16 pixels, two tiles one above the other.

20.120.2.50 SPRITES_8x8 `#define SPRITES_8x8 __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) &= (~R1_SPR_8X16))`
Sets sprite size to 8x8 pixels, one tile.

20.120.2.51 DEVICE_SUPPORTS_COLOR `#define DEVICE_SUPPORTS_COLOR (TRUE)`
Macro returns TRUE if device supports color (it always does on SMS/GG)

20.120.2.52 VBL_DONE `#define VBL_DONE _vbl_done`

20.120.2.53 DIV_REG `#define DIV_REG get_r_reg()`

20.120.2.54 _current_bank `#define _current_bank MAP_FRAME1`
Tracks current active ROM bank in frame 1

20.120.2.55 CURRENT_BANK `#define CURRENT_BANK MAP_FRAME1`

20.120.2.56 BANK `#define BANK( VARNAME ) ( (uint8_t) & __bank_ ## VARNAME )`
Obtains the **bank number** of VARNAME

Parameters

<i>VARNAME</i>	Name of the variable which has a <code>__bank_</code> <i>VARNAME</i> companion symbol which is adjusted by bankpack
----------------	---

Use this to obtain the bank number from a bank reference created with [BANKREF\(\)](#).

See also

[BANKREF_EXTERN\(\)](#), [BANKREF\(\)](#)

20.120.2.57 BANKREF `#define BANKREF( VARNAME )`

Value:

```
void __func_ ## VARNAME(void) __banked __naked { \
__asm \
    .local b__func_ ## VARNAME \
    __bank_ ## VARNAME = b__func_ ## VARNAME \
    .globl __bank_ ## VARNAME \
__endasm; \
}
```

Creates a reference for retrieving the bank number of a variable or function

Parameters

<i>VARNAME</i>	Variable name to use, which may be an existing identifier
----------------	---

See also

[BANK\(\)](#) for obtaining the bank number of the included data.

More than one [BANKREF \(\)](#) may be created per file, but each call should always use a unique *VARNAME*. Use [BANKREF_EXTERN\(\)](#) within another source file to make the variable and it's data accesible there.

20.120.2.58 BANKREF_EXTERN `#define BANKREF_EXTERN(
VARNAME) extern const void __bank_ ## VARNAME;`

Creates extern references for accessing a [BANKREF\(\)](#) generated variable.

Parameters

<i>VARNAME</i>	Name of the variable used with BANKREF()
----------------	--

This makes a [BANKREF\(\)](#) reference in another source file accessible in the current file for use with [BANK\(\)](#).

See also

[BANKREF\(\)](#), [BANK\(\)](#)

20.120.2.59 SWITCH_ROM `#define SWITCH_ROM(
b) MAP_FRAME1=(b)`

Makes switch the active ROM bank in frame 1

Parameters

<i>b</i>	ROM bank to switch to
----------	-----------------------

20.120.2.60 SWITCH_ROM1 `#define SWITCH_ROM1 SWITCH_ROM`

20.120.2.61 SWITCH_ROM2 `#define SWITCH_ROM2(
b) MAP_FRAME2=(b)`

Makes switch the active ROM bank in frame 2

Parameters

<i>b</i>	ROM bank to switch to
----------	-----------------------

20.120.2.62 SWITCH_RAM `#define SWITCH_RAM(
b) RAM_CONTROL=((b) &1) ? RAM_CONTROL|RAMCTL_BANK:RAM_CONTROL& (~RAMCTL_BANK)`

Switches RAM bank

Parameters

<i>b</i>	SRAM bank to switch to
----------	------------------------

20.120.2.63 ENABLE_RAM `#define ENABLE_RAM RAM_CONTROL|=RAMCTL_RAM`
Enables RAM

20.120.2.64 DISABLE_RAM `#define DISABLE_RAM RAM_CONTROL&=(~RAMCTL_RAM)`
Disables RAM

20.120.2.65 set_bkg_palette_entry `#define set_bkg_palette_entry set_palette_entry`

20.120.2.66 set_sprite_palette_entry `#define set_sprite_palette_entry(
 palette,
 entry,
 rgb_data) set_palette_entry(1,entry,rgb_data)`

20.120.2.67 set_bkg_palette `#define set_bkg_palette set_palette`

20.120.2.68 set_sprite_palette `#define set_sprite_palette(
 first_palette,
 nb_palettes,
 rgb_data) set_palette(1,1,rgb_data)`

20.120.2.69 COMPAT_PALETTE `#define COMPAT_PALETTE(
 C0,
 C1,
 C2,
 C3) (((uint16_t)(C3) << 12) | ((uint16_t)(C2) << 8) | ((uint16_t)(C1) << 4) |
(uint16_t)(C0))`

20.120.2.70 set_bkg_tiles `#define set_bkg_tiles set_tile_map_compat`

20.120.2.71 set_win_tiles `#define set_win_tiles set_tile_map_compat`

20.120.2.72 set_bkg_submap `#define set_bkg_submap set_tile_submap_compat`

20.120.2.73 set_win_submap `#define set_win_submap set_tile_submap_compat`

20.120.2.74 fill_bkg_rect `#define fill_bkg_rect fill_rect_compat`

20.120.2.75 fill_win_rect `#define fill_win_rect fill_rect_compat`

20.120.2.76 DISABLE_VBL_TRANSFER `#define DISABLE_VBL_TRANSFER _shadow_OAM_base = 0`
Disable shadow OAM to VRAM copy on each VBlank

20.120.2.77 ENABLE_VBL_TRANSFER `#define ENABLE_VBL_TRANSFER _shadow_OAM_base = (uint8_t) ((uint16_t) &shadow_OAM_base >> 8)`

Enable shadow OAM to VRAM copy on each VBlank

20.120.2.78 MAX_HARDWARE_SPRITES `#define MAX_HARDWARE_SPRITES 64`

Amount of hardware sprites in OAM

20.120.2.79 HARDWARE_SPRITE_CAN_FLIP_X `#define HARDWARE_SPRITE_CAN_FLIP_X 0`

True if sprite hardware can flip sprites by X (horizontally)

20.120.2.80 HARDWARE_SPRITE_CAN_FLIP_Y `#define HARDWARE_SPRITE_CAN_FLIP_Y 0`

True if sprite hardware can flip sprites by Y (vertically)

20.120.2.81 set_bkg_tile_xy `#define set_bkg_tile_xy set_tile_xy`

20.120.2.82 set_win_tile_xy `#define set_win_tile_xy set_tile_xy`

20.120.2.83 set_bkg_attribute_xy `#define set_bkg_attribute_xy set_attribute_xy`

20.120.2.84 set_win_attribute_xy `#define set_win_attribute_xy set_attribute_xy`

20.120.2.85 get_win_xy_addr `#define get_win_xy_addr get_bkg_xy_addr`

20.120.3 Typedef Documentation

20.120.3.1 int_handler `typedef void(* int_handler) (void) NONBANKED`

Interrupt handlers

20.120.4 Function Documentation

20.120.4.1 WRITE_VDP_CMD() `void WRITE_VDP_CMD (uint16_t cmd )`

20.120.4.2 WRITE_VDP_DATA() `void WRITE_VDP_DATA (uint16_t data )`

20.120.4.3 mode() `void mode (uint8_t m )`

Set the current screen mode - one of M_* modes

Normally used by internal functions only.

See also

[M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.120.4.4 `get_mode()` `uint8_t get_mode (`
`void )`

Returns the current mode

See also

[M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

Returns the current mode

See also

[M_DRAWING](#), [M_TEXT_OUT](#), [M_TEXT_INOUT](#), [M_NO_SCROLL](#), [M_NO_INTERP](#)

20.120.4.5 `get_system()` `uint8_t get_system (`
`void ) [inline]`

Returns the system gbdk is running on.

20.120.4.6 `set_interrupts()` `void set_interrupts (`
`uint8_t flags )`

Clears any pending interrupts and sets the interrupt mask register IO to flags.

Parameters

<i>flags</i>	A logical OR of *_IFLAGS
--------------	--------------------------

Note

This disables and then re-enables interrupts so it must be used outside of a critical section.

See also

[enable_interrupts\(\)](#), [disable_interrupts\(\)](#)

[VBL_IFLAG](#), [LCD_IFLAG](#), [TIM_IFLAG](#), [SIO_IFLAG](#), [JOY_IFLAG](#)

20.120.4.7 `remove_VBL()` `void remove_VBL (`
`int_handler h )`

Removes the VBL interrupt handler.

See also

[add_VBL\(\)](#)

20.120.4.8 `remove_LCD()` `void remove_LCD (`
`int_handler h )`

Removes the LCD interrupt handler.

See also

[add_LCD\(\)](#), [remove_VBL\(\)](#)

20.120.4.9 `remove_TIM()` `void remove_TIM (`
`int_handler h )`

20.120.4.10 remove_SIO() `void remove_SIO (`
 `int_handler h )`

20.120.4.11 remove_JOY() `void remove_JOY (`
 `int_handler h )`

20.120.4.12 add_VBL() `void add_VBL (`
 `int_handler h )`

Adds a V-blank interrupt handler.

20.120.4.13 add_LCD() `void add_LCD (`
 `int_handler h )`

Adds a LCD interrupt handler.

20.120.4.14 add_TIM() `void add_TIM (`
 `int_handler h )`

Does nothing on SMS/GG

20.120.4.15 add_SIO() `void add_SIO (`
 `int_handler h )`

Does nothing on SMS/GG

20.120.4.16 add_JOY() `void add_JOY (`
 `int_handler h )`

Does nothing on SMS/GG

20.120.4.17 cancel_pending_interrupts() `uint8_t cancel_pending_interrupts (`
 `void ) [inline]`

Cancel pending interrupts

20.120.4.18 move_bkg() `void move_bkg (`
 `uint8_t x,`
 `uint8_t y ) [inline]`

20.120.4.19 scroll_bkg() `void scroll_bkg (`
 `int8_t x,`
 `int8_t y ) [inline]`

20.120.4.20 vsync() `void vsync (`
 `void )`

HALTs the CPU and waits for the vertical blank interrupt.

This is often used in main loops to idle the CPU at low power until it's time to start the next frame. It's also useful for syncing animation with the screen re-draw.

Warning: If the VBL interrupt is disabled, this function will never return. If the screen is off this function returns immediately.

20.120.4.21 wait_vbl_done() `void wait_vbl_done (`
 `void )`

Obsolete. This function has been replaced by [vsync\(\)](#), which has identical behavior.

20.120.4.22 display_off() `void display_off (`
`void ) [inline]`

Turns the display off.

See also

[DISPLAY_ON](#)

20.120.4.23 refresh_OAM() `void refresh_OAM (`
`void )`

Copies data from shadow OAM to OAM

20.120.4.24 get_r_reg() `uint8_t get_r_reg (`
`void )`

Return R register for the DIV_REG emulation

Increments once per CPU instruction (fetches the Z80 CPU R register)

20.120.4.25 delay() `void delay (`
`uint16_t d )`

Delays the given number of milliseconds. Uses no timers or interrupts, and can be called with interrupts disabled

20.120.4.26 joypad() `uint8_t joypad (`
`void )`

Reads and returns the current state of the joypad.

20.120.4.27 waitpad() `uint8_t waitpad (`
`uint8_t mask )`

Waits until at least one of the buttons given in mask are pressed.

20.120.4.28 waitpadup() `void waitpadup (`
`void )`

Waits for the directional pad and all buttons to be released.

Note: Checks in a loop that doesn't HALT at all, so the CPU will be maxed out until this call returns.

20.120.4.29 joypad_init() `uint8_t joypad_init (`
`uint8_t npads,`
`joypads_t * joypads )`

Initializes [joypads_t](#) structure for polling multiple joypads

Parameters

<i>npads</i>	number of joypads requested (1, 2 or 4)
<i>joypads</i>	pointer to joypads_t structure to be initialized

Only required for [joypad_ex](#), not required for calls to regular [joypad\(\)](#)

Returns

number of joypads available

See also

[joypad_ex\(\)](#), [joypads_t](#)

20.120.4.30 joypad_ex() `void joypad_ex (`
`joypads_t * joypads )`

Polls all available joypads

Parameters

<i>joypads</i>	pointer to joypads_t structure to be filled with joypad statuses, must be previously initialized with joypad_init()
----------------	---

See also

[joypad_init\(\)](#), [joypads_t](#)

20.120.4.31 enable_interrupts() `void enable_interrupts (`
`void ) [inline]`

Enables unmasked interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

See also

[disable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.120.4.32 disable_interrupts() `void disable_interrupts (`
`void ) [inline]`

Disables interrupts

Note

Use [CRITICAL](#) {...} instead for creating a block of code which should execute with interrupts temporarily turned off.

This function may be called as many times as you like; however the first call to [enable_interrupts](#) will re-enable them.

See also

[enable_interrupts](#), [set_interrupts](#), [CRITICAL](#)

20.120.4.33 set_default_palette() `void set_default_palette (`
`void )`

20.120.4.34 cgb_compatibility() `void cgb_compatibility (`
`void ) [inline]`

Obsolete. This function has been replaced by [set_default_palette\(\)](#), which has identical behavior.

20.120.4.35 cpu_fast() `void cpu_fast (`
`void ) [inline]`

Set CPU speed to fast (CGB Double Speed) operation.

On startup the CGB operates in Normal Speed Mode and can be switched into Double speed mode (faster processing but also higher power consumption). See the Pan Docs for more information about which hardware features operate faster and which remain at Normal Speed.

- Interrupts are temporarily disabled and then re-enabled during this call.
- You can check to see if `_cpu == CGB_TYPE` before using this function.

See also

[cpu_slow\(\)](#), [_cpu](#)

20.120.4.36 `set_palette_entry()` `void set_palette_entry (`
 [uint8_t](#) *palette*,
 [uint8_t](#) *entry*,
 [uint16_t](#) *rgb_data*)

20.120.4.37 `set_palette()` `void set_palette (`
 [uint8_t](#) *first_palette*,
 [uint8_t](#) *nb_palettes*,
 const [palette_color_t](#) * *rgb_data*)

Set color palette(s)

Parameters

<i>first_palette</i>	Index of the first 16 color palette to write (0-1)
<i>nb_palettes</i>	Number of palettes to write (1-2, max depends on <i>first_palette</i>)
<i>rgb_data</i>	Pointer to source palette data

Writes **nb_palettes** to palette data starting at **first_palette**, Palette data is sourced from **rgb_data**.

- Palette 0 can be used for the Background.
- Palette 1 is shared between Background and Sprites.

On the Game Gear

- Each Palette is 32 bytes in size: 16 colors x 2 bytes per palette color entry.
- Each color (16 per palette) is packed as BGR-444 format (x:4:4:4, MSBits [15..12] are unused).
- Each component (R, G, B) may have values from 0 - 15 (4 bits), 15 is brightest.

On the SMS

- On SMS each Palette is 16 bytes in size: 16 colors x 1 byte per palette color entry.
- Each color (16 per palette) is packed as BGR-222 format (x:2:2:2, MSBits [7..6] are unused).
- Each component (R, G, B) may have values from 0 - 3 (2 bits), 3 is brightest.

See also

[RGB\(\)](#), [set_sprite_palette\(\)](#), [set_bkg_palette\(\)](#), [set_palette_entry\(\)](#), [set_sprite_palette_entry\(\)](#), [set_bkg_palette_entry\(\)](#), [set_sprite_palette\(\)](#)

20.120.4.38 `set_native_tile_data()` `void set_native_tile_data (`
 [uint16_t](#) *start*,
 [uint16_t](#) *ntiles*,
 const void * *src*)

20.120.4.39 set_bkg_4bpp_data() void set_bkg_4bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src)

20.120.4.40 set_bkg_native_data() void set_bkg_native_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src)

20.120.4.41 set_sprite_4bpp_data() void set_sprite_4bpp_data (
 uint8_t start,
 uint16_t ntiles,
 const void * src)

20.120.4.42 set_sprite_native_data() void set_sprite_native_data (
 uint8_t start,
 uint16_t ntiles,
 const void * src)

20.120.4.43 set_2bpp_palette() void set_2bpp_palette (
 uint16_t palette) [inline]

20.120.4.44 set_tile_2bpp_data() void set_tile_2bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src,
 uint16_t palette)

20.120.4.45 set_bkg_data() void set_bkg_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.120.4.46 set_sprite_data() void set_sprite_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.120.4.47 set_bkg_2bpp_data() void set_bkg_2bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.120.4.48 set_sprite_2bpp_data() void set_sprite_2bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.120.4.49 set_1bpp_colors() void set_1bpp_colors (
 uint8_t fgcolor,
 uint8_t bgcolor) [inline]

20.120.4.50 set_tile_1bpp_data() void set_tile_1bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src,
 uint16_t colors)

20.120.4.51 set_bkg_1bpp_data() void set_bkg_1bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.120.4.52 set_sprite_1bpp_data() void set_sprite_1bpp_data (
 uint16_t start,
 uint16_t ntiles,
 const void * src) [inline]

20.120.4.53 set_data() void set_data (
 uint16_t dst,
 const void * src,
 uint16_t size)

Copies arbitrary data to an address in VRAM

Parameters

<i>dst</i>	destination VRAM Address
<i>src</i>	Pointer to source buffer
<i>size</i>	Number of bytes to copy

Copies **size** bytes from a buffer at **_src__** to VRAM starting at **dst**.

20.120.4.54 vmemcpy() void vmemcpy (
 uint16_t dst,
 const void * src,
 uint16_t size)

20.120.4.55 set_tile_map() void set_tile_map (
 uint8_t x,
 uint8_t y,
 uint8_t w,
 uint8_t h,
 const uint8_t * tiles)

20.120.4.56 set_tile_map_compat() `void set_tile_map_compat (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles )`

20.120.4.57 set_bkg_based_tiles() `void set_bkg_based_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles,`
`uint8_t base_tile ) [inline]`

Sets a rectangular region of Background Tile Map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 32
<i>h</i>	Height of area to set in tiles. Range 1 - 32
<i>tiles</i>	Pointer to source tile map data
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_tiles\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_tiles](#) for more details

20.120.4.58 set_win_based_tiles() `void set_win_based_tiles (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles,`
`uint8_t base_tile ) [inline]`

20.120.4.59 set_bkg_attributes() `void set_bkg_attributes (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * tiles ) [inline]`

20.120.4.60 set_tile_submap() `void set_tile_submap (`
`uint8_t x,`

```

uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * map,
uint8_t map_w )

```

20.120.4.61 set_tile_submap_compat() void set_tile_submap_compat (

```

uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * map,
uint8_t map_w )

```

20.120.4.62 set_bkg_based_submap() void set_bkg_based_submap (

```

uint8_t x,
uint8_t y,
uint8_t w,
uint8_t h,
const uint8_t * map,
uint8_t map_w,
uint8_t base_tile ) [inline]

```

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in Background Map tile coordinates. Range 0 - 31
<i>y</i>	Y Start position in Background Map tile coordinates. Range 0 - 31
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255
<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255

This is identical to [set_bkg_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_submap](#) for more details

Sets a rectangular area of the Background Tile Map using a sub-region from a source tile map. The offset value in **base_tile** is added to the tile ID for each map entry.

Parameters

<i>x</i>	X Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>y</i>	Y Start position in both the Source Tile Map and hardware Background Map tile coordinates. Range 0 - 255
<i>w</i>	Width of area to set in tiles. Range 1 - 255
<i>h</i>	Height of area to set in tiles. Range 1 - 255
<i>map</i>	Pointer to source tile map data
<i>map_w</i>	Width of source tile map in tiles. Range 1 - 255

Parameters

<i>base_tile</i>	Offset each tile ID entry of the source map by this value. Range 1 - 255
------------------	--

This is identical to [set_bkg_submap\(\)](#) except that it adds the **base_tile** parameter for when a tile map's tiles don't start at index zero. (For example, the tiles used by the map range from 100 -> 120 in VRAM instead of 0 -> 20).

See also

[set_bkg_submap](#) for more details

20.120.4.63 set_win_based_submap() `void set_win_based_submap (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * map,`
`uint8_t map_w,`
`uint8_t base_tile ) [inline]`

20.120.4.64 set_bkg_submap_attributes() `void set_bkg_submap_attributes (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint8_t * map,`
`uint8_t map_w ) [inline]`

20.120.4.65 fill_rect() `void fill_rect (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint16_t tile )`

20.120.4.66 fill_rect_compat() `void fill_rect_compat (`
`uint8_t x,`
`uint8_t y,`
`uint8_t w,`
`uint8_t h,`
`const uint16_t tile )`

20.120.4.67 SET_SHADOW_OAM_ADDRESS() `void SET_SHADOW_OAM_ADDRESS (`
`void * address ) [inline]`

Sets address of 256-byte aligned array of shadow OAM to be transferred on each VBlank

20.120.4.68 set_sprite_tile() `void set_sprite_tile (`
`uint8_t nb,`
`uint8_t tile ) [inline]`

Sets sprite number **nb** in the OAM to display tile number **__tile**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>tile</i>	Selects a tile (0 - 255) from memory at 8000h - 8FFFh In CGB Mode this could be either in VRAM Bank 0 or 1, depending on Bit 3 of the OAM Attribute Flag (see set_sprite_prop)

In 8x16 mode:

- The sprite will also display the next tile (**tile** + 1) directly below (y + 8) the first tile.
- The lower bit of the tile number is ignored: the upper 8x8 tile is (**tile** & 0xFE), and the lower 8x8 tile is (**tile** | 0x01).
- See: [SPRITES_8x16](#)

20.120.4.69 get_sprite_tile() `uint8_t get_sprite_tile (`
`uint8_t nb ) [inline]`

Returns the tile number of sprite number **nb** in the OAM.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_tile](#) for more details

20.120.4.70 set_sprite_prop() `void set_sprite_prop (`
`uint8_t nb,`
`uint8_t prop ) [inline]`

Function has no affect on sms.

This function is only here to enable game portability

20.120.4.71 get_sprite_prop() `uint8_t get_sprite_prop (`
`uint8_t nb ) [inline]`

Returns the OAM Property Flags of sprite number **nb**.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

See also

[set_sprite_prop](#) for property bitfield settings

20.120.4.72 move_sprite() `void move_sprite (`
`uint8_t nb,`
`uint8_t x,`
`uint8_t y ) [inline]`

Moves sprite number **nb** to the **x**, **y** position on the screen.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	X Position. Specifies the sprites horizontal position on the screen (minus 8). An offscreen value (X=0 or X>=168) hides the sprite, but the sprite still affects the priority ordering - a better way to hide a sprite is to set its Y-coordinate offscreen.
<i>y</i>	Y Position. Specifies the sprites vertical position on the screen (minus 16). An offscreen value (for example, Y=0 or Y>=160) hides the sprite.

Moving the sprite to 0,0 (or similar off-screen location) will hide it.

20.120.4.73 scroll_sprite() `void scroll_sprite (`
`uint8_t nb,`
`int8_t x,`
`int8_t y ) [inline]`

Moves sprite number **nb** relative to its current position.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
<i>x</i>	Number of pixels to move the sprite on the X axis Range: -128 - 127
<i>y</i>	Number of pixels to move the sprite on the Y axis Range: -128 - 127

See also

[move_sprite](#) for more details about the X and Y position

20.120.4.74 hide_sprite() `void hide_sprite (`
`uint8_t nb ) [inline]`

Hides sprite number **nb** by moving it to zero position by Y.

Parameters

<i>nb</i>	Sprite number, range 0 - 39
-----------	-----------------------------

20.120.4.75 set_vram_byte() `void set_vram_byte (`
`uint8_t * addr,`
`uint8_t v )`

Set byte in vram at given memory location

Parameters

<i>addr</i>	address to write to
<i>v</i>	value

20.120.4.76 set_attributed_tile_xy() `uint8_t * set_attributed_tile_xy (`
`uint8_t x,`


```
uint8_t y,  
uint16_t t )
```

Set single tile t with attributes on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>t</i>	tile index

Returns

returns the address of tile, so you may use faster [set_vram_byte\(\)](#) later

20.120.4.77 **set_tile_xy()** `uint8_t * set_tile_xy (`

```
uint8_t x,  
uint8_t y,  
uint8_t t )
```

Set single tile t on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>t</i>	tile index

Returns

returns the address of tile, so you may use faster [set_vram_byte\(\)](#) later

20.120.4.78 **set_attribute_xy()** `uint8_t * set_attribute_xy (`

```
uint8_t x,  
uint8_t y,  
uint8_t a ) [inline]
```

Set single attribute data a on background layer at x,y

Parameters

<i>x</i>	X-coordinate
<i>y</i>	Y-coordinate
<i>a</i>	tile attributes

Returns

returns the address of tile attribute, so you may use faster [set_vram_byte\(\)](#) later

20.120.4.79 **get_bkg_xy_addr()** `uint8_t * get_bkg_xy_addr (`

```
uint8_t x,  
uint8_t y )
```

Get address of X,Y tile of background map

20.120.5 Variable Documentation

20.120.5.1 `_BIOS` `const UBYTE _BIOS [extern]`

20.120.5.2 `_SYSTEM` `const uint8_t _SYSTEM [extern]`

20.120.5.3 `c` `void c`

20.120.5.4 `d` `void d`

20.120.5.5 `e` `void e`

20.120.5.6 `iyh` `void iyh`

20.120.5.7 `iyi` `uint8_t iyi`

Initial value:

```
{  
    __asm__("ei")  
}
```

20.120.5.8 `h` `void h`

20.120.5.9 `l` `void l`

20.120.5.10 `sys_time` `volatile uint16_t sys_time [extern]`

Global Time Counter in VBL periods (60Hz)

Increments once per Frame

Will wrap around every ~18 minutes (unsigned 16 bits = 65535 / 60 / 60 = 18.2)

20.120.5.11 `_vbl_done` `volatile uint8_t _vbl_done [extern]`

Flag indicating the VBlank ISR has run

Flag gets cleared at the start of `vsync()` / `wait_vbl_done()` and set in the default VBlank ISR handler.

20.120.5.12 `b` `void b`

20.120.5.13 `_current_2bpp_palette` `uint16_t _current_2bpp_palette [extern]`

20.120.5.14 `_current_1bpp_colors` `uint16_t _current_1bpp_colors [extern]`

20.120.5.15 `_map_tile_offset` `uint8_t _map_tile_offset [extern]`

20.120.5.16 `_submap_tile_offset` `uint8_t` `_submap_tile_offset` [extern]

20.120.5.17 `shadow_OAM` `volatile uint8_t` `shadow_OAM[]` [extern]

Shadow OAM array in WRAM, that is transferred into the real OAM each VBlank

20.120.5.18 `_shadow_OAM_base` `volatile uint8_t` `_shadow_OAM_base` [extern]

MSB of shadow_OAM address is used by OAM copying routine

MSB of shadow_OAM address is used by OAM DMA copying routine

20.120.5.19 `_shadow_OAM_OFF` `volatile uint8_t` `_shadow_OAM_OFF` [extern]

Flag for disabling of OAM copying routine

Values:

- 1: OAM copy routine is disabled (non-isr VDP operation may be in progress)
- 0: OAM copy routine is enabled

This flag is modified by all sms/gg GBDK API calls that write to the VDP. It is set to DISABLED when they start and ENABLED when they complete.

Note

It is recommended to avoid writing to the Video Display Processor (VDP) during an interrupt service routine (ISR) since it can corrupt the VDP pointer of an VDP operation already in progress.

If it is necessary, this flag can be used during an ISR to determine whether a VDP operation is already in progress.

If the value is 1 then avoid writing to the VDP (tiles, map, scrolling, colors, etc).

```
// at the beginning of and ISR that would write to the VDP
if (_shadow_OAM_OFF) return;
```

See also

[docs_consoles_safe_display_controller_access](#)

20.120.5.20 `_sprites_OFF` `volatile uint8_t` `_sprites_OFF` [extern]

20.121 sms.h

[Go to the documentation of this file.](#)

```
1
2
3
4 #ifndef _SMS_H
5 #define _SMS_H
6
7 #include <types.h>
8 #include <stdint.h>
9 #include <gbdk/version.h>
10 #include <sms/hardware.h>
11
12 #define SEGA
13
14 // Here NINTENDO means Game Boy & related clones
15 #ifdef NINTENDO
16 #undef NINTENDO
17 #endif
18
19 #ifdef NINTENDO_NES
20 #undef NINTENDO_NES
21 #endif
22
23 #ifdef MSX
24 #undef MSX
25 #endif
26
27 #if defined(__TARGET_sms)
28 #define MASTERSYSTEM
29 #elif defined(__TARGET_gg)
30 #define GAMEGEAR
31 #endif
```

```

32
33
34 extern const UBYTE _BIOS;
35
36 extern const uint8_t _SYSTEM;
37
38 #define SYSTEM_60HZ      0x00
39 #define SYSTEM_50HZ      0x01
40
41 #define VBK_REG VDP_ATTR_SHIFT
42
43 #define J_UP      0b00000001
44 #define J_DOWN    0b00000010
45 #define J_LEFT    0b00000100
46 #define J_RIGHT   0b00001000
47 #define J_B       0b00010000
48 #define J_A       0b00100000
49 #define J_START   0b01000000
50 #define J_SELECT  0b10000000
51
52 #define M_TEXT_OUT 0x02U
53 #define M_TEXT_INOUT 0x03U
54 #define M_NO_SCROLL 0x04U
55 #define M_NO_INTERP 0x08U
56
57 #define S_BANK      0x01U
58 #define S_FLIPX     0x02U
59 #define S_FLIPY     0x04U
60 #define S_PALETTE   0x08U
61 #define S_PRIORITY  0x10U
62 #define S_PAL(n)    ((n) & 0x01U) << 3
63
64 // VDP helper macros
65 #define __WRITE_VDP_REG_UNSAFE(REG, v) shadow_##REG=(v),VDP_CMD=(shadow_##REG),VDP_CMD=REG
66 #define __WRITE_VDP_REG(REG, v)
67     shadow_##REG=(v);__asm__("di");VDP_CMD=(shadow_##REG);VDP_CMD=REG;__asm__("ei")
68 #define __READ_VDP_REG(REG) shadow_##REG
69
70 void WRITE_VDP_CMD(uint16_t cmd) Z88DK_FASTCALL PRESERVES_REGS(b, c, d, e, iyh, iyl);
71 void WRITE_VDP_DATA(uint16_t data) Z88DK_FASTCALL PRESERVES_REGS(b, c, d, e, iyh, iyl);
72
73 void mode(uint8_t m) OLDCALL;
74
75 uint8_t get_mode(void) OLDCALL;
76
77 inline uint8_t get_system(void) {
78     return _SYSTEM;
79 }
80
81 /* Interrupt flags */
82 #define EMPTY_IFLAG 0x00U
83 #define VBL_IFLAG   0x01U
84 #define LCD_IFLAG   0x02U
85 #define TIM_IFLAG   0x04U
86 #define SIO_IFLAG   0x08U
87 #define JOY_IFLAG   0x10U
88
89 void set_interrupts(uint8_t flags) Z88DK_FASTCALL;
90
91 /* Limits */
92 #define SCREENWIDTH  DEVICE_SCREEN_PX_WIDTH
93 #define SCREENHEIGHT DEVICE_SCREEN_PX_HEIGHT
94 #define MINWNDPOSX    0x00U
95 #define MINWNDPOSY    0x00U
96 #define MAXWNDPOSX    0x00U
97 #define MAXWNDPOSY    0x00U
98
99 typedef void (*int_handler)(void) NONBANKED;
100
101 void remove_VBL(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(iyh, iyl);
102
103 void remove_LCD(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b, c, iyh, iyl);
104
105 void remove_TIM(int_handler h) Z88DK_FASTCALL;
106 void remove_SIO(int_handler h) Z88DK_FASTCALL;
107 void remove_JOY(int_handler h) Z88DK_FASTCALL;
108
109 void add_VBL(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(d, e, iyh, iyl);
110
111 void add_LCD(int_handler h) Z88DK_FASTCALL PRESERVES_REGS(b, c, iyh, iyl);
112
113 void add_TIM(int_handler h) Z88DK_FASTCALL;
114 void add_SIO(int_handler h) Z88DK_FASTCALL;
115 void add_JOY(int_handler h) Z88DK_FASTCALL;

```

```

215
216 inline uint8_t cancel_pending_interrupts(void) {
217     return VDP_STATUS;
218 }
219
220 inline void move_bkg(uint8_t x, uint8_t y) {
221     __WRITE_VDP_REG(VDP_RSCX, -x);
222     __WRITE_VDP_REG(VDP_RSCY, y);
223 }
224
225 inline void scroll_bkg(int8_t x, int8_t y) {
226     __WRITE_VDP_REG(VDP_RSCX, __READ_VDP_REG(VDP_RSCX) - x);
227     int16_t tmp = __READ_VDP_REG(VDP_RSCY) + y;
228     __WRITE_VDP_REG(VDP_RSCY, (tmp < 0) ? 224 + tmp : tmp % 224u);
229 }
230
231 void vsync(void) PRESERVES_REGS(b, c, d, e, h, l, iyh, iyl);
232
233 void wait_vbl_done(void) PRESERVES_REGS(b, c, d, e, h, l, iyh, iyl);
234
235 inline void display_off(void) {
236     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) &= (~R1_DISP_ON));
237 }
238
239 #define DISPLAY_ON \
240     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) |= R1_DISP_ON)
241
242 #define DISPLAY_OFF \
243     display_off();
244
245 void refresh_OAM(void);
246
247 #define HIDE_LEFT_COLUMN \
248     __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) |= R0_LCB)
249
250 #define SHOW_LEFT_COLUMN \
251     __WRITE_VDP_REG(VDP_R0, __READ_VDP_REG(VDP_R0) &= (~R0_LCB))
252
253 #define SET_BORDER_COLOR(C) __WRITE_VDP_REG(VDP_R7, ((C) | 0xf0u))
254
255 #define SHOW_BKG
256
257 #define HIDE_BKG
258
259 #define SHOW_WIN
260
261 #define HIDE_WIN
262
263 #define SHOW_SPRITES \
264     (_sprites_OFF = 0)
265
266 #define HIDE_SPRITES \
267     (_sprites_OFF = 1)
268
269 #define SPRITES_8x16 \
270     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) |= R1_SPR_8X16)
271
272 #define SPRITES_8x8 \
273     __WRITE_VDP_REG(VDP_R1, __READ_VDP_REG(VDP_R1) &= (~R1_SPR_8X16))
274
275 #define DEVICE_SUPPORTS_COLOR (TRUE)
276
277 extern volatile uint16_t sys_time;
278
279 extern volatile uint8_t _vbl_done;
280 #define VBL_DONE _vbl_done
281
282 uint8_t get_r_reg(void) PRESERVES_REGS(b, c, d, e, h, l, iyh, iyl);
283
284 #define DIV_REG get_r_reg()
285
286 #define _current_bank MAP_FRAME1
287 #define CURRENT_BANK MAP_FRAME1
288
289 #ifndef BANK
290 #define BANK(VARNAME) ( (uint8_t) &__bank_ ## VARNAME )
291 #endif
292
293 #define BANKREF(VARNAME) void __func_ ## VARNAME(void) __banked __naked { \
294     __asm \
295     .local b__func_ ## VARNAME \
296     __bank_ ## VARNAME = b__func_ ## VARNAME \
297     .globl __bank_ ## VARNAME \
298     __endasm; \
299 }
300
301

```

```

407 #define BANKREF_EXTERN(VARNAME) extern const void __bank_ ## VARNAME;
408
409
414 #define SWITCH_ROM(b) MAP_FRAME1=(b)
415 #define SWITCH_ROM1 SWITCH_ROM
416
421 #define SWITCH_ROM2(b) MAP_FRAME2=(b)
422
427 #define SWITCH_RAM(b) RAM_CONTROL=( (b)&1)?RAM_CONTROL|RAMCTL_BANK:RAM_CONTROL&(~RAMCTL_BANK)
428
432 #define ENABLE_RAM RAM_CONTROL|=RAMCTL_RAM
433
437 #define DISABLE_RAM RAM_CONTROL&=~RAMCTL_RAM
438
439
444 void delay(uint16_t d) Z88DK_FASTCALL;
445
446
449 uint8_t joypad(void) OLDCALL PRESERVES_REGS(b, c, d, e, iyh, iyl);
450
453 uint8_t waitpad(uint8_t mask) Z88DK_FASTCALL PRESERVES_REGS(d, e, iyh, iyl);
454
460 void waitpadup(void) PRESERVES_REGS(d, e, iyh, iyl);
461
467 typedef struct {
468     uint8_t npads;
469     union {
470         struct {
471             uint8_t joy0, joy1, joy2, joy3;
472         };
473         uint8_t joypads[4];
474     };
475 } joypads_t;
476
485 uint8_t joypad_init(uint8_t npads, joypads_t * joypads) Z88DK_CALLEE;
486
493 void joypad_ex(joypads_t * joypads) Z88DK_FASTCALL PRESERVES_REGS(iyh, iyl);
494
503 inline void enable_interrupts(void) PRESERVES_REGS(a, b, c, d, e, h, l, iyh, iyl) {
504     __asm__("ei");
505 }
506
519 inline void disable_interrupts(void) PRESERVES_REGS(a, b, c, d, e, h, l, iyh, iyl) {
520     __asm__("di");
521 }
522
523
524 #if defined(__TARGET_sms)
525
526 #define RGB(r,g,b) ((r) | ((g) << 2) | ((b) << 4))
527 #define RGB8(r,g,b) (((r) >> 6) | (((g) >> 6) << 2) | (((b) >> 6) << 4))
528 #define RGBHTML(RGB24bit) (((RGB24bit) >> 22) | (((RGB24bit) & 0xFFFF) >> 14) << 2) | (((RGB24bit) &
    0xFF) >> 6) << 4))
529
532 #define RGB_RED RGB( 3, 0, 0)
533 #define RGB_DARKRED RGB( 2, 0, 0)
534 #define RGB_GREEN RGB( 0, 3, 0)
535 #define RGB_DARKGREEN RGB( 0, 2, 0)
536 #define RGB_BLUE RGB( 0, 0, 3)
537 #define RGB_DARKBLUE RGB( 0, 0, 2)
538 #define RGB_YELLOW RGB( 3, 3, 0)
539 #define RGB_DARKYELLOW RGB( 2, 2, 0)
540 #define RGB_CYAN RGB( 0, 3, 3)
541 #define RGB_AQUA RGB( 3, 1, 2)
542 #define RGB_PINK RGB( 3, 0, 3)
543 #define RGB_PURPLE RGB( 2, 0, 2)
544 #define RGB_BLACK RGB( 0, 0, 0)
545 #define RGB_DARKGRAY RGB( 1, 1, 1)
546 #define RGB_LIGHTGRAY RGB( 2, 2, 2)
547 #define RGB_WHITE RGB( 3, 3, 3)
548
549 typedef uint8_t palette_color_t;
550
551 #elif defined(__TARGET_gg)
552
553 #define RGB(r,g,b) ((uint16_t)(r) | (uint16_t)((g) << 4) | (uint16_t)((b) << 8))
554 #define RGB8(r,g,b) ((uint16_t)((r) >> 4) | ((uint16_t)((g) >> 4) << 4) | ((uint16_t)((b) >> 4) << 8))
555 #define RGBHTML(RGB24bit) (((RGB24bit) >> 20) | (((RGB24bit) & 0xFFFF) >> 12) << 4) | (((RGB24bit) & 0xFF)
    >> 4) << 8))
556
559 #define RGB_RED RGB(15, 0, 0)
560 #define RGB_DARKRED RGB( 7, 0, 0)
561 #define RGB_GREEN RGB( 0, 15, 0)
562 #define RGB_DARKGREEN RGB( 0, 7, 0)
563 #define RGB_BLUE RGB( 0, 0, 15)
564 #define RGB_DARKBLUE RGB( 0, 0, 7)
565 #define RGB_YELLOW RGB(15, 15, 0)

```

```

566 #define RGB_DARKYELLOW RGB( 7, 7, 0)
567 #define RGB_CYAN       RGB( 0, 15, 15)
568 #define RGB_AQUA       RGB(14, 2, 11)
569 #define RGB_PINK       RGB(15, 0, 15)
570 #define RGB_PURPLE     RGB(10, 0, 10)
571 #define RGB_BLACK      RGB( 0, 0, 0)
572 #define RGB_DARKGRAY   RGB( 5, 5, 5)
573 #define RGB_LIGHTGRAY  RGB(10, 10, 10)
574 #define RGB_WHITE      RGB(15, 15, 15)
575
576 #define RGB_LIGHTFLESH RGB(15, 10, 7)
577 #define RGB_BROWN      RGB( 5, 5, 0)
578 #define RGB_ORANGE     RGB(15, 10, 0)
579 #define RGB_TEAL       RGB( 7, 7, 0)
580
581 typedef uint16_t palette_color_t;
582
583 #else
584 #error Unrecognized port
585 #endif
586
587 void set_default_palette(void);
588 inline void cgb_compatibility(void) {
589     set_default_palette();
590 }
591
592 inline void cpu_fast(void) {}
593
594 void set_palette_entry(uint8_t palette, uint8_t entry, uint16_t rgb_data) Z88DK_CALLEE
    PRESERVES_REGS(iyh, iyl);
595 #define set_bkg_palette_entry set_palette_entry
596 #define set_sprite_palette_entry(palette,entry,rgb_data) set_palette_entry(1,entry,rgb_data)
597
598
623 void set_palette(uint8_t first_palette, uint8_t nb_palettes, const palette_color_t *rgb_data)
    Z88DK_CALLEE;
624 #define set_bkg_palette set_palette
625 #define set_sprite_palette(first_palette,nb_palettes,rgb_data) set_palette(1,1,rgb_data)
626
627 void set_native_tile_data(uint16_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh, iyl);
628 void set_bkg_4bpp_data(uint16_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh, iyl);
629 void set_bkg_native_data(uint16_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh, iyl);
630
631 void set_sprite_4bpp_data(uint8_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh, iyl);
632 void set_sprite_native_data(uint8_t start, uint16_t ntiles, const void *src) PRESERVES_REGS(iyh, iyl);
633
634 #define COMPAT_PALETTE(C0,C1,C2,C3) (((uint16_t)(C3) < 12) | ((uint16_t)(C2) < 8) | ((uint16_t)(C1) < 4)
    | (uint16_t)(C0))
635 extern uint16_t _current_2bpp_palette;
636 inline void set_2bpp_palette(uint16_t palette) {
637     _current_2bpp_palette = palette;
638 }
639 void set_tile_2bpp_data(uint16_t start, uint16_t ntiles, const void *src, uint16_t palette) Z88DK_CALLEE
    PRESERVES_REGS(iyh, iyl);
640 inline void set_bkg_data(uint16_t start, uint16_t ntiles, const void *src) {
641     set_tile_2bpp_data(start, ntiles, src, _current_2bpp_palette);
642 }
643 inline void set_sprite_data(uint16_t start, uint16_t ntiles, const void *src) {
644     set_tile_2bpp_data((uint8_t)(start) + 0x100u, ntiles, src, _current_2bpp_palette);
645 }
646 inline void set_bkg_2bpp_data(uint16_t start, uint16_t ntiles, const void *src) {
647     set_tile_2bpp_data(start, ntiles, src, _current_2bpp_palette);
648 }
649 inline void set_sprite_2bpp_data(uint16_t start, uint16_t ntiles, const void *src) {
650     set_tile_2bpp_data((uint8_t)(start) + 0x100u, ntiles, src, _current_2bpp_palette);
651 }
652
653 extern uint16_t _current_1bpp_colors;
654 inline void set_1bpp_colors(uint8_t fgcolor, uint8_t bgcolor) {
655     _current_1bpp_colors = ((uint16_t)bgcolor < 8) | fgcolor;
656 }
657 void set_tile_1bpp_data(uint16_t start, uint16_t ntiles, const void *src, uint16_t colors) Z88DK_CALLEE
    PRESERVES_REGS(iyh, iyl);
658 inline void set_bkg_1bpp_data(uint16_t start, uint16_t ntiles, const void *src) {
659     set_tile_1bpp_data(start, ntiles, src, _current_1bpp_colors);
660 }
661 inline void set_sprite_1bpp_data(uint16_t start, uint16_t ntiles, const void *src) {
662     set_tile_1bpp_data((uint8_t)(start) + 0x100u, ntiles, src, _current_1bpp_colors);
663 }
664
665
674 void set_data(uint16_t dst, const void *src, uint16_t size) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
675 void vmemcpy(uint16_t dst, const void *src, uint16_t size) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
676
677 void set_tile_map(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) Z88DK_CALLEE;
678 void set_tile_map_compat(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles) Z88DK_CALLEE;
679 #define set_bkg_tiles set_tile_map_compat

```

```

680 #define set_win_tiles set_tile_map_compat
681
682 extern uint8_t _map_tile_offset;
683 inline void set_bkg_based_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles,
    uint8_t base_tile) {
684     _map_tile_offset = base_tile;
685     set_tile_map_compat(x, y, w, h, tiles);
686     _map_tile_offset = 0;
687 }
688 inline void set_win_based_tiles(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles,
    uint8_t base_tile) {
689     _map_tile_offset = base_tile;
690     set_tile_map_compat(x, y, w, h, tiles);
691     _map_tile_offset = 0;
692 }
693
694 inline void set_bkg_attributes(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *tiles)
695 {
696     VBK_REG = VBK_ATTRIBUTES;
697     set_bkg_tiles(x, y, w, h, tiles);
698     VBK_REG = VBK_TILES;
699 }
700
701 void set_tile_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t map_w)
    Z88DK_CALLEE;
702 void set_tile_submap_compat(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
    map_w) Z88DK_CALLEE;
703 #define set_bkg_submap set_tile_submap_compat
704 #define set_win_submap set_tile_submap_compat
705
706 extern uint8_t _submap_tile_offset;
707 inline void set_bkg_based_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
    map_w, uint8_t base_tile) {
708     _submap_tile_offset = base_tile;
709     set_tile_submap_compat(x, y, w, h, map, map_w);
710     _submap_tile_offset = 0;
711 }
712 inline void set_win_based_submap(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map, uint8_t
    map_w, uint8_t base_tile) {
713     _submap_tile_offset = base_tile;
714     set_tile_submap_compat(x, y, w, h, map, map_w);
715     _submap_tile_offset = 0;
716 }
717
718 inline void set_bkg_submap_attributes(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint8_t *map,
    uint8_t map_w) {
719     VBK_REG = VBK_ATTRIBUTES;
720     set_tile_submap_compat(x, y, w, h, map, map_w);
721     VBK_REG = VBK_TILES;
722 }
723
724 void fill_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint16_t tile) Z88DK_CALLEE;
725 void fill_rect_compat(uint8_t x, uint8_t y, uint8_t w, uint8_t h, const uint16_t tile) Z88DK_CALLEE;
726 #define fill_bkg_rect fill_rect_compat
727 #define fill_win_rect fill_rect_compat
728
729 extern volatile uint8_t shadow_OAM[];
730
731 extern volatile uint8_t _shadow_OAM_base;
732
733 extern volatile uint8_t _shadow_OAM_OFF;
734
735 extern volatile uint8_t _sprites_OFF;
736
737 #define DISABLE_VBL_TRANSFER \
738     _shadow_OAM_base = 0
739
740 #define ENABLE_VBL_TRANSFER \
741     _shadow_OAM_base = (uint8_t)((uint16_t)&shadow_OAM >> 8)
742
743 #define MAX_HARDWARE_SPRITES 64
744
745 #define HARDWARE_SPRITE_CAN_FLIP_X 0
746
747 #define HARDWARE_SPRITE_CAN_FLIP_Y 0
748
749 inline void SET_SHADOW_OAM_ADDRESS(void * address) {
750     _shadow_OAM_base = (uint8_t)((uint16_t)address >> 8);
751 }
752
753 inline void set_sprite_tile(uint8_t nb, uint8_t tile) {
754     shadow_OAM[0x41+(nb << 1)] = tile;
755 }
756
757 inline uint8_t get_sprite_tile(uint8_t nb) {
758     return shadow_OAM[0x41+(nb << 1)];
759 }

```



```

822 }
823
829 inline void set_sprite_prop(uint8_t nb, uint8_t prop) {
830     nb; prop;
831 }
832
833 inline uint8_t get_sprite_prop(uint8_t nb) {
834     nb;
835     return 0;
836 }
837
850 inline void move_sprite(uint8_t nb, uint8_t x, uint8_t y) {
851     shadow_OAM[nb] = (y < VDP_SAT_TERM) ? y : 0xC0;
852     shadow_OAM[0x40+(nb << 1)] = x;
853 }
854
855
866 inline void scroll_sprite(uint8_t nb, int8_t x, int8_t y) {
867     uint8_t new_y = shadow_OAM[nb] + y;
868     shadow_OAM[nb] = (new_y < VDP_SAT_TERM) ? new_y : 0xC0;
869     shadow_OAM[0x40+(nb << 1)] += x;
870 }
871
872
877 inline void hide_sprite(uint8_t nb) {
878     shadow_OAM[nb] = 0xC0;
879 }
880
887 void set_vram_byte(uint8_t * addr, uint8_t v) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
888
896 uint8_t * set_attributed_tile_xy(uint8_t x, uint8_t y, uint16_t t) Z88DK_CALLEE PRESERVES_REGS(iyh,
    iyl);
897
905 uint8_t * set_tile_xy(uint8_t x, uint8_t y, uint8_t t) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
906 #define set_bkg_tile_xy set_tile_xy
907 #define set_win_tile_xy set_tile_xy
908
916 inline uint8_t * set_attribute_xy(uint8_t x, uint8_t y, uint8_t a) Z88DK_CALLEE PRESERVES_REGS(iyh,
    iyl);
917 #define set_bkg_attribute_xy set_attribute_xy
918 #define set_win_attribute_xy set_attribute_xy
919
923 uint8_t * get_bkg_xy_addr(uint8_t x, uint8_t y) Z88DK_CALLEE PRESERVES_REGS(iyh, iyl);
924 #define get_win_xy_addr get_bkg_xy_addr
925
926 #endif /* _SMS_H */

```

20.122 gbdk-lib/include/stdatomic.h File Reference

```
#include <types.h>
```

Data Structures

- struct [atomic_flag](#)

Functions

- `_Bool` [atomic_flag_test_and_set](#) (volatile [atomic_flag](#) *object) `OLDCALL`
- void [atomic_flag_clear](#) (volatile [atomic_flag](#) *object)

20.122.1 Function Documentation

20.122.1.1 [atomic_flag_test_and_set\(\)](#) `_Bool` [atomic_flag_test_and_set](#) (
 volatile [atomic_flag](#) * *object*)

20.122.1.2 [atomic_flag_clear\(\)](#) void [atomic_flag_clear](#) (
 volatile [atomic_flag](#) * *object*)

20.123 stdatomic.h

[Go to the documentation of this file.](#)

```

1 #ifndef __SDCC_STDATOMIC_H
2 #define __SDCC_STDATOMIC_H 1
3
4 #include <types.h>
5
6 typedef struct {unsigned char flag;} atomic_flag;
7
8 #if defined(__SDCC_z80) || defined(__SDCC_z180) || defined(__SDCC_ez80_z80) || defined(__SDCC_sm83) ||
    defined(__SDCC_r2k) || defined(__SDCC_r3ka) || defined(__SDCC_stm8) || defined(__SDCC_hc08) ||
    defined(__SDCC_s08) || defined(__SDCC_mos6502)
9 #define ATOMIC_FLAG_INIT {1}
10 //elif defined(__SDCC_mcs51)
11 //define ATOMIC_FLAG_INIT {0}
12 #else
13 #error Support for atomic_flag not implemented
14 #endif
15
16 _Bool atomic_flag_test_and_set(volatile atomic_flag *object) OLDCALL;
17
18 void atomic_flag_clear(volatile atomic_flag *object);
19
20 #endif
21
```

20.124 gbdk-lib/include/stdbool.h File Reference

Macros

- #define [true](#) ((_Bool)+1)
- #define [false](#) ((_Bool)+0)
- #define [bool](#) _Bool
- #define [__bool_true_false_are_defined](#) 1

20.124.1 Macro Definition Documentation

20.124.1.1 true #define true ((_Bool)+1)

20.124.1.2 false #define false ((_Bool)+0)

20.124.1.3 bool #define bool _Bool

20.124.1.4 __bool_true_false_are_defined #define __bool_true_false_are_defined 1

20.125 stdbool.h

[Go to the documentation of this file.](#)

```

1 /*-----
2     stdbool.h - ANSI functions forward declarations
3
4     Copyright (C) 2004, Maarten Brock, sourceforge.brock@dse.nl
5
6     This library is free software; you can redistribute it and/or modify it
7     under the terms of the GNU General Public License as published by the
8     Free Software Foundation; either version 2, or (at your option) any
9     later version.
10
11     This library is distributed in the hope that it will be useful,
12     but WITHOUT ANY WARRANTY; without even the implied warranty of
13     MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14     GNU General Public License for more details.
15
16     You should have received a copy of the GNU General Public License
17     along with this library; see the file COPYING. If not, write to the
18     Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,

```

```

19  MA 02110-1301, USA.
20
21  As a special exception, if you link this library with other files,
22  some of which are compiled with SDCC, to produce an executable,
23  this library does not by itself cause the resulting executable to
24  be covered by the GNU General Public License. This exception does
25  not however invalidate any other reasons why the executable file
26  might be covered by the GNU General Public License.
27  -----*/
28
29  #ifndef __SDC51_STDBOOL_H
30  #define __SDC51_STDBOOL_H 1
31
32  /* Define true and false of type _Bool in a way compatible with the preprocessor (see N 2229 for
   details). */
33  #define true ((_Bool)+1)
34  #define false ((_Bool)+0)
35
36  #define bool _Bool
37  #define __bool_true_false_are_defined 1
38
39  #endif
40

```

20.126 gbdk-lib/include/stddef.h File Reference

Macros

- `#define NULL (void *)0`
- `#define __PTRDIFF_T_DEFINED`
- `#define __SIZE_T_DEFINED`
- `#define __WCHAR_T_DEFINED`
- `#define offsetof(s, m) __builtin_offsetof (s, m)`

Typedefs

- `typedef int ptrdiff\_t`
- `typedef unsigned int size\_t`
- `typedef unsigned long int wchar\_t`

20.126.1 Macro Definition Documentation

20.126.1.1 NULL `#define NULL (void *)0`

20.126.1.2 __PTRDIFF_T_DEFINED `#define __PTRDIFF_T_DEFINED`

20.126.1.3 __SIZE_T_DEFINED `#define __SIZE_T_DEFINED`

20.126.1.4 __WCHAR_T_DEFINED `#define __WCHAR_T_DEFINED`

20.126.1.5 offsetof `#define offsetof(`
`s,`
`m ) __builtin_offsetof (s, m)`

20.126.2 Typedef Documentation

20.126.2.1 ptrdiff_t typedef int `ptrdiff_t`

20.126.2.2 size_t typedef unsigned int `size_t`

20.126.2.3 wchar_t typedef unsigned long int `wchar_t`

20.127 stddef.h

[Go to the documentation of this file.](#)

```

1  /*-----
2      stddef.h - ANSI functions forward declarations
3
4      Copyright (C) 2004, Maarten Brock / sourceforge.brock@dse.nl
5      Copyright (C) 2011, Philipp Klaus Krause / pkk@spth.de
6
7      This library is free software; you can redistribute it and/or modify it
8      under the terms of the GNU General Public License as published by the
9      Free Software Foundation; either version 2, or (at your option) any
10     later version.
11
12     This library is distributed in the hope that it will be useful,
13     but WITHOUT ANY WARRANTY; without even the implied warranty of
14     MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
15     GNU General Public License for more details.
16
17     You should have received a copy of the GNU General Public License
18     along with this library; see the file COPYING. If not, write to the
19     Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,
20     MA 02110-1301, USA.
21
22     As a special exception, if you link this library with other files,
23     some of which are compiled with SDCC, to produce an executable,
24     this library does not by itself cause the resulting executable to
25     be covered by the GNU General Public License. This exception does
26     not however invalidate any other reasons why the executable file
27     might be covered by the GNU General Public License.
28  -----*/
29
30 #ifndef __SDCC_STDDEF_H
31 #define __SDCC_STDDEF_H 1
32
33 #ifndef NULL
34     #define NULL (void *)0
35 #endif
36
37 #ifndef __PTRDIFF_T_DEFINED
38 #define __PTRDIFF_T_DEFINED
39 #if defined (__SDCC_mcs51) || defined (__SDCC_ds390)
40     typedef long int ptrdiff_t;
41 #else
42     typedef int ptrdiff_t;
43 #endif
44 #endif
45
46 #ifndef __SIZE_T_DEFINED
47 #define __SIZE_T_DEFINED
48     typedef unsigned int size_t;
49 #endif
50
51 #if __STDC_VERSION__ >= 201112L
52     typedef unsigned char max_align_t;
53 #endif
54
55 #ifndef __WCHAR_T_DEFINED
56 #define __WCHAR_T_DEFINED
57     typedef unsigned long int wchar_t;
58 #endif
59
60 /* Bounds-checking interfaces from annex K of the C11 standard. */
61 #if defined (__STDC_WANT_LIB_EXT1__) && __STDC_WANT_LIB_EXT1__
62
63 #ifndef __RSIZE_T_DEFINED
64 #define __RSIZE_T_DEFINED
65     typedef size_t rsize_t;
66 #endif
67
68 #ifndef __ERRNO_T_DEFINED
69 #define __ERRNO_T_DEFINED
70     typedef int errno_t;

```

```

71 #endif
72
73 #endif
74
75 #define offsetof(s, m) __builtin_offsetof (s, m)
76
77 #endif
78

```

20.128 gbdk-lib/include/stdint.h File Reference

Macros

- `#define INT8_MIN` (-128)
- `#define INT16_MIN` (-32767-1)
- `#define INT32_MIN` (-2147483647L-1)
- `#define INT8_MAX` (127)
- `#define INT16_MAX` (32767)
- `#define INT32_MAX` (2147483647L)
- `#define UINT8_MAX` (255)
- `#define UINT16_MAX` (65535)
- `#define UINT32_MAX` (4294967295UL)
- `#define INT_LEAST8_MIN` INT8_MIN
- `#define INT_LEAST16_MIN` INT16_MIN
- `#define INT_LEAST32_MIN` INT32_MIN
- `#define INT_LEAST8_MAX` INT8_MAX
- `#define INT_LEAST16_MAX` INT16_MAX
- `#define INT_LEAST32_MAX` INT32_MAX
- `#define UINT_LEAST8_MAX` UINT8_MAX
- `#define UINT_LEAST16_MAX` UINT16_MAX
- `#define UINT_LEAST32_MAX` UINT32_MAX
- `#define INT_FAST8_MIN` INT8_MIN
- `#define INT_FAST16_MIN` INT16_MIN
- `#define INT_FAST32_MIN` INT32_MIN
- `#define INT_FAST8_MAX` INT8_MAX
- `#define INT_FAST16_MAX` INT16_MAX
- `#define INT_FAST32_MAX` INT32_MAX
- `#define UINT_FAST8_MAX` UINT8_MAX
- `#define UINT_FAST16_MAX` UINT16_MAX
- `#define UINT_FAST32_MAX` UINT32_MAX
- `#define INTPTR_MIN` (-32767-1)
- `#define INTPTR_MAX` (32767)
- `#define UINTPTR_MAX` (65535)
- `#define INTMAX_MIN` (-2147483647L-1)
- `#define INTMAX_MAX` (2147483647L)
- `#define UINTMAX_MAX` (4294967295UL)
- `#define PTRDIFF_MIN` (-32767-1)
- `#define PTRDIFF_MAX` (32767)
- `#define SIG_ATOMIC_MIN` (0)
- `#define SIG_ATOMIC_MAX` (255)
- `#define SIZE_MAX` (65535u)
- `#define INT8_C(c)` c
- `#define INT16_C(c)` c
- `#define INT32_C(c)` c ## L
- `#define UINT8_C(c)` c ## U
- `#define UINT16_C(c)` c ## U
- `#define UINT32_C(c)` c ## UL
- `#define WCHAR_MIN` 0

- `#define WCHAR_MAX 0xffffffff`
- `#define WINT_MIN 0`
- `#define WINT_MAX 0xffffffff`
- `#define INTMAX_C(c) c ## L`
- `#define UINTMAX_C(c) c ## UL`

Typedefs

- `typedef signed char int8_t`
- `typedef short int int16_t`
- `typedef long int int32_t`
- `typedef unsigned char uint8_t`
- `typedef unsigned short int uint16_t`
- `typedef unsigned long int uint32_t`
- `typedef signed char int_least8_t`
- `typedef short int int_least16_t`
- `typedef long int int_least32_t`
- `typedef unsigned char uint_least8_t`
- `typedef unsigned short int uint_least16_t`
- `typedef unsigned long int uint_least32_t`
- `typedef signed char int_fast8_t`
- `typedef int int_fast16_t`
- `typedef long int int_fast32_t`
- `typedef unsigned char uint_fast8_t`
- `typedef unsigned int uint_fast16_t`
- `typedef unsigned long int uint_fast32_t`
- `typedef int intptr_t`
- `typedef unsigned int uintptr_t`
- `typedef long int intmax_t`
- `typedef unsigned long int uintmax_t`

20.128.1 Macro Definition Documentation

20.128.1.1 INT8_MIN `#define INT8_MIN (-128)`

20.128.1.2 INT16_MIN `#define INT16_MIN (-32767-1)`

20.128.1.3 INT32_MIN `#define INT32_MIN (-2147483647L-1)`

20.128.1.4 INT8_MAX `#define INT8_MAX (127)`

20.128.1.5 INT16_MAX `#define INT16_MAX (32767)`

20.128.1.6 INT32_MAX `#define INT32_MAX (2147483647L)`

20.128.1.7 UINT8_MAX `#define UINT8_MAX (255)`

20.128.1.8 **UINT16_MAX** `#define UINT16_MAX (65535)`

20.128.1.9 **UINT32_MAX** `#define UINT32_MAX (4294967295UL)`

20.128.1.10 **INT_LEAST8_MIN** `#define INT_LEAST8_MIN INT8_MIN`

20.128.1.11 **INT_LEAST16_MIN** `#define INT_LEAST16_MIN INT16_MIN`

20.128.1.12 **INT_LEAST32_MIN** `#define INT_LEAST32_MIN INT32_MIN`

20.128.1.13 **INT_LEAST8_MAX** `#define INT_LEAST8_MAX INT8_MAX`

20.128.1.14 **INT_LEAST16_MAX** `#define INT_LEAST16_MAX INT16_MAX`

20.128.1.15 **INT_LEAST32_MAX** `#define INT_LEAST32_MAX INT32_MAX`

20.128.1.16 **UINT_LEAST8_MAX** `#define UINT_LEAST8_MAX UINT8_MAX`

20.128.1.17 **UINT_LEAST16_MAX** `#define UINT_LEAST16_MAX UINT16_MAX`

20.128.1.18 **UINT_LEAST32_MAX** `#define UINT_LEAST32_MAX UINT32_MAX`

20.128.1.19 **INT_FAST8_MIN** `#define INT_FAST8_MIN INT8_MIN`

20.128.1.20 **INT_FAST16_MIN** `#define INT_FAST16_MIN INT16_MIN`

20.128.1.21 **INT_FAST32_MIN** `#define INT_FAST32_MIN INT32_MIN`

20.128.1.22 **INT_FAST8_MAX** `#define INT_FAST8_MAX INT8_MAX`

20.128.1.23 **INT_FAST16_MAX** `#define INT_FAST16_MAX INT16_MAX`

20.128.1.24 **INT_FAST32_MAX** `#define INT_FAST32_MAX INT32_MAX`

20.128.1.25 **UINT_FAST8_MAX** `#define UINT_FAST8_MAX UINT8_MAX`

20.128.1.26 **UINT_FAST16_MAX** `#define UINT_FAST16_MAX UINT16\_MAX`

20.128.1.27 **UINT_FAST32_MAX** `#define UINT_FAST32_MAX UINT32\_MAX`

20.128.1.28 **INTPTR_MIN** `#define INTPTR_MIN (-32767-1)`

20.128.1.29 **INTPTR_MAX** `#define INTPTR_MAX (32767)`

20.128.1.30 **UINTPTR_MAX** `#define UINTPTR_MAX (65535)`

20.128.1.31 **INTMAX_MIN** `#define INTMAX_MIN (-2147483647L-1)`

20.128.1.32 **INTMAX_MAX** `#define INTMAX_MAX (2147483647L)`

20.128.1.33 **UINTMAX_MAX** `#define UINTMAX_MAX (4294967295UL)`

20.128.1.34 **PTRDIFF_MIN** `#define PTRDIFF_MIN (-32767-1)`

20.128.1.35 **PTRDIFF_MAX** `#define PTRDIFF_MAX (32767)`

20.128.1.36 **SIG_ATOMIC_MIN** `#define SIG_ATOMIC_MIN (0)`

20.128.1.37 **SIG_ATOMIC_MAX** `#define SIG_ATOMIC_MAX (255)`

20.128.1.38 **SIZE_MAX** `#define SIZE_MAX (65535u)`

20.128.1.39 **INT8_C** `#define INT8_C(
 c) c`

20.128.1.40 **INT16_C** `#define INT16_C(
 c) c`

20.128.1.41 **INT32_C** `#define INT32_C(
 c) c ## L`

20.128.1.42 **UINT8_C** `#define UINT8_C(
 c) c ## U`

20.128.1.43 **UINT16_C** `#define UINT16_C(
 c) c ## U`

20.128.1.44 **UINT32_C** `#define UINT32_C(
 c) c ## UL`

20.128.1.45 **WCHAR_MIN** `#define WCHAR_MIN 0`

20.128.1.46 **WCHAR_MAX** `#define WCHAR_MAX 0xffffffff`

20.128.1.47 **WINT_MIN** `#define WINT_MIN 0`

20.128.1.48 **WINT_MAX** `#define WINT_MAX 0xffffffff`

20.128.1.49 **INTMAX_C** `#define INTMAX_C(
 c) c ## L`

20.128.1.50 **UINTMAX_C** `#define UINTMAX_C(
 c) c ## UL`

20.128.2 Typedef Documentation

20.128.2.1 **int8_t** `typedef signed char int8_t`

20.128.2.2 **int16_t** `typedef short int int16_t`

20.128.2.3 **int32_t** `typedef long int int32_t`

20.128.2.4 **uint8_t** `typedef unsigned char uint8_t`

20.128.2.5 **uint16_t** `typedef unsigned short int uint16_t`

20.128.2.6 **uint32_t** `typedef unsigned long int uint32_t`

20.128.2.7 **int_least8_t** `typedef signed char int_least8_t`

20.128.2.8 **int_least16_t** `typedef short int int_least16_t`

20.128.2.9 `int_least32_t` typedef long int `int_least32_t`

20.128.2.10 `uint_least8_t` typedef unsigned char `uint_least8_t`

20.128.2.11 `uint_least16_t` typedef unsigned short int `uint_least16_t`

20.128.2.12 `uint_least32_t` typedef unsigned long int `uint_least32_t`

20.128.2.13 `int_fast8_t` typedef signed char `int_fast8_t`

20.128.2.14 `int_fast16_t` typedef int `int_fast16_t`

20.128.2.15 `int_fast32_t` typedef long int `int_fast32_t`

20.128.2.16 `uint_fast8_t` typedef unsigned char `uint_fast8_t`

20.128.2.17 `uint_fast16_t` typedef unsigned int `uint_fast16_t`

20.128.2.18 `uint_fast32_t` typedef unsigned long int `uint_fast32_t`

20.128.2.19 `intptr_t` typedef int `intptr_t`

20.128.2.20 `uintptr_t` typedef unsigned int `uintptr_t`

20.128.2.21 `intmax_t` typedef long int `intmax_t`

20.128.2.22 `uintmax_t` typedef unsigned long int `uintmax_t`

20.129 stdint.h

[Go to the documentation of this file.](#)

```
1 /*-----
2  stdint.h - ISO C99 7.18 Integer types <stdint.h>
3
4  Copyright (C) 2005, Maarten Brock, sourceforge.brock@dse.nl
5  Copyright (C) 2011, Philipp Klaus Krause, pkk@spth.de
6
7  This library is free software; you can redistribute it and/or modify it
8  under the terms of the GNU General Public License as published by the
9  Free Software Foundation; either version 2, or (at your option) any
10  later version.
11
12  This library is distributed in the hope that it will be useful,
13  but WITHOUT ANY WARRANTY; without even the implied warranty of
14  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
15  GNU General Public License for more details.
16
```

```

17     You should have received a copy of the GNU General Public License
18     along with this library; see the file COPYING. If not, write to the
19     Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,
20     MA 02110-1301, USA.
21
22     As a special exception, if you link this library with other files,
23     some of which are compiled with SDCC, to produce an executable,
24     this library does not by itself cause the resulting executable to
25     be covered by the GNU General Public License. This exception does
26     not however invalidate any other reasons why the executable file
27     might be covered by the GNU General Public License.
28     -----*/
29
30 #ifndef _STDINT_H
31 #define _STDINT_H      1
32
33 /* Exact integral types. */
34
35 #if !defined(__SDCC_pic14) && !defined(__SDCC_pic16)
36 #if __STDC_VERSION__ >= 199901L
37 #define __SDCC_LONGLONG
38 #endif
39 #endif
40
41 /* Signed. */
42
43 typedef signed char      int8_t;
44 typedef short int       int16_t;
45 typedef long int        int32_t;
46 #ifdef __SDCC_LONGLONG
47 typedef long long int    int64_t;
48 #endif
49
50 /* Unsigned. */
51 typedef unsigned char    uint8_t;
52 typedef unsigned short int uint16_t;
53 typedef unsigned long int uint32_t;
54 #ifdef __SDCC_LONGLONG
55 typedef unsigned long long int uint64_t;
56 #endif
57
58 /* Small types. */
59
60 /* Signed. */
61 typedef signed char      int_least8_t;
62 typedef short int       int_least16_t;
63 typedef long int        int_least32_t;
64 #ifdef __SDCC_LONGLONG
65 typedef long long int    int_least64_t;
66 #endif
67
68 /* Unsigned. */
69 typedef unsigned char    uint_least8_t;
70 typedef unsigned short int uint_least16_t;
71 typedef unsigned long int uint_least32_t;
72 #ifdef __SDCC_LONGLONG
73 typedef unsigned long long int uint_least64_t;
74 #endif
75
76 /* Fast types. */
77
78 /* Signed. */
79 typedef signed char      int_fast8_t;
80 typedef int             int_fast16_t;
81 typedef long int        int_fast32_t;
82 #ifdef __SDCC_LONGLONG
83 typedef long long int    int_fast64_t;
84 #endif
85
86 /* Unsigned. */
87 typedef unsigned char    uint_fast8_t;
88 typedef unsigned int     uint_fast16_t;
89 typedef unsigned long int uint_fast32_t;
90 #ifdef __SDCC_LONGLONG
91 typedef unsigned long long int uint_fast64_t;
92 #endif
93
94 /* Types for 'void *' pointers. */
95 #if defined (__SDCC_mcs51) || defined (__SDCC_ds390)
96 typedef long int         intptr_t;
97 typedef unsigned long int uintptr_t;
98 #else
99 typedef int             intptr_t;
100 typedef unsigned int     uintptr_t;
101 #endif
102
103

```

```

104 /* Largest integral types. */
105 #ifndef __SDCC_LONGLONG
106 typedef long int          intmax_t;
107 typedef unsigned long int uintmax_t;
108 #else
109 typedef long long int      intmax_t;
110 typedef unsigned long long int uintmax_t;
111 #endif
112
113 /* Limits of integral types. */
114
115 /* Minimum of signed integral types. */
116 #define INT8_MIN      (-128)
117 #define INT16_MIN     (-32767-1)
118 #define INT32_MIN     (-2147483647L-1)
119 #ifdef __SDCC_LONGLONG
120 #define INT64_MIN     (-9223372036854775807LL-1)
121 #endif
122
123 /* Maximum of signed integral types. */
124 #define INT8_MAX      (127)
125 #define INT16_MAX     (32767)
126 #define INT32_MAX     (2147483647L)
127 #ifdef __SDCC_LONGLONG
128 #define INT64_MAX     (9223372036854775807LL)
129 #endif
130
131 /* Maximum of unsigned integral types. */
132 #define UINT8_MAX     (255)
133 #define UINT16_MAX    (65535)
134 #define UINT32_MAX    (4294967295UL)
135 #ifdef __SDCC_LONGLONG
136 #define UINT64_MAX    (18446744073709551615ULL)
137 #endif
138
139 /* Minimum of signed integral types having a minimum size. */
140 #define INT_LEAST8_MIN  INT8_MIN
141 #define INT_LEAST16_MIN INT16_MIN
142 #define INT_LEAST32_MIN INT32_MIN
143 #ifdef __SDCC_LONGLONG
144 #define INT_LEAST64_MIN INT64_MIN
145 #endif
146
147 /* Maximum of signed integral types having a minimum size. */
148 #define INT_LEAST8_MAX  INT8_MAX
149 #define INT_LEAST16_MAX INT16_MAX
150 #define INT_LEAST32_MAX INT32_MAX
151 #ifdef __SDCC_LONGLONG
152 #define INT_LEAST64_MAX INT64_MAX
153 #endif
154
155 /* Maximum of unsigned integral types having a minimum size. */
156 #define UINT_LEAST8_MAX  UINT8_MAX
157 #define UINT_LEAST16_MAX UINT16_MAX
158 #define UINT_LEAST32_MAX UINT32_MAX
159 #ifdef __SDCC_LONGLONG
160 #define UINT_LEAST64_MAX UINT64_MAX
161 #endif
162
163 /* Minimum of fast signed integral types having a minimum size. */
164 #define INT_FAST8_MIN  INT8_MIN
165 #define INT_FAST16_MIN INT16_MIN
166 #define INT_FAST32_MIN INT32_MIN
167 #ifdef __SDCC_LONGLONG
168 #define INT_FAST64_MIN INT64_MIN
169 #endif
170
171 /* Maximum of fast signed integral types having a minimum size. */
172 #define INT_FAST8_MAX  INT8_MAX
173 #define INT_FAST16_MAX INT16_MAX
174 #define INT_FAST32_MAX INT32_MAX
175 #ifdef __SDCC_LONGLONG
176 #define INT_FAST64_MAX INT64_MAX
177 #endif
178
179 /* Maximum of fast unsigned integral types having a minimum size. */
180 #define UINT_FAST8_MAX  UINT8_MAX
181 #define UINT_FAST16_MAX UINT16_MAX
182 #define UINT_FAST32_MAX UINT32_MAX
183 #ifdef __SDCC_LONGLONG
184 #define UINT_FAST64_MAX UINT64_MAX
185 #endif
186
187 /* Values to test for integral types holding 'void *' pointer. */
188 #if defined (__SDCC_mcs51) || defined (__SDCC_ds390)
189 #define INTPTR_MIN      (-2147483647L-1)
190 #define INTPTR_MAX      (2147483647L)

```

```

191 #define UINTPTR_MAX          (4294967295UL)
192 #else
193 #define INTPTR_MIN           (-32767-1)
194 #define INTPTR_MAX           (32767)
195 #define UINTPTR_MAX          (65535)
196 #endif
197
198 /* Minimum for largest signed integral type. */
199 #ifndef __SDCC_LONGLONG
200 #define INTMAX_MIN           (-2147483647L-1)
201 #else
202 #define INTMAX_MIN           (-9223372036854775807LL-1)
203 #endif
204
205 /* Maximum for largest signed integral type. */
206 #ifndef __SDCC_LONGLONG
207 #define INTMAX_MAX           (2147483647L)
208 #else
209 #define INTMAX_MAX           (9223372036854775807LL)
210 #endif
211
212 /* Maximum for largest unsigned integral type. */
213 #ifndef __SDCC_LONGLONG
214 #define UINTMAX_MAX          (4294967295UL)
215 #else
216 #define UINTMAX_MAX          (18446744073709551615ULL)
217 #endif
218
219 /* Limits of other integer types. */
220
221 /* Limits of 'ptrdiff_t' type. */
222 #if defined (__SDCC_mcs51) || defined (__SDCC_ds390)
223 #define PTRDIFF_MIN          (-2147483647L-1)
224 #define PTRDIFF_MAX          (2147483647L)
225 #else
226 #define PTRDIFF_MIN          (-32767-1)
227 #define PTRDIFF_MAX          (32767)
228 #endif
229
230 /* */
231 #define SIG_ATOMIC_MIN       (0)
232 #define SIG_ATOMIC_MAX       (255)
233
234 /* Limit of 'size_t' type. */
235 #define SIZE_MAX              (65535u)
236
237 /* Signed. */
238 #define INT8_C(c)             c
239 #define INT16_C(c)            c
240 #define INT32_C(c)            c ## L
241 #ifdef __SDCC_LONGLONG
242 #define INT64_C(c)            c ## LL
243 #endif
244
245 /* Unsigned. */
246 #define UINT8_C(c)            c ## U
247 #define UINT16_C(c)           c ## U
248 #define UINT32_C(c)           c ## UL
249 #ifdef __SDCC_LONGLONG
250 #define UINT64_C(c)           c ## ULL
251 #endif
252
253 #define WCHAR_MIN             0
254 #define WCHAR_MAX             0xffffffff
255
256 #define WINT_MIN              0
257 #define WINT_MAX              0xffffffff
258
259 /* Maximal type. */
260 #ifdef __SDCC_LONGLONG
261 #define INTMAX_C(c)           c ## LL
262 #define UINTMAX_C(c)          c ## ULL
263 #else
264 #define INTMAX_C(c)           c ## L
265 #define UINTMAX_C(c)          c ## UL
266 #endif
267
268 /* Bounds-checking interfaces from annex K of the C11 standard. */
269 #if defined (__STDC_WANT_LIB_EXT1__) && __STDC_WANT_LIB_EXT1__
270 #define RSIZE_MAX SIZE_MAX
271 #endif
272
273 #endif /* stdint.h */
274

```

20.130 gbdk-lib/include/stdio.h File Reference

```
#include <types.h>
```

Functions

- void [putchar](#) (char *c*) [OLDCALL](#) REENTRANT
- void [printf](#) (const char **format*,...)
- void [sprintf](#) (char **str*, const char **format*,...)
- void [puts](#) (const char **s*)
- char * [gets](#) (char **s*) [OLDCALL](#)
- char [getchar](#) (void) [OLDCALL](#)

20.130.1 Detailed Description

Basic file/console input output functions.

Including stdio.h will use a large number of the background tiles for font characters. If stdio.h is not included then that space will be available for use with other tiles instead.

20.130.2 Function Documentation

20.130.2.1 putchar() `void putchar (`
 `char c )`

Print char to stdout.

Parameters

<i>c</i>	Character to print
----------	--------------------

20.130.2.2 printf() `void printf (`
 `const char * format,`
 `... )`

Print the string and arguments given by *format* to stdout.

Parameters

<i>format</i>	The format string as per printf
---------------	---------------------------------

Does not return the number of characters printed.

Currently supported:

- %hx (char as hex)
- %hu (unsigned char)
- %hd (signed char)
- %c (character)
- %u (unsigned int)
- %d (signed int)
- %x (unsigned int as hex)
- %s (string)

Warning: to correctly pass parameters (such as chars, ints, etc) **all of them should always be explicitly cast** as when calling the function. See [docs_chars_varargs](#) for more details.

20.130.2.3 sprintf() `void sprintf (`
 `char * str,`
 `const char * format,`
 `... )`

Print the string and arguments given by format to a buffer.

Parameters

<i>str</i>	The buffer to print into
<i>format</i>	The format string as per printf

Does not return the number of characters printed.

Warning: to correctly pass parameters (such as chars, ints, etc) **all of them should always be explicitly cast** as when calling the function. See [docs_chars_varargs](#) for more details.

20.130.2.4 puts() `void puts (`
 `const char * s )`

[puts\(\)](#) writes the string **s** and a trailing newline to stdout.

20.130.2.5 gets() `char * gets (`
 `char * s )`

[gets\(\)](#) Reads a line from stdin into a buffer pointed to by **s**.

Parameters

<i>s</i>	Buffer to store string in
----------	---------------------------

Reads until either a terminating newline or an EOF, which it replaces with '\0'. No check for buffer overrun is performed.

Returns: Buffer pointed to by **s**

20.130.2.6 getchar() `char getchar (`
 `void )`

[getchar\(\)](#) Reads and returns a single character from stdin.

20.131 stdio.h

[Go to the documentation of this file.](#)

```

1
9 #ifndef STDIO_INCLUDE
10 #define STDIO_INCLUDE
11
12 #include <types.h>
13
18 void putchar(char c) OLDCALL REENTRANT;
19
41 void printf(const char *format, ...);
42
55 void sprintf(char *str, const char *format, ...);
56
59 void puts(const char *s);
60
70 char *gets(char *s) OLDCALL;
71
74 char getchar(void) OLDCALL;
75
76 #endif

```

20.132 gbdk-lib/include/stdlib.h File Reference

```
#include <types.h>
```

Functions

- void [exit](#) (int status) [OLDCALL](#)
- int [abs](#) (int i)
- long [labs](#) (long num) [OLDCALL](#)
- int [atoi](#) (const char *s)
- long [atol](#) (const char *s)
- char * [itoa](#) (int n, char *s, unsigned char radix) [OLDCALL](#)
- char * [uitoa](#) (unsigned int n, char *s, unsigned char radix) [OLDCALL](#)
- char * [ltoa](#) (long n, char *s, unsigned char radix) [OLDCALL](#)
- char * [ultoa](#) (unsigned long n, char *s, unsigned char radix) [OLDCALL](#)
- void * [calloc](#) (size_t nmemb, size_t size)
- void * [malloc](#) (size_t size)
- void * [realloc](#) (void *ptr, size_t size)
- void [free](#) (void *ptr)
- void * [bsearch](#) (const void *key, const void *base, size_t nmemb, size_t size, int(*compar)(const void *, const void *)) REENTRANT)
- void [qsort](#) (void *base, size_t nmemb, size_t size, int(*compar)(const void *, const void *)) REENTRANT)

20.132.1 Function Documentation

20.132.1.1 exit() void exit (
int status)

file stdlib.h 'Standard library' functions, for whatever that means. Causes normal program termination and the value of status is returned to the parent. All open streams are flushed and closed.

20.132.1.2 abs() int abs (
int i)

Returns the absolute value of int i

Parameters

<i>i</i>	Int to obtain absolute value of
----------	---------------------------------

If i is negative, returns -i; else returns i.

20.132.1.3 labs() long labs (
long num)

Returns the absolute value of long int num

Parameters

<i>num</i>	Long integer to obtain absolute value of
------------	--

20.132.1.4 atoi() int atoi (
const char * s)

Converts an ASCII string to an int

Parameters

<i>s</i>	String to convert to an int
----------	-----------------------------

The string may be of the format

`[\s]*[+-][\d]+[\D]*`

i.e. any number of spaces, an optional + or -, then an arbitrary number of digits.

The result is undefined if the number doesn't fit in an int.

Returns: Int value of string

20.132.1.5 atol() `long atol (`
`const char * s )`

Converts an ASCII string to a long.

Parameters

<i>s</i>	String to convert to a long int
----------	---------------------------------

See also

[atoi\(\)](#)

Returns: Long int value of string

20.132.1.6 itoa() `char * itoa (`
`int n,`
`char * s,`
`unsigned char radix )`

Converts an int into a base 10 ASCII string.

Parameters

<i>n</i>	Int to convert to a string
<i>s</i>	String to store the converted number
<i>radix</i>	Numerical base for converted number, ex: 10 is decimal base (parameter is required but not utilized on Game Boy and Analogue Pocket)

Can be used with [set_bkg_based_tiles\(\)](#) for printing if the digit character tiles are not ascii-mapped.

Returns: Pointer to converted string

20.132.1.7 uitoa() `char * uitoa (`
`unsigned int n,`
`char * s,`
`unsigned char radix )`

Converts an unsigned int into a base 10 ASCII string.

Parameters

<i>n</i>	Unsigned Int to convert to a string
<i>s</i>	String to store the converted number
<i>radix</i>	Numerical base for converted number, ex: 10 is decimal base (parameter is required but not utilized on Game Boy and Analogue Pocket)

Can be used with [set_bkg_based_tiles\(\)](#) for printing if the digit character tiles are not ascii-mapped.

Returns: Pointer to converted string

20.132.1.8 ltoa() `char * ltoa (`
 `long n,`
 `char * s,`
 `unsigned char radix )`

Converts a long into a base 10 ASCII string.

Parameters

<i>n</i>	Long int to convert to a string
<i>s</i>	String to store the converted number
<i>radix</i>	Numerical base for converted number, ex: 10 is decimal base (parameter is required but not utilized on Game Boy and Analogue Pocket)

Can be used with [set_bkg_based_tiles\(\)](#) for printing if the digit character tiles are not ascii-mapped.

Returns: Pointer to converted string

20.132.1.9 ultoa() `char * ultoa (`
 `unsigned long n,`
 `char * s,`
 `unsigned char radix )`

Converts an unsigned long into a base 10 ASCII string.

Parameters

<i>n</i>	Unsigned Long Int to convert to a string
<i>s</i>	String to store the converted number
<i>radix</i>	Numerical base for converted number, ex: 10 is decimal base (parameter is required but not utilized on Game Boy and Analogue Pocket)

Can be used with [set_bkg_based_tiles\(\)](#) for printing if the digit character tiles are not ascii-mapped.

Returns: Pointer to converted string

20.132.1.10 calloc() `void * calloc (`
 `size_t nmemb,`
 `size_t size )`

Memory allocation functions

20.132.1.11 malloc() `void * malloc (`
 `size_t size )`

20.132.1.12 realloc() `void * realloc (`
 `void * ptr,`
 `size_t size )`

20.132.1.13 free() `void free (`
 `void * ptr )`

20.132.1.14 bsearch() `void * bsearch (`
 `const void * key,`
 `const void * base,`
 `size_t nmemb,`

```

    size_t size,
    int(*) (const void *, const void *) REENTRANT compar )

```

search a sorted array of **nmemb** items

Parameters

<i>key</i>	Pointer to object that is the key for the search
<i>base</i>	Pointer to first object in the array to search
<i>nmemb</i>	Number of elements in the array
<i>size</i>	Size in bytes of each element in the array
<i>compar</i>	Function used to compare two elements of the array

Returns: Pointer to array entry that matches the search key. If key is not found, NULL is returned.

20.132.1.15 qsort() `void qsort (`
`void * base,`
`size_t nmemb,`
`size_t size,`
`int(*) (const void *, const void *) REENTRANT compar )`

Sort an array of **nmemb** items

Parameters

<i>base</i>	Pointer to first object in the array to sort
<i>nmemb</i>	Number of elements in the array
<i>size</i>	Size in bytes of each element in the array
<i>compar</i>	Function used to compare and sort two elements of the array

20.133 stdlib.h

[Go to the documentation of this file.](#)

```

1
4 #ifndef STDLIB_INCLUDE
5 #define STDLIB_INCLUDE
6
7 #include <types.h>
8
13 void exit(int status) OLDCALL;
14
15 #if 0
18 int getkey(void) OLDCALL;
19 #endif
20
26 int abs(int i);
27
28
33 long labs(long num) OLDCALL;
34
35
51 int atoi(const char *s);
52
53
60 long atol(const char *s);
61
73 char *itoa(int n, char *s, unsigned char radix) OLDCALL;
74
86 char *uitoa(unsigned int n, char *s, unsigned char radix) OLDCALL;
87
99 char *ltoa(long n, char *s, unsigned char radix) OLDCALL;
100
112 char *ultoa(unsigned long n, char *s, unsigned char radix) OLDCALL;
113
114
117 void *calloc (size_t nmemb, size_t size);
118 void *malloc (size_t size);
119 void *realloc (void *ptr, size_t size);
120 #if __STDC_VERSION__ >= 201112L

```

```

121 inline void *aligned_alloc(size_t alignment, size_t size)
122 {
123     (void)alignment;
124     return malloc(size);
125 }
126 #endif
127 extern void free (void * ptr);
128
129 /* Searching and sorting utilities (ISO C11 7.22.5) */
140 extern void *bsearch(const void *key, const void *base, size_t nmemb, size_t size, int (*compar)(const
    void *, const void *) REENTRANT);
141
142
149 extern void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *)
    REENTRANT);
150
151 #endif

```

20.134 gbdk-lib/include/stdnoreturn.h File Reference

Macros

- #define `noreturn` `_Noreturn`

20.134.1 Macro Definition Documentation

20.134.1.1 `noreturn` #define `noreturn` `_Noreturn`

20.135 stdnoreturn.h

[Go to the documentation of this file.](#)

```

1 #ifndef __SDCC_STDNORETURN_H
2 #define __SDCC_STDNORETURN_H 1
3
4 #define noreturn _Noreturn
5
6 #endif
7

```

20.136 gbdk-lib/include/asm/mos6502/string.h File Reference

#include <types.h>

Macros

- #define `memcpy`(dst, src, n) `__memcpy`(dst, src, n)

Functions

- char * `strcpy` (char *dest, const char *src) `OLDCALL`
- int `strcmp` (const char *s1, const char *s2)
- void * `__memcpy` (void *dest, const void *src, `size_t` len)
- void * `memmove` (void *dest, const void *src, `size_t` n) `OLDCALL`
- void * `memset` (void *s, int c, `size_t` n)
- char * `reverse` (char *s) `NONBANKED`
- char * `strcat` (char *s1, const char *s2) `NONBANKED`
- int `strlen` (const char *s) `OLDCALL`
- char * `strncat` (char *s1, const char *s2, int n) `NONBANKED`
- int `strncmp` (const char *s1, const char *s2, int n) `NONBANKED`
- char * `strncpy` (char *s1, const char *s2, int n) `NONBANKED`
- int `memcmp` (const void *buf1, const void *buf2, `size_t` count)

20.136.1 Detailed Description

Generic string functions.

20.136.2 Macro Definition Documentation

20.136.2.1 memcpy `#define memcpy(
 dst,
 src,
 n) __memcpy(dst, src, n)`

20.136.3 Function Documentation

20.136.3.1 strcpy() `char * strcpy (
 char * dest,
 const char * src)`

Copies the string pointed to by **src** (including the terminating '\0' character) to the array pointed to by **dest**. The strings may not overlap, and the destination string *dest* must be large enough to receive the copy.

Parameters

<i>dest</i>	Array to copy into
<i>src</i>	Array to copy from

Returns

A pointer to *dest*

20.136.3.2 strcmp() `int strcmp (
 const char * s1,
 const char * s2)`

Compares strings

Parameters

<i>s1</i>	First string to compare
<i>s2</i>	Second string to compare

Returns:

- > 0 if **s1** > **s2**
- 0 if **s1** == **s2**
- < 0 if **s1** < **s2**

20.136.3.3 __memcpy() `void * __memcpy (
 void * dest,
 const void * src,
 size_t len)`

Copies *n* bytes from memory area *src* to memory area *dest*.

The memory areas may not overlap.

Parameters

<i>dest</i>	Buffer to copy into
<i>src</i>	Buffer to copy from
<i>len</i>	Number of Bytes to copy

20.136.3.4 memmove() `void * memmove (`
`void * dest,`
`const void * src,`
`size_t n )`

Copies *n* bytes from memory area *src* to memory area *dest*, areas may overlap

20.136.3.5 memset() `void * memset (`
`void * s,`
`int c,`
`size_t n )`

Fills the memory region *s* with *n* bytes using value *c*

Parameters

<i>s</i>	Buffer to fill
<i>c</i>	char value to fill with (truncated from int)
<i>n</i>	Number of bytes to fill

20.136.3.6 reverse() `char * reverse (`
`char * s )`

Reverses the characters in a string

Parameters

<i>s</i>	Pointer to string to reverse.
----------	-------------------------------

For example 'abcdefg' will become 'gfedcba'.

Banked as the string must be modifiable.

Returns: Pointer to *s*

20.136.3.7 strcat() `char * strcat (`
`char * s1,`
`const char * s2 )`

Concatenate Strings. Appends string *s2* to the end of string *s1*

Parameters

<i>s1</i>	String to append onto
<i>s2</i>	String to copy from

For example 'abc' and 'def' will become 'abcdef'.

String *s1* must be large enough to store both *s1* and *s2*.

Returns: Pointer to *s1*

20.136.3.8 strlen() `int strlen (`

```
const char * s )
```

Calculates the length of a string

Parameters

<i>s</i>	String to calculate length of
----------	-------------------------------

Returns: Length of string not including the terminating '\0' character.

20.136.3.9 strncat()

```
char * strncat (
    char * s1,
    const char * s2,
    int n )
```

Concatenate at most **n** characters from string **s2** onto the end of **s1**.

Parameters

<i>s1</i>	String to append onto
<i>s2</i>	String to copy from
<i>n</i>	Max number of characters to copy from s2

String **s1** must be large enough to store both **s1** and **n** characters of **s2**

Returns: Pointer to **s1**

20.136.3.10 strncmp()

```
int strncmp (
    const char * s1,
    const char * s2,
    int n )
```

Compare strings (at most **n** characters):

Parameters

<i>s1</i>	First string to compare
<i>s2</i>	Second string to compare
<i>n</i>	Max number of characters to compare

Returns:

- > 0 if **s1** $>$ **s2**
- 0 if **s1** $==$ **s2**
- < 0 if **s1** $<$ **s2**

20.136.3.11 strncpy()

```
char * strncpy (
    char * s1,
    const char * s2,
    int n )
```

Copy **n** characters from string **s2** to **s1**

Parameters

<i>s1</i>	String to copy into
<i>s2</i>	String to copy from
<i>n</i>	Max number of characters to copy from s2

If **s2** is shorter than **n**, the remaining bytes in **s1** are filled with `\0`.

Warning: If there is no `\0` in the first **n** bytes of **s2** then **s1** will not be null terminated.

Returns: Pointer to **s1**

20.136.3.12 memcmp() `int memcmp (`
`const void * buf1,`
`const void * buf2,`
`size_t count )`

Compares buffers

Parameters

<i>buf1</i>	First buffer to compare
<i>buf2</i>	Second buffer to compare
<i>count</i>	Buffer length

Returns:

- `> 0` if **buf1** `>` **buf2**
- `0` if **buf1** `==` **buf2**
- `< 0` if **buf1** `<` **buf2**

20.137 string.h

[Go to the documentation of this file.](#)

```
1
4 #ifndef STRING_INCLUDE
5 #define STRING_INCLUDE
6
7 #include <types.h>
8
20 char *strcpy(char *dest, const char *src) OLDCALL;
21
32 int strcmp(const char *s1, const char *s2);
33
42 void *__memcpy(void *dest, const void *src, size_t len);
43 #define memcpy(dst, src, n) __memcpy(dst, src, n)
44
47 void *memmove (void *dest, const void *src, size_t n) OLDCALL;
48
55 void *memset (void *s, int c, size_t n);
56
67 char *reverse(char *s) NONBANKED;
68
80 char *strcat(char *s1, const char *s2) NONBANKED;
81
88 int strlen(const char *s) OLDCALL;
89
100 char *strncat(char *s1, const char *s2, int n) NONBANKED;
101
113 int strncmp(const char *s1, const char *s2, int n) NONBANKED;
114
130 char *strncpy(char *s1, const char *s2, int n) NONBANKED;
131
143 int memcmp(const void *buf1, const void *buf2, size_t count);
144
145 #endif
```

20.138 gbdk-lib/include/asm/sm83/string.h File Reference

```
#include <types.h>
```

Functions

- char * [strcpy](#) (char *dest, const char *src) [OLDCALL PRESERVES_REGS\(b](#)
- int [strcmp](#) (const char *s1, const char *s2) [OLDCALL PRESERVES_REGS\(b](#)

- void * [memcpy](#) (void *dest, const void *src, [size_t](#) len)
- void * [memmove](#) (void *dest, const void *src, [size_t](#) n)
- void * [memset](#) (void *s, int c, [size_t](#) n) [OLDCALL PRESERVES_REGS\(b](#)
- char * [reverse](#) (char *s) [OLDCALL PRESERVES_REGS\(b](#)
- char * [strcat](#) (char *s1, const char *s2)
- int [strlen](#) (const char *s) [OLDCALL PRESERVES_REGS\(b](#)
- char * [strncat](#) (char *s1, const char *s2, int n)
- int [strncmp](#) (const char *s1, const char *s2, int n)
- char * [strncpy](#) (char *s1, const char *s2, int n)
- int [memcmp](#) (const void *buf1, const void *buf2, [size_t](#) count) [OLDCALL](#)

Variables

- char [c](#)

20.138.1 Detailed Description

Generic string functions.

20.138.2 Function Documentation

20.138.2.1 strcpy() `char * strcpy (`
 `char * dest,`
 `const char * src )`

Copies the string pointed to by **src** (including the terminating '\0' character) to the array pointed to by **dest**. The strings may not overlap, and the destination string dest must be large enough to receive the copy.

Parameters

<i>dest</i>	Array to copy into
<i>src</i>	Array to copy from

Returns

A pointer to dest

20.138.2.2 strcmp() `int strcmp (`
 `const char * s1,`
 `const char * s2 )`

Compares strings

Parameters

<i>s1</i>	First string to compare
<i>s2</i>	Second string to compare

Returns:

- > 0 if **s1** > **s2**
- 0 if **s1** == **s2**
- < 0 if **s1** < **s2**

20.138.2.3 memcpy() `void * memcpy (`
 `void * dest,`
 `const void * src,`
 `size_t len )`

Copies *n* bytes from memory area *src* to memory area *dest*.
 The memory areas may not overlap.

Parameters

<i>dest</i>	Buffer to copy into
<i>src</i>	Buffer to copy from
<i>len</i>	Number of Bytes to copy

20.138.2.4 memmove() `void * memmove (`
 `void * dest,`
 `const void * src,`
 `size_t n )`

Copies *n* bytes from memory area *src* to memory area *dest*, areas may overlap

20.138.2.5 memset() `void * memset (`
 `void * s,`
 `int c,`
 `size_t n )`

Fills the memory region *s* with *n* bytes using value *c*

Parameters

<i>s</i>	Buffer to fill
<i>c</i>	char value to fill with (truncated from int)
<i>n</i>	Number of bytes to fill

20.138.2.6 reverse() `char * reverse (`
 `char * s )`

Reverses the characters in a string

Parameters

<i>s</i>	Pointer to string to reverse.
----------	-------------------------------

For example 'abcdefg' will become 'gfedcba'.
 Banked as the string must be modifiable.
 Returns: Pointer to **s**

20.138.2.7 strcat() `char * strcat (`
 `char * s1,`
 `const char * s2 )`

Concatenate Strings. Appends string *s2* to the end of string *s1*

Parameters

<i>s1</i>	String to append onto
<i>s2</i>	String to copy from

For example 'abc' and 'def' will become 'abcdef'.
String **s1** must be large enough to store both **s1** and **s2**.
Returns: Pointer to **s1**

20.138.2.8 strlen() `int strlen (`
 `const char * s )`

Calculates the length of a string

Parameters

<i>s</i>	String to calculate length of
----------	-------------------------------

Returns: Length of string not including the terminating '\0' character.

20.138.2.9 strncat() `char * strncat (`
 `char * s1,`
 `const char * s2,`
 `int n )`

Concatenate at most **n** characters from string **s2** onto the end of **s1**.

Parameters

<i>s1</i>	String to append onto
<i>s2</i>	String to copy from
<i>n</i>	Max number of characters to copy from s2

String **s1** must be large enough to store both **s1** and **n** characters of **s2**
Returns: Pointer to **s1**

20.138.2.10 strncmp() `int strncmp (`
 `const char * s1,`
 `const char * s2,`
 `int n )`

Compare strings (at most **n** characters):

Parameters

<i>s1</i>	First string to compare
<i>s2</i>	Second string to compare
<i>n</i>	Max number of characters to compare

Returns zero if the strings are identical, or non-zero if they are not (see below).
Returns:

- > 0 if **s1** > **s2** (at first non-matching byte)
- 0 if **s1** == **s2**
- < 0 if **s1** < **s2** (at first non-matching byte)

20.138.2.11 strncpy() `char * strncpy (`
 `char * s1,`
 `const char * s2,`
 `int n )`

Copy **n** characters from string **s2** to **s1**

Parameters

<i>s1</i>	String to copy into
<i>s2</i>	String to copy from
<i>n</i>	Max number of characters to copy from s2

If **s2** is shorter than **n**, the remaining bytes in **s1** are filled with `\0`.

Warning: If there is no `\0` in the first **n** bytes of **s2** then **s1** will not be null terminated.

Returns: Pointer to **s1**

20.138.2.12 memcmp() `int memcmp (`
`const void * buf1,`
`const void * buf2,`
`size_t count )`

Compare up to **count** bytes in buffers **buf1** and **buf2**

Parameters

<i>buf1</i>	Pointer to First buffer to compare
<i>buf2</i>	Pointer to Second buffer to compare
<i>count</i>	Max number of bytes to compare

Returns zero if the buffers are identical, or non-zero if they are not (see below).

Returns:

- `> 0` if **buf1** `>` **buf2** (at first non-matching byte)
- `0` if **buf1** `==` **buf2**
- `< 0` if **buf1** `<` **buf2** (at first non-matching byte)

20.138.3 Variable Documentation

20.138.3.1 `c` `void c`

20.139 string.h

[Go to the documentation of this file.](#)

```

1
4 #ifndef STRING_INCLUDE
5 #define STRING_INCLUDE
6
7 #include <types.h>
8
20 char *strcpy(char *dest, const char *src) OLD_CALL PRESERVES_REGS(b, c);
21
32 int strcmp(const char *s1, const char *s2) OLD_CALL PRESERVES_REGS(b, c);
33
42 void *memcpy(void *dest, const void *src, size_t len);
43
46 void *memmove (void *dest, const void *src, size_t n);
47
54 void *memset (void *s, int c, size_t n) OLD_CALL PRESERVES_REGS(b, c);
55
66 char *reverse(char *s) OLD_CALL PRESERVES_REGS(b, c);
67
79 char *strcat(char *s1, const char *s2);
80
87 int strlen(const char *s) OLD_CALL PRESERVES_REGS(b, c);
88
99 char *strncat(char *s1, const char *s2, int n);
100
115 int strncmp(const char *s1, const char *s2, int n);
116

```

```

132 char *strncpy(char *s1, const char *s2, int n);
133
148 int memcmp(const void *buf1, const void *buf2, size_t count) OLDCALL;
149
150 #endif

```

20.140 gbdk-lib/include/asm/z80/string.h File Reference

```
#include <types.h>
```

Functions

- char * [strcpy](#) (char *dest, const char *src) [OLDCALL](#)
- int [strcmp](#) (const char *s1, const char *s2)
- void * [memcpy](#) (void *dest, const void *src, [size_t](#) len)
- void * [memmove](#) (void *dest, const void *src, [size_t](#) n) [OLDCALL](#)
- void * [memset](#) (void *s, int c, [size_t](#) n) [Z88DK_CALLEE](#)
- char * [reverse](#) (char *s) [NONBANKED](#)
- char * [strcat](#) (char *s1, const char *s2) [NONBANKED](#)
- int [strlen](#) (const char *s) [OLDCALL](#)
- char * [strncat](#) (char *s1, const char *s2, int n) [NONBANKED](#)
- int [strncmp](#) (const char *s1, const char *s2, int n) [NONBANKED](#)
- char * [strncpy](#) (char *s1, const char *s2, int n) [NONBANKED](#)
- int [memcmp](#) (const void *buf1, const void *buf2, [size_t](#) count) [Z88DK_CALLEE](#)

20.140.1 Detailed Description

Generic string functions.

20.140.2 Function Documentation

20.140.2.1 strcpy() char * strcpy (

```

    char * dest,
    const char * src )

```

Copies the string pointed to by **src** (including the terminating ‘\0’ character) to the array pointed to by **dest**. The strings may not overlap, and the destination string dest must be large enough to receive the copy.

Parameters

<i>dest</i>	Array to copy into
<i>src</i>	Array to copy from

Returns

A pointer to dest

20.140.2.2 strcmp() int strcmp (

```

    const char * s1,
    const char * s2 )

```

Compares strings

Parameters

<i>s1</i>	First string to compare
<i>s2</i>	Second string to compare

Returns:

- > 0 if **s1** $>$ **s2**
- 0 if **s1** $==$ **s2**
- < 0 if **s1** $<$ **s2**

20.140.2.3 memcpy() `void * memcpy (`
`void * dest,`
`const void * src,`
`size_t len )`

Copies *n* bytes from memory area *src* to memory area *dest*.
 The memory areas may not overlap.

Parameters

<i>dest</i>	Buffer to copy into
<i>src</i>	Buffer to copy from
<i>len</i>	Number of Bytes to copy

20.140.2.4 memmove() `void * memmove (`
`void * dest,`
`const void * src,`
`size_t n )`

Copies *n* bytes from memory area *src* to memory area *dest*, areas may overlap

20.140.2.5 memset() `void * memset (`
`void * s,`
`int c,`
`size_t n )`

Fills the memory region **s** with **n** bytes using value **c**

Parameters

<i>s</i>	Buffer to fill
<i>c</i>	char value to fill with (truncated from int)
<i>n</i>	Number of bytes to fill

20.140.2.6 reverse() `char * reverse (`
`char * s )`

Reverses the characters in a string

Parameters

<i>s</i>	Pointer to string to reverse.
----------	-------------------------------

For example 'abcdefg' will become 'gfedcba'.

Banked as the string must be modifiable.

Returns: Pointer to **s**

20.140.2.7 strcat() `char * strcat (`
 `char * s1,`
 `const char * s2 )`

Concatenate Strings. Appends string **s2** to the end of string **s1**

Parameters

<i>s1</i>	String to append onto
<i>s2</i>	String to copy from

For example 'abc' and 'def' will become 'abcdef'.
String **s1** must be large enough to store both **s1** and **s2**.
Returns: Pointer to **s1**

20.140.2.8 strlen() `int strlen (`
 `const char * s )`

Calculates the length of a string

Parameters

<i>s</i>	String to calculate length of
----------	-------------------------------

Returns: Length of string not including the terminating '\0' character.

20.140.2.9 strncat() `char * strncat (`
 `char * s1,`
 `const char * s2,`
 `int n )`

Concatenate at most **n** characters from string **s2** onto the end of **s1**.

Parameters

<i>s1</i>	String to append onto
<i>s2</i>	String to copy from
<i>n</i>	Max number of characters to copy from s2

String **s1** must be large enough to store both **s1** and **n** characters of **s2**
Returns: Pointer to **s1**

20.140.2.10 strncmp() `int strncmp (`
 `const char * s1,`
 `const char * s2,`
 `int n )`

Compare strings (at most **n** characters):

Parameters

<i>s1</i>	First string to compare
<i>s2</i>	Second string to compare
<i>n</i>	Max number of characters to compare

Returns:

- > 0 if **s1** $>$ **s2**
- 0 if **s1** $==$ **s2**

- < 0 if **s1** < **s2**

20.140.2.11 strncpy() char * strncpy (

```

    char * s1,
    const char * s2,
    int n )

```

Copy **n** characters from string **s2** to **s1**

Parameters

<i>s1</i>	String to copy into
<i>s2</i>	String to copy from
<i>n</i>	Max number of characters to copy from s2

If **s2** is shorter than **n**, the remaining bytes in **s1** are filled with \0.

Warning: If there is no \0 in the first **n** bytes of **s2** then **s1** will not be null terminated.

Returns: Pointer to **s1**

20.140.2.12 memcmp() int memcmp (

```

    const void * buf1,
    const void * buf2,
    size_t count )

```

Compares buffers

Parameters

<i>buf1</i>	First buffer to compare
<i>buf2</i>	Second buffer to compare
<i>count</i>	Buffer length

Returns:

- > 0 if **buf1** > **buf2**
- 0 if **buf1** == **buf2**
- < 0 if **buf1** < **buf2**

20.141 string.h

[Go to the documentation of this file.](#)

```

1
4 #ifndef STRING_INCLUDE
5 #define STRING_INCLUDE
6
7 #include <types.h>
8
20 char *strcpy(char *dest, const char *src) OLDCALL;
21
32 int strcmp(const char *s1, const char *s2);
33
42 void *memcpy(void *dest, const void *src, size_t len);
43
46 void *memmove (void *dest, const void *src, size_t n) OLDCALL;
47
54 void *memset (void *s, int c, size_t n) Z88DK_CALLEE;
55
66 char *reverse(char *s) NONBANKED;
67
79 char *strcat(char *s1, const char *s2) NONBANKED;
80
87 int strlen(const char *s) OLDCALL;
88

```

```
99 char *strncat(char *s1, const char *s2, int n) NONBANKED;
100
112 int strncmp(const char *s1, const char *s2, int n) NONBANKED;
113
129 char *strncpy(char *s1, const char *s2, int n) NONBANKED;
130
142 int memcmp(const void *buf1, const void *buf2, size_t count) Z88DK_CALLEE;
143
144 #endif
```

20.142 gbdk-lib/include/string.h File Reference

```
#include <asm/sm83/string.h>
```

20.142.1 Detailed Description

Generic string functions.

20.143 string.h

[Go to the documentation of this file.](#)

```
1
4 #ifndef STD_STRING_INCLUDE
5 #define STD_STRING_INCLUDE
6
7 #if defined(__PORT_sm83)
8     #include <asm/sm83/string.h>
9 #elif defined(__PORT_z80)
10    #include <asm/z80/string.h>
11 #elif defined(__PORT_mos6502)
12    #include <asm/mos6502/string.h>
13 #else
14    #error Unrecognized port
15 #endif
16
17 #endif
```

20.144 gbdk-lib/include/time.h File Reference

```
#include <types.h>
#include <stdint.h>
```

Macros

- `#define CLOCKS_PER_SEC 60`

Typedefs

- `typedef uint16_t time_t`

Functions

- `clock_t clock (void) OLDCALL`
- `time_t time (time_t *)`

20.144.1 Detailed Description

Sort of ANSI compliant time functions.

20.144.2 Macro Definition Documentation

20.144.2.1 CLOCKS_PER_SEC `#define CLOCKS_PER_SEC 60`

20.144.3 Typedef Documentation

20.144.3.1 time_t typedef uint16_t time_t

20.144.4 Function Documentation

20.144.4.1 clock() clock_t clock (
void)

Returns an approximation of processor time used by the program in Clocks

The value returned is the CPU time (ticks) used so far as a clock_t.

To get the number of seconds used, divide by CLOCKS_PER_SEC.

This is based on sys_time, which will wrap around every ~18 minutes. (unsigned 16 bits = 65535 / 60 / 60 = 18.2)

See also

sys_time, time()

20.144.4.2 time() time_t time (
time_t * t)

Converts clock() time to Seconds

Parameters

<code>t</code>	If pointer <code>t</code> is not NULL, it's value will be set to the same seconds calculation as returned by the function.
----------------	--

The calculation is clock() / CLOCKS_PER_SEC

Returns: time in seconds

See also

sys_time, clock()

20.145 time.h

[Go to the documentation of this file.](#)

```

1
4 #ifndef TIME_INCLUDE
5 #define TIME_INCLUDE
6
7 #include <types.h>
8 #include <stdint.h>
9
10 #define CLOCKS_PER_SEC 60
11
12 typedef uint16_t time_t;
13
25 clock_t clock(void) OLDCALL;
26
36 time_t time(time_t *t);
37
38 #endif

```

20.146 gbdk-lib/include/typeof.h File Reference

Macros

- #define TYPEOF_INT 1
- #define TYPEOF_SHORT 2
- #define TYPEOF_CHAR 3

- `#define TYPEOF_LONG 4`
- `#define TYPEOF_FLOAT 5`
- `#define TYPEOF_FIXED16X16 6`
- `#define TYPEOF_BIT 7`
- `#define TYPEOF_BITFIELD 8`
- `#define TYPEOF_SBIT 9`
- `#define TYPEOF_SFR 10`
- `#define TYPEOF_VOID 11`
- `#define TYPEOF_STRUCT 12`
- `#define TYPEOF_ARRAY 13`
- `#define TYPEOF_FUNCTION 14`
- `#define TYPEOF_POINTER 15`
- `#define TYPEOF_FPOINTER 16`
- `#define TYPEOF_CPOINTER 17`
- `#define TYPEOF_GPOINTER 18`
- `#define TYPEOF_PPOINTER 19`
- `#define TYPEOF_IPOINTER 20`
- `#define TYPEOF_EEPPPOINTER 21`

20.146.1 Macro Definition Documentation

20.146.1.1 TYPEOF_INT `#define TYPEOF_INT 1`

20.146.1.2 TYPEOF_SHORT `#define TYPEOF_SHORT 2`

20.146.1.3 TYPEOF_CHAR `#define TYPEOF_CHAR 3`

20.146.1.4 TYPEOF_LONG `#define TYPEOF_LONG 4`

20.146.1.5 TYPEOF_FLOAT `#define TYPEOF_FLOAT 5`

20.146.1.6 TYPEOF_FIXED16X16 `#define TYPEOF_FIXED16X16 6`

20.146.1.7 TYPEOF_BIT `#define TYPEOF_BIT 7`

20.146.1.8 TYPEOF_BITFIELD `#define TYPEOF_BITFIELD 8`

20.146.1.9 TYPEOF_SBIT `#define TYPEOF_SBIT 9`

20.146.1.10 TYPEOF_SFR `#define TYPEOF_SFR 10`

20.146.1.11 TYPEOF_VOID `#define TYPEOF_VOID 11`

20.146.1.12 **__typeof_struct** #define __typeof_struct 12

20.146.1.13 **__typeof_array** #define __typeof_array 13

20.146.1.14 **__typeof_function** #define __typeof_function 14

20.146.1.15 **__typeof_pointer** #define __typeof_pointer 15

20.146.1.16 **__typeof_fpointer** #define __typeof_fpointer 16

20.146.1.17 **__typeof_cpointer** #define __typeof_cpointer 17

20.146.1.18 **__typeof_gpointer** #define __typeof_gpointer 18

20.146.1.19 **__typeof_ppointer** #define __typeof_ppointer 19

20.146.1.20 **__typeof_ipointer** #define __typeof_ipointer 20

20.146.1.21 **__typeof_eepointer** #define __typeof_eepointer 21

20.147 typeof.h

[Go to the documentation of this file.](#)

```
1 /*-----
2  typeof.h - Contains enumerations of values returned by __typeof
3
4  Copyright (C) 2001, Sandeep Dutta . sandeep.dutta@usa.net
5
6  This library is free software; you can redistribute it and/or modify it
7  under the terms of the GNU General Public License as published by the
8  Free Software Foundation; either version 2, or (at your option) any
9  later version.
10
11  This library is distributed in the hope that it will be useful,
12  but WITHOUT ANY WARRANTY; without even the implied warranty of
13  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14  GNU General Public License for more details.
15
16  You should have received a copy of the GNU General Public License
17  along with this library; see the file COPYING. If not, write to the
18  Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,
19  MA 02110-1301, USA.
20
21  As a special exception, if you link this library with other files,
22  some of which are compiled with SDCC, to produce an executable,
23  this library does not by itself cause the resulting executable to
24  be covered by the GNU General Public License. This exception does
25  not however invalidate any other reasons why the executable file
26  might be covered by the GNU General Public License.
27 -----*/
28
29 #ifndef __SDC51_TYPEOF_H
30 #define __SDC51_TYPEOF_H 1
31
32 #define __typeof_int 1
33 #define __typeof_short 2
34 #define __typeof_char 3
35 #define __typeof_long 4
36 #define __typeof_float 5
```

```
37 #define TYPEOF_FIXED16X16 6
38 #define TYPEOF_BIT 7
39 #define TYPEOF_BITFIELD 8
40 #define TYPEOF_SBIT 9
41 #define TYPEOF_SFR 10
42 #define TYPEOF_VOID 11
43 #define TYPEOF_STRUCT 12
44 #define TYPEOF_ARRAY 13
45 #define TYPEOF_FUNCTION 14
46 #define TYPEOF_POINTER 15
47 #define TYPEOF_FPOINTER 16
48 #define TYPEOF_CPOINTER 17
49 #define TYPEOF_GPOINTER 18
50 #define TYPEOF_PPOINTER 19
51 #define TYPEOF_IPOINTER 20
52 #define TYPEOF_EEPPPOINTER 21
53
54 #endif
```

20.148 gbdk-lib/include/asm/mos6502/types.h File Reference

Macros

- `#define` [__SIZE_T_DEFINED](#)

Typedefs

- typedef signed char [INT8](#)
- typedef unsigned char [UINT8](#)
- typedef signed int [INT16](#)
- typedef unsigned int [UINT16](#)
- typedef signed long [INT32](#)
- typedef unsigned long [UINT32](#)
- typedef unsigned int [size_t](#)
- typedef unsigned int [clock_t](#)

20.148.1 Detailed Description

Types definitions for the gb.

20.148.2 Macro Definition Documentation

20.148.2.1 `__SIZE_T_DEFINED` `#define __SIZE_T_DEFINED`

20.148.3 Typedef Documentation

20.148.3.1 `INT8` typedef signed char [INT8](#)
Signed eight bit.

20.148.3.2 `UINT8` typedef unsigned char [UINT8](#)
Unsigned eight bit.

20.148.3.3 `INT16` typedef signed int [INT16](#)
Signed sixteen bit.

20.148.3.4 `UINT16` typedef unsigned int [UINT16](#)
Unsigned sixteen bit.

20.148.3.5 INT32 typedef signed long [INT32](#)
Signed 32 bit.

20.148.3.6 UINT32 typedef unsigned long [UINT32](#)
Unsigned 32 bit.

20.148.3.7 size_t typedef unsigned int [size_t](#)

20.148.3.8 clock_t typedef unsigned int [clock_t](#)
Returned from clock

See also

[clock](#)

20.149 types.h

[Go to the documentation of this file.](#)

```

1
5 #ifndef ASM_MOS6502_TYPES_INCLUDE
6 #define ASM_MOS6502_TYPES_INCLUDE
7
8 #ifndef __PORT_mos6502
9     #error mos6502 only.
10 #endif
11
12 #ifdef __SDCC
13
14 #define NONBANKED        __nonbanked
15 #define BANKED           __banked
16 #define REENTRANT        __reentrant
17 #define NO_OVERLAY_LOCALS __no_overlay_locals
18 #define CRITICAL         __critical
19
20 #define INTERRUPT        __interrupt
21
22 #endif
23
24 typedef signed char      INT8;
25 typedef unsigned char    UINT8;
26 typedef signed int       INT16;
27 typedef unsigned int     UINT16;
28 typedef signed long      INT32;
29 typedef unsigned long    UINT32;
30
31 #ifndef __SIZE_T_DEFINED
32 #define __SIZE_T_DEFINED
33 typedef unsigned int      size_t;
34 #endif
35
36 typedef unsigned int      clock_t;
37
38 #endif

```

20.150 gbdk-lib/include/asm/sm83/types.h File Reference

Macros

- `#define` [__SIZE_T_DEFINED](#)

Typedefs

- typedef signed char [INT8](#)
- typedef unsigned char [UINT8](#)
- typedef signed int [INT16](#)
- typedef unsigned int [UINT16](#)
- typedef signed long [INT32](#)
- typedef unsigned long [UINT32](#)
- typedef unsigned int [size_t](#)
- typedef unsigned int [clock_t](#)

20.150.1 Detailed Description

Types definitions for the gb.

20.150.2 Macro Definition Documentation

20.150.2.1 `__SIZE_T_DEFINED` `#define __SIZE_T_DEFINED`

20.150.3 Typedef Documentation

20.150.3.1 `INT8` `typedef signed char INT8`
Signed eight bit.

20.150.3.2 `UINT8` `typedef unsigned char UINT8`
Unsigned eight bit.

20.150.3.3 `INT16` `typedef signed int INT16`
Signed sixteen bit.

20.150.3.4 `UINT16` `typedef unsigned int UINT16`
Unsigned sixteen bit.

20.150.3.5 `INT32` `typedef signed long INT32`
Signed 32 bit.

20.150.3.6 `UINT32` `typedef unsigned long UINT32`
Unsigned 32 bit.

20.150.3.7 `size_t` `typedef unsigned int size_t`

20.150.3.8 `clock_t` `typedef unsigned int clock_t`
Returned from clock

See also

[clock](#)

20.151 `types.h`

[Go to the documentation of this file.](#)

```
1
5 #ifndef ASM_SM83_TYPES_INCLUDE
6 #define ASM_SM83_TYPES_INCLUDE
7
8 #ifndef __PORT_sm83
9   #error sm83 only.
10 #endif
11
12 #ifdef __SDCC
13
14 #define NONBANKED      __nonbanked
15 #define BANKED         __banked
16 #define REENTRANT
17 #define NO_OVERLAY_LOCALS
18 #define CRITICAL       __critical
19
20 #define INTERRUPT      __interrupt
21
22 #endif
```



```
44
47 typedef signed char      INT8;
50 typedef unsigned char    UINT8;
53 typedef signed int       INT16;
56 typedef unsigned int     UINT16;
59 typedef signed long      INT32;
62 typedef unsigned long    UINT32;
63
64 #ifndef __SIZE_T_DEFINED
65 #define __SIZE_T_DEFINED
66 typedef unsigned int      size_t;
67 #endif
68
72 typedef unsigned int      clock_t;
73
74 #endif
```

20.152 gbdk-lib/include/asm/types.h File Reference

#include <asm/sm83/types.h>

Data Structures

- union [_fixed](#)

Macros

- #define [OLDCALL](#)
- #define [PRESERVES_REGS](#)(...)
- #define [NAKED](#)
- #define [SFR](#)
- #define [AT](#)(A)
- #define [NORETURN](#)
- #define [NONBANKED](#)
- #define [BANKED](#)
- #define [CRITICAL](#)
- #define [INTERRUPT](#)

Typedefs

- typedef [INT8](#) [BOOLEAN](#)
- typedef [INT8](#) [BYTE](#)
- typedef [UINT8](#) [UBYTE](#)
- typedef [INT16](#) [WORD](#)
- typedef [UINT16](#) [UWORD](#)
- typedef [INT32](#) [LWORD](#)
- typedef [UINT32](#) [ULWORD](#)
- typedef [INT32](#) [DWORD](#)
- typedef [UINT32](#) [UDWORD](#)
- typedef union [_fixed](#) [fixed](#)

20.152.1 Detailed Description

Shared types definitions.

20.152.2 Macro Definition Documentation

20.152.2.1 OLDCALL

```
#define OLDCALL
```

20.152.2.2 PRESERVES_REGS `#define PRESERVES_REGS(
...)`

20.152.2.3 NAKED `#define NAKED`

20.152.2.4 SFR `#define SFR`

20.152.2.5 AT `#define AT(
A)`

20.152.2.6 NORETURN `#define NORETURN`

20.152.2.7 NONBANKED `#define NONBANKED`

20.152.2.8 BANKED `#define BANKED`

20.152.2.9 CRITICAL `#define CRITICAL`

20.152.2.10 INTERRUPT `#define INTERRUPT`

20.152.3 Typedef Documentation

20.152.3.1 BOOLEAN `typedef INT8 BOOLEAN`
TRUE or FALSE.

20.152.3.2 BYTE `typedef INT8 BYTE`
Signed 8 bit.

20.152.3.3 UBYTE `typedef UINT8 UBYTE`
Unsigned 8 bit.

20.152.3.4 WORD `typedef INT16 WORD`
Signed 16 bit

20.152.3.5 UWORD `typedef UINT16 UWORD`
Unsigned 16 bit

20.152.3.6 LWORD `typedef INT32 LWORD`
Signed 32 bit

20.152.3.7 ULWORD `typedef UINT32 ULWORD`
Unsigned 32 bit

20.152.3.8 DWORD `typedef INT32 DWORD`
Signed 32 bit

20.152.3.9 UDWORD `typedef UINT32 UDWORD`
 Unsigned 32 bit

20.152.3.10 fixed `typedef union _fixed fixed`
 Useful definition for working with 8 bit + 8 bit fixed point values
 Use `.w` to access the variable as unsigned 16 bit type.
 Use `.b.h` and `.b.l` (or just `.h` and `.l`) to directly access it's high and low unsigned 8 bit values.

20.153 types.h

[Go to the documentation of this file.](#)

```

1
4 #ifndef ASM_TYPES_INCLUDE
5 #define ASM_TYPES_INCLUDE
6
7 #if defined(__PORT_sm83)
8 #include <asm/sm83/types.h>
9 #elif defined(__PORT_z80)
10 #include <asm/z80/types.h>
11 #elif defined(__PORT_mos6502)
12 #include <asm/mos6502/types.h>
13 #else
14 #error Unrecognised port
15 #endif
16
17 #ifndef OLDCALL
18 #if __SDCC_REVISION >= 12608
19 #define OLDCALL __sdcccall(0)
20 #else
21 #define OLDCALL
22 #endif
23 #endif
24
25 #ifdef __SDCC
26 #define PRESERVES_REGS(...) __preserves_regs(__VA_ARGS__)
27 #define NAKED __naked
28 #define SFR __sfr
29 #define AT(A) __at(A)
30 #define NORETURN __noreturn
31 #else
32 #define PRESERVES_REGS(...)
33 #define NAKED
34 #define SFR
35 #define AT(A)
36 #define NORETURN
37 #endif
38
39 #ifndef NONBANKED
40 #define NONBANKED
41 #endif
42 #ifndef BANKED
43 #define BANKED
44 #endif
45 #ifndef CRITICAL
46 #define CRITICAL
47 #endif
48 #ifndef INTERRUPT
49 #define INTERRUPT
50 #endif
51
55 typedef INT8    BOOLEAN;
56
59 typedef INT8    BYTE;
62 typedef UINT8   UBYTE;
64 typedef INT16   WORD;
66 typedef UINT16  UWORD;
68 typedef INT32   LWORD;
70 typedef UINT32  ULWORD;
72 typedef INT32   DWORD;
74 typedef UINT32  UDWORD;
75
82 typedef union _fixed {
83     struct {
84         UBYTE l;
85         UBYTE h;
86     };
87     struct {
88         UBYTE l;
89         UBYTE h;
90     } b;
91     UWORD w;
92 } fixed;

```

```
93
94 #endif
```

20.154 gbdk-lib/include/asm/z80/types.h File Reference

Macros

- `#define Z88DK_CALLEE`
- `#define Z88DK_FASTCALL`
- `#define __SIZE_T_DEFINED`

Typedefs

- `typedef signed char INT8`
- `typedef unsigned char UINT8`
- `typedef signed int INT16`
- `typedef unsigned int UINT16`
- `typedef signed long INT32`
- `typedef unsigned long UINT32`
- `typedef unsigned int size_t`
- `typedef unsigned int clock_t`

20.154.1 Detailed Description

Types definitions for the gb.

20.154.2 Macro Definition Documentation

20.154.2.1 Z88DK_CALLEE `#define Z88DK_CALLEE`

20.154.2.2 Z88DK_FASTCALL `#define Z88DK_FASTCALL`

20.154.2.3 __SIZE_T_DEFINED `#define __SIZE_T_DEFINED`

20.154.3 Typedef Documentation

20.154.3.1 INT8 `typedef signed char INT8`
Signed eight bit.

20.154.3.2 UINT8 `typedef unsigned char UINT8`
Unsigned eight bit.

20.154.3.3 INT16 `typedef signed int INT16`
Signed sixteen bit.

20.154.3.4 UINT16 `typedef unsigned int UINT16`
Unsigned sixteen bit.

20.154.3.5 INT32 `typedef signed long INT32`
Signed 32 bit.

20.154.3.6 UINT32 typedef unsigned long [UINT32](#)
 Unsigned 32 bit.

20.154.3.7 size_t typedef unsigned int [size_t](#)

20.154.3.8 clock_t typedef unsigned int [clock_t](#)
 Returned from clock

See also

[clock](#)

20.155 types.h

[Go to the documentation of this file.](#)

```

1
2
3
4
5 #ifndef ASM_Z80_TYPES_INCLUDE
6 #define ASM_Z80_TYPES_INCLUDE
7
8 #ifndef __PORT_Z80
9     #error z80 only.
10 #endif
11
12 #ifdef __SDCC
13
14 #define Z88DK_CALLEE __sdcccall(0) __z88dk_callee
15 #define Z88DK_FASTCALL __z88dk_fastcall
16
17 #define NONBANKED      __nonbanked
18 #define BANKED         __banked
19 #define REENTRANT
20 #define NO_OVERLAY_LOCALS
32 #define CRITICAL      __critical
33
42 #define INTERRUPT      __interrupt
43
44 #else
45
46 #define Z88DK_CALLEE
47 #define Z88DK_FASTCALL
48
49 #endif
50
53 typedef signed char      INT8;
56 typedef unsigned char   UINT8;
59 typedef signed int      INT16;
62 typedef unsigned int    UINT16;
65 typedef signed long     INT32;
68 typedef unsigned long   UINT32;
69
70 #ifndef __SIZE_T_DEFINED
71 #define __SIZE_T_DEFINED
72 typedef unsigned int     size_t;
73 #endif
74
78 typedef unsigned int     clock_t;
79
80 #endif

```

20.156 gbdk-lib/include/types.h File Reference

```
#include <asm/types.h>
```

Macros

- #define [NULL](#) (void *)0
- #define [FALSE](#) 0
- #define [TRUE](#) 1

Typedefs

- typedef void * [POINTER](#)

20.156.1 Detailed Description

Basic types.
Directly include the port specific file.

20.156.2 Macro Definition Documentation

20.156.2.1 NULL `#define NULL (void *)0`

20.156.2.2 FALSE `#define FALSE 0`
A 'false' value.

20.156.2.3 TRUE `#define TRUE 1`
A 'true' value.

20.156.3 Typedef Documentation

20.156.3.1 POINTER `typedef void* POINTER`
No longer used.

20.157 types.h

[Go to the documentation of this file.](#)

```
1
5 #ifndef TYPES_INCLUDE
6 #define TYPES_INCLUDE
7
8 #include <asm/types.h>
9
10 #ifndef NULL
11     #define NULL (void *)0
12 #endif
13
16 #define FALSE 0
19 #define TRUE 1
20
23 typedef void * POINTER;
24
25 #endif
```

Index

- `_AUD3WAVERAM`
 - `hardware.h`, [259](#)
- `_BIOS`
 - `sms.h`, [434](#)
- `_HRAM`
 - `hardware.h`, [259](#)
- `_IO`
 - `hardware.h`, [259](#)
- `_MEGADUCK_LAPTOP_IO_H`
 - `laptop_io.h`, [112](#)
- `_OAMRAM`
 - `hardware.h`, [259](#)
- `_RAM`
 - `hardware.h`, [259](#)
- `_RAMBANK`
 - `hardware.h`, [259](#)
- `_SCRN0`
 - `hardware.h`, [258](#)
- `_SCRN1`
 - `hardware.h`, [259](#)
- `_SRAM`
 - `hardware.h`, [259](#)
- `_SYSTEM`
 - `hardware.h`, [275](#)
 - `msx.h`, [350](#)
 - `nes.h`, [397](#)
 - `sms.h`, [434](#)
- `_VRAM`
 - `hardware.h`, [258](#)
- `_VRAM8000`
 - `hardware.h`, [258](#)
- `_VRAM8800`
 - `hardware.h`, [258](#)
- `_VRAM9000`
 - `hardware.h`, [258](#)
- `__BYTES`
 - `hardware.h`, [245](#), [270](#), [287](#)
- `__BYTE_REG`
 - `hardware.h`, [245](#), [270](#), [287](#)
- `__GBDK_VERSION`
 - `version.h`, [236](#)
- `__HandleCrash`
 - `crash_handler.h`, [145](#)
- `__PTRDIFF_T_DEFINED`
 - `stddef.h`, [443](#)
- `__READ_VDP_REG`
 - `msx.h`, [330](#)
 - `sms.h`, [414](#)
- `__REG`
 - `hardware.h`, [245](#), [279](#), [281](#), [282](#)
- `__SHADOW_REG`
 - `hardware.h`, [279](#)
- `__SIZE_T_DEFINED`
 - `stddef.h`, [443](#)
 - `types.h`, [478](#), [480](#), [484](#)
- `__WCHAR_T_DEFINED`
 - `stddef.h`, [443](#)
- `__WRITE_VDP_REG`
 - `msx.h`, [329](#)
 - `sms.h`, [414](#)
- `__WRITE_VDP_REG_UNSAFE`
 - `msx.h`, [329](#)
 - `sms.h`, [414](#)
- `__assert`
 - `assert.h`, [108](#)
- `__bool_true_false_are_defined`
 - `stdbool.h`, [442](#)
- `__call_banked`
 - `far_ptr.h`, [228](#)
- `__call_banked_addr`
 - `far_ptr.h`, [228](#)
- `__call_banked_bank`
 - `far_ptr.h`, [228](#)
- `__call_banked_ptr`
 - `far_ptr.h`, [228](#)
- `__current_base_prop`
 - `metasprites.h`, [306](#), [317](#)
- `__current_base_tile`
 - `metasprites.h`, [306](#), [311](#), [317](#), [323](#)
- `__current metasprite`
 - `metasprites.h`, [306](#), [311](#), [317](#), [323](#)
- `__far_ptr`, [96](#)
 - `fn`, [96](#)
 - `ofs`, [96](#)
 - `ptr`, [96](#)
 - `seg`, [96](#)
 - `segfn`, [96](#)
 - `segofs`, [96](#)
- `__memcpy`
 - `string.h`, [461](#)
- `__rand_seed`
 - `rand.h`, [405](#)
- `__render_shadow_OAM`
 - `metasprites.h`, [306](#), [311](#), [317](#), [323](#)
- `__setjmp`
 - `setjmp.h`, [407](#)
- `__cpu`
 - `gb.h`, [205](#)
- `__current_1bpp_colors`
 - `gb.h`, [206](#)
 - `msx.h`, [351](#)
 - `nes.h`, [397](#)
 - `sms.h`, [434](#)
- `__current_2bpp_palette`
 - `msx.h`, [351](#)
 - `sms.h`, [434](#)
- `__current_bank`
 - `gb.h`, [206](#)
 - `msx.h`, [351](#)
 - `nes.h`, [397](#)

- sms.h, [417](#)
- _fixed, [96](#)
 - b, [97](#)
 - h, [97](#)
 - l, [97](#)
 - w, [97](#)
- _io_in
 - gb.h, [206](#)
- _io_out
 - gb.h, [206](#)
- _io_status
 - gb.h, [206](#)
- _is_GBA
 - gb.h, [205](#)
- _lcd_scanline
 - hardware.h, [282](#)
- _map_tile_offset
 - gb.h, [206](#)
 - msx.h, [352](#)
 - sms.h, [434](#)
- _shadow_OAM_OFF
 - msx.h, [352](#)
 - sms.h, [435](#)
- _shadow_OAM_base
 - gb.h, [206](#)
 - msx.h, [352](#)
 - nes.h, [397](#)
 - sms.h, [435](#)
- _sprites_OFF
 - sms.h, [435](#)
- _submap_tile_offset
 - gb.h, [206](#)
 - msx.h, [352](#)
 - sms.h, [434](#)
- _switch_prg0
 - nes.h, [397](#)
- _vbl_done
 - gb.h, [205](#)
 - msx.h, [351](#)
 - sms.h, [434](#)
- abs
 - stdlib.h, [456](#)
- add_JOY
 - gb.h, [175](#)
 - msx.h, [337](#)
 - sms.h, [422](#)
- add_LCD
 - gb.h, [174](#)
 - msx.h, [337](#)
 - nes.h, [372](#)
 - sms.h, [422](#)
- add_low_priority_TIM
 - gb.h, [174](#)
- add_SIO
 - gb.h, [175](#)
 - msx.h, [337](#)
 - sms.h, [422](#)
- add_TIM
 - gb.h, [174](#)
 - msx.h, [337](#)
 - nes.h, [372](#)
 - sms.h, [422](#)
- add_VBL
 - gb.h, [173](#)
 - msx.h, [337](#)
 - nes.h, [371](#)
 - sms.h, [422](#)
- AND
 - drawing.h, [147](#)
- arand
 - rand.h, [405](#)
- assert
 - assert.h, [107](#)
- assert.h
 - __assert, [108](#)
 - assert, [107](#)
- AT
 - types.h, [482](#)
- atoi
 - stdlib.h, [456](#)
- atol
 - stdlib.h, [457](#)
- atomic_flag, [97](#)
 - flag, [97](#)
- atomic_flag_clear
 - stdatomic.h, [441](#)
- atomic_flag_test_and_set
 - stdatomic.h, [441](#)
- AUD1SWEEP_DOWN
 - hardware.h, [248](#)
- AUD1SWEEP_LENGTH
 - hardware.h, [248](#)
- AUD1SWEEP_TIME
 - hardware.h, [248](#)
- AUD1SWEEP_UP
 - hardware.h, [248](#)
- AUD3WAVE
 - hardware.h, [261](#)
- AUD4POLY_WIDTH_15BIT
 - hardware.h, [249](#)
- AUD4POLY_WIDTH_7BIT
 - hardware.h, [249](#)
- AUDENA_OFF
 - hardware.h, [250](#)
- AUDENA_ON
 - hardware.h, [250](#)
- AUDENV_DOWN
 - hardware.h, [256](#)
- AUDENV_LENGTH
 - hardware.h, [256](#)
- AUDENV_UP
 - hardware.h, [256](#)
- AUDENV_VOL
 - hardware.h, [256](#)
- AUDHIGH_LENGTH_OFF
 - hardware.h, [256](#)

AUDHIGH_LENGTH_ON
 hardware.h, 256

AUDHIGH_RESTART
 hardware.h, 256

AUDLEN_DUTY_12_5
 hardware.h, 256

AUDLEN_DUTY_25
 hardware.h, 256

AUDLEN_DUTY_50
 hardware.h, 256

AUDLEN_DUTY_75
 hardware.h, 256

AUDLEN_LENGTH
 hardware.h, 256

AUDTERM_1_LEFT
 hardware.h, 249

AUDTERM_1_RIGHT
 hardware.h, 250

AUDTERM_2_LEFT
 hardware.h, 249

AUDTERM_2_RIGHT
 hardware.h, 250

AUDTERM_3_LEFT
 hardware.h, 249

AUDTERM_3_RIGHT
 hardware.h, 250

AUDTERM_4_LEFT
 hardware.h, 249

AUDTERM_4_RIGHT
 hardware.h, 250

AUDVOL_VIN_LEFT
 hardware.h, 249

AUDVOL_VIN_RIGHT
 hardware.h, 249

AUDVOL_VOL_LEFT
 hardware.h, 249

AUDVOL_VOL_RIGHT
 hardware.h, 249

b
 _fixed, 97
 emu_debug.h, 155
 gb.h, 206
 msx.h, 351
 sms.h, 434

BANK
 gb.h, 166
 incbin.h, 233
 msx.h, 332
 nes.h, 366
 sms.h, 417

BANKED
 types.h, 482

BANKREF
 gb.h, 166
 msx.h, 333
 nes.h, 366
 sms.h, 417

BANKREF_EXTERN
 gb.h, 167
 msx.h, 333
 nes.h, 367
 sms.h, 418

BCD
 bcd.h, 132, 135, 137

bcd.h
 BCD, 132, 135, 137
 bcd2text, 133, 135, 138
 bcd_add, 132, 135, 137
 BCD_HEX, 132, 134, 137
 bcd_sub, 133, 135, 137
 MAKE_BCD, 132, 134, 137
 uint2bcd, 132, 135, 137

bcd2text
 bcd.h, 133, 135, 138

bcd_add
 bcd.h, 132, 135, 137

BCD_HEX
 bcd.h, 132, 134, 137

bcd_sub
 bcd.h, 133, 135, 137

BCPD_REG
 hardware.h, 263

BCPS_REG
 hardware.h, 263

BCPSF_AUTOINC
 hardware.h, 255

BGB_BREAKPOINT
 emu_debug.h, 154

BGB_MESSAGE
 emu_debug.h, 153

BGB_printf
 emu_debug.h, 154

BGB_PROFILE_BEGIN
 emu_debug.h, 153

BGB_PROFILE_END
 emu_debug.h, 153

BGB_profiler_message
 emu_debug.h, 154

BGB_TEXT
 emu_debug.h, 153

BGP_REG
 hardware.h, 262

bkg_scroll_x
 hardware.h, 282

bkg_scroll_y
 hardware.h, 282

BKGF_BANK0
 hardware.h, 254

BKGF_BANK1
 hardware.h, 254

BKGF_CGB_PAL0
 hardware.h, 254

BKGF_CGB_PAL1
 hardware.h, 254

BKGF_CGB_PAL2
 hardware.h, 254

- BKGF_CGB_PAL3
 - hardware.h, [254](#)
- BKGF_CGB_PAL4
 - hardware.h, [254](#)
- BKGF_CGB_PAL5
 - hardware.h, [254](#)
- BKGF_CGB_PAL6
 - hardware.h, [254](#)
- BKGF_CGB_PAL7
 - hardware.h, [254](#)
- BKGF_PRI
 - hardware.h, [253](#)
- BKGF_XFLIP
 - hardware.h, [254](#)
- BKGF_YFLIP
 - hardware.h, [254](#)
- BLACK
 - drawing.h, [147](#)
- bool
 - stdbool.h, [442](#)
- BOOLEAN
 - types.h, [482](#)
- box
 - drawing.h, [149](#)
- BP_SIZE
 - setjmp.h, [407](#)
- BPX_SIZE
 - setjmp.h, [407](#)
- bsearch
 - stdlib.h, [458](#)
- BYTE
 - types.h, [482](#)
- c
 - emu_debug.h, [155](#)
 - gb.h, [205](#)
 - gbdecompress.h, [217](#)
 - msx.h, [351](#)
 - rand.h, [406](#)
 - sgb.h, [224](#)
 - sms.h, [434](#)
 - string.h, [469](#)
- calloc
 - stdlib.h, [458](#)
- cancel_pending_interrupts
 - gb.h, [176](#)
 - msx.h, [337](#)
 - sms.h, [422](#)
- cgb.h
 - cgb_compatibility, [144](#)
 - cpu_fast, [144](#)
 - cpu_slow, [144](#)
 - palette_color_t, [142](#)
 - RGB, [140](#)
 - RGB8, [140](#)
 - RGB_AQUA, [141](#)
 - RGB_BLACK, [141](#)
 - RGB_BLUE, [141](#)
 - RGB_BROWN, [142](#)
 - RGB_CYAN, [141](#)
 - RGB_DARKBLUE, [141](#)
 - RGB_DARKGRAY, [141](#)
 - RGB_DARKGREEN, [141](#)
 - RGB_DARKRED, [141](#)
 - RGB_DARKYELLOW, [141](#)
 - RGB_GREEN, [141](#)
 - RGB_LIGHTFLESH, [142](#)
 - RGB_LIGHTGRAY, [141](#)
 - RGB_ORANGE, [142](#)
 - RGB_PINK, [141](#)
 - RGB_PURPLE, [141](#)
 - RGB_RED, [141](#)
 - RGB_TEAL, [142](#)
 - RGB_WHITE, [142](#)
 - RGB_YELLOW, [141](#)
 - RGBHTML, [140](#)
 - set_bkg_palette, [142](#)
 - set_bkg_palette_entry, [143](#)
 - set_default_palette, [144](#)
 - set_sprite_palette, [142](#)
 - set_sprite_palette_entry, [143](#)
- cgb_compatibility
 - cgb.h, [144](#)
 - sms.h, [424](#)
- CGB_TYPE
 - gb.h, [165](#)
- CHAR_BIT
 - limits.h, [237](#)
- CHAR_MAX
 - limits.h, [238](#)
- CHAR_MIN
 - limits.h, [238](#)
- circle
 - drawing.h, [149](#)
- clock
 - time.h, [475](#)
- clock_t
 - types.h, [479](#), [480](#), [485](#)
- CLOCKS_PER_SEC
 - time.h, [474](#)
- cls
 - console.h, [225](#)
- color
 - drawing.h, [150](#)
- COMPAT_PALETTE
 - gb.h, [171](#)
 - msx.h, [334](#)
 - nes.h, [369](#)
 - sms.h, [419](#)
- console.h
 - cls, [225](#)
 - gotoxy, [225](#)
 - posx, [225](#)
 - posy, [225](#)
 - setchar, [225](#)
- cpu_fast
 - cgb.h, [144](#)

- msx.h, [340](#)
- sms.h, [424](#)
- cpu_slow
 - cgb.h, [144](#)
- crash_handler.h
 - __HandleCrash, [145](#)
- CRITICAL
 - types.h, [482](#)
- ctype.h
 - isalpha, [109](#)
 - isdigit, [109](#)
 - islower, [109](#)
 - isspace, [109](#)
 - isupper, [109](#)
 - tolower, [110](#)
 - toupper, [110](#)
- CURRENT_BANK
 - gb.h, [166](#)
 - msx.h, [332](#)
 - nes.h, [366](#)
 - sms.h, [417](#)
- d
 - gb.h, [205](#)
 - msx.h, [351](#)
 - sms.h, [434](#)
- delay
 - gb.h, [177](#)
 - msx.h, [338](#)
 - nes.h, [373](#)
 - sms.h, [423](#)
- DEVICE_SCREEN_BUFFER_HEIGHT
 - hardware.h, [258](#), [280](#)
- DEVICE_SCREEN_BUFFER_WIDTH
 - hardware.h, [258](#), [280](#)
- DEVICE_SCREEN_HEIGHT
 - hardware.h, [258](#), [280](#)
- DEVICE_SCREEN_MAP_ENTRY_SIZE
 - hardware.h, [258](#), [280](#)
- DEVICE_SCREEN_PX_HEIGHT
 - hardware.h, [258](#), [274](#), [280](#), [296](#)
- DEVICE_SCREEN_PX_WIDTH
 - hardware.h, [258](#), [274](#), [280](#), [296](#)
- DEVICE_SCREEN_WIDTH
 - hardware.h, [258](#), [280](#)
- DEVICE_SCREEN_X_OFFSET
 - hardware.h, [257](#), [280](#)
- DEVICE_SCREEN_Y_OFFSET
 - hardware.h, [257](#), [280](#)
- DEVICE_SPRITE_PX_OFFSET_X
 - hardware.h, [258](#), [280](#)
- DEVICE_SPRITE_PX_OFFSET_Y
 - hardware.h, [258](#), [280](#)
- DEVICE_SUPPORTS_COLOR
 - gb.h, [166](#)
 - msx.h, [332](#)
 - sms.h, [417](#)
- DEVICE_WINDOW_PX_OFFSET_X
 - hardware.h, [258](#), [280](#)
- DEVICE_WINDOW_PX_OFFSET_Y
 - hardware.h, [258](#), [280](#)
- disable_interrupts
 - gb.h, [178](#)
 - msx.h, [339](#)
 - nes.h, [375](#)
 - sms.h, [424](#)
- DISABLE_OAM_DMA
 - gb.h, [171](#)
 - nes.h, [369](#)
- DISABLE_RAM
 - gb.h, [168](#)
 - msx.h, [334](#)
 - nes.h, [368](#)
 - sms.h, [419](#)
- DISABLE_RAM_MBC1
 - gb.h, [169](#)
- DISABLE_RAM_MBC5
 - gb.h, [170](#)
- DISABLE_VBL_TRANSFER
 - gb.h, [171](#)
 - msx.h, [334](#)
 - nes.h, [369](#)
 - sms.h, [419](#)
- DISPLAY_OFF
 - gb.h, [170](#)
 - msx.h, [331](#)
 - nes.h, [368](#)
 - sms.h, [416](#)
- display_off
 - gb.h, [179](#)
 - msx.h, [338](#)
 - nes.h, [376](#)
 - sms.h, [422](#)
- DISPLAY_ON
 - gb.h, [170](#)
 - msx.h, [331](#)
 - nes.h, [368](#)
 - sms.h, [416](#)
- display_on
 - nes.h, [376](#)
- DIV_REG
 - hardware.h, [260](#)
 - msx.h, [332](#)
 - sms.h, [417](#)
- DKGREY
 - drawing.h, [147](#)
- DMA_REG
 - hardware.h, [262](#)
- DMG_BLACK
 - gb.h, [164](#)
 - nes.h, [365](#)
- DMG_DARK_GRAY
 - gb.h, [164](#)
 - nes.h, [365](#)
- DMG_LITE_GRAY
 - gb.h, [164](#)
 - nes.h, [365](#)

DMG_PALETTE
 gb.h, 164
 nes.h, 365
 DMG_TYPE
 gb.h, 165
 DMG_WHITE
 gb.h, 164
 nes.h, 365
 docs/pages/01_getting_started.md, 103
 docs/pages/02_links_and_tools.md, 103
 docs/pages/03_using_gbdk.md, 103
 docs/pages/04_coding_guidelines.md, 103
 docs/pages/05_banking_mbc5.md, 103
 docs/pages/06_toolchain.md, 103
 docs/pages/06b_supported_consoles.md, 103
 docs/pages/07_sample_programs.md, 103
 docs/pages/08_faq.md, 103
 docs/pages/09_migrating_new_versions.md, 103
 docs/pages/10_release_notes.md, 103
 docs/pages/20_toolchain_settings.md, 103
 docs/pages/docs_index.md, 103
 draw_image
 drawing.h, 149
 drawing.h
 AND, 147
 BLACK, 147
 box, 149
 circle, 149
 color, 150
 DKGREY, 147
 draw_image, 149
 getpix, 150
 gotogxy, 150
 gprint, 148
 gprintf, 148
 gprintln, 148
 gprintn, 148
 GRAPHICS_HEIGHT, 147
 GRAPHICS_WIDTH, 147
 line, 149
 LTGREY, 147
 M_FILL, 147
 M_NOFILL, 147
 OR, 147
 plot, 149
 plot_point, 149
 SIGNED, 147
 SOLID, 147
 switch_data, 149
 UNSIGNED, 148
 WHITE, 147
 wrtchr, 150
 XOR, 147
 dtile
 metasprite_t, 100
 duck_check_model
 model.h, 131
 DUCK_IO_CMD_ABORT_OR_FAIL
 laptop_io.h, 112
 DUCK_IO_CMD_DONE_OR_OK
 laptop_io.h, 112
 DUCK_IO_CMD_GET_KEYS
 laptop_io.h, 112
 DUCK_IO_CMD_GET_RTC
 laptop_io.h, 112
 DUCK_IO_CMD_INIT_START
 laptop_io.h, 112
 DUCK_IO_CMD_PLAY_SPEECH
 laptop_io.h, 112
 DUCK_IO_CMD_PRINT_INIT_EXT_IO
 laptop_io.h, 112
 DUCK_IO_CMD_PRINT_SEND_BYTES
 laptop_io.h, 112
 DUCK_IO_CMD_RUN_CART_IN_SLOT
 laptop_io.h, 112
 DUCK_IO_CMD_SET_RTC
 laptop_io.h, 112
 duck_io_enable_read_byte
 laptop_io.h, 115
 DUCK_IO_KBD_FLAGS
 laptop_io.h, 114
 DUCK_IO_KBD_KEYCODE
 laptop_io.h, 114
 DUCK_IO_KEY_0
 laptop_keycodes.h, 123
 DUCK_IO_KEY_1
 laptop_keycodes.h, 122
 DUCK_IO_KEY_2
 laptop_keycodes.h, 123
 DUCK_IO_KEY_3
 laptop_keycodes.h, 123
 DUCK_IO_KEY_4
 laptop_keycodes.h, 123
 DUCK_IO_KEY_5
 laptop_keycodes.h, 123
 DUCK_IO_KEY_6
 laptop_keycodes.h, 123
 DUCK_IO_KEY_7
 laptop_keycodes.h, 123
 DUCK_IO_KEY_8
 laptop_keycodes.h, 123
 DUCK_IO_KEY_9
 laptop_keycodes.h, 123
 DUCK_IO_KEY_A
 laptop_keycodes.h, 124
 DUCK_IO_KEY_ARROW_DOWN
 laptop_keycodes.h, 126
 DUCK_IO_KEY_ARROW_LEFT
 laptop_keycodes.h, 126
 DUCK_IO_KEY_ARROW_RIGHT
 laptop_keycodes.h, 126
 DUCK_IO_KEY_ARROW_UP
 laptop_keycodes.h, 126
 DUCK_IO_KEY_B
 laptop_keycodes.h, 125
 DUCK_IO_KEY_BACKSPACE

laptop_keycodes.h, 123
DUCK_IO_KEY_BACKTICK
laptop_keycodes.h, 124
DUCK_IO_KEY_BASE
laptop_keycodes.h, 122
DUCK_IO_KEY_BASE_BIT
laptop_keycodes.h, 122
DUCK_IO_KEY_C
laptop_keycodes.h, 125
DUCK_IO_KEY_COMMA
laptop_keycodes.h, 125
DUCK_IO_KEY_D
laptop_keycodes.h, 124
DUCK_IO_KEY_DASH
laptop_keycodes.h, 125
DUCK_IO_KEY_DELETE
laptop_keycodes.h, 125
DUCK_IO_KEY_DIVIDE
laptop_keycodes.h, 126
DUCK_IO_KEY_E
laptop_keycodes.h, 123
DUCK_IO_KEY_ENTER
laptop_keycodes.h, 124
DUCK_IO_KEY_EQUALS
laptop_keycodes.h, 126
DUCK_IO_KEY_ESCAPE
laptop_keycodes.h, 122
DUCK_IO_KEY_EXCLAMATION_FLIPPED
laptop_keycodes.h, 123
DUCK_IO_KEY_F
laptop_keycodes.h, 124
DUCK_IO_KEY_F1
laptop_keycodes.h, 122
DUCK_IO_KEY_F10
laptop_keycodes.h, 122
DUCK_IO_KEY_F11
laptop_keycodes.h, 122
DUCK_IO_KEY_F12
laptop_keycodes.h, 122
DUCK_IO_KEY_F2
laptop_keycodes.h, 122
DUCK_IO_KEY_F3
laptop_keycodes.h, 122
DUCK_IO_KEY_F4
laptop_keycodes.h, 122
DUCK_IO_KEY_F5
laptop_keycodes.h, 122
DUCK_IO_KEY_F6
laptop_keycodes.h, 122
DUCK_IO_KEY_F7
laptop_keycodes.h, 122
DUCK_IO_KEY_F8
laptop_keycodes.h, 122
DUCK_IO_KEY_F9
laptop_keycodes.h, 122
DUCK_IO_KEY_FLAG_CAPSLOCK
laptop_keycodes.h, 121
DUCK_IO_KEY_FLAG_CAPSLOCK_BIT
laptop_keycodes.h, 121
DUCK_IO_KEY_FLAG_KEY_REPEAT
laptop_keycodes.h, 121
DUCK_IO_KEY_FLAG_KEY_REPEAT_BIT
laptop_keycodes.h, 121
DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT
laptop_keycodes.h, 121
DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT_BIT
laptop_keycodes.h, 122
DUCK_IO_KEY_FLAG_SHIFT
laptop_keycodes.h, 121
DUCK_IO_KEY_FLAG_SHIFT_BIT
laptop_keycodes.h, 121
DUCK_IO_KEY_G
laptop_keycodes.h, 124
DUCK_IO_KEY_H
laptop_keycodes.h, 124
DUCK_IO_KEY_HELP
laptop_keycodes.h, 123
DUCK_IO_KEY_I
laptop_keycodes.h, 124
DUCK_IO_KEY_J
laptop_keycodes.h, 124
DUCK_IO_KEY_K
laptop_keycodes.h, 124
DUCK_IO_KEY_L
laptop_keycodes.h, 124
DUCK_IO_KEY_LAST_KEY
laptop_keycodes.h, 128
DUCK_IO_KEY_LESS_THAN
laptop_keycodes.h, 125
DUCK_IO_KEY_M
laptop_keycodes.h, 125
DUCK_IO_KEY_MAYBE_RX_NOT_A_KEY
laptop_keycodes.h, 128
DUCK_IO_KEY_MAYBE_SYST_CODES_START
laptop_keycodes.h, 128
DUCK_IO_KEY_MEMORY_MINUS
laptop_keycodes.h, 125
DUCK_IO_KEY_MEMORY_PLUS
laptop_keycodes.h, 126
DUCK_IO_KEY_MEMORY_RECALL
laptop_keycodes.h, 126
DUCK_IO_KEY_MINUS
laptop_keycodes.h, 126
DUCK_IO_KEY_MULTIPLY
laptop_keycodes.h, 126
DUCK_IO_KEY_N
laptop_keycodes.h, 125
DUCK_IO_KEY_N_TILDE
laptop_keycodes.h, 124
DUCK_IO_KEY_O
laptop_keycodes.h, 124
DUCK_IO_KEY_O_OVER_LINE
laptop_keycodes.h, 125
DUCK_IO_KEY_P
laptop_keycodes.h, 124
DUCK_IO_KEY_PAGE_DOWN

[laptop_keycodes.h](#), [125](#)
 DUCK_IO_KEY_PAGE_UP
[laptop_keycodes.h](#), [125](#)
 DUCK_IO_KEY_PERIOD
[laptop_keycodes.h](#), [125](#)
 DUCK_IO_KEY_PIANO_DO
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_DO_2
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_DO_2_SHARP
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_PIANO_DO_SHARP
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_PIANO_FA
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_FA_2
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_FA_2_SHARP
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_FA_SHARP
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_PIANO_LA
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_LA_2
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_LA_2_SHARP
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_LA_SHARP
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_PIANO_MI
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_MI_2
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_RE
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_RE_2
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_RE_2_SHARP
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_RE_SHARP
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_PIANO_SI
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_SI_2
[laptop_keycodes.h](#), [128](#)
 DUCK_IO_KEY_PIANO_SOL
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_SOL_2
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_SOL_2_SHARP
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_PIANO_SOL_SHARP
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_PLUS
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_PRINTSCREEN_RIGHT
[laptop_keycodes.h](#), [127](#)
 DUCK_IO_KEY_Q

[laptop_keycodes.h](#), [123](#)
 DUCK_IO_KEY_R
[laptop_keycodes.h](#), [123](#)
 DUCK_IO_KEY_RIGHT_SQ_BRACKET
[laptop_keycodes.h](#), [124](#)
 DUCK_IO_KEY_S
[laptop_keycodes.h](#), [124](#)
 DUCK_IO_KEY_SINGLE_QUOTE
[laptop_keycodes.h](#), [123](#)
 DUCK_IO_KEY_SPACE
[laptop_keycodes.h](#), [125](#)
 DUCK_IO_KEY_SQUAREROOT
[laptop_keycodes.h](#), [126](#)
 DUCK_IO_KEY_T
[laptop_keycodes.h](#), [123](#)
 DUCK_IO_KEY_U
[laptop_keycodes.h](#), [124](#)
 DUCK_IO_KEY_U_UMLAUT
[laptop_keycodes.h](#), [125](#)
 DUCK_IO_KEY_V
[laptop_keycodes.h](#), [125](#)
 DUCK_IO_KEY_W
[laptop_keycodes.h](#), [123](#)
 DUCK_IO_KEY_X
[laptop_keycodes.h](#), [125](#)
 DUCK_IO_KEY_Y
[laptop_keycodes.h](#), [124](#)
 DUCK_IO_KEY_Z
[laptop_keycodes.h](#), [125](#)
[duck_io_laptop_init](#)
[laptop_io.h](#), [115](#)
 DUCK_IO_LEN_KBD_GET
[laptop_io.h](#), [113](#)
 DUCK_IO_LEN_PLAY_SPEECH
[laptop_io.h](#), [113](#)
 DUCK_IO_LEN_RTC_GET
[laptop_io.h](#), [113](#)
 DUCK_IO_LEN_RTC_SET
[laptop_io.h](#), [113](#)
 DUCK_IO_LEN_RX_MAX
[laptop_io.h](#), [113](#)
 DUCK_IO_LEN_TX_MAX
[laptop_io.h](#), [113](#)
 DUCK_IO_PRINTER_FAIL
[laptop_io.h](#), [114](#)
[duck_io_printer_last_status](#)
[laptop_io.h](#), [115](#)
 DUCK_IO_PRINTER_MAYBE_BUSY
[laptop_io.h](#), [114](#)
[duck_io_printer_query](#)
[laptop_io.h](#), [115](#)
 DUCK_IO_PRINTER_TYPE_1_PASS
[laptop_io.h](#), [114](#)
 DUCK_IO_PRINTER_TYPE_2_PASS
[laptop_io.h](#), [114](#)
[duck_io_read_byte_no_timeout](#)
[laptop_io.h](#), [114](#)
[duck_io_read_byte_with_msecs_timeout](#)

- laptop_io.h, [115](#)
- DUCK_IO_REPLY_BOOT_FAIL
 - laptop_io.h, [113](#)
- DUCK_IO_REPLY_BOOT_OK
 - laptop_io.h, [113](#)
- DUCK_IO_REPLY_BOOT_UNSET
 - laptop_io.h, [113](#)
- DUCK_IO_REPLY_BUFFER_XFER_OK
 - laptop_io.h, [113](#)
- DUCK_IO_REPLY_NO_CART_IN_SLOT
 - laptop_io.h, [113](#)
- DUCK_IO_REPLY_SEND_BUFFER_OK
 - laptop_io.h, [113](#)
- DUCK_IO_RTC_AMPM
 - laptop_io.h, [114](#)
- DUCK_IO_RTC_DAY
 - laptop_io.h, [114](#)
- DUCK_IO_RTC_HOUR
 - laptop_io.h, [114](#)
- DUCK_IO_RTC_MIN
 - laptop_io.h, [114](#)
- DUCK_IO_RTC_MON
 - laptop_io.h, [114](#)
- DUCK_IO_RTC_SEC
 - laptop_io.h, [114](#)
- DUCK_IO_RTC_WEEKDAY
 - laptop_io.h, [114](#)
- DUCK_IO_RTC_YEAR
 - laptop_io.h, [113](#)
- duck_io_rx_buf
 - laptop_io.h, [117](#)
- duck_io_rx_buf_len
 - laptop_io.h, [117](#)
- duck_io_rx_byte
 - laptop_io.h, [117](#)
- duck_io_rx_byte_done
 - laptop_io.h, [117](#)
- duck_io_send_byte
 - laptop_io.h, [114](#)
- duck_io_send_byte_and_check_ack_msecs_timeout
 - laptop_io.h, [116](#)
- duck_io_send_cmd_and_buffer
 - laptop_io.h, [116](#)
- duck_io_send_cmd_and_receive_buffer
 - laptop_io.h, [116](#)
- DUCK_IO_SPEECH_CMD_MAX
 - laptop_io.h, [113](#)
- DUCK_IO_SPEECH_CMD_MIN
 - laptop_io.h, [113](#)
- DUCK_IO_TIMEOUT_100_MSEC
 - laptop_io.h, [113](#)
- DUCK_IO_TIMEOUT_200_MSEC
 - laptop_io.h, [113](#)
- DUCK_IO_TIMEOUT_2_MSEC
 - laptop_io.h, [113](#)
- duck_io_tx_buf
 - laptop_io.h, [117](#)
- duck_io_tx_buf_len
 - laptop_io.h, [117](#)
- laptop_io.h, [117](#)
- DWORD
 - types.h, [482](#)
- dx
 - metasprite_t, [100](#)
- dy
 - metasprite_t, [100](#)
- e
 - gb.h, [205](#)
 - msx.h, [351](#)
 - sms.h, [434](#)
- EMPTY_IFLAG
 - gb.h, [163](#)
 - msx.h, [330](#)
 - nes.h, [365](#)
 - sms.h, [414](#)
- EMU_BREAKPOINT
 - emu_debug.h, [154](#)
- emu_debug.h
 - b, [155](#)
 - BGB_BREAKPOINT, [154](#)
 - BGB_MESSAGE, [153](#)
 - BGB_printf, [154](#)
 - BGB_PROFILE_BEGIN, [153](#)
 - BGB_PROFILE_END, [153](#)
 - BGB_profiler_message, [154](#)
 - BGB_TEXT, [153](#)
 - c, [155](#)
 - EMU_BREAKPOINT, [154](#)
 - EMU_fmtbuf, [155](#)
 - EMU_MESSAGE, [152](#)
 - EMU_printf, [154](#)
 - EMU_PROFILE_BEGIN, [153](#)
 - EMU_PROFILE_END, [153](#)
 - EMU_profiler_message, [154](#)
 - EMU_TEXT, [153](#)
- EMU_fmtbuf
 - emu_debug.h, [155](#)
- EMU_MESSAGE
 - emu_debug.h, [152](#)
- EMU_printf
 - emu_debug.h, [154](#)
- EMU_PROFILE_BEGIN
 - emu_debug.h, [153](#)
- EMU_PROFILE_END
 - emu_debug.h, [153](#)
- EMU_profiler_message
 - emu_debug.h, [154](#)
- EMU_TEXT
 - emu_debug.h, [153](#)
- enable_interrupts
 - gb.h, [178](#)
 - msx.h, [339](#)
 - nes.h, [375](#)
 - sms.h, [424](#)
- ENABLE_OAM_DMA
 - gb.h, [172](#)
 - nes.h, [369](#)

ENABLE_RAM
 gb.h, 168
 msx.h, 334
 nes.h, 368
 sms.h, 419
 ENABLE_RAM_MBC1
 gb.h, 168
 ENABLE_RAM_MBC5
 gb.h, 170
 ENABLE_VBL_TRANSFER
 gb.h, 172
 msx.h, 334
 nes.h, 370
 sms.h, 419
 exit
 stdlib.h, 456
 FALSE
 types.h, 486
 false
 stdbool.h, 442
 FAR_CALL
 far_ptr.h, 227
 FAR_FUNC
 far_ptr.h, 227
 FAR_OFS
 far_ptr.h, 227
 FAR_PTR
 far_ptr.h, 228
 far_ptr.h
 __call_banked, 228
 __call_banked_addr, 228
 __call_banked_bank, 228
 __call_banked_ptr, 228
 FAR_CALL, 227
 FAR_FUNC, 227
 FAR_OFS, 227
 FAR_PTR, 228
 FAR_SEG, 227
 TO_FAR_PTR, 226
 to_far_ptr, 228
 FAR_SEG
 far_ptr.h, 227
 fill_bkg_rect
 gb.h, 204
 msx.h, 334
 nes.h, 396
 sms.h, 419
 fill_rect
 gb.h, 172
 msx.h, 345
 nes.h, 370
 sms.h, 430
 fill_rect_compat
 sms.h, 430
 fill_win_rect
 gb.h, 204
 msx.h, 334
 sms.h, 419
 first_tile
 sfont_handle, 102
 fixed
 types.h, 483
 flag
 atomic_flag, 97
 flush_shadow_attributes
 nes.h, 397
 fn
 __far_ptr, 96
 font
 sfont_handle, 102
 font.h
 FONT_128ENCODING, 230
 FONT_256ENCODING, 230
 font_color, 231
 FONT_COMPRESSED, 230
 font_init, 230
 font_load, 230
 FONT_NOENCODING, 230
 font_set, 231
 font_t, 230
 mfont_handle, 230
 pmfont_handle, 230
 FONT_128ENCODING
 font.h, 230
 FONT_256ENCODING
 font.h, 230
 font_color
 font.h, 231
 FONT_COMPRESSED
 font.h, 230
 font_ibm
 List of gbk fonts, 95
 font_ibm_fixed
 List of gbk fonts, 95
 font_init
 font.h, 230
 font_italic
 List of gbk fonts, 95
 font_load
 font.h, 230
 font_min
 List of gbk fonts, 95
 FONT_NOENCODING
 font.h, 230
 font_set
 font.h, 231
 font_spect
 List of gbk fonts, 95
 font_t
 font.h, 230
 free
 stdlib.h, 458
 func
 isr_nested_vector_t, 98
 isr_vector_t, 98
 GAMEBOY

- gb.h, 161
- gb.h
 - _cpu, 205
 - _current_1bpp_colors, 206
 - _current_bank, 206
 - _io_in, 206
 - _io_out, 206
 - _io_status, 206
 - _is_GBA, 205
 - _map_tile_offset, 206
 - _shadow_OAM_base, 206
 - _submap_tile_offset, 206
 - _vbl_done, 205
 - add_JOY, 175
 - add_LCD, 174
 - add_low_priority_TIM, 174
 - add_SIO, 175
 - add_TIM, 174
 - add_VBL, 173
 - b, 206
 - BANK, 166
 - BANKREF, 166
 - BANKREF_EXTERN, 167
 - c, 205
 - cancel_pending_interrupts, 176
 - CGB_TYPE, 165
 - COMPAT_PALETTE, 171
 - CURRENT_BANK, 166
 - d, 205
 - delay, 177
 - DEVICE_SUPPORTS_COLOR, 166
 - disable_interrupts, 178
 - DISABLE_OAM_DMA, 171
 - DISABLE_RAM, 168
 - DISABLE_RAM_MBC1, 169
 - DISABLE_RAM_MBC5, 170
 - DISABLE_VBL_TRANSFER, 171
 - DISPLAY_OFF, 170
 - display_off, 179
 - DISPLAY_ON, 170
 - DMG_BLACK, 164
 - DMG_DARK_GRAY, 164
 - DMG_LITE_GRAY, 164
 - DMG_PALETTE, 164
 - DMG_TYPE, 165
 - DMG_WHITE, 164
 - e, 205
 - EMPTY_IFLAG, 163
 - enable_interrupts, 178
 - ENABLE_OAM_DMA, 172
 - ENABLE_RAM, 168
 - ENABLE_RAM_MBC1, 168
 - ENABLE_RAM_MBC5, 170
 - ENABLE_VBL_TRANSFER, 172
 - fill_bkg_rect, 204
 - fill_rect, 172
 - fill_win_rect, 204
 - GAMEBOY, 161
 - GBA_DETECTED, 166
 - GBA_NOT_DETECTED, 165
 - get_bkg_data, 182
 - get_bkg_tile_xy, 189
 - get_bkg_tiles, 187
 - get_bkg_xy_addr, 180
 - get_data, 200
 - get_mode, 176
 - get_sprite_data, 196
 - get_sprite_prop, 198
 - get_sprite_tile, 197
 - get_system, 176
 - get_tiles, 202
 - get_vram_byte, 180
 - get_win_data, 191
 - get_win_tile_xy, 194
 - get_win_tiles, 194
 - get_win_xy_addr, 190
 - h, 205
 - HARDWARE_SPRITE_CAN_FLIP_X, 172
 - HARDWARE_SPRITE_CAN_FLIP_Y, 172
 - HIDE_BKG, 170
 - HIDE_LEFT_COLUMN, 170
 - hide_sprite, 199
 - HIDE_SPRITES, 171
 - HIDE_WIN, 171
 - hiramcpy, 180
 - init_bkg, 204
 - init_win, 203
 - int_handler, 172
 - IO_ERROR, 166
 - IO_IDLE, 166
 - IO_RECEIVING, 166
 - IO_SENDING, 166
 - J_A, 161
 - J_B, 161
 - J_DOWN, 161
 - J_LEFT, 161
 - J_RIGHT, 161
 - J_SELECT, 161
 - J_START, 162
 - J_UP, 161
 - JOY_IFLAG, 163
 - joypad, 177
 - joypad_ex, 178
 - joypad_init, 177
 - l, 205
 - LCD_IFLAG, 163
 - M_DRAWING, 162
 - M_NO_INTERP, 162
 - M_NO_SCROLL, 162
 - M_TEXT_INOUT, 162
 - M_TEXT_OUT, 162
 - MAX_HARDWARE_SPRITES, 172
 - MAXWNDPOSX, 165
 - MAXWNDPOSY, 165
 - MGB_TYPE, 165
 - MINWNDPOSX, 164

MINWNDPOSY, 165
mode, 176
move_bkg, 189
move_sprite, 198
move_win, 195
NINTENDO, 161
nowait_int_handler, 175
OAM_item_t, 172
receive_byte, 176
refresh_OAM, 179
remove_JOY, 173
remove_LCD, 173
remove_SIO, 173
remove_TIM, 173
remove_VBL, 172
reset, 179
S_BANK, 162
S_FLIPX, 162
S_FLIPY, 162
S_PAL, 163
S_PALETTE, 162
S_PRIORITY, 163
SCREENHEIGHT, 164
SCREENWIDTH, 164
scroll_bkg, 189
scroll_sprite, 199
scroll_win, 195
send_byte, 176
set_1bpp_colors, 181
set_1bpp_colors_ex, 180
set_2bpp_palette, 180
set_attribute_xy, 171
set_bkg_1bpp_data, 182
set_bkg_2bpp_data, 171
set_bkg_attribute_xy, 188
set_bkg_attributes, 184
set_bkg_based_submap, 186
set_bkg_based_tiles, 184
set_bkg_data, 181
set_bkg_native_data, 203
set_bkg_submap, 185
set_bkg_submap_attributes, 187
set_bkg_tile_xy, 188
set_bkg_tiles, 183
SET_BORDER_COLOR, 170
set_data, 199
set_interrupts, 179
set_native_tile_data, 202
SET_SHADOW_OAM_ADDRESS, 197
set_sprite_1bpp_data, 196
set_sprite_2bpp_data, 171
set_sprite_data, 195
set_sprite_native_data, 203
set_sprite_prop, 197
set_sprite_tile, 197
set_tile_data, 201
set_tile_map, 171
set_tile_submap, 171
set_tile_xy, 171
set_tiles, 201
set_vram_byte, 180
set_win_1bpp_data, 190
set_win_based_submap, 193
set_win_based_tiles, 192
set_win_data, 190
set_win_submap, 192
set_win_tile_xy, 194
set_win_tiles, 191
shadow_OAM, 206
SHOW_BKG, 170
SHOW_LEFT_COLUMN, 170
SHOW_SPRITES, 171
SHOW_WIN, 170
SIO_IFLAG, 163
SPRITES_8x16, 171
SPRITES_8x8, 171
SWITCH_16_8_MODE_MBC1, 169
SWITCH_4_32_MODE_MBC1, 169
SWITCH_RAM, 167
SWITCH_RAM_MBC1, 168
SWITCH_RAM_MBC5, 169
SWITCH_ROM, 167
SWITCH_ROM_MBC1, 168
SWITCH_ROM_MBC5, 169
SWITCH_ROM_MBC5_8M, 169
SWITCH_ROM_MEGADUCK, 168
sys_time, 205
SYSTEM_50HZ, 161
SYSTEM_60HZ, 161
TIM_IFLAG, 163
VBL_DONE, 166
VBL_IFLAG, 163
vmemcpy, 200
vmemset, 204
vsync, 179
wait_int_handler, 175
wait_vbl_done, 179
waitpad, 177
waitpadup, 177
gb_decompress
 gbdecompress.h, 214, 216
gb_decompress_bkg_data
 gbdecompress.h, 214
gb_decompress_sprite_data
 gbdecompress.h, 215
gb_decompress_win_data
 gbdecompress.h, 215
GBA_DETECTED
 gb.h, 166
GBA_NOT_DETECTED
 gb.h, 165
gbdecompress.h
 c, 217
 gb_decompress, 214, 216
 gb_decompress_bkg_data, 214
 gb_decompress_sprite_data, 215

gb_decompress_win_data, 215

gbdk-lib/include/asm/mos6502/provides.h, 103

gbdk-lib/include/asm/mos6502/stdarg.h, 104, 105

gbdk-lib/include/asm/mos6502/string.h, 460, 465

gbdk-lib/include/asm/mos6502/types.h, 478, 479

gbdk-lib/include/asm/sm83/provides.h, 103, 104

gbdk-lib/include/asm/sm83/stdarg.h, 105, 106

gbdk-lib/include/asm/sm83/string.h, 465, 469

gbdk-lib/include/asm/sm83/types.h, 479, 480

gbdk-lib/include/asm/types.h, 481, 483

gbdk-lib/include/asm/z80/provides.h, 104

gbdk-lib/include/asm/z80/stdarg.h, 106, 107

gbdk-lib/include/asm/z80/string.h, 470, 473

gbdk-lib/include/asm/z80/types.h, 484, 485

gbdk-lib/include/assert.h, 107, 108

gbdk-lib/include/ctype.h, 108, 110

gbdk-lib/include/duck/laptop_io.h, 110, 117

gbdk-lib/include/duck/laptop_keycodes.h, 119, 128

gbdk-lib/include/duck/model.h, 130, 131

gbdk-lib/include/gb/bcd.h, 131, 133

gbdk-lib/include/gb/bgb_emu.h, 138

gbdk-lib/include/gb/cgb.h, 139, 145

gbdk-lib/include/gb/crash_handler.h, 145, 146

gbdk-lib/include/gb/drawing.h, 146, 150

gbdk-lib/include/gb/emu_debug.h, 151

gbdk-lib/include/gb/gb.h, 156, 206

gbdk-lib/include/gb/gbdecompress.h, 213, 215

gbdk-lib/include/gb/hardware.h, 239, 264

gbdk-lib/include/gb/hblankcpy.h, 217, 218

gbdk-lib/include/gb/isr.h, 219, 221

gbdk-lib/include/gb/metasprites.h, 300, 307

gbdk-lib/include/gb/sgb.h, 221, 224

gbdk-lib/include/gbdk/bcd.h, 134

gbdk-lib/include/gbdk/console.h, 224, 225

gbdk-lib/include/gbdk/emu_debug.h, 151, 155

gbdk-lib/include/gbdk/far_ptr.h, 226, 228

gbdk-lib/include/gbdk/font.h, 229, 231

gbdk-lib/include/gbdk/gbdecompress.h, 216

gbdk-lib/include/gbdk/gbdk-lib.h, 232

gbdk-lib/include/gbdk/incbin.h, 232, 234

gbdk-lib/include/gbdk/metasprites.h, 308

gbdk-lib/include/gbdk/platform.h, 234

gbdk-lib/include/gbdk/rledecompress.h, 235, 236

gbdk-lib/include/gbdk/version.h, 236

gbdk-lib/include/gbdk/zx0decompress.h, 236, 237

gbdk-lib/include/limits.h, 237, 238

gbdk-lib/include/msx/hardware.h, 269, 276

gbdk-lib/include/msx/metasprites.h, 308, 311

gbdk-lib/include/msx/msx.h, 324, 352

gbdk-lib/include/nes/bcd.h, 134, 136

gbdk-lib/include/nes/hardware.h, 277, 282

gbdk-lib/include/nes/metasprites.h, 312, 317

gbdk-lib/include/nes/nes.h, 358, 398

gbdk-lib/include/nes/rgb_to_nes_macro.h, 403

gbdk-lib/include/rand.h, 404, 406

gbdk-lib/include/setjmp.h, 406, 407

gbdk-lib/include/sms/bcd.h, 136, 138

gbdk-lib/include/sms/gbdecompress.h, 216, 217

gbdk-lib/include/sms/hardware.h, 284, 297

gbdk-lib/include/sms/metasprites.h, 318, 323

gbdk-lib/include/sms/sms.h, 408, 435

gbdk-lib/include/stdarg.h, 107

gbdk-lib/include/stdatomic.h, 441, 442

gbdk-lib/include/stdbool.h, 442

gbdk-lib/include/stddef.h, 443, 444

gbdk-lib/include/stdint.h, 445, 450

gbdk-lib/include/stdio.h, 454, 455

gbdk-lib/include/stdlib.h, 456, 459

gbdk-lib/include/stdnoreturn.h, 460

gbdk-lib/include/string.h, 474

gbdk-lib/include/time.h, 474, 475

gbdk-lib/include/typeof.h, 475, 477

gbdk-lib/include/types.h, 485, 486

get_bkg_data

gb.h, 182

get_bkg_tile_xy

gb.h, 189

nes.h, 386

get_bkg_tiles

gb.h, 187

nes.h, 385

get_bkg_xy_addr

gb.h, 180

msx.h, 350

nes.h, 377

sms.h, 433

get_data

gb.h, 200

get_mode

gb.h, 176

msx.h, 336

nes.h, 373

sms.h, 420

get_r_reg

msx.h, 338

sms.h, 423

get_sprite_data

gb.h, 196

get_sprite_prop

gb.h, 198

msx.h, 347

nes.h, 391

sms.h, 431

get_sprite_tile

gb.h, 197

msx.h, 346

nes.h, 389

sms.h, 431

get_system

gb.h, 176

msx.h, 336

nes.h, 373

sms.h, 421

get_tiles

gb.h, 202

get_vram_byte

- gb.h, 180
- get_win_data
 - gb.h, 191
- get_win_tile_xy
 - gb.h, 194
- get_win_tiles
 - gb.h, 194
- get_win_xy_addr
 - gb.h, 190
 - msx.h, 335
 - sms.h, 420
- getchar
 - stdio.h, 455
- getpix
 - drawing.h, 150
- gets
 - stdio.h, 455
- GGEXT_NINT
 - hardware.h, 287
- GGSTATE_NJAP
 - hardware.h, 287
- GGSTATE_NNTS
 - hardware.h, 287
- GGSTATE_STT
 - hardware.h, 287
- gotogxy
 - drawing.h, 150
- gotoxy
 - console.h, 225
- gprint
 - drawing.h, 148
- gprintf
 - drawing.h, 148
- gprintln
 - drawing.h, 148
- gprintn
 - drawing.h, 148
- GRAPHICS_HEIGHT
 - drawing.h, 147
- GRAPHICS_WIDTH
 - drawing.h, 147
- GUN_P1_LATCH
 - hardware.h, 289
- GUN_P2_LATCH
 - hardware.h, 289
- h
 - _fixed, 97
 - gb.h, 205
 - msx.h, 351
 - sms.h, 434
- hardware.h
 - _AUD3WAVERAM, 259
 - _HRAM, 259
 - _IO, 259
 - _OAMRAM, 259
 - _RAM, 259
 - _RAMBANK, 259
 - _SCRN0, 258
 - _SCRN1, 259
 - _SRAM, 259
 - _SYSTEM, 275
 - _VRAM, 258
 - _VRAM8000, 258
 - _VRAM8800, 258
 - _VRAM9000, 258
 - __BYTES, 245, 270, 287
 - __BYTE_REG, 245, 270, 287
 - __REG, 245, 279, 281, 282
 - __SHADOW_REG, 279
 - _lcd_scanline, 282
 - AUD1SWEEP_DOWN, 248
 - AUD1SWEEP_LENGTH, 248
 - AUD1SWEEP_TIME, 248
 - AUD1SWEEP_UP, 248
 - AUD3WAVE, 261
 - AUD4POLY_WIDTH_15BIT, 249
 - AUD4POLY_WIDTH_7BIT, 249
 - AUDENA_OFF, 250
 - AUDENA_ON, 250
 - AUDENV_DOWN, 256
 - AUDENV_LENGTH, 256
 - AUDENV_UP, 256
 - AUDENV_VOL, 256
 - AUDHIGH_LENGTH_OFF, 256
 - AUDHIGH_LENGTH_ON, 256
 - AUDHIGH_RESTART, 256
 - AUDLEN_DUTY_12_5, 256
 - AUDLEN_DUTY_25, 256
 - AUDLEN_DUTY_50, 256
 - AUDLEN_DUTY_75, 256
 - AUDLEN_LENGTH, 256
 - AUDTERM_1_LEFT, 249
 - AUDTERM_1_RIGHT, 250
 - AUDTERM_2_LEFT, 249
 - AUDTERM_2_RIGHT, 250
 - AUDTERM_3_LEFT, 249
 - AUDTERM_3_RIGHT, 250
 - AUDTERM_4_LEFT, 249
 - AUDTERM_4_RIGHT, 250
 - AUDVOL_VIN_LEFT, 249
 - AUDVOL_VIN_RIGHT, 249
 - AUDVOL_VOL_LEFT, 249
 - AUDVOL_VOL_RIGHT, 249
 - BCPD_REG, 263
 - BCPS_REG, 263
 - BCPSF_AUTOINC, 255
 - BGP_REG, 262
 - bkg_scroll_x, 282
 - bkg_scroll_y, 282
 - BKGF_BANK0, 254
 - BKGF_BANK1, 254
 - BKGF_CGB_PAL0, 254
 - BKGF_CGB_PAL1, 254
 - BKGF_CGB_PAL2, 254
 - BKGF_CGB_PAL3, 254
 - BKGF_CGB_PAL4, 254

BKGF_CGB_PAL5, [254](#)
BKGF_CGB_PAL6, [254](#)
BKGF_CGB_PAL7, [254](#)
BKGF_PRI, [253](#)
BKGF_XFLIP, [254](#)
BKGF_YFLIP, [254](#)
DEVICE_SCREEN_BUFFER_HEIGHT, [258](#), [280](#)
DEVICE_SCREEN_BUFFER_WIDTH, [258](#), [280](#)
DEVICE_SCREEN_HEIGHT, [258](#), [280](#)
DEVICE_SCREEN_MAP_ENTRY_SIZE, [258](#), [280](#)
DEVICE_SCREEN_PX_HEIGHT, [258](#), [274](#), [280](#),
[296](#)
DEVICE_SCREEN_PX_WIDTH, [258](#), [274](#), [280](#),
[296](#)
DEVICE_SCREEN_WIDTH, [258](#), [280](#)
DEVICE_SCREEN_X_OFFSET, [257](#), [280](#)
DEVICE_SCREEN_Y_OFFSET, [257](#), [280](#)
DEVICE_SPRITE_PX_OFFSET_X, [258](#), [280](#)
DEVICE_SPRITE_PX_OFFSET_Y, [258](#), [280](#)
DEVICE_WINDOW_PX_OFFSET_X, [258](#), [280](#)
DEVICE_WINDOW_PX_OFFSET_Y, [258](#), [280](#)
DIV_REG, [260](#)
DMA_REG, [262](#)
GGEXT_NINT, [287](#)
GGSTATE_NJAP, [287](#)
GGSTATE_NNTS, [287](#)
GGSTATE_STT, [287](#)
GUN_P1_LATCH, [289](#)
GUN_P2_LATCH, [289](#)
HDMA1_REG, [263](#)
HDMA2_REG, [263](#)
HDMA3_REG, [263](#)
HDMA4_REG, [263](#)
HDMA5_REG, [263](#)
HDMA5F_BUSY, [255](#)
HDMA5F_MODE_GP, [254](#)
HDMA5F_MODE_HBL, [255](#)
IE_REG, [263](#)
IEF_HILO, [256](#)
IEF_SERIAL, [256](#)
IEF_STAT, [256](#)
IEF_TIMER, [256](#)
IEF_VBLANK, [256](#)
IF_REG, [260](#)
JOY_P1_DOWN, [294](#)
JOY_P1_LEFT, [294](#)
JOY_P1_LIGHT, [295](#)
JOY_P1_MD_A, [294](#)
JOY_P1_MD_MODE, [294](#)
JOY_P1_MD_START, [294](#)
JOY_P1_MD_X, [294](#)
JOY_P1_MD_Y, [294](#)
JOY_P1_MD_Z, [294](#)
JOY_P1_RIGHT, [294](#)
JOY_P1_SW1, [294](#)
JOY_P1_SW2, [294](#)
JOY_P1_TH_DIR_IN, [289](#)
JOY_P1_TH_DIR_OUT, [289](#)
JOY_P1_TH_OUT_HI, [290](#)
JOY_P1_TH_OUT_LO, [290](#)
JOY_P1_TR_DIR_IN, [289](#)
JOY_P1_TR_DIR_OUT, [289](#)
JOY_P1_TR_OUT_HI, [289](#)
JOY_P1_TR_OUT_LO, [289](#)
JOY_P1_TRIGGER, [294](#)
JOY_P1_UP, [293](#)
JOY_P2_DOWN, [294](#)
JOY_P2_LEFT, [294](#)
JOY_P2_LIGHT, [295](#)
JOY_P2_MD_A, [295](#)
JOY_P2_MD_MODE, [295](#)
JOY_P2_MD_START, [295](#)
JOY_P2_MD_X, [294](#)
JOY_P2_MD_Y, [294](#)
JOY_P2_MD_Z, [294](#)
JOY_P2_RIGHT, [295](#)
JOY_P2_SW1, [295](#)
JOY_P2_SW2, [295](#)
JOY_P2_TH_DIR_IN, [289](#)
JOY_P2_TH_DIR_OUT, [289](#)
JOY_P2_TH_OUT_HI, [290](#)
JOY_P2_TH_OUT_LO, [290](#)
JOY_P2_TR_DIR_IN, [289](#)
JOY_P2_TR_DIR_OUT, [289](#)
JOY_P2_TR_OUT_HI, [290](#)
JOY_P2_TR_OUT_LO, [290](#)
JOY_P2_TRIGGER, [295](#)
JOY_P2_UP, [294](#)
JOY_RESET, [295](#)
JOY_TH_HI, [290](#)
JOY_TH_LO, [290](#)
KEY1_REG, [262](#)
KEY1F_DBLSPD, [253](#)
KEY1F_PREPARE, [253](#)
LCDC_REG, [261](#)
LCDCF_B_BG8000, [251](#)
LCDCF_B_BG9C00, [251](#)
LCDCF_B_BGON, [251](#)
LCDCF_B_OBJ16, [251](#)
LCDCF_B_OBJON, [251](#)
LCDCF_B_ON, [251](#)
LCDCF_B_WIN9C00, [251](#)
LCDCF_B_WINON, [251](#)
LCDCF_BG8000, [250](#)
LCDCF_BG8800, [250](#)
LCDCF_BG9800, [250](#)
LCDCF_BG9C00, [250](#)
LCDCF_BGOFF, [251](#)
LCDCF_BGON, [251](#)
LCDCF_OBJ16, [251](#)
LCDCF_OBJ8, [251](#)
LCDCF_OBJOFF, [251](#)
LCDCF_OBJON, [251](#)
LCDCF_OFF, [250](#)
LCDCF_ON, [250](#)
LCDCF_WIN9800, [250](#)

LDCDCF_WIN9C00, [250](#)
LDCDCF_WINOFF, [250](#)
LDCDCF_WINON, [250](#)
LY_REG, [262](#), [281](#)
LYC_REG, [262](#), [281](#)
MBC7_LATCH_CAPTURE, [245](#)
MBC7_LATCH_ERASE, [245](#)
MBC7_SRAM_ENABLE_KEY_1, [245](#)
MBC7_SRAM_ENABLE_KEY_2, [246](#)
MEMCTL_BASEOFF, [288](#)
MEMCTL_BASEON, [288](#)
MEMCTL_CROMOFF, [289](#)
MEMCTL_CROMON, [289](#)
MEMCTL_EXTOFF, [289](#)
MEMCTL_EXTON, [289](#)
MEMCTL_JOYOFF, [288](#)
MEMCTL_JOYON, [288](#)
MEMCTL_RAMOFF, [288](#)
MEMCTL_RAMON, [288](#)
MEMCTL_ROMOFF, [289](#)
MEMCTL_ROMON, [289](#)
NR10_REG, [260](#)
NR11_REG, [260](#)
NR12_REG, [260](#)
NR13_REG, [260](#)
NR14_REG, [260](#)
NR21_REG, [260](#)
NR22_REG, [261](#)
NR23_REG, [261](#)
NR24_REG, [261](#)
NR30_REG, [261](#)
NR31_REG, [261](#)
NR32_REG, [261](#)
NR33_REG, [261](#)
NR34_REG, [261](#)
NR41_REG, [261](#)
NR42_REG, [261](#)
NR43_REG, [261](#)
NR44_REG, [261](#)
NR50_REG, [261](#)
NR51_REG, [261](#)
NR52_REG, [261](#)
OAMF_BANK0, [257](#)
OAMF_BANK1, [257](#)
OAMF_CGB_PAL0, [257](#)
OAMF_CGB_PAL1, [257](#)
OAMF_CGB_PAL2, [257](#)
OAMF_CGB_PAL3, [257](#)
OAMF_CGB_PAL4, [257](#)
OAMF_CGB_PAL5, [257](#)
OAMF_CGB_PAL6, [257](#)
OAMF_CGB_PAL7, [257](#)
OAMF_PAL0, [257](#)
OAMF_PAL1, [257](#)
OAMF_PALMASK, [257](#)
OAMF_PRI, [257](#)
OAMF_XFLIP, [257](#)
OAMF_YFLIP, [257](#)
OBP0_REG, [262](#)
OBP1_REG, [262](#)
OCPD_REG, [263](#)
OCPS_REG, [263](#)
OCPSF_AUTOINC, [255](#)
P1_REG, [260](#)
P1F_0, [246](#)
P1F_1, [246](#)
P1F_2, [246](#)
P1F_3, [246](#)
P1F_4, [246](#)
P1F_5, [246](#)
P1F_GET_BTN, [246](#)
P1F_GET_DPAD, [246](#)
P1F_GET_NONE, [246](#)
PCM12_REG, [263](#)
PCM34_REG, [263](#)
PCM_SAMPLE, [261](#)
PPUCTRL_BG_CHR, [279](#)
PPUCTRL_INC32, [279](#)
PPUCTRL_NMI, [279](#)
PPUCTRL_SPR_8X16, [279](#)
PPUCTRL_SPR_8X8, [279](#)
PPUCTRL_SPR_CHR, [279](#)
PPUMASK_BLUE, [279](#)
PPUMASK_GREEN, [279](#)
PPUMASK_MONOCHROME, [280](#)
PPUMASK_RED, [279](#)
PPUMASK_SHOW_BG, [279](#)
PPUMASK_SHOW_BG_LC, [280](#)
PPUMASK_SHOW_SPR, [279](#)
PPUMASK_SHOW_SPR_LC, [280](#)
PSG_CH0, [271](#), [290](#)
PSG_CH1, [271](#), [290](#)
PSG_CH2, [271](#), [290](#)
PSG_CH3, [271](#), [290](#)
PSG_LATCH, [271](#), [290](#)
PSG_VOLUME, [271](#), [290](#)
R0_CB_INPUT, [271](#)
R0_CB_OUTPUT, [271](#)
R0_DEFAULT, [271](#), [291](#)
R0_ES, [272](#), [291](#)
R0_ES_OFF, [272](#), [291](#)
R0_HSCRL, [291](#)
R0_HSCRL_INH, [291](#)
R0_IE1, [271](#), [291](#)
R0_IE1_OFF, [271](#), [291](#)
R0_IE2, [271](#)
R0_IE2_OFF, [271](#)
R0_LCB, [291](#)
R0_NO_LCB, [291](#)
R0_SCR_MODE1, [272](#)
R0_SCR_MODE2, [272](#)
R0_SCR_MODE3, [272](#)
R0_SS, [291](#)
R0_SS_OFF, [291](#)
R0_VSCRL, [291](#)
R0_VSCRL_INH, [291](#)

R10_INT_EVERY, [274](#), [293](#)
R10_INT_OFF, [274](#), [293](#)
R1_DEFAULT, [272](#), [291](#)
R1_DISP_OFF, [272](#), [291](#)
R1_DISP_ON, [272](#), [292](#)
R1_IE, [272](#), [292](#)
R1_IE_OFF, [272](#), [292](#)
R1_SCR_MODE1, [272](#)
R1_SCR_MODE2, [272](#)
R1_SCR_MODE3, [272](#)
R1_SPR_16X16, [272](#)
R1_SPR_8X16, [292](#)
R1_SPR_8X8, [272](#), [292](#)
R1_SPR_MAG, [272](#)
R1_SPR_MAG_OFF, [272](#)
R2_MAP_0x0000, [273](#), [292](#)
R2_MAP_0x0800, [273](#), [292](#)
R2_MAP_0x1000, [273](#), [292](#)
R2_MAP_0x1800, [273](#), [292](#)
R2_MAP_0x2000, [273](#), [292](#)
R2_MAP_0x2800, [273](#), [292](#)
R2_MAP_0x3000, [273](#), [292](#)
R2_MAP_0x3800, [273](#), [292](#)
R5_SAT_0x1F00, [293](#)
R5_SAT_0x3F00, [273](#), [292](#)
R5_SAT_MASK, [273](#), [293](#)
R6_BANK0, [273](#), [293](#)
R6_BANK1, [273](#), [293](#)
R6_DATA_0x0000, [273](#), [293](#)
R6_DATA_0x2000, [274](#), [293](#)
R7_COLOR_MASK, [274](#), [293](#)
RAMCTL_BANK, [295](#)
RAMCTL_PROT, [295](#)
RAMCTL_RAM, [295](#)
RAMCTL_RO, [295](#)
RAMCTL_ROM, [295](#)
rAUD1ENV, [248](#)
rAUD1HIGH, [248](#)
rAUD1LEN, [248](#)
rAUD1LOW, [248](#)
rAUD1SWEEP, [247](#)
rAUD2ENV, [248](#)
rAUD2HIGH, [248](#)
rAUD2LEN, [248](#)
rAUD2LOW, [248](#)
rAUD3ENA, [248](#)
rAUD3HIGH, [248](#)
rAUD3LEN, [248](#)
rAUD3LEVEL, [248](#)
rAUD3LOW, [248](#)
rAUD4ENV, [249](#)
rAUD4GO, [249](#)
rAUD4LEN, [249](#)
rAUD4POLY, [249](#)
rAUDENA, [250](#)
rAUDTERM, [249](#)
rAUDVOL, [249](#)
rBCPD, [255](#)
rBCPS, [255](#)
rBGP, [253](#)
rDIV, [247](#)
rDMA, [253](#)
rHDMA1, [254](#)
rHDMA2, [254](#)
rHDMA3, [254](#)
rHDMA4, [254](#)
rHDMA5, [254](#)
rIE, [255](#)
rIF, [247](#)
rKEY1, [253](#)
rLDCD, [250](#)
rLY, [253](#), [281](#)
rLYC, [253](#), [281](#)
rMBC7_ACCEL_X_HI, [259](#)
rMBC7_ACCEL_X_LO, [259](#)
rMBC7_ACCEL_Y_HI, [260](#)
rMBC7_ACCEL_Y_LO, [260](#)
rMBC7_LATCH_1, [259](#)
rMBC7_LATCH_2, [259](#)
rMBC7_SRAM_ENABLE_1, [259](#)
rMBC7_SRAM_ENABLE_2, [259](#)
rOBP0, [253](#)
rOBP1, [253](#)
rOCPD, [255](#)
rOCPS, [255](#)
rP1, [246](#)
RP_REG, [263](#)
rPCM12, [255](#)
rPCM34, [255](#)
RPF_DATAIN, [255](#)
RPF_ENREAD, [255](#)
RPF_WRITE_HI, [255](#)
RPF_WRITE_LO, [255](#)
rRAMB, [259](#)
rRAMG, [259](#)
rROMB0, [259](#)
rROMB1, [259](#)
rRP, [255](#)
rSB, [246](#)
rSC, [246](#)
rSCX, [252](#), [281](#)
rSCY, [252](#), [281](#)
rSMBK, [255](#)
rSPD, [253](#)
rSTAT, [251](#)
rSVBK, [255](#)
rTAC, [247](#)
rTIMA, [247](#)
rTMA, [247](#)
rVBK, [253](#)
rWX, [253](#)
rWY, [253](#)
SB_REG, [260](#)
SC_REG, [260](#)
SCF_SOURCE, [247](#)
SCF_SPEED, [247](#)

SCF_START, [247](#)
 scroll_x_t, [281](#)
 scroll_y_t, [281](#)
 SCX_REG, [262](#), [281](#)
 SCY_REG, [262](#), [280](#)
 shadow_PPUCTRL, [282](#)
 shadow_PPUMASK, [282](#)
 shadow_VDP_R0, [275](#), [296](#)
 shadow_VDP_R1, [275](#), [296](#)
 shadow_VDP_R10, [275](#), [296](#)
 shadow_VDP_R2, [275](#), [296](#)
 shadow_VDP_R3, [275](#), [296](#)
 shadow_VDP_R4, [275](#), [296](#)
 shadow_VDP_R5, [275](#), [296](#)
 shadow_VDP_R6, [275](#), [296](#)
 shadow_VDP_R7, [275](#), [296](#)
 shadow_VDP_R8, [275](#), [296](#)
 shadow_VDP_R9, [275](#), [296](#)
 shadow_VDP_RBORDER, [275](#), [296](#)
 shadow_VDP_RSCX, [275](#), [296](#)
 shadow_VDP_RSCY, [275](#), [296](#)
 SIOCTL_BS0, [288](#)
 SIOCTL_BS1, [288](#)
 SIOCTL_FRER, [287](#)
 SIOCTL_INT, [287](#)
 SIOCTL_RON, [288](#)
 SIOCTL_RXRD, [287](#)
 SIOCTL_TON, [288](#)
 SIOCTL_TXFL, [287](#)
 SIOF_B_CLOCK, [247](#)
 SIOF_B_SPEED, [247](#)
 SIOF_B_XFER_START, [247](#)
 SIOF_CLOCK_EXT, [246](#)
 SIOF_CLOCK_INT, [246](#)
 SIOF_SPEED_1X, [246](#)
 SIOF_SPEED_32X, [246](#)
 SIOF_XFER_START, [246](#)
 SOUNDPAN_NOSL, [288](#)
 SOUNDPAN_NOSR, [288](#)
 SOUNDPAN_TN1L, [288](#)
 SOUNDPAN_TN1R, [288](#)
 SOUNDPAN_TN2L, [288](#)
 SOUNDPAN_TN2R, [288](#)
 SOUNDPAN_TN3L, [288](#)
 SOUNDPAN_TN3R, [288](#)
 STAT_REG, [262](#)
 STATF_9_SPR, [271](#), [290](#)
 STATF_B_BUSY, [252](#)
 STATF_B_LYC, [252](#)
 STATF_B_LYCF, [252](#)
 STATF_B_MODE00, [252](#)
 STATF_B_MODE01, [252](#)
 STATF_B_MODE10, [252](#)
 STATF_B_OAM, [252](#)
 STATF_B_VBL, [252](#)
 STATF_BUSY, [252](#)
 STATF_HBL, [252](#)
 STATF_INT_VBL, [271](#), [290](#)
 STATF_LCD, [252](#)
 STATF_LYC, [251](#)
 STATF_LYCF, [252](#)
 STATF_MODE00, [252](#)
 STATF_MODE01, [251](#)
 STATF_MODE10, [251](#)
 STATF_OAM, [252](#)
 STATF_SPR_COLL, [271](#), [290](#)
 STATF_VBL, [252](#)
 SVBK_REG, [263](#)
 SYSTEM_NTSC, [274](#)
 SYSTEM_PAL, [274](#)
 TAC_REG, [260](#), [282](#), [297](#)
 TACF_16KHZ, [247](#)
 TACF_262KHZ, [247](#)
 TACF_4KHZ, [247](#)
 TACF_65KHZ, [247](#)
 TACF_START, [247](#)
 TACF_STOP, [247](#)
 TIMA_REG, [260](#), [282](#), [296](#)
 TMA_REG, [260](#), [282](#), [297](#)
 VBK_ATTRIBUTES, [253](#), [274](#), [295](#)
 VBK_BANK_0, [253](#)
 VBK_BANK_1, [253](#)
 VBK_REG, [262](#)
 VBK_TILES, [253](#), [274](#), [295](#)
 VDP_ATTR_SHIFT, [275](#), [297](#)
 VDP_R0, [271](#), [291](#)
 VDP_R1, [272](#), [291](#)
 VDP_R10, [274](#), [293](#)
 VDP_R2, [273](#), [292](#)
 VDP_R3, [273](#), [292](#)
 VDP_R4, [273](#), [292](#)
 VDP_R5, [273](#), [292](#)
 VDP_R6, [273](#), [293](#)
 VDP_R7, [274](#), [293](#)
 VDP_R8, [274](#), [293](#)
 VDP_R9, [274](#), [293](#)
 VDP_RBORDER, [274](#), [293](#)
 VDP_REG_MASK, [271](#), [291](#)
 VDP_RSCX, [274](#), [293](#)
 VDP_RSCY, [274](#), [293](#)
 VDP_SAT_TERM, [274](#), [295](#)
 WX_REG, [262](#)
 WY_REG, [262](#)
 HARDWARE_SPRITE_CAN_FLIP_X
 gb.h, [172](#)
 msx.h, [335](#)
 nes.h, [370](#)
 sms.h, [420](#)
 HARDWARE_SPRITE_CAN_FLIP_Y
 gb.h, [172](#)
 msx.h, [335](#)
 nes.h, [370](#)
 sms.h, [420](#)
 hblank_copy
 hblankcpy.h, [218](#)
 hblank_copy_destination

hblankcpy.h, [218](#)
hblank_copy_vram
 hblankcpy.h, [217](#)
hblank_cpy_vram
 hblankcpy.h, [218](#)
hblankcpy.h
 hblank_copy, [218](#)
 hblank_copy_destination, [218](#)
 hblank_copy_vram, [217](#)
 hblank_cpy_vram, [218](#)
HDMA1_REG
 hardware.h, [263](#)
HDMA2_REG
 hardware.h, [263](#)
HDMA3_REG
 hardware.h, [263](#)
HDMA4_REG
 hardware.h, [263](#)
HDMA5_REG
 hardware.h, [263](#)
HDMA5F_BUSY
 hardware.h, [255](#)
HDMA5F_MODE_GP
 hardware.h, [254](#)
HDMA5F_MODE_HBL
 hardware.h, [255](#)
HIDE_BKG
 gb.h, [170](#)
 msx.h, [332](#)
 nes.h, [369](#)
 sms.h, [416](#)
HIDE_LEFT_COLUMN
 gb.h, [170](#)
 msx.h, [331](#)
 nes.h, [368](#)
 sms.h, [416](#)
hide metasprite
 metasprites.h, [306](#), [310](#), [316](#), [322](#)
hide_sprite
 gb.h, [199](#)
 msx.h, [349](#)
 nes.h, [393](#)
 sms.h, [432](#)
HIDE_SPRITES
 gb.h, [171](#)
 msx.h, [332](#)
 nes.h, [369](#)
 sms.h, [417](#)
hide_sprites_range
 metasprites.h, [303](#), [309](#), [313](#), [320](#)
HIDE_WIN
 gb.h, [171](#)
 msx.h, [332](#)
 sms.h, [416](#)
hramcpy
 gb.h, [180](#)
IE_REG
 hardware.h, [263](#)
IEF_HILO
 hardware.h, [256](#)
IEF_SERIAL
 hardware.h, [256](#)
IEF_STAT
 hardware.h, [256](#)
IEF_TIMER
 hardware.h, [256](#)
IEF_VBLANK
 hardware.h, [256](#)
IF_REG
 hardware.h, [260](#)
INCBIN
 incbin.h, [233](#)
incbin.h
 BANK, [233](#)
 INCBIN, [233](#)
 INCBIN_EXTERN, [232](#)
 INCBIN_SIZE, [233](#)
INCBIN_EXTERN
 incbin.h, [232](#)
INCBIN_SIZE
 incbin.h, [233](#)
init_bkg
 gb.h, [204](#)
 nes.h, [395](#)
init_win
 gb.h, [203](#)
initarand
 rand.h, [405](#)
initrand
 rand.h, [405](#)
INT16
 types.h, [478](#), [480](#), [484](#)
INT16_C
 stdint.h, [448](#)
INT16_MAX
 stdint.h, [446](#)
INT16_MIN
 stdint.h, [446](#)
int16_t
 stdint.h, [449](#)
INT32
 types.h, [478](#), [480](#), [484](#)
INT32_C
 stdint.h, [448](#)
INT32_MAX
 stdint.h, [446](#)
INT32_MIN
 stdint.h, [446](#)
int32_t
 stdint.h, [449](#)
INT8
 types.h, [478](#), [480](#), [484](#)
INT8_C
 stdint.h, [448](#)
INT8_MAX
 stdint.h, [446](#)

INT8_MIN
 stdint.h, 446
 int8_t
 stdint.h, 449
 INT_FAST16_MAX
 stdint.h, 447
 INT_FAST16_MIN
 stdint.h, 447
 int_fast16_t
 stdint.h, 450
 INT_FAST32_MAX
 stdint.h, 447
 INT_FAST32_MIN
 stdint.h, 447
 int_fast32_t
 stdint.h, 450
 INT_FAST8_MAX
 stdint.h, 447
 INT_FAST8_MIN
 stdint.h, 447
 int_fast8_t
 stdint.h, 450
 int_handler
 gb.h, 172
 msx.h, 335
 nes.h, 370
 sms.h, 420
 INT_LEAST16_MAX
 stdint.h, 447
 INT_LEAST16_MIN
 stdint.h, 447
 int_least16_t
 stdint.h, 449
 INT_LEAST32_MAX
 stdint.h, 447
 INT_LEAST32_MIN
 stdint.h, 447
 int_least32_t
 stdint.h, 449
 INT_LEAST8_MAX
 stdint.h, 447
 INT_LEAST8_MIN
 stdint.h, 447
 int_least8_t
 stdint.h, 449
 INT_MAX
 limits.h, 238
 INT_MIN
 limits.h, 238
 INTERRUPT
 types.h, 482
 INTMAX_C
 stdint.h, 449
 INTMAX_MAX
 stdint.h, 448
 INTMAX_MIN
 stdint.h, 448
 intmax_t
 stdint.h, 450
 INTPTR_MAX
 stdint.h, 448
 INTPTR_MIN
 stdint.h, 448
 intptr_t
 stdint.h, 450
 IO_ERROR
 gb.h, 166
 IO_IDLE
 gb.h, 166
 IO_RECEIVING
 gb.h, 166
 IO_SENDING
 gb.h, 166
 isalpha
 ctype.h, 109
 isdigit
 ctype.h, 109
 islower
 ctype.h, 109
 isr.h
 ISR_NESTED_VECTOR, 220
 isr_nested_vector_t, 220
 ISR_VECTOR, 220
 isr_vector_t, 220
 VECTOR_JOYPAD, 219
 VECTOR_SERIAL, 219
 VECTOR_STAT, 219
 VECTOR_TIMER, 219
 ISR_NESTED_VECTOR
 isr.h, 220
 isr_nested_vector_t, 97
 func, 98
 isr.h, 220
 opcode, 98
 ISR_VECTOR
 isr.h, 220
 isr_vector_t, 98
 func, 98
 isr.h, 220
 opcode, 98
 isspace
 ctype.h, 109
 isupper
 ctype.h, 109
 itoa
 stdlib.h, 457
 iyh
 msx.h, 351
 sms.h, 434
 iyl
 metasprites.h, 311, 323
 msx.h, 351
 sms.h, 434
 J_A
 gb.h, 161
 msx.h, 328

- nes.h, [363](#)
- sms.h, [413](#)
- J_B
 - gb.h, [161](#)
 - msx.h, [328](#)
 - nes.h, [363](#)
 - sms.h, [413](#)
- J_DOWN
 - gb.h, [161](#)
 - msx.h, [328](#)
 - nes.h, [363](#)
 - sms.h, [413](#)
- J_LEFT
 - gb.h, [161](#)
 - msx.h, [328](#)
 - nes.h, [363](#)
 - sms.h, [413](#)
- J_RIGHT
 - gb.h, [161](#)
 - msx.h, [328](#)
 - nes.h, [363](#)
 - sms.h, [413](#)
- J_SELECT
 - gb.h, [161](#)
 - msx.h, [328](#)
 - nes.h, [363](#)
 - sms.h, [413](#)
- J_START
 - gb.h, [162](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [413](#)
- J_UP
 - gb.h, [161](#)
 - msx.h, [328](#)
 - nes.h, [363](#)
 - sms.h, [413](#)
- jmp_buf
 - setjmp.h, [407](#)
- joy0
 - joypads_t, [99](#)
- joy1
 - joypads_t, [99](#)
- joy2
 - joypads_t, [99](#)
- joy3
 - joypads_t, [99](#)
- JOY_IFLAG
 - gb.h, [163](#)
 - msx.h, [330](#)
 - sms.h, [415](#)
- JOY_P1_DOWN
 - hardware.h, [294](#)
- JOY_P1_LEFT
 - hardware.h, [294](#)
- JOY_P1_LIGHT
 - hardware.h, [295](#)
- JOY_P1_MD_A
 - hardware.h, [294](#)
- JOY_P1_MD_MODE
 - hardware.h, [294](#)
- JOY_P1_MD_START
 - hardware.h, [294](#)
- JOY_P1_MD_X
 - hardware.h, [294](#)
- JOY_P1_MD_Y
 - hardware.h, [294](#)
- JOY_P1_MD_Z
 - hardware.h, [294](#)
- JOY_P1_RIGHT
 - hardware.h, [294](#)
- JOY_P1_SW1
 - hardware.h, [294](#)
- JOY_P1_SW2
 - hardware.h, [294](#)
- JOY_P1_TH_DIR_IN
 - hardware.h, [289](#)
- JOY_P1_TH_DIR_OUT
 - hardware.h, [289](#)
- JOY_P1_TH_OUT_HI
 - hardware.h, [290](#)
- JOY_P1_TH_OUT_LO
 - hardware.h, [290](#)
- JOY_P1_TR_DIR_IN
 - hardware.h, [289](#)
- JOY_P1_TR_DIR_OUT
 - hardware.h, [289](#)
- JOY_P1_TR_OUT_HI
 - hardware.h, [289](#)
- JOY_P1_TR_OUT_LO
 - hardware.h, [289](#)
- JOY_P1_TRIGGER
 - hardware.h, [294](#)
- JOY_P1_UP
 - hardware.h, [293](#)
- JOY_P2_DOWN
 - hardware.h, [294](#)
- JOY_P2_LEFT
 - hardware.h, [294](#)
- JOY_P2_LIGHT
 - hardware.h, [295](#)
- JOY_P2_MD_A
 - hardware.h, [295](#)
- JOY_P2_MD_MODE
 - hardware.h, [295](#)
- JOY_P2_MD_START
 - hardware.h, [295](#)
- JOY_P2_MD_X
 - hardware.h, [294](#)
- JOY_P2_MD_Y
 - hardware.h, [294](#)
- JOY_P2_MD_Z
 - hardware.h, [294](#)
- JOY_P2_RIGHT
 - hardware.h, [295](#)
- JOY_P2_SW1
 - hardware.h, [294](#)

- hardware.h, 295
- JOY_P2_SW2
 - hardware.h, 295
- JOY_P2_TH_DIR_IN
 - hardware.h, 289
- JOY_P2_TH_DIR_OUT
 - hardware.h, 289
- JOY_P2_TH_OUT_HI
 - hardware.h, 290
- JOY_P2_TH_OUT_LO
 - hardware.h, 290
- JOY_P2_TR_DIR_IN
 - hardware.h, 289
- JOY_P2_TR_DIR_OUT
 - hardware.h, 289
- JOY_P2_TR_OUT_HI
 - hardware.h, 290
- JOY_P2_TR_OUT_LO
 - hardware.h, 290
- JOY_P2_TRIGGER
 - hardware.h, 295
- JOY_P2_UP
 - hardware.h, 294
- JOY_RESET
 - hardware.h, 295
- JOY_TH_HI
 - hardware.h, 290
- JOY_TH_LO
 - hardware.h, 290
- joypad
 - gb.h, 177
 - msx.h, 338
 - nes.h, 373
 - sms.h, 423
- joypad_ex
 - gb.h, 178
 - msx.h, 339
 - nes.h, 374
 - sms.h, 423
- joypad_init
 - gb.h, 177
 - msx.h, 338
 - nes.h, 374
 - sms.h, 423
- joypads
 - joypads_t, 99
- joypads_t, 98
 - joy0, 99
 - joy1, 99
 - joy2, 99
 - joy3, 99
 - joypads, 99
 - npads, 99
- KEY1_REG
 - hardware.h, 262
- KEY1F_DBLSPED
 - hardware.h, 253
- KEY1F_PREPARE
 - hardware.h, 253
- I
 - _fixed, 97
 - gb.h, 205
 - msx.h, 351
 - sms.h, 434
- labs
 - stdlib.h, 456
- laptop_io.h
 - _MEGADUCK_LAPTOP_IO_H, 112
 - DUCK_IO_CMD_ABORT_OR_FAIL, 112
 - DUCK_IO_CMD_DONE_OR_OK, 112
 - DUCK_IO_CMD_GET_KEYS, 112
 - DUCK_IO_CMD_GET_RTC, 112
 - DUCK_IO_CMD_INIT_START, 112
 - DUCK_IO_CMD_PLAY_SPEECH, 112
 - DUCK_IO_CMD_PRINT_INIT_EXT_IO, 112
 - DUCK_IO_CMD_PRINT_SEND_BYTES, 112
 - DUCK_IO_CMD_RUN_CART_IN_SLOT, 112
 - DUCK_IO_CMD_SET_RTC, 112
 - duck_io_enable_read_byte, 115
 - DUCK_IO_KBD_FLAGS, 114
 - DUCK_IO_KBD_KEYCODE, 114
 - duck_io_laptop_init, 115
 - DUCK_IO_LEN_KBD_GET, 113
 - DUCK_IO_LEN_PLAY_SPEECH, 113
 - DUCK_IO_LEN_RTC_GET, 113
 - DUCK_IO_LEN_RTC_SET, 113
 - DUCK_IO_LEN_RX_MAX, 113
 - DUCK_IO_LEN_TX_MAX, 113
 - DUCK_IO_PRINTER_FAIL, 114
 - duck_io_printer_last_status, 115
 - DUCK_IO_PRINTER_MAYBE_BUSY, 114
 - duck_io_printer_query, 115
 - DUCK_IO_PRINTER_TYPE_1_PASS, 114
 - DUCK_IO_PRINTER_TYPE_2_PASS, 114
 - duck_io_read_byte_no_timeout, 114
 - duck_io_read_byte_with_msecs_timeout, 115
 - DUCK_IO_REPLY_BOOT_FAIL, 113
 - DUCK_IO_REPLY_BOOT_OK, 113
 - DUCK_IO_REPLY_BOOT_UNSET, 113
 - DUCK_IO_REPLY_BUFFER_XFER_OK, 113
 - DUCK_IO_REPLY_NO_CART_IN_SLOT, 113
 - DUCK_IO_REPLY_SEND_BUFFER_OK, 113
 - DUCK_IO_RTC_AMPM, 114
 - DUCK_IO_RTC_DAY, 114
 - DUCK_IO_RTC_HOUR, 114
 - DUCK_IO_RTC_MIN, 114
 - DUCK_IO_RTC_MON, 114
 - DUCK_IO_RTC_SEC, 114
 - DUCK_IO_RTC_WEEKDAY, 114
 - DUCK_IO_RTC_YEAR, 113
 - duck_io_rx_buf, 117
 - duck_io_rx_buf_len, 117
 - duck_io_rx_byte, 117
 - duck_io_rx_byte_done, 117
 - duck_io_send_byte, 114

- duck_io_send_byte_and_check_ack_msecs_timeout, 116
- duck_io_send_cmd_and_buffer, 116
- duck_io_send_cmd_and_receive_buffer, 116
- DUCK_IO_SPEECH_CMD_MAX, 113
- DUCK_IO_SPEECH_CMD_MIN, 113
- DUCK_IO_TIMEOUT_100_MSEC, 113
- DUCK_IO_TIMEOUT_200_MSEC, 113
- DUCK_IO_TIMEOUT_2_MSEC, 113
- duck_io_tx_buf, 117
- duck_io_tx_buf_len, 117
- laptop_keycodes.h
 - DUCK_IO_KEY_0, 123
 - DUCK_IO_KEY_1, 122
 - DUCK_IO_KEY_2, 123
 - DUCK_IO_KEY_3, 123
 - DUCK_IO_KEY_4, 123
 - DUCK_IO_KEY_5, 123
 - DUCK_IO_KEY_6, 123
 - DUCK_IO_KEY_7, 123
 - DUCK_IO_KEY_8, 123
 - DUCK_IO_KEY_9, 123
 - DUCK_IO_KEY_A, 124
 - DUCK_IO_KEY_ARROW_DOWN, 126
 - DUCK_IO_KEY_ARROW_LEFT, 126
 - DUCK_IO_KEY_ARROW_RIGHT, 126
 - DUCK_IO_KEY_ARROW_UP, 126
 - DUCK_IO_KEY_B, 125
 - DUCK_IO_KEY_BACKSPACE, 123
 - DUCK_IO_KEY_BACKTICK, 124
 - DUCK_IO_KEY_BASE, 122
 - DUCK_IO_KEY_BASE_BIT, 122
 - DUCK_IO_KEY_C, 125
 - DUCK_IO_KEY_COMMA, 125
 - DUCK_IO_KEY_D, 124
 - DUCK_IO_KEY_DASH, 125
 - DUCK_IO_KEY_DELETE, 125
 - DUCK_IO_KEY_DIVIDE, 126
 - DUCK_IO_KEY_E, 123
 - DUCK_IO_KEY_ENTER, 124
 - DUCK_IO_KEY_EQUALS, 126
 - DUCK_IO_KEY_ESCAPE, 122
 - DUCK_IO_KEY_EXCLAMATION_FLIPPED, 123
 - DUCK_IO_KEY_F, 124
 - DUCK_IO_KEY_F1, 122
 - DUCK_IO_KEY_F10, 122
 - DUCK_IO_KEY_F11, 122
 - DUCK_IO_KEY_F12, 122
 - DUCK_IO_KEY_F2, 122
 - DUCK_IO_KEY_F3, 122
 - DUCK_IO_KEY_F4, 122
 - DUCK_IO_KEY_F5, 122
 - DUCK_IO_KEY_F6, 122
 - DUCK_IO_KEY_F7, 122
 - DUCK_IO_KEY_F8, 122
 - DUCK_IO_KEY_F9, 122
 - DUCK_IO_KEY_FLAG_CAPSLOCK, 121
 - DUCK_IO_KEY_FLAG_CAPSLOCK_BIT, 121
 - DUCK_IO_KEY_FLAG_KEY_REPEAT, 121
 - DUCK_IO_KEY_FLAG_KEY_REPEAT_BIT, 121
 - DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT, 121
 - DUCK_IO_KEY_FLAG_PRINTSCREEN_LEFT_BIT, 122
 - DUCK_IO_KEY_FLAG_SHIFT, 121
 - DUCK_IO_KEY_FLAG_SHIFT_BIT, 121
 - DUCK_IO_KEY_G, 124
 - DUCK_IO_KEY_H, 124
 - DUCK_IO_KEY_HELP, 123
 - DUCK_IO_KEY_I, 124
 - DUCK_IO_KEY_J, 124
 - DUCK_IO_KEY_K, 124
 - DUCK_IO_KEY_L, 124
 - DUCK_IO_KEY_LAST_KEY, 128
 - DUCK_IO_KEY_LESS_THAN, 125
 - DUCK_IO_KEY_M, 125
 - DUCK_IO_KEY_MAYBE_RX_NOT_A_KEY, 128
 - DUCK_IO_KEY_MAYBE_SYST_CODES_START, 128
 - DUCK_IO_KEY_MEMORY_MINUS, 125
 - DUCK_IO_KEY_MEMORY_PLUS, 126
 - DUCK_IO_KEY_MEMORY_RECALL, 126
 - DUCK_IO_KEY_MINUS, 126
 - DUCK_IO_KEY_MULTIPLY, 126
 - DUCK_IO_KEY_N, 125
 - DUCK_IO_KEY_N_TILDE, 124
 - DUCK_IO_KEY_O, 124
 - DUCK_IO_KEY_O_OVER_LINE, 125
 - DUCK_IO_KEY_P, 124
 - DUCK_IO_KEY_PAGE_DOWN, 125
 - DUCK_IO_KEY_PAGE_UP, 125
 - DUCK_IO_KEY_PERIOD, 125
 - DUCK_IO_KEY_PIANO_DO, 127
 - DUCK_IO_KEY_PIANO_DO_2, 127
 - DUCK_IO_KEY_PIANO_DO_2_SHARP, 126
 - DUCK_IO_KEY_PIANO_DO_SHARP, 126
 - DUCK_IO_KEY_PIANO_FA, 127
 - DUCK_IO_KEY_PIANO_FA_2, 127
 - DUCK_IO_KEY_PIANO_FA_2_SHARP, 127
 - DUCK_IO_KEY_PIANO_FA_SHARP, 126
 - DUCK_IO_KEY_PIANO_LA, 127
 - DUCK_IO_KEY_PIANO_LA_2, 127
 - DUCK_IO_KEY_PIANO_LA_2_SHARP, 127
 - DUCK_IO_KEY_PIANO_LA_SHARP, 126
 - DUCK_IO_KEY_PIANO_MI, 127
 - DUCK_IO_KEY_PIANO_MI_2, 127
 - DUCK_IO_KEY_PIANO_RE, 127
 - DUCK_IO_KEY_PIANO_RE_2, 127
 - DUCK_IO_KEY_PIANO_RE_2_SHARP, 127
 - DUCK_IO_KEY_PIANO_RE_SHARP, 126
 - DUCK_IO_KEY_PIANO_SI, 127
 - DUCK_IO_KEY_PIANO_SI_2, 128
 - DUCK_IO_KEY_PIANO_SOL, 127
 - DUCK_IO_KEY_PIANO_SOL_2, 127
 - DUCK_IO_KEY_PIANO_SOL_2_SHARP, 127
 - DUCK_IO_KEY_PIANO_SOL_SHARP, 126

- DUCK_IO_KEY_PLUS, [126](#)
- DUCK_IO_KEY_PRINTSCREEN_RIGHT, [127](#)
- DUCK_IO_KEY_Q, [123](#)
- DUCK_IO_KEY_R, [123](#)
- DUCK_IO_KEY_RIGHT_SQ_BRACKET, [124](#)
- DUCK_IO_KEY_S, [124](#)
- DUCK_IO_KEY_SINGLE_QUOTE, [123](#)
- DUCK_IO_KEY_SPACE, [125](#)
- DUCK_IO_KEY_SQUAREROOT, [126](#)
- DUCK_IO_KEY_T, [123](#)
- DUCK_IO_KEY_U, [124](#)
- DUCK_IO_KEY_U_UMLAUT, [125](#)
- DUCK_IO_KEY_V, [125](#)
- DUCK_IO_KEY_W, [123](#)
- DUCK_IO_KEY_X, [125](#)
- DUCK_IO_KEY_Y, [124](#)
- DUCK_IO_KEY_Z, [125](#)
- LCD_IFLAG
 - gb.h, [163](#)
 - msx.h, [330](#)
 - nes.h, [365](#)
 - sms.h, [415](#)
- LCDC_REG
 - hardware.h, [261](#)
- LCDCF_B_BG8000
 - hardware.h, [251](#)
- LCDCF_B_BG9C00
 - hardware.h, [251](#)
- LCDCF_B_BGON
 - hardware.h, [251](#)
- LCDCF_B_OBJ16
 - hardware.h, [251](#)
- LCDCF_B_OBJON
 - hardware.h, [251](#)
- LCDCF_B_ON
 - hardware.h, [251](#)
- LCDCF_B_WIN9C00
 - hardware.h, [251](#)
- LCDCF_B_WINON
 - hardware.h, [251](#)
- LCDCF_BG8000
 - hardware.h, [250](#)
- LCDCF_BG8800
 - hardware.h, [250](#)
- LCDCF_BG9800
 - hardware.h, [250](#)
- LCDCF_BG9C00
 - hardware.h, [250](#)
- LCDCF_BGOFF
 - hardware.h, [251](#)
- LCDCF_BGON
 - hardware.h, [251](#)
- LCDCF_OBJ16
 - hardware.h, [251](#)
- LCDCF_OBJ8
 - hardware.h, [251](#)
- LCDCF_OBJOFF
 - hardware.h, [251](#)
- LCDCF_OBJON
 - hardware.h, [251](#)
- LCDCF_OFF
 - hardware.h, [250](#)
- LCDCF_ON
 - hardware.h, [250](#)
- LCDCF_WIN9800
 - hardware.h, [250](#)
- LCDCF_WIN9C00
 - hardware.h, [250](#)
- LCDCF_WINOFF
 - hardware.h, [250](#)
- LCDCF_WINON
 - hardware.h, [250](#)
- limits.h
 - CHAR_BIT, [237](#)
 - CHAR_MAX, [238](#)
 - CHAR_MIN, [238](#)
 - INT_MAX, [238](#)
 - INT_MIN, [238](#)
 - LONG_MAX, [238](#)
 - LONG_MIN, [238](#)
 - SCHAR_MAX, [237](#)
 - SCHAR_MIN, [237](#)
 - SHRT_MAX, [238](#)
 - SHRT_MIN, [238](#)
 - UCHAR_MAX, [238](#)
 - UINT_MAX, [238](#)
 - UINT_MIN, [238](#)
 - ULONG_MAX, [238](#)
 - ULONG_MIN, [238](#)
 - USHRT_MAX, [238](#)
 - USHRT_MIN, [238](#)
- line
 - drawing.h, [149](#)
- List of gbk fonts, [95](#)
 - font_ibm, [95](#)
 - font_ibm_fixed, [95](#)
 - font_italic, [95](#)
 - font_min, [95](#)
 - font_spect, [95](#)
- LONG_MAX
 - limits.h, [238](#)
- LONG_MIN
 - limits.h, [238](#)
- longjmp
 - setjmp.h, [407](#)
- LTGREY
 - drawing.h, [147](#)
- ltoa
 - stdlib.h, [457](#)
- LWORD
 - types.h, [482](#)
- LY_REG
 - hardware.h, [262](#), [281](#)
- LYC_REG
 - hardware.h, [262](#), [281](#)
- M_DRAWING

- gb.h, [162](#)
- nes.h, [364](#)
- M_FILL
 - drawing.h, [147](#)
- M_NO_INTERP
 - gb.h, [162](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [414](#)
- M_NO_SCROLL
 - gb.h, [162](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [414](#)
- M_NOFILL
 - drawing.h, [147](#)
- M_TEXT_INOUT
 - gb.h, [162](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [413](#)
- M_TEXT_OUT
 - gb.h, [162](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [413](#)
- MAKE_BCD
 - bcd.h, [132](#), [134](#), [137](#)
- malloc
 - stdlib.h, [458](#)
- MAX_HARDWARE_SPRITES
 - gb.h, [172](#)
 - msx.h, [335](#)
 - nes.h, [370](#)
 - sms.h, [420](#)
- MAX_LCD_ISR_CALLS
 - nes.h, [366](#)
- MAXWNDPOSX
 - gb.h, [165](#)
 - msx.h, [331](#)
 - sms.h, [415](#)
- MAXWNDPOSY
 - gb.h, [165](#)
 - msx.h, [331](#)
 - sms.h, [415](#)
- MBC7_LATCH_CAPTURE
 - hardware.h, [245](#)
- MBC7_LATCH_ERASE
 - hardware.h, [245](#)
- MBC7_SRAM_ENABLE_KEY_1
 - hardware.h, [245](#)
- MBC7_SRAM_ENABLE_KEY_2
 - hardware.h, [246](#)
- MEGADUCK_HANDHELD_STANDARD
 - model.h, [131](#)
- MEGADUCK_LAPTOP_GERMAN
 - model.h, [131](#)
- MEGADUCK_LAPTOP_SPANISH
 - model.h, [131](#)
- memcmp
 - string.h, [465](#), [469](#), [473](#)
- memcpy
 - string.h, [461](#), [466](#), [471](#)
- MEMCTL_BASEOFF
 - hardware.h, [288](#)
- MEMCTL_BASEON
 - hardware.h, [288](#)
- MEMCTL_CROMOFF
 - hardware.h, [289](#)
- MEMCTL_CROMON
 - hardware.h, [289](#)
- MEMCTL_EXTOFF
 - hardware.h, [289](#)
- MEMCTL_EXTON
 - hardware.h, [289](#)
- MEMCTL_JOYOFF
 - hardware.h, [288](#)
- MEMCTL_JOYON
 - hardware.h, [288](#)
- MEMCTL_RAMOFF
 - hardware.h, [288](#)
- MEMCTL_RAMON
 - hardware.h, [288](#)
- MEMCTL_ROMOFF
 - hardware.h, [289](#)
- MEMCTL_ROMON
 - hardware.h, [289](#)
- memmove
 - string.h, [463](#), [467](#), [471](#)
- memset
 - string.h, [463](#), [467](#), [471](#)
- METASPR_ITEM
 - metasprites.h, [302](#), [309](#), [313](#), [319](#)
- METASPR_TERM
 - metasprites.h, [302](#), [309](#), [313](#), [319](#)
- metasprite_end
 - metasprites.h, [302](#), [309](#), [312](#), [319](#)
- metasprite_t, [100](#)
 - dtile, [100](#)
 - dx, [100](#)
 - dy, [100](#)
 - metasprites.h, [302](#), [309](#), [313](#), [319](#)
 - props, [101](#)
- metasprites.h
 - __current_base_prop, [306](#), [317](#)
 - __current_base_tile, [306](#), [311](#), [317](#), [323](#)
 - __current_metasprite, [306](#), [311](#), [317](#), [323](#)
 - __render_shadow_OAM, [306](#), [311](#), [317](#), [323](#)
 - hide_metasprite, [306](#), [310](#), [316](#), [322](#)
 - hide_sprites_range, [303](#), [309](#), [313](#), [320](#)
 - iyl, [311](#), [323](#)
 - METASPR_ITEM, [302](#), [309](#), [313](#), [319](#)
 - METASPR_TERM, [302](#), [309](#), [313](#), [319](#)
 - metasprite_end, [302](#), [309](#), [312](#), [319](#)
 - metasprite_t, [302](#), [309](#), [313](#), [319](#)
 - move_metasprite, [303](#), [310](#), [314](#), [320](#)

- move metasprite_ex, [303](#), [310](#), [313](#), [320](#)
 - move metasprite_flipx, [304](#), [314](#), [321](#)
 - move metasprite_flipxy, [305](#), [315](#), [322](#)
 - move metasprite_flipy, [304](#), [315](#), [321](#)
 - move metasprite_hflip, [305](#), [315](#)
 - move metasprite_hvflip, [306](#), [316](#)
 - move metasprite_vflip, [304](#), [315](#)
- mfont_handle
 - font.h, [230](#)
- MGB_TYPE
 - gb.h, [165](#)
- MINWNDPOSX
 - gb.h, [164](#)
 - msx.h, [330](#)
 - sms.h, [415](#)
- MINWNDPOSY
 - gb.h, [165](#)
 - msx.h, [330](#)
 - sms.h, [415](#)
- mode
 - gb.h, [176](#)
 - msx.h, [335](#)
 - nes.h, [372](#)
 - sms.h, [420](#)
- model.h
 - duck_check_model, [131](#)
 - MEGADUCK_HANDHELD_STANDARD, [131](#)
 - MEGADUCK_LAPTOP_GERMAN, [131](#)
 - MEGADUCK_LAPTOP_SPANISH, [131](#)
- move_bkg
 - gb.h, [189](#)
 - msx.h, [337](#)
 - nes.h, [387](#)
 - sms.h, [422](#)
- move metasprite
 - metasprites.h, [303](#), [310](#), [314](#), [320](#)
- move metasprite_ex
 - metasprites.h, [303](#), [310](#), [313](#), [320](#)
- move metasprite_flipx
 - metasprites.h, [304](#), [314](#), [321](#)
- move metasprite_flipxy
 - metasprites.h, [305](#), [315](#), [322](#)
- move metasprite_flipy
 - metasprites.h, [304](#), [315](#), [321](#)
- move metasprite_hflip
 - metasprites.h, [305](#), [315](#)
- move metasprite_hvflip
 - metasprites.h, [306](#), [316](#)
- move metasprite_vflip
 - metasprites.h, [304](#), [315](#)
- move_sprite
 - gb.h, [198](#)
 - msx.h, [349](#)
 - nes.h, [391](#)
 - sms.h, [431](#)
- move_win
 - gb.h, [195](#)
- MSX
 - msx.h, [328](#)
- msx.h
 - _SYSTEM, [350](#)
 - __READ_VDP_REG, [330](#)
 - __WRITE_VDP_REG, [329](#)
 - __WRITE_VDP_REG_UNSAFE, [329](#)
 - _current_1bpp_colors, [351](#)
 - _current_2bpp_palette, [351](#)
 - _current_bank, [351](#)
 - _map_tile_offset, [352](#)
 - _shadow_OAM_OFF, [352](#)
 - _shadow_OAM_base, [352](#)
 - _submap_tile_offset, [352](#)
 - _vbl_done, [351](#)
 - add_JOY, [337](#)
 - add_LCD, [337](#)
 - add_SIO, [337](#)
 - add_TIM, [337](#)
 - add_VBL, [337](#)
 - b, [351](#)
 - BANK, [332](#)
 - BANKREF, [333](#)
 - BANKREF_EXTERN, [333](#)
 - c, [351](#)
 - cancel_pending_interrupts, [337](#)
 - COMPAT_PALETTE, [334](#)
 - cpu_fast, [340](#)
 - CURRENT_BANK, [332](#)
 - d, [351](#)
 - delay, [338](#)
 - DEVICE_SUPPORTS_COLOR, [332](#)
 - disable_interrupts, [339](#)
 - DISABLE_RAM, [334](#)
 - DISABLE_VBL_TRANSFER, [334](#)
 - DISPLAY_OFF, [331](#)
 - display_off, [338](#)
 - DISPLAY_ON, [331](#)
 - DIV_REG, [332](#)
 - e, [351](#)
 - EMPTY_IFLAG, [330](#)
 - enable_interrupts, [339](#)
 - ENABLE_RAM, [334](#)
 - ENABLE_VBL_TRANSFER, [334](#)
 - fill_bkg_rect, [334](#)
 - fill_rect, [345](#)
 - fill_win_rect, [334](#)
 - get_bkg_xy_addr, [350](#)
 - get_mode, [336](#)
 - get_r_reg, [338](#)
 - get_sprite_prop, [347](#)
 - get_sprite_tile, [346](#)
 - get_system, [336](#)
 - get_win_xy_addr, [335](#)
 - h, [351](#)
 - HARDWARE_SPRITE_CAN_FLIP_X, [335](#)
 - HARDWARE_SPRITE_CAN_FLIP_Y, [335](#)
 - HIDE_BKG, [332](#)
 - HIDE_LEFT_COLUMN, [331](#)

hide_sprite, 349
HIDE_SPRITES, 332
HIDE_WIN, 332
int_handler, 335
iyh, 351
iyl, 351
J_A, 328
J_B, 328
J_DOWN, 328
J_LEFT, 328
J_RIGHT, 328
J_SELECT, 328
J_START, 329
J_UP, 328
JOY_IFLAG, 330
joypad, 338
joypad_ex, 339
joypad_init, 338
l, 351
LCD_IFLAG, 330
M_NO_INTERP, 329
M_NO_SCROLL, 329
M_TEXT_INOUT, 329
M_TEXT_OUT, 329
MAX_HARDWARE_SPRITES, 335
MAXWNDPOSX, 331
MAXWNDPOSY, 331
MINWNDPOSX, 330
MINWNDPOSY, 330
mode, 335
move_bkg, 337
move_sprite, 349
MSX, 328
OAM_item_t, 335
refresh_OAM, 338
remove_JOY, 337
remove_LCD, 336
remove_SIO, 337
remove_TIM, 336
remove_VBL, 336
S_BANK, 329
S_FLIPX, 329
S_FLIPY, 329
S_PAL, 329
S_PALETTE, 329
S_PRIORITY, 329
SCREENHEIGHT, 330
SCREENWIDTH, 330
scroll_bkg, 337
scroll_sprite, 349
set_1bpp_colors, 341
set_2bpp_palette, 340
set_attributed_tile_xy, 350
set_bkg_1bpp_data, 341
set_bkg_4bpp_data, 340
set_bkg_based_submap, 344
set_bkg_based_tiles, 342
set_bkg_data, 341
set_bkg_palette, 334
set_bkg_palette_entry, 334
set_bkg_submap, 343
set_bkg_tile_xy, 335
set_bkg_tiles, 334
SET_BORDER_COLOR, 331
set_data, 341
set_default_palette, 339
set_interrupts, 336
set_native_sprite_data, 340
set_native_tile_data, 340
set_palette, 340
set_palette_entry, 340
SET_SHADOW_OAM_ADDRESS, 345
set_sprite_1bpp_data, 340
set_sprite_data, 341
set_sprite_palette, 334
set_sprite_palette_entry, 334
set_sprite_prop, 346
set_sprite_tile, 345
set_tile_1bpp_data, 341
set_tile_map, 341
set_tile_submap, 342
set_tile_submap_compat, 342
set_tile_xy, 350
set_vram_byte, 349
set_win_based_submap, 345
set_win_based_tiles, 342
set_win_submap, 343
set_win_tile_xy, 335
set_win_tiles, 334
shadow_OAM, 352
SHOW_BKG, 332
SHOW_LEFT_COLUMN, 331
SHOW_SPRITES, 332
SHOW_WIN, 332
SIO_IFLAG, 330
SPRITES_16x16, 332
SPRITES_8x8, 332
SWITCH_RAM, 333
SWITCH_ROM, 338
SWITCH_ROM1, 333
SWITCH_ROM2, 333
sys_time, 351
SYSTEM_50HZ, 328
SYSTEM_60HZ, 328
TIM_IFLAG, 330
VBK_REG, 328
VBL_DONE, 332
VBL_IFLAG, 330
vmemcpy, 341
vsync, 337
wait_vbl_done, 337
waitpad, 338
waitpadup, 338
WRITE_VDP_CMD, 335
WRITE_VDP_DATA, 335
NAKED

types.h, [482](#)

nes.h

- [_SYSTEM](#), [397](#)
- [_current_1bpp_colors](#), [397](#)
- [_current_bank](#), [397](#)
- [_shadow_OAM_base](#), [397](#)
- [_switch_prg0](#), [397](#)
- [add_LCD](#), [372](#)
- [add_TIM](#), [372](#)
- [add_VBL](#), [371](#)
- [BANK](#), [366](#)
- [BANKREF](#), [366](#)
- [BANKREF_EXTERN](#), [367](#)
- [COMPAT_PALETTE](#), [369](#)
- [CURRENT_BANK](#), [366](#)
- [delay](#), [373](#)
- [disable_interrupts](#), [375](#)
- [DISABLE_OAM_DMA](#), [369](#)
- [DISABLE_RAM](#), [368](#)
- [DISABLE_VBL_TRANSFER](#), [369](#)
- [DISPLAY_OFF](#), [368](#)
- [display_off](#), [376](#)
- [DISPLAY_ON](#), [368](#)
- [display_on](#), [376](#)
- [DMG_BLACK](#), [365](#)
- [DMG_DARK_GRAY](#), [365](#)
- [DMG_LITE_GRAY](#), [365](#)
- [DMG_PALETTE](#), [365](#)
- [DMG_WHITE](#), [365](#)
- [EMPTY_IFLAG](#), [365](#)
- [enable_interrupts](#), [375](#)
- [ENABLE_OAM_DMA](#), [369](#)
- [ENABLE_RAM](#), [368](#)
- [ENABLE_VBL_TRANSFER](#), [370](#)
- [fill_bkg_rect](#), [396](#)
- [fill_rect](#), [370](#)
- [flush_shadow_attributes](#), [397](#)
- [get_bkg_tile_xy](#), [386](#)
- [get_bkg_tiles](#), [385](#)
- [get_bkg_xy_addr](#), [377](#)
- [get_mode](#), [373](#)
- [get_sprite_prop](#), [391](#)
- [get_sprite_tile](#), [389](#)
- [get_system](#), [373](#)
- [HARDWARE_SPRITE_CAN_FLIP_X](#), [370](#)
- [HARDWARE_SPRITE_CAN_FLIP_Y](#), [370](#)
- [HIDE_BKG](#), [369](#)
- [HIDE_LEFT_COLUMN](#), [368](#)
- [hide_sprite](#), [393](#)
- [HIDE_SPRITES](#), [369](#)
- [init_bkg](#), [395](#)
- [int_handler](#), [370](#)
- [J_A](#), [363](#)
- [J_B](#), [363](#)
- [J_DOWN](#), [363](#)
- [J_LEFT](#), [363](#)
- [J_RIGHT](#), [363](#)
- [J_SELECT](#), [363](#)
- [J_START](#), [364](#)
- [J_UP](#), [363](#)
- [joypad](#), [373](#)
- [joypad_ex](#), [374](#)
- [joypad_init](#), [374](#)
- [LCD_IFLAG](#), [365](#)
- [M_DRAWING](#), [364](#)
- [M_NO_INTERP](#), [364](#)
- [M_NO_SCROLL](#), [364](#)
- [M_TEXT_INOUT](#), [364](#)
- [M_TEXT_OUT](#), [364](#)
- [MAX_HARDWARE_SPRITES](#), [370](#)
- [MAX_LCD_ISR_CALLS](#), [366](#)
- [mode](#), [372](#)
- [move_bkg](#), [387](#)
- [move_sprite](#), [391](#)
- [NINTENDO_NES](#), [361](#)
- [OAM_item_t](#), [370](#)
- [palette_color_t](#), [370](#)
- [refresh_OAM](#), [377](#)
- [remove_LCD](#), [371](#)
- [remove_TIM](#), [371](#)
- [remove_VBL](#), [371](#)
- [reset](#), [376](#)
- [RGB](#), [362](#)
- [RGB8](#), [362](#)
- [RGB_AQUA](#), [363](#)
- [RGB_BLACK](#), [363](#)
- [RGB_BLUE](#), [362](#)
- [RGB_CYAN](#), [363](#)
- [RGB_DARKBLUE](#), [362](#)
- [RGB_DARKGRAY](#), [363](#)
- [RGB_DARKGREEN](#), [362](#)
- [RGB_DARKRED](#), [362](#)
- [RGB_DARKYELLOW](#), [363](#)
- [RGB_GREEN](#), [362](#)
- [RGB_LIGHTGRAY](#), [363](#)
- [RGB_PINK](#), [363](#)
- [RGB_PURPLE](#), [363](#)
- [RGB_RED](#), [362](#)
- [RGB_WHITE](#), [363](#)
- [RGB_YELLOW](#), [362](#)
- [RGBHTML](#), [362](#)
- [S_FLIPX](#), [364](#)
- [S_FLIPY](#), [364](#)
- [S_PAL](#), [365](#)
- [S_PALETTE](#), [364](#)
- [S_PRIORITY](#), [364](#)
- [SCREENHEIGHT](#), [366](#)
- [SCREENWIDTH](#), [366](#)
- [scroll_bkg](#), [387](#)
- [scroll_sprite](#), [392](#)
- [set_1bpp_colors](#), [377](#)
- [set_1bpp_colors_ex](#), [377](#)
- [set_2bpp_palette](#), [377](#)
- [set_attribute_xy](#), [369](#)
- [set_bkg_1bpp_data](#), [378](#)
- [set_bkg_2bpp_data](#), [369](#)

set_bkg_attribute_xy, 386
set_bkg_attribute_xy_nes16x16, 386
set_bkg_attributes, 381
set_bkg_attributes_nes16x16, 380
set_bkg_based_submap, 384
set_bkg_based_tiles, 382
set_bkg_data, 377
set_bkg_native_data, 394
set_bkg_palette, 370
set_bkg_palette_entry, 370
set_bkg_submap, 383
set_bkg_submap_attributes, 382
set_bkg_submap_attributes_nes16x16, 381
set_bkg_tile_xy, 385
set_bkg_tiles, 379
SET_BORDER_COLOR, 368
set_data, 393
set_interrupts, 375
set_native_tile_data, 395
SET_SHADOW_OAM_ADDRESS, 389
set_sprite_1bpp_data, 388
set_sprite_2bpp_data, 369
set_sprite_data, 387
set_sprite_native_data, 395
set_sprite_palette, 370
set_sprite_palette_entry, 371
set_sprite_prop, 390
set_sprite_tile, 389
set_tile_data, 394
set_tile_map, 369
set_tile_submap, 369
set_tile_xy, 369
set_tiles, 393
set_vram_byte, 377
shadow_OAM, 397
SHOW_BKG, 369
SHOW_LEFT_COLUMN, 368
SHOW_SPRITES, 369
SPRITES_8x16, 369
SPRITES_8x8, 369
SWITCH_RAM, 368
SWITCH_ROM, 367
SWITCH_ROM_DUMMY, 367
SWITCH_ROM_UNROM, 367
sys_time, 397
SYSTEM_50HZ, 362
SYSTEM_60HZ, 362
SYSTEM_BITS_DENDY, 362
SYSTEM_BITS_NTSC, 361
SYSTEM_BITS_PAL, 361
TIM_IFLAG, 365
TIMER_VBLANK_PARITY_MODE_SYSTEM_50HZ, 362
TIMER_VBLANK_PARITY_MODE_SYSTEM_60HZ, 362
VBL_IFLAG, 365
vmemset, 396
vsync, 376
wait_vbl_done, 376
waitpad, 373
waitpadup, 374
NINTENDO
 gb.h, 161
NINTENDO_NES
 nes.h, 361
NONBANKED
 types.h, 482
NORETURN
 types.h, 482
noreturn
 stdnoreturn.h, 460
nowait_int_handler
 gb.h, 175
npads
 joypads_t, 99
NR10_REG
 hardware.h, 260
NR11_REG
 hardware.h, 260
NR12_REG
 hardware.h, 260
NR13_REG
 hardware.h, 260
NR14_REG
 hardware.h, 260
NR21_REG
 hardware.h, 260
NR22_REG
 hardware.h, 261
NR23_REG
 hardware.h, 261
NR24_REG
 hardware.h, 261
NR30_REG
 hardware.h, 261
NR31_REG
 hardware.h, 261
NR32_REG
 hardware.h, 261
NR33_REG
 hardware.h, 261
NR34_REG
 hardware.h, 261
NR41_REG
 hardware.h, 261
NR42_REG
 hardware.h, 261
NR43_REG
 hardware.h, 261
NR44_REG
 hardware.h, 261
NR50_REG
 hardware.h, 261
NR51_REG
 hardware.h, 261
NR52_REG

- hardware.h, 261
- NULL
 - stddef.h, 443
 - types.h, 486
- OAM_item_t, 101
 - gb.h, 172
 - msx.h, 335
 - nes.h, 370
 - prop, 101
 - tile, 101
 - x, 101
 - y, 101
- OAMF_BANK0
 - hardware.h, 257
- OAMF_BANK1
 - hardware.h, 257
- OAMF_CGB_PAL0
 - hardware.h, 257
- OAMF_CGB_PAL1
 - hardware.h, 257
- OAMF_CGB_PAL2
 - hardware.h, 257
- OAMF_CGB_PAL3
 - hardware.h, 257
- OAMF_CGB_PAL4
 - hardware.h, 257
- OAMF_CGB_PAL5
 - hardware.h, 257
- OAMF_CGB_PAL6
 - hardware.h, 257
- OAMF_CGB_PAL7
 - hardware.h, 257
- OAMF_PAL0
 - hardware.h, 257
- OAMF_PAL1
 - hardware.h, 257
- OAMF_PALMASK
 - hardware.h, 257
- OAMF_PRI
 - hardware.h, 257
- OAMF_XFLIP
 - hardware.h, 257
- OAMF_YFLIP
 - hardware.h, 257
- OBP0_REG
 - hardware.h, 262
- OBP1_REG
 - hardware.h, 262
- OCPD_REG
 - hardware.h, 263
- OCPS_REG
 - hardware.h, 263
- OCPSF_AUTOINC
 - hardware.h, 255
- offsetof
 - stddef.h, 443
- ofs
 - __far_ptr, 96
- OLDCALL
 - types.h, 481
- opcode
 - isr_nested_vector_t, 98
 - isr_vector_t, 98
- OR
 - drawing.h, 147
- P1_REG
 - hardware.h, 260
- P1F_0
 - hardware.h, 246
- P1F_1
 - hardware.h, 246
- P1F_2
 - hardware.h, 246
- P1F_3
 - hardware.h, 246
- P1F_4
 - hardware.h, 246
- P1F_5
 - hardware.h, 246
- P1F_GET_BTN
 - hardware.h, 246
- P1F_GET_DPAD
 - hardware.h, 246
- P1F_GET_NONE
 - hardware.h, 246
- palette_color_t
 - cgb.h, 142
 - nes.h, 370
- PCM12_REG
 - hardware.h, 263
- PCM34_REG
 - hardware.h, 263
- PCM_SAMPLE
 - hardware.h, 261
- plot
 - drawing.h, 149
- plot_point
 - drawing.h, 149
- pmfont_handle
 - font.h, 230
- POINTER
 - types.h, 486
- posx
 - console.h, 225
- posy
 - console.h, 225
- PPUCTRL_BG_CHR
 - hardware.h, 279
- PPUCTRL_INC32
 - hardware.h, 279
- PPUCTRL_NMI
 - hardware.h, 279
- PPUCTRL_SPR_8X16
 - hardware.h, 279
- PPUCTRL_SPR_8X8
 - hardware.h, 279

PPUCTRL_SPR_CHR
 [hardware.h, 279](#)

PPUMASK_BLUE
 [hardware.h, 279](#)

PPUMASK_GREEN
 [hardware.h, 279](#)

PPUMASK_MONOCHROME
 [hardware.h, 280](#)

PPUMASK_RED
 [hardware.h, 279](#)

PPUMASK_SHOW_BG
 [hardware.h, 279](#)

PPUMASK_SHOW_BG_LC
 [hardware.h, 280](#)

PPUMASK_SHOW_SPR
 [hardware.h, 279](#)

PPUMASK_SHOW_SPR_LC
 [hardware.h, 280](#)

PRESERVES_REGS
 [types.h, 481](#)

printf
 [stdio.h, 454](#)

prop
 OAM_item_t, [101](#)

props
 metasprite_t, [101](#)

provides.h
 USE_C_MEMCPY, [103](#), [104](#)
 USE_C_STRCMP, [103](#), [104](#)
 USE_C_STRCPY, [103](#), [104](#)

PSG_CH0
 [hardware.h, 271](#), [290](#)

PSG_CH1
 [hardware.h, 271](#), [290](#)

PSG_CH2
 [hardware.h, 271](#), [290](#)

PSG_CH3
 [hardware.h, 271](#), [290](#)

PSG_LATCH
 [hardware.h, 271](#), [290](#)

PSG_VOLUME
 [hardware.h, 271](#), [290](#)

ptr
 __far_ptr, [96](#)

PTRDIFF_MAX
 [stdint.h, 448](#)

PTRDIFF_MIN
 [stdint.h, 448](#)

ptrdiff_t
 [stddef.h, 443](#)

putchar
 [stdio.h, 454](#)

puts
 [stdio.h, 455](#)

qsort
 [stdlib.h, 459](#)

R0_CB_INPUT
 [hardware.h, 271](#)

R0_CB_OUTPUT
 [hardware.h, 271](#)

R0_DEFAULT
 [hardware.h, 271](#), [291](#)

R0_ES
 [hardware.h, 272](#), [291](#)

R0_ES_OFF
 [hardware.h, 272](#), [291](#)

R0_HSCRL
 [hardware.h, 291](#)

R0_HSCRL_INH
 [hardware.h, 291](#)

R0_IE1
 [hardware.h, 271](#), [291](#)

R0_IE1_OFF
 [hardware.h, 271](#), [291](#)

R0_IE2
 [hardware.h, 271](#)

R0_IE2_OFF
 [hardware.h, 271](#)

R0_LCB
 [hardware.h, 291](#)

R0_NO_LCB
 [hardware.h, 291](#)

R0_SCR_MODE1
 [hardware.h, 272](#)

R0_SCR_MODE2
 [hardware.h, 272](#)

R0_SCR_MODE3
 [hardware.h, 272](#)

R0_SS
 [hardware.h, 291](#)

R0_SS_OFF
 [hardware.h, 291](#)

R0_VSCRL
 [hardware.h, 291](#)

R0_VSCRL_INH
 [hardware.h, 291](#)

R10_INT_EVERY
 [hardware.h, 274](#), [293](#)

R10_INT_OFF
 [hardware.h, 274](#), [293](#)

R1_DEFAULT
 [hardware.h, 272](#), [291](#)

R1_DISP_OFF
 [hardware.h, 272](#), [291](#)

R1_DISP_ON
 [hardware.h, 272](#), [292](#)

R1_IE
 [hardware.h, 272](#), [292](#)

R1_IE_OFF
 [hardware.h, 272](#), [292](#)

R1_SCR_MODE1
 [hardware.h, 272](#)

R1_SCR_MODE2
 [hardware.h, 272](#)

R1_SCR_MODE3

- hardware.h, [272](#)
- R1_SPR_16X16
 - hardware.h, [272](#)
- R1_SPR_8X16
 - hardware.h, [292](#)
- R1_SPR_8X8
 - hardware.h, [272](#), [292](#)
- R1_SPR_MAG
 - hardware.h, [272](#)
- R1_SPR_MAG_OFF
 - hardware.h, [272](#)
- R2_MAP_0x0000
 - hardware.h, [273](#), [292](#)
- R2_MAP_0x0800
 - hardware.h, [273](#), [292](#)
- R2_MAP_0x1000
 - hardware.h, [273](#), [292](#)
- R2_MAP_0x1800
 - hardware.h, [273](#), [292](#)
- R2_MAP_0x2000
 - hardware.h, [273](#), [292](#)
- R2_MAP_0x2800
 - hardware.h, [273](#), [292](#)
- R2_MAP_0x3000
 - hardware.h, [273](#), [292](#)
- R2_MAP_0x3800
 - hardware.h, [273](#), [292](#)
- R5_SAT_0x1F00
 - hardware.h, [293](#)
- R5_SAT_0x3F00
 - hardware.h, [273](#), [292](#)
- R5_SAT_MASK
 - hardware.h, [273](#), [293](#)
- R6_BANK0
 - hardware.h, [273](#), [293](#)
- R6_BANK1
 - hardware.h, [273](#), [293](#)
- R6_DATA_0x0000
 - hardware.h, [273](#), [293](#)
- R6_DATA_0x2000
 - hardware.h, [274](#), [293](#)
- R7_COLOR_MASK
 - hardware.h, [274](#), [293](#)
- RAMCTL_BANK
 - hardware.h, [295](#)
- RAMCTL_PROT
 - hardware.h, [295](#)
- RAMCTL_RAM
 - hardware.h, [295](#)
- RAMCTL_RO
 - hardware.h, [295](#)
- RAMCTL_ROM
 - hardware.h, [295](#)
- rand
 - rand.h, [405](#)
- rand.h
 - __rand_seed, [405](#)
 - arand, [405](#)
 - c, [406](#)
 - initarand, [405](#)
 - initrand, [405](#)
 - rand, [405](#)
 - RAND_MAX, [404](#)
 - randw, [405](#)
 - RANDW_MAX, [404](#)
- RAND_MAX
 - rand.h, [404](#)
- randw
 - rand.h, [405](#)
- RANDW_MAX
 - rand.h, [404](#)
- rAUD1ENV
 - hardware.h, [248](#)
- rAUD1HIGH
 - hardware.h, [248](#)
- rAUD1LEN
 - hardware.h, [248](#)
- rAUD1LOW
 - hardware.h, [248](#)
- rAUD1SWEEP
 - hardware.h, [247](#)
- rAUD2ENV
 - hardware.h, [248](#)
- rAUD2HIGH
 - hardware.h, [248](#)
- rAUD2LEN
 - hardware.h, [248](#)
- rAUD2LOW
 - hardware.h, [248](#)
- rAUD3ENA
 - hardware.h, [248](#)
- rAUD3HIGH
 - hardware.h, [248](#)
- rAUD3LEN
 - hardware.h, [248](#)
- rAUD3LEVEL
 - hardware.h, [248](#)
- rAUD3LOW
 - hardware.h, [248](#)
- rAUD4ENV
 - hardware.h, [249](#)
- rAUD4GO
 - hardware.h, [249](#)
- rAUD4LEN
 - hardware.h, [249](#)
- rAUD4POLY
 - hardware.h, [249](#)
- rAUDENA
 - hardware.h, [250](#)
- rAUDTERM
 - hardware.h, [249](#)
- rAUDVOL
 - hardware.h, [249](#)
- rBCPD
 - hardware.h, [255](#)
- rBCPS

hardware.h, [255](#)
rBGP
hardware.h, [253](#)
rDIV
hardware.h, [247](#)
rDMA
hardware.h, [253](#)
realloc
stdlib.h, [458](#)
receive_byte
gb.h, [176](#)
refresh_OAM
gb.h, [179](#)
msx.h, [338](#)
nes.h, [377](#)
sms.h, [423](#)
remove_JOY
gb.h, [173](#)
msx.h, [337](#)
sms.h, [422](#)
remove_LCD
gb.h, [173](#)
msx.h, [336](#)
nes.h, [371](#)
sms.h, [421](#)
remove_SIO
gb.h, [173](#)
msx.h, [337](#)
sms.h, [421](#)
remove_TIM
gb.h, [173](#)
msx.h, [336](#)
nes.h, [371](#)
sms.h, [421](#)
remove_VBL
gb.h, [172](#)
msx.h, [336](#)
nes.h, [371](#)
sms.h, [421](#)
reset
gb.h, [179](#)
nes.h, [376](#)
RET_SIZE
setjmp.h, [407](#)
reverse
string.h, [463](#), [467](#), [471](#)
RGB
cgb.h, [140](#)
nes.h, [362](#)
RGB8
cgb.h, [140](#)
nes.h, [362](#)
RGB_AQUA
cgb.h, [141](#)
nes.h, [363](#)
RGB_BLACK
cgb.h, [141](#)
nes.h, [363](#)
RGB_BLUE
cgb.h, [141](#)
nes.h, [362](#)
RGB_BROWN
cgb.h, [142](#)
RGB_CYAN
cgb.h, [141](#)
nes.h, [363](#)
RGB_DARKBLUE
cgb.h, [141](#)
nes.h, [362](#)
RGB_DARKGRAY
cgb.h, [141](#)
nes.h, [363](#)
RGB_DARKGREEN
cgb.h, [141](#)
nes.h, [362](#)
RGB_DARKRED
cgb.h, [141](#)
nes.h, [362](#)
RGB_DARKYELLOW
cgb.h, [141](#)
nes.h, [363](#)
RGB_GREEN
cgb.h, [141](#)
nes.h, [362](#)
RGB_LIGHTFLESH
cgb.h, [142](#)
RGB_LIGHTGRAY
cgb.h, [141](#)
nes.h, [363](#)
RGB_ORANGE
cgb.h, [142](#)
RGB_PINK
cgb.h, [141](#)
nes.h, [363](#)
RGB_PURPLE
cgb.h, [141](#)
nes.h, [363](#)
RGB_RED
cgb.h, [141](#)
nes.h, [362](#)
RGB_TEAL
cgb.h, [142](#)
RGB_TO_NES
rgb_to_nes_macro.h, [403](#)
rgb_to_nes_macro.h
RGB_TO_NES, [403](#)
RGB_WHITE
cgb.h, [142](#)
nes.h, [363](#)
RGB_YELLOW
cgb.h, [141](#)
nes.h, [362](#)
RGBHTML
cgb.h, [140](#)
nes.h, [362](#)
rHDMA1

- hardware.h, [254](#)
- rHDMA2
 - hardware.h, [254](#)
- rHDMA3
 - hardware.h, [254](#)
- rHDMA4
 - hardware.h, [254](#)
- rHDMA5
 - hardware.h, [254](#)
- rIE
 - hardware.h, [255](#)
- rIF
 - hardware.h, [247](#)
- rKEY1
 - hardware.h, [253](#)
- rLDC
 - hardware.h, [250](#)
- rle_decompress
 - rledecompress.h, [235](#)
- rle_init
 - rledecompress.h, [235](#)
- RLE_STOP
 - rledecompress.h, [235](#)
- rledecompress.h
 - rle_decompress, [235](#)
 - rle_init, [235](#)
 - RLE_STOP, [235](#)
- rLY
 - hardware.h, [253](#), [281](#)
- rLYC
 - hardware.h, [253](#), [281](#)
- rMBC7_ACCEL_X_HI
 - hardware.h, [259](#)
- rMBC7_ACCEL_X_LO
 - hardware.h, [259](#)
- rMBC7_ACCEL_Y_HI
 - hardware.h, [260](#)
- rMBC7_ACCEL_Y_LO
 - hardware.h, [260](#)
- rMBC7_LATCH_1
 - hardware.h, [259](#)
- rMBC7_LATCH_2
 - hardware.h, [259](#)
- rMBC7_SRAM_ENABLE_1
 - hardware.h, [259](#)
- rMBC7_SRAM_ENABLE_2
 - hardware.h, [259](#)
- rOBP0
 - hardware.h, [253](#)
- rOBP1
 - hardware.h, [253](#)
- rOCPD
 - hardware.h, [255](#)
- rOCPS
 - hardware.h, [255](#)
- rP1
 - hardware.h, [246](#)
- RP_REG
 - hardware.h, [263](#)
- rPCM12
 - hardware.h, [255](#)
- rPCM34
 - hardware.h, [255](#)
- RPF_DATAIN
 - hardware.h, [255](#)
- RPF_ENREAD
 - hardware.h, [255](#)
- RPF_WRITE_HI
 - hardware.h, [255](#)
- RPF_WRITE_LO
 - hardware.h, [255](#)
- rRAMB
 - hardware.h, [259](#)
- rRAMG
 - hardware.h, [259](#)
- rROMB0
 - hardware.h, [259](#)
- rROMB1
 - hardware.h, [259](#)
- rRP
 - hardware.h, [255](#)
- rSB
 - hardware.h, [246](#)
- rSC
 - hardware.h, [246](#)
- rSCX
 - hardware.h, [252](#), [281](#)
- rSCY
 - hardware.h, [252](#), [281](#)
- rSMBK
 - hardware.h, [255](#)
- rSPD
 - hardware.h, [253](#)
- rSTAT
 - hardware.h, [251](#)
- rSVBK
 - hardware.h, [255](#)
- rTAC
 - hardware.h, [247](#)
- rTIMA
 - hardware.h, [247](#)
- rTMA
 - hardware.h, [247](#)
- rVBK
 - hardware.h, [253](#)
- rWX
 - hardware.h, [253](#)
- rWY
 - hardware.h, [253](#)
- S_BANK
 - gb.h, [162](#)
 - msx.h, [329](#)
 - sms.h, [414](#)
- S_FLIPX
 - gb.h, [162](#)
 - msx.h, [329](#)

- nes.h, [364](#)
- sms.h, [414](#)
- S_FLIPY
 - gb.h, [162](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [414](#)
- S_PAL
 - gb.h, [163](#)
 - msx.h, [329](#)
 - nes.h, [365](#)
 - sms.h, [414](#)
- S_PALETTE
 - gb.h, [162](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [414](#)
- S_PRIORITY
 - gb.h, [163](#)
 - msx.h, [329](#)
 - nes.h, [364](#)
 - sms.h, [414](#)
- SB_REG
 - hardware.h, [260](#)
- SC_REG
 - hardware.h, [260](#)
- SCF_SOURCE
 - hardware.h, [247](#)
- SCF_SPEED
 - hardware.h, [247](#)
- SCF_START
 - hardware.h, [247](#)
- SCHAR_MAX
 - limits.h, [237](#)
- SCHAR_MIN
 - limits.h, [237](#)
- SCREENHEIGHT
 - gb.h, [164](#)
 - msx.h, [330](#)
 - nes.h, [366](#)
 - sms.h, [415](#)
- SCREENWIDTH
 - gb.h, [164](#)
 - msx.h, [330](#)
 - nes.h, [366](#)
 - sms.h, [415](#)
- scroll_bkg
 - gb.h, [189](#)
 - msx.h, [337](#)
 - nes.h, [387](#)
 - sms.h, [422](#)
- scroll_sprite
 - gb.h, [199](#)
 - msx.h, [349](#)
 - nes.h, [392](#)
 - sms.h, [432](#)
- scroll_win
 - gb.h, [195](#)
- scroll_x_t
 - hardware.h, [281](#)
- scroll_y_t
 - hardware.h, [281](#)
- SCX_REG
 - hardware.h, [262](#), [281](#)
- SCY_REG
 - hardware.h, [262](#), [280](#)
- seg
 - __far_ptr, [96](#)
- SEGA
 - sms.h, [413](#)
- segn
 - __far_ptr, [96](#)
- segofs
 - __far_ptr, [96](#)
- send_byte
 - gb.h, [176](#)
- set_1bpp_colors
 - gb.h, [181](#)
 - msx.h, [341](#)
 - nes.h, [377](#)
 - sms.h, [427](#)
- set_1bpp_colors_ex
 - gb.h, [180](#)
 - nes.h, [377](#)
- set_2bpp_palette
 - gb.h, [180](#)
 - msx.h, [340](#)
 - nes.h, [377](#)
 - sms.h, [426](#)
- set_attribute_xy
 - gb.h, [171](#)
 - nes.h, [369](#)
 - sms.h, [433](#)
- set_attributed_tile_xy
 - msx.h, [350](#)
 - sms.h, [432](#)
- set_bkg_1bpp_data
 - gb.h, [182](#)
 - msx.h, [341](#)
 - nes.h, [378](#)
 - sms.h, [427](#)
- set_bkg_2bpp_data
 - gb.h, [171](#)
 - nes.h, [369](#)
 - sms.h, [426](#)
- set_bkg_4bpp_data
 - msx.h, [340](#)
 - sms.h, [425](#)
- set_bkg_attribute_xy
 - gb.h, [188](#)
 - nes.h, [386](#)
 - sms.h, [420](#)
- set_bkg_attribute_xy_nes16x16
 - nes.h, [386](#)
- set_bkg_attributes
 - gb.h, [184](#)

- nes.h, [381](#)
- sms.h, [428](#)
- set_bkg_attributes_nes16x16
 - nes.h, [380](#)
- set_bkg_based_submap
 - gb.h, [186](#)
 - msx.h, [344](#)
 - nes.h, [384](#)
 - sms.h, [429](#)
- set_bkg_based_tiles
 - gb.h, [184](#)
 - msx.h, [342](#)
 - nes.h, [382](#)
 - sms.h, [428](#)
- set_bkg_data
 - gb.h, [181](#)
 - msx.h, [341](#)
 - nes.h, [377](#)
 - sms.h, [426](#)
- set_bkg_native_data
 - gb.h, [203](#)
 - nes.h, [394](#)
 - sms.h, [426](#)
- set_bkg_palette
 - cgb.h, [142](#)
 - msx.h, [334](#)
 - nes.h, [370](#)
 - sms.h, [419](#)
- set_bkg_palette_entry
 - cgb.h, [143](#)
 - msx.h, [334](#)
 - nes.h, [370](#)
 - sms.h, [419](#)
- set_bkg_submap
 - gb.h, [185](#)
 - msx.h, [343](#)
 - nes.h, [383](#)
 - sms.h, [419](#)
- set_bkg_submap_attributes
 - gb.h, [187](#)
 - nes.h, [382](#)
 - sms.h, [430](#)
- set_bkg_submap_attributes_nes16x16
 - nes.h, [381](#)
- set_bkg_tile_xy
 - gb.h, [188](#)
 - msx.h, [335](#)
 - nes.h, [385](#)
 - sms.h, [420](#)
- set_bkg_tiles
 - gb.h, [183](#)
 - msx.h, [334](#)
 - nes.h, [379](#)
 - sms.h, [419](#)
- SET_BORDER_COLOR
 - gb.h, [170](#)
 - msx.h, [331](#)
 - nes.h, [368](#)
- sms.h, [416](#)
- set_data
 - gb.h, [199](#)
 - msx.h, [341](#)
 - nes.h, [393](#)
 - sms.h, [427](#)
- set_default_palette
 - cgb.h, [144](#)
 - msx.h, [339](#)
 - sms.h, [424](#)
- set_interrupts
 - gb.h, [179](#)
 - msx.h, [336](#)
 - nes.h, [375](#)
 - sms.h, [421](#)
- set_native_sprite_data
 - msx.h, [340](#)
- set_native_tile_data
 - gb.h, [202](#)
 - msx.h, [340](#)
 - nes.h, [395](#)
 - sms.h, [425](#)
- set_palette
 - msx.h, [340](#)
 - sms.h, [425](#)
- set_palette_entry
 - msx.h, [340](#)
 - sms.h, [425](#)
- SET_SHADOW_OAM_ADDRESS
 - gb.h, [197](#)
 - msx.h, [345](#)
 - nes.h, [389](#)
 - sms.h, [430](#)
- set_sprite_1bpp_data
 - gb.h, [196](#)
 - msx.h, [340](#)
 - nes.h, [388](#)
 - sms.h, [427](#)
- set_sprite_2bpp_data
 - gb.h, [171](#)
 - nes.h, [369](#)
 - sms.h, [426](#)
- set_sprite_4bpp_data
 - sms.h, [426](#)
- set_sprite_data
 - gb.h, [195](#)
 - msx.h, [341](#)
 - nes.h, [387](#)
 - sms.h, [426](#)
- set_sprite_native_data
 - gb.h, [203](#)
 - nes.h, [395](#)
 - sms.h, [426](#)
- set_sprite_palette
 - cgb.h, [142](#)
 - msx.h, [334](#)
 - nes.h, [370](#)
 - sms.h, [419](#)

- set_sprite_palette_entry
 - cgb.h, [143](#)
 - msx.h, [334](#)
 - nes.h, [371](#)
 - sms.h, [419](#)
- set_sprite_prop
 - gb.h, [197](#)
 - msx.h, [346](#)
 - nes.h, [390](#)
 - sms.h, [431](#)
- set_sprite_tile
 - gb.h, [197](#)
 - msx.h, [345](#)
 - nes.h, [389](#)
 - sms.h, [430](#)
- set_tile_1bpp_data
 - msx.h, [341](#)
 - sms.h, [427](#)
- set_tile_2bpp_data
 - sms.h, [426](#)
- set_tile_data
 - gb.h, [201](#)
 - nes.h, [394](#)
- set_tile_map
 - gb.h, [171](#)
 - msx.h, [341](#)
 - nes.h, [369](#)
 - sms.h, [427](#)
- set_tile_map_compat
 - sms.h, [428](#)
- set_tile_submap
 - gb.h, [171](#)
 - msx.h, [342](#)
 - nes.h, [369](#)
 - sms.h, [428](#)
- set_tile_submap_compat
 - msx.h, [342](#)
 - sms.h, [429](#)
- set_tile_xy
 - gb.h, [171](#)
 - msx.h, [350](#)
 - nes.h, [369](#)
 - sms.h, [433](#)
- set_tiles
 - gb.h, [201](#)
 - nes.h, [393](#)
- set_vram_byte
 - gb.h, [180](#)
 - msx.h, [349](#)
 - nes.h, [377](#)
 - sms.h, [432](#)
- set_win_1bpp_data
 - gb.h, [190](#)
- set_win_attribute_xy
 - sms.h, [420](#)
- set_win_based_submap
 - gb.h, [193](#)
 - msx.h, [345](#)
 - sms.h, [430](#)
- set_win_based_tiles
 - gb.h, [192](#)
 - msx.h, [342](#)
 - sms.h, [428](#)
- set_win_data
 - gb.h, [190](#)
- set_win_submap
 - gb.h, [192](#)
 - msx.h, [343](#)
 - sms.h, [419](#)
- set_win_tile_xy
 - gb.h, [194](#)
 - msx.h, [335](#)
 - sms.h, [420](#)
- set_win_tiles
 - gb.h, [191](#)
 - msx.h, [334](#)
 - sms.h, [419](#)
- setchar
 - console.h, [225](#)
- setjmp
 - setjmp.h, [407](#)
- setjmp.h
 - __setjmp, [407](#)
 - BP_SIZE, [407](#)
 - BPX_SIZE, [407](#)
 - jmp_buf, [407](#)
 - longjmp, [407](#)
 - RET_SIZE, [407](#)
 - setjmp, [407](#)
 - SP_SIZE, [407](#)
 - SPX_SIZE, [407](#)
- sfont_handle, [102](#)
 - first_tile, [102](#)
 - font, [102](#)
- SFR
 - types.h, [482](#)
- sgb.h
 - c, [224](#)
 - SGB_ATTRC_EN, [222](#)
 - SGB_ATTR_BLK, [222](#)
 - SGB_ATTR_CHR, [222](#)
 - SGB_ATTR_DIV, [222](#)
 - SGB_ATTR_LIN, [222](#)
 - SGB_ATTR_SET, [223](#)
 - SGB_ATTR_TRN, [223](#)
 - sgb_check, [223](#)
 - SGB_CHR_TRN, [223](#)
 - SGB_DATA_SND, [223](#)
 - SGB_DATA_TRN, [223](#)
 - SGB_ICON_EN, [223](#)
 - SGB_JUMP, [223](#)
 - SGB_MASK_EN, [223](#)
 - SGB_MLT_REQ, [223](#)
 - SGB_OBJ_TRN, [223](#)
 - SGB_PAL_01, [222](#)
 - SGB_PAL_03, [222](#)

SGB_PAL_12, [222](#)
 SGB_PAL_23, [222](#)
 SGB_PAL_SET, [222](#)
 SGB_PAL_TRN, [222](#)
 SGB_PCT_TRN, [223](#)
 SGB_SOU_TRN, [222](#)
 SGB_SOUND, [222](#)
 SGB_TEST_EN, [222](#)
 sgb_transfer, [223](#)
 SGB_ATTRC_EN
 sgb.h, [222](#)
 SGB_ATTR_BLK
 sgb.h, [222](#)
 SGB_ATTR_CHR
 sgb.h, [222](#)
 SGB_ATTR_DIV
 sgb.h, [222](#)
 SGB_ATTR_LIN
 sgb.h, [222](#)
 SGB_ATTR_SET
 sgb.h, [223](#)
 SGB_ATTR_TRN
 sgb.h, [223](#)
 sgb_check
 sgb.h, [223](#)
 SGB_CHR_TRN
 sgb.h, [223](#)
 SGB_DATA_SND
 sgb.h, [223](#)
 SGB_DATA_TRN
 sgb.h, [223](#)
 SGB_ICON_EN
 sgb.h, [223](#)
 SGB_JUMP
 sgb.h, [223](#)
 SGB_MASK_EN
 sgb.h, [223](#)
 SGB_MLT_REQ
 sgb.h, [223](#)
 SGB_OBJ_TRN
 sgb.h, [223](#)
 SGB_PAL_01
 sgb.h, [222](#)
 SGB_PAL_03
 sgb.h, [222](#)
 SGB_PAL_12
 sgb.h, [222](#)
 SGB_PAL_23
 sgb.h, [222](#)
 SGB_PAL_SET
 sgb.h, [222](#)
 SGB_PAL_TRN
 sgb.h, [222](#)
 SGB_PCT_TRN
 sgb.h, [223](#)
 SGB_SOU_TRN
 sgb.h, [222](#)
 SGB_SOUND
 sgb.h, [222](#)
 SGB_TEST_EN
 sgb.h, [222](#)
 sgb_transfer
 sgb.h, [223](#)
 shadow_OAM
 gb.h, [206](#)
 msx.h, [352](#)
 nes.h, [397](#)
 sms.h, [435](#)
 shadow_PPUCTRL
 hardware.h, [282](#)
 shadow_PPUMASK
 hardware.h, [282](#)
 shadow_VDP_R0
 hardware.h, [275](#), [296](#)
 shadow_VDP_R1
 hardware.h, [275](#), [296](#)
 shadow_VDP_R10
 hardware.h, [275](#), [296](#)
 shadow_VDP_R2
 hardware.h, [275](#), [296](#)
 shadow_VDP_R3
 hardware.h, [275](#), [296](#)
 shadow_VDP_R4
 hardware.h, [275](#), [296](#)
 shadow_VDP_R5
 hardware.h, [275](#), [296](#)
 shadow_VDP_R6
 hardware.h, [275](#), [296](#)
 shadow_VDP_R7
 hardware.h, [275](#), [296](#)
 shadow_VDP_R8
 hardware.h, [275](#), [296](#)
 shadow_VDP_R9
 hardware.h, [275](#), [296](#)
 shadow_VDP_RBORDER
 hardware.h, [275](#), [296](#)
 shadow_VDP_RSCX
 hardware.h, [275](#), [296](#)
 shadow_VDP_RSCY
 hardware.h, [275](#), [296](#)
 SHOW_BKG
 gb.h, [170](#)
 msx.h, [332](#)
 nes.h, [369](#)
 sms.h, [416](#)
 SHOW_LEFT_COLUMN
 gb.h, [170](#)
 msx.h, [331](#)
 nes.h, [368](#)
 sms.h, [416](#)
 SHOW_SPRITES
 gb.h, [171](#)
 msx.h, [332](#)
 nes.h, [369](#)
 sms.h, [416](#)
 SHOW_WIN

- gb.h, [170](#)
- msx.h, [332](#)
- sms.h, [416](#)
- SHRT_MAX
 - limits.h, [238](#)
- SHRT_MIN
 - limits.h, [238](#)
- SIG_ATOMIC_MAX
 - stdint.h, [448](#)
- SIG_ATOMIC_MIN
 - stdint.h, [448](#)
- SIGNED
 - drawing.h, [147](#)
- SIO_IFLAG
 - gb.h, [163](#)
 - msx.h, [330](#)
 - sms.h, [415](#)
- SIOCTL_BS0
 - hardware.h, [288](#)
- SIOCTL_BS1
 - hardware.h, [288](#)
- SIOCTL_FRER
 - hardware.h, [287](#)
- SIOCTL_INT
 - hardware.h, [287](#)
- SIOCTL_ROM
 - hardware.h, [288](#)
- SIOCTL_RXRD
 - hardware.h, [287](#)
- SIOCTL_TON
 - hardware.h, [288](#)
- SIOCTL_TXFL
 - hardware.h, [287](#)
- SIOF_B_CLOCK
 - hardware.h, [247](#)
- SIOF_B_SPEED
 - hardware.h, [247](#)
- SIOF_B_XFER_START
 - hardware.h, [247](#)
- SIOF_CLOCK_EXT
 - hardware.h, [246](#)
- SIOF_CLOCK_INT
 - hardware.h, [246](#)
- SIOF_SPEED_1X
 - hardware.h, [246](#)
- SIOF_SPEED_32X
 - hardware.h, [246](#)
- SIOF_XFER_START
 - hardware.h, [246](#)
- SIZE_MAX
 - stdint.h, [448](#)
- size_t
 - stddef.h, [444](#)
 - types.h, [479](#), [480](#), [485](#)
- sms.h
 - _BIOS, [434](#)
 - _SYSTEM, [434](#)
 - __READ_VDP_REG, [414](#)
 - __WRITE_VDP_REG, [414](#)
 - __WRITE_VDP_REG_UNSAFE, [414](#)
 - _current_1bpp_colors, [434](#)
 - _current_2bpp_palette, [434](#)
 - _current_bank, [417](#)
 - _map_tile_offset, [434](#)
 - _shadow_OAM_OFF, [435](#)
 - _shadow_OAM_base, [435](#)
 - _sprites_OFF, [435](#)
 - _submap_tile_offset, [434](#)
 - _vbl_done, [434](#)
 - add_JOY, [422](#)
 - add_LCD, [422](#)
 - add_SIO, [422](#)
 - add_TIM, [422](#)
 - add_VBL, [422](#)
 - b, [434](#)
 - BANK, [417](#)
 - BANKREF, [417](#)
 - BANKREF_EXTERN, [418](#)
 - c, [434](#)
 - cancel_pending_interrupts, [422](#)
 - cgb_compatibility, [424](#)
 - COMPAT_PALETTE, [419](#)
 - cpu_fast, [424](#)
 - CURRENT_BANK, [417](#)
 - d, [434](#)
 - delay, [423](#)
 - DEVICE_SUPPORTS_COLOR, [417](#)
 - disable_interrupts, [424](#)
 - DISABLE_RAM, [419](#)
 - DISABLE_VBL_TRANSFER, [419](#)
 - DISPLAY_OFF, [416](#)
 - display_off, [422](#)
 - DISPLAY_ON, [416](#)
 - DIV_REG, [417](#)
 - e, [434](#)
 - EMPTY_IFLAG, [414](#)
 - enable_interrupts, [424](#)
 - ENABLE_RAM, [419](#)
 - ENABLE_VBL_TRANSFER, [419](#)
 - fill_bkg_rect, [419](#)
 - fill_rect, [430](#)
 - fill_rect_compat, [430](#)
 - fill_win_rect, [419](#)
 - get_bkg_xy_addr, [433](#)
 - get_mode, [420](#)
 - get_r_reg, [423](#)
 - get_sprite_prop, [431](#)
 - get_sprite_tile, [431](#)
 - get_system, [421](#)
 - get_win_xy_addr, [420](#)
 - h, [434](#)
 - HARDWARE_SPRITE_CAN_FLIP_X, [420](#)
 - HARDWARE_SPRITE_CAN_FLIP_Y, [420](#)
 - HIDE_BKG, [416](#)
 - HIDE_LEFT_COLUMN, [416](#)
 - hide_sprite, [432](#)

HIDE_SPRITES, 417
HIDE_WIN, 416
int_handler, 420
iyh, 434
iyl, 434
J_A, 413
J_B, 413
J_DOWN, 413
J_LEFT, 413
J_RIGHT, 413
J_SELECT, 413
J_START, 413
J_UP, 413
JOY_IFLAG, 415
joypad, 423
joypad_ex, 423
joypad_init, 423
l, 434
LCD_IFLAG, 415
M_NO_INTERP, 414
M_NO_SCROLL, 414
M_TEXT_INOUT, 413
M_TEXT_OUT, 413
MAX_HARDWARE_SPRITES, 420
MAXWNDPOSX, 415
MAXWNDPOSY, 415
MINWNDPOSX, 415
MINWNDPOSY, 415
mode, 420
move_bkg, 422
move_sprite, 431
refresh_OAM, 423
remove_JOY, 422
remove_LCD, 421
remove_SIO, 421
remove_TIM, 421
remove_VBL, 421
S_BANK, 414
S_FLIPX, 414
S_FLIPY, 414
S_PAL, 414
S_PALETTE, 414
S_PRIORITY, 414
SCREENHEIGHT, 415
SCREENWIDTH, 415
scroll_bkg, 422
scroll_sprite, 432
SEGA, 413
set_1bpp_colors, 427
set_2bpp_palette, 426
set_attribute_xy, 433
set_attributed_tile_xy, 432
set_bkg_1bpp_data, 427
set_bkg_2bpp_data, 426
set_bkg_4bpp_data, 425
set_bkg_attribute_xy, 420
set_bkg_attributes, 428
set_bkg_based_submap, 429
set_bkg_based_tiles, 428
set_bkg_data, 426
set_bkg_native_data, 426
set_bkg_palette, 419
set_bkg_palette_entry, 419
set_bkg_submap, 419
set_bkg_submap_attributes, 430
set_bkg_tile_xy, 420
set_bkg_tiles, 419
SET_BORDER_COLOR, 416
set_data, 427
set_default_palette, 424
set_interrupts, 421
set_native_tile_data, 425
set_palette, 425
set_palette_entry, 425
SET_SHADOW_OAM_ADDRESS, 430
set_sprite_1bpp_data, 427
set_sprite_2bpp_data, 426
set_sprite_4bpp_data, 426
set_sprite_data, 426
set_sprite_native_data, 426
set_sprite_palette, 419
set_sprite_palette_entry, 419
set_sprite_prop, 431
set_sprite_tile, 430
set_tile_1bpp_data, 427
set_tile_2bpp_data, 426
set_tile_map, 427
set_tile_map_compat, 428
set_tile_submap, 428
set_tile_submap_compat, 429
set_tile_xy, 433
set_vram_byte, 432
set_win_attribute_xy, 420
set_win_based_submap, 430
set_win_based_tiles, 428
set_win_submap, 419
set_win_tile_xy, 420
set_win_tiles, 419
shadow_OAM, 435
SHOW_BKG, 416
SHOW_LEFT_COLUMN, 416
SHOW_SPRITES, 416
SHOW_WIN, 416
SIO_IFLAG, 415
SPRITES_8x16, 417
SPRITES_8x8, 417
SWITCH_RAM, 418
SWITCH_ROM, 418
SWITCH_ROM1, 418
SWITCH_ROM2, 418
sys_time, 434
SYSTEM_50HZ, 413
SYSTEM_60HZ, 413
TIM_IFLAG, 415
VBK_REG, 413
VBL_DONE, 417

- VBL_IFLAG, [414](#)
- vmemcpy, [427](#)
- vsync, [422](#)
- wait_vbl_done, [422](#)
- waitpad, [423](#)
- waitpadup, [423](#)
- WRITE_VDP_CMD, [420](#)
- WRITE_VDP_DATA, [420](#)
- SOLID
 - drawing.h, [147](#)
- SOUNDPAN_NOSL
 - hardware.h, [288](#)
- SOUNDPAN_NOSR
 - hardware.h, [288](#)
- SOUNDPAN_TN1L
 - hardware.h, [288](#)
- SOUNDPAN_TN1R
 - hardware.h, [288](#)
- SOUNDPAN_TN2L
 - hardware.h, [288](#)
- SOUNDPAN_TN2R
 - hardware.h, [288](#)
- SOUNDPAN_TN3L
 - hardware.h, [288](#)
- SOUNDPAN_TN3R
 - hardware.h, [288](#)
- SP_SIZE
 - setjmp.h, [407](#)
- sprintf
 - stdio.h, [455](#)
- SPRITES_16x16
 - msx.h, [332](#)
- SPRITES_8x16
 - gb.h, [171](#)
 - nes.h, [369](#)
 - sms.h, [417](#)
- SPRITES_8x8
 - gb.h, [171](#)
 - msx.h, [332](#)
 - nes.h, [369](#)
 - sms.h, [417](#)
- SPX_SIZE
 - setjmp.h, [407](#)
- STAT_REG
 - hardware.h, [262](#)
- STATF_9_SPR
 - hardware.h, [271](#), [290](#)
- STATF_B_BUSY
 - hardware.h, [252](#)
- STATF_B_LYC
 - hardware.h, [252](#)
- STATF_B_LYCF
 - hardware.h, [252](#)
- STATF_B_MODE00
 - hardware.h, [252](#)
- STATF_B_MODE01
 - hardware.h, [252](#)
- STATF_B_MODE10
 - hardware.h, [252](#)
- hardware.h, [252](#)
- STATF_B_OAM
 - hardware.h, [252](#)
- STATF_B_VBL
 - hardware.h, [252](#)
- STATF_BUSY
 - hardware.h, [252](#)
- STATF_HBL
 - hardware.h, [252](#)
- STATF_INT_VBL
 - hardware.h, [271](#), [290](#)
- STATF_LCD
 - hardware.h, [252](#)
- STATF_LYC
 - hardware.h, [251](#)
- STATF_LYCF
 - hardware.h, [252](#)
- STATF_MODE00
 - hardware.h, [252](#)
- STATF_MODE01
 - hardware.h, [251](#)
- STATF_MODE10
 - hardware.h, [251](#)
- STATF_OAM
 - hardware.h, [252](#)
- STATF_SPR_COLL
 - hardware.h, [271](#), [290](#)
- STATF_VBL
 - hardware.h, [252](#)
- stdarg.h
 - va_arg, [105](#), [106](#)
 - va_end, [105](#), [106](#)
 - va_list, [105–107](#)
 - va_start, [105](#), [106](#)
- stdatomic.h
 - atomic_flag_clear, [441](#)
 - atomic_flag_test_and_set, [441](#)
- stdbool.h
 - __bool_true_false_are_defined, [442](#)
 - bool, [442](#)
 - false, [442](#)
 - true, [442](#)
- stddef.h
 - __PTRDIFF_T_DEFINED, [443](#)
 - __SIZE_T_DEFINED, [443](#)
 - __WCHAR_T_DEFINED, [443](#)
 - NULL, [443](#)
 - offsetof, [443](#)
 - ptrdiff_t, [443](#)
 - size_t, [444](#)
 - wchar_t, [444](#)
- stdint.h
 - INT16_C, [448](#)
 - INT16_MAX, [446](#)
 - INT16_MIN, [446](#)
 - int16_t, [449](#)
 - INT32_C, [448](#)
 - INT32_MAX, [446](#)

INT32_MIN, 446
 int32_t, 449
 INT8_C, 448
 INT8_MAX, 446
 INT8_MIN, 446
 int8_t, 449
 INT_FAST16_MAX, 447
 INT_FAST16_MIN, 447
 int_fast16_t, 450
 INT_FAST32_MAX, 447
 INT_FAST32_MIN, 447
 int_fast32_t, 450
 INT_FAST8_MAX, 447
 INT_FAST8_MIN, 447
 int_fast8_t, 450
 INT_LEAST16_MAX, 447
 INT_LEAST16_MIN, 447
 int_least16_t, 449
 INT_LEAST32_MAX, 447
 INT_LEAST32_MIN, 447
 int_least32_t, 449
 INT_LEAST8_MAX, 447
 INT_LEAST8_MIN, 447
 int_least8_t, 449
 INTMAX_C, 449
 INTMAX_MAX, 448
 INTMAX_MIN, 448
 intmax_t, 450
 INTPTR_MAX, 448
 INTPTR_MIN, 448
 intptr_t, 450
 PTRDIFF_MAX, 448
 PTRDIFF_MIN, 448
 SIG_ATOMIC_MAX, 448
 SIG_ATOMIC_MIN, 448
 SIZE_MAX, 448
 UINT16_C, 448
 UINT16_MAX, 446
 uint16_t, 449
 UINT32_C, 449
 UINT32_MAX, 447
 uint32_t, 449
 UINT8_C, 448
 UINT8_MAX, 446
 uint8_t, 449
 UINT_FAST16_MAX, 447
 uint_fast16_t, 450
 UINT_FAST32_MAX, 448
 uint_fast32_t, 450
 UINT_FAST8_MAX, 447
 uint_fast8_t, 450
 UINT_LEAST16_MAX, 447
 uint_least16_t, 450
 UINT_LEAST32_MAX, 447
 uint_least32_t, 450
 UINT_LEAST8_MAX, 447
 uint_least8_t, 450
 UINTMAX_C, 449
 UINTMAX_MAX, 448
 uintmax_t, 450
 UINTPTR_MAX, 448
 uintptr_t, 450
 WCHAR_MAX, 449
 WCHAR_MIN, 449
 WINT_MAX, 449
 WINT_MIN, 449
 stdio.h
 getchar, 455
 gets, 455
 printf, 454
 putchar, 454
 puts, 455
 sprintf, 455
 stdlib.h
 abs, 456
 atoi, 456
 atol, 457
 bsearch, 458
 calloc, 458
 exit, 456
 free, 458
 itoa, 457
 labs, 456
 ltoa, 457
 malloc, 458
 qsort, 459
 realloc, 458
 uitoa, 457
 ultoa, 458
 stdnoreturn.h
 noreturn, 460
 strcat
 string.h, 463, 467, 471
 strcmp
 string.h, 461, 466, 470
 strcpy
 string.h, 461, 466, 470
 string.h
 __memcpy, 461
 c, 469
 memcmp, 465, 469, 473
 memcpy, 461, 466, 471
 memmove, 463, 467, 471
 memset, 463, 467, 471
 reverse, 463, 467, 471
 strcat, 463, 467, 471
 strcmp, 461, 466, 470
 strcpy, 461, 466, 470
 strlen, 463, 468, 472
 strncat, 464, 468, 472
 strncmp, 464, 468, 472
 strncpy, 464, 468, 473
 strlen
 string.h, 463, 468, 472
 strncat
 string.h, 464, 468, 472

strncmp
 string.h, [464](#), [468](#), [472](#)

strncpy
 string.h, [464](#), [468](#), [473](#)

SVBK_REG
 hardware.h, [263](#)

SWITCH_16_8_MODE_MBC1
 gb.h, [169](#)

SWITCH_4_32_MODE_MBC1
 gb.h, [169](#)

switch_data
 drawing.h, [149](#)

SWITCH_RAM
 gb.h, [167](#)
 msx.h, [333](#)
 nes.h, [368](#)
 sms.h, [418](#)

SWITCH_RAM_MBC1
 gb.h, [168](#)

SWITCH_RAM_MBC5
 gb.h, [169](#)

SWITCH_ROM
 gb.h, [167](#)
 msx.h, [338](#)
 nes.h, [367](#)
 sms.h, [418](#)

SWITCH_ROM1
 msx.h, [333](#)
 sms.h, [418](#)

SWITCH_ROM2
 msx.h, [333](#)
 sms.h, [418](#)

SWITCH_ROM_DUMMY
 nes.h, [367](#)

SWITCH_ROM_MBC1
 gb.h, [168](#)

SWITCH_ROM_MBC5
 gb.h, [169](#)

SWITCH_ROM_MBC5_8M
 gb.h, [169](#)

SWITCH_ROM_MEGADUCK
 gb.h, [168](#)

SWITCH_ROM_UNROM
 nes.h, [367](#)

sys_time
 gb.h, [205](#)
 msx.h, [351](#)
 nes.h, [397](#)
 sms.h, [434](#)

SYSTEM_50HZ
 gb.h, [161](#)
 msx.h, [328](#)
 nes.h, [362](#)
 sms.h, [413](#)

SYSTEM_60HZ
 gb.h, [161](#)
 msx.h, [328](#)
 nes.h, [362](#)

 sms.h, [413](#)

SYSTEM_BITS_DENDY
 nes.h, [362](#)

SYSTEM_BITS_NTSC
 nes.h, [361](#)

SYSTEM_BITS_PAL
 nes.h, [361](#)

SYSTEM_NTSC
 hardware.h, [274](#)

SYSTEM_PAL
 hardware.h, [274](#)

TAC_REG
 hardware.h, [260](#), [282](#), [297](#)

TACF_16KHZ
 hardware.h, [247](#)

TACF_262KHZ
 hardware.h, [247](#)

TACF_4KHZ
 hardware.h, [247](#)

TACF_65KHZ
 hardware.h, [247](#)

TACF_START
 hardware.h, [247](#)

TACF_STOP
 hardware.h, [247](#)

tile
 OAM_item_t, [101](#)

TIM_IFLAG
 gb.h, [163](#)
 msx.h, [330](#)
 nes.h, [365](#)
 sms.h, [415](#)

TIMA_REG
 hardware.h, [260](#), [282](#), [296](#)

time
 time.h, [475](#)

time.h
 clock, [475](#)
 CLOCKS_PER_SEC, [474](#)
 time, [475](#)
 time_t, [475](#)

time_t
 time.h, [475](#)

TIMER_VBLANK_PARITY_MODE_SYSTEM_50HZ
 nes.h, [362](#)

TIMER_VBLANK_PARITY_MODE_SYSTEM_60HZ
 nes.h, [362](#)

TMA_REG
 hardware.h, [260](#), [282](#), [297](#)

TO_FAR_PTR
 far_ptr.h, [226](#)

to_far_ptr
 far_ptr.h, [228](#)

tolower
 ctype.h, [110](#)

toupper
 ctype.h, [110](#)

TRUE

- types.h, 486
- true
 - stdbool.h, 442
- typedef.h
 - TYPEOF_ARRAY, 477
 - TYPEOF_BIT, 476
 - TYPEOF_BITFIELD, 476
 - TYPEOF_CHAR, 476
 - TYPEOF_CPOINTER, 477
 - TYPEOF_EEPPPOINTER, 477
 - TYPEOF_FIXED16X16, 476
 - TYPEOF_FLOAT, 476
 - TYPEOF_FPOINTER, 477
 - TYPEOF_FUNCTION, 477
 - TYPEOF_GPOINTER, 477
 - TYPEOF_INT, 476
 - TYPEOF_IPOINTER, 477
 - TYPEOF_LONG, 476
 - TYPEOF_POINTER, 477
 - TYPEOF_PPOINTER, 477
 - TYPEOF_SBIT, 476
 - TYPEOF_SFR, 476
 - TYPEOF_SHORT, 476
 - TYPEOF_STRUCT, 476
 - TYPEOF_VOID, 476
- TYPEOF_ARRAY
 - typedef.h, 477
- TYPEOF_BIT
 - typedef.h, 476
- TYPEOF_BITFIELD
 - typedef.h, 476
- TYPEOF_CHAR
 - typedef.h, 476
- TYPEOF_CPOINTER
 - typedef.h, 477
- TYPEOF_EEPPPOINTER
 - typedef.h, 477
- TYPEOF_FIXED16X16
 - typedef.h, 476
- TYPEOF_FLOAT
 - typedef.h, 476
- TYPEOF_FPOINTER
 - typedef.h, 477
- TYPEOF_FUNCTION
 - typedef.h, 477
- TYPEOF_GPOINTER
 - typedef.h, 477
- TYPEOF_INT
 - typedef.h, 476
- TYPEOF_IPOINTER
 - typedef.h, 477
- TYPEOF_LONG
 - typedef.h, 476
- TYPEOF_POINTER
 - typedef.h, 477
- TYPEOF_PPOINTER
 - typedef.h, 477
- TYPEOF_SBIT
- typedef.h, 476
- TYPEOF_SFR
 - typedef.h, 476
- TYPEOF_SHORT
 - typedef.h, 476
- TYPEOF_STRUCT
 - typedef.h, 476
- TYPEOF_VOID
 - typedef.h, 476
- types.h
 - __SIZE_T_DEFINED, 478, 480, 484
 - AT, 482
 - BANKED, 482
 - BOOLEAN, 482
 - BYTE, 482
 - clock_t, 479, 480, 485
 - CRITICAL, 482
 - DWORD, 482
 - FALSE, 486
 - fixed, 483
 - INT16, 478, 480, 484
 - INT32, 478, 480, 484
 - INT8, 478, 480, 484
 - INTERRUPT, 482
 - LWORD, 482
 - NAKED, 482
 - NONBANKED, 482
 - NORETURN, 482
 - NULL, 486
 - OLDCALL, 481
 - POINTER, 486
 - PRESERVES_REGS, 481
 - SFR, 482
 - size_t, 479, 480, 485
 - TRUE, 486
 - UBYTE, 482
 - UDWORD, 482
 - UINT16, 478, 480, 484
 - UINT32, 479, 480, 484
 - UINT8, 478, 480, 484
 - ULWORD, 482
 - UWORD, 482
 - WORD, 482
 - Z88DK_CALLEE, 484
 - Z88DK_FASTCALL, 484
- UBYTE
 - types.h, 482
- UCHAR_MAX
 - limits.h, 238
- UDWORD
 - types.h, 482
- UINT16
 - types.h, 478, 480, 484
- UINT16_C
 - stdint.h, 448
- UINT16_MAX
 - stdint.h, 446
- uint16_t

- stdint.h, 449
- uint2bcd
 - bcd.h, 132, 135, 137
- UINT32
 - types.h, 479, 480, 484
- UINT32_C
 - stdint.h, 449
- UINT32_MAX
 - stdint.h, 447
- uint32_t
 - stdint.h, 449
- UINT8
 - types.h, 478, 480, 484
- UINT8_C
 - stdint.h, 448
- UINT8_MAX
 - stdint.h, 446
- uint8_t
 - stdint.h, 449
- UINT_FAST16_MAX
 - stdint.h, 447
- uint_fast16_t
 - stdint.h, 450
- UINT_FAST32_MAX
 - stdint.h, 448
- uint_fast32_t
 - stdint.h, 450
- UINT_FAST8_MAX
 - stdint.h, 447
- uint_fast8_t
 - stdint.h, 450
- UINT_LEAST16_MAX
 - stdint.h, 447
- uint_least16_t
 - stdint.h, 450
- UINT_LEAST32_MAX
 - stdint.h, 447
- uint_least32_t
 - stdint.h, 450
- UINT_LEAST8_MAX
 - stdint.h, 447
- uint_least8_t
 - stdint.h, 450
- UINT_MAX
 - limits.h, 238
- UINT_MIN
 - limits.h, 238
- UINTMAX_C
 - stdint.h, 449
- UINTMAX_MAX
 - stdint.h, 448
- uintmax_t
 - stdint.h, 450
- UINTPTR_MAX
 - stdint.h, 448
- uintptr_t
 - stdint.h, 450
- uitoa
 - stdlib.h, 457
- ULONG_MAX
 - limits.h, 238
- ULONG_MIN
 - limits.h, 238
- ultoa
 - stdlib.h, 458
- ULWORD
 - types.h, 482
- UNSIGNED
 - drawing.h, 148
- USE_C_MEMCPY
 - provides.h, 103, 104
- USE_C_STRCMP
 - provides.h, 103, 104
- USE_C_STRCPY
 - provides.h, 103, 104
- USHRT_MAX
 - limits.h, 238
- USHRT_MIN
 - limits.h, 238
- UWORD
 - types.h, 482
- va_arg
 - stdarg.h, 105, 106
- va_end
 - stdarg.h, 105, 106
- va_list
 - stdarg.h, 105–107
- va_start
 - stdarg.h, 105, 106
- VBK_ATTRIBUTES
 - hardware.h, 253, 274, 295
- VBK_BANK_0
 - hardware.h, 253
- VBK_BANK_1
 - hardware.h, 253
- VBK_REG
 - hardware.h, 262
 - msx.h, 328
 - sms.h, 413
- VBK_TILES
 - hardware.h, 253, 274, 295
- VBL_DONE
 - gb.h, 166
 - msx.h, 332
 - sms.h, 417
- VBL_IFLAG
 - gb.h, 163
 - msx.h, 330
 - nes.h, 365
 - sms.h, 414
- VDP_ATTR_SHIFT
 - hardware.h, 275, 297
- VDP_R0
 - hardware.h, 271, 291
- VDP_R1
 - hardware.h, 272, 291

VDP_R10
 hardware.h, [274](#), [293](#)
 VDP_R2
 hardware.h, [273](#), [292](#)
 VDP_R3
 hardware.h, [273](#), [292](#)
 VDP_R4
 hardware.h, [273](#), [292](#)
 VDP_R5
 hardware.h, [273](#), [292](#)
 VDP_R6
 hardware.h, [273](#), [293](#)
 VDP_R7
 hardware.h, [274](#), [293](#)
 VDP_R8
 hardware.h, [274](#), [293](#)
 VDP_R9
 hardware.h, [274](#), [293](#)
 VDP_RBORDER
 hardware.h, [274](#), [293](#)
 VDP_REG_MASK
 hardware.h, [271](#), [291](#)
 VDP_RSCX
 hardware.h, [274](#), [293](#)
 VDP_RSCY
 hardware.h, [274](#), [293](#)
 VDP_SAT_TERM
 hardware.h, [274](#), [295](#)
 VECTOR_JOYPAD
 isr.h, [219](#)
 VECTOR_SERIAL
 isr.h, [219](#)
 VECTOR_STAT
 isr.h, [219](#)
 VECTOR_TIMER
 isr.h, [219](#)
 version.h
 __GBDK_VERSION, [236](#)
 vmemcpy
 gb.h, [200](#)
 msx.h, [341](#)
 sms.h, [427](#)
 vmemset
 gb.h, [204](#)
 nes.h, [396](#)
 vsync
 gb.h, [179](#)
 msx.h, [337](#)
 nes.h, [376](#)
 sms.h, [422](#)

 w
 _fixed, [97](#)
 wait_int_handler
 gb.h, [175](#)
 wait_vbl_done
 gb.h, [179](#)
 msx.h, [337](#)
 nes.h, [376](#)

 sms.h, [422](#)
 waitpad
 gb.h, [177](#)
 msx.h, [338](#)
 nes.h, [373](#)
 sms.h, [423](#)
 waitpadup
 gb.h, [177](#)
 msx.h, [338](#)
 nes.h, [374](#)
 sms.h, [423](#)
 WCHAR_MAX
 stdint.h, [449](#)
 WCHAR_MIN
 stdint.h, [449](#)
 wchar_t
 stddef.h, [444](#)
 WHITE
 drawing.h, [147](#)
 WINT_MAX
 stdint.h, [449](#)
 WINT_MIN
 stdint.h, [449](#)
 WORD
 types.h, [482](#)
 WRITE_VDP_CMD
 msx.h, [335](#)
 sms.h, [420](#)
 WRITE_VDP_DATA
 msx.h, [335](#)
 sms.h, [420](#)
 wrtchr
 drawing.h, [150](#)
 WX_REG
 hardware.h, [262](#)
 WY_REG
 hardware.h, [262](#)

 x
 OAM_item_t, [101](#)
 XOR
 drawing.h, [147](#)

 y
 OAM_item_t, [101](#)

 Z88DK_CALLEE
 types.h, [484](#)
 Z88DK_FASTCALL
 types.h, [484](#)
 zx0_decompress
 zx0decompress.h, [236](#)
 zx0decompress.h
 zx0_decompress, [236](#)